

Solidum FEM

Manual de Referencia

Especificaciones físicas, formulaciones numéricas y contratos
de los componentes del programa

Generado automáticamente desde docs/specs/

Autor: Jaime Retama Velasco

Facultad de Estudios Superiores Aragón
Universidad Nacional Autónoma de México

25 de agosto de 2026

Índice general

Sobre este Manual

XXI

1. Elementos 1D — Armaduras	1
1.1. Truss2D	1
1.2. Especificación física	1
1.2.1. 0. Descripción general	1
1.2.2. 1. Cinemática de desplazamientos	1
1.2.3. 2. Cinemática de deformaciones	1
1.2.4. 3. Ecuación constitutiva	1
1.2.5. 4. Equilibrio — forma fuerte	1
1.2.6. 5. Equilibrio — forma débil	2
1.3. Formulación numérica (FEM)	2
1.3.1. 6. Discretización	2
1.3.2. 7. Funciones de forma	2
1.3.3. 8. Matriz deformación-desplazamiento	2
1.3.4. 9. Rigidez elemental	2
1.3.5. 10. Fuerzas internas	2
1.3.6. 11. Cuadratura	2
1.4. Contrato de implementación	2
1.5. Implementación	3
1.6. Diálogo	4
1.7. Truss2DCorot	5
1.8. Especificación física	5
1.8.1. 0. Descripción general	5
1.8.2. 1. Cinemática	5
1.8.3. 2. Medida de deformación	5
1.8.4. 3. Ecuación constitutiva	5
1.8.5. 4. Equilibrio — forma débil incremental	5
1.9. Formulación numérica (FEM)	6
1.9.1. 6. Discretización	6
1.9.2. 7. Vector dirección actual	6
1.9.3. 8. Matriz B corotacional	6
1.9.4. 9. Rigidez tangente	6
1.9.5. 10. Fuerzas internas	6
1.9.6. 11. Cuadratura	6
1.10. Contrato de implementación	6
1.11. Implementación	7

1.12. Diálogo	8
1.13. Truss3D	9
1.14. Especificación física	9
1.14.1. 0. Descripción general	9
1.14.2. 1. Cinemática de desplazamientos	9
1.14.3. 2. Cinemática de deformaciones	9
1.14.4. 3. Ecuación constitutiva	9
1.14.5. 4. Equilibrio — forma fuerte	9
1.14.6. 5. Equilibrio — forma débil	9
1.15. Formulación numérica (FEM)	10
1.15.1. 6. Discretización	10
1.15.2. 7. Funciones de forma	10
1.15.3. 8. Matriz deformación-desplazamiento	10
1.15.4. 9. Rigidez elemental	10
1.15.5. 10. Fuerzas internas	10
1.15.6. 11. Cuadratura	10
1.16. Contrato de implementación	10
1.17. Implementación	11
1.18. Diálogo	12
1.19. Truss3DCorot	13
1.20. Especificación física	13
1.20.1. 0. Descripción general	13
1.20.2. 1. Cinemática	13
1.20.3. 2. Medida de deformación	13
1.20.4. 3. Ecuación constitutiva	13
1.20.5. 4. Equilibrio — forma débil incremental	13
1.21. Formulación numérica (FEM)	13
1.21.1. 6. Discretización	13
1.21.2. 7. Vector dirección actual	14
1.21.3. 8. Matriz B corotacional	14
1.21.4. 9. Rigidez tangente	14
1.21.5. 10. Fuerzas internas	14
1.21.6. 11. Cuadratura	14
1.22. Contrato de implementación	14
1.23. Implementación	15
1.24. Diálogo	16
2. Elementos 1D — Cables	17
2.1. Cable2DCorot	17
2.2. Especificación física	17
2.2.1. 0. Descripción general	17
2.2.2. 1. Cinemática	17
2.2.3. 2. Medida de deformación	17
2.2.4. 3. Ecuación constitutiva	18
2.2.5. 4. Equilibrio — forma débil incremental	18
2.3. Formulación numérica (FEM)	18
2.3.1. 6. Discretización	18
2.3.2. 7. Vectores de dirección (configuración corriente)	18

2.3.3.	8. Matriz B corotacional	18
2.3.4.	9. Rigidez tangente	18
2.3.5.	10. Fuerzas internas	19
2.3.6.	11. Cuadratura	19
2.4.	Contrato de implementación	19
2.5.	Implementación	20
2.6.	Diálogo	21
2.7.	Cable3DCorot	22
2.8.	Especificación física	22
2.8.1.	0. Descripción general	22
2.8.2.	1. Cinemática	22
2.8.3.	2. Medida de deformación	22
2.8.4.	3. Ecuación constitutiva	22
2.8.5.	4. Equilibrio — forma débil incremental	22
2.9.	Formulación numérica (FEM)	23
2.9.1.	6. Discretización	23
2.9.2.	7. Vector dirección actual	23
2.9.3.	8. Matriz B corotacional	23
2.9.4.	9. Rigidez tangente	23
2.9.5.	10. Fuerzas internas	23
2.9.6.	11. Cuadratura	23
2.10.	Contrato de implementación	23
2.11.	Implementación	25
2.12.	Diálogo	26
3.	Elementos 1D — Marcos / Vigas	27
3.1.	Frame2DEuler	27
3.2.	Especificación física	27
3.2.1.	0. Descripción general	27
3.2.2.	1. Hipótesis cinemática de Euler-Bernoulli	27
3.2.3.	2. Campos de desplazamiento	27
3.2.4.	3. Medidas de deformación	28
3.2.5.	4. Ecuaciones constitutivas	28
3.2.6.	5. Equilibrio — forma débil	28
3.3.	Formulación numérica (FEM)	28
3.3.1.	6. Discretización	28
3.3.2.	7. Sistema local y transformación	28
3.3.3.	8. Funciones de forma	29
3.3.4.	9. Rigidez local	29
3.3.5.	10. Fuerzas internas	29
3.3.6.	11. Ensamblaje global	29
3.3.7.	12. Cuadratura	29
3.4.	Contrato de implementación	30
3.5.	Implementación	31
3.6.	Diálogo	31
3.7.	Frame2DEulerCorot	33
3.8.	Especificación física	33
3.8.1.	0. Descripción general	33

3.8.2.	1. Cinemática corotacional	33
3.8.3.	2. Medidas de deformación	33
3.8.4.	3. Ecuaciones constitutivas	33
3.8.5.	4. Equilibrio — forma débil	34
3.9.	Formulación numérica (FEM)	34
3.9.1.	5. DOFs locales deformacionales	34
3.9.2.	6. Rigidez local	34
3.9.3.	7. Matriz de transformación $\mathbf{B}$ (3×6)	34
3.9.4.	8. Rigidez tangente	34
3.9.5.	9. Cuadratura	35
3.10.	Contrato de implementación	35
3.11.	Implementación	36
3.12.	Diálogo	37
3.13.	Frame2DTimoshenko	38
3.14.	Especificación física	38
3.14.1.	0. Descripción general	38
3.14.2.	1. Hipótesis cinemática de Timoshenko	38
3.14.3.	2. Campos de desplazamiento	38
3.14.4.	3. Medidas de deformación	38
3.14.5.	4. Ecuaciones constitutivas	39
3.14.6.	5. Factor de Phi (shear locking)	39
3.14.7.	6. Equilibrio — forma débil	39
3.15.	Formulación numérica (FEM)	39
3.15.1.	7. Discretización	39
3.15.2.	8. Sistema local y transformación	39
3.15.3.	9. Rigidez local con corrección por shear locking	40
3.15.4.	10. Fuerzas internas	40
3.15.5.	11. Ensamblaje global	40
3.15.6.	12. Cuadratura	40
3.16.	Contrato de implementación	40
3.17.	Implementación	41
3.18.	Diálogo	42
3.19.	Frame3D	43
3.20.	Especificación física	43
3.20.1.	0. Descripción general	43
3.20.2.	1. Hipótesis	43
3.20.3.	2. Sistema de ejes locales	43
3.20.4.	3. Medidas de deformación / acciones internas	43
3.20.5.	4. Equilibrio — forma débil	44
3.21.	Formulación numérica (FEM)	44
3.21.1.	5. Discretización	44
3.21.2.	6. Construcción de ejes locales	44
3.21.3.	7. Matriz de transformación global $\rightarrow$ local	44
3.21.4.	8. Matriz de rigidez local	45
3.21.5.	9. Fuerzas internas	45
3.21.6.	10. Ensamblaje global	45
3.21.7.	11. Cuadratura	45
3.22.	Contrato de implementación	45

3.23. Implementación	47
3.24. Diálogo	48
4. Elementos 2D — Sólidos	49
4.1. Tri3	49
4.2. Especificación física	49
4.2.1. 0. Descripción general	49
4.2.2. 1. Cinemática de desplazamientos	49
4.2.3. 2. Cinemática de deformaciones	49
4.2.4. 3. Ecuación constitutiva	49
4.2.5. 4. Equilibrio — forma fuerte	49
4.2.6. 5. Equilibrio — forma débil	50
4.3. Formulación numérica (FEM)	50
4.3.1. 6. Discretización	50
4.3.2. 7. Funciones de forma	50
4.3.3. 8. Matriz deformación-desplazamiento	50
4.3.4. 9. Rigidez elemental	50
4.3.5. 10. Fuerzas internas	50
4.3.6. 11. Cuadratura	50
4.3.7. 12. Cargas distribuidas consistentes	51
4.3.8. 13. Salida por punto de Gauss	51
4.4. Limitación crítica — <i>shear locking</i>	51
4.5. Contrato de implementación	51
4.6. Implementación	52
4.7. Diálogo	53
4.8. Quad4	54
4.9. Especificación física	54
4.9.1. 0. Descripción general	54
4.9.2. 1. Cinemática de desplazamientos	54
4.9.3. 2. Cinemática de deformaciones	54
4.9.4. 3. Ecuación constitutiva	54
4.9.5. 4. Equilibrio — forma fuerte	54
4.9.6. 5. Equilibrio — forma débil	54
4.10. Formulación numérica (FEM)	54
4.10.1. 6. Discretización	55
4.10.2. 7. Funciones de forma	55
4.10.3. 8. Matriz deformación-desplazamiento	55
4.10.4. 9. Rigidez elemental	55
4.10.5. 10. Fuerzas internas	55
4.10.6. 11. Cuadratura	55
4.10.7. 12. Cargas distribuidas consistentes	56
4.10.8. 13. Salida por punto de Gauss	56
4.11. Contrato de implementación	56
4.12. Implementación	58
4.13. Diálogo	58
4.14. Tri6	59
4.15. Especificación física	59
4.15.1. 7. Funciones de forma	59

4.15.2. 11. Cuadratura	59
4.15.3. 12. Cargas distribuidas	59
4.16. Contrato de implementación	59
4.17. Implementación	60
4.18. Quad8	61
4.19. Especificación física	61
4.19.1. 0. Descripción general	61
4.19.2. 1. Cinemática de desplazamientos	61
4.19.3. 2. Cinemática de deformaciones	61
4.19.4. 3. Ecuación constitutiva	61
4.19.5. 4-5. Equilibrio	61
4.20. Formulación numérica	61
4.20.1. 6. Discretización	61
4.20.2. 7. Funciones de forma serendípitas	61
4.20.3. 8. Matriz $\mathbf{B}$	62
4.20.4. 9-10. Rigidez y fuerzas internas	62
4.20.5. 11. Cuadratura	62
4.20.6. 12. Cargas distribuidas consistentes	62
4.20.7. 13. Salida por punto de Gauss	62
4.21. Contrato de implementación	62
4.22. Implementación	63
4.23. Quad9	64
4.24. Especificación física	64
4.24.1. 7. Funciones de forma	64
4.24.2. 11. Cuadratura	64
4.24.3. 12. Cargas distribuidas	64
4.25. Contrato de implementación	64
4.26. Implementación	65
5. Elementos 2D — Discontinuidades embebidas	66
5.1. CST_Embedded2D	66
5.2. Especificación física	66
5.2.1. 0. Descripción general	66
5.2.2. 1. Cinemática de desplazamientos (formulación KOS)	67
5.2.3. 2. Cinemática de deformaciones	67
5.2.4. 3. Ecuación constitutiva	68
5.2.5. 4. Equilibrio — forma fuerte	68
5.2.6. 5. Equilibrio — forma débil (residuales acoplados)	68
5.3. Formulación numérica (FEM)	68
5.3.1. 6. Discretización del CST padre	68
5.3.2. 7. Funciones de forma + función φ	69
5.3.3. 8. Matrices B estándar + $\hat{B}^\varphi$	69
5.3.4. 9. Sistema acoplado — matrices elementales	69
5.3.5. 10. Condensación estática local	70
5.3.6. 11. Activación: criterio Rankine + dirección $\mathbf{n} + \mathbf{l}_d$ del Cap. 6	70
5.3.7. 12. Estado de la discontinuidad — <code>DiscontinuityState</code>	71
5.3.8. 13. Cuadratura	71
5.3.9. 14. Activación al principio del paso — interacción con el solver	71

5.4.	Caveats numéricos	71
5.5.	Contrato de implementación	72
5.6.	Implementación	75
5.7.	Diálogo	76
6.	Elementos 3D — Sólidos	78
6.1.	Tet4	78
6.2.	Especificación física	78
6.2.1.	0. Descripción general	78
6.2.2.	1. Cinemática de desplazamientos	78
6.2.3.	2. Cinemática de deformaciones	78
6.2.4.	3. Ecuación constitutiva	78
6.2.5.	4. Equilibrio — forma fuerte	78
6.2.6.	5. Equilibrio — forma débil	79
6.3.	Formulación numérica (FEM)	79
6.3.1.	6. Discretización	79
6.3.2.	7. Funciones de forma	79
6.3.3.	8. Matriz deformación-desplazamiento	79
6.3.4.	9. Rigidez elemental	79
6.3.5.	10. Fuerzas internas	80
6.3.6.	11. Cuadratura	80
6.3.7.	12. Cargas distribuidas consistentes	80
6.3.8.	13. Salida por punto de Gauss	80
6.4.	Limitación crítica — <i>shear locking</i>	80
6.5.	Contrato de implementación	81
6.6.	Implementación	82
6.7.	Diálogo	83
6.8.	Hex8	84
6.9.	Especificación física	84
6.9.1.	0. Descripción general	84
6.9.2.	1. Cinemática de desplazamientos	84
6.9.3.	2. Cinemática de deformaciones	84
6.9.4.	3. Ecuación constitutiva	84
6.9.5.	4. Equilibrio — forma fuerte	84
6.9.6.	5. Equilibrio — forma débil	84
6.10.	Formulación numérica (FEM)	85
6.10.1.	6. Discretización	85
6.10.2.	7. Funciones de forma	85
6.10.3.	8. Matriz deformación-desplazamiento	85
6.10.4.	9. Rigidez elemental	86
6.10.5.	10. Fuerzas internas	86
6.10.6.	11. Cuadratura	86
6.10.7.	12. Cargas distribuidas consistentes	86
6.10.8.	13. Salida por punto de Gauss	87
6.11.	Limitaciones declaradas	87
6.11.1.	Locking volumétrico con $\nu \rightarrow 0,5$	87
6.11.2.	Hourglass con integración reducida	87
6.11.3.	Mass lumping en orientación arbitraria	87

6.12. Contrato de implementación	87
6.13. Implementación	89
6.14. Diálogo	90
6.15. Tet10	91
6.16. Especificación física	91
6.16.1. 0. Descripción general	91
6.16.2. 1-5. Cinemática, ecuación constitutiva, equilibrio	91
6.17. Formulación numérica (FEM)	91
6.17.1. 6. Discretización — convención VTK_QUADRATIC_TETRA	91
6.17.2. 7. Funciones de forma	91
6.17.3. 8. Matriz B	92
6.17.4. 9-10. Rigidez y fuerzas internas	92
6.17.5. 11. Cuadratura	92
6.17.6. 12. Cargas distribuidas consistentes	92
6.17.7. 13. Salida por punto de Gauss	93
6.18. Limitaciones declaradas	93
6.18.1. Locking volumétrico atenuado pero presente	93
6.18.2. Distorsión severa	93
6.19. Contrato de implementación	93
6.20. Implementación	95
6.21. Diálogo	95
6.22. Hex20	96
6.23. Especificación física	96
6.23.1. 0. Descripción general	96
6.23.2. 1. Cinemática de desplazamientos	96
6.23.3. 2. Cinemática de deformaciones	96
6.23.4. 3. Ecuación constitutiva	96
6.23.5. 4-5. Equilibrio	96
6.24. Formulación numérica (FEM)	96
6.24.1. 6. Discretización	96
6.24.2. 7. Funciones de forma serendípitas	97
6.24.3. 8. Matriz deformación-desplazamiento	98
6.24.4. 9. Rigidez elemental	99
6.24.5. 10. Fuerzas internas	99
6.24.6. 11. Cuadratura	99
6.24.7. 12. Cargas distribuidas consistentes	99
6.24.8. 13. Salida por punto de Gauss	100
6.25. Limitaciones declaradas	100
6.25.1. Locking volumétrico con $\nu \rightarrow 0,5$	100
6.25.2. Hourglass con integración reducida	100
6.25.3. Mass lumping	101
6.26. Contrato de implementación	101
6.27. Implementación	103
6.28. Diálogo	103
6.29. Hex27	105
6.30. Especificación física	105
6.30.1. 0. Descripción general	105
6.30.2. 1-5. Cinemática, ecuación constitutiva, equilibrio	105

6.31. Formulación numérica (FEM)	105
6.31.1. 6. Discretización — convención VTK_TRIQUADRATIC_HEXAHEDRON	105
6.31.2. 7. Funciones de forma — producto tensorial de Lagrange cuadrático	105
6.31.3. 8. Matriz B	106
6.31.4. 9-10. Rigidez y fuerzas internas	106
6.31.5. 11. Cuadratura	106
6.31.6. 12. Cargas distribuidas consistentes	106
6.31.7. 13. Salida por punto de Gauss	107
6.32. Limitaciones declaradas	107
6.32.1. Coste computacional vs ‘Hex20’	107
6.32.2. Locking volumétrico con $\nu \rightarrow 0,5$	107
6.32.3. Hourglass con integración reducida	107
6.33. Contrato de implementación	108
6.34. Implementación	109
6.35. Diálogo	110
7. Modelos Constitutivos — 1D	111
7.1. CableMaterial1D	111
7.2. Especificación física	111
7.2.1. 0. Descripción general	111
7.2.2. 1. Ecuación constitutiva	111
7.2.3. 2. Módulo tangente	111
7.2.4. 3. Variables internas	111
7.2.5. 4. Interpretación física	112
7.3. Contrato de implementación	112
7.4. Implementación	113
7.5. Diálogo	114
7.6. Elastic1D	115
7.7. Especificación física	115
7.7.1. 0. Descripción general	115
7.7.2. 1. Descomposición de la deformación	115
7.7.3. 2. Ley elástica (Hooke 1D)	115
7.7.4. 3. Superficie de fluencia / criterio de daño	115
7.7.5. 4. Regla de flujo	115
7.7.6. 5. Endurecimiento	115
7.7.7. 6. Condiciones de Kuhn-Tucker	115
7.7.8. 7. Variables internas	115
7.7.9. 8. Notación	116
7.8. Formulación numérica	116
7.8.1. 9. Esquema temporal	116
7.8.2. 10. Return mapping / actualización	116
7.8.3. 11. Tangente algorítmica consistente	116
7.8.4. 12. Caveats numéricos	116
7.9. Contrato de implementación	116
7.10. Implementación	117
7.11. Diálogo	118
7.12. Elastoplastic1D	119
7.13. Especificación física	119

7.13.1. 0. Descripción general	119
7.13.2. 1. Descomposición aditiva	119
7.13.3. 2. Ley elástica	119
7.13.4. 3. Criterio de fluencia	119
7.13.5. 4. Regla de flujo (asociada)	119
7.13.6. 5. Endurecimiento isótropo lineal	119
7.13.7. 6. Condiciones de Kuhn-Tucker	119
7.13.8. 7. Variables internas	120
7.13.9. 8. Notación	120
7.14. Formulación numérica	120
7.14.1. 9. Esquema temporal — Backward Euler implícito	120
7.14.2. 10. Return mapping — forma cerrada	120
7.14.3. 11. Tangente algorítmica consistente	120
7.14.4. 12. Caveats numéricos	121
7.15. Contrato de implementación	121
7.16. Implementación	122
7.17. Diálogo	123
7.18. IsotropicDamage1D	124
7.19. Especificación física	124
7.19.1. Ecuaciones	124
7.20. Formulación numérica	124
7.20.1. Tangente algorítmica consistente	124
7.20.2. Signo de la tangente consistente	125
7.20.3. Por qué no tests de integración en esta spec	125
7.21. Contrato de implementación	125
7.22. Implementación	127
7.23. Diálogo	128
8. Modelos Constitutivos — 2D	129
8.1. DruckerPrager2D	129
8.2. Especificación física	129
8.2.1. 0. Descripción general	129
8.2.2. 1. Descomposición aditiva y elasticidad	129
8.2.3. 2. Criterio de fluencia (Drucker-Prager)	129
8.2.4. 3. Calibración con Mohr-Coulomb	130
8.2.5. 4. Regla de flujo (no asociada por defecto)	130
8.2.6. 5. Endurecimiento isótropo lineal	130
8.2.7. 6. Condiciones de Kuhn-Tucker	131
8.2.8. 7. Variables internas	131
8.3. Formulación numérica	131
8.3.1. 8. Esquema implícito — Backward Euler	131
8.3.2. 9. Return regular (cone surface)	131
8.3.3. 10. Return al ápice	132
8.3.4. 11. Detección del régimen de retorno	133
8.3.5. 12. Tangente algorítmica consistente	133
8.3.6. 13. Tolerancia de fluencia	133
8.4. Contrato de implementación	134
8.5. Implementación	137

8.6. Diálogo	137
8.7. Elastic2D	138
8.8. Especificación física	138
8.8.1. 0. Descripción general	138
8.8.2. 1. Descomposición de la deformación	138
8.8.3. 2. Ley elástica	138
8.8.4. 3. Superficie de fluencia / criterio de daño	138
8.8.5. 4-6. Flujo, endurecimiento, Kuhn-Tucker	139
8.8.6. 7. Variables internas	139
8.8.7. 8. Notación Voigt	139
8.9. Formulación numérica	139
8.9.1. 9. Esquema temporal	139
8.9.2. 10. Return mapping / actualización	139
8.9.3. 11. Tangente algorítmica consistente	139
8.9.4. 12. Caveats numéricos	139
8.10. Contrato de implementación	140
8.11. Implementación	141
8.12. Diálogo	141
8.13. IsotropicDamage2D	142
8.14. Especificación física	142
8.14.1. 0. Descripción general	142
8.14.2. 1. Cinemática	142
8.14.3. 2. Esfuerzo nominal vs esfuerzo efectivo	142
8.14.4. 3. Deformación equivalente	142
8.14.5. 4. Variable histórica y condición de carga (Kuhn-Tucker)	142
8.14.6. 5. Ley de evolución del daño	143
8.15. Formulación numérica	143
8.15.1. 6. Algoritmo en un paso	143
8.15.2. 7. Tangente algorítmica consistente (ampliación)	143
8.15.3. 8. Pérdida de simetría	144
8.15.4. 9. Decisión de régimen durante el Newton	144
8.16. Contrato de implementación	145
8.17. Implementación	147
8.18. Diálogo	148
8.19. VonMises2D	149
8.20. Especificación física	149
8.20.1. 0. Descripción general	149
8.20.2. 1. Descomposición aditiva	149
8.20.3. 2. Ley elástica	149
8.20.4. 3. Superficie de fluencia (J2)	149
8.20.5. 4. Regla de flujo (asociada)	150
8.20.6. 5. Endurecimiento isótropo lineal	150
8.20.7. 6. Condiciones de Kuhn-Tucker	150
8.20.8. 7. Variables internas	150
8.20.9. 8. Notación Voigt y manejo de ε_{zz}^p	150
8.21. Formulación numérica	151
8.21.1. 9. Esquema temporal — Backward Euler implícito	151
8.21.2. 10. Return mapping plane strain — algoritmo radial cerrado (existente)	151

8.21.3. 11. Return mapping plane stress — algoritmo proyectado (a implementar)	151
8.21.4. 12. Tolerancia de fluencia	153
8.22. Contrato de implementación	153
8.23. Implementación	156
8.23.1. Bug fixes detectados al implementar	157
8.24. Diálogo	157
9. Modelos Constitutivos — 3D	159
9.1. DruckerPrager3D	159
9.2. Especificación física	159
9.2.1. 0. Descripción general	159
9.2.2. 1. Descomposición aditiva y elasticidad	160
9.2.3. 2. Criterio de fluencia (Drucker-Prager)	160
9.2.4. 3. Calibración con Mohr-Coulomb (3D)	160
9.2.5. 4. Regla de flujo (no asociada por defecto)	161
9.2.6. 5. Endurecimiento isótropo lineal	161
9.2.7. 6. Condiciones de Kuhn-Tucker	161
9.2.8. 7. Variables internas	161
9.2.9. 8. Notación Voigt 6D	162
9.2.10. 9. Relación con DruckerPrager2D plane strain	162
9.3. Formulación numérica	162
9.3.1. 10. Esquema implícito — Backward Euler	162
9.3.2. 11. Return regular (cone surface)	163
9.3.3. 12. Return al ápice	163
9.3.4. 13. Detección del régimen de retorno	164
9.3.5. 14. Tangente algorítmica consistente	164
9.3.6. 15. Tolerancia de fluencia	164
9.4. Contrato de implementación	165
9.5. Implementación	168
9.6. Diálogo	169
9.7. Elastic3D	171
9.8. Especificación física	171
9.8.1. 0. Descripción general	171
9.8.2. 1. Descomposición de la deformación	171
9.8.3. 2. Ley elástica	171
9.8.4. 3-6. Superficie de fluencia, flujo, endurecimiento, Kuhn-Tucker	171
9.8.5. 7. Variables internas	171
9.8.6. 8. Notación Voigt	172
9.9. Formulación numérica	172
9.9.1. 9. Esquema temporal	172
9.9.2. 10. Return mapping / actualización	172
9.9.3. 11. Tangente algorítmica consistente	172
9.9.4. 12. Caveats numéricos	172
9.10. Contrato de implementación	173
9.11. Implementación	174
9.12. Diálogo	174
9.13. IsotropicDamage3D	175
9.14. Especificación física	175

9.14.1. 0. Descripción general	175
9.14.2. 1. Cinemática	175
9.14.3. 2. Esfuerzo nominal vs esfuerzo efectivo	175
9.14.4. 3. Deformación equivalente	176
9.14.5. 4. Variable histórica y condición de carga (Kuhn-Tucker)	176
9.14.6. 5. Ley de evolución del daño	176
9.15. Formulación numérica	176
9.15.1. 6. Algoritmo en un paso	176
9.15.2. 7. Tangente algorítmica consistente	177
9.15.3. 8. Pérdida de simetría	178
9.15.4. 9. Decisión de régimen durante el Newton	178
9.16. Contrato de implementación	178
9.17. Implementación	181
9.18. Diálogo	182
9.19. VonMises3D	184
9.20. Especificación física	184
9.20.1. 0. Descripción general	184
9.20.2. 1. Descomposición aditiva	184
9.20.3. 2. Ley elástica	184
9.20.4. 3. Superficie de fluencia (J2)	185
9.20.5. 4. Regla de flujo (asociada)	185
9.20.6. 5. Endurecimiento isótropo lineal	185
9.20.7. 6. Condiciones de Kuhn-Tucker	185
9.20.8. 7. Variables internas	185
9.20.9. 8. Notación Voigt y manejo de ϵ^p	186
9.20.10. 9. Relación con VonMises2D plane strain	186
9.21. Formulación numérica	186
9.21.1. 10. Esquema temporal — Backward Euler implícito	186
9.21.2. 11. Return mapping 3D — algoritmo radial cerrado	186
9.21.3. 12. Tangente algorítmica consistente	187
9.21.4. 13. Caveats numéricos	188
9.22. Contrato de implementación	188
9.23. Implementación	191
9.24. Diálogo	192
10. Modelos Constitutivos — Cohesivos	194
10.1. CohesiveDamageIsotropic	194
10.2. Especificación física	194
10.2.1. 0. Descripción general	194
10.2.2. 1. Descomposición del salto	194
10.2.3. 2. Ley elástica del salto (sin daño)	195
10.2.4. 3. Variable equivalente y función de fluencia	195
10.2.5. 4. Variable histórica y evolución (Kuhn-Tucker)	195
10.2.6. 5. Ley de evolución del daño — softening lineal y exponencial	196
10.2.7. 6. Variables internas	196
10.2.8. 7. Frame local y notación	197
10.3. Formulación numérica	197
10.3.1. 8. Algoritmo en un paso (explícito, sin Newton local)	197

10.3.2. 9. Tangente algorítmica consistente	197
10.3.3. 10. Simetría de la tangente	198
10.3.4. 11. Decisión de régimen durante el Newton global	199
10.3.5. 12. Caveats numéricos	199
10.4. Contrato de implementación	199
10.5. Implementación	202
10.6. Diálogo	203
11. Esquemas de Solución — Estáticos	205
11.1. ArcLengthSolver	205
11.2. Especificación física	205
11.2.1. 0. Descripción general	205
11.2.2. 1. Ecuación de equilibrio resuelta	205
11.2.3. 2. Condiciones de contorno	205
11.2.4. 3. Salidas físicas	206
11.3. Formulación numérica	206
11.3.1. 4. Esquema operativo	206
11.3.2. 5. Predictor / corrector	206
11.3.3. 6. Criterio de convergencia (ADR 0007)	207
11.3.4. 7. Imposición de Dirichlet y MPC	207
11.3.5. 8. Backend algebraico	207
11.3.6. 9. Adaptatividad — ajuste de dl	207
11.3.7. 10. Caveats numéricos	207
11.4. Contrato de implementación	208
11.5. Implementación	209
11.6. Diálogo	210
11.7. DissipationArcLengthSolver	211
11.8. Especificación física	211
11.8.1. 0. Descripción general	211
11.8.2. 1. Ecuación de equilibrio resuelta	211
11.8.3. 2. Condiciones de contorno	212
11.8.4. 3. Salidas físicas	212
11.9. Formulación numérica — solo lo que cambia respecto al padre	212
11.9.1. 4. Esquema operativo	212
11.9.2. 5. Predictor / corrector — punto clave del cambio	213
11.9.3. 6. Criterio de convergencia (ADR 0007)	213
11.9.4. 7. Imposición de Dirichlet y MPC	214
11.9.5. 8. Backend algebraico	214
11.9.6. 9. Adaptatividad — ajuste de τ	214
11.9.7. 10. Caveats numéricos	214
11.10. Contrato de implementación	215
11.11. Implementación	217
11.11.1. Tests (suite tests/test_dissipation_arclength.py)	217
11.11.2. Validación parcial — qué falta para status <code>validated</code>	218
11.12. Diálogo	218
11.13. LinearSolver	220
11.14. Especificación física	220
11.14.1. 0. Descripción general	220

11.14.21. Ecuación de equilibrio resuelta	220
11.14.32. Condiciones de contorno	220
11.14.43. Salidas físicas	220
11.15 Formulación numérica	220
11.15.14. Esquema operativo	220
11.15.25. Predictor / corrector	221
11.15.36. Criterio de convergencia	221
11.15.47. Imposición de Dirichlet y MPC	221
11.15.58. Backend algebraico	221
11.15.69. Adaptatividad / control de paso	221
11.15.710. Caveats numéricos	221
11.16 Contrato de implementación	222
11.17 Implementación	223
11.18 Diálogo	223
11.19 NonlinearSolver	224
11.20 Especificación física	224
11.20.10. Descripción general	224
11.20.21. Ecuación de equilibrio resuelta	224
11.20.32. Condiciones de contorno	224
11.20.43. Salidas físicas	224
11.21 Formulación numérica	224
11.21.14. Esquema operativo	224
11.21.25. Predictor / corrector	225
11.21.36. Criterio de convergencia (ADR 0007)	225
11.21.47. Imposición de Dirichlet y MPC	225
11.21.58. Backend algebraico	225
11.21.69. Adaptatividad / control de paso	226
11.21.710. Caveats numéricos	226
11.22 Contrato de implementación	226
11.23 Implementación	228
11.24 Diálogo	228
12. Esquemas de Solución — Modal y dinámicos	229
12.1. CentralDifferenceSolver	229
12.2. Especificación física	229
12.2.1. 0. Descripción general	229
12.2.2. 1. Ecuación de equilibrio resuelta	229
12.2.3. 2. Condiciones iniciales / de contorno	229
12.2.4. 3. Salidas físicas	230
12.3. Formulación numérica	230
12.3.1. 4. Esquema operativo — leapfrog Belytschko-Liu-Moran	230
12.3.2. 5. Predictor / corrector	230
12.3.3. 6. Criterio de convergencia	230
12.3.4. 7. Imposición de Dirichlet y MPC	231
12.3.5. 8. Backend algebraico	231
12.3.6. 9. Adaptatividad / control de paso	231
12.3.7. 10. Caveats numéricos	231
12.4. Contrato de implementación	231

12.5. Implementación	233
12.6. Diálogo	234
12.7. HHTSolver	235
12.8. Qué cambia respecto a NewmarkSolver	235
12.8.1. Ecuación de movimiento HHT- α	235
12.8.2. Sistema efectivo	235
12.8.3. Disipación numérica controlada	236
12.8.4. Recuperación del comportamiento Newmark	236
12.8.5. Variante no lineal	236
12.9. Contrato de implementación	236
12.10 Implementación	238
12.11 Diálogo	238
12.12 HarmonicSolver	240
12.13 Especificación física	240
12.13.10. Descripción general	240
12.13.2.1. Ecuación de equilibrio resuelta	240
12.13.3.2. Condiciones iniciales / de contorno	240
12.13.4.3. Salidas físicas	240
12.14 Formulación numérica	241
12.14.1.4. Esquema operativo	241
12.14.2.5. Predictor / corrector	241
12.14.3.6. Criterio de convergencia	241
12.14.4.7. Imposición de Dirichlet y MPC	241
12.14.5.8. Backend algebraico	241
12.14.6.9. Adaptatividad / control de paso	241
12.14.7.10. Caveats numéricos	242
12.15 Contrato de implementación	242
12.16 Implementación	243
12.17 Diálogo	244
12.18 ModalSolver	245
12.19 Especificación física	245
12.19.1.0. Descripción general	245
12.19.2.1. Problema físico	245
12.19.3.2. Problema de autovalor generalizado	245
12.19.4.3. Propiedades del par (K, M)	245
12.19.5.4. Salidas	245
12.20 Formulación numérica	246
12.20.1.5. Reducción por Dirichlet	246
12.20.2.6. Algoritmo numérico: Lanczos con shift-invert	246
12.20.3.7. Normalización	246
12.20.4.8. Post-procesado	246
12.21 Contrato de implementación	246
12.22 Implementación	248
12.23 Diálogo	249
12.24 NewmarkSolver	250
12.25 Especificación física	250
12.25.1.0. Descripción general	250
12.25.2.1. Ecuación de movimiento semidiscreta	250

12.25.32. Amortiguamiento Rayleigh	250
12.25.43. Salidas	250
12.26 Formulación numérica	251
12.26.14. Método de Newmark- β (a-form)	251
12.26.25. Aceleración inicial	251
12.26.36. Parámetros canónicos	251
12.26.47. Imposición de Dirichlet con apoyos constantes	251
12.26.58. Factorización reutilizable	251
12.27 Contrato de implementación	252
12.28 Implementación	254
12.29 Diálogo	255
12.30 NewtonHHTSolver	256
12.31 Qué cambia respecto a <code>NewtonNewmarkSolver</code>	256
12.31.1. Residuo dinámico HHT- α no lineal	256
12.31.2. Jacobiano efectivo	256
12.31.3. Parámetros heredados de <code>HHTSolver</code>	256
12.31.4. Comportamiento que NO cambia	257
12.31.5. Recuperación de <code>NewtonNewmark</code> con $\alpha = 0$	257
12.31.6. Recuperación de <code>HHTSolver</code> con materiales lineales	257
12.32 Contrato de implementación	257
12.33 Implementación	259
12.34 Diálogo	259
12.35 NewtonNewmarkSolver	260
12.36 Qué cambia respecto a <code>NewmarkSolver</code>	260
12.36.1. Residuo dinámico en cada paso	260
12.36.2. Jacobiano dinámico tangente	260
12.36.3. Amortiguamiento Rayleigh — heredado, constante en el tiempo	260
12.36.4. Reducción y commit de estado	261
12.36.5. Convergencia (ADR 0007)	261
12.36.6. Factorización por iteración	261
12.36.7. Recuperación del comportamiento lineal	261
12.37 Contrato de implementación	261
12.38 Implementación	262
12.39 Diálogo	263
12.40 ResponseSpectrumSolver	265
12.41 Especificación física	265
12.41.1.0. Descripción general	265
12.41.2.1. Ecuación de equilibrio resuelta	265
12.41.3.2. Condiciones iniciales / de contorno	265
12.41.4.3. Salidas físicas	266
12.42 Formulación numérica	266
12.42.1.4. Esquema operativo	266
12.42.2.5. Predictor / corrector	266
12.42.3.6. Criterio de convergencia	266
12.42.4.7. Imposición de Dirichlet y MPC	266
12.42.5.8. Backend algebraico	267
12.42.6.9. Adaptatividad / control de paso	267
12.42.7.1.0. Caveats numéricos	267

12.43	Contrato de implementación	267
12.44	Implementación	269
12.45	Diálogo	269
13.	Elementos — catálogo	270
13.1.	Truss2D — barra axial 2D	270
13.1.1.	Cinemática	270
13.1.2.	Formulación	270
13.1.3.	Convenciones	270
13.1.4.	Parámetros	270
13.1.5.	Cargas distribuidas	271
13.1.6.	Régimen de validez	271
13.1.7.	Validación	271
13.2.	Truss2DCorot — armadura 2D corotacional	271
13.2.1.	Cinemática	271
13.2.2.	Formulación	271
13.2.3.	Cargas distribuidas	271
13.2.4.	Régimen de validez	272
13.2.5.	Validación	272
13.3.	Truss3D — barra axial 3D	272
13.3.1.	Cinemática	272
13.3.2.	Formulación	272
13.3.3.	Convenciones	272
13.3.4.	Parámetros	273
13.3.5.	Cargas distribuidas	273
13.3.6.	Régimen de validez	273
13.3.7.	Validación	273
13.4.	Truss3DCorot — barra axial 3D corotacional	273
13.4.1.	Cinemática	273
13.4.2.	Formulación	273
13.4.3.	Cargas distribuidas	274
13.4.4.	Régimen de validez	274
13.4.5.	Validación	274
13.5.	Cable2DCorot — cable 2D corotacional	274
13.5.1.	Cinemática	274
13.5.2.	Formulación	274
13.5.3.	Cargas distribuidas	274
13.5.4.	Régimen de validez	275
13.5.5.	Independencia del diseño	275
13.5.6.	Validación	275
13.6.	Cable3DCorot — cable 3D corotacional	275
13.6.1.	Cinemática	275
13.6.2.	Formulación	275
13.6.3.	Cargas distribuidas	276
13.6.4.	Régimen de validez	276
13.6.5.	Independencia del diseño	276
13.6.6.	Validación	276
13.7.	Frame2DEuler — marco/viga 2D Euler-Bernoulli	276

13.7.1. Cinemática	276
13.7.2. Formulación	276
13.7.3. Cargas distribuidas	277
13.7.4. Régimen de validez	277
13.7.5. Independencia del diseño	277
13.7.6. Validación	277
13.8. Frame2DTimoshenko — marco/viga 2D Timoshenko	277
13.8.1. Cinemática	277
13.8.2. Formulación	278
13.8.3. Parámetros	278
13.8.4. Cargas distribuidas	278
13.8.5. Régimen de validez	278
13.8.6. Independencia del diseño	278
13.8.7. Validación	278
13.9. Frame2DEulerCorot — viga 2D Euler-Bernoulli corotacional	279
13.9.1. Cinemática corotacional	279
13.9.2. Formulación	279
13.9.3. Cargas distribuidas	279
13.9.4. Régimen de validez	279
13.9.5. Dominios que abre	279
13.9.6. Independencia del diseño	280
13.9.7. Validación	280
13.10Frame3D — marco/viga 3D Euler-Bernoulli	280
13.10.1.Cinemática	280
13.10.2.Ejes locales y orientación de la sección	280
13.10.3Formulación	280
13.10.4Parámetros	281
13.10.5.Cargas distribuidas	281
13.10.6.Régimen de validez	281
13.10.7Masa lumped: limitación en orientación oblicua	281
13.10.8Independencia del diseño	281
13.10.9.Validación	281
13.11Quad4 — cuadrilátero bilineal 2D	282
13.12Tri3 — triángulo lineal 2D (CST)	282
13.13Quad8 — cuadrilátero serendípito 2D de orden 2	283
13.14Quad9 — cuadrilátero Lagrangiano 2D de orden 2	283
13.15Tri6 — triángulo 2D cuadrático completo P_2	283
13.16Hex8 — hexaedro trilineal 3D	284
13.17Hex20 — hexaedro serendípito 3D de orden 2 (20 nodos)	285
13.18Hex27 — hexaedro Lagrangiano 3D triquadrático (27 nodos)	286
13.19Tet10 — tetraedro cuadrático 3D (10 nodos)	287
13.20Tet4 — tetraedro lineal 3D (CST 3D)	288
13.21CST_Embedded2D — triángulo CST con discontinuidad interior embebida (KOS)	289
13.22Cómo añadir un elemento nuevo	290

14. Modelos constitutivos — catálogo	291
14.1. Elastic1D — elástico lineal 1D	291
14.2. Elastic2D — elástico lineal 2D	291
14.3. Elastic3D — elástico lineal isótropo 3D	292
14.4. IsotropicDamage1D — daño isótropo 1D con softening exponencial	292
14.5. IsotropicDamage3D — daño isótropo 3D con softening exponencial	293
14.6. IsotropicDamage2D — daño isótropo 2D con softening exponencial	294
14.7. Elastoplastic1D — plasticidad J2 1D con endurecimiento isótropo lineal	295
14.8. CableMaterial1D — cable axial 1D con elasticidad unilateral	296
14.9. DruckerPrager3D — plasticidad friccional cohesivo-friccional 3D	296
14.10. DruckerPrager2D — plasticidad friccional cohesivo-friccional 2D plane strain	298
14.11. VonMises3D — plasticidad J2 3D con endurecimiento isótropo lineal	299
14.12. VonMises2D — plasticidad J2 2D con endurecimiento isótropo lineal	300
14.13. CohesiveDamageIsotropic — daño cohesivo isótropo Modo-I con softening lineal/exponencial	302
14.14. Cómo añadir un material nuevo	303
15. Solvers — catálogo	304
15.1. LinearSolver — solución lineal directa en un paso	304
15.2. NonlinearSolver — Newton-Raphson incremental con paso adaptativo	304
15.3. ArcLengthSolver — método de longitud de arco cilíndrico (Crisfield)	305
15.4. DissipationArcLengthSolver — variante de arc-length por disipación (Gutiérrez 2004)	306
15.5. ModalSolver — análisis modal por autovalores generalizados (ADR 0009)	307
15.6. NewmarkSolver — análisis dinámico transitorio lineal (ADR 0009 fase 3)	308
15.7. NewtonNewmarkSolver — análisis dinámico transitorio no lineal (ADR 0009 fase 4)	309
15.8. HHTSolver — análisis dinámico transitorio lineal con disipación numérica (Hilber-Hughes-Taylor α)	310
15.9. NewtonHHTSolver — análisis dinámico transitorio no lineal HHT- α	311
15.10. CentralDifferenceSolver — integración explícita por diferencias centradas (ADR 0009 fase 5)	312
15.11. HarmonicSolver — respuesta forzada armónica en el dominio de la frecuencia (ADR 0009 fase 6)	313
15.12. ResponseSpectrumSolver — análisis sísmico por combinación modal (ADR 0009 fase 7)	313
15.13. Cómo añadir un solver nuevo	314
16. Anexos técnicos	316
16.1. Dos capas que se llaman ambas «solver»	316
16.2. Backends disponibles y criterio de selección	316
16.3. Fallback automático SPD $\rightarrow$ LU	317
16.4. Factorización reusable y Newton modificado	317
16.5. Override desde YAML — herramienta de diagnóstico	318
16.6. Activación de Cholesky (opcional)	318

Sobre este Manual

Este manual contiene la **referencia formal** de los componentes de Solidum FEM: cada elemento finito y cada modelo constitutivo se documenta con su especificación física (ecuaciones), su formulación numérica (matrices **B**, rigidez tangente, integración) y su contrato YAML.

El contenido se genera automáticamente desde los archivos `docs/specs/*.md` del repositorio, que constituyen la *fuentes única de verdad* de cada componente. No editar este PDF manualmente; cualquier corrección debe hacerse sobre la spec correspondiente y regenerar el manual mediante:

```
python manuals/build_reference_manual.py
```

Para una guía orientada al uso del programa (sintaxis YAML, ejemplos, post-procesamiento), consulte el **Manual de Usuario** en `manuals/User_manual.pdf`.

Capítulo 1

Elementos 1D — Armaduras

1.1 Truss2D

*Orden de trabajo. El usuario escribe la **especificación física**, la **formulación numérica** y el **contrato**. La IA rellena **implementación** y responde en **diálogo** durante el trabajo.*

1.2 Especificación física

1.2.1 0. Descripción general

Elemento sólido 1D de primer orden, inmerso en el plano. Dos nodos articulados; transmite únicamente fuerza axial. Aproximación C^0 del desplazamiento.

1.2.2 1. Cinemática de desplazamientos

Interpolación lineal del desplazamiento axial a lo largo del eje $s \in [0, L]$:

$$u(s) = a_0 + a_1 s$$

1.2.3 2. Cinemática de deformaciones

Única componente (axial, en el sistema local de la barra):

$$\varepsilon_{ss}(s) = \frac{du}{ds}$$

1.2.4 3. Ecuación constitutiva

$$\sigma_{ss} = E \varepsilon_{ss}$$

1.2.5 4. Equilibrio — forma fuerte

$$-\frac{d}{ds} \left(EA \frac{du}{ds} \right) = b(s), \quad s \in (0, L),$$

con condiciones de contorno Dirichlet o Neumann ($\sigma A = \bar{F}$) en los extremos.

1.2.6 5. Equilibrio — forma débil

$$\int_0^L \frac{d\delta u}{ds} EA \frac{du}{ds} ds = \int_0^L \delta u b ds + [\delta u \bar{F}]_{\partial\Omega}, \quad \forall \delta u \in V_0.$$

1.3 Formulación numérica (FEM)

1.3.1 6. Discretización

Dos nodos en $s = 0$ y $s = L$. Cosenos directores del eje en globales:

$$c = \frac{x_2 - x_1}{L}, \quad s_\theta = \frac{y_2 - y_1}{L}.$$

1.3.2 7. Funciones de forma

$$N_1(s) = 1 - \frac{s}{L}, \quad N_2(s) = \frac{s}{L}.$$

1.3.3 8. Matriz deformación-desplazamiento

Local (1 DOF axial por nodo): $\mathbf{B}_{\text{loc}} = \frac{1}{L}[-1, 1]$. Global ($\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_x^{(2)}, u_y^{(2)}]^\top$):

$$\mathbf{B} = \frac{1}{L}[-c, -s_\theta, c, s_\theta], \quad \varepsilon = \mathbf{B} \mathbf{u}_e.$$

1.3.4 9. Rigidez elemental

Integración analítica exacta (EA constante):

$$\mathbf{K}_e = \int_0^L \mathbf{B}^\top EA \mathbf{B} ds = \frac{EA}{L} \mathbf{d}^\top \mathbf{d}, \quad \mathbf{d} = [-c, -s_\theta, c, s_\theta].$$

1.3.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \sigma A \mathbf{d}.$$

1.3.6 11. Cuadratura

No aplica (forma cerrada).

1.4 Contrato de implementación

```
name: Truss2D
kind: element
status: validated           # draft → implemented → validated

interface:
  dof_names: [ux, uy]
  n_nodes: 2
  strain_dim: 1
```



```

n_integration_points: 1

parameters:
- { name: A, type: float, required: true, desc: Área de la sección transversal }

material_contract:
signature: "compute_state(, state) -> (, E_tangent, state'"
strain_kind: axial escalar

conventions:
sign: " > 0 > 0 (elongación)"
voigt: "N/A (escalar)"
node_orientation: "libre; K_e y F_int invariantes bajo permutación"

validity:
- " || 1e-2"
- pequeños desplazamientos y rotaciones
- cargas exclusivamente axiales

out_of_scope:
- flexión, cortante, momentos en extremos
- grandes desplazamientos
- pandeo

acceptance:
- name: barra axial bajo carga puntual
setup: "empotrada en s=0, carga F axial en s=L"
expect: "u(L) = F·L / (E·A)"
tol_rel: 1.0e-10

references:
- "Bathe K.-J., Finite Element Procedures, §4.2.1"

```

1.5 Implementación

- **Archivo:** solidum/elements/truss.py
- **Clase:** Truss2D (registrada vía `@ElementRegistry.register`)
- **Tests:**
 - tests/test_truss.py · `TestTruss2D` — verifica L_0 , cosenos directores, entradas de $\mathbf{K}_e$ (incluyendo simetría), y evaluación de $\mathbf{F}_{\text{int}}$ sobre desplazamientos conocidos. Los valores numéricos esperados coinciden con $\mathbf{K}_e = (EA/L) \mathbf{d} \mathbf{d}^\top$ y $\mathbf{F}_{\text{int}} = N \mathbf{d}$ para geometría de prueba $L = 5$, $c = 0,6$, $s = 0,8$.
 - tests/test_integration.py · `TestSolversIntegration` — resuelve end-to-end el caso de aceptación (barra empotrada en un extremo, carga axial en el otro) con `NonlinearSolver` y `ArcLengthSolver`; en régimen elástico precedente a la fluencia reproduce $u(L) = FL/(EA)$.
- **Notas de traducción:**
 - L_0 , c , s se calculan una vez en `__init__` sobre la configuración inicial y no se actualizan — coherente con el régimen infinitesimal declarado.
 - El contrato con el material es el estándar del proyecto: `material.compute_state( $\epsilon$ , state) → ( $\sigma$ ,  $E_t$ , state')`.

- Para grandes desplazamientos/rotaciones usar `Truss2DCorot`, que hereda de esta clase.

1.6 Diálogo

- **2026-04-21** · Cierre de ciclo retroactivo. La clase `Truss2D` preexistía al flujo `spec-first`; la `spec` se creó documentando la formulación ya implementada. Los tests unitarios y de integración satisfacen el único criterio de **acceptance** declarado (analíticamente, vía los valores de $\mathbf{K}_e$ y end-to-end vía el solver). Promovido **status: validated**.

1.7 Truss2DCorot

*Orden de trabajo. Usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

1.8 Especificación física

1.8.1 0. Descripción general

Barra articulada 2D equivalente a **Truss2D** pero en régimen de **grandes desplazamientos y rotaciones** con **pequeña deformación axial**. La cinemática se actualiza en configuración corriente (corotacional / Updated Lagrangian): el eje de la barra gira con el movimiento y la deformación se mide sobre la longitud actual. Rigidez tangente = rigidez material + rigidez geométrica debida a la fuerza axial.

1.8.2 1. Cinemática

Configuraciones:

- Referencia: nodos $\mathbf{X}_1, \mathbf{X}_2$, longitud $L_0 = \|\mathbf{X}_2 - \mathbf{X}_1\|$.
- Corriente: $\mathbf{x}_i = \mathbf{X}_i + \mathbf{u}_i$, longitud $l = \|\mathbf{x}_2 - \mathbf{x}_1\|$.

Cosenos directores en configuración corriente:

$$c_\theta = \frac{x_2 - x_1}{l}, \quad s_\theta = \frac{y_2 - y_1}{l}.$$

1.8.3 2. Medida de deformación

Ingenieril corotacional (válida para pequeña deformación aunque la rotación sea grande):

$$\varepsilon = \frac{l - L_0}{L_0}.$$

1.8.4 3. Ecuación constitutiva

Material 1D: $\sigma = E \varepsilon$ (elástico lineal); el contrato admite cualquier material con **STRAIN_DIM** = 1.

1.8.5 4. Equilibrio — forma débil incremental

Principio de trabajos virtuales completo:

$$\underbrace{\int_V \delta \varepsilon \sigma dV}_{\delta W_{\text{int}}} = \underbrace{\int_V \delta \mathbf{u}^\top \mathbf{b} dV + \int_{\partial V_t} \delta \mathbf{u}^\top \bar{\mathbf{t}} dA}_{\delta W_{\text{ext}}}.$$

El elemento calcula **solo el lado interno**:

$$\delta W_{\text{int}} = \delta \mathbf{u}_e^\top \mathbf{F}_{\text{int}},$$

con la rigidez tangente consistente separada en parte material y geométrica (ver §9). Las cargas externas (nodales y, si las hubiera, de cuerpo o tracción) las ensambla el solver a partir del input del usuario; este elemento **no** produce vector nodal equivalente de fuerzas de cuerpo distribuidas a lo largo del eje.

1.9 Formulación numérica (FEM)

1.9.1 6. Discretización

Dos nodos, 2 DOFs por nodo (u_x, u_y) . Vector elemental $\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_x^{(2)}, u_y^{(2)}]^\top$.

1.9.2 7. Vector dirección actual

$$\mathbf{d} = [-c_\theta, -s_\theta, c_\theta, s_\theta]^\top, \quad \mathbf{n} = [-s_\theta, c_\theta, s_\theta, -c_\theta]^\top,$$

con c_θ, s_θ evaluados en la configuración corriente.

1.9.3 8. Matriz B corotacional

$$\mathbf{B} = \frac{1}{L_0} \mathbf{d}^\top, \quad \varepsilon = \mathbf{B} \mathbf{u}_e.$$

1.9.4 9. Rigidez tangente

$$\begin{aligned} \mathbf{K}_T &= \mathbf{K}_M + \mathbf{K}_G, \\ \mathbf{K}_M &= \frac{EA}{L_0} \mathbf{d} \mathbf{d}^\top, \\ \mathbf{K}_G &= \frac{N}{l} \mathbf{n} \mathbf{n}^\top, \quad N = \sigma A. \end{aligned}$$

1.9.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = N \mathbf{d}.$$

1.9.6 11. Cuadratura

No aplica (forma cerrada; un único punto conceptual en el eje).

1.10 Contrato de implementación

```
name: Truss2DCorot
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: Área de la sección transversal }
```

```

material_contract:
  signature: "compute_state(, state) -> (, E_tangent, state')"
  strain_kind: axial escalar (deformación ingenieril corotacional)

conventions:
  sign: " > 0 > 0 (elongación)"
  voigt: "N/A (escalar)"
  node_orientation: "libre; K_T y F_int invariantes bajo permutación"
  configuration: "Updated Lagrangian - c_, s_, l se recalculan en cada evaluación"

validity:
  - "|| 1e-2 (pequeña deformación axial)"
  - rotaciones y desplazamientos de cualquier magnitud
  - cargas exclusivamente axiales

out_of_scope:
  - flexión, cortante, momentos
  - pandeo por bifurcación (captura de snap-through requiere arc-length)
  - plasticidad con grandes deformaciones
  - "fuerzas de cuerpo distribuidas (peso propio sobre la barra): el elemento no
    calcula vector nodal equivalente; el usuario reparte masa a los nodos en el input"

acceptance:
  - name: equivalencia con Truss2D en régimen lineal
    setup: "barra axial empotrada, carga F pequeña ( ~ 1e-6)"
    expect: "u(L) = F·L/(E·A) con tol_rel 1e-8"
    tol_rel: 1.0e-8

  - name: invariancia bajo rotación de cuerpo rígido
    setup: "aplicar rotación rígida de 45° a ambos nodos (sin deformar)"
    expect: "F_int = 0 y = 0"
    tol_abs: 1.0e-10

  - name: rigidez geométrica bajo tensión
    setup: "barra en tensión con N > 0; verificar autovalor transversal"
    expect: "autovalor de K_T en dirección n = N/l"
    tol_rel: 1.0e-10

references:
  - "Crisfield M.A., Non-linear Finite Element Analysis of Solids and Structures, vol
    .1, §3.3"
  - "Belytschko T., Nonlinear Finite Elements for Continua and Structures, §4.5"

```

1.11 Implementación

- **Archivo:** solidum/elements/truss.py
- **Clase:** Truss2DCorot — hereda de Truss2D (reutiliza `__init__`, `ClassVars` y metadatos de registro) y sobrescribe `compute_element_state` y `compute_internal_forces` para evaluar cosenos directores y longitud en configuración corriente.
- **Tests:** tests/test_truss.py · TestTruss2DCorot — tres tests, uno por cada acceptance:
- test_acceptance_linear_limit_matches_truss2d (criterio 1).
- test_acceptance_rigid_body_rotation (criterio 2).

- `test_acceptance_geometric_stiffness_under_traction` (criterio 3).
- **Notas de traducción:**
- La longitud corriente l y los cosenos c_θ, s_θ se recalculan en cada evaluación (no se cachean entre llamadas del solver, como exige Updated Lagrangian).
- El vector $\mathbf{d}$ de §7 se construye con signos $[-c_\theta, -s_\theta, c_\theta, s_\theta]$; el transverso $\mathbf{n}$ como $[-s_\theta, c_\theta, s_\theta, -c_\theta]$. Ambos con norma $\sqrt{2}$ — el factor se absorbe en los coeficientes EA/L_0 y N/l .
- La rigidez tangente se devuelve ya sumada $\mathbf{K}_M + \mathbf{K}_G$; el solver no distingue las dos contribuciones.
- El test 3 confronta $\mathbf{K}_T$ con la descomposición esperada y además verifica $\mathbf{K}_G \mathbf{n} = (2N/l)\mathbf{n}$ (autovalor sobre el vector sin normalizar, que es N/l sobre el versor — así queda la ambigüedad resuelta explícitamente).

1.12 Diálogo

- **2026-04-21** · Implementación inicial tras validar el diseño spec-first. Se optó por herencia `Truss2DCorot(Truss2D)` para reutilizar el constructor (L_0 , cosenos iniciales) y los `ClassVars` del registro; los métodos que dependen de configuración corriente se sobrescriben. La bandera `large_strains` que tenía `Truss2D` previamente se elimina en el mismo commit — el régimen corotacional vive ahora como elemento independiente.
- **2026-04-21** · Normalización del autovalor transverso (criterio 3): la fórmula del libro da N/l sobre el **versor** $\hat{\mathbf{n}} = \mathbf{n}/\sqrt{2}$; sobre el vector $\mathbf{n}$ directo (norma $\sqrt{2}$) el autovalor es $2N/l$. El test verifica la segunda forma para evitar raíces cuadradas y la nota de traducción documenta la equivalencia.

1.13 Truss3D

*Orden de trabajo. El usuario escribe la **especificación física**, la **formulación numérica** y el **contrato**. La IA rellena **implementación** y responde en **diálogo** durante el trabajo.*

1.14 Especificación física

1.14.1 0. Descripción general

Elemento sólido 1D de primer orden, inmerso en el espacio tridimensional. Dos nodos articulados; transmite únicamente fuerza axial. Aproximación C^0 del desplazamiento. Régimen estrictamente de **linealidad geométrica**: pequeños desplazamientos, pequeñas rotaciones, pequeñas deformaciones.

1.14.2 1. Cinemática de desplazamientos

Interpolación lineal del desplazamiento axial a lo largo del eje $s \in [0, L]$:

$$u(s) = a_0 + a_1 s.$$

1.14.3 2. Cinemática de deformaciones

Única componente (axial, en el sistema local de la barra):

$$\varepsilon_{ss}(s) = \frac{du}{ds}.$$

1.14.4 3. Ecuación constitutiva

$$\sigma_{ss} = E \varepsilon_{ss}.$$

El elemento admite cualquier material 1D con STRAIN_DIM = 1 (elástico lineal, plástico 1D, daño 1D).

1.14.5 4. Equilibrio — forma fuerte

$$-\frac{d}{ds} \left(EA \frac{du}{ds} \right) = b(s), \quad s \in (0, L),$$

con condiciones de contorno Dirichlet o Neumann ($\sigma A = \bar{F}$) en los extremos.

1.14.6 5. Equilibrio — forma débil

$$\int_0^L \frac{d\delta u}{ds} EA \frac{du}{ds} ds = \int_0^L \delta u b ds + [\delta u \bar{F}]_{\partial\Omega}, \quad \forall \delta u \in V_0.$$

El elemento calcula **solo el lado interno**; las cargas externas las ensambla el solver a partir del input del usuario.

1.15 Formulación numérica (FEM)

1.15.1 6. Discretización

Dos nodos en $s = 0$ y $s = L$. Cosenos directores del eje en coordenadas globales:

$$c_x = \frac{x_2 - x_1}{L}, \quad c_y = \frac{y_2 - y_1}{L}, \quad c_z = \frac{z_2 - z_1}{L},$$

con $L = \|\mathbf{X}_2 - \mathbf{X}_1\|$ (configuración inicial, fija).

1.15.2 7. Funciones de forma

$$N_1(s) = 1 - \frac{s}{L}, \quad N_2(s) = \frac{s}{L}.$$

1.15.3 8. Matriz deformación-desplazamiento

Local (1 DOF axial por nodo): $\mathbf{B}_{\text{loc}} = \frac{1}{L}[-1, 1]$. Global con $\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_z^{(1)}, u_x^{(2)}, u_y^{(2)}, u_z^{(2)}]^\top$:

$$\mathbf{B} = \frac{1}{L}[-c_x, -c_y, -c_z, c_x, c_y, c_z], \quad \varepsilon = \mathbf{B} \mathbf{u}_e.$$

1.15.4 9. Rigidez elemental

Integración analítica exacta (EA constante, $\mathbf{B}$ constante):

$$\mathbf{K}_e = \int_0^L \mathbf{B}^\top EA \mathbf{B} ds = \frac{EA}{L} \mathbf{d} \mathbf{d}^\top, \quad \mathbf{d} = [-c_x, -c_y, -c_z, c_x, c_y, c_z]^\top.$$

$\mathbf{K}_e$ es una matriz 6×6 de rango 1.

1.15.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \sigma A \mathbf{d} = N \mathbf{d}.$$

1.15.6 11. Cuadratura

No aplica (forma cerrada, $\mathbf{B}$ y σ uniformes por elemento). El campo `n_integration_points: 1` se refiere al número de estados materiales conceptuales (uno por elemento), no a puntos de Gauss.

1.16 Contrato de implementación

```
name: Truss3D
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1
```



```

parameters:
- { name: A, type: float, required: true, desc: Área de la sección transversal }

material_contract:
signature: "compute_state(, state) -> (, E_tangent, state)"
strain_kind: axial escalar

conventions:
sign: " > 0 > 0 (elongación)"
voigt: "N/A (escalar)"
node_orientation: "libre; K_e y F_int invariantes bajo permutación"
configuration: "inicial fija - L, c_x, c_y, c_z no se actualizan con los
desplazamientos"

validity:
- "|| 1e-2"
- pequeños desplazamientos y rotaciones
- cargas exclusivamente axiales

out_of_scope:
- flexión, cortante, momentos
- grandes desplazamientos o rotaciones (para ese régimen usar `Truss3DCorot`)
- pandeo
- "fuerzas de cuerpo distribuidas sobre el eje: el usuario reparte masa a los nodos"

acceptance:
- name: barra axial bajo carga puntual (3D)
setup: "empotrada en s=0, carga F axial en s=L, orientación arbitraria en el
espacio"
expect: "u(L) = F·L / (E·A) proyectado sobre el eje"
tol_rel: 1.0e-10

references:
- "Bathe K.-J., Finite Element Procedures, §4.2.1"
- "Cook R.D., Concepts and Applications of FEA, §2.3"

```

1.17 Implementación

- **Archivo:** solidum/elements/truss.py
- **Clase:** Truss3D — hereda directamente de Element (sin relación de herencia con Truss2D ni con Truss2DCorot).
- **Tests:** tests/test_truss.py · TestTruss3D — verifica L0, cosenos directores (c_x, c_y, c_z), entradas de $\mathbf{K}_e$ (incluyendo simetría, dimensiones 6×6) y evaluación de $\mathbf{F}_{\text{int}}$ sobre desplazamientos conocidos con geometría de prueba $L = 13$, $(c_x, c_y, c_z) = (3/13, 4/13, 12/13)$.
- **Notas de traducción:**
 - L0, c_x , c_y , c_z se calculan una vez en `__init__` sobre la configuración inicial y no se actualizan — coherente con el régimen geoméricamente lineal declarado.
 - El constructor tolera nodos con 2 ó 3 coordenadas (completa con $z = 0$ si falta), para facilitar transición de problemas planos embebidos.

- El contrato con el material es el estándar del proyecto: `material.compute_state( $\varepsilon$ , state)` $\rightarrow$ (σ , E_t , state').
- Para grandes desplazamientos/rotaciones en 3D usar `Truss3DCorot`, que hereda de esta clase.

1.18 Diálogo

- **2026-04-21** · Apertura de spec retroactiva. La clase `Truss3D` preexistía al flujo spec-first; la spec se crea documentando la formulación ya implementada y estableciendo explícitamente su alcance: régimen de linealidad geométrica, sin bandera ni variante corotacional. Los tests unitarios existentes cubren el criterio de **acceptance** (vía los valores de $\mathbf{K}_e = (EA/L)\mathbf{d}\mathbf{d}^\top$ que implican algebraicamente $u(L) = FL/(EA)$). Promovido **status: validated**.

1.19 Truss3DCorot

*Orden de trabajo. Usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

1.20 Especificación física

1.20.1 0. Descripción general

Barra articulada 3D equivalente a Truss3D pero en régimen de **grandes desplazamientos y rotaciones** con **pequeña deformación axial**. Updated Lagrangian / corotacional: el eje de la barra se redefine en cada iteración sobre la configuración corriente y la deformación se mide sobre la longitud actual. La rigidez tangente suma material + geométrica.

1.20.2 1. Cinemática

Configuraciones:

- Referencia: $\mathbf{X}_1, \mathbf{X}_2 \in \mathbb{R}^3$, longitud $L_0 = \|\mathbf{X}_2 - \mathbf{X}_1\|$.
- Corriente: $\mathbf{x}_i = \mathbf{X}_i + \mathbf{u}_i$, longitud $l = \|\mathbf{x}_2 - \mathbf{x}_1\|$.

Vector unitario del eje corriente:

$$\hat{\mathbf{e}} = \frac{\mathbf{x}_2 - \mathbf{x}_1}{l} = (c_x, c_y, c_z), \quad c_x^2 + c_y^2 + c_z^2 = 1.$$

1.20.3 2. Medida de deformación

Ingenieril corotacional:

$$\varepsilon = \frac{l - L_0}{L_0}.$$

1.20.4 3. Ecuación constitutiva

Material 1D con STRAIN_DIM = 1: $\sigma = \sigma(\varepsilon)$ (el elemento no hardcodea la ley; la provee el material).

1.20.5 4. Equilibrio — forma débil incremental

PTV estándar. El elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \delta \mathbf{u}_e^\top \mathbf{F}_{\text{int}}, \quad \mathbf{K}_T = \frac{\partial \mathbf{F}_{\text{int}}}{\partial \mathbf{u}_e} = \mathbf{K}_M + \mathbf{K}_G.$$

1.21 Formulación numérica (FEM)

1.21.1 6. Discretización

Dos nodos, 3 DOFs por nodo (u_x, u_y, u_z). Vector elemental de 6 componentes:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_z^{(1)}, u_x^{(2)}, u_y^{(2)}, u_z^{(2)}]^\top.$$

1.21.2 7. Vector dirección actual

$$\mathbf{d} = [-c_x, -c_y, -c_z, c_x, c_y, c_z]^\top,$$

con los cosenos evaluados en configuración **corriente**. $\|\mathbf{d}\|^2 = 2$.

1.21.3 8. Matriz B corotacional

$$\mathbf{B} = \frac{1}{L_0} \mathbf{d}^\top, \quad \varepsilon = \mathbf{B} \mathbf{u}_e.$$

1.21.4 9. Rigidez tangente

Material:

$$\mathbf{K}_M = \frac{E_t A}{L_0} \mathbf{d} \mathbf{d}^\top \quad (6 \times 6, \text{ rango } 1).$$

Geométrica: en 3D la dirección perpendicular al eje es un **plano** (no un vector único). Se usa el proyector perpendicular $\mathbf{P}_3 = \mathbf{I}_3 - \hat{\mathbf{e}}\hat{\mathbf{e}}^\top$ (matriz 3×3 , rango 2). La rigidez geométrica en el elemento de 6 DOFs:

$$\mathbf{K}_G = \frac{N}{l} \begin{bmatrix} \mathbf{P}_3 & -\mathbf{P}_3 \\ -\mathbf{P}_3 & \mathbf{P}_3 \end{bmatrix} \quad (6 \times 6, \text{ rango } 2), \quad N = \sigma A.$$

1.21.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = N \mathbf{d}.$$

1.21.6 11. Cuadratura

No aplica (forma cerrada).

1.22 Contrato de implementación

```
name: Truss3DCorot
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: Área de la sección transversal }

material_contract:
  signature: "compute_state(, state) -> (, E_tangent, state)"
  strain_kind: axial escalar (deformación ingenieril corotacional)

conventions:
```

```

sign: " > 0    > 0 (elongación)"
voigt: "N/A (escalar)"
node_orientation: "libre"
configuration: "Updated Lagrangian - l, c_x, c_y, c_z se recalculan en cada evaluación"

validity:
- "|| 1e-2"
- rotaciones y desplazamientos de cualquier magnitud en el espacio
- cargas exclusivamente axiales

out_of_scope:
- flexión, cortante, momentos
- pandeo por bifurcación
- plasticidad con grandes deformaciones
- "fuerzas de cuerpo distribuidas: el usuario reparte masa a los nodos"

acceptance:
- name: equivalencia con Truss3D en régimen lineal
  setup: "barra 3D con orientación arbitraria, carga axial pequeña ( ~ 1e-6)"
  expect: "K_T y F_int coinciden con Truss3D lineal a tolerancia rtol=1e-4"
  tol_rel: 1.0e-4

- name: invariancia bajo rotación rígida 3D
  setup: "aplicar una rotación rígida arbitraria (p. ej. 30° sobre eje z, luego 45° sobre eje x)"
  expect: "F_int = 0 y    = 0"
  tol_abs: 1.0e-10

- name: rigidez geométrica transversa (plano perpendicular)
  setup: "barra en tensión con N > 0; aplicar K_G sobre dos vectores perpendiculares al eje linealmente independientes"
  expect: "K_G·v_perp = (N/l)·v_perp_struct, donde v_perp_struct es la extensión del vector transverso al espacio de 6 DOFs con signos opuestos en los dos nodos"
  tol_rel: 1.0e-10

references:
- "Crisfield M.A., Non-linear Finite Element Analysis of Solids and Structures, vol .1, §3.3"
- "Belytschko T., Nonlinear Finite Elements for Continua and Structures, §4.5"

```

1.23 Implementación

- **Archivo:** solidum/elements/truss.py
- **Clase:** Truss3DCorot — hereda de Truss3D (reutiliza `__init__`, tolerancia a nodos 2D/3D, metadatos de registro) y sobrescribe `compute_element_state` y `compute_internal_forces` para evaluar longitud y cosenos directores en configuración corriente.
- **Tests:** tests/test_truss.py · TestTruss3DCorot — tres tests, uno por cada acceptance:
- test_acceptance_linear_limit_matches_truss3d (criterio 1).
- test_acceptance_rigid_body_rotation (criterio 2 — dos rotaciones rígidas distintas, verifica $\sigma=0$ y $F_{int}=0$ en ambas).

- `test_acceptance_geometric_stiffness_transverse_plane` (criterio 3 — aplica $\mathbf{K}_G$ a dos direcciones linealmente independientes del plano perpendicular al eje).
- **Notas de traducción:**
- La clave respecto a 2D es el proyector $\mathbf{P}_3 = \mathbf{I} - \hat{\mathbf{e}}\hat{\mathbf{e}}^\top$ (3×3 , rango 2): el plano perpendicular al eje es bidimensional. $\mathbf{K}_G$ resulta entonces de rango 2, no rango 1.
- El proyector se construye con `solidum.math.geometry.perpendicular_projector( $\hat{\mathbf{e}}$ )`, helper compartido con `Cable3DCorot` (operación geométrica pura, no específica de armaduras ni de cables).
- La longitud corriente l y los cosenos c_x, c_y, c_z se recalculan en cada evaluación (Updated Lagrangian); no se cachean entre llamadas del solver.
- Autovalores de $\mathbf{K}_G$ sobre modos transversos (rotación de la barra alrededor de su centro): $2N/l$ sobre el vector sin normalizar — igual que en 2D, el factor 2 viene de que el vector de 6 componentes tiene norma $\sqrt{2}$. Sobre el versor el autovalor es N/l .

1.24 Diálogo

- **2026-04-21** · Implementación tras validar el diseño spec-first. Se optó por herencia `Truss3DCorot(Truss3D)` por paralelismo con `Truss2DCorot(Truss2D)` y reutilización del constructor. La diferencia estructural con el caso 2D es el proyector $\mathbf{P}$: en 2D tenía rango 1 y generaba un $\mathbf{K}_G$ de rango 1 (vector $\mathbf{n}$ único); en 3D tiene rango 2 y $\mathbf{K}_G$ rango 2 (plano perpendicular al eje). Los tests del criterio 3 verifican explícitamente **dos** direcciones transversas linealmente independientes, capturando este cambio dimensional.
- **2026-04-21** · Test de invariancia bajo rotación rígida 3D: se verifican dos rotaciones distintas (plano x-y y plano x-z) para descartar coincidencias accidentales en ejes canónicos.

Capítulo 2

Elementos 1D — Cables

2.1 Cable2DCorot

*Orden de trabajo. El usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

2.2 Especificación física

2.2.1 0. Descripción general

Elemento 1D inmerso en el plano 2D que modela un **cable**: dos nodos articulados que solo transmiten **fuerza axial de tensión**. En compresión o estado sin tensar, el elemento no transmite nada — se destensa. Cinemática **corotacional** (Updated Lagrangian): el eje rota con el movimiento, la longitud se mide sobre la configuración corriente.

La propiedad de unilateralidad vive en el **material**; el elemento es la maquinaria cinemática que aplica al material la deformación correcta. Para obtener el comportamiento propio de cable se empareja este elemento con un material unilateral como **CableMaterial1D**. Con un material elástico clásico, el elemento se comporta como una barra articulada corotacional (útil para pruebas, no para modelado realista de cables).

2.2.2 1. Cinemática

Configuraciones:

- Referencia: $\mathbf{X}_1, \mathbf{X}_2 \in \mathbb{R}^2$, longitud $L_0 = \|\mathbf{X}_2 - \mathbf{X}_1\|$.
- Corriente: $\mathbf{x}_i = \mathbf{X}_i + \mathbf{u}_i$, longitud $l = \|\mathbf{x}_2 - \mathbf{x}_1\|$.

Cosenos directores del eje **corriente**:

$$c_\theta = \frac{x_2 - x_1}{l}, \quad s_\theta = \frac{y_2 - y_1}{l}.$$

2.2.3 2. Medida de deformación

Ingenieril corotacional:

$$\varepsilon = \frac{l - L_0}{L_0}.$$

2.2.4 3. Ecuación constitutiva

Delegada al material con `STRAIN_DIM = 1`. El elemento invoca `material.compute_state(ε, state)` y recibe $(\sigma, E_t, \text{nuevo estado})$ sin inspeccionar su naturaleza. La unilateralidad (si la hay) es responsabilidad del material — el elemento nunca filtra σ ni ε por signo.

2.2.5 4. Equilibrio — forma débil incremental

Principio de trabajos virtuales, lado interno:

$$\delta W_{\text{int}} = \int_V \delta \varepsilon \sigma dV = \delta \mathbf{u}_e^\top \mathbf{F}_{\text{int}}, \quad \mathbf{K}_T = \frac{\partial \mathbf{F}_{\text{int}}}{\partial \mathbf{u}_e} = \mathbf{K}_M + \mathbf{K}_G.$$

Las cargas externas las ensambla el solver. El elemento no calcula vector nodal equivalente de fuerzas de cuerpo.

2.3 Formulación numérica (FEM)

2.3.1 6. Discretización

Dos nodos, 2 DOFs por nodo (u_x, u_y) . Vector elemental:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_x^{(2)}, u_y^{(2)}]^\top.$$

2.3.2 7. Vectores de dirección (configuración corriente)

$$\mathbf{d} = [-c_\theta, -s_\theta, c_\theta, s_\theta]^\top, \quad \mathbf{n} = [-s_\theta, c_\theta, s_\theta, -c_\theta]^\top.$$

$\mathbf{d}$ apunta a lo largo del eje corriente (norma $\sqrt{2}$); $\mathbf{n}$ al perpendicular (norma $\sqrt{2}$). Son ortogonales: $\mathbf{d}^\top \mathbf{n} = 0$.

2.3.3 8. Matriz B corotacional

$$\mathbf{B} = \frac{1}{L_0} \mathbf{d}^\top, \quad \varepsilon = \mathbf{B} \mathbf{u}_e.$$

2.3.4 9. Rigidez tangente

$$\mathbf{K}_T = \mathbf{K}_M + \mathbf{K}_G, \\ \mathbf{K}_M = \frac{E_t A}{L_0} \mathbf{d} \mathbf{d}^\top, \quad \mathbf{K}_G = \frac{N}{l} \mathbf{n} \mathbf{n}^\top, \quad N = \sigma A.$$

Caso destensado (material unilateral con $\varepsilon \leq 0$): el material devuelve $\sigma = 0$, $E_t = 0$, luego $N = 0$ y **ambas contribuciones se anulan** — $\mathbf{K}_T = \mathbf{0}$. El elemento aporta cero al sistema global, como físicamente corresponde a un cable flojo.

2.3.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = N \mathbf{d}.$$

Cuando el cable está destensado ($N = 0$), $\mathbf{F}_{\text{int}} = \mathbf{0}$.

2.3.6 11. Cuadratura

No aplica (forma cerrada).

2.4 Contrato de implementación

```

name: Cable2DCorot
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: "Área de la sección transversal del cable" }

material_contract:
  signature: "compute_state(, state) -> (, E_tangent, state')"
  strain_kind: axial escalar (deformación ingenieril corotacional)
  expected_behaviour: "material unilateral (p. ej. CableMaterial1D) para obtener respuesta física de cable; cualquier material con STRAIN_DIM=1 es técnicamente aceptado pero producirá comportamiento de barra"

conventions:
  sign: " > 0 > 0 (elongación); la responsabilidad de anular en compresión es del material"
  voigt: "N/A (escalar)"
  node_orientation: "libre; K_T y F_int invariantes bajo permutación"
  configuration: "Updated Lagrangian - l, c_, s_ se recalculan en cada evaluación"

validity:
  - "|| 1e-2 en régimen tensado"
  - rotaciones y desplazamientos de cualquier magnitud
  - cargas axiales (las transversales se transmiten por el destensado-retensado del cable)

out_of_scope:
  - flexión, cortante, momentos
  - dinámica con inercia distribuida (sin matriz de masa)
  - pandeo (irrelevante: el cable no transmite compresión)
  - "fuerzas de cuerpo distribuidas: el usuario reparte masa/peso a los nodos"
  - pretensión mediante parámetro del elemento (modelar con longitud inicial ajustada)

numerical_caveats:

```

```

- "Un cable completamente destensado aporta  $K_T = 0$  al sistema global. Si otros
  elementos no garantizan rigidez suficiente en los DOFs compartidos, la matriz
  tangente global puede ser singular o mal condicionada."
- "En transiciones tensado destensado ( cruza 0), Newton-Raphson puede oscilar por
  el salto discontinuo en  $E_t$ . Usar arc-length o pasos de carga finos si el
  problema atraviesa destensiones."

acceptance:
- name: cable tensado coincide con barra corotacional
  setup: "cable horizontal con material lineal (Elastic1D,  $E=1000$ ),  $u_x$  nodo 2 =  $1e-3$ 
        =  $1e-3 > 0$ "
  expect: " =  $E$  ,  $E_t = E$ ,  $K_T = K_M + K_G$  con valores de barra corotacional"
  tol_rel: 1.0e-10

- name: cable destensado produce aportación nula
  setup: "cable horizontal con CableMaterial1D,  $u_x$  nodo 2 =  $-1e-3$  < 0"
  expect: " = 0,  $F_{int} = 0$ ,  $K_T =$  matriz cero"
  tol_abs: 1.0e-12

- name: invariancia bajo rotación rígida
  setup: "cable inicialmente horizontal, rotación rígida de  $45^\circ$  de ambos nodos"
  expect: " = 0 y  $F_{int} = 0$  (independiente del material)"
  tol_abs: 1.0e-10

- name: cruce por cero
  setup: " $u_e$  tal que = 0 exactamente, material unilateral"
  expect: " = 0,  $E_t = 0$ ,  $F_{int} = 0$ ,  $K_T = 0$ "
  tol_abs: 1.0e-12

references:
- "Crisfield M.A., Non-linear Finite Element Analysis of Solids and Structures, vol
  .1, §3.3"
- "Irvine H.M., Cable Structures, MIT Press, §1.2"

```

2.5 Implementación

- **Archivo:** solidum/elements/cable.py — archivo propio, no referencia a los módulos de armadura ni a ningún otro elemento.
- **Clase:** Cable2DCorot — hereda directamente de Element (no de Truss2DCorot ni de ningún elemento de armadura). La maquinaria cinemática corotacional se reimplementa íntegra dentro de la clase.
- **Tests:** tests/test_cable_elements.py · TestCable2DCorot — los cuatro criterios de acceptance más un test de registro:
- test_acceptance_cable_tensado_como_barra_corot (criterio 1).
- test_acceptance_cable_destensado_aportacion_nula (criterio 2).
- test_acceptance_rotacion_rigida (criterio 3).
- test_acceptance_cruce_por_cero (criterio 4).
- test_registro_en_registry — autodiscover.

- **Notas de traducción:**

- La clase duplica deliberadamente la lógica de `Truss2DCorot`. No hay herencia. Es el precio de la independencia: si mañana se toca la armadura corotacional, este cable no se mueve.
- `_current_geometry(u_e)` devuelve $(1, c_\theta, s_\theta)$ — método privado para aislar la única operación que depende de configuración corriente.
- En estado destensado, $\mathbf{K}_M = \mathbf{0}$ (porque $E_t = 0$ viene del material) y $\mathbf{K}_G = \mathbf{0}$ (porque $N = 0$); el elemento aporta literalmente ceros al ensamblaje, como debe ser físicamente un cable flojo. Los tests lo verifican con tolerancia absoluta.
- El elemento no valida que el material sea unilateral: es responsabilidad del usuario emparejar con `CableMaterial1D`. Emparejar con `Elastic1D` produce respuesta de barra corotacional y se usa internamente en el test del criterio 1 para comparar contra valores analíticos.

2.6 Diálogo

- **2026-04-21** · Elemento creado como componente totalmente independiente de `Truss2DCorot` por decisión explícita del usuario: la maquinaria cinemática se duplica dentro de la clase de cable para desacoplar su evolución futura del elemento de armadura. El coste es ~40 líneas duplicadas; la ganancia es que ninguna refactorización de las armaduras puede contaminar silenciosamente el comportamiento de los cables.
- **2026-04-21** · Validación de material unilateral: se optó por **no** imponerla en `__init__`. Motivo: el contrato elemento↔material del proyecto es genérico sobre `STRAIN_DIM=1`; introducir una verificación específica crearía un acoplamiento entre la clase del elemento y la identidad del material. Se documenta en `material_contract.expected_behaviour` del YAML para orientar al usuario.

2.7 Cable3DCorot

*Orden de trabajo. El usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

2.8 Especificación física

2.8.1 0. Descripción general

Elemento 1D inmerso en el espacio 3D que modela un **cable**: dos nodos articulados que solo transmiten **fuerza axial de tensión**. En compresión o estado sin tensar, el elemento no transmite nada. Cinemática **corotacional** (Updated Lagrangian): el eje rota libremente en el espacio, la longitud se mide sobre la configuración corriente.

La unilateralidad vive en el **material**; el elemento es la maquinaria cinemática 3D. Para obtener respuesta física de cable, emparejar con **CableMaterial1D**. Con un material elástico clásico, el elemento se comporta como una barra corotacional 3D.

2.8.2 1. Cinemática

- Referencia: $\mathbf{X}_1, \mathbf{X}_2 \in \mathbb{R}^3$, longitud $L_0 = \|\mathbf{X}_2 - \mathbf{X}_1\|$.
- Corriente: $\mathbf{x}_i = \mathbf{X}_i + \mathbf{u}_i$, longitud $l = \|\mathbf{x}_2 - \mathbf{x}_1\|$.

Vector unitario del eje corriente:

$$\hat{\mathbf{e}} = \frac{\mathbf{x}_2 - \mathbf{x}_1}{l} = (c_x, c_y, c_z), \quad c_x^2 + c_y^2 + c_z^2 = 1.$$

2.8.3 2. Medida de deformación

Ingenieril corotacional:

$$\varepsilon = \frac{l - L_0}{L_0}.$$

2.8.4 3. Ecuación constitutiva

Delegada al material con `STRAIN_DIM = 1`. El elemento llama `material.compute_state( $\varepsilon$ , state)` y recibe (σ , E_t , nuevo estado) sin inspeccionar su naturaleza. La unilateralidad (si la hay) es responsabilidad del material.

2.8.5 4. Equilibrio — forma débil incremental

PTV estándar; el elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \delta \mathbf{u}_e^T \mathbf{F}_{\text{int}}, \quad \mathbf{K}_T = \frac{\partial \mathbf{F}_{\text{int}}}{\partial \mathbf{u}_e} = \mathbf{K}_M + \mathbf{K}_G.$$

2.9 Formulación numérica (FEM)

2.9.1 6. Discretización

Dos nodos, 3 DOFs por nodo (u_x, u_y, u_z) . Vector elemental de 6 componentes:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_z^{(1)}, u_x^{(2)}, u_y^{(2)}, u_z^{(2)}]^\top.$$

2.9.2 7. Vector dirección actual

$$\mathbf{d} = [-c_x, -c_y, -c_z, c_x, c_y, c_z]^\top, \quad \|\mathbf{d}\|^2 = 2,$$

con los cosenos evaluados en configuración **corriente**.

2.9.3 8. Matriz B corotacional

$$\mathbf{B} = \frac{1}{L_0} \mathbf{d}^\top, \quad \varepsilon = \mathbf{B} \mathbf{u}_e.$$

2.9.4 9. Rigidez tangente

Material:

$$\mathbf{K}_M = \frac{E_t A}{L_0} \mathbf{d} \mathbf{d}^\top \quad (6 \times 6, \text{ rango } 1).$$

Geométrica: en 3D la dirección perpendicular al eje es un plano bidimensional. Se usa el proyector perpendicular $\mathbf{P}_3 = \mathbf{I}_3 - \hat{\mathbf{e}} \hat{\mathbf{e}}^\top$ (3×3 , rango 2):

$$\mathbf{K}_G = \frac{N}{l} \begin{bmatrix} \mathbf{P}_3 & -\mathbf{P}_3 \\ -\mathbf{P}_3 & \mathbf{P}_3 \end{bmatrix} \quad (6 \times 6, \text{ rango } 2), \quad N = \sigma A.$$

Caso destensado (material unilateral con $\varepsilon \leq 0$): el material devuelve $\sigma = 0$, $E_t = 0$, luego $N = 0$ y **ambas contribuciones se anulan** — $\mathbf{K}_T = \mathbf{0}$. El elemento aporta cero al sistema global.

2.9.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = N \mathbf{d}.$$

2.9.6 11. Cuadratura

No aplica (forma cerrada).

2.10 Contrato de implementación

```
name: Cable3DCorot
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
```

```

n_nodes: 2
strain_dim: 1
n_integration_points: 1

parameters:
- { name: A, type: float, required: true, desc: "Área de la sección transversal del cable" }

material_contract:
signature: "compute_state(, state) -> (, E_tangent, state')"
strain_kind: axial escalar (deformación ingenieril corotacional)
expected_behaviour: "material unilateral (p. ej. CableMaterial1D) para obtener respuesta física de cable; cualquier material con STRAIN_DIM=1 es técnicamente aceptado pero producirá comportamiento de barra"

conventions:
sign: " > 0 > 0 (elongación); la responsabilidad de anular en compresión es del material"
voigt: "N/A (escalar)"
node_orientation: "libre; K_T y F_int invariantes bajo permutación"
configuration: "Updated Lagrangian - l, c_x, c_y, c_z se recalculan en cada evaluación"

validity:
- "|| 1e-2 en régimen tensado"
- rotaciones y desplazamientos de cualquier magnitud en el espacio
- cargas axiales (las transversales se transmiten por destensado-retensado del cable)

out_of_scope:
- flexión, cortante, momentos, torsión
- dinámica con inercia distribuida (sin matriz de masa)
- pandeo (irrelevante: el cable no transmite compresión)
- "fuerzas de cuerpo distribuidas: el usuario reparte masa/peso a los nodos"
- pretensión mediante parámetro del elemento

numerical_caveats:
- "Cable destensado K_T = 0 en los DOFs del elemento. Si otros elementos no garantizan rigidez en esos DOFs, la matriz tangente global puede ser singular."
- "En transiciones tensado destensado (cruza 0), Newton-Raphson puede oscilar. Usar arc-length o pasos finos si el problema atraviesa destensiones."

acceptance:
- name: cable tensado coincide con barra corotacional 3D
  setup: "cable con orientación arbitraria, material lineal (Elastic1D), desplazamiento axial pequeño (~ 1e-6)"
  expect: "K_T y F_int coinciden con Truss3DCorot a tolerancia rtol = 1e-4"
  tol_rel: 1.0e-4

- name: cable destensado produce aportación nula
  setup: "cable sobre eje x con CableMaterial1D, u_x nodo 2 = -1e-3 < 0"
  expect: " = 0, F_int = 0, K_T = matriz cero"
  tol_abs: 1.0e-12

- name: invariancia bajo rotación rígida 3D
  setup: "cable inicial sobre eje x, rotación rígida de 45° en plano x-z"
  expect: " = 0 y F_int = 0 (independiente del material)"
  tol_abs: 1.0e-10

- name: cruce por cero

```

```

setup: "u_e = 0 con material unilateral"
expect: " = 0, E_t = 0, F_int = 0, K_T = 0"
tol_abs: 1.0e-12

references:
- "Crisfield M.A., Non-linear Finite Element Analysis of Solids and Structures, vol
  .1, §3.3"
- "Belytschko T., Nonlinear Finite Elements for Continua and Structures, §4.5"
- "Irvine H.M., Cable Structures, MIT Press, §1.2"

```

2.11 Implementación

- **Archivo:** solidum/elements/cable.py — comparte archivo con `Cable2DCorot` por convención temática del proyecto; no comparte código.
- **Clase:** `Cable3DCorot` — hereda directamente de `Element` (no de `Cable2DCorot`, no de `Truss3DCorot`, no de ningún otro elemento). Maquinaria cinemática 3D autónoma.
- **Tests:** tests/test_cable_elements.py · `TestCable3DCorot` — los 4 criterios de `acceptance` más un test de registro:
 - `test_acceptance_cable_tensado_como_barra_corot_3d` (criterio 1).
 - `test_acceptance_cable_destensado_aportacion_nula` (criterio 2).
 - `test_acceptance_rotacion_rigida_3d` (criterio 3).
 - `test_acceptance_cruce_por_cero` (criterio 4).
 - `test_registro_en_registry` — autodiscover.
- **Notas de traducción:**
 - La clase duplica la maquinaria corotacional 3D de `Truss3DCorot`. Duplicación deliberada por la misma razón que en 2D: desacoplar la evolución futura de armaduras y cables.
 - El proyector $\mathbf{P}_3 = \mathbf{I}_3 - \hat{\mathbf{e}}\hat{\mathbf{e}}^\top$ se construye con `solidum.math.geometry.perpendicular_projector( $\hat{\mathbf{e}}$ )`, helper compartido con `Truss3DCorot` (operación geométrica pura, ortogonal al hecho de que el material sea unilateral o no).
 - `_current_geometry` tolera nodos con 2 ó 3 coordenadas (completa con $z = 0$ si falta), heredando la flexibilidad de `Truss3D` para problemas embebidos.
 - En estado destensado, $\mathbf{K}_M = \mathbf{0}$ y $\mathbf{K}_G = \mathbf{0}$ simultáneamente; el elemento aporta literalmente ceros al ensamblaje.
 - El test del criterio 1 compara directamente contra `Truss3DCorot` con material `Elastic1D`: si ambos producen $\mathbf{K}$ y $\mathbf{F}_{\text{int}}$ iguales a tolerancia, la cinemática del cable y de la barra son consistentes entre sí en el régimen tensado.

2.12 Diálogo

- **2026-04-21** · Elemento creado como pieza totalmente autónoma por decisión del usuario: no hereda de `Cable2DCorot` ni de `Truss3DCorot`. Compartir archivo con `Cable2DCorot` (`solidum/elements/cable.py`) es convención temática del proyecto (elementos del mismo dominio conviven en un archivo, como las 4 armaduras y los 2 frames 2D en `structural.py`), no implica dependencia de código.
- **2026-04-21** · La única diferencia estructural con `Cable2DCorot` es el proyector 3D: en 2D la perpendicular es un vector único ($\mathbf{n}$); en 3D es un plano (dimensión 2). Esto se refleja en el rango de $\mathbf{K}_G$: 1 en 2D, 2 en 3D.

Capítulo 3

Elementos 1D — Marcos / Vigas

3.1 Frame2DEuler

*Orden de trabajo. El usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

3.2 Especificación física

3.2.1 0. Descripción general

Elemento 1D inmerso en el plano 2D que modela una **viga esbelta** según la teoría de **Euler-Bernoulli**. Dos nodos rígidamente conectados (transmiten momento), 3 grados de libertad por nodo (u_x, u_y, r_z) . Captura tres acciones internas: **fuerza axial**, **cortante transverso** y **momento flector**. Régimen de **linealidad geométrica** (pequeños desplazamientos y rotaciones).

3.2.2 1. Hipótesis cinemática de Euler-Bernoulli

- Las secciones transversales se mantienen **planas** tras la deformación.
- Las secciones permanecen **perpendiculares al eje neutro deformado** (cizalladura transversal nula: $\gamma_{xy} = 0$).

Consecuencia: la rotación de la sección es igual a la pendiente de la elástica:

$$\theta(s) = \frac{dv}{ds}.$$

Válido solo para vigas **esbeltas** ($L/h \gtrsim 10$). En vigas peraltadas la cizalladura no es despreciable
→ Frame2DTimoshenko.

3.2.3 2. Campos de desplazamiento

- Axial: $u(s)$ a lo largo del eje local $s \in [0, L]$.
- Transverso: $v(s)$ perpendicular al eje local.
- Rotación de la sección: $\theta(s) = dv/ds$ (derivada de la flecha).

3.2.4 3. Medidas de deformación

- Axial: $\varepsilon_{axial}(s) = du/ds$ (constante para interpolación lineal en u).
- Curvatura: $\kappa(s) = d^2v/ds^2$.

La deformación axial en un punto genérico de la sección es $\varepsilon(s, y) = \varepsilon_{axial}(s) - y \kappa(s)$; al integrar sobre la sección se obtienen axial (N) y momento flector (M) desacoplados.

3.2.5 4. Ecuaciones constitutivas

- Axial: $N = EA \varepsilon_{axial}$ (o $N = A \sigma$ con σ del material).
- Flexión: $M = EI \kappa$.

El material delega la respuesta axial; la rigidez a flexión escala con el **módulo tangente** E_t devuelto por el material (aproximación: toda la sección entra en régimen no-elástico simultáneamente — no hay plasticidad distribuida).

3.2.6 5. Equilibrio — forma débil

Principio de trabajos virtuales. El elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \int_0^L (N \delta \varepsilon_{axial} + M \delta \kappa) ds = \delta \mathbf{u}_e^\top \mathbf{F}_{\text{int}}.$$

3.3 Formulación numérica (FEM)

3.3.1 6. Discretización

Dos nodos ($s = 0$ y $s = L$), 3 DOFs por nodo en coordenadas globales:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, r_z^{(1)}, u_x^{(2)}, u_y^{(2)}, r_z^{(2)}]^\top.$$

3.3.2 7. Sistema local y transformación

Cosenos directores del eje (configuración inicial, fija): $c = (x_2 - x_1)/L$, $s_\theta = (y_2 - y_1)/L$. Matriz de transformación ortogonal 6×6 :

$$\mathbf{T} = \begin{bmatrix} c & s_\theta & 0 & 0 & 0 & 0 \\ -s_\theta & c & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & c & s_\theta & 0 \\ 0 & 0 & 0 & -s_\theta & c & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

Las rotaciones r_z son invariantes ante rotación del sistema (escalares en el plano). $\mathbf{u}_{\text{local}} = \mathbf{T} \mathbf{u}_e$.

3.3.3 8. Funciones de forma

- Axial: lineal, $N_1^u(s) = 1 - s/L$, $N_2^u(s) = s/L$.
- Transverso (Hermite cúbicos, cuatro funciones que interpolan v y θ):

$$\begin{aligned} N_1^v &= 1 - 3\xi^2 + 2\xi^3, & N_1^\theta &= L(\xi - 2\xi^2 + \xi^3), \\ N_2^v &= 3\xi^2 - 2\xi^3, & N_2^\theta &= L(-\xi^2 + \xi^3), \end{aligned}$$

con $\xi = s/L$.

3.3.4 9. Rigidez local

En el sistema local, la matriz 6×6 tiene forma cerrada por integración analítica:

$$\mathbf{K}_{\text{local}} = \begin{bmatrix} \frac{EA}{L} & 0 & 0 & -\frac{EA}{L} & 0 & 0 \\ 0 & \frac{12EI}{L^3} & \frac{6EI}{L^2} & 0 & -\frac{12EI}{L^3} & \frac{6EI}{L^2} \\ 0 & \frac{6EI}{L^2} & \frac{4EI}{L} & 0 & -\frac{6EI}{L^2} & \frac{2EI}{L} \\ -\frac{EA}{L} & 0 & 0 & \frac{EA}{L} & 0 & 0 \\ 0 & -\frac{12EI}{L^3} & -\frac{6EI}{L^2} & 0 & \frac{12EI}{L^3} & -\frac{6EI}{L^2} \\ 0 & \frac{6EI}{L^2} & \frac{2EI}{L} & 0 & -\frac{6EI}{L^2} & \frac{4EI}{L} \end{bmatrix}.$$

En no-linealidad material, $E \rightarrow E_t(\varepsilon_{axial})$ devuelto por el material — escala **todos** los términos de la matriz por igual. Esta es una aproximación: supone que la plasticidad/daño afecta uniformemente a la sección.

3.3.5 10. Fuerzas internas

En el sistema local:

$$\mathbf{F}_{\text{int,local}} = \mathbf{K}_{\text{local}} \mathbf{u}_{\text{local}}.$$

La componente axial se **corrige** con el esfuerzo axial verdadero del material: $F_1 = -\sigma A$, $F_4 = +\sigma A$. Esto permite que el material aporte σ no-lineal sin que la formulación de flexión interfiera.

3.3.6 11. Ensamblaje global

$$\mathbf{K}_{\text{global}} = \mathbf{T}^\top \mathbf{K}_{\text{local}} \mathbf{T}, \quad \mathbf{F}_{\text{int}} = \mathbf{T}^\top \mathbf{F}_{\text{int,local}}.$$

3.3.7 12. Cuadratura

No aplica (forma cerrada para la matriz; integración analítica de los polinomios de Hermite). El campo `n_integration_points`: 1 se refiere al único estado material conceptual del elemento (la deformación axial media).

3.4 Contrato de implementación

```

name: Frame2DEuler
kind: element
status: validated

interface:
  dof_names: [ux, uy, rz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: "Área de la sección transversal" }
  - { name: I, type: float, required: true, desc: "Momento de inercia respecto al eje perpendicular al plano (z)" }

material_contract:
  signature: "compute_state(, state) -> (, E_tangent, state')"
  strain_kind: "axial escalar (deformación media axial = (u_2 - u_1)/L en sistema local)"
  nonlinearity_model: "E_tangent escala toda la matriz local - no hay distinción axial vs flexión en régimen no-elástico"

conventions:
  sign: " > 0 > 0 (elongación) en el eje axial"
  rotation_sign: "r_z positivo antihorario (convención matemática estándar)"
  node_orientation: "el eje local va del nodo 1 al nodo 2; permutar nodos invierte el signo de las fuerzas internas"
  configuration: "inicial fija - L, c, s, T no se actualizan con los desplazamientos"

validity:
  - "vigas esbeltas: L/h > 10 (h = canto de la sección)"
  - "pequeños desplazamientos y pequeñas rotaciones"
  - "|_axial| < 1e-2"

out_of_scope:
  - vigas peraltadas con deformación por cortante significativa (usar Frame2DTimoshenko)
  - grandes rotaciones o desplazamientos (no hay variante corotacional en el catálogo actual)
  - plasticidad distribuida en la sección (fibras); el modelo escala rigidez global por E_t
  - pandeo (requiere rigidez geométrica, no implementada en esta viga lineal)
  - "fuerzas de cuerpo distribuidas: el usuario reparte carga equivalente a los nodos"

acceptance:
  - name: respuesta axial pura
    setup: "viga horizontal, empotrada en extremo izquierdo, carga axial F en extremo derecho"
    expect: "u_x (extremo libre) = F·L/(E·A); momentos y cortantes nulos"
    tol_rel: 1.0e-10

  - name: voladizo con carga transversal (flecha analítica)
    setup: "viga horizontal, empotrada en extremo izquierdo, carga vertical P en extremo derecho"
    expect: "v (extremo libre) = P·L³/(3·E·I); rotación = P·L²/(2·E·I)"
    tol_rel: 1.0e-10

```

```

- name: simetría de K
  setup: "cualquier configuración"
  expect: "K_global = K_global (elemento conservativo en régimen elástico)"
  tol_abs: 1.0e-12

references:
- "Bathe K.-J., Finite Element Procedures, §4.2 (formulación isoparamétrica) y cap. 5 (vigas)"
- "Cook R.D., Concepts and Applications of FEA, §2.7 (elementos de viga)"

```

3.5 Implementación

- **Archivo:** `solidum/elements/frame/euler.py` — submódulo del paquete `solidum/elements/frame/` que aloja también `Frame2DTimoshenko` y `Frame2DEulerCorot`.
- **Clase:** `Frame2DEuler` — hereda directamente de `Element`. La construcción de la matriz de transformación **T** se delega a `build_geometry_2d` en `solidum/elements/frame/_shared.py`, compartida con `Frame2DTimoshenko` (la versión corotacional reconstruye **T** desde `alpha0` por su lógica propia).
- **Tests:** `tests/test_frame.py · TestFrame2DEulerAcceptance` — los tres criterios físicos y el registro:
 - `test_acceptance_respuesta_axial_pura` (criterio 1) — resuelve el voladizo con `solver`, verifica $u_x(L) = FL/(EA)$.
 - `test_acceptance_voladizo_carga_transversal` (criterio 2) — carga transversal P , verifica flecha $v = PL^3/(3EI)$ y rotación $\theta = PL^2/(2EI)$.
 - `test_acceptance_simetria_K` (criterio 3) — viga oblicua ($c=0.6$, $s=0.8$) para evitar ejes canónicos, verifica $\mathbf{K} = \mathbf{K}^\top$.
- `test_registro_en_registry` — `autodiscover`.
- **Notas de traducción:**
 - La matriz **T** se calcula una sola vez en `__init__` sobre la configuración inicial y se guarda como atributo — coherente con el régimen geoméricamente lineal.
 - En `compute_element_state`, la componente axial de $\mathbf{F}_{\text{int,local}}$ se sobrescribe tras el producto $\mathbf{K}_{\text{local}}\mathbf{u}_{\text{local}}$ con el valor σA devuelto por el material. Esto permite usar materiales no-lineales en la rama axial sin que la parte lineal de flexión interfiera.
 - La aproximación del modelo no-lineal es que E_t escala **toda** la matriz local: no hay distinción entre rigidez axial y de flexión en régimen no-elástico. Documentado en `material_contract.nonlinearity_mode`.

3.6 Diálogo

- **2026-04-21** · Elemento movido a archivo propio `solidum/elements/frame.py` y desacoplado del helper compartido `_frame_geometry`. El método estático `_build_geometry` reimplementa la construcción de **T** dentro de la clase. `Frame2DTimoshenko` sigue en `structural.py` usando su

propio acceso al helper compartido; si mañana también se independiza, duplicará o internalizará su propia versión.

- **2026-04-21** · Tests de aceptación cubren la flecha analítica del voladizo a 8 decimales ($v = PL^3/(3EI)$) usando el solver completo del proyecto — validan no solo la cinemática del elemento sino también su integración con ensamblador y solver.

- **2026-05-13** · `frame.py` se parte en paquete `solidum/elements/frame/{euler,timoshenko,euler_corot,_,_}`

La duplicación de `_build_geometry` entre Euler y Timoshenko se elimina: la construcción de **T** vuelve a vivir en un único lugar (`build_geometry_2d` en `_shared.py`), pero esta vez sin función helper externa flotante — pertenece al paquete `frame/`. `Frame2DEulerCorot` mantiene su propia reconstrucción de **T** desde `alpha0` porque su cinemática corotacional no se beneficia del helper común.

3.7 Frame2DEulerCorot

*Orden de trabajo. El usuario escribe **especificación física, formulación numérica y contrato**; la IA rellena **implementación** y responde en **diálogo**.*

3.8 Especificación física

3.8.1 0. Descripción general

Viga 2D esbelta según Euler-Bernoulli en formulación **corotacional** (Updated Lagrangian). Dos nodos rígidamente conectados, 3 DOFs por nodo (u_x, u_y, r_z). Captura grandes desplazamientos y grandes rotaciones del elemento como un todo con **pequeñas deformaciones axiales** ($|\varepsilon_{axial}| \lesssim 1\%$) y rotaciones nodales deformacionales moderadas.

Abre dominios que la viga lineal no captura: **pandeo por flexión, post-pandeo, snap-through** de arcos planos, brazos flexibles esbeltos.

3.8.2 1. Cinemática corotacional

Se separa el movimiento del elemento en:

1. **Rotación rígida** α_e : el ángulo que el eje de la barra ha girado desde la configuración inicial.
2. **Rotación deformacional de cada sección** $\bar{\theta}_i$: la rotación nodal menos la rotación rígida (es lo que genera momento flector).

Sean:

$$\alpha_0 = \arctan 2(Y_2 - Y_1, X_2 - X_1) \quad (\text{ángulo del eje en config. inicial, fijo}),$$

$$\alpha = \arctan 2(y_2 - y_1, x_2 - x_1) \quad (\text{ángulo corriente}),$$

$$\alpha_e = \alpha - \alpha_0 \quad (\text{rotación rígida del elemento}).$$

Rotaciones deformacionales:

$$\bar{\theta}_1 = \theta_1 - \alpha_e, \quad \bar{\theta}_2 = \theta_2 - \alpha_e,$$

donde $\theta_i = r_z^{(i)}$ son las rotaciones totales de los nodos.

3.8.3 2. Medidas de deformación

- Axial: $\varepsilon_{axial} = (l - L_0)/L_0$ con l longitud corriente.
- Curvatura efectiva: $\kappa = (\bar{\theta}_2 - \bar{\theta}_1)/L_0$ (constante si se usa interpolación cúbica).

3.8.4 3. Ecuaciones constitutivas

- Axial: delegado al material (σ, E_t) .
- Flexión: $M = E_t I \kappa$ (el mismo E_t del material escala la rigidez de flexión, como en **Frame2DEuler**).

3.8.5 4. Equilibrio — forma débil

PTV en configuración corriente. El elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \delta \mathbf{u}_e^\top \mathbf{F}_{\text{int}}, \quad \mathbf{K}_T = \mathbf{K}_{\text{material}} + \mathbf{K}_\sigma.$$

3.9 Formulación numérica (FEM)

3.9.1 5. DOFs locales deformacionales

Se reduce la descripción del estado deformacional del elemento a **3 magnitudes escalares**:

$$\mathbf{q}_l = [u_l, \bar{\theta}_1, \bar{\theta}_2]^\top, \quad u_l = l - L_0.$$

3.9.2 6. Rigidez local

En el sistema de coordenadas corrotado (eje de la barra corriente), la rigidez local 3×3 :

$$\mathbf{K}_{ll} = \begin{bmatrix} \frac{E_t A}{L_0} & 0 & 0 \\ 0 & \frac{4E_t I}{L_0} & \frac{2E_t I}{L_0} \\ 0 & \frac{2E_t I}{L_0} & \frac{4E_t I}{L_0} \end{bmatrix}.$$

Fuerzas locales $\mathbf{F}_l = \mathbf{K}_{ll} \mathbf{q}_l = [N, M_1, M_2]^\top$, con N sustituido por σA directamente del material para permitir respuestas no-lineales.

3.9.3 7. Matriz de transformación $\mathbf{B}$ (3×6)

Relación entre variaciones de $\mathbf{q}_l$ y de los DOFs globales $\delta \mathbf{u}_e$:

$$\mathbf{B} = \begin{bmatrix} -c & -s & 0 & c & s & 0 \\ -s/l & c/l & 1 & s/l & -c/l & 0 \\ -s/l & c/l & 0 & s/l & -c/l & 1 \end{bmatrix}, \quad c = \cos \alpha, \quad s = \sin \alpha.$$

c, s y l se evalúan en **configuración corriente** — esta es la esencia corrotacional.

Fuerzas internas globales:

$$\mathbf{F}_{\text{int}} = \mathbf{B}^\top \mathbf{F}_l.$$

3.9.4 8. Rigidez tangente

Se descompone en contribución material más geométrica:

$$\begin{aligned} \mathbf{K}_T &= \mathbf{K}_{\text{material}} + \mathbf{K}_\sigma, \\ \mathbf{K}_{\text{material}} &= \mathbf{B}^\top \mathbf{K}_{ll} \mathbf{B}, \\ \mathbf{K}_\sigma &= \frac{N}{l} \mathbf{z} \mathbf{z}^\top + \frac{M_1 + M_2}{l^2} (\mathbf{r} \mathbf{z}^\top + \mathbf{z} \mathbf{r}^\top), \end{aligned}$$

con vectores geométricos de 6 componentes:

$$\begin{aligned} \mathbf{r} &= [-c, -s, 0, c, s, 0]^\top \quad (\text{dirección axial}), \\ \mathbf{z} &= [-s, c, 0, s, -c, 0]^\top \quad (\text{perpendicular}). \end{aligned}$$

La parte $(N/l)\mathbf{z}\mathbf{z}^\top$ es análoga al $\mathbf{K}_G$ de **Truss2DCorot**: captura la rigidización por tensión (y el ablandamiento precrítico por compresión, base del pandeo). La parte con momentos acopla rotaciones con traslaciones — crítica para problemas de flexión con desplazamientos significativos.

3.9.5 9. Cuadratura

No aplica (forma cerrada con Hermite cúbicos).

3.10 Contrato de implementación

```

name: Frame2DEulerCorot
kind: element
status: validated

interface:
  dof_names: [ux, uy, rz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: "Área de la sección transversal" }
  - { name: I, type: float, required: true, desc: "Momento de inercia respecto al eje perpendicular al plano" }

material_contract:
  signature: "compute_state(, state) -> (, E_tangent, state')"
  strain_kind: "axial escalar corotacional ( = (1 - L)/L)"
  nonlinearity_model: "E_tangent escala K_material local; el K_ geométrico depende solo de N, M, M corrientes"

conventions:
  sign: " > 0 > 0 (elongación); r_z positivo antihorario"
  node_orientation: "eje local del nodo 1 al nodo 2"
  configuration: "Updated Lagrangian - l, c, s, se recalculan en cada evaluación"

validity:
  - "|_axial| 1e-2 (pequeña deformación axial)"
  - rotaciones rígidas del elemento de cualquier magnitud, pero acumuladas en |_e| < entre commits (rotaciones continuas > requerirían tracking de vueltas - fuera de alcance)
  - rotaciones deformacionales de las secciones moderadas (~|_i| 30°); para valores mayores la interpolación Hermite cúbica pierde precisión
  - vigas esbeltas (L/h 10)

out_of_scope:
  - rotaciones continuas del eje > sin pasar por commit (pendulums; aliasing en atan2)
  - deformaciones axiales grandes (requiere medida logarítmica)
  - plasticidad distribuida en la sección (fibras)
  - cortante transverso significativo (usar versión Timoshenko, pendiente)
  - "fuerzas de cuerpo distribuidas: el usuario reparte carga a los nodos"

numerical_caveats:
  - "El ángulo _e se unwrappea a (-, ] por llamada. Si el problema tiene saltos de rotación > entre iteraciones consecutivas del solver, la formulación produce resultados incorrectos. Usar pasos de carga finos."
  - "En problemas con pandeo, la rigidez tangente se hace singular al cruzar el punto crítico. Usar `ArcLengthSolver` para seguir la rama post-crítica."

acceptance:

```

```

- name: límite lineal coincide con Frame2DEuler
  setup: "voladizo horizontal, cargas pequeñas ( ~ 1e-8, v/L ~ 1e-6)"
  expect: "K_T y F_int coinciden con Frame2DEuler lineal a tolerancia rtol=1e-4"
  tol_rel: 1.0e-4

- name: invariancia bajo rotación rígida
  setup: "viga inicialmente horizontal, rotada rígidamente 45° (ambos nodos)"
  expect: "F_int = 0, = 0 (ningún esfuerzo interno bajo rotación rígida pura)"
  tol_abs: 1.0e-10

- name: coherencia tangente/fuerza interna (chequeo por diferencias finitas)
  setup: "configuración arbitraria no trivial; K_T analítica vs derivada numérica de
        F_int"
  expect: "||K_T_analítica - K_T_numérica|| / ||K_T_analítica|| 1e-5"
  tol_rel: 1.0e-5

- name: simetría de K
  setup: "configuración arbitraria"
  expect: "K_T = K_T"
  tol_abs: 1.0e-6

- name: Bathe-Bolourchi - cuarto de círculo # añadido 2026-05-19
  setup: "voladizo recto, 10 elementos, momento puro M = (/2)·EI/L en la punta, 8
        pasos"
  expect: "tip se mueve a (R·sin (/2)-L, R·(1-cos (/2))) con R=EI/M; _tip = /2"
  tol_abs: 0.005 # sobre L (cuerda ~0.2% en 10 elementos rectos)

- name: Bathe-Bolourchi - medio círculo # añadido 2026-05-19
  setup: "voladizo recto, 20 elementos, momento puro M = ·EI/L en la punta, 20 pasos
        "
  expect: "tip a (-L, 2L/) y _tip = - la viga se enrosca en semicírculo cerrado"
  tol_abs: 0.01 # sobre L

references:
- "Crisfield M.A., Non-linear Finite Element Analysis of Solids and Structures, vol
  .1, §7.3 (corotational beams)"
- "Belytschko T., Nonlinear Finite Elements for Continua and Structures, §4.11"
- "Wriggers P., Nonlinear Finite Element Methods, cap. 4"

```

3.11 Implementación

- **Archivo:** `solidum/elements/frame/euler_corot.py` — convive con `Frame2DEuler` y `Frame2DTimoshenko` en el paquete `solidum/elements/frame/`, sin heredar de ninguno. Comparte con ellos los helpers libres de `_shared.py` (carga de cuerpo, masa consistente, traducción a `ElementForces`), pero no `build_geometry_2d`: reconstruye `T` desde `alpha0` por su lógica corotacional propia.
- **Clase:** `Frame2DEulerCorot` — hereda directamente de `Element`. Métodos internos `_current_geometry` (longitud, cosenos, ángulo corriente y rigid-body α_e) y `_unwrap` (reducción a $(-\pi, \pi]$).
- **Tests:** `tests/test_frame.py · TestFrame2DEulerCorotAcceptance` — 4 criterios + registro:
- `test_acceptance_limite_lineal_coincide_con_frame2deuler`
- `test_acceptance_invariancia_rotacion_rigida`

- `test_acceptance_coherencia_tangente_fuerza_interna` — **finite-difference check** de $\mathbf{K}_T$ contra derivada numérica de $\mathbf{F}_{\text{int}}$, red de seguridad clave en esta formulación.
- `test_acceptance_simetria_K`
- `test_registro_en_registry`
- **Notas de traducción:**
- La formulación se basa en 3 DOFs deformacionales $[u, \bar{\theta}_1, \bar{\theta}_2]$ y una matriz $\mathbf{B}$ 3×6 que los relaciona con los DOFs globales. Evaluada en configuración corriente.
- **Signo de la contribución de momentos en $\mathbf{K}_\sigma$:** tras derivar $\partial(\mathbf{z}/l)/\partial \mathbf{u}_e$ analíticamente se obtiene $-(1/l^2)(\mathbf{r}\mathbf{z}^\top + \mathbf{z}\mathbf{r}^\top)$. El signo es **negativo** — un error común es ponerlo positivo. El test de diferencias finitas lo cazó en la primera implementación.
- `compute_internal_forces` proyecta $\mathbf{F}_{\text{int}}$ al sistema local corriente para reportar axial/cortante separados, utilizando la submatriz 2×2 de rotación del eje corriente.
- `_unwrap` lleva los ángulos al intervalo $(-\pi, \pi]$; la formulación es válida mientras la rotación acumulada por paso sea menor que π .

3.12 Diálogo

- **2026-04-21** · Implementación siguiendo Crisfield §7.3. Convive en `frame.py` con las vigas lineales por familia temática (todas son vigas 2D), sin herencia ni helpers compartidos.
- **2026-05-13** · `frame.py` se parte en paquete `solidum/elements/frame/`. EulerCorot vive ahora en `euler_corot.py`, sin herencia con las otras vigas 2D. Pasan a compartirse los helpers libres de carga de cuerpo, masa y traducción de fuerzas a través de `_shared.py`; la lógica corotacional propia (reconstrucción de $\mathbf{T}$ desde `alpha0`, cinemática actual) se mantiene íntegra dentro de la clase.
- **2026-04-21** · La primera versión tenía el signo equivocado en la parte de momentos de $\mathbf{K}_\sigma$ (+ en lugar de -). El test de coherencia tangente/fuerza interna por diferencias finitas lo detectó inmediatamente: $\|\mathbf{K}_{T,\text{analítica}} - \mathbf{K}_{T,\text{FD}}\|_{\text{rel}}$ del orden de $1e-1$ en lugar de $1e-5$. Tras derivar $\partial(\mathbf{z}/l)/\partial \mathbf{u}_e$ cuidadosamente (ver notas) el signo correcto es negativo. Este episodio justifica retrospectivamente incluir un test FD como criterio de aceptación en toda formulación no-lineal geométrica futura.
- **2026-04-21** · El test del límite lineal usa $P = 10$ N para obtener $v/L \sim 10^{-4}$: suficientemente pequeño para estar en régimen lineal pero suficientemente grande para que el residuo del solver converja por debajo de `tol=1e-8` (cargas menores producen desplazamientos del orden del ruido numérico y el Newton oscila en la tolerancia relativa).

3.13 Frame2DTimoshenko

*Orden de trabajo. El usuario escribe **especificación física, formulación numérica y contrato**; la IA rellena **implementación** y responde en **diálogo**.*

3.14 Especificación física

3.14.1 0. Descripción general

Elemento 1D inmerso en el plano 2D que modela una **viga** según la teoría de **Timoshenko**. Dos nodos rígidamente conectados, 3 DOFs por nodo (u_x, u_y, r_z). Transmite **fuerza axial, cortante transversal y momento flector**. Incluye explícitamente la **deformación por cortante transversal**. Régimen de **linealidad geométrica** (pequeños desplazamientos y rotaciones).

3.14.2 1. Hipótesis cinemática de Timoshenko

- Las secciones transversales se mantienen **planas** tras la deformación (igual que Euler-Bernoulli).
- Las secciones **no** están obligadas a ser perpendiculares al eje neutro: se permite una **rotación adicional** respecto a la tangente.

Consecuencia: la rotación de la sección $\theta(s)$ y la pendiente de la elástica dv/ds son **variables independientes**:

$$\gamma(s) = \frac{dv}{ds} - \theta(s) \neq 0,$$

donde γ es la distorsión angular (deformación por cortante transversal). Euler-Bernoulli corresponde al caso $\gamma = 0$.

Apropiado para vigas **gruesas, cortas o peraltadas** ($L/h \lesssim 10$) donde la deformación por cortante no es despreciable.

3.14.3 2. Campos de desplazamiento

- Axial: $u(s)$ a lo largo del eje local $s \in [0, L]$.
- Transverso: $v(s)$ perpendicular al eje local.
- Rotación de la sección: $\theta(s)$, **independiente** de v .

3.14.4 3. Medidas de deformación

- Axial: $\varepsilon_{axial}(s) = du/ds$.
- Curvatura (flexión): $\kappa(s) = d\theta/ds$.
- Cortante: $\gamma(s) = dv/ds - \theta(s)$.

3.14.5 4. Ecuaciones constitutivas

- Axial: $N = EA \varepsilon_{axial}$ (el material entrega σ y E_t).
- Flexión: $M = EI \kappa$.
- Cortante: $V = GA_s \gamma$, con $G = E/[2(1 + \nu)]$ (módulo de corte) y A_s el **área efectiva de cortante** (menor que A por la distribución no uniforme de la tensión tangencial en la sección).

3.14.6 5. Factor de Phi (shear locking)

La formulación estándar con interpolaciones lineales de v y θ sufre **shear locking** para vigas esbeltas (la rigidez por cortante domina numéricamente aunque γ físicamente sea pequeño). La matriz local incorpora un factor correctivo:

$$\Phi = \frac{12 EI}{G A_s L^2}.$$

Para $\Phi \rightarrow 0$ (viga esbelta) la matriz se reduce a la de Euler-Bernoulli.

3.14.7 6. Equilibrio — forma débil

Principio de trabajos virtuales (PTV). El elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \int_0^L (N \delta \varepsilon_{axial} + M \delta \kappa + V \delta \gamma) ds = \delta \mathbf{u}_e^\top \mathbf{F}_{\text{int}}.$$

3.15 Formulación numérica (FEM)

3.15.1 7. Discretización

Dos nodos ($s = 0$ y $s = L$), 3 DOFs por nodo en coordenadas globales:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, r_z^{(1)}, u_x^{(2)}, u_y^{(2)}, r_z^{(2)}]^\top.$$

3.15.2 8. Sistema local y transformación

Cosenos directores del eje (configuración inicial, fija): $c = (x_2 - x_1)/L$, $s_\theta = (y_2 - y_1)/L$. Matriz de transformación ortogonal 6×6 :

$$\mathbf{T} = \begin{bmatrix} c & s_\theta & 0 & 0 & 0 & 0 \\ -s_\theta & c & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & c & s_\theta & 0 \\ 0 & 0 & 0 & -s_\theta & c & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}, \quad \mathbf{u}_{\text{local}} = \mathbf{T} \mathbf{u}_e.$$

3.15.3 9. Rigidez local con corrección por shear locking

Coefficientes:

$$a = \frac{12 EI}{L^3(1 + \Phi)}, \quad b = \frac{6 EI}{L^2(1 + \Phi)}, \quad c_m = \frac{(4 + \Phi) EI}{L(1 + \Phi)}, \quad d = \frac{(2 - \Phi) EI}{L(1 + \Phi)}.$$

$$\mathbf{K}_{\text{local}} = \begin{bmatrix} EA/L & 0 & 0 & -EA/L & 0 & 0 \\ 0 & a & b & 0 & -a & b \\ 0 & b & c_m & 0 & -b & d \\ -EA/L & 0 & 0 & EA/L & 0 & 0 \\ 0 & -a & -b & 0 & a & -b \\ 0 & b & d & 0 & -b & c_m \end{bmatrix}.$$

Para $\Phi \rightarrow 0$: $a \rightarrow 12EI/L^3$, $b \rightarrow 6EI/L^2$, $c_m \rightarrow 4EI/L$, $d \rightarrow 2EI/L$ — se recupera Euler-Bernoulli.

3.15.4 10. Fuerzas internas

$$\mathbf{F}_{\text{int,local}} = \mathbf{K}_{\text{local}} \mathbf{u}_{\text{local}}.$$

La componente axial se corrige con el esfuerzo axial del material: $F_1 = -\sigma A$, $F_4 = +\sigma A$ (permite materiales no-lineales en la rama axial).

3.15.5 11. Ensamblaje global

$$\mathbf{K}_{\text{global}} = \mathbf{T}^\top \mathbf{K}_{\text{local}} \mathbf{T}, \quad \mathbf{F}_{\text{int}} = \mathbf{T}^\top \mathbf{F}_{\text{int,local}}.$$

3.15.6 12. Cuadratura

No aplica (forma cerrada).

3.16 Contrato de implementación

```
name: Frame2DTimoshenko
kind: element
status: validated

interface:
  dof_names: [ux, uy, rz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: "Área de la sección transversal" }
  - { name: I, type: float, required: true, desc: "Momento de inercia respecto al eje perpendicular al plano" }
  - { name: As, type: float, required: true, desc: "Área efectiva de cortante" }
  - { name: nu, type: float, required: false, default: 0.3, desc: "Coeficiente de Poisson (si el material no lo expone)" }
```

```

material_contract:
  signature: "compute_state(, state) -> (, E_tangent, state')"
  strain_kind: "axial escalar (deformación media = (u_2 - u_1)/L en sistema local)"
  nonlinearity_model: "E_tangent escala toda la matriz local (axial, flexión y cortante por igual)"
  poisson_source: "si material.nu existe, se usa; en su defecto el parámetro `nu` del elemento con default 0.3 y advertencia por consola"

conventions:
  sign: " > 0 > 0 (elongación) en el eje axial"
  rotation_sign: "r_z positivo antihorario"
  node_orientation: "eje local del nodo 1 al nodo 2"
  configuration: "inicial fija - L, c, s, T,  $\phi$  (en forma funcional) se calculan sin actualizar"

validity:
  - "vigas peraltadas o cortas (L/h > 10) donde el cortante no es despreciable"
  - "pequeños desplazamientos y pequeñas rotaciones"
  - " $|_{axial}| < 1e-2$ "

out_of_scope:
  - "grandes rotaciones o desplazamientos (no hay variante corotacional en el catálogo actual)"
  - "plasticidad distribuida en la sección"
  - "pandeo"
  - "fuerzas de cuerpo distribuidas: el usuario reparte carga equivalente a los nodos"

acceptance:
  - name: convergencia a Euler-Bernoulli para viga esbelta
    setup: "voladizo con carga transversal P, viga esbelta (L/h > 100)  $\phi \rightarrow 0$ "
    expect: " $v(L) = PL^3/(3EI)$  (solución Euler-Bernoulli) con tolerancia relativa  $1e-3$ "
    tol_rel: 1.0e-3

  - name: respuesta axial pura desacoplada
    setup: "voladizo con carga axial F"
    expect: " $u_x(L) = F \cdot L / (E \cdot A)$ ; momentos y cortantes nulos"
    tol_rel: 1.0e-10

  - name: simetría de K
    setup: "viga oblicua"
    expect: " $K_{global} = K_{global}$ "
    tol_abs: 1.0e-12

references:
  - "Reddy J.N., An Introduction to the Finite Element Method, cap. 5 (vigas de Timoshenko)"
  - "Bathe K.-J., Finite Element Procedures, §5.4 (formulación con factor  $\phi$ )"

```

3.17 Implementación

- **Archivo:** `solidum/elements/frame/timoshenko.py` — submódulo del paquete `solidum/elements/frame/` que aloja también `Frame2DEuler` y `Frame2DEulerCorot`.
- **Clase:** `Frame2DTimoshenko` — hereda directamente de `Element`, sin herencia con las otras vigas 2D. Comparte `build_geometry_2d` con `Frame2DEuler` en `solidum/elements/frame/_shared.py`;

además, los helpers libres de carga de cuerpo, masa consistente y traducción a `ElementForces` también viven en `_shared.py`.

- **Tests:** `tests/test_frame.py` · `TestFrame2DTimoshenkoAcceptance`:
- `test_acceptance_convergencia_euler_en_viga_esbelta` (criterio 1) — viga con L/h muy grande, verifica que la flecha tiende a $PL^3/(3EI)$ con tolerancia relativa 10^{-3} .
- `test_acceptance_respuesta_axial_pura` (criterio 2) — carga axial pura, verifica $u_x = FL/(EA)$.
- `test_acceptance_simetria_K` (criterio 3) — viga oblicua, verifica $\mathbf{K} = \mathbf{K}^\top$.
- `test_registro_en_registry` — autodiscover.
- **Notas de traducción:**
- Para no colisionar con la variable cinemática c (coseno director) dentro de `compute_element_state`, el coeficiente c_m de la matriz local se llama `c_coef` en el código y `d_coef` su análogo (antes eran `c` y `d` inline, ambiguos).
- El Poisson ν se toma de `material.nu` si el material lo expone; en su defecto, del parámetro `nu` del elemento con default 0.3 y una advertencia por consola la primera vez que se construye con ese default — atajo a errores silenciosos de usuarios que olvidan especificarlo.
- La matriz $\mathbf{T}$ se calcula una sola vez en `__init__` y se guarda como atributo (régimen geométricamente lineal).
- El factor Φ se recalcula en cada llamada porque depende de E_t devuelto por el material (en régimen no-lineal Φ cambia con el estado).

3.18 Diálogo

- **2026-04-21** · Elemento movido a archivo propio `solidum/elements/frame.py` y desacoplado del helper `_frame_geometry`. Con este movimiento, el helper compartido se elimina completamente del repositorio: `Frame2DEuler` y `Frame2DTimoshenko` replican la construcción de $\mathbf{T}$ como método estático `_build_geometry`, cada una en su clase. La duplicación (~15 líneas) es el precio aceptado de la independencia mutua.
- **2026-04-21** · Test del criterio 1: se eligió L/h grande en lugar de reproducir un caso específico de libro porque la convergencia Timoshenko $\rightarrow$ Euler es el comportamiento **esperado por construcción** (factor $\Phi \rightarrow 0$). Verificarlo directamente con el solver completo valida simultáneamente la cinemática de Timoshenko, el tratamiento del shear locking y la integración con ensamblador + solver.
- **2026-05-13** · `frame.py` se parte en paquete `solidum/elements/frame/`. La duplicación de `_build_geometry` ya no se justifica: ambas vigas comparten `build_geometry_2d` en `_shared.py`, helper interno al paquete (no flotante). Las clases siguen sin herencia entre sí.

3.19 Frame3D

*Orden de trabajo. El usuario escribe **especificación física, formulación numérica y contrato**; la IA rellena **implementación** y responde en **diálogo**.*

3.20 Especificación física

3.20.1 0. Descripción general

Elemento 1D inmerso en el espacio tridimensional que modela una **viga esbelta 3D**. Dos nodos rígidamente conectados, 6 DOFs por nodo ($u_x, u_y, u_z, r_x, r_y, r_z$). Captura seis acciones internas:

- **Axial** (fuerza normal N a lo largo del eje local x).
- **Cortante en dos planos** (V_y en el plano xy , V_z en el plano xz).
- **Flexión en dos planos** (M_z alrededor del eje local z , M_y alrededor del eje local y).
- **Torsión** (M_x alrededor del eje longitudinal).

Régimen de **linealidad geométrica** (pequeños desplazamientos y rotaciones). Hipótesis de Euler-Bernoulli: secciones planas perpendiculares al eje deformado (sin cizalladura transversal).

3.20.2 1. Hipótesis

- Secciones planas y perpendiculares al eje neutro deformado (igual que Frame2DEuler).
- Flexiones en los dos planos **desacopladas** entre sí (ejes principales de inercia).
- Torsión de Saint-Venant pura (sin alabeo). Válida para secciones macizas o cerradas; abiertas de pared delgada requerirían formulación con alabeo (fuera de alcance).
- Material isótropo 1D; $G = E/[2(1 + \nu)]$.

3.20.3 2. Sistema de ejes locales

Tres ejes ortonormales en cada elemento:

- **$\hat{\mathbf{x}}$ local**: dirección del eje de la barra ($\hat{\mathbf{x}} = (\mathbf{x}_2 - \mathbf{x}_1)/L$).
- **$\hat{\mathbf{y}}$ local y $\hat{\mathbf{z}}$ local**: ejes principales de inercia de la sección, ortogonales al eje.

La orientación de $\hat{\mathbf{y}}, \hat{\mathbf{z}}$ no está determinada solo por la dirección del eje — hace falta un **vector de referencia** proporcionado por el usuario (o un default). Es la diferencia estructural respecto al caso 2D, donde el plano era único.

3.20.4 3. Medidas de deformación / acciones internas

- Axial: $\varepsilon_{axial} = du/ds$, tensión $N = EA \varepsilon_{axial}$ (o $N = A \sigma$ con σ del material).
- Flexión en plano xy : curvatura $\kappa_z = d^2v/ds^2$, momento $M_z = EI_z \kappa_z$.
- Flexión en plano xz : curvatura $\kappa_y = d^2w/ds^2$, momento $M_y = EI_y \kappa_y$.
- Torsión: $d\theta_x/ds$, momento torsor $M_x = GJ d\theta_x/ds$.

3.20.5 4. Equilibrio — forma débil

PTV. El elemento calcula solo el lado interno:

$$\delta W_{\text{int}} = \int_0^L (N \delta \varepsilon + M_z \delta \kappa_z + M_y \delta \kappa_y + M_x \delta \theta'_x) ds.$$

3.21 Formulación numérica (FEM)

3.21.1 5. Discretización

Dos nodos, 6 DOFs por nodo:

$$\mathbf{u}_e = [u_x^{(1)}, u_y^{(1)}, u_z^{(1)}, r_x^{(1)}, r_y^{(1)}, r_z^{(1)}, u_x^{(2)}, u_y^{(2)}, u_z^{(2)}, r_x^{(2)}, r_y^{(2)}, r_z^{(2)}]^\top.$$

Vector de 12 componentes $\rightarrow$ matrices 12×12 .

3.21.2 6. Construcción de ejes locales

Dados $\mathbf{X}_1, \mathbf{X}_2$ y un vector de referencia $\mathbf{v}_{\text{ref}}$ proporcionado:

1. $\hat{\mathbf{x}} = (\mathbf{X}_2 - \mathbf{X}_1)/L$.
2. Proyectar $\mathbf{v}_{\text{ref}}$ al plano perpendicular a $\hat{\mathbf{x}}$:

$$\tilde{\mathbf{y}} = \mathbf{v}_{\text{ref}} - (\mathbf{v}_{\text{ref}} \cdot \hat{\mathbf{x}}) \hat{\mathbf{x}}.$$

1. $\hat{\mathbf{y}} = \tilde{\mathbf{y}}/\|\tilde{\mathbf{y}}\|$ (falla si $\|\tilde{\mathbf{y}}\| \approx 0$: $\mathbf{v}_{\text{ref}}$ paralelo al eje).
2. $\hat{\mathbf{z}} = \hat{\mathbf{x}} \times \hat{\mathbf{y}}$.

Default de $\mathbf{v}_{\text{ref}}$ (cuando el usuario no lo proporciona): $[0, 0, 1]$ (eje z global como "vertical"). Si el eje del elemento es cercanamente paralelo a z global (viga vertical), se usa $[1, 0, 0]$ como fallback y se emite una advertencia.

3.21.3 7. Matriz de transformación global $\rightarrow$ local

$$\boldsymbol{\lambda} = \begin{bmatrix} \hat{\mathbf{x}}^\top \\ \hat{\mathbf{y}}^\top \\ \hat{\mathbf{z}}^\top \end{bmatrix} \quad (3 \times 3, \text{ ortogonal}).$$

$$\mathbf{T} = \text{bloques diag}(\boldsymbol{\lambda}, \boldsymbol{\lambda}, \boldsymbol{\lambda}, \boldsymbol{\lambda}) \quad (12 \times 12).$$

Aplicada a los 4 bloques de 3 DOFs: desplazamientos nodo 1, rotaciones nodo 1, desplazamientos nodo 2, rotaciones nodo 2. Las rotaciones infinitesimales en 3D transforman como vectores.

$$\mathbf{u}_{\text{local}} = \mathbf{T} \mathbf{u}_e.$$

3.21.4 8. Matriz de rigidez local

Desacoplada en cuatro contribuciones: axial, torsión, flexión xy (plano con u_y y r_z , inercia I_z), flexión xz (plano con u_z y r_y , inercia I_y).

Definiciones:

$$a_z = \frac{12EI_z}{L^3}, \quad b_z = \frac{6EI_z}{L^2}, \quad c_z = \frac{4EI_z}{L}, \quad d_z = \frac{2EI_z}{L},$$

$$a_y = \frac{12EI_y}{L^3}, \quad b_y = \frac{6EI_y}{L^2}, \quad c_y = \frac{4EI_y}{L}, \quad d_y = \frac{2EI_y}{L}.$$

La rigidez $\mathbf{K}_{\text{local}}$ es simétrica, con los bloques desacoplados (0s fuera de su plano correspondiente). Los signos del bloque de flexión xz se invierten respecto al xy porque u_z y r_y forman una pareja dextrógira opuesta (consecuencia del producto vectorial $\hat{\mathbf{x}} \times \hat{\mathbf{y}} = \hat{\mathbf{z}}$).

Forma concreta en términos de los 12 DOFs locales (la matriz completa vive en la implementación).

3.21.5 9. Fuerzas internas

$$\mathbf{F}_{\text{int,local}} = \mathbf{K}_{\text{local}} \mathbf{u}_{\text{local}},$$

con la componente axial corregida por el σ que devuelve el material: $F_1 = -\sigma A$, $F_7 = +\sigma A$. El resto de componentes viene del producto lineal (permite materiales no-lineales en la rama axial).

3.21.6 10. Ensamblaje global

$$\mathbf{K}_{\text{global}} = \mathbf{T}^\top \mathbf{K}_{\text{local}} \mathbf{T}, \quad \mathbf{F}_{\text{int}} = \mathbf{T}^\top \mathbf{F}_{\text{int,local}}.$$

3.21.7 11. Cuadratura

No aplica (forma cerrada; integración analítica de los polinomios de Hermite).

3.22 Contrato de implementación

```
name: Frame3D
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz, rx, ry, rz]
  n_nodes: 2
  strain_dim: 1
  n_integration_points: 1

parameters:
  - { name: A, type: float, required: true, desc: "Área de la sección transversal" }
  - { name: Iy, type: float, required: true, desc: "Momento de inercia alrededor del eje local y" }
  - { name: Iz, type: float, required: true, desc: "Momento de inercia alrededor del eje local z" }
  - { name: J, type: float, required: true, desc: "Constante torsional de Saint-Venant" }
  - { name: nu, type: float, required: false, default: 0.3, desc: "Coeficiente de Poisson (si el material no lo expone)" }
  - { name: ref_vector, type: list, required: false, default: "[0, 0, 1]",
```

```

    desc: "Vector 3D de referencia que fija la orientación de los ejes locales y, z
          de la sección. Default: eje z global." }

material_contract:
  signature: "compute_state(, state) -> (, E_tangent, state')"
  strain_kind: "axial escalar ( = (u_2 - u_1)/L en sistema local)"
  nonlinearity_model: "E_tangent escala toda la matriz local; G se recalcula como E_t
                      /[2(1+)]"
  poisson_source: "material.nu si existe; en su defecto parámetro `nu` del elemento"

conventions:
  sign: " > 0    > 0 (elongación); momentos siguen regla de la mano derecha respecto a
         los ejes locales"
  local_axes: "x_local = eje de la barra; y_local, z_local = ejes principales de
               inercia definidos por ref_vector"
  node_orientation: "eje local del nodo 1 al nodo 2"
  configuration: "inicial fija - L, T, ejes locales no se actualizan"

validity:
  - "vigas esbeltas (L/h  10 en ambos planos de flexión)"
  - "pequeños desplazamientos y pequeñas rotaciones en el espacio"
  - "|_axial|  1e-2"
  - "sección con ejes principales de inercia bien definidos por ref_vector"

out_of_scope:
  - grandes rotaciones espaciales (requeriría Frame3DCorot; pendiente)
  - torsión con alabeo restringido (secciones abiertas de pared delgada)
  - plasticidad distribuida en la sección (fibras)
  - pandeo por bifurcación
  - deformación por cortante transversal significativa (Timoshenko 3D, fuera de alcance
    )
  - "fuerzas de cuerpo distribuidas: el usuario reparte carga a los nodos"

numerical_caveats:
  - "Si ref_vector es paralelo al eje de la barra, la proyección al plano perpendicular
    se anula y el constructor aborta con mensaje claro - el usuario debe elegir otro
    vector."
  - "Para barras verticales (eje casi paralelo a z global) con ref_vector default, el
    constructor usa [1, 0, 0] como fallback y emite advertencia por consola."

acceptance:
  - name: respuesta axial pura
    setup: "voladizo alineado con eje x, carga F axial en extremo libre"
    expect: "u_x = F·L/(E·A); demás componentes nulas"
    tol_rel: 1.0e-10

  - name: flexión en plano xy (coherencia con Frame2D)
    setup: "voladizo alineado con eje x, carga P en dirección y, ref_vector = [0,0,1]"
    expect: "v_y = P·L³/(3·E·Iz), _z = P·L²/(2·E·Iz)"
    tol_rel: 1.0e-10

  - name: flexión en plano xz
    setup: "voladizo alineado con eje x, carga P en dirección z, ref_vector = [0,1,0]"
    expect: "v_z = P·L³/(3·E·Iy), _y = -P·L²/(2·E·Iy) (signo por mano derecha)"
    tol_rel: 1.0e-10

  - name: torsión pura
    setup: "voladizo con momento T aplicado alrededor del eje de la barra en el extremo
           libre"

```

```

expect: "_x = T·L/(G·J); demás componentes nulas"
tol_rel: 1.0e-10

- name: simetría de K
  setup: "viga en orientación arbitraria con ref_vector genérico"
  expect: "K_global = K_global"
  tol_abs: 1.0e-8

references:
- "Przemieniecki J.S., Theory of Matrix Structural Analysis, Tabla 11.3"
- "Cook R.D., Concepts and Applications of FEA, §2.8"
- "Bathe K.-J., Finite Element Procedures, cap. 5"

```

3.23 Implementación

- **Archivo:** solidum/elements/frame3d.py — archivo propio, sin dependencia de ningún otro elemento.
- **Clase:** Frame3D — hereda directamente de Element. Métodos privados `_build_local_frame` (longitud, matriz λ 3×3 y $\mathbf{T}$ 12×12) y `_build_local_stiffness` (matriz $\mathbf{K}_{\text{local}}$ 12×12 con bloques axial, torsión y flexión en ambos planos).
- **Tests:** tests/test_frame3d.py · TestFrame3DAcceptance — cubre los 5 criterios + validación de `ref_vector` paralelo al eje + registro:
 - test_acceptance_respuesta_axial_pura
 - test_acceptance_flexion_xy_coincide_con_frame2d
 - test_acceptance_flexion_xz
 - test_acceptance_torsion_pura
 - test_acceptance_simetria_K
 - test_rechazo_ref_vector_paralelo
 - test_registro_en_registry
- **Notas de traducción:**
 - La matriz λ 3×3 se construye como $[\hat{\mathbf{x}}^\top; \hat{\mathbf{y}}^\top; \hat{\mathbf{z}}^\top]$ — las filas son los ejes locales expresados en coordenadas globales. $\mathbf{T}$ 12×12 es un `blkdiag` de cuatro copias de λ (traslaciones y rotaciones transforman como vectores en 3D infinitesimal).
 - El default de `ref_vector` es $[0, 0, 1]$; el fallback $[1, 0, 0]$ se activa cuando $|\hat{\mathbf{x}} \cdot \hat{\mathbf{z}}_{\text{global}}| > 0,99$ (barra cercanamente vertical). Ambas rutas emiten advertencia por consola la primera vez que se usan con default.
 - La matriz $\mathbf{K}_{\text{local}}$ se construye rellenando entradas no nulas en sus 4 bloques desacoplados (axial, torsión, flexión xy , flexión xz). Los signos de la flexión xz están invertidos respecto a la flexión xy porque la pareja (u_z, r_y) tiene orientación dextrógira opuesta a (u_y, r_z) : este punto se verifica explícitamente en `test_acceptance_flexion_xz`.
 - `compute_internal_forces` devuelve cortantes, torsión y momentos en ambos extremos separados — útil para post-proceso estructural (diagramas de fuerzas internas).

3.24 Diálogo

- **2026-04-21** · Elemento creado como componente totalmente autónomo: no hereda de `Frame2DEuler` ni de ninguna otra viga. La maquinaria cinemática 3D se implementa íntegra dentro de `frame3d.py`, coherente con la filosofía aplicada a cables.
- **2026-04-21** · Decisión sobre `ref_vector` vs "tercer nodo de orientación". Se optó por `ref_vector` (vector 3D proyectado al plano perpendicular al eje) por ser más compacto en YAML y no requerir nodos fantasma en la malla. El default `[0, 0, 1]` cubre la mayoría de casos de ingeniería estructural (estructuras terrestres con gravedad en $-z$); el fallback para barras verticales usa `[1, 0, 0]` con advertencia explícita.
- **2026-04-21** · Verificación cruzada con `Frame2DEuler`: el criterio 2 aplica una viga alineada con el eje x global bajo carga en y ; los desplazamientos resultantes deben coincidir con la solución 2D clásica $v = PL^3/(3EI_z)$, $\theta = PL^2/(2EI_z)$. El test lo verifica a 8 decimales. Esto blindo la coherencia dimensional: el 3D se reduce al 2D sin error acumulado cuando el problema es efectivamente plano.

Capítulo 4

Elementos 2D — Sólidos

4.1 Tri3

Documentación retroactiva del elemento ya implementado.

4.2 Especificación física

4.2.1 0. Descripción general

Elemento sólido bidimensional plano de primer orden, triangular de tres nodos. Aproximación lineal C^0 del campo de desplazamientos en coordenadas naturales (ξ, η) . Conocido en la literatura como elemento *constant strain triangle* (CST): la matriz $\mathbf{B}$ es constante sobre el elemento, por lo que la deformación $\boldsymbol{\varepsilon}$ y la tensión $\boldsymbol{\sigma}$ son uniformes dentro de cada Tri3. Régimen de pequeños desplazamientos y deformaciones.

4.2.2 1. Cinemática de desplazamientos

$$u_x(\xi, \eta) = \sum_{i=1}^3 N_i(\xi, \eta) u_x^{(i)}, \quad u_y(\xi, \eta) = \sum_{i=1}^3 N_i(\xi, \eta) u_y^{(i)}.$$

4.2.3 2. Cinemática de deformaciones

Notación Voigt 2D $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$, idéntica a la de Quad4. Por la linealidad de N_i , las derivadas $\partial N_i / \partial x$ son constantes y $\boldsymbol{\varepsilon}$ es uniforme en el elemento.

4.2.4 3. Ecuación constitutiva

Delegada al material 2D (STRAIN_DIM = 3). Soporta Elastic2D, IsotropicDamage2D, VonMises2D (este último solo plane_strain).

4.2.5 4. Equilibrio — forma fuerte

$$-\nabla \cdot \boldsymbol{\sigma} = \mathbf{b} \quad \text{en } \Omega, \quad \boldsymbol{\sigma} \cdot \mathbf{n} = \bar{\mathbf{t}} \quad \text{en } \partial\Omega_N, \quad \mathbf{u} = \bar{\mathbf{u}} \quad \text{en } \partial\Omega_D.$$

4.2.6 5. Equilibrio — forma débil

Idéntica a la del Quad4; el principio variacional no depende de la forma del elemento.

4.3 Formulación numérica (FEM)

4.3.1 6. Discretización

Tres nodos en orden antihorario sobre el plano (x, y) . Mapeo isoparamétrico desde el triángulo de referencia con vértices en $(0, 0)$, $(1, 0)$, $(0, 1)$. Jacobiano $\mathbf{J} = \partial(x, y)/\partial(\xi, \eta)$ es constante sobre el elemento; el código aborta con error si $\det \mathbf{J} \leq \text{tol}$ (nodos colineales o invertidos).

4.3.2 7. Funciones de forma

Coordenadas baricéntricas:

$$N_1 = 1 - \xi - \eta, \quad N_2 = \xi, \quad N_3 = \eta.$$

Sus derivadas en coordenadas naturales son constantes:

$$\partial N_1 / \partial \xi = -1, \quad \partial N_2 / \partial \xi = 1, \quad \partial N_3 / \partial \xi = 0;$$

$$\partial N_1 / \partial \eta = -1, \quad \partial N_2 / \partial \eta = 0, \quad \partial N_3 / \partial \eta = 1.$$

4.3.3 8. Matriz deformación-desplazamiento

$\mathbf{B}$ tiene tamaño 3×6 y, una vez transformadas las derivadas a globales vía $\mathbf{J}^{-1}$, sus entradas son **constantes** sobre el elemento:

$$\mathbf{B}_i = \begin{bmatrix} \partial N_i / \partial x & 0 \\ 0 & \partial N_i / \partial y \\ \partial N_i / \partial y & \partial N_i / \partial x \end{bmatrix}.$$

4.3.4 9. Rigidez elemental

$$\mathbf{K}_e = \mathbf{B}^\top \mathbf{D} \mathbf{B} A_e t,$$

con $A_e = \frac{1}{2} \det \mathbf{J}$ el área del triángulo y t el espesor. La integración es exacta con un único punto central porque $\mathbf{B}$ es constante.

4.3.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \mathbf{B}^\top \boldsymbol{\sigma} A_e t.$$

4.3.6 11. Cuadratura

Un punto central (en el baricentro) con peso $w = 1/2$ sobre el triángulo de referencia. Es exacto para formas constantes de $\mathbf{B}$ y $\boldsymbol{\sigma}$.

4.3.7 12. Cargas distribuidas consistentes

Fuerza de cuerpo $\mathbf{b}$. Con N_i lineales y $\mathbf{b}$ uniforme la integral es exacta:

$$\mathbf{f}_e^b = \int_{\Omega_e} \mathbf{N}^\top \mathbf{b} t dA \implies \text{cada nodo recibe } \frac{1}{3} \mathbf{b} A_e t.$$

Tracción de superficie $\bar{\mathbf{t}}$ sobre un borde recto. Con $\bar{\mathbf{t}}$ constante el reparto es $\frac{L}{2} \bar{\mathbf{t}} t$ en cada uno de los dos nodos del borde. Bordes numerados según la conectividad:

Edge	Nodos
0	(0, 1)
1	(1, 2)
2	(2, 0)

$\bar{\mathbf{t}}$ se especifica en coordenadas globales (t_x, t_y) . Tracción variable o presión normal no soportadas en este paso.

4.3.8 13. Salida por punto de Gauss

`compute_gauss_state(U)` devuelve la misma estructura que en `Quad4` pero con un único punto (centroide). Para `Tri3` ε y σ son constantes sobre el elemento, así que el dato del punto coincide con el promedio.

4.4 Limitación crítica — *shear locking*

El `Tri3` es notoriamente rígido en flexión: tres DOFs de desplazamiento no bastan para representar el modo de flexión puro de una viga discretizada en triángulos, por lo que el elemento responde con cortante espurio (*shear locking*) y subestima sistemáticamente la deflexión. La severidad crece con la relación L/h del problema. Recomendación operativa: usar `Quad4` por defecto; reservar `Tri3` para zonas de transición de malla en regiones donde el campo de tensiones varía suavemente.

4.5 Contrato de implementación

```
name: Tri3
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 3
  strain_dim: 3          # Voigt 2D [_xx, _yy, _xy]
  n_integration_points: 1

parameters:
  - { name: thickness, type: float, required: false, default: 1.0, desc: Espesor para
      estado plano }

material_contract:
  signature: "compute_state(, state) -> (, C_tangent, state)"
  strain_kind: "Voigt 2D [_xx, _yy, _xy]"
```

```

conventions:
    sign: "tensión positiva = tracción"
    voigt: "[xx, yy, xy] con _xy = 2·_xy"
    node_orientation: "antihorario (asegura det J > 0)"

validity:
    - "pequeños desplazamientos y deformaciones"
    - "campos de tensión que varían suavemente sobre el elemento"

out_of_scope:
    - "flexión dominante (shear locking severo)"
    - "elementos de alto orden (Tri6)"
    - "grandes deformaciones"

acceptance:
    verification:
        - name: patch_test_macneal_harder
          setup: "4 Tri3 distorsionados alrededor de un nodo interior descentrado
                  con campo lineal  $u = 1e-3 \cdot (x + y/2)$ ,  $v = 1e-3 \cdot (y + x/2)$ 
                  impuesto en los 4 nodos del contorno del cuadrado unitario"
          expect: "nodo interior adopta el campo lineal  $y = (1e-3, 1e-3, 1e-3)$  en cada elemento; uniforme"
          tol_rel: 1.0e-10
    specific:
        - name: parche de deformación constante sobre malla triangular regular
          setup: "malla triangular con campo de desplazamiento lineal y material Elastic2D"
          expect: "constante (CST reproduce campos lineales por construcción)"
          tol_rel: 1.0e-12
        - name: jacobiano degenerado abortado
          setup: "tres nodos colineales"
          expect: "ValueError en _compute_kinematics_tri3"
        - name: cargas consistentes - body load uniforme
          setup: "Tri3 con b uniforme"
          expect: "cada nodo recibe  $(1/3) \cdot b \cdot A_e \cdot t$ ;  $\Sigma = b \cdot A_e \cdot t$ "
          tol_rel: 1.0e-12
        - name: cargas consistentes - tracción uniforme en un borde
          setup: "Tri3 con tracción constante en un borde"
          expect: " $\Sigma_f = t \cdot L \cdot t$  y reparto  $L/2$  a cada nodo del borde, 0 en el opuesto"
          tol_rel: 1.0e-12

references:
    - "Cook, Malkus, Plesha, Witt, Concepts and Applications of FEA, §3.6 (CST)"
    - "Zienkiewicz O.C., The Finite Element Method, vol. 1, §5 (limitations of CST)"

```

4.6 Implementación

- **Archivo:** solidum/elements/solid_2d/tri3.py
- **Clase:** Tri3 (registrada vía `@ElementRegistry.register`)
- **Función núcleo:** `_compute_kinematics_tri3(coords)`, decorada con `@njit`, en `_shared.py`. Reutiliza `_compute_integrands` (también en `_shared`, compartido con `Quad4`) con peso $w = 0.5$.
- **Tests:**
 - `tests/test_solid_2d.py` — patch test sobre malla triangular regular y modos de cuerpo rígido.

- `tests/test_patch_solid_2d.py` — patch test de MacNeal-Harder con nodo interior libre (NAFEMS, V&V).

4.7 Diálogo

- **2026-04-30** · Spec retroactiva. `Tri3` precedía al protocolo `spec-first`; la spec se creó documentando la formulación ya implementada. Promovido `status: validated`.
- **2026-04-30** · Añadido patch test de MacNeal-Harder (NAFEMS) en `acceptance.verification`. Cuatro triángulos alrededor de un nodo interior descentrado reproducen un campo lineal impuesto en el contorno con ε constante en cada elemento.
- **2026-05-04** · Expuesta `compute_gauss_state(U)` (1 punto central). El `VtkExporter` consume σ por elemento y promedia a nodos (`Sigma*_nodal`).
- **2026-05-04** · Añadidas cargas distribuidas consistentes (`compute_body_load`, `compute_edge_traction`) con la misma semántica que `Quad4`. Para `Tri3` ambos integrandos son exactos analíticamente: body load reparte 1/3 por nodo, tracción de borde reparte $L/2$ a cada uno de los dos nodos del borde.

4.8 Quad4

Documentación retroactiva del elemento ya implementado. Cubre la formulación, el contrato de implementación y la trazabilidad con tests existentes.

4.9 Especificación física

4.9.1 0. Descripción general

Elemento sólido bidimensional plano de primer orden, cuadrilátero de cuatro nodos. Aproximación bilineal C^0 del campo de desplazamientos en coordenadas naturales $(\xi, \eta) \in [-1, 1]^2$. Formulación isoparamétrica: la geometría y los desplazamientos comparten las mismas funciones de forma. Régimen de pequeños desplazamientos y deformaciones.

4.9.2 1. Cinemática de desplazamientos

$$u_x(\xi, \eta) = \sum_{i=1}^4 N_i(\xi, \eta) u_x^{(i)}, \quad u_y(\xi, \eta) = \sum_{i=1}^4 N_i(\xi, \eta) u_y^{(i)}.$$

4.9.3 2. Cinemática de deformaciones

Tensor infinitesimal en notación Voigt 2D $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$:

$$\varepsilon_{xx} = \frac{\partial u_x}{\partial x}, \quad \varepsilon_{yy} = \frac{\partial u_y}{\partial y}, \quad \gamma_{xy} = \frac{\partial u_x}{\partial y} + \frac{\partial u_y}{\partial x}.$$

4.9.4 3. Ecuación constitutiva

Delegada al material 2D (STRAIN_DIM = 3). El elemento llama `material.compute_state( $\varepsilon$ , state) → ( $\sigma$ , C_tangent, state')` por punto de Gauss y soporta cualquier ley constitutiva 2D (Elastic2D, IsotropicDamage2D, VonMises2D).

4.9.5 4. Equilibrio — forma fuerte

$$-\nabla \cdot \boldsymbol{\sigma} = \mathbf{b} \quad \text{en } \Omega, \quad \boldsymbol{\sigma} \cdot \mathbf{n} = \bar{\mathbf{t}} \quad \text{en } \partial\Omega_N, \quad \mathbf{u} = \bar{\mathbf{u}} \quad \text{en } \partial\Omega_D.$$

4.9.6 5. Equilibrio — forma débil

$$\int_{\Omega} \delta \boldsymbol{\varepsilon}^\top \boldsymbol{\sigma} dV = \int_{\Omega} \delta \mathbf{u}^\top \mathbf{b} dV + \int_{\partial\Omega_N} \delta \mathbf{u}^\top \bar{\mathbf{t}} dS, \quad \forall \delta \mathbf{u} \in V_0.$$

4.10 Formulación numérica (FEM)

4.10.1 6. Discretización

Cuatro nodos en orden antihorario sobre el plano (x, y) . Mapeo isoparamétrico desde el cuadrado de referencia $[-1, 1]^2$:

$$x(\xi, \eta) = \sum_{i=1}^4 N_i(\xi, \eta) x^{(i)}, \quad y(\xi, \eta) = \sum_{i=1}^4 N_i(\xi, \eta) y^{(i)}.$$

Jacobiano $\mathbf{J} = \partial(x, y)/\partial(\xi, \eta)$ y su determinante $\det \mathbf{J}$ entran en el cambio de variable de la integración. El código aborta con error si $\det \mathbf{J} \leq \text{tol}$ (elemento degenerado o nodos en orden invertido).

4.10.2 7. Funciones de forma

Producto tensorial bilineal:

$$N_i(\xi, \eta) = \frac{1}{4}(1 + \xi_i \xi)(1 + \eta_i \eta), \quad (\xi_i, \eta_i) = (\pm 1, \pm 1).$$

Convención de numeración (antihorario): $(\xi_1, \eta_1) = (-1, -1)$, $(\xi_2, \eta_2) = (+1, -1)$, $(\xi_3, \eta_3) = (+1, +1)$, $(\xi_4, \eta_4) = (-1, +1)$.

4.10.3 8. Matriz deformación-desplazamiento

Las derivadas en globales se obtienen vía $\partial N_i/\partial x = \mathbf{J}^{-1} \partial N_i/\partial \xi$. La matriz $\mathbf{B}$ de tamaño 3×8 se ensambla por nodo:

$$\mathbf{B}_i = \begin{bmatrix} \partial N_i/\partial x & 0 \\ 0 & \partial N_i/\partial y \\ \partial N_i/\partial y & \partial N_i/\partial x \end{bmatrix}, \quad \boldsymbol{\varepsilon} = \mathbf{B} \mathbf{u}_e.$$

4.10.4 9. Rigidez elemental

$$\mathbf{K}_e = \int_{\Omega} \mathbf{B}^\top \mathbf{D} \mathbf{B} t dA = \sum_{p=1}^{n_g} \mathbf{B}(\xi_p, \eta_p)^\top \mathbf{D}_p \mathbf{B}(\xi_p, \eta_p) \det \mathbf{J}_p w_p t,$$

donde t es el espesor (**thickness**) y $\mathbf{D}_p$ es el tensor constitutivo tangente devuelto por el material en cada punto.

4.10.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \sum_{p=1}^{n_g} \mathbf{B}(\xi_p, \eta_p)^\top \boldsymbol{\sigma}_p \det \mathbf{J}_p w_p t.$$

4.10.6 11. Cuadratura

Por defecto Gauss–Legendre 2×2 (cuatro puntos): orden de exactitud 3 — integra exactamente $\mathbf{K}_e$ para Jacobiano constante (geometría de paralelogramo) y captura el comportamiento dominante en geometrías irregulares moderadas. Soporta esquemas alternativos vía el parámetro **quadrature** (lista nombrada en **QuadratureRegistry**).

Aviso sobre integración reducida: el esquema 1×1 está disponible pero el código emite advertencia explícita; un único punto central no detecta los modos de hourglass (energía nula con

desplazamientos de signo alternado), que pueden contaminar la respuesta sin estabilización adicional. No se implementa estabilización tipo Flanagan-Belytschko.

4.10.7 12. Cargas distribuidas consistentes

Fuerza de cuerpo $\mathbf{b}$ (e.g. peso propio $\mathbf{b} = (0, -\rho g)$, fuerza centrífuga). Vector nodal equivalente:

$$\mathbf{f}_e^b = \int_{\Omega_e} \mathbf{N}^\top \mathbf{b} t dA = \sum_{p=1}^{n_g} \mathbf{N}(\xi_p, \eta_p)^\top \mathbf{b} \det \mathbf{J}_p w_p t,$$

donde $\mathbf{N}$ es la matriz 2×8 de funciones de forma. Se reutiliza la cuadratura del elemento (Gauss 2×2 por defecto) — exacta para $\mathbf{b}$ uniforme y geometrías de paralelogramo, y adecuada para geometrías irregulares moderadas.

Tracción de superficie $\bar{\mathbf{t}}$ sobre un borde $\Gamma_e \subset \partial\Omega_N$:

$$\mathbf{f}_e^t = \int_{\Gamma_e} \mathbf{N}^\top \bar{\mathbf{t}} t dS.$$

Cada borde es una recta entre dos nodos; sobre él las funciones de forma son lineales y, para $\bar{\mathbf{t}}$ constante, la integral se reduce a un reparto exacto de $\frac{L}{2} \bar{\mathbf{t}} t$ a cada uno de los dos nodos del borde, donde L es la longitud del segmento. Bordes numerados según la conectividad nodal:

Edge	Nodos
0	(0, 1)
1	(1, 2)
2	(2, 3)
3	(3, 0)

$\bar{\mathbf{t}}$ se especifica en **coordenadas globales** (t_x, t_y) ; presiones normales se obtienen multiplicando previamente por la normal exterior del borde. Tracción variable a lo largo del borde no está soportada en este paso.

4.10.8 13. Salida por punto de Gauss

`compute_gauss_state(U)` devuelve un dict con $\boldsymbol{\varepsilon}$ y $\boldsymbol{\sigma}$ en cada uno de los n_g puntos de Gauss del elemento, junto con sus coordenadas naturales y globales. Habilita post-proceso fino (mapas no promediados, suavizado nodal, extrapolación tipo Barlow). `compute_internal_forces` queda como atajo de promedio.

4.11 Contrato de implementación

```
name: Quad4
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 4
  strain_dim: 3          # Voigt 2D [_xx, _yy, _xy]
  n_integration_points: 4 # default Gauss 2x2
```

```

parameters:
- { name: thickness, type: float, required: false, default: 1.0, desc: Espesor para
  estado plano }
- { name: quadrature, type: tuple, required: false, default: "2x2", desc: Regla de
  cuadratura desde QuadratureRegistry }

material_contract:
  signature: "compute_state(, state) -> (, C_tangent, state)"
  strain_kind: "Voigt 2D [_xx, _yy, _xy]"

conventions:
  sign: "tensión positiva = tracción"
  voigt: "[xx, yy, xy] con _xy = 2·_xy (engineering shear strain)"
  node_orientation: "antihorario (asegura det J > 0)"

validity:
- "pequeños desplazamientos y deformaciones"
- "geometría sin distorsión severa (det J > tol en todos los puntos de Gauss)"
- "compatible con materiales Elastic2D, IsotropicDamage2D, VonMises2D (este último
  solo plane_strain)"

out_of_scope:
- "grandes deformaciones, formulaciones corotacionales o totales lagrangianas"
- "estabilización contra hourglass para integración reducida"
- "elementos de alto orden (Q8, Q9)"

acceptance:
  verification:
- name: patch_test_macneal_harder
  setup: "5 Quad4 distorsionados (4 nodos exteriores + 4 interiores)
    con campo lineal u = 1e-3·(x + y/2), v = 1e-3·(y + x/2)
    impuesto en los 4 nodos del contorno"
  expect: "nodos interiores adoptan el campo lineal y = (1e-3, 1e-3,
    1e-3) en todos los puntos de Gauss; uniforme"
  tol_rel: 1.0e-10
  specific:
- name: parche de tracción uniaxial sobre malla regular
  setup: "malla regular con campo de desplazamiento lineal y material Elastic2D"
  expect: "_xx constante en todos los puntos de Gauss"
  tol_rel: 1.0e-12
- name: jacobiano degenerado abortado
  setup: "elemento con nodos colineales o en orden inverso"
  expect: "ValueError en _compute_kinematics"
- name: cargas consistentes - body load uniforme
  setup: "Quad4 regular y distorsionado con b uniforme; sumar componentes nodales"
  expect: " $\Sigma f_x = b_x \cdot A \cdot t$  y  $\Sigma f_y = b_y \cdot A \cdot t$  (invariante de geometría)"
  tol_rel: 1.0e-10
- name: cargas consistentes - tracción uniforme en un borde
  setup: "Quad4 con tracción constante en un borde"
  expect: " $\Sigma f = t \cdot L \cdot t$  y reparto L/2 a cada nodo del borde, 0 en los demás"
  tol_rel: 1.0e-12
- name: patch físico de tracción uniaxial
  setup: "Quad4 cuadrado L×H, borde izquierdo con ux=0 (n0,n3) + uy=0 (n0); borde
    derecho con tracción uniforme (p,0)"
  expect: "u reproduce ux=p·x/E, uy=-p·y/E (plane stress); _xx=p, _yy=_xy=0"
  tol_rel: 1.0e-10

references:
- "Bathe K.-J., Finite Element Procedures, §5.3 (isoparametric elements)"

```

```
- "Cook, Malkus, Plesha, Witt, Concepts and Applications of FEA, §6 (plane stress/strain)"
```

4.12 Implementación

- **Archivo:** `solidum/elements/solid_2d/quad4.py`
- **Clase:** `Quad4` (registrada vía `@ElementRegistry.register`)
- **Funciones núcleo:** `_compute_kinematics(xi, eta, coords)` y `_compute_integrands(B, C,  $\sigma$ , detJ, w, t)`, ambas decoradas con `@njit` (Numba) para evaluar $\mathbf{B}$, $\det \mathbf{J}$ y los integrandos a velocidad cercana a Fortran. Viven en `solidum/elements/solid_2d/_shared.py` y se comparten con `Tri3`.
- **Tests:**
 - `tests/test_solid_2d.py` — ensamblaje de $\mathbf{B}$, simetría de $\mathbf{K}_e$, modos de cuerpo rígido y tracción uniaxial sobre malla regular.
 - `tests/test_patch_solid_2d.py` — patch test de MacNeal-Harder sobre 5 `Quad4` distorsionados (NAFEMS, V&V).

4.13 Diálogo

- **2026-04-30** · Spec retroactiva. `Quad4` precedía al protocolo `spec-first`; la spec se creó documentando la formulación ya implementada y verificada. Promovido `status: validated`.
- **2026-04-30** · Añadido patch test de MacNeal-Harder (NAFEMS) en `acceptance.verification`. Cinco `Quad4` distorsionados con cuatro nodos interiores libres reproducen exactamente un campo lineal impuesto en el contorno, con ε constante e igual al gradiente analítico en todos los puntos de Gauss.
- **2026-05-04** · Expuesta `compute_gauss_state(U)` con σ y ε por punto de Gauss y coordenadas naturales/globales. `compute_internal_forces` queda como promedio derivado. El `VtkExporter` añade campos nodales `Sigma_XX_nodal`, `Sigma_YY_nodal`, `Tau_XY_nodal` y `Von_Mises_nodal` por promedio simple sobre los elementos contiguos a cada nodo (suavizado de orden 0).
- **2026-05-04** · Añadidas cargas distribuidas consistentes (`compute_body_load`, `compute_edge_traction`). Para tracciones uniformes en bordes rectos el reparto es exacto ($L/2$ a cada nodo del borde) y se evita la cuadratura 1D; las fuerzas de cuerpo se integran con la cuadratura del elemento. Solo tracciones **constantes por borde** y en **coordenadas globales** en este paso; tracción variable y presión normal quedan para una iteración posterior.

4.14 Tri6

*Triángulo isoparamétrico de 6 nodos. Cura el shear locking severo del **Tri3** y reproduce campos cuadráticos exactamente. Útil cuando la malla es triangular por geometría.*

4.15 Especificación física

Idéntica al **Tri3** salvo por la cinemática y la cuadratura.

4.15.1 7. Funciones de forma

Coordenadas baricéntricas $L_1 = 1 - \xi - \eta$, $L_2 = \xi$, $L_3 = \eta$:

$$N_i^{vert} = L_i(2L_i - 1), \quad i = 1, 2, 3$$

$$N_4 = 4L_1L_2, \quad N_5 = 4L_2L_3, \quad N_6 = 4L_3L_1.$$

Numeración: 0,1,2 vértices en (0,0), (1,0), (0,1); 3 medio del borde 0-1, 4 medio del 1-2, 5 medio del 2-0.

4.15.2 11. Cuadratura

Cuadratura triangular de 3 puntos (puntos medios de los lados; peso 1/6 cada uno) — exacta para polinomios hasta grado 2 sobre el triángulo de referencia.

4.15.3 12. Cargas distribuidas

- Body load con la cuadratura del elemento.
- Edge traction uniforme: reparto 1/6, 4/6, 1/6 a (vértice, medio, vértice).

4.16 Contrato de implementación

```
name: Tri6
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 6
  strain_dim: 3
  n_integration_points: 3

acceptance:
  verification:
    - name: patch_cuadratico
      setup: "u = (x2, 0) impuesto en los 6 nodos"
      expect: "_xx = 2x exacto en cada Gauss"
      tol_rel: 1.0e-10
    - name: cooks_membrane_4x4
      setup: "Trapezoide de Cook triangulado 4x4 (32 Tri6, diagonal →c1c3 con center node compartido)"
      expect: "u_y(48, 52) 23.91 (Bathe; Hughes)"
```

```
tol_abs: 1.5
- name: cooks_membrane_refinamiento
  setup: "Mallas 2x2 → 4x4 → 6x6 con la misma triangulación"
  expect: "Error |u_y - 23.91| decreciente monótonamente"
  tol_rel: 0.0
```

4.17 Implementación

- **Archivo:** solidum/elements/solid_2d/tri6.py · clase `Tri6` (subclase de la base interna `_HigherOrderSolid2D` en `_shared.py`, compartida con `Quad8` y `Quad9`). Declara `_MASS_QUADRATURE = "tri_6"` (Dunavant 6 puntos, orden 4) porque la cuadratura del elemento (`tri_3`, orden 2) subintegra el producto cuadrático×cuadrático de la masa consistente.
- **Tests:** tests/test_higher_order_solid_2d.py (patch cuadrático), tests/test_cooks_membrane.py (benchmark Bathe/Hughes con malla triangulada 4×4 + refinamiento monótono, añadido Fase B 2026-05-19).

4.18 Quad8

Elemento sólido 2D de orden cuadrático con interpolación serendípita.

*Reproduce exactamente campos hasta Q_2^{seren} (todos los polinomios de grado total ≤ 2 más algunos de orden 3 sin el término $\xi^2\eta^2$). Mucho más preciso que **Quad4** en flexión y geometrías curvas con el mismo grado de malla.*

4.19 Especificación física

4.19.1 0. Descripción general

Cuadrilátero plano de 8 nodos: 4 vértices + 4 nodos en los puntos medios de los bordes. Pequeñas deformaciones, isoparamétrico.

4.19.2 1. Cinemática de desplazamientos

$$u_x(\xi, \eta) = \sum_{i=1}^8 N_i(\xi, \eta) u_x^{(i)}, \quad u_y(\xi, \eta) = \sum_{i=1}^8 N_i(\xi, \eta) u_y^{(i)}.$$

4.19.3 2. Cinemática de deformaciones

Voigt 2D $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$, idéntica al Quad4.

4.19.4 3. Ecuación constitutiva

Material 2D (STRAIN_DIM = 3).

4.19.5 4-5. Equilibrio

Idénticas a Quad4.

4.20 Formulación numérica

4.20.1 6. Discretización

Nodos en orden estándar: 0-3 vértices antihorarios desde $(-1, -1)$; 4 medio del borde 0-1, 5 medio del 1-2, 6 medio del 2-3, 7 medio del 3-0. Mapeo isoparamétrico.

4.20.2 7. Funciones de forma serendípitas

Vértices ($i = 0, 1, 2, 3$ con $(\xi_i, \eta_i) \in \{\pm 1\}^2$):

$$N_i = \frac{1}{4}(1 + \xi_i\xi)(1 + \eta_i\eta)(\xi_i\xi + \eta_i\eta - 1).$$

Medios de borde:

- Bordes con $\xi = 0$ (nodos 4, 6): $N_i = \frac{1}{2}(1 - \xi^2)(1 + \eta_i\eta)$ con $\eta_i = \mp 1$.
- Bordes con $\eta = 0$ (nodos 5, 7): $N_i = \frac{1}{2}(1 + \xi_i\xi)(1 - \eta^2)$ con $\xi_i = \pm 1$.

4.20.3 8. Matriz B

Ensamblaje estándar a partir de $\partial N_i / \partial x = \mathbf{J}^{-1} \partial N_i / \partial \xi$. Tamaño 3×16 .

4.20.4 9-10. Rigidez y fuerzas internas

$$\mathbf{K}_e = \int \mathbf{B}^\top \mathbf{D} \mathbf{B} t dA, \quad \mathbf{F}_{\text{int}}^e = \int \mathbf{B}^\top \boldsymbol{\sigma} t dA.$$

Cuadratura Gauss 3×3 por defecto.

4.20.5 11. Cuadratura

Gauss-Legendre 3×3 (9 puntos) — exacta para polinomios hasta grado 5, suficiente para integrar $\mathbf{B}^\top \mathbf{D} \mathbf{B}$ del Quad8 sobre Jacobiano constante. La opción 2x2 queda disponible (reducida) pero introduce modos espurios; emite advertencia.

4.20.6 12. Cargas distribuidas consistentes

- **Body load:** $\int N^\top \mathbf{b} t dA$ con la cuadratura del elemento (3×3).
- **Edge traction** uniforme: en un borde recto con N cuadráticas la integral analítica reparte $L/6$ a cada nodo extremo y $4L/6$ al nodo medio (regla 1-4-1, Simpson). Bordes:

Edge	Nodos extremos	Nodo medio
0	(0, 1)	4
1	(1, 2)	5
2	(2, 3)	6
3	(3, 0)	7

4.20.7 13. Salida por punto de Gauss

`compute_gauss_state(U)` con la misma estructura que Quad4: 9 puntos por defecto.

4.21 Contrato de implementación

```

name: Quad8
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 8
  strain_dim: 3
  n_integration_points: 9      # default Gauss 3x3

parameters:
  - { name: thickness, type: float, required: false, default: 1.0 }
  - { name: quadrature, type: tuple, required: false, default: "3x3" }

material_contract:
  signature: "compute_state(, state) -> (, C_tangent, state)"

```

```

    strain_kind: "Voigt 2D [_xx, _yy, _xy]"

conventions:
    sign: "tensión positiva = tracción"
    voigt: "[xx, yy, xy] con _xy = 2·_xy"
    node_orientation: "vértices antihorarios; medios en el orden de los bordes"

validity:
    - "pequeños desplazamientos y deformaciones"

out_of_scope:
    - "grandes deformaciones"
    - "estabilización contra modos espurios bajo integración reducida"

acceptance:
    verification:
        - name: patch_lineal
          setup: "campo u = a + a·x + a·y impuesto en el contorno"
          expect: "constante en todos los Gauss e igual al gradiente analítico"
          tol_rel: 1.0e-10
        - name: patch_cuadratico
          setup: "campo u_x = x2·1e-3 impuesto en los 8 nodos"
          expect: "_xx(x, y) = 2e-3·x exacto en todos los puntos de Gauss"
          tol_rel: 1.0e-10
    specific:
        - name: cargas consistentes - body load uniforme
          setup: "Σf = b·A·t (regular y distorsionado)"
          tol_rel: 1.0e-10
        - name: cargas consistentes - tracción uniforme en un borde
          setup: "Σf = t·L·t y reparto 1/6, 4/6, 1/6 en (vértice, medio, vértice)"
          tol_rel: 1.0e-12
        - name: Cook's membrane (Cook 1974) # 2026-05-19
          setup: "trapezoid (0,0)-(48,44)-(48,60)-(0,44), E=1, ν=1/3, plane stress,
                cortante total F=1 distribuido en borde derecho; malla 4×4 Q8"
          expect: "u_y en (48,52) 23.91 ± 0.5; refinamiento 2×→24×→46×6 reduce error monot
                ónicamente"
          tol_abs: 0.5

references:
    - "Bathe K.-J., Finite Element Procedures, §5.3"
    - "Cook, Malkus, Plesha, Witt, Concepts and Applications of FEA, §6"
    - "Zienkiewicz O.C., The Finite Element Method, vol. 1, §8"

```

4.22 Implementación

- **Archivo:** solidum/elements/solid_2d/quad8.py · clase `Quad8` (subclase de la base interna `_HigherOrderSolid2D` en `_shared.py`, compartida con `Quad9` y `Tri6`).
- **Tests:** tests/test_higher_order_solid_2d.py.

4.23 Quad9

Variante Lagrangiana del Quad8: añade un noveno nodo central interior.

Espacio polinómico completo Q_2 (incluye el término $\xi^2\eta^2$ que el serendípito omite).

4.24 Especificación física

Idéntica a Quad8 salvo por las funciones de forma.

4.24.1 7. Funciones de forma

Producto tensorial de los polinomios de Lagrange 1D de orden 2:

$$L_0(\xi) = \frac{1}{2}\xi(\xi - 1), \quad L_1(\xi) = 1 - \xi^2, \quad L_2(\xi) = \frac{1}{2}\xi(\xi + 1).$$

$N_i(\xi, \eta) = L_a(\xi) L_b(\eta)$ con (a, b) asociado al nodo i según la numeración:

- 0..3: vértices $(-1, -1)$, $(+1, -1)$, $(+1, +1)$, $(-1, +1)$.
- 4: medio del borde 0-1 ($\eta = -1$).
- 5: medio del borde 1-2 ($\xi = +1$).
- 6: medio del borde 2-3 ($\eta = +1$).
- 7: medio del borde 3-0 ($\xi = -1$).
- 8: nodo central interior $(\xi, \eta) = (0, 0)$ — *bubble*.

4.24.2 11. Cuadratura

Gauss 3×3 por defecto.

4.24.3 12. Cargas distribuidas

Body load con la cuadratura del elemento. Tracción en bordes idéntica a Quad8 (1/6, 4/6, 1/6 — el nodo 8 no participa, no toca bordes).

4.25 Contrato de implementación

```
name: Quad9
kind: element
status: validated

interface:
  dof_names: [ux, uy]
  n_nodes: 9
  strain_dim: 3
  n_integration_points: 9

acceptance:
  verification:
    - name: patch_cuadratico
```

```
setup: "u = (x2, 0) impuesto en los 9 nodos"
expect: "_xx = 2x exacto en todos los Gauss"
tol_rel: 1.0e-10
- name: Cook's membrane (Cook 1974) # 2026-05-19
  setup: "trapezoid (0,0)-(48,44)-(48,60)-(0,44), E=1, =1/3, plane stress,
         cortante total F=1 distribuido en borde derecho; malla 4x4 Q9"
  expect: "u_y en (48,52) 23.91 ± 0.5"
  tol_abs: 0.5
```

4.26 Implementación

- **Archivo:** solidum/elements/solid_2d/quad9.py · clase `Quad9` (subclase de la base interna `_HigherOrderSolid2D` en `_shared.py`, compartida con `Quad8` y `Tri6`).
- **Tests:** tests/test_higher_order_solid_2d.py.

Capítulo 5

Elementos 2D — Discontinuidades embebidas

5.1 CST_Embedded2D

*Orden de trabajo. Pre-redactada por la IA como borrador para validación del usuario (autor de la formulación, Retama 2010). La cinemática KOS y la fórmula $\mathbf{l}_d = (A/h) \cdot \cos(\theta - \alpha)$ son traducción directa de los Caps. 2, 5, 6 y 7 de la tesis; la mecánica de implementación (Voigt, registry, signatura, contrato *Element*, condensación) sigue las convenciones del proyecto. El usuario revisa, corrige donde proceda, y aprueba antes de que la IA toque código.*

5.2 Especificación física

5.2.1 0. Descripción general

Elemento finito sólido 2D triangular de tres nodos (CST padre) **enriquecido** con una **discontinuidad interior embebida** Γ_d que cruza el dominio del elemento cuando se cumple un criterio de activación. Es la primera implementación de la familia de elementos con DOFs enriquecidos elementales introducida por **ADR 0010**, y la **fase 2** de la hoja de ruta de discontinuidades embebidas.

Estructura del elemento:

- **Estado intacto** (`discontinuity_state is None`): se comporta exactamente como un **Tri3** estándar — CST con un único punto de Gauss, B constante, integración exacta del bulk con peso **A_e.t**.
- **Estado agrietado** (`discontinuity_state is not None`): el campo de desplazamientos del bulk se enriquece con un salto $[[\mathbf{u}]] \in \mathbb{R}^2$ localizado en Γ_d . El bulk vecino descarga elásticamente; la disipación se concentra en Γ_d y la gobierna un material cohesivo (**CohesiveMaterial**, ADR 0010 §1).

Aproximación adoptada: **discrete approach** (Retama 2010), **KOS** (Kinematically Optimal Symmetric) según la clasificación de Jirásek (2000). Tracking trivial: la normal $\mathbf{n}$ y la posición de Γ_d se fijan al activar el elemento (perpendicular a σ_I por criterio Rankine; Γ_d pasa por el centroide) y no se reorientan después. Modo-I puro (consistente con **CohesiveDamageIsotropic** fase 1); modo mixto y KSON quedan diferidos (fases G y H del ADR 0010).

Régimen: pequeños desplazamientos y deformaciones, cuasi-estático. Pensado para hormigón, roca, cerámicos cuasi-frágiles. Bulk **elástico** por construcción de la formulación (la disipación está localizada en Γ_d); materiales no lineales del bulk quedan **out_of_scope** en fase 1.

5.2.2 1. Cinemática de desplazamientos (formulación KOS)

Campo total de desplazamientos en el elemento:

$$\mathbf{u}(\mathbf{x}) = \underbrace{\sum_{i=1}^3 N_i(\mathbf{x}) \mathbf{d}_i}_{\mathbf{u}^{\text{std}}(\mathbf{x})} + \underbrace{(H_{\Gamma_d}(\mathbf{x}) - \varphi(\mathbf{x})) \mathbf{G} \llbracket \mathbf{u} \rrbracket}_{\mathbf{u}^{\text{enr}}(\mathbf{x})}$$

donde:

- $N_i(\mathbf{x})$ son las funciones de forma estándar del CST (coordenadas baricéntricas; ver **Tri3** §7).
- $\mathbf{d}_i \in \mathbb{R}^2$ son los desplazamientos nodales estándar (DOFs globales del modelo, 2 por nodo).
- $\llbracket \mathbf{u} \rrbracket \in \mathbb{R}^2$ es el salto de desplazamientos a través de Γ_d , expresado en el **frame local** ($\mathbf{n}$, $\mathbf{s}$) de la discontinuidad. Es un **DOF enriquecido elemental** — vive en el elemento, no se ensambla al sistema global (ADR 0010 §2).
- $H_{\{\Gamma_d\}}(\mathbf{x})$ es la función escalón de Heaviside relativa a Γ_d (0 en Ω^- , 1 en Ω^+). Ω^+ es el subdominio del lado al que apunta $\mathbf{n}$.
- $\varphi(\mathbf{x})$ es una función de soporte continua que vale 1 en los nodos de Ω^+ y 0 en los de Ω^- . Para CST con Γ_d cortando dos lados, $\varphi(\mathbf{x})$ es la combinación lineal $\sum N_i^+(\mathbf{x})$ sobre los nodos del lado positivo (típicamente un único nodo solitario, ver §6).
- $\mathbf{G}$ es la matriz que transforma el salto del frame local al sistema cartesiano global:

$$\mathbf{G} = \begin{bmatrix} n_x & s_x \\ n_y & s_y \end{bmatrix}$$

La construcción $(H_{\{\Gamma_d\}} - \varphi)$ garantiza que $\mathbf{u}^{\text{enr}}$ sea **continuo en el contorno del elemento** (porque $H_{\{\Gamma_d\}} = \varphi$ en los nodos) y que el salto correcto $\llbracket \mathbf{u} \rrbracket = \mathbf{G}^T \cdot \llbracket \mathbf{u} \rrbracket_{\text{global}}$ ocurra únicamente sobre Γ_d interior. Es la elección **kinematically optimal**: el campo discontinuo está bien localizado y el salto no se transmite a elementos vecinos (a diferencia de XFEM, donde el salto sí cruza fronteras inter-elemento).

5.2.3 2. Cinemática de deformaciones

Deformación en el bulk (parte regular del campo):

$$\boldsymbol{\varepsilon}^{\text{bulk}}(\mathbf{x}) = \mathbf{B}_{\text{std}} \mathbf{d} - \mathbf{B}^\varphi(\mathbf{x}) \mathbf{G} \llbracket \mathbf{u} \rrbracket$$

donde $\mathbf{B}_{\text{std}}$ es la matriz $\mathbf{B}$ del CST estándar (Voigt 2D, 3×6 , constante, ver **Tri3** §8) y $\mathbf{B}^\varphi(\mathbf{x})$ es la matriz Voigt construida con $\nabla \varphi$:

$$\mathbf{B}^\varphi = \begin{bmatrix} \partial \varphi / \partial x & 0 \\ 0 & \partial \varphi / \partial y \\ \partial \varphi / \partial y & \partial \varphi / \partial x \end{bmatrix}$$

Como $\varphi = \sum N_i \hat{\varphi}_i$, las derivadas $\partial\varphi/\partial\mathbf{x}$, $\partial\varphi/\partial\mathbf{y}$ son **constantes sobre el elemento** (CST lineal). Por tanto $\mathbf{B}^T\varphi$ es constante.

Salto cinemático en Γ_d (en frame local):

$$[\![\mathbf{u}(\mathbf{x}_{\Gamma_d})]\!] = \mathbf{G}^T [\mathbf{u}(\mathbf{x}^+) - \mathbf{u}(\mathbf{x}^-)] = [\![\mathbf{u}]\!] \quad (\text{i.e. los propios DOFs enriquecidos})$$

por construcción de la cinemática KOS — el salto en Γ_d coincide exactamente con la variable enriquecida, sin contribución de $\mathbf{d}$ (continuidad del campo estándar atravesando Γ_d).

5.2.4 3. Ecuación constitutiva

Bulk (en todo el dominio del elemento Ω_e):

$$\boldsymbol{\sigma}(\mathbf{x}) = \mathbf{C}_e \boldsymbol{\varepsilon}^{\text{bulk}}(\mathbf{x})$$

con $\mathbf{C}_e$ la matriz constitutiva elástica isotropa (parámetros $\mathbf{E}$, ν , hipótesis plane stress / plane strain del material `Elastic2D`).

Discontinuidad (sobre Γ_d):

$$\mathbf{t}(\mathbf{x}_{\Gamma_d}) = \mathbf{T}_{\text{coh}}([\![\mathbf{u}]\!], \text{estado cohesivo})$$

con $\mathbf{T}_{\text{coh}}$ el operador del material cohesivo (en frame local; ver spec `CohesiveDamageIsotropic`).

5.2.5 4. Equilibrio — forma fuerte

Balance estático en cada subdominio (Ω^- , Ω^+) más continuidad de tracciones a través de Γ_d :

$$\nabla \cdot \boldsymbol{\sigma} + \mathbf{b} = \mathbf{0} \quad \text{en } \Omega \setminus \Gamma_d, \quad \boldsymbol{\sigma} \cdot \mathbf{n} = \mathbf{t} \quad \text{en } \Gamma_d.$$

5.2.6 5. Equilibrio — forma débil (residuales acoplados)

Aplicando el principio de trabajos virtuales con variaciones independientes $\delta\mathbf{d}$ y $\delta[\![\mathbf{u}]\!]$, se obtienen **dos residuales acoplados**:

$$\begin{aligned} \mathbf{R}^d &= \int_{\Omega_e} \mathbf{B}_{\text{std}}^T \boldsymbol{\sigma} d\Omega - \mathbf{f}_d^{\text{ext}} = \mathbf{0} \\ \mathbf{R}^{[\![u]\!]} &= - \int_{\Omega_e} \mathbf{G}^T (\mathbf{B}^\varphi)^T \boldsymbol{\sigma} d\Omega + l_d \mathbf{t} = \mathbf{0} \end{aligned}$$

con l_d la longitud efectiva de Γ_d dentro del elemento (ver §11). El primer residual se ensambla al sistema global; el segundo se resuelve **localmente** dentro del elemento (ver §10, condensación estática).

5.3 Formulación numérica (FEM)

5.3.1 6. Discretización del CST padre

Idéntica a `Tri3`: tres nodos en orden antihorario, mapeo isoparamétrico desde el triángulo de referencia con vértices $(0,0)$, $(1,0)$, $(0,1)$. Jacobiano constante; aborta con error si $\det \mathbf{J} \leq \text{tol}$.

Identificación del nodo solitario. Cuando Γ_d corta dos lados del triángulo, exactamente un nodo queda aislado en Ω^+ (lado al que apunta $\mathbf{n}$); los otros dos quedan en Ω^- . El nodo solitario

es el **único** que tiene $\varphi = 1$; los otros dos tienen $\varphi = 0$. La identificación se hace al activar el elemento, comparando el producto $(\mathbf{x}_i - \mathbf{x}_c) \cdot \mathbf{n}$ para cada nodo ($\mathbf{x}_c = \text{centroide}$), donde el signo del producto determina el lado.

5.3.2 7. Funciones de forma + función φ

$N_i(\mathbf{x})$ idénticas a Tri3 §7. Para φ :

$$\varphi(\mathbf{x}) = N_{i^*}(\mathbf{x})$$

donde i^* es el índice del nodo solitario. Es lineal y C^0 continua, vale 1 en i^* y 0 en los otros dos. Sus derivadas son $(\partial N_{i^*}/\partial x, \partial N_{i^*}/\partial y)$, **constantes** sobre el elemento.

5.3.3 8. Matrices B estándar + B^φ

B_{std} es la matriz B clásica del CST (constante, 3×6); ver Tri3 §8.

B^φ es la matriz Voigt 3×2 construida con $(\partial\varphi/\partial x, \partial\varphi/\partial y)$:

$$\mathbf{B}^\varphi = \begin{bmatrix} \partial\varphi/\partial x & 0 \\ 0 & \partial\varphi/\partial y \\ \partial\varphi/\partial y & \partial\varphi/\partial x \end{bmatrix}$$

Por construcción, B^φ coincide con la **submatriz B_{std} correspondiente al nodo solitario** (las columnas $2 \cdot i^*$ y $2 \cdot i^* + 1$ de B_{std}). Eso permite implementación trivial sin recomputar derivadas.

5.3.4 9. Sistema acoplado — matrices elementales

Linealización del residual acoplado conduce al sistema 8×8 local (6 DOFs estándar + 2 enriquecidos):

$$\begin{bmatrix} \mathbf{K}_{dd} & \mathbf{K}_{d[u]} \\ \mathbf{K}_{[u]d} & \mathbf{K}_{[u][u]} \end{bmatrix} \begin{bmatrix} \Delta \mathbf{d} \\ \Delta [u] \end{bmatrix} = \begin{bmatrix} -\mathbf{R}^d \\ -\mathbf{R}^{[u]} \end{bmatrix}$$

con (todas las cantidades evaluadas en el único punto de Gauss central, peso $A_e \cdot t$):

- $K_{dd} = B_{\text{std}}^T \cdot C_e \cdot B_{\text{std}} \cdot A_e \cdot t$ (6×6) — rigidez estándar del CST.
- $K_{\{d[[u]]\}} = -B_{\text{std}}^T \cdot C_e \cdot B^\varphi \cdot G \cdot A_e \cdot t$ (6×2) — acoplamiento.
- $K_{\{[[u]] d\}} = -G^T \cdot (B^\varphi)^T \cdot C_e \cdot B_{\text{std}} \cdot A_e \cdot t$ (2×6) — traspuesta de $K_{\{d[[u]]\}}$ en KOS (simétrica por construcción).
- $K_{\{[[u]][[u]]\}} = G^T \cdot (B^\varphi)^T \cdot C_e \cdot B^\varphi \cdot G \cdot A_e \cdot t + l_d \cdot T_{\text{coh}}$ (2×2) — rigidez del salto, suma de contribución del bulk (descarga elástica del modo enriquecido) y de la rigidez tangente cohesiva $T_{\text{coh}} = \partial t / \partial [[u]]$ proyectada sobre Γ_d .

Convención de signos: el $-$ en $K_{\{d[[u]]\}}$ y $K_{\{[[u]]d\}}$ viene del signo del término enriquecido en $B^{\text{enr}} = -B^\varphi \cdot G$ (recordar $(H - \varphi)$ con $\nabla H = \delta_{\{\Gamma_d\}} \cdot \mathbf{n}$).

5.3.5 10. Condensación estática local

El sistema 8×8 se condensa eliminando los DOFs enriquecidos $\Delta[[u]]$ antes de devolver K_e al ensamblador (ADR 0010 §3):

$$\Delta[[u]] = -K_{[[u]][[u]]}^{-1} (R^{[u]} + K_{[[u]]d} \Delta d)$$

Sustituyendo en la primera ecuación:

$$K_{\text{cond}} = K_{dd} - K_{d[[u]]} K_{[[u]][[u]]}^{-1} K_{[[u]]d}$$

$$F_{\text{int}}^{\text{cond}} = F_{\text{int}}^d - K_{d[[u]]} K_{[[u]][[u]]}^{-1} R^{[u]}$$

donde $F_{\text{int}}^d = B_{\text{std}}^T \cdot \sigma \cdot A_e \cdot t$ es la fuerza interna estándar antes de condensar.

Inversión de $K_{[[u]][[u]]}$ (2×2): cerrada analíticamente. El despachador algebraico (ADR 0003) no participa al ser una matriz elemental densa de 2×2 .

Recuperación local tras converger el Newton global: $[[u]]_{\text{committed}} += \Delta[[u]]$ con la fórmula de arriba aplicada a $\Delta d = d_{n+1} - d_n$. Esta recuperación se hace en `commit_state()`, no en `compute_element_state()`.

5.3.6 11. Activación: criterio Rankine + dirección $n + 1_d$ del Cap. 6

Criterio de activación. Al inicio de cada paso de carga (no dentro del Newton — ver §14), el elemento intacto evalúa su tensión principal mayor σ_I en el centroide:

$$\sigma_I = \frac{\sigma_{xx} + \sigma_{yy}}{2} + \sqrt{\left(\frac{\sigma_{xx} - \sigma_{yy}}{2}\right)^2 + \sigma_{xy}^2}$$

Si $\sigma_I > \sigma_{t0}$ (resistencia a tracción del material cohesivo), se activa:

- n = autovector unitario de σ_I (perpendicular a la dirección de tracción máxima, criterio Rankine).
- s = ortogonal a n orientada por regla de la mano derecha con $+z$ saliendo del plano (consistente con Reglas.md §5).
- Γ_d pasa por el centroide del CST (decisión cerrada: la posición no es DOF; se fija en el activación, igual que n).
- Identificación del nodo solitario i^* (§6).

Longitud efectiva 1_d — Cap. 6 de Retama (2010). Fórmula cerrada:

$$l_d = \frac{A_e}{h} \cos(\theta - \alpha)$$

donde:

- A_e es el área del triángulo.
- h es la altura desde el nodo solitario i^* al lado opuesto.

- θ es el ángulo de Γ_d respecto a $\mathbf{x}$ global (i.e. $\theta = \text{atan2}(s_y, s_x)$ con $\mathbf{s} = (s_x, s_y)$ la tangente a Γ_d).
- α es el ángulo del lado opuesto al nodo solitario respecto a $\mathbf{x}$ global.

Esta fórmula es **decisión cerrada en el ADR 0010** (§6); no se relitiga aquí. La alternativa ingenua $l_d = A_e/h$ (independiente de θ) produce stress locking cuando la grieta no es paralela al lado opuesto; ver fase 3 del ADR 0010 para test numérico.

5.3.7 12. Estado de la discontinuidad — DiscontinuityState

Dataclass paralela a `ElementState` (ADR 0010 §4), específica de la discontinuidad. Vive en `solidum/core/discontinuity_state.py`:

```
@dataclass
class DiscontinuityState:
    normal: np.ndarray      #  $\mathbf{n}^2$ , unitario, fijado al activar
    tangent: np.ndarray      #  $\mathbf{s}^2$ ,  $\mathbf{s} = -(\mathbf{n}_y, \mathbf{n}_x)$ 
    centroid: np.ndarray     # punto por el que pasa  $\Gamma_d$ 
    solitary_node: int       # índice (0, 1 o 2) del nodo en  $\Omega$ 
    l_d: float               #  $(A_e/h) \cdot \cos(-)$ 
    jump_committed: np.ndarray #  $[[u]]$  al cierre del último paso convergido
    jump_trial: np.ndarray    #  $[[u]]$  dentro de la iteración Newton
    cohesive_state_committed: dict # estado interno del CohesiveMaterial (committed)
    cohesive_state_trial: dict    # estado interno del CohesiveMaterial (trial)
```

Semántica **trial** / **commit** consistente con `ElementState`: `jump_trial` y `cohesive_state_trial` evolucionan en cada iteración del Newton global; al converger el paso, `commit_state()` copia `trial` → `committed` y aplica la recuperación local de $\Delta[[u]]$ (§10).

5.3.8 13. Cuadratura

Un único punto de Gauss central, peso $A_e \cdot t$ — idéntico a `Tri3`. Las matrices $\mathbf{B}_{std}$, $\mathbf{B}^T \varphi$ y la deformación del bulk son constantes; la integración es exacta. El término cohesivo $l_d \cdot t$ se evalúa directamente sobre Γ_d (no requiere cuadratura porque t es constante a lo largo de Γ_d por la cinemática KOS — el salto es constante).

5.3.9 14. Activación al principio del paso — interacción con el solver

ADR 0010 §5 fija que la activación se evalúa **al inicio de cada paso de carga, no dentro del Newton**. Razón: dentro del Newton, σ_I oscila por el predictor lineal y puede cruzar/descruzar el umbral varias veces antes de converger — *chattering*.

Implementación propuesta (a validar — punto abierto en §Diálogo): añadir al contrato de `Element` un método opcional `prepare_step(U_committed)` que el `NonlinearSolver/ArcLengthSolver` invoca **una vez por paso** antes del primer Newton. La base lo hace no-op (la mayoría de elementos no necesitan preparación); `CST_Embedded2D` lo sobrescribe para evaluar la activación con el estado convergido del paso anterior. Una vez activado, el elemento queda agrietado **irreversiblemente** (no revierte si el siguiente paso lo descarga).

5.4 Caveats numéricos

- **Singularidad de $K_{[[u]][[u]]}$ en Modo-I puro saturado.** Cuando $\omega \rightarrow 1$ en el cohesivo, la rigidez tangente cohesiva T_{coh} se aproxima a la diagonal $[(1-DAMAGE_MAX) \cdot K_e,$

0] (la componente tangencial es cero por construcción del Modo-I). La suma con la contribución del bulk $\mathbf{G}^T \cdot (\mathbf{B}^T \boldsymbol{\varphi})^T \cdot \mathbf{C}_e \cdot \mathbf{B}^T \boldsymbol{\varphi} \cdot \mathbf{G}$ introduce rigidez en la dirección tangencial vía el bulk; la matriz 2×2 sigue siendo invertible. Verificable analíticamente.

- **Activación al principio del paso.** Si el incremento es muy grande, σ_I puede saltar muy por encima de σ_{t0} antes de la activación, generando un $\Delta[[u]]$ grande en el primer Newton. Mitigación: pasos suficientemente pequeños o `ArcLengthSolver` cerca del pico.
- **Bulk descarga elástica.** Por la cinemática KOS, el bulk evalúa $\varepsilon^{\text{bulk}} = \mathbf{B}_{\text{std}} \cdot \mathbf{d} - \mathbf{B}^T \boldsymbol{\varphi} \cdot \mathbf{G} \cdot [[u]]$ — descarga conforme $[[u]]$ crece. Esto es físicamente correcto (la disipación va a Γ_d) y consistente con la discrete approach de Retama 2010. El bulk **no acumula daño** propio (`out_of_scope` fase 1).
- **Γ_d paralelo a un lado.** Caso degenerado: $\cos(\theta - \alpha) = 1$, $l_d = A_e/h$. La fórmula del Cap. 6 sigue siendo correcta, pero numéricamente el nodo solitario debe identificarse con tolerancia para evitar oscilación entre dos nodos casi en el mismo plano.
- **Γ_d casi vertical en CST con jacobiano malacondicionado.** Sin patología nueva: hereda los caveats del CST padre. El test `det J > tol` de `Tri3` cubre el caso.
- **Stress locking.** La fase 3 del ADR 0010 valida numéricamente que $l_d = (A_e/h) \cdot \cos(\theta - \alpha)$ evita el locking que aparece con $l_d = A_e/h$ ingenuo. Test cubierto en `tests/test_ld_chapter_6_validation` con Γ_d oblicua respecto al lado opuesto al nodo solitario ($\cos(\theta - \alpha) = \sqrt{3}/2$ en el setup), la versión correcta produce una apertura $[[u_n]] \approx 5.45\%$ mayor que la versión ingenua para una misma carga, en rama de softening exponencial activo. La versión ingenua infla la rigidez cohesiva ($l_d \cdot T_{\text{coh}}$) por un factor $1/\cos(\theta - \alpha) > 1$, restringe la apertura y el elemento se comporta como si fuera más rígido — eso es el *stress locking*. Cuando Γ_d es paralela al lado opuesto ($\cos(\theta - \alpha) = \pm 1$), las dos fórmulas coinciden exactamente y no hay locking.

5.5 Contrato de implementación

```
name: CST_Embedded2D
kind: element
status: validated

interface:
  dof_names: [ux, uy]          # 2 DOFs globales por nodo (los enriquecidos NO son
                                # globales)
  n_nodes: 3
  strain_dim: 3                # Voigt 2D [_xx, _yy, _xy]
  n_integration_points: 1      # CST con cuadratura central

parameters:
- { name: thickness,           type: float, required: false, default: 1.0,
    desc: "Espesor para estado plano (idéntico a Tri3)." }
- { name: cohesive_material,   type: str,   required: true,
    desc: "Nombre (en CohesiveMaterialRegistry) del material cohesivo que gobierna
          \Gamma_d cuando se activa." }
- { name: activation_criterion, type: str,   required: false, default: rankine,
    desc: "Criterio de activación: 'rankine' (única opción fase 1)." }

material_contract:
  bulk:
```

```

signature: "compute_state(, state) -> (, C_tangent, state')"
strain_kind: "Voigt 2D [_xx, _yy, _xy]"
accepted: ["Elastic2D"] # fase 1: bulk elástico; otros → out_of_scope
cohesive:
  signature: "compute_traction([[u]], state) -> (t, T_tan, state')"
  jump_kind: "frame local - [[u]] = ([[u_n]], [[u_s]]) en ejes (n, s) de  $\Gamma_d$ "
  accepted: ["CohesiveDamageIsotropic"] # cualquier CohesiveMaterial con JUMP_DIM=2

conventions:
  sign: "tracción interna positiva; convención stress-resultant del proyecto (Reglas.md §5)."
  voigt: "[xx, yy, xy] con _xy = 2·_xy."
  node_orientation: "antihorario (det J > 0)."
  frame_local_discontinuity: "n perpendicular a _I al activarse; s = -(n_y, n_x) (RHR con +z saliendo del plano)."
  activation: "irreversible; evaluada al principio de cada paso con el estado convergido previo."

validity:
  - "Pequeños desplazamientos y deformaciones (no large strain)."
  - "Bulk elástico (Elastic2D) y cohesivo Modo-I (JUMP_DIM = 2)."
  - " $\Gamma_d$  cortando exactamente dos lados del triángulo (configuración geométrica estándar)."

out_of_scope:
  - "Bulk no lineal (J2, daño continuo, Drucker-Prager): la disipación debe localizarse en  $\Gamma_d$ , no co-existir con disipación de bulk."
  - "Modo mixto I-II en el cohesivo (fase G del ADR 0010)."
  - "Reorientación de n tras la activación (tracking no trivial, fase F)."
  - "Múltiples discontinuidades por elemento."
  - "Contacto unilateral en compresión (cierre rígido)."
  - "Elementos de orden superior con discontinuidad embebida (Tri6_Embedded, Quad8_Embedded; fase J)."
  - "3D (fase I)."

acceptance:
  verification:
    - name: estado_intacto_reproduce_Tri3
      setup: "elemento sin activar (discontinuity_state=None), bulk Elastic2D; aplicar campo lineal de desplazamientos."
      expect: "K_e y F_int_e idénticos (bit-exact en doble) a los de un Tri3 con el mismo bulk."
      tol_rel: 1.0e-14

    - name: patch_test_intacto
      setup: "patch MacNeal-Harder con 4 CST_Embedded2D distorsionados, ninguno activado, < _t0 en todo el dominio."
      expect: "campo lineal exacto en nodos interiores; constante por elemento."
      tol_rel: 1.0e-10

    - name: activacion_rankine_correcta
      setup: "tracción uniaxial pura en x sobre un único CST con _t0 conocido; incrementar carga hasta cruzar _t0."
      expect: "activación en el paso donde _xx > _t0; n = (1, 0) y s = (0, 1) con error angular < 1e-10."
      tol_rel: 1.0e-10

    - name: bulk_descarga_elasticamente_post_activacion

```

```

    setup: "activar elemento, después fijar  $[[u]]$  e imponer  $\Delta d$ ; verificar que la
           deformación del bulk responde como elástica con  $C_e$ ."
    expect: "_bulk =  $C_e \cdot (B_{std} \cdot d - B^{\wedge} G \cdot [[u]])$ ; coincide con  $C_e$  aplicado a la
           neta."
    tol_rel: 1.0e-12

- name: condensacion_simetrica_KOS
  setup: "elemento activado con cohesivo en carga activa simétrica (Modo-I puro).".
  expect: "K_cond simétrica:  $||K_{cond} - K_{cond}^{\wedge} T_F / |||K_{cond}_F < 1e-12$ ."
  tol_rel: 1.0e-12

- name: ld_formula_cerrada
  setup: "varias orientaciones de  $\Gamma_d$  ( {0, /6, /4, /3, /2}) sobre un triángulo
           conocido."
  expect: "l_d coincide con  $(A_e/h) \cdot \cos - ()$  calculado analíticamente."
  tol_rel: 1.0e-14

- name: recuperacion_local_del_salto
  setup: "carga monótona hasta varios pasos post-activación; tras cada commit,  $[[u]]_{committed}$  debe satisfacer  $R^{\wedge} [[u]] = 0$ ."
  expect: " $||R^{\wedge} [[u]]|| < 1e-8$  tras commit_state."
  tol_rel: 1.0e-8

specific:
- name: bulk_no_lineal_rechazado
  setup: "instanciar CST_Embedded2D con bulk_material=IsotropicDamage2D."
  expect: "ValueError clara en construcción, mensaje listando los bulks aceptados (
           fase 1: Elastic2D).".

- name: jump_dim_validado
  setup: "instanciar con cohesive_material cuyo JUMP_DIM=3 (futuro 3D).".
  expect: "ValueError con mensaje sobre incompatibilidad dimensional (elemento 2D
           requiere JUMP_DIM=2).".

- name: activacion_no_chattering_dentro_de_newton
  setup: "elemento cerca del umbral; correr iteraciones Newton sin invocar
           prepare_step."
  expect: "estado discontinuity_state no cambia durante las iteraciones; sólo
           prepare_step modifica activación.".

- name: identificacion_nodo_solitario
  setup: "construir CST con coordenadas tales que solo un nodo cae en  $\Omega$  para una n
           dada."
  expect: "DiscontinuityState.solitary_node identifica correctamente el índice; los
           otros dos tienen =0.".

- name: ld_invariante_bajo_reflejo_de_n
  setup: "activar con n y comparar con activar con -n (mismo problema físico).".
  expect: "l_d, K_cond, F_int^cond idénticos (la convención de n positivo está bien
           definida - apunta hacia el nodo solitario).".
  tol_rel: 1.0e-14

arch:
- name: registro_en_ElementRegistry
  expect: "CST_Embedded2D registrado con kind='element'.".

- name: STRAIN_DIM_y_DOF_NAMES_correctos
  expect: "STRAIN_DIM=3, DOF_NAMES=['ux', 'uy'], N_NODES=3, N_INTEGRATION_POINTS=1."

```



```

- name: prepare_step_es_no_op_en_Element_base
  expect: "Element.prepare_step existe y no hace nada por defecto; CST_Embedded2D
          lo sobrescribe."

references:
- "Retama Velasco, J. (2010). Formulation and Approximation to Problems in Solids by
  Embedded Discontinuity Models. Tesis Doctoral, UNAM. **Caps. 2 (cinemática KOS),
  5 (formulación variacional discreta), 6 (l_d correcto), 7 (condensación estática)
  **."
- "Jirásek, M. (2000). Comparative study on finite elements with embedded
  discontinuities. CMAME 188, 307-330. - Clasificación SOS/KOS/SKON."
- "Oliver, J., Huespe, A.E., Sánchez, P.J. (2006). A comparative study on finite
  elements for capturing strong discontinuities. CMAME 195, 4732-4752."
- "Linder, C., Armero, F. (2007). Finite elements with embedded strong
  discontinuities. IJNME 72, 1391-1433."
- "ADR 0010 - Discontinuidades interiores embebidas (hoja de ruta; fase 2 = este
  elemento)."
```

```

- "Spec [CohesiveDamageIsotropic](CohesiveDamageIsotropic.md) - material cohesivo
  consumido en  $\Gamma_d$ ."
```

```

- "Spec [Tri3](Tri3.md) - CST padre del elemento; mismo bulk en estado intacto."
```

5.6 Implementación

- Archivo: `solidum/elements/solid_2d/embedded_cst.py`
- Clase: `CST_Embedded2D` (registrada en `ElementRegistry`)
- Estado de la discontinuidad: `solidum/core/discontinuity_state.py` (`DiscontinuityState`)
- Hook de paso en clase base: `Element.prepare_step(U_committed)` (no-op por defecto)
- Integración en solvers:
- `NonlinearSolver`: invoca `self.assembler.prepare_all_steps(U_current)` al inicio de cada paso, antes del Newton.
- `ArcLengthSolver`: idem, antes del predictor.
- Parser YAML: nueva sección `cohesive_materials`: + campo `cohesive_material`: por elemento.
- Tests: `tests/test_cst_embedded.py` — 24 tests cubriendo `acceptance` completo (verification, specific, arch) + integración end-to-end del YAML.
- Notas de traducción:
- Estado intacto (`discontinuity_state` is `None`) → bit-exact con `Tri3` (mismo `B`, mismo `_compute_integrands`).
- Estado agrietado: Newton local sobre $[[u]]$ hasta $R^{\{[[u]]\}} = 0$, después condensación estática local cerrada (K_{jj} es 2×2). Tolerancia interna: `_LOCAL_JUMP_RTOL` = `1e-10`, `_LOCAL_JUMP_MAX_ITER` = `30`.
- B^φ se obtiene como la submatriz `B_std[:, 2*i*:2*i*+2]` correspondiente al nodo solitario (consistente con $\varphi = N_{\{i*\}}$).

- Activación: $\mathbf{n}$ apunta hacia el nodo solitario (convención fija); si el autovector de $\sigma_{\mathbf{I}}$ deja 2 nodos en lado positivo, se invierte el signo de $\mathbf{n}$.
- `_compute_ld` usa $|\cos(\theta - \alpha)|$ (valor absoluto): la longitud es positiva y simétrica bajo $\mathbf{n} \rightarrow -\mathbf{n}$ (test `ld_invariante_bajo_reflejo_de_n`).
- `compute_gauss_state` extendido: en estado agrietado añade clave `'discontinuity': {normal, tangent, centroid, solitary_node, l_d, jump, traction, damage}` para post-proceso VTK.
- Bulks aceptados en fase 1: `_ACCEPTED_BULKS = ('Elastic2D',)`. Restricción declarativa, ampliación futura es aditiva.

5.7 Diálogo

- **2026-05-18** · Spec pre-redactada por la IA como **orden de trabajo** sobre los Caps. 2, 5, 6 y 7 de Retama 2010. La cinemática KOS, la condensación estática y el $\mathbf{l}_d = (\mathbf{A}/h) \cdot \cos(\theta - \alpha)$ son traducción directa de la tesis; las decisiones arquitecturales del ADR 0010 (familia paralela, condensación local, irreversibilidad, activación al principio del paso) están fijadas. Pendiente de validación del usuario.
- **2026-05-18** · Usuario delega las decisiones a la IA. Cierre con justificación arquitectural:
- **A — Bulk restringido a Elastic2D en fase 1.** Adoptado. Razón: la *discrete approach* del Cap. 3-7 de la tesis presupone bulk elástico (la disipación se localiza en Γ_d); mezclar plasticidad continua del bulk con cohesivo de Γ_d es una formulación distinta no cubierta por el ADR 0010. Validación cruzada en `__init__` con mensaje listando bulks aceptados. Apertura futura a otros materiales es aditiva.
- **B — Hook prepare_step(U_committed) en Element base, no-op por defecto.** Adoptado. Lo prescribe el ADR 0010 §5 explícitamente; aditivo (no rompe llamadores) y testeable directamente. Alternativa de meter activación en `compute_element_state` con guarda `_step_id` descartada por ser frágil y mezclar conceptos (preparación vs cálculo).
- **C — INTERNAL_DOF_NAMES no se añade al contrato declarativo todavía.** Resistir abstracción especulativa (Reglas.md §1: justificar con dos casos reales). Cuando entre el segundo elemento enriquecido (`Quad4_Embedded` o `Tet4_Embedded`), se centraliza la convención.
- **D — Post-proceso vía compute_gauss_state(U) extendido.** Adoptado. En estado intacto se comporta como `Tri3`; en estado agrietado añade la clave `'discontinuity': {'normal', 'jump', 'traction', 'damage', 'l_d'}`. Consistente con el patrón actual del `Tri3` y aditivo para `VtkExporter`. Un método separado `crack_state()` duplicaría el patrón sin beneficio.
- **E — Constructor __init__(element_id, nodes, material, cohesive_material, thickness=1.0).** Mantengo el nombre `material` para el bulk (consistente con `Element.__init__`, que ya lo recibe así); `cohesive_material` es el parámetro nuevo. El YAML mapea `bulk_material: <name> → material=...` y `cohesive_material: <name> → cohesive_material=...`
- **F — Tolerancia 1e-14 (bit-exact) en estado_intacto_reproduce_Tri3.** Si en la implementación emerge un desliz numérico documentado (por canceladas algebraicas tipo $\mathbf{G} \cdot \mathbf{T} = \mathbf{I}$), se relaja con razón explícita en el Diálogo. Empezar estricta es mejor que pre-laxar.

- **2026-05-18** · Status → en draft hasta que el código + tests cierren; se promoverá a **validated** cuando los **acceptance** pasen. La IA arranca implementación.

Capítulo 6

Elementos 3D — Sólidos

6.1 Tet4

Elemento sólido 3D de primer orden, espejo natural de Tri3. Pieza de la Etapa 7. Convención Voigt 6D y numeración de caras por ADR 0012.

6.2 Especificación física

6.2.1 0. Descripción general

Elemento sólido tridimensional de primer orden, tetraedro de cuatro nodos. Aproximación **lineal** C^0 del campo de desplazamientos en coordenadas baricéntricas. Análogo 3D del Tri3: la matriz **B** es **constante sobre el elemento**, por lo que ε y σ son uniformes dentro de cada Tet4 (*constant strain tetrahedron* en la literatura). Régimen de pequeños desplazamientos y deformaciones.

6.2.2 1. Cinemática de desplazamientos

$$u_x = \sum_{i=1}^4 N_i u_x^{(i)}, \quad u_y = \sum N_i u_y^{(i)}, \quad u_z = \sum N_i u_z^{(i)}.$$

6.2.3 2. Cinemática de deformaciones

Notación Voigt 6D del proyecto (ADR 0012):

$$\varepsilon = [\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{zz}, \gamma_{xy}, \gamma_{yz}, \gamma_{xz}]^\top$$

Por la linealidad de N_i , las derivadas $\partial N_i / \partial \mathbf{x}$ son constantes y ε es uniforme en el elemento. Idéntico patrón conceptual al Tri3 en 2D.

6.2.4 3. Ecuación constitutiva

Delegada al material 3D (STRAIN_DIM = 6). En esta etapa solo Elastic3D está disponible.

6.2.5 4. Equilibrio — forma fuerte

Idéntica a la del Hex8. El principio variacional no depende de la forma del elemento.

6.2.6 5. Equilibrio — forma débil

Idéntica a la del Hex8.

6.3 Formulación numérica (FEM)

6.3.1 6. Discretización

Cuatro nodos en orden estándar de tetraedro VTK. En el tetraedro de referencia con vértices:

Nodo local	(ξ_i, η_i, ζ_i)
0	$(0, 0, 0)$
1	$(1, 0, 0)$
2	$(0, 1, 0)$
3	$(0, 0, 1)$

El orden de nodos asegura que el vector $(\mathbf{x}_1 - \mathbf{x}_0) \times (\mathbf{x}_2 - \mathbf{x}_0)$ apunte hacia $\mathbf{x}_3$ (volumen orientado positivo). Mapeo isoparamétrico desde el tetraedro de referencia; jacobiano $\mathbf{J}$ constante en todo el elemento; $\det \mathbf{J} = 6 V_e$ con V_e el volumen del tetraedro. El código aborta con `ValueError` si $\det \mathbf{J} \leq \text{tol}$ (nodos coplanares o en orden invertido).

6.3.2 7. Funciones de forma

Coordenadas baricéntricas / tetraédricas:

$$N_1 = 1 - \xi - \eta - \zeta, \quad N_2 = \xi, \quad N_3 = \eta, \quad N_4 = \zeta.$$

Sus derivadas en coordenadas naturales son constantes:

$$\partial N_i / \partial \xi = (-1, 1, 0, 0), \quad \partial N_i / \partial \eta = (-1, 0, 1, 0), \quad \partial N_i / \partial \zeta = (-1, 0, 0, 1).$$

6.3.3 8. Matriz deformación-desplazamiento

$\mathbf{B}$ tiene tamaño 6×12 y, una vez transformadas las derivadas a globales vía $\mathbf{J}^{-1}$, sus entradas son **constantes** sobre el elemento:

$$\mathbf{B}_i = \begin{bmatrix} \partial N_i / \partial x & 0 & 0 \\ 0 & \partial N_i / \partial y & 0 \\ 0 & 0 & \partial N_i / \partial z \\ \partial N_i / \partial y & \partial N_i / \partial x & 0 \\ 0 & \partial N_i / \partial z & \partial N_i / \partial y \\ \partial N_i / \partial z & 0 & \partial N_i / \partial x \end{bmatrix}, \quad \boldsymbol{\varepsilon} = \mathbf{B} \mathbf{u}_e.$$

Mismo patrón filas/columnas que Hex8 (ver §8 de su spec).

6.3.4 9. Rigidez elemental

$$\mathbf{K}_e = \mathbf{B}^\top \mathbf{D} \mathbf{B} V_e,$$

con $V_e = \frac{1}{6} \det \mathbf{J}$ el volumen del tetraedro. La integración es **exacta con un único punto central** porque $\mathbf{B}$ y $\boldsymbol{\sigma}$ son constantes.

6.3.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \mathbf{B}^\top \boldsymbol{\sigma} V_e.$$

6.3.6 11. Cuadratura

Un punto central (baricentro del tetraedro) con peso $w = 1/6$ sobre el tetraedro de referencia. Es exacto para formas constantes de $\mathbf{B}$ y $\boldsymbol{\sigma}$.

6.3.7 12. Cargas distribuidas consistentes

Fuerza de cuerpo $\mathbf{b}$. Con N_i lineales y $\mathbf{b}$ uniforme la integral es exacta:

$$\mathbf{f}_e^\phi = \int_{\Omega_e} \mathbf{N}^\top \mathbf{b} dV \implies \text{cada nodo recibe } \frac{1}{4} \mathbf{b} V_e.$$

Tracción de superficie $\bar{\mathbf{t}}$ sobre una cara triangular. Caras numeradas con normal saliente (ADR 0012):

Cara	Nodos	Nodo opuesto
0	(1, 2, 3)	0
1	(0, 3, 2)	1
2	(0, 1, 3)	2
3	(0, 2, 1)	3

Sobre una cara triangular con tracción constante el reparto es exacto:

$$\mathbf{f}_e^t = \frac{1}{3} \bar{\mathbf{t}} A_{\text{cara}} \text{ por nodo del triángulo, } \quad 0 \text{ al nodo opuesto.}$$

Cuadratura 1 punto centroide de la cara (suficiente por linealidad de las funciones de forma sobre la cara). $\bar{\mathbf{t}}$ se especifica en coordenadas globales (t_x, t_y, t_z) . Tracción variable o presión normal no soportadas en este paso.

6.3.8 13. Salida por punto de Gauss

`compute_gauss_state(U)` devuelve la misma estructura que `Hex8` pero con un único punto (centroide). Para `Tet4` $\boldsymbol{\varepsilon}$ y $\boldsymbol{\sigma}$ son constantes sobre el elemento, así que el dato del punto coincide con el promedio del elemento. Por ADR 0012, `internal_forces` devuelve `None`.

6.4 Limitación crítica — *shear locking*

El `Tet4` es notoriamente rígido en flexión y dominado por cortante, exactamente como su análogo `Tri3` en 2D: cuatro DOFs de desplazamiento por dirección no bastan para representar el modo de flexión puro de una viga discretizada en tetraedros, por lo que el elemento responde con cortante espurio y subestima sistemáticamente la deflexión. La severidad crece con la relación L/h del problema.

Recomendación operativa: usar `Hex8` por defecto en mallas hexaédricas o mixtas; reservar `Tet4` para zonas de transición de malla donde la generación automática de hexaedros falla y se requiere malla tetraédrica de baja regularidad. Para problemas dominados por flexión se necesitan tetraedros cuadráticos `Tet10` (sub-etapa posterior). Locking volumétrico con $\nu \rightarrow 0,5$ es **aún peor** que en `Hex8` por la misma razón: el espacio lineal es muy pobre.

6.5 Contrato de implementación

```

name: Tet4
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
  n_nodes: 4
  strain_dim: 6          # Voigt 3D [_xx, _yy, _zz, _xy, _yz, _xz]
  n_integration_points: 1

parameters: []

material_contract:
  signature: "compute_state(, state) -> (, C_tangent, state')"
  strain_kind: "Voigt 3D [_xx, _yy, _zz, _xy, _yz, _xz] con _ij = 2·_ij"

conventions:
  sign: " > 0  tracción (Reglas §5)"
  voigt: "[xx, yy, zz, xy, yz, xz] con _ij = 2·_ij (ADR 0012)"
  node_orientation: "convención VTK_TETRA; nodos 0-2 base con normal hacia 3 (volumen orientado positivo); asegura det J > 0"
  face_numbering: "ADR 0012 - 4 caras con normal saliente; cara i opuesta al nodo i"

validity:
  - "pequeños desplazamientos y deformaciones"
  - "campos de tensión que varían suavemente sobre el elemento"
  - "compatible con materiales con STRAIN_DIM = 6 (en esta etapa: Elastic3D)"

out_of_scope:
  - "flexión dominante (shear locking severo)"
  - "casi-incompresibilidad (→ 0.5 - locking volumétrico aún peor que Hex8)"
  - "elementos de alto orden (Tet10 - sub-etapa posterior)"
  - "grandes deformaciones"
  - "internal_forces (ADR 0012: sólidos exponen compute_gauss_state)"

acceptance:
  verification:
    - name: patch_test_3d
      setup: "malla tetraédrica con al menos un nodo interior (e.g. división de un cubo en 5 tetraedros con un nodo central libre); campo lineal u_i = a_ij·x_j impuesto en el contorno"
      expect: "nodo interior adopta el campo lineal exacto y constante e igual a 0.5·(a_ij + a_ji) en cada elemento; uniforme"
      tol_rel: 1.0e-10
    - name: modos_de_cuerpo_rigido
      setup: "evaluar K_e sobre un Tet4 cualquiera y proyectar sobre los 6 modos de cuerpo rígido"
      expect: "K_e · u_rigid = 0 en cada uno de los 6 modos"
      tol_abs: 1.0e-9
    - name: simetria_K_e
      setup: "evaluar K_e con Elastic3D"
      expect: "K_e == K_e.T exacto"
      tol_abs: 1.0e-12
  specific:
    - name: parche_deformacion_constante
      setup: "malla tetraédrica regular con campo lineal y Elastic3D"

```

```

expect: " constante (CST 3D reproduce campos lineales por construcción)"
tol_rel: 1.0e-12
- name: jacobiano_degenerado_abortado
  setup: "cuatro nodos coplanares"
  expect: "ValueError en _compute_kinematics_tet4"
- name: body_load_uniforme
  setup: "Tet4 con b uniforme"
  expect: "cada nodo recibe (1/4)·b·V_e;  $\Sigma = b \cdot V_e$ "
  tol_rel: 1.0e-12
- name: traccion_uniforme_cara
  setup: "Tet4 con tracción constante en una cara"
  expect: " $\Sigma_f = -t \cdot A_{cara}$ ; reparto  $A_{cara}/3 \cdot t$  a cada uno de los 3
           nodos de la cara, 0 al nodo opuesto"
  tol_rel: 1.0e-12
- name: cubo_lame_3d_malla_tetraedrica
  setup: "cubo unitario discretizado con tetraedros (e.g. 5 o 6 Tet4
           por cubo); tracción uniaxial  $_{xx} = p$  en cara +"
  expect: " $u_x(L) = p \cdot L/E$  con error decreciente al refinar; orden
           de convergencia  $O(h)$  en norma energética"
  tol_rel: "documentar convergencia, sin tolerancia absoluta"

references:
- "Cook R.D., Malkus D.S., Plesha M.E., Witt R.J. (2002). Concepts and Applications
  of FEA. §6.8 (CST 3D)."
```

```

- "Zienkiewicz O.C., The Finite Element Method, vol. 1, §6.7 (limitations of linear
  tets)."
```

```

- "Bathe K.J. (2014). Finite Element Procedures. §6.6.2."
```

```

- "ADR 0012 - Sólidos 3D: convención Voigt 6D y caras."
```

6.6 Implementación

- **Archivo:** solidum/elements/solid_3d/tet4.py.
- **Clase:** Tet4 (registrada vía `@ElementRegistry.register`).
- **Función núcleo:** `_compute_kinematics_tet4(coords)` con `@njit` en solidum/elements/solid_3d/_shared.py. Reutiliza `_compute_integrands_3d` (también en `_shared`, compartido con Hex8) con peso $1/6$ (volumen del tet de referencia).
- **Masa consistente analítica:** $M_{ij} = \rho \cdot V_e \cdot (1 + \delta_{ij})/20$ (matriz escalar 4×4 , expandida 3D vía `_expand_scalar_mass_3d`). Evita cuadratura insuficiente en el centroide.
- **Tests:**
 - tests/test_solid_3d.py — clase `TestTet4Element` (10 tests: dimensiones/DOFs, simetría exacta de K (no numérica, por fórmula cerrada), volumen, patch tracción uniaxial, jacobiano degenerado abortado, body load, face traction (cara opuesta a nodo recibe cero, reparto $A/3$ a los 3 nodos restantes), masa consistente y lumped HRZ (reparto equitativo $\rho V/4$ por DOF)).
 - tests/test_rigid_body_modes.py — `TestRBM Tet4` (5 tests: traslación, 3 rotaciones, rank-deficiency = 6).
 - tests/validation/test_cube_lame_3d.py — `test_uniaxial_traction_tet4_mesh_exact` (cubo dividido en 5 tetraedros con tracción uniaxial; solución exacta nodal).

6.7 Diálogo

- **2026-05-19** · Spec inicial. Elemento espejo natural de **Tri3**; convención Voigt 6D y numeración de caras fijadas por ADR 0012. Sin elementos de alto orden en esta etapa. Shear locking y locking volumétrico declarados como limitaciones; **Hex8** es la opción preferente cuando la malla lo permite.

6.8 Hex8

Elemento sólido 3D de primer orden, espejo natural de Quad4. Pieza central de la Etapa 7. Convención Voigt 6D y caras fijadas por ADR 0012.

6.9 Especificación física

6.9.1 0. Descripción general

Elemento sólido tridimensional de primer orden, hexaedro de ocho nodos. Aproximación **trilineal** C^0 del campo de desplazamientos en coordenadas naturales $(\xi, \eta, \zeta) \in [-1, 1]^3$. Formulación isoparamétrica: la geometría y los desplazamientos comparten las mismas funciones de forma. Régimen de pequeños desplazamientos y deformaciones.

6.9.2 1. Cinemática de desplazamientos

$$u_x(\xi, \eta, \zeta) = \sum_{i=1}^8 N_i(\xi, \eta, \zeta) u_x^{(i)}, \quad u_y = \sum N_i u_y^{(i)}, \quad u_z = \sum N_i u_z^{(i)}.$$

6.9.3 2. Cinemática de deformaciones

Tensor infinitesimal en notación Voigt 6D del proyecto (ADR 0012):

$$\boldsymbol{\varepsilon} = [\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{zz}, \gamma_{xy}, \gamma_{yz}, \gamma_{xz}]^\top$$

con

$$\varepsilon_{xx} = \frac{\partial u_x}{\partial x}, \quad \varepsilon_{yy} = \frac{\partial u_y}{\partial y}, \quad \varepsilon_{zz} = \frac{\partial u_z}{\partial z},$$

$$\gamma_{xy} = \frac{\partial u_x}{\partial y} + \frac{\partial u_y}{\partial x}, \quad \gamma_{yz} = \frac{\partial u_y}{\partial z} + \frac{\partial u_z}{\partial y}, \quad \gamma_{xz} = \frac{\partial u_x}{\partial z} + \frac{\partial u_z}{\partial x}.$$

6.9.4 3. Ecuación constitutiva

Delegada al material 3D (STRAIN_DIM = 6). El elemento llama `material.compute_state( $\varepsilon$ , state) → ( $\sigma$ , C_tangent, state')` por punto de Gauss. En esta etapa solo Elastic3D está disponible; el contrato deja la puerta abierta a futuros VonMises3D, DruckerPrager3D, IsotropicDamage3D.

6.9.5 4. Equilibrio — forma fuerte

$$-\nabla \cdot \boldsymbol{\sigma} = \mathbf{b} \quad \text{en } \Omega, \quad \boldsymbol{\sigma} \cdot \mathbf{n} = \bar{\mathbf{t}} \quad \text{en } \partial\Omega_N, \quad \mathbf{u} = \bar{\mathbf{u}} \quad \text{en } \partial\Omega_D.$$

6.9.6 5. Equilibrio — forma débil

$$\int_{\Omega} \delta \boldsymbol{\varepsilon}^\top \boldsymbol{\sigma} dV = \int_{\Omega} \delta \mathbf{u}^\top \mathbf{b} dV + \int_{\partial\Omega_N} \delta \mathbf{u}^\top \bar{\mathbf{t}} dS, \quad \forall \delta \mathbf{u} \in V_0.$$

6.10 Formulación numérica (FEM)

6.10.1 6. Discretización

Ocho nodos en orden estándar de hexaedro VTK (compatible con la mayoría de pre-procesadores). En el cubo de referencia $[-1, 1]^3$:

Nodo local	(ξ_i, η_i, ζ_i)
0	$(-1, -1, -1)$
1	$(+1, -1, -1)$
2	$(+1, +1, -1)$
3	$(-1, +1, -1)$
4	$(-1, -1, +1)$
5	$(+1, -1, +1)$
6	$(+1, +1, +1)$
7	$(-1, +1, +1)$

Cara inferior $\zeta = -1$ recorrida (0, 1, 2, 3) antihorario visto desde dentro; cara superior $\zeta = +1$ recorrida (4, 5, 6, 7) antihorario visto desde fuera. Mapeo isoparamétrico desde el cubo de referencia:

$$x(\xi, \eta, \zeta) = \sum_{i=0}^7 N_i x^{(i)}, \quad y = \sum N_i y^{(i)}, \quad z = \sum N_i z^{(i)}.$$

Jacobiano $\mathbf{J} = \partial(x, y, z)/\partial(\xi, \eta, \zeta)$ es una matriz 3×3 con determinante $\det \mathbf{J}$. El código aborta con `ValueError` si $\det \mathbf{J} \leq \text{tol}$ (elemento degenerado o nodos en orden invertido).

6.10.2 7. Funciones de forma

Producto tensorial trilineal:

$$N_i(\xi, \eta, \zeta) = \frac{1}{8}(1 + \xi_i \xi)(1 + \eta_i \eta)(1 + \zeta_i \zeta), \quad (\xi_i, \eta_i, \zeta_i) \in \{-1, +1\}^3.$$

6.10.3 8. Matriz deformación-desplazamiento

Las derivadas en globales se obtienen vía $\partial N_i / \partial \mathbf{x} = \mathbf{J}^{-1} \partial N_i / \partial \boldsymbol{\xi}$. La matriz $\mathbf{B}$ tiene tamaño 6×24 y se ensambla por nodo $i \in \{0, \dots, 7\}$:

$$\mathbf{B}_i = \begin{bmatrix} \partial N_i / \partial x & 0 & 0 \\ 0 & \partial N_i / \partial y & 0 \\ 0 & 0 & \partial N_i / \partial z \\ \partial N_i / \partial y & \partial N_i / \partial x & 0 \\ 0 & \partial N_i / \partial z & \partial N_i / \partial y \\ \partial N_i / \partial z & 0 & \partial N_i / \partial x \end{bmatrix}, \quad \boldsymbol{\varepsilon} = \mathbf{B} \mathbf{u}_e.$$

El orden de las filas reproduce exactamente el orden Voigt del proyecto (ADR 0012). El orden de las columnas por nodo es $[\mathbf{u}_x, \mathbf{u}_y, \mathbf{u}_z]$, agrupado por nodo i para $i \in \{0..7\}$.

6.10.4 9. Rigidez elemental

$$\mathbf{K}_e = \int_{\Omega} \mathbf{B}^T \mathbf{D} \mathbf{B} dV = \sum_{p=1}^{n_g} \mathbf{B}(\boldsymbol{\xi}_p)^T \mathbf{D}_p \mathbf{B}(\boldsymbol{\xi}_p) \det \mathbf{J}_p w_p.$$

A diferencia del 2D, no hay parámetro de espesor — el `.espesor` está incluido en el volumen integrado.

6.10.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \sum_{p=1}^{n_g} \mathbf{B}(\boldsymbol{\xi}_p)^T \boldsymbol{\sigma}_p \det \mathbf{J}_p w_p.$$

6.10.6 11. Cuadratura

Por defecto **Gauss–Legendre $2 \times 2 \times 2$** (ocho puntos): orden de exactitud 3 en cada dirección — integra exactamente $\mathbf{K}_e$ para Jacobiano constante (geometría de paralelepípedo) y captura el comportamiento dominante en geometrías irregulares moderadas. Soporta esquemas alternativos vía el parámetro `quadrature` desde `QuadratureRegistry` (e.g. `hex_3x3x3` para problemas con materiales no lineales severos).

Aviso sobre integración reducida: el esquema $1 \times 1 \times 1$ (un solo punto central) está disponible pero el código emite advertencia explícita; un único punto no detecta los modos de hourglass (energía nula con desplazamientos alternados) que en 3D son **doce** modos espurios (no cuatro como en 2D). No se implementa estabilización tipo Flanagan-Belytschko. Recomendación operativa: usar el default $2 \times 2 \times 2$ salvo necesidad explícita.

6.10.7 12. Cargas distribuidas consistentes

Fuerza de cuerpo $\mathbf{b}$ (e.g. peso propio $\mathbf{b} = (0, 0, -\rho g)$). Vector nodal equivalente:

$$\mathbf{f}_e^b = \int_{\Omega_e} \mathbf{N}^T \mathbf{b} dV = \sum_{p=1}^{n_g} \mathbf{N}(\boldsymbol{\xi}_p)^T \mathbf{b} \det \mathbf{J}_p w_p,$$

donde $\mathbf{N}$ es la matriz 3×24 de funciones de forma. Se reutiliza la cuadratura del elemento (Gauss $2 \times 2 \times 2$ por defecto). Exacto para $\mathbf{b}$ uniforme y geometría de paralelepípedo.

Tracción de superficie $\bar{\mathbf{t}}$ sobre una cara $\Gamma_e \subset \partial\Omega_N$. Caras numeradas con normal saliente (ADR 0012):

Cara	Nodos	Normal saliente
0	(0, 3, 2, 1)	$-\zeta$ (inferior)
1	(4, 5, 6, 7)	$+\zeta$ (superior)
2	(0, 1, 5, 4)	$-\eta$ (frontal)
3	(1, 2, 6, 5)	$+\xi$ (derecha)
4	(2, 3, 7, 6)	$+\eta$ (trasera)
5	(3, 0, 4, 7)	$-\xi$ (izquierda)

$$\mathbf{f}_e^t = \int_{\Gamma_e} \mathbf{N}^\top \bar{\mathbf{t}} dS.$$

Cuadratura 2D sobre la cara: Gauss 2×2 (cuatro puntos por cara). Exacto para tracción constante sobre cara plana. $\bar{\mathbf{t}}$ se especifica en **coordenadas globales** (t_x, t_y, t_z); presiones normales se obtienen multiplicando previamente por la normal exterior de la cara. Tracción variable a lo largo de la cara no soportada en este paso.

6.10.8 13. Salida por punto de Gauss

`compute_gauss_state(U)` devuelve un dict con ε y σ en cada uno de los n_g puntos de Gauss del elemento (8 por defecto), junto con coordenadas naturales y globales. Habilita post-proceso fino (mapas no promediados, suavizado nodal, recovery superconvergente futuro). Por ADR 0012, `internal_forces` devuelve `None` (sólidos no tienen fuerzas internas seccionales discretas — son su campo $\sigma(x)$ continuo).

6.11 Limitaciones declaradas

6.11.1 Locking volumétrico con $\nu \rightarrow 0,5$

Hex8 con integración completa $2 \times 2 \times 2$ y material casi-incompresible (`Elastic3D` con $\nu \rightarrow 0,5$, o un futuro `VonMises3D` en régimen plástico desarrollado) sufre **locking volumétrico**: la rigidez crece artificialmente porque el espacio de funciones trilineal no contiene desplazamientos puramente isocóricos suficientes. El elemento se queda atrapado en respuestas erróneamente rígidas.

Tratamiento en este proyecto: declarar limitación, **blindar con test** (`test_volumetric_locking_3d.py`, espejo del análogo 2D) que documente cuantitativamente el colapso, **no implementar mitigación** (B-bar, F-bar, mixed u-p). Política idéntica al `Quad4` (ver `STATUS.md` "Limitaciones declaradas").

6.11.2 Hourglass con integración reducida

Hex8 integrado con un solo punto central tiene **12 modos de hourglass** (energía nula con desplazamientos alternados). Reducción disponible vía `quadrature` con warning; estabilización Flanagan-Belytschko no implementada.

6.11.3 Mass lumping en orientación arbitraria

`compute_mass_matrix(lumping="lumped")` usa HRZ canónico (`solidum/math/mass_lumping.py::lump_hrz`) Diagonal en ejes locales del elemento. En sólidos 3D la matriz lumped traslacional es estrictamente diagonal porque $\mathbf{m}_t \cdot \mathbf{I}_3$ es invariante bajo $\text{SO}(3)$ — sin el caveat de `Frame3D` que sí aparece en su bloque rotacional (no hay rotaciones nodales en sólidos).

6.12 Contrato de implementación

```
name: Hex8
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
  n_nodes: 8
```

```

    strain_dim: 6                # Voigt 3D [_xx, _yy, _zz, _xy, _yz, _xz]
    n_integration_points: 8      # default Gauss 2x2x2

parameters:
- { name: quadrature, type: str, required: false, default: "hex_2x2x2",
    desc: "Regla de cuadratura desde QuadratureRegistry (hex_2x2x2 default, hex_3x3x3
        alternativa)" }

material_contract:
    signature: "compute_state(, state) -> (, C_tangent, state')"
    strain_kind: "Voigt 3D [_xx, _yy, _zz, _xy, _yz, _xz] con _ij = 2·_ij"

conventions:
    sign: " > 0  tracción (Reglas §5)"
    voigt: "[xx, yy, zz, xy, yz, xz] con _ij = 2·_ij (ADR 0012)"
    node_orientation: "convención VTK_HEXAHEDRON; nodos 0-3 cara inferior antihorario
        visto desde dentro, 4-7 cara superior antihorario visto desde fuera; asegura det
        J > 0"
    face_numbering: "ADR 0012 - 6 caras con normal saliente: 0 - (), 1 (+), 2 - (), 3 (+)
        , 4 (+), 5 - ()"

validity:
- "pequeños desplazamientos y deformaciones"
- "geometría sin distorsión severa (det J > tol en todos los puntos de Gauss)"
- "compatible con materiales con STRAIN_DIM = 6 (en esta etapa: Elastic3D)"

out_of_scope:
- "grandes deformaciones, formulaciones corotacionales o lagrangianas totales"
- "estabilización contra hourglass para integración reducida"
- "elementos de alto orden (Hex20, Hex27 - sub-etapa posterior)"
- "tracción variable sobre cara o presión normal"
- "internal_forces (ADR 0012: sólidos exponen compute_gauss_state)"

acceptance:
    verification:
- name: patch_test_3d
    setup: "8 Hex8 alrededor de un nodo interior (cubo 2x2x2 partido en
        ocho octantes) con campo lineal u_i = a_ij·x_j impuesto en
        los 26 nodos del contorno; nodo interior libre"
    expect: "nodo interior adopta el campo lineal exacto y constante e
        igual a 0.5·(a_ij + a_ji) en todos los puntos de Gauss;
        uniforme"
    tol_rel: 1.0e-10
- name: modos_de_cuerpo_rigido
    setup: "evaluar K_e sobre un Hex8 cualquiera y proyectar sobre los 6
        modos de cuerpo rígido (3 traslaciones + 3 rotaciones)"
    expect: "K_e · u_rigid  0 en cada uno de los 6 modos"
    tol_abs: 1.0e-9
- name: simetria_K_e
    setup: "evaluar K_e con Elastic3D"
    expect: "K_e == K_e.T exacto"
    tol_abs: 1.0e-12
    specific:
- name: cubo_lame_3d
    setup: "cubo unitario con Elastic3D; tracción uniaxial _xx = p en
        cara +; cara opuesta restringida en ux; restricciones
        mínimas para evitar movimiento rígido"
    expect: "u_x(L, y, z) = p·L/E; u_y = -·p·y/E; u_z = -·p·z/E
        exactos en todos los nodos"

```

```

tol_rel: 1.0e-10
- name: traccion_uniaxial_sobre_cara
  setup: "Hex8 con tracción constante (p, 0, 0) sobre cara 3 (+)"
  expect: "Σf_x = p·A_cara, Σf_y = Σf_z = 0; reparto p·A_cara/4 a cada
          nodo de la cara para Hex8 regular"
tol_rel: 1.0e-12
- name: body_load_uniforme
  setup: "Hex8 regular y distorsionado con b uniforme"
  expect: "Σf = b·V_e (invariante de geometría); por simetría en Hex8
          regular cada nodo recibe b·V_e / 8"
tol_rel: 1.0e-10
- name: jacobiano_degenerado_abortado
  setup: "Hex8 con cuatro nodos coplanares de la cara superior sobre
          la inferior (det J  0)"
  expect: "ValueError en _compute_kinematics"
- name: locking_volumetrico_documentado
  setup: "viga esbelta empotrada-libre con Hex8 plane strain forzado
          (cara z restringida); {0.3, 0.49, 0.4999}"
  expect: "deflexión vs Frame3D decrece monotonamente con → 0.5
          (locking documentado, no mitigado)"
tol_rel: "documentar valores, sin tolerancia de aceptación"

references:
- "Bathe K.J. (2014). Finite Element Procedures. §6.6 (sólidos 3D isoparamétricos)."
- "Cook R.D., Malkus D.S., Plesha M.E., Witt R.J. (2002). Concepts and Applications
  of FEA. §6.8."
- "Zienkiewicz O.C., Taylor R.L. (2005). The Finite Element Method, vol. 1, §6.7."
- "ADR 0012 - Sólidos 3D: convención Voigt 6D y caras."

```

6.13 Implementación

- **Archivo:** solidum/elements/solid_3d/hex8.py.
- **Clase:** Hex8 (registrada vía `@ElementRegistry.register`).
- **Funciones núcleo:** `_compute_kinematics_hex8(xi, eta, zeta, coords)`, `_det_jacobian_hex8(...)` y `_shape_functions_hex8(...)` con `@njit` (Numba) en `solidum/elements/solid_3d/_shared.py`. Integrandos comunes 3D (`_compute_integrands_3d`) y expansor masa traslacional (`_expand_scalar_mass_3d`) también en `_shared`.
- **Tests:**
- `tests/test_solid_3d.py` — clase `TestHex8Element` (10 tests: dimensiones/DOFs, simetría K, patch tracción uniaxial, jacobiano negativo abortado, body load, face traction (balance + nodos activos + índice fuera de rango), masa consistente y lumped HRZ, gauss state).
- `tests/test_rigid_body_modes.py` — `TestRBMHex8` (5 tests: traslación, 3 rotaciones independientes en (x, y, z), rank-deficiency = 6 modos rígidos).
- `tests/validation/test_cube_lame_3d.py` — `test_uniaxial_traction_hex8_exact`, `test_hydrostatic_com` (solución exacta a precisión máquina).
- `tests/validation/test_macneal_beam_3d.py` — shear locking documentado en malla coarse + convergencia h monótona.
- `tests/test_volumetric_locking_3d.py` — locking volumétrico declarado y blindado por test.

6.14 Diálogo

- **2026-05-19** · Spec inicial. Elemento espejo natural de `Quad4`; convención Voigt 6D y numeración de caras fijadas por ADR 0012. Sin variantes corotacionales ni de alto orden en esta etapa. Locking volumétrico declarado como limitación con test que lo blindo; sin mitigación implementada (política idéntica al 2D).

6.15 Tet10

Elemento sólido 3D de orden cuadrático sobre el simplex tetraédrico. Análogo 3D del ‘Tri6 2D’. Reproduce campos cuadráticos completos en (ξ, η, ζ) . Mitiga drásticamente el shear locking y el locking volumétrico del Tet4 (CST 3D). Sub-fase 3 de A.ter; subclase de _HigherOrderSolid.

6.16 Especificación física

6.16.1 0. Descripción general

Tetraedro tridimensional de segundo orden con **10 nodos**: 4 vértices + 6 medios de arista. Aproximación cuadrática C^0 en coordenadas baricéntricas L_1, L_2, L_3, L_4 con $\sum_i L_i = 1$. Formulación isoparamétrica. Régimen de pequeños desplazamientos y deformaciones.

6.16.2 1-5. Cinemática, ecuación constitutiva, equilibrio

Idénticas al ‘Hex20/Hex27’ — la diferencia con los hexaedros está exclusivamente en la geometría de referencia (simplex tetraédrico vs cubo).

6.17 Formulación numérica (FEM)

6.17.1 6. Discretización — convención VTK_QUADRATIC_TETRA

Los 10 nodos sobre el tetraedro de referencia con vértices $(0, 0, 0)$, $(1, 0, 0)$, $(0, 1, 0)$, $(0, 0, 1)$:

Nodo	Tipo	Coords naturales (ξ, η, ζ)
0	vértice (idéntico Tet4)	$(0, 0, 0)$
1	vértice (idéntico Tet4)	$(1, 0, 0)$
2	vértice (idéntico Tet4)	$(0, 1, 0)$
3	vértice (idéntico Tet4)	$(0, 0, 1)$
4	medio arista 0-1	$(0,5, 0, 0)$
5	medio arista 1-2	$(0,5, 0,5, 0)$
6	medio arista 2-0	$(0, 0,5, 0)$
7	medio arista 0-3	$(0, 0, 0,5)$
8	medio arista 1-3	$(0,5, 0, 0,5)$
9	medio arista 2-3	$(0, 0,5, 0,5)$

Mapeo isoparamétrico desde el tetraedro de referencia con $L_1 = 1 - \xi - \eta - \zeta$, $L_2 = \xi$, $L_3 = \eta$, $L_4 = \zeta$.

6.17.2 7. Funciones de forma

Vértices ($i \in \{1, 2, 3, 4\}$ — corresponde a los nodos locales $\{0, 1, 2, 3\}$):

$$N_i = L_i (2 L_i - 1).$$

Medios de arista entre vértices i, j :

$$N_{ij} = 4 L_i L_j.$$

Las seis aristas se etiquetan en el orden VTK_QUADRATIC_TETRA: (1, 2), (2, 3), (3, 1), (1, 4), (2, 4), (3, 4) — nodos locales (0, 1), (1, 2), (2, 0), (0, 3), (1, 3), (2, 3). Garantizan partición de la unidad $\sum N_i = 1$ y propiedad delta de Kronecker.

Espacio reproducido: el ‘Tet10’ reproduce **completamente** el espacio cuadrático en barycentric coords — todos los polinomios de grado total ≤ 2 en (ξ, η, ζ) . 10 monomios, idéntico al número de nodos.

6.17.3 8. Matriz B

Tamaño **6×30**, ensamblada con la misma plantilla 6×3 del ‘Hex8/Hex20/Hex27’ por nodo. Derivadas en coords naturales obtenidas con regla del producto: para vértices $\partial N_i / \partial x_k = (4L_i - 1) \partial L_i / \partial x_k$; para medios $\partial N_{ij} / \partial x_k = 4(L_j \partial L_i / \partial x_k + L_i \partial L_j / \partial x_k)$.

6.17.4 9-10. Rigidez y fuerzas internas

Implementadas en la base ‘_HigherOrderSolid3D’. Tamaño 30×30 para **K_e**.

6.17.5 11. Cuadratura

Por defecto **Stroud 4 puntos** (‘tet_4’): orden de exactitud 2. Para Tet10 con Jacobiano constante el integrando de **B^TDB** es a lo sumo de grado 2 (B es lineal en barycentric), así que la regla es **exacta** para K.

Para problemas con Jacobiano variable (geometría distorsionada, mid-edges fuera del centro real) el integrando puede subir de grado; en esos casos el usuario puede seleccionar **tet_15** (Keast 15 puntos, orden 5) vía el parámetro **quadrature**.

Cuadratura de masa: el ‘Tet10 declara **_MASS_QUADRATURE = "tet_15"** (Keast 15 puntos, orden 5) ****independientemente**** de la cuadratura de K. El integrando **@MINL38@@** es de grado 4 (cuadrático × cuadrático) y la regla de 4 puntos lo subintegra; **tet_15** lo integra exactamente. Esto garantiza que la masa total sea correcta y que análisis modal/transitorio con Tet10 no sufran errores de cuadratura.

6.17.6 12. Cargas distribuidas consistentes

Body load: $\int N^T b \, dV$ con la cuadratura del elemento. Reparto no uniforme entre vértices y medios; suma global **b·V_e** exacta para **b** uniforme y geometría sin distorsión severa.

Face traction: cada cara del ‘Tet10 tiene ****6 nodos**** (3 vértices + 3 medios de arista) y se aproxima por funciones cuadráticas **Tri6** sobre el triángulo de referencia **@MINL39@@-@MINL40@@-@MINL41@@**. Cuadratura **tri_3** (3 puntos, orden 2) — exacta para tracción uniforme sobre cara plana.

Cara	Vértices Tet10	Medios Tet10 (en orden Tri6)	Normal saliente
0	(1, 2, 3)	(5, 9, 8)	opuesta al nodo 0
1	(0, 3, 2)	(7, 9, 6)	opuesta al nodo 1
2	(0, 1, 3)	(4, 8, 7)	opuesta al nodo 2
3	(0, 2, 1)	(6, 5, 4)	opuesta al nodo 3

El orden dentro de cada cara reproduce la convención de ‘**_N_tri6** del paquete 2D: vértices (0, 1, 2) antihorarios, luego medios (0-1, 1-2, 2-0)’.

6.17.7 13. Salida por punto de Gauss

`compute_gauss_state(U)` con 4 puntos por defecto. Sin `internal_forces` (ADR 0012).

6.18 Limitaciones declaradas

6.18.1 Locking volumétrico atenuado pero presente

Tet10 con cuadratura ‘tet_4 y material casi-incompresible (Elastic3D con @@MINL42@@, o VonMises3D plástico desarrollado) sufre ****locking volumétrico**** mucho menor que el Tet4 (cuyo shear locking severo es la motivación principal del Tet10), pero todavía presente. Sin mitigación implementada. Blindado por test (mismo patrón que Hex20/Hex27‘).

6.18.2 Distorsión severa

Para Tet10 con mid-edges desplazados fuera del centro real de la arista (mid-edges curvos para representar geometría curva), el Jacobiano deja de ser constante y la cuadratura ‘tet_4 puede subintegrar K. Recomendación: usar tet_15‘ en mallas con curvatura severa.

6.19 Contrato de implementación

```
name: Tet10
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
  n_nodes: 10
  strain_dim: 6
  n_integration_points: 4      # default Stroud tet_4

parameters:
  - { name: quadrature, type: str, required: false, default: "tet_4",
      desc: "Cuadratura tetraédrica desde QuadratureRegistry. Default
            tet_4 (Stroud 4 puntos, orden 2, suficiente para K con J
            constante). Alternativa tet_15 (Keast 15 puntos, orden 5)
            para geometrías distorsionadas." }

material_contract:
  signature: "compute_state(, state) -> (, C_tangent, state')"
  strain_kind: "Voigt 3D [_xx, _yy, _zz, _xy, _yz, _xz] con _ij = 2·_ij"

conventions:
  sign: "> 0  tracción (Reglas §5)"
  voigt: "[xx, yy, zz, xy, yz, xz] con _ij = 2·_ij (ADR 0012)"
  node_orientation: "VTK_QUADRATIC_TETRA; 0-3 vértices (Tet4 con volumen
                    positivo), 4-9 medios de arista en orden (0-1, 1-2,
                    2-0, 0-3, 1-3, 2-3)"
  face_numbering: "ADR 0012 - 4 caras triangulares con normal saliente,
                  paritarias con Tet4 en los vértices; cada cara añade
                  3 medios de arista en orden Tri6"

validity:
  - "pequeños desplazamientos y deformaciones"
```

```

- "geometría sin distorsión severa (det J > tol en todos los Gauss)"
- "compatible con materiales con STRAIN_DIM = 6"

out_of_scope:
- "grandes deformaciones, formulaciones corotacionales"
- "internal_forces (ADR 0012)"
- "mid-edges severamente curvos (usar tet_15 si es necesario)"

acceptance:
  verification:
    - name: patch_test_lineal
      setup: "campo lineal  $u_i = a_{ij} \cdot x_j$  en los 10 nodos del contorno"
      expect: "constante = sym(a) en todos los Gauss"
      tol_rel: 1.0e-12
    - name: patch_test_cuadratico
      setup: "campo  $u_x = c \cdot x^2$  en los 10 nodos"
      expect: " $_{xx}(x) = 2c \cdot x$  exacto en todos los Gauss"
      tol_rel: 1.0e-12
    - name: modos_de_cuerpo_rigido
      setup: "K_e proyectado sobre 6 modos rígidos 3D"
      expect: "exactamente 6 autovalores nulos"
      tol_abs: 1.0e-9
    - name: simetria_K_e
      setup: "K_e con Elastic3D"
      expect: " $K_e == K_e.T$  exacto"
      tol_abs: 1.0e-12
  specific:
    - name: cubo_lame_3d_uniaxial
      setup: "malla 5 tetraedros del cubo unitario, refinados a Tet10,
              con tracción uniaxial; el campo analítico es lineal y
              Tet10 lo reproduce exacto"
      expect: " $u_x = p \cdot x/E$  etc. exacto a precisión máquina"
      tol_rel: 1.0e-10
    - name: macneal_3d_tet10_dominates_tet4
      setup: "cantilever de Tet10 con la misma malla que Tet4 sufre
              dramáticamente menos shear locking"
      expect: "Tet10 alcanza > 80%  $u_{EB}$  con ~12 tetraedros (Tet4 << 20%)"
    - name: body_load_uniforme
      setup: "Tet10 ref con  $b = (0, 0, -g)$ "
      expect: " $\Sigma f_z = -g \cdot V_e = -g/6$ "
      tol_rel: 1.0e-12
    - name: face_traction_balance
      setup: "tracción uniforme sobre una cara del Tet10"
      expect: " $\Sigma f = A_{face} \cdot t$  exacto; nodos fuera de la cara reciben 0"
      tol_rel: 1.0e-12
    - name: mass_consistent_total_with_tet15
      setup: "Tet10 cubo ref con  $\rho = 7850$ , masa consistente con tet_15"
      expect: " $\Sigma M = 3 \cdot \rho \cdot V_e = 3 \cdot \rho / 6$ "
      tol_rel: 1.0e-9
    - name: mass_lumped_HRZ_positivo
      setup: "masa lumped HRZ del Tet10"
      expect: "diagonal estricta, todas las entradas positivas,
              conservación de masa total"
      tol_abs: 1.0e-12

references:
- "Bathe K.J. (2014). Finite Element Procedures. §5.3.3 (tetraedro 3D)."
```

```

- "Cook R.D., Malkus D.S., Plesha M.E., Witt R.J. (2002). Concepts and
  Applications of FEA. §6.5."
```

- "Keast P. (1986). Moderate-degree tetrahedral quadrature formulas. CMAME 55(3), 339-348 (regla tet_15 de masa)."
- "Stroud A.H. (1971). Approximate calculation of multiple integrals (regla tet_4 de K)."
- "ADR 0012 - Sólidos 3D: convención Voigt 6D y caras."

6.20 Implementación

- **Archivo:** solidum/elements/solid_3d/tet10.py.
- **Clase:** Tet10 (registrada vía @ElementRegistry.register); subclase de _HigherOrderSolid3D.
- **Funciones núcleo:** _N_tet10(xi, eta, zeta), _dN_tet10(xi, eta, zeta) en solidum/elements/solid_3d/_shared.py. Implementadas en numpy puro vía coordenadas baricéntricas L_i .
- **Cuadraturas registradas:** tet_4 (Stroud, orden 2) y tet_15 (Keast, orden 5) en solidum/math/integration.py. El tet_4 cubre la integración de K; el tet_15 se usa exclusivamente para la masa consistente (declarada en Tet10._MASS_QUADRATURE).
- **Tests:**
 - tests/test_solid_3d_higher_order.py — TestTet10Element (patch lineal + cuadrático, simetría K, body load, face traction, masa consistente con tet_15 y lumped HRZ, gauss state).
 - tests/test_rigid_body_modes.py — TestRBM Tet10 (3 trans + 3 rot + rank-deficiency = 6).
 - tests/validation/test_cube_lame_3d.py — extensión con malla Tet10 del cubo unitario; uniaxial e hidrostática exactas a precisión máquina.

6.21 Diálogo

- **2026-05-27** · Spec inicial. Sub-fase 3 de A.ter — cierra el catálogo cuadrático 3D con la rama tetraédrica. Subclase de _HigherOrderSolid3D igual que Hex20/Hex27, pero con cara triangular (Tri6 shape functions importadas del paquete 2D, cuadratura tri_3) y cuadratura volumétrica tetraédrica nueva (tet_4 + tet_15).
- **2026-05-27** · Implementado y validado. Patch test lineal y cuadrático exactos a precisión máquina, 6 modos rígidos, body load suma exacta $\mathbf{b} \cdot \mathbf{V}_e$, face traction balance exacto sobre la cara triangular, masa consistente con tet_15 exacta a precisión máquina (conservación $3 \cdot \rho \cdot V$), lumped HRZ con todas las entradas positivas. Spec a status: validated.

6.22 Hex20

Elemento sólido 3D de orden cuadrático con interpolación serendípita.

Reproduce campos cuadráticos completos en 3D (todos los polinomios de grado total ≤ 2 más algunos de orden 3 sin los términos triquadráticos). Es al Hex8 lo que el Quad8 al Quad4: mucho más preciso en flexión y geometrías curvas con el mismo número de elementos. Primer elemento de la sub-etapa A.ter.

6.23 Especificación física

6.23.1 0. Descripción general

Hexaedro tridimensional de segundo orden con **20 nodos**: 8 vértices + 12 medios de arista, sin nodos de cara ni centroide. Aproximación serendípita C^0 en coordenadas naturales $(\xi, \eta, \zeta) \in [-1, 1]^3$. Formulación isoparamétrica: la geometría y los desplazamientos comparten las mismas funciones de forma. Régimen de pequeños desplazamientos y deformaciones.

6.23.2 1. Cinemática de desplazamientos

$$u_x(\xi, \eta, \zeta) = \sum_{i=0}^{19} N_i(\xi, \eta, \zeta) u_x^{(i)}, \quad u_y = \sum N_i u_y^{(i)}, \quad u_z = \sum N_i u_z^{(i)}.$$

6.23.3 2. Cinemática de deformaciones

Tensor infinitesimal en notación Voigt 6D del proyecto (ADR 0012):

$$\boldsymbol{\varepsilon} = [\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{zz}, \gamma_{xy}, \gamma_{yz}, \gamma_{xz}]^\top$$

con $\gamma_{ij} = 2\varepsilon_{ij}$ *engineering*. Idéntica al Hex8.

6.23.4 3. Ecuación constitutiva

Delegada al material 3D (STRAIN_DIM = 6). El elemento llama `material.compute_state( $\varepsilon$ , state)  $\rightarrow$  ( $\sigma$ , C_tangent, state')` por punto de Gauss. Compatible con todo el catálogo 3D actual: Elastic3D, VonMises3D, DruckerPrager3D, IsotropicDamage3D.

6.23.5 4-5. Equilibrio

Forma fuerte y débil idénticas al Hex8 — la diferencia entre Hex8 y Hex20 está exclusivamente en el espacio de aproximación discreto.

6.24 Formulación numérica (FEM)

6.24.1 6. Discretización

Veinte nodos en orden VTK_QUADRATIC_HEXAHEDRON (estándar gmsh/VTK/Abaqus). En el cubo de referencia $[-1, 1]^3$:

Nodo local	(ξ_i, η_i, ζ_i)	Tipo
0	$(-1, -1, -1)$	vértice
1	$(+1, -1, -1)$	vértice
2	$(+1, +1, -1)$	vértice
3	$(-1, +1, -1)$	vértice
4	$(-1, -1, +1)$	vértice
5	$(+1, -1, +1)$	vértice
6	$(+1, +1, +1)$	vértice
7	$(-1, +1, +1)$	vértice
8	$(0, -1, -1)$	medio arista 0-1 (eje ξ , cara inferior)
9	$(+1, 0, -1)$	medio arista 1-2 (eje η , cara inferior)
10	$(0, +1, -1)$	medio arista 2-3 (eje ξ , cara inferior)
11	$(-1, 0, -1)$	medio arista 3-0 (eje η , cara inferior)
12	$(0, -1, +1)$	medio arista 4-5 (eje ξ , cara superior)
13	$(+1, 0, +1)$	medio arista 5-6 (eje η , cara superior)
14	$(0, +1, +1)$	medio arista 6-7 (eje ξ , cara superior)
15	$(-1, 0, +1)$	medio arista 7-4 (eje η , cara superior)
16	$(-1, -1, 0)$	medio arista 0-4 (eje ζ , vertical)
17	$(+1, -1, 0)$	medio arista 1-5 (eje ζ , vertical)
18	$(+1, +1, 0)$	medio arista 2-6 (eje ζ , vertical)
19	$(-1, +1, 0)$	medio arista 3-7 (eje ζ , vertical)

Mapecto isoparamétrico desde el cubo de referencia:

$$x(\xi, \eta, \zeta) = \sum_{i=0}^{19} N_i x^{(i)}, \quad y = \sum N_i y^{(i)}, \quad z = \sum N_i z^{(i)}.$$

Jacobiano $\mathbf{J} = \partial(x, y, z)/\partial(\xi, \eta, \zeta)$ es una matriz 3×3 con determinante $\det \mathbf{J}$. El código aborta con **ValueError** si $\det \mathbf{J} \leq \text{tol}$ en cualquier punto de Gauss (elemento severamente distorsionado o nodos en orden invertido).

6.24.2 7. Funciones de forma serendípitas

Vértices ($i \in \{0, \dots, 7\}$, $(\xi_i, \eta_i, \zeta_i) \in \{\pm 1\}^3$):

$$N_i = \frac{1}{8}(1 + \xi_i \xi)(1 + \eta_i \eta)(1 + \zeta_i \zeta)(\xi_i \xi + \eta_i \eta + \zeta_i \zeta - 2).$$

El multiplicador $(\xi_i \xi + \eta_i \eta + \zeta_i \zeta - 2)$ vale 1 en el propio vértice y 0 en los demás vértices y en todos los medios de arista — garantía δ_{ij} del esquema.

Medios de arista paralelos a ξ (nodos 8, 10, 12, 14; $\xi_i = 0$, $\eta_i, \zeta_i \in \{\pm 1\}$):

$$N_i = \frac{1}{4}(1 - \xi^2)(1 + \eta_i \eta)(1 + \zeta_i \zeta).$$

Medios de arista paralelos a η (nodos 9, 11, 13, 15; $\eta_i = 0$):

$$N_i = \frac{1}{4}(1 + \xi_i \xi)(1 - \eta^2)(1 + \zeta_i \zeta).$$

Medios de arista paralelos a ζ (nodos 16, 17, 18, 19; $\zeta_i = 0$):

$$N_i = \frac{1}{4}(1 + \xi_i \xi)(1 + \eta_i \eta)(1 - \zeta^2).$$

Espacio reproducido: $\{1, \xi, \eta, \zeta, \xi^2, \eta^2, \zeta^2, \xi\eta, \eta\zeta, \xi\zeta, \xi^2\eta, \xi\eta^2, \eta^2\zeta, \eta\zeta^2, \xi^2\zeta, \xi\zeta^2, \xi\eta\zeta, \xi^2\eta\zeta, \xi\eta^2\zeta, \xi\eta\zeta^2\}$
— 20 monomios, omitiendo los puramente cúbicos (ξ^3 , etc.) y el término $\xi^2\eta^2\zeta^2$.

6.24.3 8. Matriz deformación-desplazamiento

Las derivadas en globales se obtienen vía $\partial N_i / \partial \mathbf{x} = \mathbf{J}^{-1} \partial N_i / \partial \boldsymbol{\xi}$. La matriz $\mathbf{B}$ tiene tamaño 6×60 y se ensambla por nodo $i \in \{0, \dots, 19\}$ con la misma plantilla 6×3 del **Hex8**:

$$\mathbf{B}_i = \begin{bmatrix} \partial N_i / \partial x & 0 & 0 \\ 0 & \partial N_i / \partial y & 0 \\ 0 & 0 & \partial N_i / \partial z \\ \partial N_i / \partial y & \partial N_i / \partial x & 0 \\ 0 & \partial N_i / \partial z & \partial N_i / \partial y \\ \partial N_i / \partial z & 0 & \partial N_i / \partial x \end{bmatrix}, \quad \boldsymbol{\varepsilon} = \mathbf{B} \mathbf{u}_e.$$

El orden de las filas reproduce exactamente el orden Voigt del proyecto (ADR 0012). El orden de las columnas por nodo es **[ux, uy, uz]**, agrupado por nodo i para $i \in \{0 \dots 19\}$.

Derivadas explícitas (cerradas vía regla del producto):

Vértices ($i \in \{0, \dots, 7\}$, con $u = \xi_i \xi$, $v = \eta_i \eta$, $w = \zeta_i \zeta$):

$$\frac{\partial N_i}{\partial \xi} = \frac{\xi_i}{8}(1 + v)(1 + w)(2u + v + w - 1),$$

$$\frac{\partial N_i}{\partial \eta} = \frac{\eta_i}{8}(1 + u)(1 + w)(u + 2v + w - 1),$$

$$\frac{\partial N_i}{\partial \zeta} = \frac{\zeta_i}{8}(1 + u)(1 + v)(u + v + 2w - 1).$$

Medio paralelo a ξ (nodos 8, 10, 12, 14):

$$\frac{\partial N_i}{\partial \xi} = -\frac{\xi}{2}(1 + \eta_i \eta)(1 + \zeta_i \zeta), \quad \frac{\partial N_i}{\partial \eta} = \frac{\eta_i}{4}(1 - \xi^2)(1 + \zeta_i \zeta), \quad \frac{\partial N_i}{\partial \zeta} = \frac{\zeta_i}{4}(1 - \xi^2)(1 + \eta_i \eta).$$

Medio paralelo a η (nodos 9, 11, 13, 15):

$$\frac{\partial N_i}{\partial \xi} = \frac{\xi_i}{4}(1 - \eta^2)(1 + \zeta_i \zeta), \quad \frac{\partial N_i}{\partial \eta} = -\frac{\eta}{2}(1 + \xi_i \xi)(1 + \zeta_i \zeta), \quad \frac{\partial N_i}{\partial \zeta} = \frac{\zeta_i}{4}(1 + \xi_i \xi)(1 - \eta^2).$$

Medio paralelo a ζ (nodos 16, 17, 18, 19):

$$\frac{\partial N_i}{\partial \xi} = \frac{\xi_i}{4}(1 + \eta_i \eta)(1 - \zeta^2), \quad \frac{\partial N_i}{\partial \eta} = \frac{\eta_i}{4}(1 + \xi_i \xi)(1 - \zeta^2), \quad \frac{\partial N_i}{\partial \zeta} = -\frac{\zeta}{2}(1 + \xi_i \xi)(1 + \eta_i \eta).$$

6.24.4 9. Rigidez elemental

$$\mathbf{K}_e = \int_{\Omega} \mathbf{B}^T \mathbf{D} \mathbf{B} dV = \sum_{p=1}^{n_g} \mathbf{B}(\xi_p)^T \mathbf{D}_p \mathbf{B}(\xi_p) \det \mathbf{J}_p w_p.$$

Sin parámetro de espesor — el volumen ya está incluido en $\det \mathbf{J}_p \cdot w_p$.

6.24.5 10. Fuerzas internas

$$\mathbf{F}_{\text{int}} = \sum_{p=1}^{n_g} \mathbf{B}(\xi_p)^T \boldsymbol{\sigma}_p \det \mathbf{J}_p w_p.$$

6.24.6 11. Cuadratura

Por defecto **Gauss–Legendre $3 \times 3 \times 3$** (27 puntos): orden de exactitud 5 en cada dirección. Suficiente para integrar exactamente $\mathbf{B}^T \mathbf{D} \mathbf{B}$ del **Hex20** sobre Jacobiano constante (el integrando es a lo sumo de grado 3 en cada dirección) y para integrar exactamente la masa consistente $\rho \mathbf{N}^T \mathbf{N}$ (grado 4 en cada dirección).

Aviso sobre integración reducida: el esquema $2 \times 2 \times 2$ (**hex_2x2x2**, 8 puntos) está disponible y reduce el coste computacional, pero introduce **6 modos de hourglass espurios** sobre un elemento aislado (verificación numérica directa del proyecto; Belytschko-Liu-Moran §8.4.4). Estos modos **no propagan** a través de mallas ensambladas con condiciones de Dirichlet razonables — la rigidez global ensamblada queda rank-sufficient en la mayoría de casos prácticos, pero un elemento aislado o una capa con BC pobres puede mostrar hourglass. El código permite seleccionar la cuadratura reducida vía el parámetro **quadrature** pero **no estabiliza** los modos espurios. Recomendación operativa: usar el default $3 \times 3 \times 3$ salvo necesidad experimental. La matriz de masa se integra con $3 \times 3 \times 3$ independientemente de la cuadratura elegida para **K**, para evitar subintegrarla.

6.24.7 12. Cargas distribuidas consistentes

Fuerza de cuerpo b. Vector nodal equivalente:

$$\mathbf{f}_e^b = \int_{\Omega_e} \mathbf{N}^T \mathbf{b} dV = \sum_{p=1}^{n_g} \mathbf{N}(\xi_p)^T \mathbf{b} \det \mathbf{J}_p w_p,$$

donde **N** es la matriz 3×60 de funciones de forma. Se reutiliza la cuadratura del elemento. Para **b** uniforme y geometría regular el resultado es exacto. **Para Hex20 el reparto no es uniforme entre nodos:** la fórmula serendípita asigna pesos distintos a vértices y medios (los medios reciben más carga total que los vértices, con suma global $\rho \cdot V_e$ correcta).

Tracción de superficie $\bar{\mathbf{t}}$ sobre una cara. Cada cara del **Hex20** tiene **8 nodos** (4 vértices + 4 medios de arista) y se aproxima por funciones serendípticas Quad8 sobre el parámetro 2D $(s, t) \in [-1, 1]^2$. La numeración de caras conserva la convención del **Hex8** (ADR 0012) extendida con los medios:

Cara	Vértices	Medios de arista	Normal saliente
0	(0,3,2,1)	(11, 10, 9, 8)	$-\zeta$ (inferior)
1	(4,5,6,7)	(12, 13, 14, 15)	$+\zeta$ (superior)
2	(0,1,5,4)	(8, 17, 12, 16)	$-\eta$ (frontal)
3	(1,2,6,5)	(9, 18, 13, 17)	$+\xi$ (derecha)
4	(2,3,7,6)	(10, 19, 14, 18)	$+\eta$ (trasera)
5	(3,0,4,7)	(11, 16, 15, 19)	$-\xi$ (izquierda)

El orden dentro de cada cara reproduce la convención VTK_QUADRATIC_HEXAHEDRON: primero los 4 vértices, luego los 4 medios en el mismo orden cíclico (medio entre vértice k y vértice $k + 1$).

$$\mathbf{f}_e^t = \int_{\Gamma_e} \mathbf{N}^\top \bar{\mathbf{t}} dS.$$

Cuadratura 2D sobre la cara: Gauss 3×3 (nueve puntos) — exacta para tracción constante sobre cara plana (integrando degree 2 en (s, t)). $\bar{\mathbf{t}}$ se especifica en **coordenadas globales** (t_x, t_y, t_z) ; presiones normales se obtienen multiplicando previamente por la normal exterior de la cara. Tracción variable sobre la cara no soportada en este paso. **Para Hex20 el reparto entre vértices y medios no es trivial** (la integral cuadrática de Quad8 sobre cara plana asigna fracciones específicas: los vértices reciben $-1/12$ de la fuerza total cada uno y los medios $4/12$ cada uno, con suma global exacta — ver test de balance).

6.24.8 13. Salida por punto de Gauss

`compute_gauss_state(U)` devuelve un dict con ε y σ en cada uno de los n_g puntos de Gauss del elemento (27 por defecto), junto con coordenadas naturales y globales. Habilita post-proceso fino (mapas no promediados, suavizado nodal, recovery superconvergente futuro). Por ADR 0012, `internal_forces` devuelve `None` (sólidos no tienen fuerzas internas seccionales discretas).

6.25 Limitaciones declaradas

6.25.1 Locking volumétrico con $\nu \rightarrow 0,5$

Hex20 con integración completa $3 \times 3 \times 3$ y material casi-incompresible (Elastic3D con $\nu \rightarrow 0,5$, o VonMises3D en régimen plástico desarrollado) sufre **locking volumétrico** menos severo que el Hex8 pero todavía presente: el espacio serendípito triquadrático tampoco contiene desplazamientos puramente isocóricos suficientes en mallas distorsionadas (Cook-Malkus-Plesha §13.6).

Tratamiento en este proyecto: declarar limitación, **blindar con test cuantitativo** (`test_volumetric_locking_` espejo del análogo Hex8) que documente la mitigación parcial respecto al lineal, **no implementar B-bar/F-bar/mixed u-p**. Política idéntica a Quad4/Quad8 y Hex8 (ver STATUS.md "Limitaciones declaradas").

6.25.2 Hourglass con integración reducida

Hex20 integrado con `hex_2x2x2` tiene **6 modos de hourglass espurios** por elemento aislado (vs 12 del Hex8 reducido). En mallas ensambladas con Dirichlet razonable los modos suelen no propagar — la rigidez global queda rank-sufficient — pero un elemento aislado o una capa con BC pobres lo mostraría. Reducción disponible vía `quadrature`; estabilización no implementada. Modo recomendado: full integration $3 \times 3 \times 3$.

6.25.3 Mass lumping

`compute_mass_matrix(lumping="lumped")` usa HRZ canónico (`solidum/math/mass_lumping.py::lump_hrz`). Para Hex20 (con nodos de medio de arista) HRZ es la única opción razonable: row-sum produciría masas **negativas** en los vértices (los pesos de Gauss son negativos para los vértices en la cuadratura serendípita Lobatto equivalente), inadmisibles para análisis dinámico (Bathe FEP §9.2.4). HRZ preserva masa total escalando la diagonal consistente por un factor común.

6.26 Contrato de implementación

```

name: Hex20
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
  n_nodes: 20
  strain_dim: 6          # Voigt 3D [_xx, _yy, _zz, _xy, _yz, _xz]
  n_integration_points: 27 # default Gauss 3x3x3

parameters:
  - { name: quadrature, type: str, required: false, default: "hex_3x3x3",
      desc: "Regla de cuadratura desde QuadratureRegistry (hex_3x3x3 default,
            hex_2x2x2 reducida con 4 modos de hourglass espurios)" }

material_contract:
  signature: "compute_state(, state) -> (, C_tangent, state)"
  strain_kind: "Voigt 3D [_xx, _yy, _zz, _xy, _yz, _xz] con _ij = 2·_ij"

conventions:
  sign: " > 0  tracción (Reglas §5)"
  voigt: "[xx, yy, zz, xy, yz, xz] con _ij = 2·_ij (ADR 0012)"
  node_orientation: "convención VTK_QUADRATIC_HEXAHEDRON; vértices 0-7 idénticos
                    al Hex8; medios 8-11 bottom edges, 12-15 top edges, 16-19
                    vertical edges; asegura det J > 0 en todos los Gauss"
  face_numbering: "ADR 0012 - 6 caras con normal saliente, idéntica a Hex8 en
                  los vértices; cada cara añade 4 nodos medios en el mismo
                  orden cíclico (Quad8 serendipity)"

validity:
  - "pequeños desplazamientos y deformaciones"
  - "geometría sin distorsión severa (det J > tol en todos los puntos de Gauss)"
  - "compatible con materiales con STRAIN_DIM = 6 (Elastic3D, VonMises3D,
      DruckerPrager3D, IsotropicDamage3D)"

out_of_scope:
  - "grandes deformaciones, formulaciones corotacionales o lagrangianas totales"
  - "estabilización contra hourglass para integración reducida hex_2x2x2"
  - "tracción variable sobre cara o presión normal con gradiente"
  - "internal_forces (ADR 0012: sólidos exponen compute_gauss_state)"
  - "Hex27 lagrangiano (sub-fase 2 de A.ter)"

acceptance:
  verification:
    - name: patch_test_lineal
      setup: "Hex20 con campo lineal u_i = a_ij·x_j impuesto en los 20 nodos

```

```

        del contorno; sin nodos interiores libres (todos están en la
        frontera de un elemento aislado, pero al menos los medios no
        están fijados en su posición referencial)"
expect: " constante e igual a sym(a) en todos los puntos de Gauss;
        uniforme"
tol_rel: 1.0e-12
- name: patch_test_cuadratico
  setup: "Hex20 con campo  $u_x = c \cdot x^2$ ,  $u_y = u_z = 0$  impuesto en los 20 nodos
        del contorno (todos)"
  expect: " $_{xx}(x) = 2c \cdot x$  exacto en todos los puntos de Gauss"
  tol_rel: 1.0e-12
- name: modos_de_cuerpo_rigido
  setup: "K_e del Hex20 proyectado sobre los 6 modos de cuerpo rígido"
  expect: "K_e · u_rigid 0 (autovalores 0: exactamente 6)"
  tol_abs: 1.0e-9
- name: simetria_K_e
  setup: "evaluar K_e con Elastic3D"
  expect: "K_e == K_e.T exacto"
  tol_abs: 1.0e-12
specific:
- name: cubo_lame_3d
  setup: "cubo unitario con un Hex20; tracción uniaxial  $_{xx} = p$  en cara +;
        cara opuesta restringida en ux; restricciones mínimas para evitar
        movimiento rígido"
  expect: " $u_x(L, y, z) = p \cdot L/E$ ;  $u_y = - \cdot p \cdot y/E$ ;  $u_z = - \cdot p \cdot z/E$  exactos en
        todos los nodos"
  tol_rel: 1.0e-10
- name: body_load_uniforme
  setup: "Hex20 regular cubo unitario con  $b = (0, 0, -g)$ "
  expect: " $\Sigma f_z = -g \cdot V_e = -g$ ; reparto no uniforme entre vértices y medios"
  tol_rel: 1.0e-12
- name: face_traction_balance
  setup: "Hex20 cubo unitario con tracción  $(p, 0, 0)$  sobre cara + (face 3)"
  expect: " $\Sigma f_x = p \cdot A_{cara} = p \cdot 1$ ; vértices reciben  $-p/12$  cada uno y medios
         $4p/12$  cada uno (suma =  $4 \cdot -(1/12) + 4 \cdot (4/12) = 12/12 = 1$ , OK)"
  tol_rel: 1.0e-12
- name: jacobiano_degenerado_abortado
  setup: "Hex20 con un nodo medio desplazado fuera del rango cuasi-recto"
  expect: "ValueError en _compute_kinematics_hex20 cuando det J tol"
- name: convergencia_cantilever_macneal
  setup: "cantilever esbelto  $L/h = 30$  con  $1 \times 1$  sección + N elementos a lo
        largo; flecha vs Euler-Bernoulli"
  expect: "Hex20 con  $1 \times 1 \times 6$  alcanza  $>0.9 \cdot u_{EB}$  (Hex8 con  $1 \times 1 \times 6$  solo 0.5);
        convergencia h monotónica al refinar"
  tol_rel: "documentar - sin tolerancia estricta"
- name: locking_volumetrico_atenuado
  setup: "cantilever esbelto con Hex20; {0.3, 0.4999}"
  expect: "ratio  $u_{tip}(=0.4999) / u_{tip}(=0.3)$  significativamente mayor
        que el del Hex8 (Hex20  $< 0.6 \rightarrow$  Hex20  $> 0.8$  esperado al menos)"
  tol_rel: "documentar valores; cota relativa Hex20 vs Hex8"
- name: mass_consistent_total
  setup: "Hex20 cubo unitario con  $\rho = 7850$ , masa consistente"
  expect: " $\Sigma M$  (todas las entradas) =  $3 \cdot \rho \cdot V = 3 \cdot 7850 \cdot 1$  (conservación 3D)"
  tol_rel: 1.0e-9
- name: mass_lumped_total_HRZ
  setup: "Hex20 cubo unitario con  $\rho = 7850$ , masa lumped HRZ"
  expect: "diag(M) suma =  $3 \cdot \rho \cdot V$ ; matriz estrictamente diagonal; todas las
        entradas estrictamente positivas (HRZ canónico, no row-sum)"
  tol_abs: 1.0e-12

```

references:

- "Bathe K.J. (2014). Finite Element Procedures. §5.3.2 (serendipity 3D)."
- "Cook R.D., Malkus D.S., Plesha M.E., Witt R.J. (2002). Concepts and Applications of FEA. §6.5, §13.6 (locking volumétrico Hex20)."
- "Zienkiewicz O.C., Taylor R.L. (2005). The Finite Element Method, vol. 1, §8.7 (familia serendípita 3D)."
- "ADR 0012 - Sólidos 3D: convención Voigt 6D y caras."

6.27 Implementación

- **Archivo:** solidum/elements/solid_3d/hex20.py.
- **Clase:** Hex20 (registrada vía `@ElementRegistry.register`); subclase de `_HigherOrderSolid3D` desde **sub-fase 2** de A.ter — la entrada del Hex27 disparó la centralización (regla de los dos casos reales). El cuerpo de `hex20.py` quedó como pura declaración de atributos: funciones de forma, cuadraturas, `FACE_NODES` y funciones de cara `Quad8` (`_N_quad8/_dN_quad8` del paquete 2D).
- **Funciones núcleo:** `_N_hex20(xi, eta, zeta)`, `_dN_hex20(xi, eta, zeta)` en solidum/elements/solid_3d/_shared.py. Numpy puro (no `@njit`) por la complejidad de la rama por nodo serendipity, paritario con `_N_quad8/_dN_quad8` del 2D. El kinematics genérico `'_kinematics_higher_order_3d'` (también en `_shared.py`) es reutilizado por la base con cualquier par (`shape_fn`, `grad_fn`) 3D.
- **Tests:**
 - `tests/test_solid_3d_higher_order.py` — clase `TestHex20Element` (dimensiones/DOFs, patch lineal, patch cuadrático, simetría K, jacobiano degenerado abortado, body load suma, face traction suma + reparto vértice/medio, masa consistente y lumped HRZ, gauss state).
 - `tests/test_rigid_body_modes.py` — `TestRBMHex20` (3 trans + 3 rot + rank-deficiency = 6).
 - `tests/validation/test_cube_lame_3d.py` (extender) — `test_uniaxial_traction_hex20_exact`.
 - `tests/validation/test_macneal_beam_3d.py` (extender) — convergencia Hex20 1×1 sección vs Hex8 1×1.
 - `tests/test_volumetric_locking_3d.py` (extender) — `TestHex20VolumetricLocking` documentando mitigación parcial.

6.28 Diálogo

- **2026-05-27** · Spec inicial. Sub-fase 1 de la sub-etapa A.ter (sólidos 3D cuadráticos). Hex20 standalone — sin base interna compartida en esta sub-fase; el segundo caso (Hex27) entra en sub-fase 2 y dispara la centralización en `_HigherOrderSolid3D` (regla de los dos casos reales aplicada).

- **2026-05-27** · Implementado y validado. Patch test lineal y cuadrático exactos a precisión máquina, 6 modos rígidos exactos, cubo Lamé 3D uniaxial e hidrostático exactos. Demostración cuantitativa de la mitigación del shear locking sobre MacNeal 3D: Hex20 $6\times 1\times 1$ alcanza 97 % de la flecha analítica EB, frente a $<55\%$ de Hex8 $12\times 1\times 1$. Mitigación parcial del locking volumétrico documentada: $\text{ratio } u(\nu=0.4999)/u(\nu=0.3) \approx 0.80$ (vs <0.6 del Hex8). Conteo de modos espurios con cuadratura reducida **hex_2x2x2**: $12 = 6$ rígidos + 6 hourglass por elemento aislado (corregido desde el "4inicial tras verificación numérica directa; coincide con Belytschko-Liu-Moran §8.4.4). Spec actualizada a **status: validated**.
- **2026-05-27** · Refactor a la base `_HigherOrderSolid3D` con la entrada del Hex27 (sub-fase 2). El cuerpo del archivo `hex20.py` queda como declaración de atributos (`_SHAPE_FN`, `_GRAD_FN`, `_DEFAULT_QUADRATURE`, `_MASS_QUADRATURE`, `_FACE_N_FN`, `_FACE_DN_FN`, `_FACE_QUADRATURE`, `FACE_NODES`). La maquinaria de cálculo (bucle de Gauss para K, gauss state, body load, face traction, masa consistente, HRZ) ahora vive en la base — paritaria con la centralización 2D `_HigherOrderSolid2D` (Quad8/Quad9/Tri6). Los 26 tests previos pasan sin modificaciones.

6.29 Hex27

*Elemento sólido 3D de orden cuadrático con interpolación **Lagrangiana completa** — producto tensorial de Lagrange cuadrático en las tres direcciones. Reproduce **todos** los polinomios triquadráticos $\xi^a \eta^b \zeta^c$ con $a, b, c \in \{0, 1, 2\}$, incluyendo los puramente triquadráticos ($\xi^2 \eta^2 \zeta^2$ y combinaciones) que faltan en el serendípito Hex20. Sub-fase 2 de A.ter; dispara la centralización en `_HigherOrderSolid3D` (regla de los dos casos reales).*

6.30 Especificación física

6.30.1 0. Descripción general

Hexaedro tridimensional de segundo orden con **27 nodos**: 8 vértices + 12 medios de arista + 6 centros de cara + 1 centro del cuerpo. Aproximación Lagrangiana C^0 completa en coordenadas naturales $(\xi, \eta, \zeta) \in [-1, 1]^3$. Formulación isoparamétrica. Régimen de pequeños desplazamientos y deformaciones.

6.30.2 1-5. Cinemática, ecuación constitutiva, equilibrio

Idénticas al ‘Hex20’ — la diferencia entre serendipity y Lagrange está exclusivamente en el espacio de aproximación discreto, no en la formulación variacional.

6.31 Formulación numérica (FEM)

6.31.1 6. Discretización — convención VTK_TRIQUADRATIC_HEXAHEDRON

Los 27 nodos en el cubo de referencia $[-1, 1]^3$:

Rango	Tipo	Posiciones (ξ_i, η_i, ζ_i)
0-7	vértices	idénticos al Hex8/Hex20
8-19	medios de arista	idénticos al Hex20
20	centro de cara	$(-1, 0, 0)$ — cara 5 $(-\xi)$
21	centro de cara	$(+1, 0, 0)$ — cara 3 $(+\xi)$
22	centro de cara	$(0, -1, 0)$ — cara 2 $(-\eta)$
23	centro de cara	$(0, +1, 0)$ — cara 4 $(+\eta)$
24	centro de cara	$(0, 0, -1)$ — cara 0 $(-\zeta)$
25	centro de cara	$(0, 0, +1)$ — cara 1 $(+\zeta)$
26	centro del cuerpo	$(0, 0, 0)$

6.31.2 7. Funciones de forma — producto tensorial de Lagrange cuadrático

Para cada nodo se asigna una terna de índices $(i, j, k) \in \{-1, 0, +1\}^3$ correspondiente a su posición natural. Las funciones de forma son productos tensoriales:

$$N_n(\xi, \eta, \zeta) = L_{i_n}(\xi) L_{j_n}(\eta) L_{k_n}(\zeta),$$

con los tres polinomios de Lagrange cuadrático 1D:

$$L_{-1}(x) = \frac{1}{2}x(x-1), \quad L_0(x) = 1-x^2, \quad L_{+1}(x) = \frac{1}{2}x(x+1).$$

Cumplen $L_{-1}(-1) = L_0(0) = L_{+1}(+1) = 1$ y cero en los demás. Garantiza propiedad delta de Kronecker $N_m(\xi_n, \eta_n, \zeta_n) = \delta_{mn}$ y partición de la unidad $\sum_n N_n = 1$.

Espacio reproducido: el ‘Hex27 reproduce **completamente** el espacio triquadrático @MINL20 --- 27 monomios, idéntico al número de nodos. La diferencia con el Hex20 serendípito está en los términos puramente triquadráticos $\xi^2\eta^2$, $\eta^2\zeta^2$, $\xi^2\zeta^2$, $\xi^2\eta^2\zeta$, $\xi^2\eta\zeta^2$, $\xi\eta^2\zeta^2$ que Hex20 no captura.

6.31.3 8. Matriz B

Tamaño **6×81**, ensamblada con la misma plantilla 6×3 del ‘Hex8/Hex20’ por nodo. Derivadas obtenidas vía regla del producto sobre los polinomios de Lagrange:

$$\frac{\partial N_n}{\partial \xi} = L'_{i_n}(\xi) L_{j_n}(\eta) L_{k_n}(\zeta),$$

con

$$L'_{-1}(x) = x - \frac{1}{2}, \quad L'_0(x) = -2x, \quad L'_{+1}(x) = x + \frac{1}{2}.$$

6.31.4 9-10. Rigidez y fuerzas internas

Idénticas a ‘Hex8/Hex20’ en estructura. El tamaño es 81×81.

6.31.5 11. Cuadratura

Por defecto **Gauss–Legendre 3×3×3** (27 puntos): orden de exactitud 5 en cada dirección. Suficiente para integrar exactamente $\mathbf{B}^\top \mathbf{D} \mathbf{B}$ del ‘Hex27’ sobre Jacobiano constante (el integrando es a lo sumo de grado 3 en cada dirección — derivada cuadrático×cuadrático) y la masa consistente $\rho \mathbf{N}^\top \mathbf{N}$ (grado 4 en cada dirección).

Aviso sobre integración reducida: el esquema $2 \times 2 \times 2$ (hex_2x2x2, 8 puntos) está disponible pero **introduce muchos modos espurios** sobre un elemento aislado: verificación numérica directa del proyecto cuenta **27 modos de hourglass** (= 81 DOFs — 8 Gauss × 6 strain — 6 rigid; jump espectral limpio entre eigs[32] $\approx 1e-10$ y eigs[33] $\approx 7e3$). Es muchísimo peor que en el Hex20 reducido (6 hourglass) — el espacio extra del Lagrangiano completo es subintegrado severamente por $2 \times 2 \times 2$. La cita clásica de “1 modo bubble”(Cook-Malkus-Plesha-Witt §11.6) corresponde al **único modo que persiste tras ensamblar** en mallas estructuradas; en un elemento aislado el conteo real es el de arriba. Sin estabilización implementada. Recomendación operativa estricta: usar el default $3 \times 3 \times 3$ con Hex27.

6.31.6 12. Cargas distribuidas consistentes

Fuerza de cuerpo: integración con cuadratura del elemento, reparto no uniforme entre vértices, medios, centros de cara y centro del cuerpo. Suma global $\mathbf{b} \cdot V_e$ exacta para $\mathbf{b}$ uniforme.

Tracción de superficie: cada cara del ‘Hex27’ tiene **9 nodos** (4 vértices + 4 medios de arista + 1 centro de cara) que se aproximan por funciones Lagrangianas Quad9 sobre el parámetro 2D $(s, t) \in [-1, 1]^2$.

Cara	Vértices	Medios de arista	Centro de cara	Normal saliente
0	(0,3,2,1)	(11, 10, 9, 8)	24	$-\zeta$
1	(4,5,6,7)	(12, 13, 14, 15)	25	$+\zeta$
2	(0,1,5,4)	(8, 17, 12, 16)	22	$-\eta$
3	(1,2,6,5)	(9, 18, 13, 17)	21	$+\xi$
4	(2,3,7,6)	(10, 19, 14, 18)	23	$+\eta$
5	(3,0,4,7)	(11, 16, 15, 19)	20	$-\xi$

El orden dentro de cada cara reproduce la convención de ‘_N_quad9’ del paquete 2D: primero los 4 vértices, después los 4 medios en orden cíclico, y finalmente el nodo central. Cuadratura 2D sobre la cara: Gauss 3×3 (exacta para tracción constante sobre cara plana).

6.31.7 13. Salida por punto de Gauss

`compute_gauss_state(U)` devuelve dict con ε y σ en cada uno de los 27 puntos de Gauss del elemento, junto con coordenadas naturales y globales. Sin `internal_forces` (ADR 0012).

6.32 Limitaciones declaradas

6.32.1 Coste computacional vs ‘Hex20’

Hex27 tiene **81 DOFs por elemento** vs 60 del Hex20 (35 % más). El espacio extra que captura (términos puramente triquadráticos) **rara vez gobierna la solución** en problemas físicos estándar — el Hex20 serendípito es la elección por defecto en la mayoría de aplicaciones (Bathe FEP §5.3.2). El Hex27 se prefiere cuando:

- La geometría tiene curvatura severa y se mapea con elementos isoparamétricos cuadráticos: el nodo central de cuerpo ayuda a representar la curvatura interior.
- La sensibilidad al locking es crítica y se prefiere intentar el Q_2 completo antes que recurrir a B-bar/F-bar.
- Se busca convergencia óptima $O(h^{p+1})$ con $p = 2$ en problemas donde el contenido espectral cubre el espacio completo.

Para problemas con flexión simple y geometría rectilínea, Hex20 y Hex27 dan resultados prácticamente idénticos a un coste menor para el primero.

6.32.2 Locking volumétrico con $\nu \rightarrow 0,5$

Comparable al ‘Hex20’: el espacio extra triquadrático añade flexibilidad sin eliminar la incompresibilidad — formulación mixta u-p o B-bar/F-bar son necesarias para eliminarlo. Declarado como limitación, blindado por test cuantitativo.

6.32.3 Hourglass con integración reducida

Hex27 integrado con `hex_2x2x2` tiene **27 modos de hourglass espurios por elemento aislado** (vs 6 del Hex20 reducido y 12 del Hex8 reducido). El espacio Lagrangiano completo es subintegrado severamente por 8 puntos. En mallas estructuradas la mayoría no propagan tras ensamblar, pero el conteo a nivel de elemento es alto. Sin estabilización.

6.33 Contrato de implementación

```

name: Hex27
kind: element
status: validated

interface:
  dof_names: [ux, uy, uz]
  n_nodes: 27
  strain_dim: 6
  n_integration_points: 27      # default Gauss 3x3x3

parameters:
  - { name: quadrature, type: str, required: false, default: "hex_3x3x3",
      desc: "Regla de cuadratura desde QuadratureRegistry; hex_2x2x2
            reducida con 1 modo de hourglass espurio (bubble) por
            elemento aislado, sin estabilización" }

material_contract:
  signature: "compute_state(, state) -> (, C_tangent, state')"
  strain_kind: "Voigt 3D [_xx, _yy, _zz, _xy, _yz, _xz] con _ij = 2·_ij"

conventions:
  sign: " > 0   tracción (Reglas §5)"
  voigt: "[xx, yy, zz, xy, yz, xz] con _ij = 2·_ij (ADR 0012)"
  node_orientation: "VTK_TRIQUADRATIC_HEXAHEDRON; 0-7 vértices (Hex8), 8-19
                    medios de arista (Hex20), 20-25 centros de cara en orden
                    (-x, +x, -y, +y, -z, +z), 26 centro del cuerpo"
  face_numbering: "ADR 0012 - 6 caras con normal saliente, paritarias con
                  Hex8/Hex20 en los vértices y medios; cada cara añade el
                  nodo de centro correspondiente (índice 20-25)"

validity:
  - "pequeños desplazamientos y deformaciones"
  - "geometría sin distorsión severa (det J > tol en todos los puntos de Gauss)"
  - "compatible con materiales con STRAIN_DIM = 6 (Elastic3D, VonMises3D,
      DruckerPrager3D, IsotropicDamage3D)"

out_of_scope:
  - "grandes deformaciones, formulaciones corotacionales"
  - "estabilización contra hourglass para integración reducida"
  - "tracción variable sobre cara"
  - "internal_forces (ADR 0012)"
  - "Tet10 cuadrático (sub-fase 3)"

acceptance:
  verification:
    - name: patch_test_lineal
      setup: "campo lineal u_i = a_ij·x_j en los 27 nodos del contorno"
      expect: " constante = sym(a) en todos los Gauss;  uniforme"
      tol_rel: 1.0e-12
    - name: patch_test_triquadratico
      setup: "campo u_x = c·x2·y2·z2 impuesto en los 27 nodos"
      expect: "_xx(x,y,z) = 2c·x·y2·z2 exacto en todos los Gauss (el espacio
              Q_2 lagrangiano completo lo contiene; Hex20 serendípito N0)"
      tol_rel: 1.0e-12
    - name: modos_de_cuerpo_rigido
      setup: "evaluar K_e y proyectar sobre 6 modos rígidos 3D"

```

```

expect: "K_e · u_rigid 0; exactamente 6 autovalores nulos"
tol_abs: 1.0e-9
- name: simetria_K_e
  setup: "K_e con Elastic3D"
  expect: "K_e == K_e.T exacto"
  tol_abs: 1.0e-12
specific:
- name: cubo_lame_3d
  setup: "1 Hex27 en cubo unitario con tracción uniaxial y BCs mínimas"
  expect: "u_x = p·x/E etc. exacto a precisión máquina (campo lineal)"
  tol_rel: 1.0e-10
- name: body_load_uniforme
  setup: "Hex27 cubo unitario con b = (0, 0, -g)"
  expect: "Σf_z = -g·V_e = -g"
  tol_rel: 1.0e-12
- name: face_traction_balance
  setup: "tracción (p, 0, 0) sobre cara + del cubo unitario"
  expect: "Σf_x = p·A_cara = p; reparto consistente Lagrange Q_2 (vértices,
    medios y centro de cara con sus pesos respectivos)"
  tol_rel: 1.0e-12
- name: reduced_integration_introduces_hourglass
  setup: "Hex27 con cuadratura reducida hex_2x2x2"
  expect: "81 - 8·6 = 33 modos cero totales (6 rígidos + 27 hourglass).
    Verificación con jump espectral limpio entre eigs[32] 1e-10
    y eigs[33] 0(10^3) (escala del módulo de cortante × volumen)"
- name: mass_consistent_total
  setup: "Hex27 cubo unitario con =7850, masa consistente"
  expect: "Σ M = 3 · V = 23550"
  tol_rel: 1.0e-9
- name: mass_lumped_total_HRZ
  setup: "Hex27 cubo unitario con =7850, masa lumped HRZ"
  expect: "diag(M) suma = 3 · V; matriz diagonal; todas las entradas
    positivas (HRZ canónico)"
  tol_abs: 1.0e-12

references:
- "Bathe K.J. (2014). Finite Element Procedures. §5.3.2 (Lagrangiano 3D)."
```

```

- "Cook R.D., Malkus D.S., Plesha M.E., Witt R.J. (2002). Concepts and
  Applications of FEA. §6.6, §11.6 (Q2/E27 con 2x2x2 reducida)."
```

```

- "Zienkiewicz O.C., Taylor R.L. (2005). The Finite Element Method, vol. 1,
  §8.6 (familia Lagrangiana 3D)."
```

```

- "ADR 0012 - Sólidos 3D: convención Voigt 6D y caras."
```

6.34 Implementación

- **Archivo:** solidum/elements/solid_3d/hex27.py.
- **Clase:** Hex27 (registrada vía `@ElementRegistry.register`); subclase de `_HigherOrderSolid3D`.
- **Funciones núcleo:** `_N_hex27(xi, eta, zeta)`, `_dN_hex27(xi, eta, zeta)` en `solidum/elements/solid_3d/_shared.py`. Implementadas en numpy puro vía productos tensoriales de los polinomios Lagrange 1D `_L_lagrange_quad` y `_dL_lagrange_quad`.
- **Centralización (sub-fase 2):** `_HigherOrderSolid3D` abstrae el bucle de Gauss para `K`, `F_int`, `gauss_state`, `body_load`, `face_traction` y masa consistente. Comparte con Hex20 la misma

kinematics y la misma maquinaria; sólo cambian las funciones de forma 3D, la cuadratura, las funciones de forma 2D de cara y los `FACE_NODES`.

■ **Tests:**

- `tests/test_solid_3d_higher_order.py` — `TestHex27Element` (patch lineal, patch **triquadrático**, simetría K, jacobiano degenerado, body load, face traction balance, masa consistente y lumped HRZ, reduced integration 1 hourglass).
- `tests/test_rigid_body_modes.py` — `TestRBMHex27` (3 trans + 3 rot + rank-deficiency = 6).
- `tests/validation/test_cube_lame_3d.py` — `test_uniaxial_traction_hex27_exact`, `test_hydrostatic`
- `tests/validation/test_macneal_beam_3d.py` — `test_macneal_3d_hex27_*` (convergencia espejo del Hex20).
- `tests/test_volumetric_locking_3d.py` — `TestHex27VolumetricLocking` (mitigación comparable al Hex20).

6.35 Diálogo

- **2026-05-27** · Spec inicial. Sub-fase 2 de A.ter — el segundo caso real (Hex27) dispara la centralización en `_HigherOrderSolid3D` (regla de los dos casos reales aplicada en el momento canónico, igual que el `_HigherOrderSolid2D` cuando entró el `Quad9` después del `Quad8`).
- **2026-05-27** · Implementado y validado. Patch test lineal, bilineal mixto ($u_x = c \cdot xy$) y triquadrático completo ($u_x = c \cdot x^2 y^2 z^2$) exactos a precisión máquina — esta última es la capacidad distintiva del Hex27 respecto al Hex20 serendípito. K simétrica, 6 modos rígidos, masas consistente y lumped HRZ con todas las entradas positivas, balance de cargas distribuidas correcto. Spec a `status: validated`.

Capítulo 7

Modelos Constitutivos — 1D

7.1 CableMaterial1D

*Orden de trabajo. El usuario escribe **especificación física, formulación y contrato**; la IA rellena **implementación** y responde en **diálogo**.*

7.2 Especificación física

7.2.1 0. Descripción general

Material 1D memoryless que modela la respuesta axial de un cable: **elástico lineal en tensión y nulo en compresión**. No tiene variables internas ni historia — la respuesta depende exclusivamente de la deformación instantánea. Captura la propiedad esencial de un cable: que solo transmite esfuerzo cuando está tensado.

7.2.2 1. Ecuación constitutiva

$$\sigma(\varepsilon) = E \langle \varepsilon \rangle^+ = \begin{cases} E \varepsilon & \varepsilon > 0 \\ 0 & \varepsilon \leq 0 \end{cases}$$

Rampa lineal de pendiente E en tensión ($\varepsilon > 0$), cero en compresión o estado sin deformación ($\varepsilon \leq 0$). $\langle \cdot \rangle^+$ denota la parte positiva (operador de MacAuley).

7.2.3 2. Módulo tangente

$$E_t(\varepsilon) = \frac{d\sigma}{d\varepsilon} = \begin{cases} E & \varepsilon > 0 \\ 0 & \varepsilon \leq 0 \end{cases}$$

Discontinuo en $\varepsilon = 0$. Por convención el valor en el punto de corte se asigna al régimen compresivo ($E_t = 0$), lo que evita generar rigidez espuria cuando el cable está exactamente sin tensión.

7.2.4 3. Variables internas

No hay. La respuesta es puramente función de ε actual; el estado del material es el vacío.

7.2.5 4. Interpretación física

- $\varepsilon > 0$: cable tensado — se comporta como barra elástica.
- $\varepsilon < 0$: cable destensado — no transmite nada, la carga pasa por otros caminos del modelo.
- $\varepsilon = 0$: frontera neutra — no hay tensión, no hay rigidez.

La no linealidad es **constitutiva** (en la respuesta del material), no geométrica. Un elemento con este material puede tener cualquier cinemática (lineal o corotacional); la unilateralidad solo vive aquí.

7.3 Contrato de implementación

```

name: CableMaterial1D
kind: material
status: validated

interface:
  strain_dim: 1
  primary_state_var: null      # memoryless, no exporta variable principal

parameters:
  - { name: E, type: float, required: true, desc: "Módulo de Young (en tensión)" }

signature:
  compute_state: "(: float, state_vars=None) -> (: float, E_tangent: float, state_vars)"
  strain_kind: axial escalar

conventions:
  sign: " > 0      > 0 (elongación);      0      = 0"
  state_passthrough: "state_vars se devuelve intacto (el material no almacena historia)"

validity:
  - "E > 0"
  - "|| 1e-2 en el régimen tensado (elasticidad lineal)"
  - "IS_UNILATERAL = True solo admitido en elementos con ACCEPTS_UNILATERAL = True (Cable2DCorot, Cable3DCorot). La base Element rechaza la combinación con Truss* en construcción."

out_of_scope:
  - plasticidad, fluencia, fatiga
  - viscoelasticidad / dependencia temporal
  - pretensión inicial (el usuario la modela con una longitud de referencia ajustada)
  - regularización numérica en compresión (no hay rigidez residual en 0)
  - anisotropía direccional; la respuesta es puramente axial escalar

numerical_caveats:
  - "En transiciones tensado destensado (cruza 0), E_tangent salta de E a 0 y Newton-Raphson puede oscilar. Mitigación: usar arc-length o pasos de carga finos si el problema tiene destensiones reales."
  - "Un cable completamente destensado tiene K_M = 0 y K_G = 0 matriz tangente singular en los DOFs del elemento. El usuario debe garantizar que otros elementos del modelo proporcionen rigidez suficiente, o pretensar el cable antes de aplicar cargas que puedan destensarlo."

```

```

acceptance:
- name: respuesta en tensión pura
  setup: " = 1e-4"
  expect: " = E · 1e-4, E_tangent = E"
  tol_rel: 1.0e-12

- name: respuesta en compresión pura
  setup: " = -1e-4"
  expect: " = 0, E_tangent = 0"
  tol_abs: 1.0e-15

- name: punto de corte
  setup: " = 0"
  expect: " = 0, E_tangent = 0 (régimen compresivo por convención)"
  tol_abs: 1.0e-15

- name: state_vars se devuelve intacto
  setup: "state_vars = {'dummy': 42}, arbitrario"
  expect: "el tercer retorno es idéntico al state_vars de entrada"

references:
- "Irvine H.M., Cable Structures, MIT Press, §1.2 (modelo elástico del cable)"
- "MacAuley M.A., on the deflection of beams, 1919 (operador parte positiva)"

```

7.4 Implementación

- **Archivo:** solidum/materials/cable_1d.py — archivo propio, sin dependencia de otros módulos de material salvo la clase base `Material`.
- **Clase:** `CableMaterial1D` — hereda directamente de `Material` (no de `Elastic1D` ni de ningún otro material), registrada vía `@MaterialRegistry.register`.
- **Tests:** `tests/test_cable_material.py · TestCableMaterial1D` — seis tests:
 - `test_acceptance_tension_pura` (criterio 1).
 - `test_acceptance_compresion_pura` (criterio 2).
 - `test_acceptance_punto_de_corte` (criterio 3).
 - `test_acceptance_state_vars_passthrough` (criterio 4).
 - `test_registro_en_registry` — verifica autodiscover.
 - `test_validacion_E_positivo` — rechaza $E \leq 0$.
- **Notas de traducción:**
 - La ramificación en `compute_state` es un único `if strain > 0.0`. El `>` estricto implementa la convención del punto de corte (el cero se asigna al régimen compresivo). El `else` cubre tanto $\varepsilon < 0$ como $\varepsilon = 0$ con las mismas salidas.
 - `state_vars` se devuelve como llega (identidad, no copia). El test `test_acceptance_state_vars_passthrough` usa `assertIs` para confirmar identidad.

- Validación de $E > 0$ al construir, con mensaje claro. Atrapa typos del input YAML temprano.
- El material acepta ****kwargs** en `compute_state` por compatibilidad con elementos que pudieran pasar argumentos adicionales (convención del proyecto).

7.5 Diálogo

- **2026-04-21** · Material creado como pieza independiente para la futura implementación de elementos de cable (`Cable2DCorot`, `Cable3DCorot`). Deliberadamente **no hereda** de `Elastic1D` aunque su rama en tensión sea idéntica: mantener el material autocontenido simplifica su lectura y evita acoplamientos futuros si la rama elástica cambia en otro archivo.
- **2026-04-21** · Discontinuidad en $\varepsilon = 0$: documentada explícitamente en `out_of_scope: regularización numérica`. Si aparece un caso de uso con destensiones frecuentes donde Newton-Raphson oscile, se añadirá una variante `CableMaterial1DRegularized` con $E_c \ll E$ en compresión — **no** se modificará este material (contrato limpio).
- **2026-05-12** · Restricción de emparejamiento elevada a contrato: `IS_UNILATERAL = True` en el material; `ACCEPTS_UNILATERAL = True` en `Cable2DCorot/Cable3DCorot`. La base `Element._validate_material` rechaza la combinación con `Truss*` (cuyo tangente colapsa la rigidez global sin `K_G` que lo compense). Antes el contrato lo permitía y solo se desaconsejaba en docstring; ahora falla limpio al construir.

7.6 Elastic1D

*Spec **retroactiva**: el material existe desde el origen del proyecto y está validado por tests preexistentes. Esta spec lo documenta sin cambiar comportamiento — H-5.3 de la auditoría 2026-05-18.*

7.7 Especificación física

7.7.1 0. Descripción general

Modelo elástico lineal isótropo unidimensional. Captura la relación σ - ε axial de barras, cables traccionados (no unilaterales — para unilateralidad ver `CableMaterial1D`), y la respuesta elástica de cualquier elemento 1D del catálogo (truss, frame Euler en su componente axial). Independiente de la velocidad, sin variables históricas.

Es el modelo más sencillo del catálogo: **una sola constante elástica** (módulo de Young E) y una sola ecuación constitutiva.

7.7.2 1. Descomposición de la deformación

No aplica — no hay parte plástica. Toda la deformación es elástica:

$$\varepsilon = \varepsilon^e$$

7.7.3 2. Ley elástica (Hooke 1D)

$$\sigma = E \varepsilon$$

con $E > 0$ módulo de Young. Tangente constante $E_t = E$ en todo régimen.

7.7.4 3. Superficie de fluencia / criterio de daño

No aplica — el material es indefinidamente elástico.

7.7.5 4. Regla de flujo

No aplica.

7.7.6 5. Endurecimiento

No aplica.

7.7.7 6. Condiciones de Kuhn-Tucker

No aplica.

7.7.8 7. Variables internas

Ninguna — `state_vars = None` se acepta y se devuelve intacto. Sin `PRIMARY_STATE_VAR`.

7.7.9 8. Notación

Esfuerzo y deformación escalares en convención axial: $\sigma > 0 \Leftrightarrow$ tracción, $\varepsilon > 0 \Leftrightarrow$ alargamiento (Reglas §5).

7.8 Formulación numérica

7.8.1 9. Esquema temporal

No aplica — el material no tiene historia. Cada evaluación de `compute_state` es independiente.

7.8.2 10. Return mapping / actualización

No aplica. La evaluación se reduce a una multiplicación escalar:

$$\sigma_{n+1} = E \varepsilon_{n+1}, \quad E_t = E$$

7.8.3 11. Tangente algorítmica consistente

$$C^{\text{alg}} = E$$

Coincide con la tangente continua y es simétrica trivialmente. `IS_SYMMETRIC = True` (por default del contrato base).

7.8.4 12. Caveats numéricos

Ninguno — el modelo es perfectamente bien condicionado para $E > 0$.

7.9 Contrato de implementación

```
name: Elastic1D
kind: material
status: validated

interface:
  strain_dim: 1
  primary_state_var: null      # sin variables internas
  is_symmetric: true

parameters:
  - { name: E,          type: float, required: true, desc: "Módulo de Young (>0)" }
  - { name: density,    type: float, required: false, default: null,
      desc: "Densidad (kg/m³). Opcional al construir; obligatoria solo si se ensambla
            peso propio o masa (ADR 0008)" }

signature:
  compute_state: "(: float, state_vars=None, **kwargs) -> (: float, E_t: float,
                    state_vars)"
  strain_kind: "axial escalar"

state_variables: []           # ninguna
```

```

conventions:
    sign: " > 0  tracción,  > 0  alargamiento (Reglas §5)"
    units: "[E] = [] / [] = Pa (o consistente con el sistema)"

validity:
    - "E > 0"
    - "régimen elástico indefinido - sin umbral superior"
    - "pequeñas deformaciones (|| 1e-2 por hipótesis cinemática global)"

out_of_scope:
    - "endurecimiento / fluencia  usar Elastoplastic1D"
    - "softening por daño  usar IsotropicDamage1D"
    - "unilateralidad (sin compresión)  usar CableMaterial1D"
    - "anisotropía o dependencia de temperatura"
    - "viscoelasticidad"

acceptance:
    # Cubierto por tests existentes de elementos 1D (Truss, Frame, Cable, etc.)
    # que usan Elastic1D como material por default. La verificación se hace
    # implícita en cada test de elemento.
    verification:
        - name: ley_de_hooke_exacta
          setup: "evaluar compute_state con  arbitrario; comparar  = E· "
          expect: " = E·  exacto; tangente = E"
          tol_rel: 1.0e-14
        - name: tangente_constante
          setup: "evaluar dos veces con strain distintos"
          expect: "E_t devuelto idéntico en ambas evaluaciones"
          tol_rel: 1.0e-14
        - name: ciclo_carga_descarga_sin_historia
          setup: "  creciente luego decreciente; verificar reversibilidad  "
          expect: " ()=E·  para todo  (sin histéresis, sin state_vars retornados
              modificados)"
          tol_rel: 1.0e-14
        - name: rechazo_E_no_positivo
          setup: "construir Elastic1D(E=0) y Elastic1D(E=-1)"
          expect: "ValueError con mensaje claro"
        - name: rechazo_density_negativa
          setup: "construir Elastic1D(E=210e9, density=-1)"
          expect: "ValueError con mensaje claro"

references:
    - "Cualquier texto elemental de mecánica de materiales (Beer, Hibbeler, Timoshenko §
        1)."
```

7.10 Implementación

- **Archivo:** solidum/materials/elastic.py.
- **Clase:** Elastic1D, registrada vía `@MaterialRegistry.register`.
- **Estado:** `state_vars=None` permitido; se acepta y se devuelve sin tocar (compatibilidad con elementos no lineales que pasan diccionarios históricos).
- **admissibility_scale:** hereda el comportamiento por defecto del contrato `Material` (no usado — sin criterio de admisibilidad).

- **Tests:** ningún test unitario propio de `Elastic1D` (es trivial); su correctitud se ejercita implícitamente en todos los tests de elementos 1D que lo usan como material (decenas de tests en `tests/test_truss*.py`, `tests/test_frame*.py`, `tests/test_cable*.py`, `tests/test_dynamic*.py`).

7.11 Diálogo

- **2026-05-19** · Spec creada retroactivamente para cerrar el hueco H-5.3 de la auditoría 2026-05-18. El material es anterior a la convención de specs validadas; no se modifica código. La aceptación se considera cumplida por cobertura indirecta en tests de elementos.

7.12 Elastoplastic1D

*Spec **retroactiva**: el material existe desde la fase inicial del proyecto y está validado por tests unitarios y de integración. Esta spec lo documenta sin cambiar comportamiento — H-5.3 de la auditoría 2026-05-18.*

7.13 Especificación física

7.13.1 0. Descripción general

Modelo elastoplástico unidimensional independiente de la velocidad (rate-independent”) con criterio de fluencia $|\sigma| \leq \sigma_y + H \alpha$ y **endurecimiento isótropo lineal**. Es el análogo 1D de **VonMises2D** — la versión axial sobre la que se valida y benchmarkean los pipelines plásticos. Se usa en barras (**Truss2D/3D**) y como componente axial conceptual de marcos plásticos (pendiente **FiberSection**).

7.13.2 1. Descomposición aditiva

$$\varepsilon = \varepsilon^e + \varepsilon^p$$

con ε^p histórica (memoria del material).

7.13.3 2. Ley elástica

$$\sigma = E \varepsilon^e = E (\varepsilon - \varepsilon^p)$$

7.13.4 3. Criterio de fluencia

$$f(\sigma, \alpha) = |\sigma| - (\sigma_y + H \alpha) \leq 0$$

donde $\sigma_y > 0$ es la fluencia inicial y $H \geq 0$ el módulo de endurecimiento isótropo lineal.

7.13.5 4. Regla de flujo (asociada)

$$\dot{\varepsilon}^p = \dot{\gamma} \operatorname{sign}(\sigma)$$

7.13.6 5. Endurecimiento isótropo lineal

$$\dot{\alpha} = \dot{\gamma}$$

α es la deformación plástica acumulada equivalente, escalar monótona no decreciente.

7.13.7 6. Condiciones de Kuhn-Tucker

$$\dot{\gamma} \geq 0, \quad f \leq 0, \quad \dot{\gamma} f = 0$$

7.13.8 7. Variables internas

Símbolo	Tipo	Significado
ε^p	escalar	deformación plástica acumulada con signo
α	escalar	deformación plástica acumulada equivalente (≥ 0)

`alpha` es la variable principal exportable (`PRIMARY_STATE_VAR = 'alpha'`).

7.13.9 8. Notación

Escalares en convención axial: $\sigma > 0 \Leftrightarrow$ tracción, $\varepsilon > 0 \Leftrightarrow$ alargamiento (Reglas §5). ε^p con signo (puede ser negativa tras carga compresiva).

7.14 Formulación numérica

7.14.1 9. Esquema temporal — Backward Euler implícito

Dado el estado convergido $\{\varepsilon_n^p, \alpha_n\}$ al paso n y la deformación total ε_{n+1} , resolver para $\{\sigma_{n+1}, \varepsilon_{n+1}^p, \alpha_{n+1}\}$.

7.14.2 10. Return mapping — forma cerrada

Predictor elástico:

$$\sigma^{\text{trial}} = E (\varepsilon_{n+1} - \varepsilon_n^p)$$

Función de fluencia trial:

$$f^{\text{trial}} = |\sigma^{\text{trial}}| - (\sigma_y + H \alpha_n)$$

- Si $f^{\text{trial}} \leq \text{tol} \Rightarrow$ **paso elástico:** $\sigma_{n+1} = \sigma^{\text{trial}}$, $E_t = E$, estado intacto.
- Si $f^{\text{trial}} > \text{tol} \Rightarrow$ **paso plástico,** retorno radial cerrado:

$$\Delta\gamma = \frac{f^{\text{trial}}}{E + H}, \quad \sigma_{n+1} = \sigma^{\text{trial}} - \Delta\gamma E \operatorname{sign}(\sigma^{\text{trial}})$$

$$\varepsilon_{n+1}^p = \varepsilon_n^p + \Delta\gamma \operatorname{sign}(\sigma^{\text{trial}}), \quad \alpha_{n+1} = \alpha_n + \Delta\gamma$$

Sin Newton local — la forma cerrada es exacta en 1D.

7.14.3 11. Tangente algorítmica consistente

En régimen elástico: $E_t = E$. En régimen plástico:

$$E_t = \frac{E H}{E + H}$$

con $H = 0$ recuperando $E_t = 0$ (plasticidad perfecta, descenso de carga indefinido bajo control de desplazamiento).

`IS_SYMMETRIC = True` (trivial en 1D).

7.14.4 12. Caveats numéricos

- **Plasticidad perfecta** ($H = 0$): tangente $E_t = 0$ tras la fluencia. Newton-Raphson global puede oscilar o requerir line search en problemas multibarra (modos de mecanismo). Mitigación: usar incremento pequeño cerca del umbral o activar `line_search=True` en el solver.
- **Switching exacto trial elástico $\leftrightarrow$ plástico**: la tolerancia `is_admissible` (ADR 0006) usa `admissibility_scale =  $\sigma_y + H \cdot \alpha$` y patrón `atol + rtol · escala` — invariancia bajo cambio de unidades.

7.15 Contrato de implementación

```

name: Elastoplastic1D
kind: material
status: validated

interface:
  strain_dim: 1
  primary_state_var: alpha    # deformación plástica acumulada equivalente
  is_symmetric: true

parameters:
  - { name: E,          type: float, required: true, desc: "Módulo de Young (>0)" }
  - { name: sigma_y,    type: float, required: true, desc: "Esfuerzo de fluencia inicial (>0)" }
  - { name: H,          type: float, required: false, default: 0.0,
      desc: "Módulo de endurecimiento isótropo lineal (0). H=0  plasticidad perfecta"
    }
  - { name: density,    type: float, required: false, default: null,
      desc: "Densidad (kg/m³). Opcional al construir; obligatoria solo si se ensambla peso propio o masa (ADR 0008)" }

signature:
  compute_state: "(: float, state_vars=None) -> (: float, E_t: float, state_vars)"
  strain_kind: "axial escalar"

state_schema:
  eps_p: "float - deformación plástica acumulada con signo"
  alpha: "float 0 - deformación plástica acumulada equivalente"

conventions:
  sign: "> 0  tracción, ^p con signo (puede ser negativa)"
  units: "[E]=[_y]=[H]=Pa;  adimensional"

validity:
  - "E > 0, _y > 0, H 0"
  - "carga proporcional o no proporcional (sin restricción de trayectoria)"
  - "pequeñas deformaciones (|| 1e-2)"

out_of_scope:
  - "endurecimiento cinemático (back-stress)  requiere variante 1D dedicada"
  - "endurecimiento isótropo no lineal (Voce, exponencial)"
  - "ablandamiento (H<0)  requiere regularización"
  - "viscoplasticidad / dependencia de la velocidad"
  - "grandes deformaciones (descomposición multiplicativa F = F^e · F^p)"

```

```

acceptance:
# Cubierto por tests/test_materials_unit.py y tests/test_truss_plastic.py
unit:
- name: paso_elastico
  setup: " = 0.5·_y/E (mitad del umbral); state_vars=None"
  expect: " = E· exacto, =0, ^p=0, tangente=E"
  tol_rel: 1.0e-12
- name: fluencia_uniaxial_traccion
  setup: " = 1.5·_y/E (1.5× el umbral elástico)"
  expect: " = _y (con H=0); =0.5·_y/E; ^p > 0; tangente=0"
  tol_rel: 1.0e-10
- name: endurecimiento_lineal
  setup: "carga monotónica hasta = 2·_y/E con H = E/10"
  expect: "_final = _y + H·; tangente = E·H/(E+H) en régimen plástico"
  tol_rel: 1.0e-10
- name: ciclo_carga_descarga
  setup: " creciente hasta plástico, luego decreciente"
  expect: "descarga elástica con tangente E; permanece constante durante descarga
; histéresis correcta"
  tol_rel: 1.0e-10
- name: cambio_de_signo
  setup: "tracción plástica luego compresión hasta fluencia inversa"
  expect: "fluencia inversa en = -(_y + H·) (endurecimiento isótropo es simétrico
)"
  tol_rel: 1.0e-10
- name: monotonicidad_alpha
  setup: "trayectoria arbitraria"
  expect: " no decrece nunca"

# Cubierto por tests de integración con barras plásticas
integration:
- name: truss_uniaxial_plastico
  setup: "Truss2D con Elastoplastic1D, carga axial creciente"
  expect: "respuesta fuerza-desplazamiento bilineal exacta (E hasta fluencia, E_t
después)"
  tol_rel: 1.0e-6

references:
- "Simó J.C., Hughes T.J.R. (1998). Computational Inelasticity. Springer. §1 (J2 1D
return mapping)."
- "de Souza Neto E.A., ċPeri D., Owen D.R.J. (2008). Computational Methods for
Plasticity. Wiley. §6.2 (uniaxial)."
- "Lubliner J. (1990). Plasticity Theory. Macmillan. §3.2."

```

7.16 Implementación

- **Archivo:** solidum/materials/plastic_1d.py.
- **Clase:** Elastoplastic1D, registrada vía @MaterialRegistry.register.
- **admissibility_scale:** $\sigma_y + H \alpha$ (ADR 0006). El switching elástico/plástico usa el patrón atol + rtol-escala.
- **State schema:** {'eps_p': float, 'alpha': float}. state_vars=None se interpreta como estado virgen ($\varepsilon^p = 0$, $\alpha = 0$).
- **Tests:**

- tests/test_materials_unit.py · clase `TestElastoplastic1D` (carga, descarga, endurecimiento, plasticidad perfecta).
- Integración: cualquier test de truss con material plástico.

7.17 Diálogo

- **2026-05-19** · Spec creada retroactivamente para cerrar el hueco H-5.3 de la auditoría 2026-05-18. Material anterior a la convención de specs; no se modifica código. La aceptación se considera cumplida por los tests unitarios existentes.

7.18 IsotropicDamage1D

Orden de trabajo. El usuario escribe **especificación física, formulación y contrato**; la IA rellena **implementación** y responde en **diálogo**.

>Spec **retroactiva con ampliación**: modelo implementado desde sesiones anteriores con tangente secante; esta spec lo documenta y añade la tangente algorítmica consistente, replicando la derivación de `[[IsotropicDamage2D]]` reducida a 1D escalar.

7.19 Especificación física

Versión 1D de `IsotropicDamage2D`. Modelo de daño escalar isótropo en pequeñas deformaciones para barras, cables y elementos axiales. Captura degradación progresiva de la rigidez axial con ablandamiento exponencial.

7.19.1 Ecuaciones

- **Constitutiva**: $\sigma = (1 - d) E \varepsilon$, con $\sigma^{\text{eff}} = E \varepsilon$.
- **Deformación equivalente**: $\varepsilon_{\text{eq}} = |\varepsilon|$.
- **Variable histórica**: $\kappa_{n+1} = \max(\kappa_n, |\varepsilon_{n+1}|)$, con κ_0 inicial.
- **Daño**: $d(\kappa) = \begin{cases} 0 & \kappa \leq \kappa_0 \\ 1 - (\kappa_0/\kappa) e^{-\alpha(\kappa-\kappa_0)} & \kappa > \kappa_0 \end{cases}$, saturado a `DAMAGE_MAX`.

Todas las propiedades físicas (irreversibilidad, asintótica $d \rightarrow 1$, papel de α) son las del modelo 2D restringidas a un escalar. No distingue tracción de compresión.

7.20 Formulación numérica

7.20.1 Tangente algorítmica consistente

$$\frac{\partial \sigma}{\partial \varepsilon} = (1 - d) E - E \varepsilon \frac{\partial d}{\partial \kappa} \frac{\partial \kappa}{\partial \varepsilon}$$

con:

- $\frac{\partial d}{\partial \kappa} = (1 - d) \left(\frac{1}{\kappa} + \alpha \right)$ (idéntico a 2D).
- $\frac{\partial \kappa}{\partial \varepsilon} = \text{sign}(\varepsilon)$ en **carga activa** ($|\varepsilon| > \kappa_n$); 0 en descarga.

Tangente final:

$$E^{\text{alg}} = \begin{cases} E & \kappa \leq \kappa_0 \text{ (sin daño)} \\ (1 - d) E & \text{descarga} \\ (1 - d) E [1 - (1/\kappa + \alpha) \kappa] = -(1 - d) E \alpha \kappa & \text{carga activa, } d < d_{\text{max}} \\ (1 - d_{\text{max}}) E & d = d_{\text{max}} \text{ (saturación)} \end{cases}$$

donde se ha usado $(1/\kappa + \alpha)\kappa = 1 + \alpha\kappa$ y $E \varepsilon \text{sign}(\varepsilon) = E|\varepsilon| = E\kappa$ en carga activa.

7.20.2 Signo de la tangente consistente

En carga activa **siempre es negativa**: $E^{\text{alg}} = -(1-d)E\alpha\kappa < 0$ porque $1-d > 0$, $E > 0$, $\alpha > 0$, $\kappa > 0$. Refleja físicamente el régimen de **softening** (ablandamiento): la curva σ - ε desciende tras el umbral.

Implicaciones numéricas:

- La rigidez tangente del elemento ($\mathbf{K}_e = \int B^\top E^{\text{alg}} B dV$) puede tener autovalores negativos. La matriz **sigue siendo simétrica** (escalar $\times$ forma cuadrática) — `IS_SYMMETRIC = True` se mantiene.
- La matriz global puede dejar de ser **positiva definida**. El despachador algebraico (ADR 0003) detecta el fallo de Cholesky y degrada a LU automáticamente — no requiere cambio declarativo.
- Control de carga puede no converger en problemas con suficiente daño para producir snap-back; en ese régimen se requiere control de longitud de arco (`ArcLengthSolver`).

7.20.3 Por qué no tests de integración en esta spec

A diferencia de `[[IsotropicDamage2D]]`, no se añade benchmark de integración `Truss2D + IsotropicDamage1D + NonlinearSolver`: con un solo elemento axial el comportamiento global está dominado por la inestabilidad de softening ($E_{\text{alg}} < 0$) y la única vía limpia de seguir la respuesta post-pico es arc-length. La validación de la tangente consistente se cubre con tests unitarios (incluyendo diferencia finita centrada como verificación de la derivada cerrada). Si en el futuro aparece un caso real de problema con daño 1D ramificado, se añadirá un benchmark dedicado con `ArcLengthSolver`.

7.21 Contrato de implementación

```
name: IsotropicDamage1D
kind: material
status: validated

interface:
  strain_dim: 1
  primary_state_var: damage
  is_symmetric: true          # tangente escalar    contribución elemento simétrica (puede
                              ser indefinida pero simétrica)

parameters:
- { name: E,          type: float, required: true, desc: "Módulo de Young intacto" }
- { name: kappa_0,    type: float, required: true, desc: "Umbral de deformación
  equivalente ||" }
- { name: alpha,      type: float, required: true, desc: "Velocidad de degradación
  exponencial (>0)" }
- { name: density,    type: float, required: false, default: null, desc: "Densidad (ADR
  0008)" }

signature:
  compute_state: "(: float, state_vars=None) -> (: float, E_tan: float, state_vars)"
  strain_kind: axial escalar

state_schema:
```

```

kappa: "float    _0 - máxima || histórica"
damage: "float    [0, DAMAGE_MAX]"

conventions:
  sign: " > 0   tracción; el modelo no distingue tracción de compresión en la activaci
        ón del daño"
  damage_irreversibility: " monótono no decreciente; d() no decreciente"

validity:
  - "E > 0, _0 > 0,   > 0"
  - "|| 1e-2 (régimen de pequeñas deformaciones)"

out_of_scope:
  - "split tracción/compresión"
  - "fatiga / acumulación cíclica"
  - "regularización para mesh-dependency en problemas con varios elementos en serie"
  - "test de integración con NonlinearSolver - requiere arc-length para superar el snap
    -back, fuera de scope hasta caso real"

numerical_caveats:
  - "Tangente consistente NEGATIVA en carga activa (softening). La rigidez global puede
    no ser PD; el despachador algebraico (ADR 0003) degrada a LU automáticamente al
    detectar fallo de Cholesky."
  - "Control de carga con un elemento aislado tras alcanzar el pico se vuelve inestable
    . Usar arc-length para seguir la rama post-pico."
  - "Transición cargadescarga discontinua en la tangente (rama loading vs unloading):
    aceptable para Newton convergente, problemático cerca del umbral en problemas
    patológicos."

acceptance:
  unit_existing_no_regression:
    - name: secant_response_below_threshold
      expect: " pequeño,    _0  d=0,  =E· , tangente = E"
      tol_rel: 1.0e-12

    - name: damage_evolution_exponential
      expect: "ley exponencial reproducida con la fórmula"
      tol_rel: 1.0e-10

  unit_consistent_tangent:
    - name: tangent_equals_E_below_threshold
      setup: " < _0"
      expect: "E_tan = E (sin daño)"
      tol_rel: 1.0e-12

    - name: tangent_equals_secant_on_unloading
      setup: "cargar a  > _0; descargar a || < "
      expect: "E_tan = (1-d)·E exacto"
      tol_rel: 1.0e-12

    - name: tangent_negative_on_loading
      setup: "carga activa con  > _0, d  (0, DAMAGE_MAX)"
      expect: "E_tan = -(1-d)·E· · < 0"
      tol_rel: 1.0e-12

    - name: tangent_equals_secant_at_saturation
      setup: " enorme tal que d = DAMAGE_MAX"
      expect: "E_tan = (1-DAMAGE_MAX)·E (sin término consistente al saturar)"
      tol_rel: 1.0e-12

```

```

- name: tangent_matches_finite_difference
  setup: "carga activa; / por FD centrada (h=1e-7) vs fórmula cerrada"
  expect: "error relativo < 1e-5"
  tol_rel: 1.0e-5

- name: tangent_consistent_with_2D_when_uniaxial
  setup: "evaluar IsotropicDamage1D(E, _0, ) en =eps y comparar contra
         IsotropicDamage2D(E, =0, _0, ) plane stress en =[eps, 0, 0]; la primera
         componente de y la entrada [0,0] de C_alg deben coincidir"
  expect: "_1D = _xx_2D; E_tan_1D = C_alg_2D[0,0]"
  tol_rel: 1.0e-10

references:
- "Lemaitre J., Chaboche J.-L. (1990). Mechanics of Solid Materials. Cambridge UP.
  Cap. 7."
- "Simó J.C., Ju J.W. (1987). Strain- and stress-based continuum damage models - I.
  Int. J. Solids Struct. 23, 821-840."
- "Spec hermana: [docs/specs/IsotropicDamage2D.md](IsotropicDamage2D.md) - modelo 2D,
  derivación detallada de la tangente consistente."

```

7.22 Implementación

- **Archivo:** solidum/materials/damage_1d.py.
- **Clase:** `IsotropicDamage1D`, registrada vía `@MaterialRegistry.register`. `IS_SYMMETRIC` queda `True` (default heredado de `Material`) — la contribución elemental es simétrica aunque pueda ser indefinida.
- **Algoritmo `compute_state`:**
 1. Recuperar κ_n y calcular $\varepsilon_{eq} = |\varepsilon|$.
 2. Si $\varepsilon_{eq} > \kappa_n$: carga activa, $\kappa_{n+1} = \varepsilon_{eq}$. Si no: descarga, $\kappa_{n+1} = \kappa_n$.
 3. Calcular d por la ley exponencial, saturar a `DAMAGE_MAX`.
 4. $\sigma = (1 - d) E \varepsilon$.
 5. Tangente: ramas (sin daño / descarga / saturación / carga activa) idénticas en estructura al modelo 2D, con la fórmula escalar resultante. En carga activa con daño efectivo no saturado, $E_{tan} = -(1-d) \cdot E \cdot \alpha \cdot \kappa$.
- **Tests:**
 - `tests/test_materials_unit.py · TestIsotropicDamage1D` (existente, no regresión) + nueva clase `TestIsotropicDamage1DConsistentTangent`.
 - Sin tests de integración (justificado arriba).

7.23 Diálogo

- **2026-05-14** · Spec creada retroactivamente sobre material existente (tangente secante) y extendida con tangente algorítmica consistente. Réplica directa de `[[IsotropicDamage2D]]` reducida a escalar. Sin ADR — extensión local al material, no afecta a contratos.
- **2026-05-14** · `IS_SYMMETRIC` permanece `True`: la rigidez axial es escalar y la contribución elemental $\mathbf{B}^T \mathbf{E}_{\text{tan}} \mathbf{B}$ es simétrica aunque pueda ser indefinida cuando $\mathbf{E}_{\text{tan}} < 0$. El sistema global puede dejar de ser PD; el fallback automático de Cholesky a LU del despachador algebraico (ADR 0003) lo gestiona sin cambios declarativos.
- **2026-05-14** · No se añade test de integración. Justificación: con tangente consistente < 0 en carga activa, un benchmark `Truss2D` bajo control de carga se vuelve inestable tras el pico; el seguimiento de la rama post-pico requiere arc-length que cae fuera del scope inmediato. Tests unitarios (incluyendo FD numérica y consistencia con la versión 2D reducida) cubren la corrección de la fórmula.

Capítulo 8

Modelos Constitutivos — 2D

8.1 DruckerPrager2D

*Orden de trabajo. El usuario escribe **especificación física**, **formulación numérica** y **contrato**; la IA rellena **implementación** y responde en **diálogo**.*

8.2 Especificación física

8.2.1 0. Descripción general

Modelo elastoplástico **friccional** independiente de la velocidad. Suelos, hormigón, granulares, materiales cuasi-frágiles cohesivo-friccionales. Suaviza la pirámide hexagonal de Mohr-Coulomb a un **cono circular** en el espacio de tensiones principales, eliminando las aristas que complican el return mapping.

Diferencia clave con J2:

- **Dependencia de la presión.** La resistencia al corte aumenta con la presión de confinamiento (componente friccional).
- **Plasticidad no asociada.** La dilatancia volumétrica real del flujo plástico (ψ) es típicamente menor que la fricción del criterio (ϕ). Forzar $\psi = \phi$ (asociada) sobreestima la dilatancia y el volumen post-fluencia.
- **Retorno al ápice.** El criterio define un cono con vértice; estados de prueba en zona puramente hidrostática extrema retornan al vértice (zona singular en la dirección de flujo).

Hipótesis cinemática soportada: **plane_strain** únicamente en esta spec.

8.2.2 1. Descomposición aditiva y elasticidad

Pequeñas deformaciones: $\boldsymbol{\varepsilon} = \boldsymbol{\varepsilon}^e + \boldsymbol{\varepsilon}^p$. Ley elástica isótropa: $\boldsymbol{\sigma} = \mathbf{C}_e(\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}^p)$.

8.2.3 2. Criterio de fluencia (Drucker-Prager)

$$f(\boldsymbol{\sigma}, \alpha) = \sqrt{J_2} + \eta_f I_1 - k(\alpha) \leq 0$$

con:

- $J_2 = \frac{1}{2} \mathbf{s} : \mathbf{s}$, segundo invariante del desviador.
- $I_1 = \text{tr}(\boldsymbol{\sigma}) = \sigma_{xx} + \sigma_{yy} + \sigma_{zz}$, primer invariante.
- η_f , coeficiente friccional del cono (adimensional, ≥ 0).
- $k(\alpha) = k_0 + H_k \alpha$, parámetro cohesivo con endurecimiento isótropo lineal.

La superficie es un cono con eje en la línea hidrostática ($\sigma_{xx} = \sigma_{yy} = \sigma_{zz}$), vértice en $I_1 = k/\eta_f$, abertura controlada por η_f .

8.2.4 3. Calibración con Mohr-Coulomb

Los parámetros (η_f, k_0) se derivan de los parámetros físicos (c_0, ϕ) — cohesión y ángulo de fricción interna — según la variante del cono elegida:

variant	η_f	k_0
plane_strain_matched (default)	$\tan(\phi)/\sqrt{9 + 12 \tan^2(\phi)}$	$3 c_0/\sqrt{9 + 12 \tan^2(\phi)}$
outer_cone	$2 \sin(\phi)/[\sqrt{3} (3 - \sin(\phi))]$	$6 c_0 \cos(\phi)/[\sqrt{3} (3 - \sin(\phi))]$
inner_cone	$2 \sin(\phi)/[\sqrt{3} (3 + \sin(\phi))]$	$6 c_0 \cos(\phi)/[\sqrt{3} (3 + \sin(\phi))]$

plane_strain_matched es la variante que coincide exactamente con Mohr-Coulomb en condiciones de plane strain (Drucker, 1953). Las otras dos circunscriben (outer) o inscriben (inner) el hexágono de MC en el plano π . Como esta spec solo soporta plane strain, plane_strain_matched es el default natural.

8.2.5 4. Regla de flujo (no asociada por defecto)

Potencial plástico g con coeficiente de **dilatancia** η_g (función del ángulo de dilatancia ψ , calibrado con la misma fórmula que η_f pero usando ψ):

$$g(\boldsymbol{\sigma}) = \sqrt{J_2} + \eta_g I_1$$

$$\dot{\boldsymbol{\epsilon}}^p = \dot{\gamma} \frac{\partial g}{\partial \boldsymbol{\sigma}} = \dot{\gamma} \left(\frac{\mathbf{s}}{2\sqrt{J_2}} + \eta_g \mathbf{I} \right)$$

Descomposición:

- **Parte desviadora:** $\dot{\boldsymbol{\epsilon}}^p = \dot{\gamma} \mathbf{s}/(2\sqrt{J_2})$ — análoga a J2.
- **Parte volumétrica:** $\text{tr}(\dot{\boldsymbol{\epsilon}}^p) = 3\dot{\gamma} \eta_g$ — **dilatación plástica positiva** si $\eta_g > 0$.

Si $\psi = \phi$ (i.e. $\eta_g = \eta_f$) el modelo se vuelve **asociado**; tangente algorítmica simétrica. Para $\psi < \phi$ (típico en suelos: $\psi \approx \phi/2$ o incluso $\psi = 0$, "non-dilatan"), el modelo es **no asociado** y la tangente algorítmica es **asimétrica**.

8.2.6 5. Endurecimiento isótropo lineal

$$\dot{\alpha} = \dot{\gamma}$$

con α la **deformación plástica equivalente** (convención del proyecto para Drucker-Prager). La superficie crece manteniendo eje y abertura, solo k aumenta linealmente.

Endurecimiento por fricción/dilatancia (cambio de η_f o η_g con α) queda fuera de esta spec — escenario que no necesitamos hoy y que añade no-linealidad fuerte al return mapping.

8.2.7 6. Condiciones de Kuhn-Tucker

$$\dot{\gamma} \geq 0, \quad f \leq 0, \quad \dot{\gamma} f = 0$$

8.2.8 7. Variables internas

Símbolo	Tipo	Significado
ϵ^p	tensor 4 componentes [xx, yy, zz, xy_tensorial]	deformación plástica
α	escalar ≥ 0	deformación plástica equivalente (= acumulado de $\Delta\gamma$)

alpha es la variable principal exportable (PRIMARY_STATE_VAR = 'alpha').

8.3 Formulación numérica

8.3.1 8. Esquema implícito — Backward Euler

Dado $\{\epsilon_n^p, \alpha_n\}$ y ϵ_{n+1} , predictor elástico:

$$\epsilon^{\text{trial}} = \epsilon_{n+1} - \epsilon_n^p, \quad \mathbf{s}^{\text{trial}} = 2G \mathbf{e}_{\text{dev}}^{\text{trial}}, \quad p^{\text{trial}} = K \text{tr}(\epsilon^{\text{trial}})$$

con $I_1^{\text{trial}} = 3p^{\text{trial}}$, $\sqrt{J_2^{\text{trial}}} = \|\mathbf{s}^{\text{trial}}\|/\sqrt{2}$.

Función de fluencia trial:

$$f^{\text{trial}} = \sqrt{J_2^{\text{trial}}} + \eta_f I_1^{\text{trial}} - k(\alpha_n)$$

Si $f^{\text{trial}} \leq \text{tol} \rightarrow$ paso elástico. Si no, intentar **return regular**.

8.3.2 9. Return regular (cone surface)

La dirección desviadora $\hat{\mathbf{n}} = \mathbf{s}^{\text{trial}}/\sqrt{2J_2^{\text{trial}}} = \mathbf{s}/\sqrt{2J_2}$ es invariante bajo carga radial. Las actualizaciones son lineales en $\Delta\gamma$:

$$\begin{aligned} \sqrt{J_2^{n+1}} &= \sqrt{J_2^{\text{trial}}} - G \Delta\gamma \\ p^{n+1} &= p^{\text{trial}} - 3K \eta_g \Delta\gamma, \quad I_1^{n+1} = 3p^{n+1} \end{aligned}$$

La consistencia $f(\sigma_{n+1}, \alpha_{n+1}) = 0$ con $\alpha_{n+1} = \alpha_n + \Delta\gamma$ y $k_{n+1} = k_0 + H_k(\alpha_n + \Delta\gamma)$ da:

$$\Delta\gamma = \frac{f^{\text{trial}}}{G + 9K \eta_f \eta_g + H_k}$$

Forma cerrada — sin Newton local. Cada término del denominador:

- G : cambio en $\sqrt{J_2}$ por la parte desviadora del flujo.
- $9K \eta_f \eta_g$: cambio en $\eta_f I_1$ por la parte volumétrica dilatante del flujo (multiplicado por η_f del criterio y η_g del potencial).

- H_k : endurecimiento.

Actualizaciones:

$$\mathbf{s}^{n+1} = \frac{\sqrt{J_2^{n+1}}}{\sqrt{J_2^{\text{trial}}}} \mathbf{s}^{\text{trial}}$$

$$\boldsymbol{\sigma}^{n+1} = \mathbf{s}^{n+1} + p^{n+1} \mathbf{I}$$

$$\boldsymbol{\varepsilon}_{n+1}^p = \boldsymbol{\varepsilon}_n^p + \Delta\gamma \left(\frac{\mathbf{s}^{n+1}}{2\sqrt{J_2^{n+1}}} + \eta_g \mathbf{I} \right)$$

Condición de validez del return regular: $\sqrt{J_2^{n+1}} \geq 0$. Si $\Delta\gamma_{\text{reg}} > \sqrt{J_2^{\text{trial}}}/G$, el desviador "se pasa de cero" el algoritmo regular es inválido — toca **return al ápice**.

8.3.3 10. Return al ápice

Cuando el estado de prueba cae en zona puramente hidrostática extrema (p_{trial} muy traccionante respecto a la cohesión), la proyección al cono cae en su vértice, donde $\mathbf{s} = 0$ y la dirección desviadora $\hat{\mathbf{n}}$ no está definida.

En el ápice: $\boldsymbol{\sigma}^{n+1} = (k_{n+1}/\eta_f) \mathbf{I}$ (puramente hidrostático), $\sqrt{J_2^{n+1}} = 0$.

Por conservación volumétrica:

$$p^{n+1} = p^{\text{trial}} - 3K \eta_g \Delta\gamma = \frac{k_{n+1}}{3\eta_f}$$

con $k_{n+1} = k_0 + H_k(\alpha_n + \Delta\gamma)$. Despejando:

$$\Delta\gamma_{\text{apex}} = \frac{p^{\text{trial}} \eta_f - k(\alpha_n)/3 \cdot \eta_f \cdot 3}{3K \eta_f \eta_g + H_k} = \frac{3p^{\text{trial}} \eta_f - k(\alpha_n)}{3K \cdot 3\eta_f \eta_g + H_k}$$

Reordeno: $3p^{\text{trial}} \eta_f - k(\alpha_n) = I_1^{\text{trial}} \eta_f - k(\alpha_n)$. Denominador: $9K \eta_f \eta_g + H_k$. (Equivalente a la fórmula regular sin el término G , porque la rama de apex es puramente volumétrica.)

$$\Delta\gamma_{\text{apex}} = \frac{I_1^{\text{trial}} \eta_f - k(\alpha_n)}{9K \eta_f \eta_g + H_k}$$

Actualizaciones (apex):

$$\mathbf{s}^{n+1} = 0, \quad p^{n+1} = K(\text{tr}(\boldsymbol{\varepsilon}_{n+1}) - 3\eta_g \Delta\gamma) \equiv \frac{k_{n+1}}{3\eta_f}$$

$$\boldsymbol{\sigma}^{n+1} = p^{n+1} \mathbf{I}$$

$$\boldsymbol{\varepsilon}_{n+1}^p = \boldsymbol{\varepsilon}_n^p + \Delta\gamma \eta_g \mathbf{I} + \boldsymbol{\varepsilon}_{\text{trial}}^{p,\text{dev}}$$

(la parte desviadora plástica absorbe completamente el desviador de prueba, ya que $\mathbf{s}^{n+1} = 0$.)

8.3.4 11. Detección del régimen de retorno

Algoritmo de despacho:

1. Calcular $\Delta\gamma_{\text{reg}}$ por la fórmula regular.
2. Si $\sqrt{J_2^{\text{trial}}} - G \Delta\gamma_{\text{reg}} \geq 0 \rightarrow$ return regular.
3. Si no $\rightarrow$ return al ápice con $\Delta\gamma_{\text{apex}}$.

El criterio es geométricamente: el predictor cae más allá del ápice si la proyección desviadora exigiría reducir $\sqrt{J_2}$ por debajo de cero.

8.3.5 12. Tangente algorítmica consistente

Return regular ($\sqrt{J_2^{n+1}} > 0$):

Por linealización del algoritmo:

$$\mathbf{C}^{\text{alg}} = K \mathbf{1} \otimes \mathbf{1} + 2G \left(1 - \frac{G \Delta\gamma}{\sqrt{J_2^{\text{trial}}}} \right) \mathbf{I}_{\text{dev}} + 2G \Theta \hat{\mathbf{n}} \otimes \hat{\mathbf{n}} - 3K \eta_g \mathbf{1} \otimes \mathbf{a}_f - 3K \eta_f \mathbf{a}_g \otimes \mathbf{1} + \text{corrección}$$

donde:

- $\hat{\mathbf{n}} = \mathbf{s}^{n+1} / (2\sqrt{J_2^{n+1}})$ (dirección de flujo desviador, normalizada por convención).
- $\mathbf{1} = (1, 1, 1, 0)$ en notación 4 componentes (hidrostática).
- $\Theta, \mathbf{a}_f, \mathbf{a}_g$: términos cruzados que recogen el acoplamiento entre cambio de $\sqrt{J_2}$ y endurecimiento.

Forma compacta (de Souza Neto §8.3.4):

$$\mathbf{C}^{\text{alg}} = K \mathbf{1} \otimes \mathbf{1} + 2G \left(1 - \frac{G \Delta\gamma}{\sqrt{J_2^{\text{trial}}}} \right) \mathbf{I}_{\text{dev}} - \frac{1}{G + 9K\eta_f\eta_g + H_k} \mathbf{b}_g \otimes \mathbf{b}_f$$

con $\mathbf{b}_g = G\hat{\mathbf{n}} + 3K\eta_g \mathbf{1}$, $\mathbf{b}_f = G\hat{\mathbf{n}} + 3K\eta_f \mathbf{1}$. **Asimétrica** si $\eta_f \neq \eta_g$ (no asociada).

Return al ápice ($\sqrt{J_2^{n+1}} = 0$):

$$\mathbf{C}^{\text{alg}} = K \left(1 - \frac{3K\eta_f\eta_g}{3K\eta_f\eta_g + H_k/3} \right) \mathbf{1} \otimes \mathbf{1}$$

— rigidez puramente volumétrica reducida (el material en el ápice se comporta como un fluido con compresibilidad finita; sin rigidez desviadora alguna).

Forma equivalente más limpia: $\mathbf{C}_{\text{apex}}^{\text{alg}} = \frac{K H_k}{9K\eta_f\eta_g + H_k} \mathbf{1} \otimes \mathbf{1}$ (con $H_k = 0$, el ápice se vuelve completamente plástico volumétrico y la tangente colapsa; este caso queda manejado en el código devolviendo un escalar pequeño > 0 para evitar singularidad).

8.3.6 13. Tolerancia de fluencia

ADR 0006: `admissibility_scale = k(α) = k_0 + H_k · α` (escala cohesiva corriente del cono). Tiene unidades de esfuerzo, igual que f .

8.4 Contrato de implementación

```

name: DruckerPrager2D
kind: material
status: validated

interface:
  strain_dim: 3
  primary_state_var: alpha
  is_symmetric: false      # tangente asimétrica si no asociada ( )

parameters:
- { name: E,          type: float, required: true,  desc: "Módulo de Young" }
- { name: nu,         type: float, required: true,  desc: "Coeficiente de Poisson" }
- { name: cohesion,   type: float, required: true,  desc: "Cohesión c_0 inicial ( esfuerzo)" }
- { name: phi_deg,    type: float, required: true,  desc: "Ángulo de fricción interna en grados; típico suelos 20-40°, hormigón 30-37°" }
- { name: psi_deg,    type: float, required: false, default: null, desc: "Ángulo de dilatación en grados. Default = phi_deg (asociada). Típico no asociada: < , e.g . 0 o /2." }
- { name: H,          type: float, required: false, default: 0.0, desc: "Endurecimiento isótropo lineal en cohesión H_k (0). H=0 perfectamente plástico." }
- { name: hypothesis, type: str,   required: false, default: "plane_strain", desc: "plane_strain (único soportado)" }
- { name: variant,    type: str,   required: false, default: "plane_strain_matched", desc: "plane_strain_matched | outer_cone | inner_cone - calibración con Mohr-Coulomb" }
- { name: density,    type: float, required: false, default: null, desc: "Densidad ( ADR 0008)" }

signature:
  compute_state: " (: ndarray(3,), state_vars=None) -> (: ndarray(3,), C_alg: ndarray (3,3), state_vars)"
  strain_kind: "plano Voigt - [_xx, _yy, _xy] con _xy = 2·_xy"

state_schema:
  eps_p: "ndarray(4,) - [^p_xx, ^p_yy, ^p_zz, ^p_xy_tensorial]"
  alpha: "float 0 - multiplicador plástico acumulado"

conventions:
  sign: " > 0 tracción (Reglas §5)"
  voigt: "_xy = 2·_xy en deformación; ^p_xy en tensorial (sin factor 2)"
  associated_flow: " = asociada (tangente simétrica); no asociada"

validity:
- "E > 0, c_0 > 0, 0 < 90°, 0"
- " (-1, 0.5)"
- "H 0"
- "hypothesis == 'plane_strain' (único soportado en esta spec)"
- "variant {'plane_strain_matched', 'outer_cone', 'inner_cone'}"

out_of_scope:
- "plane_stress - la proyección de Drucker-Prager con _zz=0 acoplada a regla de flujo dilatante es notoriamente delicada; difiere hasta caso real"
- "endurecimiento por cambio de y/o con - no lineal fuerte en el return mapping"
- "endurecimiento cinemático (back-stress hidrostático y desviador)"

```

```

- "cap (modelo cap-cone) para suelos compactables"
- "ablandamiento (H<0) - requiere regularización"
- "grandes deformaciones"

numerical_caveats:
- "Tangente algorítmica asimétrica si → IS_SYMMETRIC=False, el despachador
  algebraico usa LU (ADR 0003)."
```

- "Modos de carga muy traccionantes pueden caer en la rama de ápice. La detección automática del régimen (regular vs apex) está en el algoritmo; si en el futuro aparece oscilación entre ramas en pasos cerca de la transición, considerar smoothing con vertex regularization."

- "Con $\nu = 0$ (sin dilatancia) y $H_k = 0$, en la rama de ápice la tangente volumétrica se anula → singularidad. El código devuelve un valor de respaldo (rigidez volumétrica reducida no nula) para evitar matriz singular; típicamente sale del ápice en el siguiente paso."

- "El return regular puede generar λ creciente con incrementos pequeños cerca de la transición regularapex. Newton global converge igual; si en problema concreto se detecta oscilación, usar arc-length."

```

acceptance:
unit:
- name: parameter_validation
  expect: "constructor rechaza variant inválida,  $\theta = 90^\circ$ ,  $\nu > 0$ , hypothesis distinto
    de plane_strain"

- name: calibration_matched_consistency
  expect: "para variant='plane_strain_matched' con  $\nu = 0$ , asociada →  $\lambda_f = \lambda_g$ ; con
     $\nu = 0$ ,  $\lambda_g = 0$  (sin dilatación)"

- name: paso_elastico_below_yield
  expect: "  $\lambda$  pequeño;  $\lambda = C_e \cdot \epsilon$ ,  $\epsilon_p$  y  $\alpha$  intactos"
  tol_rel: 1.0e-10

- name: regular_return_under_shear
  setup: "  $\tau$  cortante puro suficiente para fluir ( $\gamma_{xy} = \gamma/2$ , sin parte hidrostática)"
  expect: "return regular activo,  $\gamma > 0$ ,  $\sqrt{J}$  del estado final =  $\sqrt{(2/3) \cdot k - \lambda_f \cdot I} = 0$ 
    (cumple criterio)"
  tol_rel: 1.0e-8

- name: apex_return_under_pure_tension
  setup: "  $\sigma$  puramente hidrostática traccionante ( $\sigma_{xx} = \sigma_{yy} > 0$ , sin cortante)"
  expect: "return al ápice;  $\sigma_{xx} = \sigma_{yy} = k/(3 \cdot \lambda_f)$  tras converger;  $\sqrt{J} = 0$ "
  tol_rel: 1.0e-8

- name: associated_tangent_is_symmetric
  setup: "  $\nu = 0$  (asociada), estado plástico activo"
  expect: "C_alg simétrica  $\|(C_{alg} - C_{alg}^T)\| < tol$ "
  tol_abs: 1.0e-8

- name: nonassociated_tangent_is_asymmetric
  setup: "  $\nu = 0$ ,  $\theta = 30^\circ$ , estado plástico activo no degenerado"
  expect: "C_alg - C_alg^T  $\neq 0$  con norma significativa"

- name: tangent_matches_finite_difference
  setup: "estado plástico regular; comparar C_alg cerrada con diferencia finita
    centrada"
  expect: "error relativo < 1e-5"
  tol_rel: 1.0e-5

```

```

- name: alpha_monotonic_under_increasing_load
  expect: "bajo carga creciente no decrece"

- name: unloading_recovers_C_e
  setup: "cargar a plástico, descargar elásticamente"
  expect: "tangente = C_e en el paso de descarga, se conserva"
  tol_rel: 1.0e-10

- name: unit_invariance_MPa_vs_Pa
  expect: "mismo path adimensional en (MPa,mm,N) y (Pa,m,N) idéntico"
  tol_rel: 1.0e-12

integration:
- name: quad4_pipeline_elastico
  setup: "Quad4 unitario plane strain, carga muy por debajo del yield"
  expect: "respuesta elástica pura, =0"
  tol_rel: 1.0e-8

- name: quad4_pipeline_plastico_converges
  setup: "Quad4 unitario plane strain, carga que produce plasticidad activa, 4
  pasos, max_iter=8"
  expect: "Newton converge en cada paso, > 0 al final"

- name: quad4_yield_onset_confined_tension # DP-1 (2026-05-19)
  setup: "Quad4 unitario plane strain con desplazamientos prescritos en los 4 nodos
  para _xx=k·_y, _yy=0, _xy=0"
  expect: "_xx=0.5·_y elástico (_xx coincide con (+2)· exacto, =0); _xx=0.99·_y
  aún =0; _xx=1.5·_y plástico con >0 idéntico en los 4 Gauss points"
  tol_rel: 1.0e-8

- name: quad4_flow_rule_invariant # DP-2 (2026-05-19)
  setup: "Quad4 unitario plane strain, carga monótona en rama regular (5 niveles
  1.2·_y → 2.6·_y), "
  expect: "invariante cinemática tr(_p) = 3·_g· en cada Gauss point y en cada
  nivel"
  tol_rel: 1.0e-8

- name: quad4_apex_under_biaxial_tension # DP-2bis (2026-05-19)
  setup: "Quad4 unitario plane strain, _xx = _yy ~ 1e-2 (predicor hidrostático
  extremo)"
  expect: "estado final cae en ápice: _xx _yy = k()/ (3·_f), _xy 0, en cada
  Gauss point"
  tol_rel: 1.0e-6

references:
- "Drucker D.C., Prager W. (1952). Soil mechanics and plastic analysis or limit
  design. Quart. Appl. Math. 10, 157-165."
- "Drucker D.C. (1953). Coulomb friction, plasticity, and limit loads. J. Appl. Mech.
  21, 71-74. (calibración plane_strain_matched)"
- "Simó J.C., Hughes T.J.R. (1998). Computational Inelasticity. Springer. §6 (cone
  plasticity, apex return)."
- "de Souza Neto E.A., ċPeri D., Owen D.R.J. (2008). Computational Methods for
  Plasticity. Wiley. §8 (Drucker-Prager, tangentes algorítmicas detalladas)."
- "Chen W.F., Han D.J. (1988). Plasticity for Structural Engineers. Springer. §7 (
  cohesivo-friccionales)."

```

8.5 Implementación

- **Archivo:** solidum/materials/drucker_prager_2d.py.
- **Clase:** DruckerPrager2D, registrada vía `@MaterialRegistry.register`. `IS_SYMMETRIC = False` declarativo (la tangente puede ser asimétrica; el despachador agrega el flag sobre todos los materiales del dominio).
- **Kernel Numba** `_compute_drucker_prager_plane_strain`:
 1. Predictor elástico desviador-volumétrico (paralelo a J2 plane strain, ε_{zz}^p libre).
 2. Check f^{trial} .
 3. Si elástico: devolver σ_{trial} restaurando contribución hidrostática y volumétrica.
 4. Si no: intentar $\Delta\gamma_{\text{reg}}$ cerrado; comprobar $\sqrt{J_2^{n+1}} \geq 0$.
 5. Si regular válido: actualizar σ , ε^p , α , tangente regular (de Souza Neto §8.3.4).
 6. Si regular inválido: rama de ápice — $\Delta\gamma_{\text{apex}}$ cerrado, σ puramente hidrostático, tangente volumétrica reducida.
- **Cálculo de** (η_f, k_0, η_g) : en `__init__` desde $(c_0, \phi, \psi, \text{variant})$ — sin coste runtime. Se valida $\psi \leq \phi$ (físicamente sensato).
- **admissibility_scale**: $k(\alpha) = k_0 + H \cdot \alpha$.
- **Tests**:
 - tests/test_materials_unit.py · TestDruckerPrager2D.
 - tests/test_solid_2d_drucker_prager.py.

8.6 Diálogo

- **2026-05-14** · Spec creada. Decisiones clave del alcance MVP:
- Solo `plane_strain` en esta entrega; `plane_stress` con cono en $\sigma_{zz}=0$ es notoriamente delicado y de uso geotécnico raro — se difiere.
- Parámetros físicos (c_0, φ, ψ) en lugar de adimensionales (k_0, η_f, η_g) — más natural para el usuario de geotecnia. Calibración interna por `variant`.
- **No asociada por defecto** (ψ es parámetro independiente con `default = \varphi` que la fuerza a asociada solo si el usuario no especifica). Plasticidad asociada en suelos sobreestima dilatación.
- Endurecimiento solo en cohesión (lineal); fricción/dilatación constantes.
- `IS_SYMMETRIC = False` declarativo a nivel material (independiente del caso particular de uso; conservador y consistente con el patrón del proyecto: el despachador agrega).
- **2026-05-14** · Sin ADR. El modelo encaja en el patrón establecido por `VonMises2D` (kernel Numba, despacho hypothesis aunque solo soporte una, semántica trial/commit) y no rompe contratos transversales.

8.7 Elastic2D

*Spec **retroactiva**: el material existe desde el origen del subsistema 2D y está validado por tests preexistentes (todos los pipelines sólido 2D elástico). Esta spec lo documenta sin cambiar comportamiento — H-5.3 de la auditoría 2026-05-18.*

8.8 Especificación física

8.8.1 0. Descripción general

Modelo elástico lineal isótropo bidimensional, parametrizado por (E, ν) . Soporta dos hipótesis cinemáticas 2D mutuamente excluyentes seleccionadas en construcción:

- **plane_stress** (default) — $\sigma_{zz} = \sigma_{xz} = \sigma_{yz} = 0$. Láminas o membranas delgadas en su plano (chapa traccionada, placa fina).
- **plane_strain** — $\varepsilon_{zz} = \varepsilon_{xz} = \varepsilon_{yz} = 0$. Cuerpos prismáticos largos cargados transversalmente (presa, túnel, cilindro largo).

Independiente de la velocidad, sin variables históricas. Es el material elástico canónico para todos los sólidos 2D del catálogo.

8.8.2 1. Descomposición de la deformación

No aplica — toda la deformación es elástica:

$$\boldsymbol{\varepsilon} = \boldsymbol{\varepsilon}^e$$

8.8.3 2. Ley elástica

$$\boldsymbol{\sigma} = \mathbf{C}_e \boldsymbol{\varepsilon}$$

con $\boldsymbol{\varepsilon} = [\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]^\top$ en notación Voigt del proyecto ($\gamma_{xy} = 2\varepsilon_{xy}$) y $\boldsymbol{\sigma} = [\sigma_{xx}, \sigma_{yy}, \sigma_{xy}]^\top$.

Matriz constitutiva plane stress (3×3):

$$\mathbf{C}_e^{ps} = \frac{E}{1-\nu^2} \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & (1-\nu)/2 \end{bmatrix}$$

Matriz constitutiva plane strain (3×3):

$$\mathbf{C}_e^{pe} = \frac{E}{(1+\nu)(1-2\nu)} \begin{bmatrix} 1-\nu & \nu & 0 \\ \nu & 1-\nu & 0 \\ 0 & 0 & (1-2\nu)/2 \end{bmatrix}$$

Tangente constante $\mathbf{C}_e$ en todo régimen. Simétrica y positiva definida en el rango admisible de ν .

8.8.4 3. Superficie de fluencia / criterio de daño

No aplica — el material es indefinidamente elástico.

8.8.5 4-6. Flujo, endurecimiento, Kuhn-Tucker

No aplican.

8.8.6 7. Variables internas

Ninguna — `state_vars = None` se acepta y se devuelve intacto. Sin `PRIMARY_STATE_VAR`.

8.8.7 8. Notación Voigt

Convención del proyecto: $\varepsilon = [\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$ con $\gamma_{xy} = 2\varepsilon_{xy}$ (deformación angular *engineering*). Los esfuerzos son los componentes tensoriales reales $[\sigma_{xx}, \sigma_{yy}, \sigma_{xy}]$.

8.9 Formulación numérica

8.9.1 9. Esquema temporal

No aplica — sin historia.

8.9.2 10. Return mapping / actualización

No aplica. Evaluación reducida a producto matriz-vector:

$$\sigma_{n+1} = \mathbf{C}_e \varepsilon_{n+1}, \quad \mathbf{C}^{\text{alg}} = \mathbf{C}_e$$

La matriz $\mathbf{C}_e$ se precalcula y guarda en el constructor — coste de `compute_state` es un producto $3 \times 3 \cdot 3$ sin asignación adicional.

8.9.3 11. Tangente algorítmica consistente

$$\mathbf{C}^{\text{alg}} = \mathbf{C}_e$$

Simétrica (`IS_SYMMETRIC = True` por default). En plane stress la matriz es PD para $\nu \in (-1, 0,5)$; en plane strain también, con singularidad en $\nu \rightarrow 0,5$ (incompresible — requiere formulación mixta, no soportada).

8.9.4 12. Caveats numéricos

- **Plane strain** $\nu \rightarrow 0,5$: la matriz constitutiva diverge ($1 - 2\nu \rightarrow 0$). Recomendado $\nu \leq 0,49$ para metales (típico $\nu = 0,3$). El constructor rechaza $\nu \geq 0,5$ con `ValueError`.
- **Locking volumétrico** en plane strain con elementos de bajo orden (Quad4, Tri3) cuando $\nu \rightarrow 0,5$: declarado limitación arquitectural, sin mitigación implementada (B-bar, F-bar diferidas).
- Para $\nu < 0$ el material es admisible termodinámicamente (auxético) pero infrecuente en la práctica; el constructor lo permite hasta $\nu > -1$.

8.10 Contrato de implementación

```

name: Elastic2D
kind: material
status: validated

interface:
  strain_dim: 3
  primary_state_var: null
  is_symmetric: true

parameters:
  - { name: E,          type: float, required: true, desc: "Módulo de Young (>0)" }
  - { name: nu,         type: float, required: true, desc: "Coeficiente de Poisson (-1, 0.5)" }
  - { name: hypothesis, type: str,   required: false, default: "plane_stress", desc: "plane_stress | plane_strain" }
  - { name: density,    type: float, required: false, default: null, desc: "Densidad (kg/m³). Opcional al construir; obligatoria solo si se ensambla peso propio o masa (ADR 0008)" }

signature:
  compute_state: "(: ndarray(3,), state_vars=None) -> (: ndarray(3,), C_e: ndarray(3,3), state_vars)"
  strain_kind: "plano Voigt - [_xx, _yy, _xy] con _xy = 2·_xy"

state_variables: []

conventions:
  sign: "> 0 tracción (Reglas §5)"
  voigt: "_xy = 2·_xy en deformación; _xy tensorial sin factor"
  hypothesis_dispatch: "selección por kwarg al construir; matriz C precalculada"

validity:
  - "E > 0"
  - "(-1, 0.5) estricto; = 0.5 rechazado (incompresible)"
  - "hypothesis {'plane_stress', 'plane_strain'}"
  - "pequeñas deformaciones (|| 1e-2)"

out_of_scope:
  - "anisotropía (ortótropo, monoclinico)"
  - "plasticidad VonMises2D, DruckerPrager2D"
  - "daño IsotropicDamage2D"
  - "incompresibilidad estricta (=0.5) requiere formulación mixta u-p"
  - "grandes deformaciones"

acceptance:
  # Cubierto por tests existentes de elementos sólidos 2D que usan
  # Elastic2D como material por default.
  verification:
    - name: ley_de_hooke_3x3_exacta
      setup: "evaluar compute_state con arbitrario; comparar = C_e· en ambas hipótesis"
      expect: "= C_e· exacto componente a componente; tangente = C_e"
      tol_rel: 1.0e-14
    - name: simetria_C_e
      setup: "construir C_e en ambas hipótesis"
      expect: "C_e == C_e.T exacto"

```

```

    tol_abs: 1.0e-14
- name: positividad_definida
  setup: "autovalores de C_e en ambas hipótesis con {0, 0.3, 0.49}"
  expect: "todos los autovalores estrictamente positivos"
  tol_abs: 0.0
- name: plane_stress_traccion_uniaxial
  setup: " = (_0, -·_0, 0); plane_stress"
  expect: "_xx = E·_0, _yy = 0, _xy = 0 (uniaxial puro emergente)"
  tol_rel: 1.0e-12
- name: plane_strain_confinamiento
  setup: " = (_0, 0, 0); plane_strain"
  expect: "_yy = /(1-) · _xx (reacción de confinamiento clásica)"
  tol_rel: 1.0e-12
- name: rechazo_inputs_invalidos
  setup: "construir Elastic2D con E0, fuera de rango, hypothesis desconocida,
        density<0"
  expect: "ValueError con mensaje claro en cada caso"

references:
- "Timoshenko S., Goodier J.N. (1970). Theory of Elasticity. McGraw-Hill. §3-4 (plane
  stress / plane strain)."
- "Cook R.D., Malkus D.S., Plesha M.E., Witt R.J. (2002). Concepts and Applications
  of Finite Element Analysis. Wiley. §3.6."
- "Bathe K.J. (2014). Finite Element Procedures. Prentice Hall. §4.2."

```

8.11 Implementación

- **Archivo:** solidum/materials/elastic_2d.py.
- **Clase:** Elastic2D, registrada vía @MaterialRegistry.register.
- **Despacho por hipótesis:** la matriz C se precalcula en el constructor según hypothesis. compute_state solo hace self.C @ strain sin ramificaciones — coste mínimo.
- **Validación temprana** en constructor: $E > 0$, $\nu \in (-1, 0.5)$, $hypothesis \in \{\text{plane_stress}, \text{plane_strain}\}$, $density \geq 0$ si se pasa. Mensajes en español.
- **Tests:** cobertura implícita amplísima vía todos los pipelines de sólidos 2D (Quad4, Quad8, Quad9, Tri3, Tri6) que usan Elastic2D como material por default (tests/test_solid_2d_*.py, decenas de tests).

8.12 Diálogo

- **2026-05-19** · Spec creada retroactivamente para cerrar el hueco H-5.3 de la auditoría 2026-05-18. Material anterior a la convención de specs validadas; no se modifica código. Aceptación cumplida por cobertura indirecta en pipelines existentes.

8.13 IsotropicDamage2D

Orden de trabajo. El usuario escribe **especificación física, formulación y contrato**; la IA rellena **implementación** y responde en **diálogo**.

>Spec **retroactiva con ampliación**: el modelo está implementado desde sesiones anteriores con **tangente secante**. Esta spec lo documenta y añade la **tangente algorítmica consistente** para recuperar convergencia cuadrática del Newton global.

8.14 Especificación física

8.14.1 0. Descripción general

Modelo de **daño isótropo escalar** en pequeñas deformaciones para sólidos 2D. Captura degradación progresiva e isótropa de la rigidez en respuesta a la deformación máxima histórica, con ablandamiento exponencial. Pensado para hormigón en tracción uniforme, cerámicos, materiales cuasi-frágiles donde la microfisuración distribuida puede modelarse mediante un escalar de daño en lugar de microestructura explícita.

8.14.2 1. Cinemática

Pequeñas deformaciones, descomposición elástica pura: la deformación plástica no existe en este modelo (sin plasticidad). El daño se manifiesta como degradación de la rigidez efectiva.

8.14.3 2. Esfuerzo nominal vs esfuerzo efectivo

$$\boldsymbol{\sigma} = (1 - d) \boldsymbol{\sigma}^{\text{eff}}, \quad \boldsymbol{\sigma}^{\text{eff}} = \mathbf{C}_e \boldsymbol{\varepsilon}$$

con $d \in [0, d_{\text{max}}]$ la variable de daño isótropa y $\mathbf{C}_e$ el tensor constitutivo elástico isótropo (2D, plane stress o plane strain según **hypothesis** heredada del **Elastic2D** interno).

El esfuerzo efectivo $\boldsymbol{\sigma}^{\text{eff}}$ es el que el material intacto soportaría con la deformación corriente. El esfuerzo nominal $\boldsymbol{\sigma}$ es lo que efectivamente transmite considerando la degradación.

8.14.4 3. Deformación equivalente

Cantidad escalar que cuantifica la “intensidad” de la deformación. Convención del proyecto, norma simétrica simple en notación Voigt $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$ con $\gamma_{xy} = 2\varepsilon_{xy}$:

$$\varepsilon_{\text{eq}}(\boldsymbol{\varepsilon}) = \sqrt{\varepsilon_{xx}^2 + \varepsilon_{yy}^2 + \frac{1}{2}\gamma_{xy}^2} = \sqrt{\boldsymbol{\varepsilon}^\top \mathbf{M} \boldsymbol{\varepsilon}}$$

con $\mathbf{M} = \text{diag}(1, 1, 1/2)$. Equivalente a la norma de Frobenius del tensor deformación 2D plana ($\boldsymbol{\varepsilon} : \boldsymbol{\varepsilon}$ con $\varepsilon_{xy} = \gamma_{xy}/2$).

Limitación: no distingue tracción de compresión. Para hormigón a compresión se necesitaría un split tipo Mazars o de Vree; aquí se diferencia explícitamente como *out-of-scope*.

8.14.5 4. Variable histórica y condición de carga (Kuhn-Tucker)

$$\kappa_{n+1} = \max(\kappa_n, \varepsilon_{\text{eq}}(\boldsymbol{\varepsilon}_{n+1})), \quad \kappa_0 \text{ inicial}$$

Esto codifica:

- **Carga activa** ($\dot{\kappa} > 0$): κ crece con ε_{eq} ; el daño puede aumentar.
- **Descarga / recarga** ($\dot{\kappa} = 0$): κ queda congelado; el daño no cambia.
- **Irreversibilidad**: κ es monótono no decreciente.

8.14.6 5. Ley de evolución del daño

$$d(\kappa) = \begin{cases} 0 & \kappa \leq \kappa_0 \\ 1 - \frac{\kappa_0}{\kappa} e^{-\alpha(\kappa - \kappa_0)} & \kappa > \kappa_0 \end{cases}$$

con saturación numérica $d \leq d_{\max} = \text{DAMAGE_MAX}$ (`solidum.constants.DAMAGE_MAX`, evita rigidez nula).

Propiedades:

- $d(\kappa_0) = 0$ (continuidad).
- $d'(\kappa) > 0$ para $\kappa > \kappa_0$ (irreversibilidad).
- $\lim_{\kappa \rightarrow \infty} d = 1$ (asintótico).
- α controla la velocidad de degradación (ablandamiento más abrupto si α grande).

8.15 Formulación numérica

8.15.1 6. Algoritmo en un paso

Dado $\{\kappa_n\}$ y ε_{n+1} :

1. $\varepsilon_{eq} = \sqrt{\varepsilon_{xx}^2 + \varepsilon_{yy}^2 + \frac{1}{2}\gamma_{xy}^2}$.
2. Si $\varepsilon_{eq} > \kappa_n$: **carga activa**, $\kappa_{n+1} = \varepsilon_{eq}$. Si no: **descarga**, $\kappa_{n+1} = \kappa_n$.
3. Aplicar ley de daño con κ_{n+1} ; saturar a $d_{\max}$.
4. $\sigma_{n+1} = (1 - d) \mathbf{C}_e \varepsilon_{n+1}$.
5. Tangente: **secante** en descarga / sin daño / saturación; **algorítmica consistente** en carga activa con daño no saturado (ver §7).

El algoritmo es **explícito**: no requiere Newton local. La actualización de κ y d es directa.

8.15.2 7. Tangente algorítmica consistente (ampliación)

$$\mathbf{C}^{\text{alg}} = \frac{\partial \sigma_{n+1}}{\partial \varepsilon_{n+1}}$$

Derivando $\sigma = (1 - d) \mathbf{C}_e \varepsilon$:

$$\mathbf{C}^{\text{alg}} = (1 - d) \mathbf{C}_e - \sigma^{\text{eff}} \otimes \frac{\partial d}{\partial \varepsilon}$$

con $\sigma^{\text{eff}} = \mathbf{C}_e \boldsymbol{\varepsilon}$. La derivada del daño respecto a la deformación, por regla de la cadena:

$$\frac{\partial d}{\partial \boldsymbol{\varepsilon}} = \frac{\partial d}{\partial \kappa} \frac{\partial \kappa}{\partial \boldsymbol{\varepsilon}}$$

donde:

- $\frac{\partial d}{\partial \kappa} = (1-d) \left(\frac{1}{\kappa} + \alpha \right)$ para la ley exponencial. (Demostración: derivando $d = 1 - (\kappa_0/\kappa)e^{-\alpha(\kappa-\kappa_0)}$ y reagrupando vía $1 - d = (\kappa_0/\kappa)e^{-\alpha(\kappa-\kappa_0)}$.)
- $\frac{\partial \kappa}{\partial \boldsymbol{\varepsilon}}$ depende del régimen:
- **Carga activa** ($\varepsilon_{\text{eq}} > \kappa_n$, $\kappa = \varepsilon_{\text{eq}}$):

$$\frac{\partial \kappa}{\partial \boldsymbol{\varepsilon}} = \frac{\partial \varepsilon_{\text{eq}}}{\partial \boldsymbol{\varepsilon}} = \frac{1}{\varepsilon_{\text{eq}}} \begin{bmatrix} \varepsilon_{xx} \\ \varepsilon_{yy} \\ \frac{1}{2}\gamma_{xy} \end{bmatrix}$$

- **Descarga** ($\varepsilon_{\text{eq}} \leq \kappa_n$): $\partial \kappa / \partial \boldsymbol{\varepsilon} = 0$.

Tangente final:

$$\mathbf{C}^{\text{alg}} = \begin{cases} \mathbf{C}_e & \kappa \leq \kappa_0 \text{ (sin daño)} \\ (1-d) \mathbf{C}_e & \text{descarga} \\ (1-d) \mathbf{C}_e - \frac{(1-d)(1/\kappa + \alpha)}{\varepsilon_{\text{eq}}} (\mathbf{C}_e \boldsymbol{\varepsilon})(\mathbf{M} \boldsymbol{\varepsilon})^\top & \text{carga activa, } d < d_{\text{max}} \\ (1-d_{\text{max}}) \mathbf{C}_e & d = d_{\text{max}} \text{ (saturación)} \end{cases}$$

con $\mathbf{M} = \text{diag}(1, 1, 1/2)$.

8.15.3 8. Pérdida de simetría

El producto externo $(\mathbf{C}_e \boldsymbol{\varepsilon})(\mathbf{M} \boldsymbol{\varepsilon})^\top$ es una matriz 3×3 que **no es simétrica en general**. $\mathbf{C}_e \boldsymbol{\varepsilon}$ y $\mathbf{M} \boldsymbol{\varepsilon}$ no son proporcionales salvo en estados de deformación muy particulares (e.g. uniaxial puro alineado con ejes).

Esto rompe la simetría heredada de la tangente secante $(1-d)\mathbf{C}_e$:

- **Tangente secante:** simétrica. `IS_SYMMETRIC = True` sería válido.
- **Tangente consistente:** no simétrica en general. `IS_SYMMETRIC = False` obligatorio.

Consecuencia algebraica (ADR 0003): el despachador elige LU en lugar de Cholesky/LDL[⊤] para sistemas globales que incluyan este material. El coste por paso aumenta ~30-50 % respecto a Cholesky, pero la convergencia cuadrática del Newton global con tangente consistente recupera ese coste con creces (típicamente 2-3 iteraciones en lugar de 5-10 con tangente secante).

8.15.4 9. Decisión de régimen durante el Newton

Durante una iteración del Newton global, el material recibe un $\boldsymbol{\varepsilon}_{\text{trial}}$ y debe decidir carga/-descarga comparando con κ_n (el estado committed del paso anterior). La transición entre ramas (loading ↔ unloading) es **discontinua en la tangente**: al cruzar $\varepsilon_{\text{eq}} = \kappa_n$ el segundo término aparece/desaparece bruscamente. En la práctica esto no genera problemas porque el Newton converge antes de oscilar entre ramas; en problemas patológicos puede requerir continuation o arc-length.

8.16 Contrato de implementación

```

name: IsotropicDamage2D
kind: material
status: validated

interface:
  strain_dim: 3
  primary_state_var: damage
  is_symmetric: false      # tangente consistente no simétrica (ver §8)

parameters:
  - { name: E,          type: float, required: true,  desc: "Módulo de Young intacto" }
  - { name: nu,         type: float, required: true,  desc: "Coeficiente de Poisson" }
  - { name: kappa_0,    type: float, required: true,  desc: "Umbral de deformación
    equivalente; daño nulo mientras _0" }
  - { name: alpha,      type: float, required: true,  desc: "Velocidad de degradación
    exponencial (>0)" }
  - { name: hypothesis, type: str,   required: false, default: "plane_stress", desc: "
    plane_stress | plane_strain (delegado a Elastic2D interno)" }
  - { name: density,    type: float, required: false, default: null, desc: "Densidad (
    ADR 0008)" }

signature:
  compute_state: " (: ndarray(3,), state_vars=None) -> (: ndarray(3,), C_tan: ndarray
    (3,3), state_vars)"
  strain_kind: "plano Voigt - [_xx, _yy, _xy] con _xy = 2·_xy"

state_schema:
  kappa: "float    _0 - máxima deformación equivalente histórica"
  damage: "float    [0, DAMAGE_MAX] - variable de daño escalar isótropa"

conventions:
  sign: " > 0  tracción; el modelo no distingue tracción de compresión en la activació
    n del daño"
  voigt: "_xy = 2·_xy en deformación; _eq usa norma M=diag(1,1,1/2)"
  damage_irreversibility: " monótono no decreciente vía max(_old, _eq); d() creciente
    "

validity:
  - "E > 0, _0 > 0,    > 0"
  - "    (-1, 0.5) (delegación a Elastic2D)"
  - "hypothesis  {'plane_stress', 'plane_strain'}"

out_of_scope:
  - "split tracción/compresión (Mazars, de Vree, Mises modificado); el modelo daña por
    igual en cualquier régimen"
  - "anisotropía del daño (tensor de daño D vs escalar)"
  - "regularización para evitar mesh-dependency en regime de ablandamiento (longitud
    característica, no-local, gradient)"
  - "acoplamiento con plasticidad"
  - "fatiga / acumulación cíclica del daño"
  - "tangente consistente para IsotropicDamage1D - fuera del alcance de esta spec;
    misma deuda gemela"

numerical_caveats:
  - "Tangente consistente NO simétrica → IS_SYMMETRIC=False obliga al despachador
    algebraico a usar LU (ADR 0003). Costo por paso ~30-50% mayor que Cholesky pero

```

- convergencia cuadrática del Newton global lo compensa."
- "Transición loadingunloading es discontinua en la tangente. Newton converge antes de oscilar entre ramas en casos típicos. Para problemas patológicos cerca del umbral, considerar arc-length."
- "Saturación $d=DAMAGE_MAX$ corta la corrección consistente (d / deja de ser efectivamente $(1-d)$ ($1/+$)); el algoritmo devuelve tangente secante en ese caso para evitar términos espurios."
- "Sin regularización, la solución es mesh-dependent en régimen de ablandamiento (localización en una banda de elementos). Para benchmark cuantitativo se requiere mismo refinamiento de malla; para comparaciones cualitativas (presencia de banda, dirección) basta con regularización de gradiente o longitud característica (fuera de scope)."

acceptance:

```
# Cubierto por tests/test_materials_unit.py · TestIsotropicDamage2D (preexistentes)
# y tests/test_damage2d_consistent_tangent.py (nuevos)
unit_existing_no_regression:
- name: secant_response_below_threshold
  expect: " _0 d=0, = C_e · , tangente = C_e (sin cambio respecto a versión
           secante anterior)"
  tol_rel: 1.0e-12

- name: damage_evolution_exponential
  expect: "para > _0, d cumple ley exponencial dentro de la fórmula"
  tol_rel: 1.0e-10

unit_consistent_tangent:
- name: tangent_equals_secant_on_unloading
  setup: "carga hasta >_0, descarga con de menor norma"
  expect: "C_tan = (1-d)·C_e exacto (segundo término se anula con / = 0)"
  tol_rel: 1.0e-12

- name: tangent_equals_secant_below_threshold
  setup: " pequeño, _old = _0"
  expect: "C_tan = C_e (sin daño activo, ni secante con d=0 ni corrección)"
  tol_rel: 1.0e-12

- name: tangent_equals_secant_at_saturation
  setup: "carga hasta enorme tal que d alcanza DAMAGE_MAX"
  expect: "C_tan = (1-DAMAGE_MAX)·C_e (el término consistente se corta porque d /
           efectivamente 0 al saturar)"
  tol_rel: 1.0e-12

- name: tangent_consistent_term_on_loading
  setup: "carga activa con > _0 y d < DAMAGE_MAX"
  expect: "C_tan (1-d)·C_e - incluye el término rank-1 -(1-d) (1/+) /_eq · (Ce · )
           (M · )"
  verification: "compara contra fórmula explícita evaluada en el mismo estado"
  tol_rel: 1.0e-9

- name: tangent_not_symmetric_in_loading
  setup: "carga activa con estado de deformación no degenerado (_xx _yy, _xy 0)
           "
  expect: "C_tan - C_tan^T 0 (al menos un elemento off-diagonal con diferencia
           significativa)"
  verification: "||C_tan - C_tan^T||_F / |||C_tan_F 0.01 en el caso de prueba"

- name: tangent_finite_difference_consistency
```



```

setup: "carga activa; calcular tangente por diferencia finita centrada (
    perturbación 1e-7) y comparar con cerrada"
expect: "||C_tan_FD -|| C_tan_closed_form / |||C_tan < 1e-5"
tol_rel: 1.0e-5

unit_arch:
- name: is_symmetric_attribute_is_false
  expect: "IsotropicDamage2D.IS_SYMMETRIC == False (atributo declarado en clase)"

integration:
- name: quad4_damage_newton_converges_in_few_iter
  setup: "Quad4 unitario, IsotropicDamage2D plane stress, carga uniaxial hasta régimen plenamente dañado"
  expect: "Newton converge en 6 iteraciones por paso (evidencia de convergencia cuadrática con tangente consistente)"

references:
- "Lemaitre J., Chaboche J.-L. (1990). Mechanics of Solid Materials. Cambridge UP. Cap. 7 (continuum damage mechanics)."
```

```

- "Oliver J. (1989). A consistent characteristic length for smeared cracking models. IJNME 28, 461-474."
```

```

- "Simó J.C., Ju J.W. (1987). Strain- and stress-based continuum damage models - I. Formulation. Int. J. Solids Struct. 23, 821-840 (tangente consistente para daño isótropo)."
```

```

- "de Souza Neto E.A., ċPeri D., Owen D.R.J. (2008). Computational Methods for Plasticity. Wiley. §12 (damage models)."
```

8.17 Implementación

- **Archivo:** solidum/materials/damage_2d.py.
- **Clase:** IsotropicDamage2D, registrada vía @MaterialRegistry.register. Declara IS_SYMMETRIC = False como ClassVar (sobreescribe el default True de Material).
- **Estado:** dict {'kappa': float, 'damage': float}. PRIMARY_STATE_VAR = 'damage'.
- **Algoritmo compute_state:**
 1. Recuperar κ_n y calcular $\varepsilon_{eq}(\varepsilon)$.
 2. Si $\varepsilon_{eq} > \kappa_n$ y $\kappa_{n+1} > \kappa_0$: rama de carga activa.
 3. Calcular d por la ley exponencial, saturar a DAMAGE_MAX.
 4. Calcular $\sigma = (1 - d)\mathbf{C}_e\varepsilon$.
 5. Tangente: rama según §7 (secante o consistente). En la rama consistente, calcular el término rank-1 una sola vez y restar a $(1 - d)\mathbf{C}_e$.
- **admissibility_scale:** sin cambios respecto a versión previa, $E \cdot \kappa_0$ (umbral inicial — la escala no evoluciona con el estado porque el criterio se mide contra κ_0 , no contra κ).
- **Compatibilidad:** el cambio rompe IS_SYMMETRIC para este material. El despachador algebraico (ADR 0003) verifica domain_is_symmetric agregando material.IS_SYMMETRIC sobre todos los materiales del dominio; basta un material con IS_SYMMETRIC=False para que el solver elija LU.

- **Tests:**
- tests/test_materials_unit.py · **TestIsotropicDamage2D** (existente, no regresión) + nueva clase **TestIsotropicDamage2DConsistentTangent**.
- tests/test_solid_2d_damage.py nuevo: integración Quad4 + IsotropicDamage2D + Nonlinear-Solver con tangente consistente.

8.18 Diálogo

- **2026-05-14** · Spec creada retroactivamente sobre material existente (tangente secante) y extendida con tangente algorítmica consistente. Sin ADR: extiende un material existente añadiendo un término a la tangente; no rompe contratos arquitecturales transversales. Sí cambia **IS_SYMMETRIC** para este material, lo cual el despachador algebraico ya soporta vía agregación sobre el dominio (no requiere refactor).
- **2026-05-14** · Decisión: la transición loading $\leftrightarrow$ unloading se decide con κ_n committed (no con κ trial dentro del Newton). Esto es lo estándar y evita oscilaciones del Newton entre ramas en pasos con ambigüedad. Si el paso converge, κ se compromete y la decisión es consistente con la trayectoria.
- **2026-05-14** · **IsotropicDamage1D** queda con tangente secante. Misma deuda gemela conceptual, pero fuera del alcance pactado (A2 era solo 2D). Si el usuario la prioriza luego, replicar es trivial: misma fórmula reducida a escalar.

8.19 VonMises2D

*Orden de trabajo. El usuario escribe **especificación física, formulación numérica y contrato**; la IA rellena **implementación** y responde en **diálogo**.*

*>Spec **retroactiva con ampliación**: la hipótesis **plane_strain** está implementada y testada desde sesiones anteriores; esta spec la documenta y añade **plane_stress** como hipótesis a implementar en esta sesión.*

8.20 Especificación física

8.20.1 0. Descripción general

Modelo elastoplástico independiente de la velocidad (rate-independent”) basado en el criterio **J2 / Von Mises** con regla de flujo **asociada** y **endurecimiento isótropo lineal**. Captura plasticidad de metales dúctiles en régimen de pequeñas deformaciones, sin efectos cinemáticos (Bauschinger), térmicos ni viscosos.

Soporta dos hipótesis cinemáticas 2D mutuamente excluyentes seleccionadas en construcción:

- **plane_strain** — $\varepsilon_{zz} = \varepsilon_{xz} = \varepsilon_{yz} = 0$. Cuerpos prismáticos largos cargados transversalmente (presa, túnel, cilindro largo).
- **plane_stress** — $\sigma_{zz} = \sigma_{xz} = \sigma_{yz} = 0$. Láminas o membranas delgadas en su plano (chapa traccionada, placa fina).

8.20.2 1. Descomposición aditiva

En pequeñas deformaciones, el tensor de deformación se descompone aditivamente:

$$\boldsymbol{\varepsilon} = \boldsymbol{\varepsilon}^e + \boldsymbol{\varepsilon}^p$$

La parte elástica $\boldsymbol{\varepsilon}^e$ gobierna el esfuerzo vía la ley elástica isótropa; la parte plástica $\boldsymbol{\varepsilon}^p$ es **histórica** y evoluciona según la regla de flujo.

8.20.3 2. Ley elástica

$$\boldsymbol{\sigma} = \mathbf{C}_e : \boldsymbol{\varepsilon}^e = \mathbf{C}_e : (\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}^p)$$

con $\mathbf{C}_e$ tensor de rigidez elástica isótropa parametrizado por (E, ν) o equivalentemente (K, G) :

$$K = \frac{E}{3(1-2\nu)}, \quad G = \frac{E}{2(1+\nu)}$$

8.20.4 3. Superficie de fluencia (J2)

$$f(\boldsymbol{\sigma}, \alpha) = \|\mathbf{s}\| - \sqrt{\frac{2}{3}} (\sigma_y + H \alpha) \leq 0$$

donde $\mathbf{s} = \boldsymbol{\sigma} - \frac{1}{3} \text{tr}(\boldsymbol{\sigma}) \mathbf{I}$ es el tensor desviador, $\|\mathbf{s}\| = \sqrt{\mathbf{s} : \mathbf{s}}$, $\sigma_y > 0$ es la fluencia inicial y $H \geq 0$ el módulo de endurecimiento isótropo lineal. El factor $\sqrt{2/3}$ alinea σ_y con la fluencia uniaxial estándar.

8.20.5 4. Regla de flujo (asociada)

$$\dot{\epsilon}^p = \dot{\gamma} \frac{\partial f}{\partial \boldsymbol{\sigma}} = \dot{\gamma} \mathbf{N}, \quad \mathbf{N} = \frac{\mathbf{s}}{\|\mathbf{s}\|}$$

El flujo es puramente desviador ($\text{tr}(\mathbf{N}) = 0$) — **plasticidad incompresible**, propiedad central de J2.

8.20.6 5. Endurecimiento isótropo lineal

$$\dot{\alpha} = \sqrt{\frac{2}{3}} \dot{\gamma}$$

α es la **deformación plástica acumulada equivalente** (escalar, monótona no decreciente). La superficie de fluencia se expande uniformemente con α manteniendo su centro en el origen del espacio de tensiones desviadoras.

8.20.7 6. Condiciones de Kuhn-Tucker

$$\dot{\gamma} \geq 0, \quad f \leq 0, \quad \dot{\gamma} f = 0$$

Garantizan que $\dot{\gamma} > 0$ solo si el estado intenta salir de la superficie, y que el estado real siempre cumple $f \leq 0$.

8.20.8 7. Variables internas

Símbolo	Tipo	Significado
ϵ^p	tensor (4 componentes Voigt extendido en plane strain, ver §8)	deformación plástica
α	escalar	deformación plástica acumulada equivalente

`alpha` es la variable principal exportable (`PRIMARY_STATE_VAR = 'alpha'`); `eps_p` es interna al algoritmo de return mapping.

8.20.9 8. Notación Voigt y manejo de ϵ_{zz}^p

Convención del proyecto para 2D: $\boldsymbol{\epsilon} = [\epsilon_{xx}, \epsilon_{yy}, \gamma_{xy}]$ con $\gamma_{xy} = 2\epsilon_{xy}$. Pero la **descomposición desviadora** requiere todas las componentes 3D, así que internamente el material maneja $\boldsymbol{\epsilon}^p = [\epsilon_{xx}^p, \epsilon_{yy}^p, \epsilon_{zz}^p, \epsilon_{xy}^p]$ (4 componentes, ϵ_{xy}^p tensorial sin factor 2).

- **Plane strain:** $\epsilon_{zz} = 0$ se impone *en deformación total*. La componente ϵ_{zz}^p es no nula y evoluciona libremente (la incompresibilidad plástica acopla $\epsilon_{zz}^p = -(\epsilon_{xx}^p + \epsilon_{yy}^p)$). Por simetría, σ_{zz} resulta no nulo pero no se expone en el API público 2D — vive solo en el cómputo interno.
- **Plane stress:** $\sigma_{zz} = 0$ se impone *en esfuerzo*. La componente ϵ_{zz} es no nula (extensión transversal de Poisson + Poisson plástico), y aparece como variable adicional en el problema local de retorno (ver §10).

8.21 Formulación numérica

8.21.1 9. Esquema temporal — Backward Euler implícito

Dado el estado convergido $\{\epsilon_n^p, \alpha_n\}$ al paso n y la deformación total prescrita ϵ_{n+1} , resolver para $\{\sigma_{n+1}, \epsilon_{n+1}^p, \alpha_{n+1}\}$ con discretización implícita:

$$\epsilon_{n+1}^p = \epsilon_n^p + \Delta\gamma \mathbf{N}_{n+1}, \quad \alpha_{n+1} = \alpha_n + \sqrt{\frac{2}{3}} \Delta\gamma$$

más $f(\sigma_{n+1}, \alpha_{n+1}) \leq 0$ con la condición de complementariedad.

8.21.2 10. Return mapping plane strain — algoritmo radial cerrado (existente)

Descomposición volumétrica-desviadora del estado de prueba:

$$\epsilon_{n+1}^{\text{trial}} = \epsilon_{n+1} - \epsilon_n^p, \quad \mathbf{s}^{\text{trial}} = 2G \mathbf{e}^{\text{trial}}, \quad p^{\text{trial}} = K \text{tr}(\epsilon_{n+1})$$

donde $\mathbf{e}^{\text{trial}}$ es la parte desviadora de $\epsilon_{n+1}^{\text{trial}}$. Función de fluencia trial:

$$f^{\text{trial}} = \|\mathbf{s}^{\text{trial}}\| - \sqrt{\frac{2}{3}} (\sigma_y + H \alpha_n)$$

Si $f^{\text{trial}} \leq \text{tol} \rightarrow$ paso elástico, $\sigma_{n+1} = \mathbf{C}_e : \epsilon_{n+1}$, estado intacto.

Si no, el flujo es radial (la dirección no cambia entre trial y final por isotropía + asociado en J2):

$$\Delta\gamma = \frac{f^{\text{trial}}}{2G + \frac{2}{3}H} \quad (\text{forma cerrada})$$

$$\mathbf{N} = \frac{\mathbf{s}^{\text{trial}}}{\|\mathbf{s}^{\text{trial}}\|}, \quad \mathbf{s}_{n+1} = \mathbf{s}^{\text{trial}} - 2G \Delta\gamma \mathbf{N}, \quad \sigma_{n+1} = \mathbf{s}_{n+1} + p^{\text{trial}} \mathbf{I}$$

Tangente consistente algorítmica (derivada del algoritmo, no de la ley continua):

$$\mathbf{C}^{\text{alg}} = K \mathbf{1} \otimes \mathbf{1} + 2G(1 - \beta) \mathbf{I}_{\text{dev}} - 2G \bar{\gamma} \mathbf{N} \otimes \mathbf{N}$$

$$\text{con } \beta = \frac{2G \Delta\gamma}{\|\mathbf{s}^{\text{trial}}\|}, \quad \bar{\gamma} = \frac{1}{1 + H/(3G)} - \beta.$$

8.21.3 11. Return mapping plane stress — algoritmo proyectado (a implementar)

Referencia: Simó & Hughes 1998, *Computational Inelasticity*, §3.4.1 ("Plane stress projected algorithm").

La restricción $\sigma_{zz} = 0$ se impone **eliminando** ϵ_{zz} del problema mediante un operador de proyección, no como ecuación extra. Resultado: el problema local sigue siendo escalar en $\Delta\gamma$, sin nested Newton.

Matriz elástica plane stress (3×3 , base Voigt $[\epsilon_{xx}, \epsilon_{yy}, \gamma_{xy}]$):

$$\mathbf{C}_e^{ps} = \frac{E}{1 - \nu^2} \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & (1 - \nu)/2 \end{bmatrix}$$

Predictor elástico en el subespacio plane stress:

$$\sigma^{\text{trial}} = \mathbf{C}_e^{ps} (\epsilon_{n+1} - \mathbf{P}_\epsilon \epsilon_n^p)$$

con $\mathbf{P}_\varepsilon$ extrayendo las componentes plane $[\varepsilon_{xx}^p, \varepsilon_{yy}^p, 2\varepsilon_{xy}^p]$ del tensor 4D.

Operador $\mathbf{P}$ — norma desviadora bajo $\sigma_{zz} = 0$ (Simó-Hughes Box 3.1):

$$\mathbf{P} = \frac{1}{3} \begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & 0 \\ 0 & 0 & 6 \end{bmatrix}$$

Esta matriz codifica el producto escalar desviador en el subespacio plane stress; reemplaza $\|\mathbf{s}\|^2$ por $\boldsymbol{\sigma}^\top \mathbf{P} \boldsymbol{\sigma}$ con todas las componentes 3D ya eliminadas analíticamente.

Función de fluencia plane stress proyectada:

$$\bar{f}(\boldsymbol{\sigma}, \alpha) = \frac{1}{2} \boldsymbol{\sigma}^\top \mathbf{P} \boldsymbol{\sigma} - \frac{1}{3} (\sigma_y + H \alpha)^2 \leq 0$$

Esquema de retorno:

$$\boldsymbol{\sigma}_{n+1} = [\mathbf{C}_e^{ps}]^{-1} (\boldsymbol{\sigma}^{\text{trial}}) - \Delta\gamma \mathbf{P} \boldsymbol{\sigma}_{n+1}$$

reordenando:

$$\boldsymbol{\sigma}_{n+1} = \mathbf{A}(\Delta\gamma)^{-1} \boldsymbol{\sigma}^{\text{trial}}, \quad \mathbf{A}(\Delta\gamma) = \mathbf{I} + \Delta\gamma \mathbf{C}_e^{ps} \mathbf{P}$$

La matriz $\mathbf{A}$ es 3×3 , diagonalizable en los autovectores ortonormales de $\mathbf{C}_e^{ps} \mathbf{P}$. Los autovalores son cerrados:

$$\mu_1 = \frac{E}{3(1-\nu)}, \quad \mu_2 = 2G, \quad \mu_3 = 2G$$

con autovectores $\mathbf{v}_1 = \frac{1}{\sqrt{2}}[1, 1, 0]$ (hidrostático plano), $\mathbf{v}_2 = \frac{1}{\sqrt{2}}[1, -1, 0]$ (desviador plano), $\mathbf{v}_3 = [0, 0, 1]$ (cortante). Diagonalizan simultáneamente $\mathbf{C}_e^{ps}$ y $\mathbf{P}$, con $\mathbf{v}_i^\top \mathbf{P} \mathbf{v}_i \in \{1/3, 1, 2\}$.

Actualización de la deformación plástica acumulada equivalente. La regla $\dot{\alpha} = \sqrt{2/3} \dot{\gamma}$ usada en plane strain **no** es válida aquí: en plane stress projected $\dot{\gamma}$ tiene unidades [1/esfuerzo] (porque $\dot{\varepsilon}^p = \dot{\gamma} \mathbf{P} \boldsymbol{\sigma}$ con $\mathbf{P} \boldsymbol{\sigma}$ en unidades de esfuerzo). La fórmula físicamente correcta y dimensionalmente consistente, obtenida de $\dot{\alpha} = \sqrt{2/3} \|\dot{\varepsilon}^p\|_{\text{Frob}}$:

$$\alpha_{n+1} = \alpha_n + \Delta\gamma \sqrt{\frac{2}{3} \boldsymbol{\sigma}_{n+1}^\top \mathbf{P} \boldsymbol{\sigma}_{n+1}}$$

Newton local sobre $\Delta\gamma$. En base autovectores $\sigma_i(\Delta\gamma) = a_i/(1 + \Delta\gamma \mu_i)$, así $\boldsymbol{\sigma}^\top \mathbf{P} \boldsymbol{\sigma}$ se expresa cerrado:

$$\boldsymbol{\sigma}^\top \mathbf{P} \boldsymbol{\sigma}(\Delta\gamma) = \frac{a_1^2}{3(1 + \Delta\gamma \mu_1)^2} + \frac{a_2^2}{(1 + \Delta\gamma \mu_2)^2} + \frac{2 a_3^2}{(1 + \Delta\gamma \mu_3)^2}$$

donde a_i son las proyecciones de $\boldsymbol{\sigma}^{\text{trial}}$ en los autovectores. La ecuación residual:

$$\bar{f}(\Delta\gamma) = \frac{1}{2} \boldsymbol{\sigma}^\top \mathbf{P} \boldsymbol{\sigma}(\Delta\gamma) - \frac{1}{3} (\sigma_y + H \alpha(\Delta\gamma))^2 = 0$$

con $\alpha(\Delta\gamma) = \alpha_n + \Delta\gamma \sqrt{(2/3) \boldsymbol{\sigma}^\top \mathbf{P} \boldsymbol{\sigma}}$ es monótona decreciente; tangente cerrada con regla de la cadena. **Newton local converge** típicamente en 3-6 iteraciones desde $\Delta\gamma = 0$. Coste: $O(1)$ por punto de Gauss, comparable a plane strain.

Tangente consistente plane stress. Sea $\mathbf{D} = \mathbf{A}^{-1} \mathbf{C}_e^{ps}$, $\mathbf{q} = \boldsymbol{\sigma}_{n+1}^\top \mathbf{P} \mathbf{D} \mathbf{P} \boldsymbol{\sigma}_{n+1}$, $w = \sqrt{(2/3) \boldsymbol{\sigma}_{n+1}^\top \mathbf{P} \boldsymbol{\sigma}_{n+1}}$, $m = (2/3) R H$, $n = 2\Delta\gamma/(3w)$. De la condición de consistencia ($\dot{f} = 0$) con la regla correcta de α :

$$\mathbf{C}_{ps}^{\text{alg}} = \mathbf{D} - \frac{(\mathbf{D} \mathbf{P} \boldsymbol{\sigma}_{n+1}) (\boldsymbol{\sigma}_{n+1}^\top \mathbf{P} \mathbf{D})}{q + m w / (1 - m n)}$$

Si $H = 0$ (plasticidad perfecta) el término m se anula y el denominador se reduce a q . Forma cerrada — sin diferenciación numérica.

Predictor elástico con $\varepsilon^p \neq 0$. Tanto en plane strain como en plane stress, el predictor debe construirse como $\boldsymbol{\sigma}^{\text{trial}} = \mathbf{C}_e (\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_n^p)$ (o equivalente desviador + presión), **no** como $\mathbf{C}_e \boldsymbol{\varepsilon}$, que ignoraría la deformación plástica acumulada. Importante en descargas elásticas desde estado plástico y al reevaluar el material con `compute_gauss_state(U_final)` tras un análisis converged.

8.21.4 12. Tolerancia de fluencia

Vía ADR 0006 (`Material.admissibility_tol`), patrón `atol + rtol·escala`:

- **Plane strain:** `admissibility_scale =  $\sqrt{(2/3)(\sigma_y + H \cdot \alpha)}$` (norma desviadora característica en la frontera).
- **Plane stress:** `admissibility_scale =  $(\sigma_y + H \cdot \alpha)^2/3$` (escala de $\bar{f}$ proyectada; los dos términos son del mismo orden).

8.22 Contrato de implementación

```
name: VonMises2D
kind: material
status: validated

interface:
  strain_dim: 3
  primary_state_var: alpha    # deformación plástica acumulada equivalente

parameters:
  - { name: E,           type: float, required: true,  desc: "Módulo de Young" }
  - { name: nu,          type: float, required: true,  desc: "Coeficiente de Poisson" }
  - { name: sigma_y,      type: float, required: true,  desc: "Tensión de fluencia inicial" }
  - { name: H,           type: float, required: false, default: 0.0, desc: "Módulo de endurecimiento isótropo lineal (0). H=0 plasticidad perfecta" }
  - { name: hypothesis,   type: str,   required: false, default: "plane_strain", desc: "plane_strain | plane_stress" }
  - { name: density,      type: float, required: false, default: null, desc: "Densidad (kg/m³). Opcional al construir; obligatoria solo si se ensambla peso propio o masa (ADR 0008)" }

signature:
  compute_state: "(: ndarray(3,), state_vars=None) -> (: ndarray(3,), C_alg: ndarray(3,3), state_vars)"
  strain_kind: "plano Voigt - [_xx, _yy, _xy] con _xy = 2·_xy"

state_schema:
  eps_p: "ndarray(4,) - [^p_xx, ^p_yy, ^p_zz, ^p_xy] (xy tensorial, sin factor 2)"
  alpha: "float 0 - deformación plástica acumulada equivalente"

conventions:
  sign: "> 0 tracción (Reglas §5)"
```

```

voigt: "_xy = 2·_xy en deformación total; ^p_xy almacenada en componente tensorial (
    sin factor 2)"
hypothesis_dispatch: "selección por kwarg al construir; mutuamente excluyentes"

validity:
- "E > 0, _y > 0, H = 0"
- "    (-1, 0.5) en plane_strain;    (-1, 0.5) en plane_stress (=0.5 no admitido por
    singularidad)"
- "|| 1e-2 (régimen de pequeñas deformaciones)"
- "hypothesis {'plane_strain', 'plane_stress'}"

out_of_scope:
- "endurecimiento cinemático (back-stress); usar variante futura VonMises2DKinematic"
- "endurecimiento isótropo no lineal (Voce, exponencial)"
- "endurecimiento por ablandamiento (H<0); requiere regularización (longitud caracter
    ística, no-local)"
- "regla de flujo no asociada"
- "acoplamiento con daño; usar modelo plástico-dañado dedicado"
- "viscoplasticidad y dependencia de la velocidad de deformación"
- "grandes deformaciones (descomposición multiplicativa F = Fe·Fp)"
- "anisotropía (Hill, Barlat) - el modelo es estrictamente J2 isótropo"

numerical_caveats:
- "Plane strain: incompresibilidad plástica genera locking volumétrico en elementos
    de bajo orden (Quad4, Tri3) cuando →0.5 o el campo plástico domina. Mitigaciones
    futuras: B-bar, F-bar, elementos mixtos u-p. No incluidas en esta spec."
- "Plane stress: el Newton local sobre Δ asume H = 0; con H = 0 (perfectamente plá
    stico) la ecuación residual sigue siendo monótona pero su pendiente al final del
    paso plástico tiende a 0 - convergencia más lenta. Mitigación: bisección de
    respaldo si Newton local diverge."
- "Para K cercano a 0.5 en plane strain, K →∞ y el predictor elástico pierde precisió
    n numérica. Recomendado K = 0.49 en metales (típico K = 0.3)."
```

acceptance:

```

# Cubierto por tests/test_materials_unit.py · TestVonMises2D y
  TestVonMises2DPlaneStress
unit_plane_strain:
- name: paso_elastico_bajo_fluencia
  expect: "[1e-5,0,0]: =C_e· exacto, alpha=0, tangente=C_e"
  tol_rel: 1.0e-10
- name: fluencia_traccion_uniaxial_confinada
  expect: "fluencia activa con eps=[0.01,-0.01,0]: alpha>0 tras un solo paso"
- name: monotonicidad_alpha
  expect: "no decrece bajo carga creciente con eps_yy=-; _final > 0"
- name: no_further_yield_on_frontier
  expect: "repetir en frontera de fluencia no incrementa (return mapping
    consistente)"
  tol_rel: 1.0e-10
- name: hypothesis_validation
  expect: "constructor rechaza hypothesis distinto de 'plane_strain'/'plane_stress'
    con ValueError"

unit_plane_stress:
- name: paso_elastico_bajo_fluencia
  expect: "pequeño: =C_e^ps·, tangente=C_e^ps, alpha=0, eps_p=0"
  tol_rel: 1.0e-10
- name: degeneracion_a_1d_traccion_pura_elastico
  expect: "=(eps,-eps,0): _xx=E·eps, _yy=0 (regimen elástico, no E/(1-ν²))"
  tol_rel: 1.0e-4

```



```

- name: yield_uniaxial_at_sigma_y
  expect: "primer yield en _xx=_y bajo tracción uniaxial pura (no E/(1-2))."
  tol_rel: 1.0e-2
- name: traccion_biaxial_isotropa
  expect: "fluencia biaxial isotropa exactamente en _xx=_yy=_y (||s||=·√(2/3)
    iguala √(2/3) _y); _yield=_y (1-)/E"
  tol_rel: 1.0e-2
- name: incompresibilidad_plastica
  expect: "tr (^p) = ^p_xx + ^p_yy + ^p_zz = 0 tras cualquier paso plástico"
  tol_abs: 1.0e-12
- name: alpha_monotonic
  expect: " no decrece bajo carga monotónica creciente"
- name: descarga_recupera_C_e_ps
  expect: "tras estado plástico, paso siguiente con menor produce tangente=C_e^ps
    y intacto"
  tol_rel: 1.0e-10
- name: tangente_simetrica
  expect: "C_alg simétrica (J2 asociado IS_SYMMETRIC=True)"
  tol_abs: 1.0e-8
- name: unit_invariance_MPa_vs_Pa
  expect: "mismo path adimensional en (MPa,mm,N) y (Pa,m,N) y ^p idénticos"
  tol_rel: 1.0e-12

# Cubierto por tests/test_solid_2d_plasticity.py
integration:
- name: pipeline_plane_strain_elastico
  setup: "Quad4 unitario, confinamiento uy=0; tracción horizontal por debajo del
    yield"
  expect: "u_x = _xx_target exacto; =0"
  tol_rel: 1.0e-8

- name: pipeline_plane_strain_plastico_vs_material_directo
  setup: "mismo setup, carga 1.5×_xx_yield → régimen plástico; integrar 10 pasos"
  expect: "_xx FEM (gauss avg) coincide con material standalone en _xx_final bajo
    trayectoria monótona; coincide"
  tol_rel: 1.0e-3

- name: pipeline_plane_stress_elastico_uniaxial
  setup: "Quad4 unitario, lado izq apoyado en ux; nodo (0,0) y (1,0) en uy; nodo
    (1,1) libre en y. Tracción horizontal por debajo del yield"
  expect: "u_x = _xx_target; contracción transversal u_y(0,1) = -·_xx (Poisson);
    =0"
  tol_rel: 1.0e-6

- name: pipeline_plane_stress_plastico_vs_1d
  setup: "mismo setup, carga 1.2_y para entrar plástico; comparar contra
    Elastoplastic1D bajo mismo path _xx"
  expect: "_xx FEM = _1D, _yy FEM 0 (uniaxial puro emergente); _FEM = _1D"
  tol_rel: 1.0e-3

- name: pipeline_converges_few_iter_plane_strain
  setup: "carga plástica significativa, 4 incrementos, max_iter=8"
  expect: "solver converge en todos los pasos sin agotar max_iter; alpha final > 0"

- name: pipeline_converges_few_iter_plane_stress
  setup: "tracción uniaxial pura plane stress 1.5_y, 4 incrementos, max_iter=8"
  expect: "solver converge en todos los pasos; alpha final > 0"

references:

```

- "Simó J.C., Hughes T.J.R. (1998). Computational Inelasticity. Springer. §3.3 (plane strain return mapping), §3.4 (plane stress projected algorithm), Box 3.1 (operador P)."
- "Crisfield M.A. (1991). Non-linear Finite Element Analysis of Solids and Structures, Vol. 1. Wiley. Cap. 6 (J2 plasticity)."
- "Chen W.F., Han D.J. (1988). Plasticity for Structural Engineers. Springer. Cap. 5 (cilindro a presión interna, sol. cerrada Hill)."
- "de Souza Neto E.A., ċPeri D., Owen D.R.J. (2008). Computational Methods for Plasticity. Wiley. §9.4 (plane stress projection alternativa)."

8.23 Implementación

- **Archivo:** solidum/materials/von_mises_2d.py.
- **Clase:** VonMises2D, registrada vía `@MaterialRegistry.register`. Despacho por `hypothesis` en construcción (sin coste runtime).
- **Kernels Numba:**
- `_compute_j2_plane_strain`: descomposición volumétrica-desviadora 3D (ε_{zz}^p libre), return mapping radial cerrado, tangente algorítmica consistente cerrada. Predictor elástico construido como $\mathbf{s}_{\text{trial}} + p\mathbf{I}$ con $\mathbf{s}_{\text{trial}} = 2G(\mathbf{e}^{\text{dev}} - \varepsilon_n^p \mathbf{e}_n)$ — respeta ε_n^p en descargas/reevaluaciones.
- `_compute_j2_plane_stress`: predictor $\boldsymbol{\sigma}^{\text{trial}} = \mathbf{C}_e^{ps}(\boldsymbol{\varepsilon} - \varepsilon_n^p \mathbf{e}_n)$, función de fluencia $\bar{f}$ proyectada, Newton local sobre $\Delta\gamma$ con tangente cerrada y máximo `_PLANE_STRESS_MAX_LOCAL_ITER` = 25 iteraciones. Construcción de $\mathbf{A}^{-1}$ y σ_{new} vía base de autovectores conocida ($\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$); incompresibilidad plástica cierra $\varepsilon_{zz}^p = -(\varepsilon_{xx}^p + \varepsilon_{yy}^p)$.
- **State schema:** `eps_p` ndarray(4), [xx, yy, zz, xy_tensorial], `alpha` escalar. Las cuatro componentes de `eps_p` son obligatorias en ambas hipótesis para preservar invariantes (incompresibilidad, normas tensoriales).
- **admissibility_scale:**
- `plane_strain`: $\sqrt{(2/3) \cdot (\sigma_y + H \cdot \alpha)}$
- `plane_stress`: $(\sigma_y + H \cdot \alpha)^2 / 3$
- **Tests:**
- `tests/test_materials_unit.py` · `TestVonMises2D` (5 tests plane strain pre-existentes, no regresión) + `TestVonMises2DPlaneStress` (9 tests plane stress).
- `tests/test_solid_2d_plasticity.py` · 6 tests de integración del pipeline Quad4 + VonMises2D + NonlinearSolver en ambas hipótesis.
- `tests/test_density_self_weight.py` · `test_vonmises2d_density` — densidad ADR 0008.
- **Notas de traducción:**
- El kernel plane stress trabaja con $\boldsymbol{\sigma}$ en Voigt mixto: las componentes $\boldsymbol{\sigma} = [\sigma_{xx}, \sigma_{yy}, \sigma_{xy}]$ son las "engineering" mientras que $\boldsymbol{\varepsilon}$ usa $\gamma_{xy} = 2\varepsilon_{xy}$. El operador $\mathbf{P}$ en esta convención tiene la forma con 6 en la entrada (3,3); $\mathbf{P}\boldsymbol{\sigma}$ devuelve la componente cortante como $2\sigma_{xy}$ (convención engineering), que se convierte a tensorial dividiendo por 2 antes de acumular en ε_{xy}^p .

- La actualización $\alpha_{n+1} = \alpha_n + \Delta\gamma\sqrt{(2/3)\boldsymbol{\sigma}^\top\mathbf{P}\boldsymbol{\sigma}}$ se evalúa **con $\boldsymbol{\sigma}$ final** del paso plástico (tras converger Newton local). Durante las iteraciones se usa la misma fórmula con el $\boldsymbol{\sigma}$ corriente.
- Validación de parámetros en constructor: $E>0$, $\sigma_y>0$, $H\geq 0$, $\nu\in(-1,0.5)$, `hypothesis` $\in$ {`plane_strain`, `plane_stress`}. Mensajes en español describiendo la violación.

8.23.1 Bug fixes detectados al implementar

- **Predictor elástico plane strain ignoraba $\boldsymbol{\varepsilon}^p$.** Versión previa devolvía $\boldsymbol{\sigma} = \mathbf{C}_e\boldsymbol{\varepsilon}$ cuando $f^{\text{trial}} \leq 0$ — correcto solo si $\boldsymbol{\varepsilon}^p = 0$. En descargas elásticas desde estado plástico, o al reevaluar el material vía `compute_gauss_state(U_final)` tras un análisis converged, devolvía $\boldsymbol{\sigma}$ inconsistente con el estado interno. Corregido a $\boldsymbol{\sigma} = \mathbf{s}_{\text{trial}} + p\mathbf{I}$. Detectado por el test de integración `test_plastic_regime_matches_standalone_material`: $\boldsymbol{\sigma}$ FEM reportado coincidía con el valor de equilibrio (F/A) en Newton (donde sí entra a return mapping), pero `compute_gauss_state` post-converged daba el valor incorrecto del predictor elástico, divergiendo del material standalone.

8.24 Diálogo

- **2026-05-14** · Spec creada retroactivamente sobre material existente (J2 plane strain) y extendida con J2 plane stress. Decisión de algoritmo plane stress: **proyección de Simó-Hughes §3.4.1** en lugar de nested Newton sobre ε_{zz} . Justificación: forma cerrada del problema local (Newton local escalar sobre $\Delta\gamma$ con tangente cerrada, 3-6 iteraciones, $O(1)$ por Gauss); Numba-compatible (kernel plano); tangente consistente cerrada vía operador $\mathbf{P}$. Nested Newton implicaría $\sim 5\text{-}10\times$ el coste por anidación de return mapping completo dentro de cada iteración externa sobre $\sigma_{zz} = 0$, justificable solo con endurecimiento no lineal complejo (Voce, plasticidad mixta) que no es el caso. Registrado aquí, no en ADR — afecta a un material, no a contratos arquitecturales transversales.
- **2026-05-14** · Corrección durante la implementación: la regla heurística $\alpha_{n+1} = \alpha_n + \sqrt{2/3}\Delta\gamma$ heredada de plane strain rompe la invariancia bajo cambio de unidades en plane stress, porque allí $\Delta\gamma$ tiene unidades [1/esfuerzo] (la regla de flujo $\dot{\boldsymbol{\varepsilon}}^p = \dot{\gamma}\mathbf{P}\boldsymbol{\sigma}$ tiene $\mathbf{P}\boldsymbol{\sigma}$ con dimensión de esfuerzo). La fórmula físicamente correcta es $\alpha_{n+1} = \alpha_n + \Delta\gamma\sqrt{(2/3)\boldsymbol{\sigma}^\top\mathbf{P}\boldsymbol{\sigma}}$ — adimensional. Detectado por el test unitario `test_unit_invariance_MPa_vs_Pa` que daba un factor $1e6$ entre los dos sistemas. Implica recalcular la tangente consistente con $\partial\alpha/\partial\Delta\gamma \neq \sqrt{2/3}$ (ver §11).
- **2026-05-14** · Corrección plane strain: el predictor elástico devolvía $\boldsymbol{\sigma} = \mathbf{C}_e\boldsymbol{\varepsilon}$, ignorando $\boldsymbol{\varepsilon}_n^p$. Bug latente preexistente sin tests que lo cubrieran (los unitarios solo probaban primer paso con $\boldsymbol{\varepsilon}^p = 0$). Detectado al validar el pipeline FEM: `compute_gauss_state(U_final)` reportaba $\boldsymbol{\sigma}$ del predictor elástico (incorrecto) en lugar del $\boldsymbol{\sigma}$ post return-mapping. Corregido a $\boldsymbol{\sigma} = \mathbf{s}_{\text{trial}} + p\mathbf{I}$. La rama plane stress ya lo hacía bien desde el principio (predictor incluía la sustracción de $\boldsymbol{\varepsilon}^p$).
- **2026-05-14** · Validación numérica de plane strain queda implícita en los tests unitarios existentes; los tests de integración `tests/test_solid_2d_plasticity.py` abren por primera vez el pipeline completo Quad4 + VonMises2D + NonlinearSolver en ambas hipótesis — hasta hoy no había cobertura de integración sólido 2D + material plástico.

- **2026-05-14** · No se introduce ADR. La spec no rompe contratos: extiende un material existente con una rama nueva del despacho por `hypothesis`. Sí se actualiza `docs/catalogo_materiales.md` al validar.

Capítulo 9

Modelos Constitutivos — 3D

9.1 DruckerPrager3D

*Orden de trabajo. El usuario revisa **especificación física, formulación numérica y contrato**; la IA propone, ejecuta y rellena **implementación + diálogo**.*

*>Pre-redactada por la IA sobre la base de `DruckerPrager2D.md` (mismo modelo físico) y `VonMises3D.md` (mismo patrón Voigt 6D). Segunda entrega de la sub-etapa **A.bis** (materiales 3D no lineales).*

9.2 Especificación física

9.2.1 0. Descripción general

Modelo elastoplástico **friccional** independiente de la velocidad. Suelos, hormigón, granulares, materiales cuasi-frágiles cohesivo-friccionales en régimen 3D completo. Suaviza la pirámide hexagonal de Mohr-Coulomb a un **cono circular** en el espacio de tensiones principales, eliminando las aristas que complican el return mapping. Formulado íntegramente en Voigt 6D del proyecto (ADR 0012).

Diferencias clave con J2 3D (`VonMises3D`):

- **Dependencia de la presión.** La resistencia al corte aumenta con la presión de confinamiento (componente friccional).
- **Plasticidad no asociada.** La dilatación volumétrica real del flujo plástico (ψ) es típicamente menor que la fricción del criterio (ϕ). Forzar $\psi = \phi$ (asociada) sobreestima la dilatación.
- **Retorno al ápice.** El criterio define un cono con vértice; estados de prueba en zona puramente hidrostática extrema retornan al vértice (zona singular en la dirección de flujo).

Comparado con `DruckerPrager2D`:

- **Sin variantes de hipótesis** (igual que VM3D): en 3D todas las componentes son activas.
- **El catálogo de calibraciones cambia:** la variante 2D `plane_strain_matched` no tiene análogo en 3D (Mohr-Coulomb en 3D es una pirámide hexagonal sin coincidencia circular global). Las calibraciones 3D disponibles son `outer_cone` (circunscribe MC en el meridiano de compresión, **default**) e `inner_cone` (inscribe MC en el meridiano de extensión).

9.2.2 1. Descomposición aditiva y elasticidad

Pequeñas deformaciones: $\boldsymbol{\varepsilon} = \boldsymbol{\varepsilon}^e + \boldsymbol{\varepsilon}^p$. Ley elástica isotrópica con la matriz $\mathbf{C}_e$ 6×6 de `Elastic3D`:

$$\boldsymbol{\sigma} = \mathbf{C}_e (\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}^p)$$

equivalente desviador + presión:

$$\boldsymbol{\sigma} = \mathbf{s} + p \mathbf{I}, \quad \mathbf{s} = 2G \mathbf{e}_{\text{dev}}^e, \quad p = K \text{tr}(\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}^p)$$

A diferencia de J2, $\text{tr}(\boldsymbol{\varepsilon}^p) \neq 0$ en general (dilatancia plástica si $\eta_g > 0$), por lo que p depende de la deformación plástica acumulada vía $\text{tr}(\boldsymbol{\varepsilon}^p)$.

9.2.3 2. Criterio de fluencia (Drucker-Prager)

$$f(\boldsymbol{\sigma}, \alpha) = \sqrt{J_2} + \eta_f I_1 - k(\alpha) \leq 0$$

con:

- $J_2 = \frac{1}{2} \mathbf{s} : \mathbf{s}$, segundo invariante del desviador.
- $I_1 = \text{tr}(\boldsymbol{\sigma}) = \sigma_{xx} + \sigma_{yy} + \sigma_{zz}$, primer invariante.
- η_f , coeficiente friccional del cono (adimensional, ≥ 0).
- $k(\alpha) = k_0 + H_k \alpha$, parámetro cohesivo con endurecimiento isotrópico lineal.

La superficie es un **cono circular** con eje en la línea hidrostática ($\sigma_{xx} = \sigma_{yy} = \sigma_{zz}$), vértice en $I_1 = k/\eta_f$, abertura controlada por η_f .

Cómputo de J_2 en Voigt 6D del proyecto (componentes cortantes tensoriales sin factor 2):

$$J_2 = \frac{1}{2} \left(s_{xx}^2 + s_{yy}^2 + s_{zz}^2 + 2(s_{xy}^2 + s_{yz}^2 + s_{xz}^2) \right)$$

El factor 2 contabiliza las dos posiciones simétricas del tensor.

9.2.4 3. Calibración con Mohr-Coulomb (3D)

Los parámetros (η_f, k_0) se derivan de los parámetros físicos (c_0, ϕ) — cohesión y ángulo de fricción interna — según la variante del cono elegida. En 3D, **el cono no puede coincidir con la pirámide hexagonal de MC en todo el plano π** (las aristas de MC quedan fuera o dentro del círculo), por lo que las calibraciones eligen un meridiano de referencia:

variant	η_f	k_0	Comportamiento
outer_cone (default)	$2 \sin(\phi) / [\sqrt{3} (3 - \sin(\phi))]$	$6 c_0 \cos(\phi) / [\sqrt{3} (3 - \sin(\phi))]$	Circumscribe MC en el meridiano de compresión triaxial ($\theta_L = -30^\circ$). Más conservador en compresión, menos conservador en extensión.
inner_cone	$2 \sin(\phi) / [\sqrt{3} (3 + \sin(\phi))]$	$6 c_0 \cos(\phi) / [\sqrt{3} (3 + \sin(\phi))]$	Inscribe MC en el meridiano de extensión triaxial ($\theta_L = +30^\circ$). Cota inferior segura para diseño con MC.

El default `outer_cone` se elige porque la compresión triaxial es el modo dominante en problemas geotécnicos típicos (cimentaciones, muros de contención, taludes). Si la aplicación prioriza la extensión (e.g. excavaciones), `inner_cone` es la elección segura.

Caveat fundamental: el cono no captura efectos de ángulo de Lode (la pirámide hexagonal de MC sí). Para problemas donde el modo de tensión rota fuera de los meridianos puros, considerar Mohr-Coulomb 3D directo (out-of-scope) o variantes con dependencia explícita del Lode angle (e.g. Matsuoka-Nakai, también out-of-scope).

No se incluye `plane_strain_matched` (variante 2D de DP2D): es un artefacto matemático que coincide con MC exactamente solo cuando $\varepsilon_{zz} = 0$ está impuesto cinemáticamente; no tiene análogo 3D porque MC en 3D depende del Lode angle activo.

9.2.5 4. Regla de flujo (no asociada por defecto)

Potencial plástico g con coeficiente de **dilatancia** η_g (función del ángulo de dilatancia ψ , calibrado con la misma fórmula que η_f pero usando ψ):

$$g(\boldsymbol{\sigma}) = \sqrt{J_2} + \eta_g I_1$$

$$\dot{\boldsymbol{\varepsilon}}^p = \dot{\gamma} \frac{\partial g}{\partial \boldsymbol{\sigma}} = \dot{\gamma} \left(\frac{\mathbf{s}}{2\sqrt{J_2}} + \eta_g \mathbf{I} \right)$$

Descomposición:

- **Parte desviadora:** $\dot{\boldsymbol{\varepsilon}}^p = \dot{\gamma} \mathbf{s} / (2\sqrt{J_2})$ — análoga a J2.
- **Parte volumétrica:** $\text{tr}(\dot{\boldsymbol{\varepsilon}}^p) = 3\dot{\gamma} \eta_g$ — **dilatación plástica positiva** si $\eta_g > 0$.

Si $\psi = \phi$ (i.e. $\eta_g = \eta_f$) el modelo se vuelve **asociado**; tangente algorítmica simétrica. Para $\psi < \phi$ (típico en suelos: $\psi \approx \phi/2$ o incluso $\psi = 0$, "non-dilatant"), el modelo es **no asociado** y la tangente algorítmica es **asimétrica**.

9.2.6 5. Endurecimiento isótropo lineal

$$\dot{\alpha} = \dot{\gamma}$$

con α la **deformación plástica equivalente** (convención del proyecto para Drucker-Prager). La superficie crece manteniendo eje y abertura, solo k aumenta linealmente. Endurecimiento por fricción/dilatancia (cambio de η_f o η_g con α) queda fuera de esta spec.

9.2.7 6. Condiciones de Kuhn-Tucker

$$\dot{\gamma} \geq 0, \quad f \leq 0, \quad \dot{\gamma} f = 0$$

9.2.8 7. Variables internas

Símbolo	Tipo	Significado
$\boldsymbol{\varepsilon}^p$	tensor 6 componentes [$\mathbf{xx}$, $\mathbf{yy}$, $\mathbf{zz}$, $\mathbf{xy}$, $\mathbf{yz}$, $\mathbf{xz}$] (cortantes tensoriales)	deformación plástica
α	escalar ≥ 0	deformación plástica equivalente (acumulado de $\Delta\gamma$)

α es la variable principal exportable (`PRIMARY_STATE_VAR = 'alpha'`). ε^p no es **incompresible** en este modelo (a diferencia de J2): $\text{tr}(\varepsilon^p) = 3\eta_g \alpha$ en cualquier estado, lo que es un invariante cinemático verificable por test.

9.2.9 8. Notación Voigt 6D

Convención del proyecto (ADR 0012):

$$\boldsymbol{\varepsilon} = [\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{zz}, \gamma_{xy}, \gamma_{yz}, \gamma_{xz}]^\top, \quad \gamma_{ij} = 2 \varepsilon_{ij}$$

Almacenamiento interno de ε^p : 6 componentes con cortantes **tensoriales sin factor 2**, idéntica convención a `VonMises3D` (consistencia entre materiales 3D no lineales del proyecto).

9.2.10 9. Relación con `DruckerPrager2D plane strain`

`DruckerPrager2D plane strain` con variante `outer_cone` (o `inner_cone`) es la restricción de este modelo bajo $\varepsilon_{zz} = \gamma_{yz} = \gamma_{xz} = 0$ con $\varepsilon_{zz}^p \neq 0$ libre (dilatancia volumétrica). Esta relación se verifica como test cruzado en §13.

No se puede establecer la equivalencia con la variante `plane_strain_matched` de `DP2D`: esa calibración es 2D-only por construcción. El test cruzado usa `outer_cone` (o `inner_cone`) que es la única variante compartida.

9.3 Formulación numérica

9.3.1 10. Esquema implícito — Backward Euler

Dado $\{\varepsilon_n^p, \alpha_n\}$ y ε_{n+1} , predictor elástico. Conversión `engineering`→`tensorial` dividiendo los cortantes por 2 (única vez al construir el desviador):

$$\boldsymbol{\varepsilon}_{n+1}^{\text{tens}} = [\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{zz}, \frac{\gamma_{xy}}{2}, \frac{\gamma_{yz}}{2}, \frac{\gamma_{xz}}{2}]$$

Deformación elástica de prueba (incluye toda la ε_n^p , incluida su parte volumétrica por dilatancia):

$$\boldsymbol{\varepsilon}_{n+1}^{e,\text{trial}} = \boldsymbol{\varepsilon}_{n+1}^{\text{tens}} - \boldsymbol{\varepsilon}_n^p$$

Descomposición volumétrica-desviadora:

$$\text{tr}(\boldsymbol{\varepsilon}_{n+1}^{e,\text{trial}}) = (\varepsilon_{xx} + \varepsilon_{yy} + \varepsilon_{zz}) - (\varepsilon_{xx,n}^p + \varepsilon_{yy,n}^p + \varepsilon_{zz,n}^p)$$

$$\mathbf{e}^{\text{trial}} = \text{dev}(\boldsymbol{\varepsilon}_{n+1}^{e,\text{trial}}), \quad \mathbf{s}^{\text{trial}} = 2G \mathbf{e}^{\text{trial}}, \quad p^{\text{trial}} = K \text{tr}(\boldsymbol{\varepsilon}_{n+1}^{e,\text{trial}})$$

con $I_1^{\text{trial}} = 3p^{\text{trial}}$ y $\sqrt{J_2^{\text{trial}}} = \|\mathbf{s}^{\text{trial}}\|/\sqrt{2}$ (norma Frobenius en Voigt 6D tensorial).

Función de fluencia trial:

$$f^{\text{trial}} = \sqrt{J_2^{\text{trial}}} + \eta_f I_1^{\text{trial}} - k(\alpha_n)$$

Si $f^{\text{trial}} \leq \text{tol} \rightarrow$ paso elástico. Si no $\rightarrow$ return mapping.

9.3.2 11. Return regular (cone surface)

La dirección desviadora $\hat{\mathbf{n}} = \mathbf{s}^{\text{trial}} / (2\sqrt{J_2^{\text{trial}}})$ es invariante bajo carga radial. Las actualizaciones son lineales en $\Delta\gamma$:

$$\begin{aligned}\sqrt{J_2^{n+1}} &= \sqrt{J_2^{\text{trial}}} - G \Delta\gamma \\ p^{n+1} &= p^{\text{trial}} - 3K \eta_g \Delta\gamma, \quad I_1^{n+1} = 3p^{n+1}\end{aligned}$$

La consistencia $f(\boldsymbol{\sigma}_{n+1}, \alpha_{n+1}) = 0$ con $\alpha_{n+1} = \alpha_n + \Delta\gamma$ y $k_{n+1} = k_0 + H_k(\alpha_n + \Delta\gamma)$ da:

$$\Delta\gamma = \frac{f^{\text{trial}}}{G + 9K \eta_f \eta_g + H_k}$$

Forma cerrada — sin Newton local. Idéntica al 2D plane strain en estructura algebraica.

Actualizaciones:

$$\begin{aligned}\mathbf{s}^{n+1} &= \frac{\sqrt{J_2^{n+1}}}{\sqrt{J_2^{\text{trial}}}} \mathbf{s}^{\text{trial}}, \quad \boldsymbol{\sigma}^{n+1} = \mathbf{s}^{n+1} + p^{n+1} \mathbf{I} \\ \boldsymbol{\varepsilon}_{n+1}^p &= \boldsymbol{\varepsilon}_n^p + \Delta\gamma \left(\frac{\mathbf{s}^{n+1}}{2\sqrt{J_2^{n+1}}} + \eta_g \mathbf{I} \right)\end{aligned}$$

donde $\mathbf{I} = [1, 1, 1, 0, 0, 0]$ en Voigt 6D (parte volumétrica) y la componente desviadora de $\Delta\boldsymbol{\varepsilon}^p$ se almacena en convención tensorial.

Condición de validez: $\sqrt{J_2^{n+1}} \geq 0$. Si $\Delta\gamma_{\text{reg}} > \sqrt{J_2^{\text{trial}}}/G$, el desviador "se pasa de cero" el algoritmo regular es inválido $\rightarrow$ toca **return al ápice**.

9.3.3 12. Return al ápice

Cuando el estado de prueba cae en zona puramente hidrostática extrema (predictor muy traccionante respecto a la cohesión), la proyección al cono cae en su vértice, donde $\mathbf{s} = 0$ y la dirección desviadora $\hat{\mathbf{n}}$ no está definida.

En el ápice: $\boldsymbol{\sigma}^{n+1} = (k_{n+1}/(3\eta_f)) \mathbf{I}$ (puramente hidrostático), $\sqrt{J_2^{n+1}} = 0$.

$$\Delta\gamma_{\text{apex}} = \frac{I_1^{\text{trial}} \eta_f - k(\alpha_n)}{9K \eta_f \eta_g + H_k}$$

(Equivalente a la fórmula regular sin el término G , porque la rama de apex es puramente volumétrica.)

Actualizaciones (apex):

$$\begin{aligned}\mathbf{s}^{n+1} &= 0, \quad p^{n+1} = \frac{k_{n+1}}{3\eta_f}, \quad \boldsymbol{\sigma}^{n+1} = p^{n+1} \mathbf{I} \\ \boldsymbol{\varepsilon}_{n+1}^p &= \boldsymbol{\varepsilon}_n^p + \mathbf{e}^{\text{trial}} + \Delta\gamma_{\text{apex}} \eta_g \mathbf{I}\end{aligned}$$

(El desviador trial se absorbe completamente; la parte volumétrica del flujo añade $3\eta_g \Delta\gamma$ a la traza, distribuida $\eta_g \Delta\gamma$ por componente diagonal.)

9.3.4 13. Detección del régimen de retorno

Algoritmo de despacho (idéntico al 2D):

1. Calcular $\Delta\gamma_{\text{reg}}$ por la fórmula regular.
2. Si $\sqrt{J_2^{\text{trial}}} - G \Delta\gamma_{\text{reg}} \geq 0 \rightarrow$ return regular.
3. Si no $\rightarrow$ return al ápice con $\Delta\gamma_{\text{apex}}$.

9.3.5 14. Tangente algorítmica consistente

Return regular ($\sqrt{J_2^{n+1}} > 0$), forma compacta (de Souza Neto §8.3.4 extendida a 6D):

$$\mathbf{C}^{\text{alg}} = K \mathbf{v} \otimes \mathbf{v} + 2G \left(1 - \frac{G \Delta\gamma}{\sqrt{J_2^{\text{trial}}}} \right) \mathbf{I}_{\text{dev}} + 4G \beta \hat{\mathbf{n}} \otimes \hat{\mathbf{n}} - \frac{1}{A} \mathbf{b}_g \otimes \mathbf{b}_f$$

con:

- $\mathbf{v} = [1, 1, 1, 0, 0, 0]$ (gradiente de I_1 en Voigt 6D engineering input).
- $\mathbf{I}_{\text{dev}}$: proyector desviador 6×6 con 1/2 en los cortantes (mapea engineering γ_{ij} a tensorial ε_{ij}).
- $\hat{\mathbf{n}} = \mathbf{s}^{\text{trial}} / (2\sqrt{J_2^{\text{trial}}})$ en Voigt 6D tensorial.
- $\beta = G \Delta\gamma / \sqrt{J_2^{\text{trial}}}$.
- $\mathbf{b}_f = 2G \hat{\mathbf{n}} + 3K \eta_f \mathbf{v}$, $\mathbf{b}_g = 2G \hat{\mathbf{n}} + 3K \eta_g \mathbf{v}$.
- $A = G + 9K \eta_f \eta_g + H_k$.

Asimétrica si $\eta_f \neq \eta_g$ (no asociada). El término $+4G\beta \hat{\mathbf{n}} \otimes \hat{\mathbf{n}}$ recoge el cambio de dirección de flujo con ε a través de $\mathbf{s}^{\text{trial}}$ (no solo el cambio de magnitud); sin él la tangente cierra mal en componentes de cortante alineados con el flujo (verificable por diferencia finita, idéntico al hallazgo del DP2D).

Return al ápice ($\sqrt{J_2^{n+1}} = 0$):

$$\mathbf{C}_{\text{apex}}^{\text{alg}} = \frac{K H_k}{9K \eta_f \eta_g + H_k} \mathbf{v} \otimes \mathbf{v}$$

Rigidez puramente volumétrica reducida; sin rigidez desviadora alguna. Con $H_k = 0$ y $\eta_g > 0$ la rigidez tangente colapsa por completo — el código devuelve un escalado mínimo de K (idéntico patrón al 2D) para evitar matriz globalmente singular hasta que el siguiente paso salga del ápice.

9.3.6 15. Tolerancia de fluencia

ADR 0006: `admissibility_scale = k(α) = k_0 + H_k · α` (escala cohesiva corriente del cono). Tiene unidades de esfuerzo, igual que f .

9.4 Contrato de implementación

```

name: DruckerPrager3D
kind: material
status: validated          # draft → implemented → validated

interface:
  strain_dim: 6
  primary_state_var: alpha
  is_symmetric: false      # tangente asimétrica si no asociada ( );
                           # override por instancia a True cuando = (mismo
                           # patrón que DP2D - el despachador algebraico ADR
                           # 0003 elige Cholesky/ LDL en problemas asociados).

parameters:
- { name: E,                type: float, required: true,  desc: "Módulo de Young (>0)" }
- { name: nu,               type: float, required: true,  desc: "Coeficiente de Poisson
  (-1, 0.5)" }
- { name: cohesion,         type: float, required: true,  desc: "Cohesión c_0 inicial (
  esfuerzo, >0)" }
- { name: phi_deg,          type: float, required: true,  desc: "Ángulo de fricción interna
  en grados; típico suelos 20-40°, hormigón 30-37°" }
- { name: psi_deg,          type: float, required: false, default: null,
  desc: "Ángulo de dilatación en grados. Default = phi_deg (asociada). Típico no
  asociada: < , e.g. 0 o /2." }
- { name: H,                type: float, required: false, default: 0.0,
  desc: "Endurecimiento isótropo lineal en cohesión H_k (0). H=0 perfectamente pl
  ástico." }
- { name: variant,          type: str,   required: false, default: "outer_cone",
  desc: "outer_cone (default, circumscribe MC en compresión) | inner_cone (inscribe
  MC en extensión)" }
- { name: density,          type: float, required: false, default: null, desc: "Densidad (
  ADR 0008)" }

signature:
  compute_state: " (: ndarray(6,), state_vars=None) -> (: ndarray(6,), C_alg: ndarray
    (6,6), state_vars)"
  strain_kind: "3D Voigt - [_xx, _yy, _zz, _xy, _yz, _xz] con _ij = 2·_ij"

state_schema:
  eps_p: "ndarray(6,) - [p_xx, p_yy, p_zz, p_xy, p_yz, p_xz] (cortantes
    tensoriales, sin factor 2). tr(p) = 3·g· invariante cinemático."
  alpha: "float 0 - multiplicador plástico acumulado (= deformación plástica
    equivalente)"

conventions:
  sign: " > 0 tracción (Reglas §5)"
  voigt: "[xx, yy, zz, xy, yz, xz] (ADR 0012); _ij = 2·_ij engineering input;
    p_ij almacenada tensorial; _ij tensorial sin factor"
  associated_flow: " = asociada (tangente simétrica); no asociada (tangente
    asimétrica)"

validity:
- "E > 0, c_0 > 0, 0 < 90°, 0"
- " (-1, 0.5) estricto"
- "H 0"
- "variant {'outer_cone', 'inner_cone'}"

```

```

out_of_scope:
- "Mohr-Coulomb 3D con aristas (pirámide hexagonal): el cono circular DP no captura efectos de ángulo de Lode"
- "Variantes con dependencia explícita del Lode angle (Matsuoka-Nakai, Lade-Duncan): formulaciones distintas"
- "Calibración 'plane_strain_matched': sin análogo 3D (la coincidencia DPMC en plane strain es 2D-only)"
- "Endurecimiento por cambio de y/o con (no lineal fuerte en return mapping)"
- "Endurecimiento cinemático (back-stress hidrostático y desviador)"
- "Cap (modelo cap-cone) para suelos compactables"
- "Ablandamiento (H<0) - requiere regularización"
- "Grandes deformaciones"

numerical_caveats:
- "Tangente algorítmica asimétrica si → IS_SYMMETRIC=False, el despachador algebraico usa LU (ADR 0003). Cuando = se vuelve simétrica y el override por instancia activa Cholesky/ LDL."
- "Modos de carga muy traccionantes pueden caer en la rama de ápice. La detección automática del régimen (regular vs apex) está en el algoritmo."
- "Con = 0 (sin dilatación) y H_k = 0, en la rama de ápice la tangente volumétrica se anula → singularidad. El código devuelve un valor de respaldo (rigidez volumétrica reducida no nula, K·1e-6) para evitar matriz singular; típicamente sale del ápice en el siguiente paso."
- "Locking volumétrico en Hex8/Tet4 cuando > 0 (flujo dilatante) combinado con moderado-alto. Mismo régimen documentado para Elastic3D/VonMises3D; mitigaciones B-bar/F-bar diferidas."
- "El cono circular DP es independiente del ángulo de Lode. Para trayectorias de carga donde el modo rota fuera del meridiano de calibración (compresión triaxial → cortante → extensión triaxial), la diferencia DP vs MC puede ser hasta el 15-20% en la frontera. Es limitación inherente del modelo, no del algoritmo."

acceptance:
unit:
- name: parameter_validation
  setup: "constructor con variant inválida, 90°, >, E0, c_00, H<0, (-1,0.5)"
  expect: "ValueError con mensaje claro en cada caso"

- name: calibration_outer_inner_consistency
  setup: "comparar _f de outer_cone vs inner_cone para =30°"
  expect: "outer_cone _f > inner_cone _f (el outer es geométricamente mayor); ambos > 0; k_0 análogo"

- name: paso_elastico_below_yield
  setup: "pequeño 6D, todas las componentes activas"
  expect: "= C_e·; eps_p y alpha intactos"
  tol_rel: 1.0e-10

- name: regular_return_under_pure_shear
  setup: "cortante puro suficiente para fluir (_xy creciente, otras componentes 0)"
  expect: "return regular activo, > 0; estado final cumple f tol"
  tol_rel: 1.0e-8

- name: apex_return_under_hydrostatic_tension
  setup: "hidrostática traccionante (_xx = _yy = _zz > 0 suficientemente grande, sin cortante)"
  expect: "return al ápice; _xx = _yy = _zz = k()/(3_f); √J 0"
  tol_rel: 1.0e-8

```

```

- name: tr_eps_p_invariant
  setup: "carga monotónica en rama regular (5 niveles),      (no asociada con _g > 0)"
  expect: "tr( $\epsilon_p$ ) = 3 ·  $\epsilon_g$  en cada paso plástico"
  tol_rel: 1.0e-10

- name: associated_tangent_is_symmetric
  setup: " = (asociada), estado plástico activo en rama regular"
  expect: "C_alg simétrica  $\|C_{alg} - C_{alg}^T\| \leq tol$ "
  tol_abs: 1.0e-8

- name: nonassociated_tangent_is_asymmetric
  setup: " = 0,  $\alpha = 30^\circ$ , estado plástico activo no degenerado"
  expect: " $\|C_{alg} - C_{alg}^T\|$  con magnitud significativa relativa a  $\|C_{alg}\|$ "

- name: tangent_matches_finite_difference
  setup: "estado plástico regular; comparar C_alg cerrada con diferencia finita centrada"
  expect: "error relativo < 1e-4 en todas las componentes"
  tol_rel: 1.0e-4

- name: alpha_monotonic
  setup: "carga creciente con plasticidad sostenida"
  expect: "  $\alpha$  no decrece nunca"

- name: unloading_recovers_C_e
  setup: "cargar a plástico, descargar elásticamente"
  expect: "tangente = C_e en el paso de descarga; se conserva"
  tol_rel: 1.0e-10

- name: unit_invariance_MPa_vs_Pa
  setup: "mismo path adimensional con (MPa,mm,N) y (Pa,m,N)"
  expect: "  $\epsilon_p$  y  $\epsilon_g$  (adimensionales) idénticos; E idéntico"
  tol_rel: 1.0e-12

cross_consistency:
- name: equivalencia_plane_strain_outer_cone
  setup: "DP3D con variant='outer_cone' y  $\epsilon = (\epsilon_{xx}, \epsilon_{yy}, 0, \epsilon_{xy}, 0, 0)$ ; DP2D con variant='outer_cone' (plane_strain) y  $\epsilon = (\epsilon_{xx}, \epsilon_{yy}, \epsilon_{xy})$ ; path multi-paso con plasticidad activa en rama regular"
  expect: " $\epsilon_{xx}, \epsilon_{yy}, \epsilon_{xy}$  y  $\epsilon$  coinciden entre 3D y 2D PS; componentes plásticas planas coinciden ( $\epsilon_{xx}, \epsilon_{yy}, \epsilon_{zz}, \epsilon_{xy\_tens}$ );  $\epsilon_{p\_yz} = \epsilon_{p\_xz} = 0$  en VM3D (no se activan)"
  tol_rel: 1.0e-10

integration:
- name: hex8_pipeline_elastico
  setup: "Hex8 unitario, carga muy por debajo del yield"
  expect: "respuesta elástica pura,  $\epsilon = 0$  en todos los Gauss points"
  tol_rel: 1.0e-8

- name: hex8_pipeline_plastico_converges
  setup: "Hex8 unitario con confinamiento + cortante; carga plástica significativa; 4 pasos, max_iter=8"
  expect: "Newton converge en cada paso;  $\epsilon > 0$  al final; tangente asimétrica activa LU"

- name: hex8_apex_under_hydrostatic

```

```

setup: "Hex8 unitario con hidrostática traccionante (todos los nodos del cubo
      desplazados radialmente desde el centro)"
expect: "estado final cae en ápice en los 8 puntos de Gauss: _xx _yy _zz, √J
      0"
tol_rel: 1.0e-6

- name: tet4_basic_pipeline
  setup: "Tet4 con confinamiento + cortante, plasticidad activa"
  expect: "convergencia + > 0 + tr(ε̂p) = 3 · g · invariante en el único Gauss
      point"

references:
- "Drucker D.C., Prager W. (1952). Soil mechanics and plastic analysis or limit
  design. Quart. Appl. Math. 10, 157-165."
- "Simó J.C., Hughes T.J.R. (1998). Computational Inelasticity. Springer. §6 (cone
  plasticity, apex return)."
- "de Souza Neto E.A., ĩPeri D., Owen D.R.J. (2008). Computational Methods for
  Plasticity. Wiley. §8 (Drucker-Prager 3D, calibraciones outer/inner, tangentes
  algorítmicas detalladas)."
- "Chen W.F., Han D.J. (1988). Plasticity for Structural Engineers. Springer. §7 (
  cohesivo-friccionales 3D)."
- "ADR 0012 - Sólidos 3D: convención Voigt 6D del proyecto."
- "ADR 0003 - Despachador algebraico: tangente asimétrica fuerza LU global."
- "ADR 0006 - Tolerancias adimensionales: patrón atol + rtol · escala con escala física
  k ()."

```

9.5 Implementación

- **Archivo:** solidum/materials/drucker_prager_3d.py.
- **Clase:** DruckerPrager3D, registrada vía `@MaterialRegistry.register`. `IS_SYMMETRIC = False` a nivel de clase; override por instancia a `True` cuando $\psi = \varphi$ (asociada).
- **Kernel Numba** `_compute_drucker_prager_3d`: predictor elástico desviador-volumétrico en Voigt 6D tensorial (sustracción de ϵ_n^p antes del desviador — *necesario* en DP por la dilatación plástica), detección automática regular/apex, tangente cerrada para cada rama.
- **Calibración:** reusa la función `_calibrate_drucker_prager` de `drucker_prager_2d.py` por composición (mismas fórmulas para outer/inner; `plane_strain_matched` queda filtrada por el validador de variant antes de llegar a la función compartida).
- **State schema:** `eps_p` ndarray(6,) [xx, yy, zz, xy, yz, xz] con cortantes tensoriales, `alpha` escalar. **No es incompresible:** $\text{tr}(\epsilon_p) = 3 \cdot \eta_g \cdot \alpha$ por construcción del flujo.
- **admissibility_scale:** $k(\alpha) = k_0 + H \cdot \alpha$ (cohesión efectiva corriente).
- **Tests:**
 - `tests/test_materials_unit.py · TestDruckerPrager3D` (17 tests unitarios) + `TestDruckerPrager3DvsPlaneStress` (1 cruzado contra DP2D outer_cone_plane_strain).
 - `tests/test_solid_3d_plasticity.py · 4 tests de integración:` `TestHex8DruckerPrager3DElastic` (1 — respuesta elástica), `TestHex8DruckerPrager3DRegularPlastic` (1 — cortante puro + tangente asimétrica activa LU + invariante $\text{tr}(\epsilon^p) = 3\eta_g\alpha$), `TestHex8DruckerPrager3DApex`

(1 — expansión hidrostática a 8 GP en rama ápice), `TestTet4DruckerPrager3DBasic` (1 — smoke test Tet4).

■ **Notas de traducción:**

- Convención mixta engineering/tensorial idéntica a VM3D: entrada `strain` engineering ($\gamma_{ij} = 2 \cdot \epsilon_{ij}$), salida `sigma` con cortantes tensoriales, estado `eps_p` con cortantes tensoriales. Conversión engineering→tensorial una sola vez al construir el desviador (`strain[3..5]/2`).
- El predictor construye el desviador como $\mathbf{s}^{\text{trial}} = 2G \text{dev}(\boldsymbol{\epsilon}^{\text{tens}} - \boldsymbol{\epsilon}_n^p)$, **no** como $2G (\text{dev}(\boldsymbol{\epsilon}^{\text{tens}}) - \epsilon_n^p)$. A diferencia de J2, ϵ_n^p en DP tiene parte volumétrica no nula (dilatancia plástica si $\eta_g > 0$), así que la sustracción debe hacerse *antes* del operador desviador para que la parte volumétrica del flujo se contabilice correctamente.
- Tangente regular usa $\mathbf{I}_{\text{dev}}$ 6×6 con 1/2 en los cortantes (mapea engineering input a tensorial output), idéntica estructura al `I_dev` de VM3D.
- Tangente apex: rigidez puramente volumétrica reducida con escalado mínimo $K \cdot 10^{-6}$ cuando $H_k = 0$ y η_g pequeño (mismo patrón que DP2D para evitar matriz globalmente singular en el paso degenerado).
- Validación de parámetros en constructor: $E > 0$, $c_0 > 0$, $0 \leq \varphi < 90^\circ$, $0 \leq \psi \leq \varphi$, $H \geq 0$, $\nu \in (-1, 0.5)$, `variant` $\in \{\text{outer_cone}, \text{inner_cone}\}$, `density` ≥ 0 . Mensajes en español; rechazo explícito de `plane_strain_matched` con explicación de por qué es 2D-only.

9.6 Diálogo

- **2026-05-21** · Spec pre-redactada por la IA al avanzar la sub-etapa **A.bis** (materiales 3D no lineales) tras cerrar `VonMises3D` con 824 tests verdes. Base: `DruckerPrager2D.md` (modelo físico, calibraciones, two-branch return mapping) + `VonMises3D.md` (estructura Voigt 6D, patrón cross-consistency).
- **2026-05-21** · **Decisión de plumbing (decide la IA, Reglas §3)**: kernels Numba separados `_compute_drucker_prager_3d` (este material) vs `_compute_drucker_prager_plane_strain` (existente en DP2D). **No** se extrae un `_DPCore` compartido en esta entrega, por las mismas razones que motivaron mantener separados los kernels de VM2D plane strain y VM3D: Numba *nopython* desfavorece arrays de tamaño variable, los kernels son cortos (~80 líneas cada uno), y el sub-patrón compartido (descomposición desviadora, branch detection, tangente cerrada) se documenta cruzadamente entre las specs.
- **2026-05-21** · **Decisión de alcance de variantes**: `DruckerPrager3D` ofrece `outer_cone` (default) e `inner_cone`, omitiendo `plane_strain_matched`. Razón: la variante 2D `plane_strain_matched` es un artefacto matemático que reproduce MC exactamente solo bajo $\epsilon_{zz} = 0$ cinemático impuesto; no tiene análogo 3D porque MC en 3D depende del Lode angle activo. Default `outer_cone` se elige por ser el modo dominante en problemas geotécnicos (cimentaciones, muros). Si la aplicación es extension-dominated (excavaciones), el usuario puede elegir `inner_cone` explícitamente.
- **2026-05-21** · **Decisión de tipo de tests cruzados**: `cross_consistency` compara DP3D (`variant='outer_cone'`) contra DP2D (`variant='outer_cone'`, `plane_strain`), no contra

`plane_strain_matched`. Razón: solo las dos calibraciones cono (outer/inner) existen en ambos modelos. La equivalencia matemática es exacta porque las fórmulas de calibración no dependen de la dimensión — solo de (c_0, ϕ, ψ) — y el cono 3D con $\varepsilon_{zz} = \gamma_{yz} = \gamma_{xz} = 0$ impuesto se reduce a la formulación 2D plane strain en las componentes activas.

- **2026-05-21 · Decisión de plumbing:** el predictor elástico construye el desviador como $\mathbf{s}^{\text{trial}} = 2G \text{dev}(\boldsymbol{\varepsilon}^{\text{tens}} - \boldsymbol{\varepsilon}_n^p)$, **no** como $2G (\text{dev}(\boldsymbol{\varepsilon}^{\text{tens}}) - \boldsymbol{\varepsilon}_n^p)$. Diferencia material: a diferencia de J2, $\boldsymbol{\varepsilon}^p$ en DP tiene parte volumétrica no nula ($\text{tr}(\boldsymbol{\varepsilon}^p) = 3\eta_g\alpha$), así que la sustracción debe hacerse *antes* del desviador para que la parte volumétrica del flujo plástico se contabilice correctamente. Esta es la lección documentada en el comentario del kernel de DP2D (línea 39–42) y se aplica desde el inicio en DP3D.
- **2026-05-21 · Implementación completa, status validated.** Kernel + clase + 17 tests unitarios + 1 test cruzado + 4 tests de integración (Hex8 elástico, Hex8 rama regular con tangente asimétrica + invariante $\text{tr}(\boldsymbol{\varepsilon}^p) = 3\eta_g\alpha$, Hex8 rama ápice bajo expansión hidrostática, Tet4 smoke test). **24 tests añadidos a la suite (824 → 848 verdes; suite global sin regresiones).** Todos los tests pasaron a la primera, incluyendo el cruzado DP3D ↔ DP2D outer_cone plane_strain a 10 decimales — señal fuerte de que la convención Voigt 6D, la calibración compartida y la sustracción de $\boldsymbol{\varepsilon}_n^p$ antes del desviador son correctas. La integración Hex8 con cortante puro y tangente asimétrica ($\psi \neq \phi$) confirma que el despachador algebraico (ADR 0003) selecciona LU correctamente vía el flag declarativo `IS_SYMMETRIC = False`. Validación contra solución analítica cerrada (cilindro con falla DP, esfera hueca bajo presión interna, etc.) **diferida a la campaña 3D consolidada** que se hará al final de la sub-etapa A.bis (tras completar `IsotropicDamage3D`).

9.7 Elastic3D

*Material elástico isótropo en 3D, espejo natural de **Elastic2D** con la convención Voigt 6D fijada en ADR 0012.*

9.8 Especificación física

9.8.1 0. Descripción general

Modelo elástico lineal isótropo tridimensional, parametrizado por (E, ν) . A diferencia de **Elastic2D**, **no tiene variantes de hipótesis cinemática**: en 3D no aplican `plane_stress` ni `plane_strain` — toda la deformación es activa. Independiente de la velocidad, sin variables internas. Es el material elástico canónico para todos los sólidos 3D del catálogo.

9.8.2 1. Descomposición de la deformación

No aplica — toda la deformación es elástica:

$$\varepsilon = \varepsilon^e$$

9.8.3 2. Ley elástica

$$\sigma = \mathbf{C}_e \varepsilon$$

con ε y σ en **notación Voigt 6D del proyecto** (ADR 0012, Reglas.md §5):

$$\varepsilon = [\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{zz}, \gamma_{xy}, \gamma_{yz}, \gamma_{xz}]^T, \quad \gamma_{ij} = 2\varepsilon_{ij}$$

$$\sigma = [\sigma_{xx}, \sigma_{yy}, \sigma_{zz}, \sigma_{xy}, \sigma_{yz}, \sigma_{xz}]^T$$

Matriz constitutiva elástica 6×6:

$$\mathbf{C}_e = \frac{E}{(1+\nu)(1-2\nu)} \begin{bmatrix} 1-\nu & \nu & \nu & 0 & 0 & 0 \\ \nu & 1-\nu & \nu & 0 & 0 & 0 \\ \nu & \nu & 1-\nu & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1-2\nu}{2} & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1-2\nu}{2} & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1-2\nu}{2} \end{bmatrix}$$

Tangente constante $\mathbf{C}_e$ en todo régimen. Simétrica y positiva definida en el rango admisible de ν .

El factor $\frac{1-2\nu}{2}$ en los cortantes (no $1-2\nu$) es consecuencia del uso de γ_{ij} *engineering*: $\sigma_{ij} = G \gamma_{ij}$ con $G = \frac{E}{2(1+\nu)} = \frac{E}{(1+\nu)(1-2\nu)} \cdot \frac{1-2\nu}{2}$.

9.8.4 3-6. Superficie de fluencia, flujo, endurecimiento, Kuhn-Tucker

No aplican — el material es indefinidamente elástico.

9.8.5 7. Variables internas

Ninguna — `state_vars = None` se acepta y se devuelve intacto. Sin `PRIMARY_STATE_VAR`.

9.8.6 8. Notación Voigt

Orden y factores fijados por **ADR 0012**:

- Componentes diagonales primero: $[\mathbf{xx}, \mathbf{yy}, \mathbf{zz}]$.
- Bloque cortante después: $[\mathbf{xy}, \mathbf{yz}, \mathbf{xz}]$ (extensión natural del orden 2D del proyecto $[\mathbf{xx}, \mathbf{yy}, \mathbf{xy}]$).
- Deformaciones angulares *engineering*: $\gamma_{ij} = 2\varepsilon_{ij}$.
- Esfuerzos tensoriales sin factor.

Compatibilidad con códigos externos: ABAQUS usa $[11, 22, 33, 12, 13, 23]$. La diferencia con la convención del proyecto es la permutación del bloque cortante ($\mathbf{yz} \leftrightarrow \mathbf{xz}$). Si se importan datos desde ABAQUS, aplicar permutación $5 \leftrightarrow 6$ una vez en el preprocesador.

9.9 Formulación numérica

9.9.1 9. Esquema temporal

No aplica — sin historia.

9.9.2 10. Return mapping / actualización

No aplica. Evaluación reducida a producto matriz-vector:

$$\boldsymbol{\sigma}_{n+1} = \mathbf{C}_e \boldsymbol{\varepsilon}_{n+1}, \quad \mathbf{C}^{\text{alg}} = \mathbf{C}_e$$

La matriz $\mathbf{C}_e$ se precalcula y guarda en el constructor — coste de `compute_state` es un producto $6 \times 6 \cdot 6$ sin asignación adicional.

9.9.3 11. Tangente algorítmica consistente

$$\mathbf{C}^{\text{alg}} = \mathbf{C}_e$$

Simétrica (`IS_SYMMETRIC = True`). Positiva definida para $\nu \in (-1, 0.5)$ estricto; diverge en $\nu \rightarrow 0.5$ (incompresible — requiere formulación mixta u-p, no soportada).

9.9.4 12. Caveats numéricos

- **Incompresibilidad** $\nu \rightarrow 0.5$: la matriz constitutiva diverge ($1 - 2\nu \rightarrow 0$). El constructor rechaza $\nu \geq 0.5$ con `ValueError`. Recomendado $\nu \leq 0.49$ para metales y polímeros casi-incompresibles.
- **Locking volumétrico** en Hex8/Tet4 con $\nu \rightarrow 0.5$: declarado limitación arquitectural en la spec de cada elemento, sin mitigación implementada (B-bar/F-bar diferidas a caso de uso real, política idéntica al 2D).
- Para $\nu < 0$ el material es admisible termodinámicamente (auxético) pero infrecuente en la práctica; el constructor lo permite hasta $\nu > -1$.

9.10 Contrato de implementación

```

name: Elastic3D
kind: material
status: validated

interface:
  strain_dim: 6
  primary_state_var: null
  is_symmetric: true

parameters:
  - { name: E,          type: float, required: true, desc: "Módulo de Young (>0)" }
  - { name: nu,         type: float, required: true, desc: "Coeficiente de Poisson (-1, 0.5)" }
  - { name: density, type: float, required: false, default: null,
      desc: "Densidad (kg/m³). Opcional al construir; obligatoria solo si se ensambla peso propio o masa (ADR 0008)" }

signature:
  compute_state: " (: ndarray(6,), state_vars=None) -> (: ndarray(6,), C_e: ndarray(6,6), state_vars)"
  strain_kind: "3D Voigt - [_xx, _yy, _zz, _xy, _yz, _xz] con _ij = 2·_ij"

state_variables: []

conventions:
  sign: " > 0  tracción (Reglas §5)"
  voigt: "[xx, yy, zz, xy, yz, xz] (ADR 0012); _ij = 2·_ij en deformación; _ij tensorial sin factor"
  hypothesis_dispatch: "no aplica en 3D"

validity:
  - "E > 0"
  - " (-1, 0.5) estricto; = 0.5 rechazado (incompresible)"
  - "pequeñas deformaciones (|| 1e-2)"

out_of_scope:
  - "anisotropía (ortótropo, monoclinico, totalmente anisótropo)"
  - "plasticidad VonMises3D, DruckerPrager3D (sub-etapa posterior)"
  - "daño IsotropicDamage3D (sub-etapa posterior)"
  - "incompresibilidad estricta (=0.5) requiere formulación mixta u-p"
  - "grandes deformaciones"

acceptance:
  verification:
    - name: simetria_C_e
      setup: "construir Elastic3D y comparar C_e con su traspuesta"
      expect: "C_e == C_e.T exacto componente a componente"
      tol_abs: 1.0e-14
    - name: positividad_definida
      setup: "autovalores de C_e con {0, 0.3, 0.49}"
      expect: "todos los autovalores estrictamente positivos"
      tol_abs: 0.0
    - name: traccion_uniaxial
      setup: " = (_0, -·_0, -·_0, 0, 0, 0)"
      expect: " = (E·_0, 0, 0, 0, 0, 0) exacto"
      tol_rel: 1.0e-12

```

```

- name: cortante_puro_xy
  setup: " = (0, 0, 0, _0, 0, 0) con _0 = 1e-3"
  expect: " = (0, 0, 0, G·_0, 0, 0) con G = E/[2(1+)]"
  tol_rel: 1.0e-12
- name: cortante_puro_yz
  setup: " = (0, 0, 0, 0, _0, 0)"
  expect: " = (0, 0, 0, 0, G·_0, 0)"
  tol_rel: 1.0e-12
- name: cortante_puro_xz
  setup: " = (0, 0, 0, 0, 0, _0)"
  expect: " = (0, 0, 0, 0, 0, G·_0)"
  tol_rel: 1.0e-12
- name: compresion_hidrostatica
  setup: " = (_0, _0, _0, 0, 0, 0) con _0 = -1e-3 (compresión isotrópica)"
  expect: " = (K·3_0, K·3_0, K·3_0, 0, 0, 0) con K = E/[3(1-2)] (módulo de
    compresibilidad)"
  tol_rel: 1.0e-12
- name: rechazo_inputs_invalidos
  setup: "construir Elastic3D con E0, fuera de rango (-1, 0.5), density<0"
  expect: "ValueError con mensaje claro en cada caso"

references:
- "Timoshenko S., Goodier J.N. (1970). Theory of Elasticity. McGraw-Hill. §6 (3D
  elasticity)."
```

```

- "Bathe K.J. (2014). Finite Element Procedures. Prentice Hall. §6.6."
- "Cook R.D., Malkus D.S., Plesha M.E., Witt R.J. (2002). Concepts and Applications
  of FEA. Wiley. §6.8."
- "ADR 0012 - Sólidos 3D: convención Voigt 6D y cierre del contrato internal_forces."
```

9.11 Implementación

- **Archivo:** solidum/materials/elastic_3d.py.
- **Clase:** Elastic3D, registrada vía `@MaterialRegistry.register`.
- **Despacho:** matriz `C` precalculada en el constructor. `compute_state` reduce a `self.C @ strain` sin ramificaciones — coste mínimo.
- **Validación temprana** en constructor: $E > 0$, $\nu \in (-1, 0.5)$, $\text{density} \geq 0$ si se pasa. Mensajes en español.
- **Tests:** tests/test_solid_3d.py — clase `TestElastic3D` (11 tests: simetría `C`, positividad definida, tracción uniaxial, los tres cortantes puros, compresión hidrostática con $K = E/[3(1-2\nu)]$, rechazos de inputs inválidos, `density` opcional).
- **Validación de integración:** tests/validation/test_cube_lame_3d.py consume `Elastic3D` end-to-end vía `Hex8` y `Tet4` (tracción uniaxial + compresión hidrostática con solución analítica exacta).

9.12 Diálogo

- **2026-05-19** · Spec creada como pieza inicial de la Etapa 7 (sólidos 3D, alcance acotado `Hex8` + `Tet4` + `Elastic3D`). Convención Voigt 6D fijada en ADR 0012. Sin variantes de hipótesis (no aplica `plane_stress/plane_strain` en 3D).

9.13 IsotropicDamage3D

*Orden de trabajo. El usuario revisa **especificación física, formulación numérica y contrato**; la IA propone, ejecuta y rellena **implementación + diálogo**.*

*>Pre-redactada por la IA sobre la base de `IsotropicDamage2D.md` (mismo modelo físico) y `Elastic3D.md` / `VonMises3D.md` (patrón Voigt 6D). Tercera y última entrega de la sub-etapa **A.bis** (materiales 3D no lineales).*

9.14 Especificación física

9.14.1 0. Descripción general

Modelo de **daño isótropo escalar** en pequeñas deformaciones para sólidos 3D, formulado íntegramente en Voigt 6D del proyecto (ADR 0012). Captura degradación progresiva e isótropa de la rigidez en respuesta a la deformación máxima histórica, con ablandamiento exponencial. Pensado para hormigón en tracción uniforme, cerámicos, materiales cuasi-frágiles donde la microfisuración distribuida puede modelarse mediante un escalar.

Diferencias respecto a `IsotropicDamage2D`:

- **Sin variantes de hipótesis** (igual que VM3D, DP3D, Elastic3D): en 3D todas las componentes son activas.
- **Algoritmo idéntico al 2D extendido a 6D**: misma ley de evolución exponencial, mismo régimen carga/descarga por κ , misma estructura de tangente algorítmica consistente. **Sin Newton local** (el algoritmo es explícito).
- **IsotropicDamage2D plane_strain** es la restricción de este modelo bajo $\varepsilon_{zz} = \gamma_{yz} = \gamma_{xz} = 0$ con la misma fórmula de ε_{eq} — el test cruzado se hace contra esta variante (no contra plane_stress, que es genuinamente distinta).

9.14.2 1. Cinemática

Pequeñas deformaciones, descomposición elástica pura: no hay deformación plástica (sin plasticidad). El daño se manifiesta como degradación de la rigidez efectiva. Sin variables internas tensoriales — solo el historial escalar κ y el daño d .

9.14.3 2. Esfuerzo nominal vs esfuerzo efectivo

$$\boldsymbol{\sigma} = (1 - d) \boldsymbol{\sigma}^{\text{eff}}, \quad \boldsymbol{\sigma}^{\text{eff}} = \mathbf{C}_e \boldsymbol{\varepsilon}$$

con $d \in [0, d_{\max}]$ la variable de daño isótropa y $\mathbf{C}_e$ el tensor constitutivo elástico isótropo 3D 6×6 de `Elastic3D` (sin variantes de hipótesis en 3D).

El esfuerzo efectivo $\boldsymbol{\sigma}^{\text{eff}}$ es el que el material intacto soportaría con la deformación corriente. El esfuerzo nominal $\boldsymbol{\sigma}$ es lo que efectivamente transmite considerando la degradación.

9.14.4 3. Deformación equivalente

Cantidad escalar que cuantifica la “intensidad” de la deformación. Convención del proyecto, norma simétrica simple en notación Voigt 6D $[\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{zz}, \gamma_{xy}, \gamma_{yz}, \gamma_{xz}]$ con $\gamma_{ij} = 2\varepsilon_{ij}$:

$$\varepsilon_{\text{eq}}(\boldsymbol{\varepsilon}) = \sqrt{\varepsilon_{xx}^2 + \varepsilon_{yy}^2 + \varepsilon_{zz}^2 + \frac{1}{2}(\gamma_{xy}^2 + \gamma_{yz}^2 + \gamma_{xz}^2)} = \sqrt{\boldsymbol{\varepsilon}^\top \mathbf{M} \boldsymbol{\varepsilon}}$$

con $\mathbf{M} = \text{diag}(1, 1, 1, 1/2, 1/2, 1/2)$. Equivalente a la norma de Frobenius del tensor deformación 3D ($\boldsymbol{\varepsilon} : \boldsymbol{\varepsilon}$ con $\varepsilon_{ij} = \gamma_{ij}/2$):

$$\boldsymbol{\varepsilon} : \boldsymbol{\varepsilon} = \sum_i \varepsilon_{ii}^2 + 2 \sum_{i < j} \varepsilon_{ij}^2 = \varepsilon_{xx}^2 + \varepsilon_{yy}^2 + \varepsilon_{zz}^2 + \frac{1}{2}(\gamma_{xy}^2 + \gamma_{yz}^2 + \gamma_{xz}^2)$$

Es **extensión natural** de la convención 2D del proyecto ($\mathbf{M}_{2D} = \text{diag}(1, 1, 1/2)$). Bajo restricción plane strain ($\varepsilon_{zz} = \gamma_{yz} = \gamma_{xz} = 0$), $\varepsilon_{\text{eq}}^{3D}$ se reduce exactamente a $\varepsilon_{\text{eq}}^{2D}$ — propiedad clave para el test cruzado con `IsotropicDamage2D plane_strain` (§13).

Limitación: no distingue tracción de compresión. Para hormigón a compresión con resistencia distinta se necesitaría un split tipo Mazars o de Vree; explícitamente *out-of-scope* (idéntica deuda gemela al 2D).

9.14.5 4. Variable histórica y condición de carga (Kuhn-Tucker)

$$\kappa_{n+1} = \max(\kappa_n, \varepsilon_{\text{eq}}(\boldsymbol{\varepsilon}_{n+1})), \quad \kappa_0 \text{ inicial}$$

Esto codifica:

- **Carga activa** ($\dot{\kappa} > 0$): κ crece con ε_{eq} ; el daño puede aumentar.
- **Descarga / recarga** ($\dot{\kappa} = 0$): κ queda congelado; el daño no cambia.
- **Irreversibilidad:** κ es monótono no decreciente.

9.14.6 5. Ley de evolución del daño

$$d(\kappa) = \begin{cases} 0 & \kappa \leq \kappa_0 \\ 1 - \frac{\kappa_0}{\kappa} e^{-\alpha(\kappa - \kappa_0)} & \kappa > \kappa_0 \end{cases}$$

con saturación numérica $d \leq d_{\text{max}} = \text{DAMAGE_MAX}$ (evita rigidez nula).

Propiedades idénticas al 2D: $d(\kappa_0) = 0$ (continuidad); $d'(\kappa) > 0$ para $\kappa > \kappa_0$ (irreversibilidad); $\lim_{\kappa \rightarrow \infty} d = 1$ (asintótico); α controla la velocidad de degradación. **Misma función de ablandamiento** que en 1D y 2D — centralizada en `solidum.materials._softening.evaluate_exponential_damage`.

9.15 Formulación numérica

9.15.1 6. Algoritmo en un paso

Dado $\{\kappa_n\}$ y $\boldsymbol{\varepsilon}_{n+1}$:

1. $\varepsilon_{\text{eq}} = \sqrt{\boldsymbol{\varepsilon}^\top \mathbf{M} \boldsymbol{\varepsilon}}$ (norma 6D).

2. Si $\varepsilon_{\text{eq}} > \kappa_n$: **carga activa**, $\kappa_{n+1} = \varepsilon_{\text{eq}}$. Si no: **descarga**, $\kappa_{n+1} = \kappa_n$.
3. Aplicar ley de daño con κ_{n+1} ; saturar a d_{max} .
4. $\boldsymbol{\sigma}_{n+1} = (1 - d) \mathbf{C}_e \boldsymbol{\varepsilon}_{n+1}$.
5. Tangente: **secante** en descarga / sin daño / saturación; **algorítmica consistente** en carga activa con daño no saturado (ver §7).

El algoritmo es **explícito**: no requiere Newton local. La actualización de κ y d es directa, idéntica al 2D.

9.15.2 7. Tangente algorítmica consistente

$$\mathbf{C}^{\text{alg}} = \frac{\partial \boldsymbol{\sigma}_{n+1}}{\partial \boldsymbol{\varepsilon}_{n+1}}$$

Derivando $\boldsymbol{\sigma} = (1 - d) \mathbf{C}_e \boldsymbol{\varepsilon}$:

$$\mathbf{C}^{\text{alg}} = (1 - d) \mathbf{C}_e - \boldsymbol{\sigma}^{\text{eff}} \otimes \frac{\partial d}{\partial \boldsymbol{\varepsilon}}$$

con $\boldsymbol{\sigma}^{\text{eff}} = \mathbf{C}_e \boldsymbol{\varepsilon}$ (vector 6×1) y la derivada del daño:

$$\frac{\partial d}{\partial \boldsymbol{\varepsilon}} = \frac{\partial d}{\partial \kappa} \frac{\partial \kappa}{\partial \boldsymbol{\varepsilon}}$$

donde:

- $\frac{\partial d}{\partial \kappa} = (1 - d) \left(\frac{1}{\kappa} + \alpha \right)$ para la ley exponencial (idéntica al 2D, derivación independiente de la dimensión).
- $\frac{\partial \kappa}{\partial \boldsymbol{\varepsilon}}$ depende del régimen:
- **Carga activa** ($\varepsilon_{\text{eq}} > \kappa_n$, $\kappa = \varepsilon_{\text{eq}}$):

$$\frac{\partial \kappa}{\partial \boldsymbol{\varepsilon}} = \frac{\partial \varepsilon_{\text{eq}}}{\partial \boldsymbol{\varepsilon}} = \frac{1}{\varepsilon_{\text{eq}}} \mathbf{M} \boldsymbol{\varepsilon} = \frac{1}{\varepsilon_{\text{eq}}} \begin{bmatrix} \varepsilon_{xx} \\ \varepsilon_{yy} \\ \varepsilon_{zz} \\ \frac{1}{2} \gamma_{xy} \\ \frac{1}{2} \gamma_{yz} \\ \frac{1}{2} \gamma_{xz} \end{bmatrix}$$

- **Descarga** ($\varepsilon_{\text{eq}} \leq \kappa_n$): $\partial \kappa / \partial \boldsymbol{\varepsilon} = 0$.

Tangente final:

$$\mathbf{C}^{\text{alg}} = \begin{cases} \mathbf{C}_e & \kappa \leq \kappa_0 \text{ (sin daño)} \\ (1 - d) \mathbf{C}_e & \text{descarga} \\ (1 - d) \mathbf{C}_e - \frac{(1 - d)(1/\kappa + \alpha)}{\varepsilon_{\text{eq}}} (\mathbf{C}_e \boldsymbol{\varepsilon}) \otimes (\mathbf{M} \boldsymbol{\varepsilon}) & \text{carga activa, } d < d_{\text{max}} \\ (1 - d_{\text{max}}) \mathbf{C}_e & d = d_{\text{max}} \text{ (saturación)} \end{cases}$$

con $\mathbf{M} = \text{diag}(1, 1, 1, 1/2, 1/2, 1/2)$ (6×6).

9.15.3 8. Pérdida de simetría

El producto externo $(\mathbf{C}_e \boldsymbol{\varepsilon}) \otimes (\mathbf{M} \boldsymbol{\varepsilon})$ es una matriz 6×6 que **no es simétrica en general**. $\mathbf{C}_e \boldsymbol{\varepsilon}$ y $\mathbf{M} \boldsymbol{\varepsilon}$ no son proporcionales salvo en estados de deformación muy particulares (e.g. uniaxial puro alineado con ejes principales).

Consecuencia: **IS_SYMMETRIC = False obligatorio**. El despachador algebraico (ADR 0003) elige LU en lugar de Cholesky/LDL^T para sistemas globales que incluyan este material. Coste por paso ~30-50 % mayor, compensado con creces por la convergencia cuadrática del Newton global (típicamente 2-3 iteraciones en lugar de 5-10 con tangente secante).

9.15.4 9. Decisión de régimen durante el Newton

Idéntica al 2D: la transición loading↔unloading se decide con κ_n committed (no con κ trial dentro del Newton). Estándar y evita oscilaciones del Newton entre ramas en pasos con ambigüedad. Si el paso converge, κ se compromete y la decisión es consistente con la trayectoria.

9.16 Contrato de implementación

```
name: IsotropicDamage3D
kind: material
status: validated          # draft → implemented → validated

interface:
  strain_dim: 6
  primary_state_var: damage
  is_symmetric: false      # tangente consistente no simétrica (ver §8)

parameters:
- { name: E,               type: float, required: true, desc: "Módulo de Young intacto (>0)" }
- { name: nu,              type: float, required: true, desc: "Coeficiente de Poisson (-1, 0.5)" }
- { name: kappa_0,         type: float, required: true, desc: "Umbral de deformación equivalente; daño nulo mientras _0 (>0)" }
- { name: alpha,           type: float, required: true, desc: "Velocidad de degradación exponencial (>0). mayor → ablandamiento más abrupto" }
- { name: density,         type: float, required: false, default: null, desc: "Densidad (ADR 0008)" }

signature:
  compute_state: "(: ndarray(6,), state_vars=None) -> (: ndarray(6,), C_tan: ndarray(6,6), state_vars)"
  strain_kind: "3D Voigt - [_xx, _yy, _zz, _xy, _yz, _xz] con _ij = 2·_ij"

state_schema:
  kappa: "float _0 - máxima deformación equivalente histórica"
  damage: "float [0, DAMAGE_MAX] - variable de daño escalar isotropa"

conventions:
  sign: " > 0 tracción; el modelo no distingue tracción de compresión en la activación del daño"
  voigt: "_ij = 2·_ij en deformación; _eq usa M = diag(1, 1, 1, 1/2, 1/2, 1/2)"
  damage_irreversibility: " monótono no decreciente vía max(_old, _eq); d() creciente "
```



```

validity:
- "E > 0,  $\nu$  > 0,  $\nu$  > 0"
- "  $\nu$  (-1, 0.5) estricto"

out_of_scope:
- "split tracción/compresión (Mazars, de Vree, Mises modificado); el modelo daña por igual en cualquier régimen"
- "anisotropía del daño (tensor de daño D vs escalar)"
- "regularización para evitar mesh-dependency en regime de ablandamiento (longitud característica, no-local, gradient)"
- "acoplamiento con plasticidad"
- "fatiga / acumulación cíclica del daño"

numerical_caveats:
- "Tangente consistente NO simétrica → IS_SYMMETRIC=False obliga al despachador algebraico a usar LU (ADR 0003). Costo por paso ~30-50% mayor que Cholesky pero convergencia cuadrática del Newton global lo compensa (Simó-Ju 1987)."
```

- "Transición loading/unloading discontinua en la tangente. Newton converge antes de oscilar entre ramas en casos típicos. Para problemas patológicos cerca del umbral, considerar arc-length."

- "Saturación d=DAMAGE_MAX corta la corrección consistente; el algoritmo devuelve tangente secante en ese caso para evitar términos espurios."

- "Sin regularización, la solución es mesh-dependent en régimen de ablandamiento (localización en una banda de elementos). Es limitación inherente del modelo continuo de daño isótropo escalar; out-of-scope esta entrega."

```

acceptance:
unit:
- name: paso_elastico_bajo_umbral
  setup: "pequeño 6D con todas las componentes activas,  $\nu$  <  $\nu$ "
  expect: " $\nu$  = C_e· exacto; d=0; C_tan = C_e (sin corrección rank-1)"
  tol_rel: 1.0e-12

- name: damage_evolution_exponential
  setup: "grande tal que  $\nu$  >>  $\nu$ "
  expect: "d cumple la ley exponencial; estado coincide con la fórmula directa"
  tol_rel: 1.0e-10

- name: rejects_loading_below_threshold
  setup: " $\nu$  apenas por encima de  $\nu$  (banda elástica + bandera de carga activa)"
  expect: "d > 0 minúsculo;  $\nu$  =  $\nu$ ; tangente con corrección rank-1"

- name: tangent_equals_secant_on_unloading
  setup: "carga hasta  $\nu$  >  $\nu$ ; descarga con  $\nu$  de menor norma"
  expect: "C_tan = (1-d)·C_e exacto (segundo término se anula con  $\nu$  / = 0)"
  tol_rel: 1.0e-12

- name: tangent_equals_secant_below_threshold
  setup: "pequeño,  $\nu$  =  $\nu$  (estado inicial intacto)"
  expect: "C_tan = C_e (sin daño activo)"
  tol_rel: 1.0e-12

- name: tangent_equals_secant_at_saturation
  setup: "carga hasta enorme tal que d alcanza DAMAGE_MAX"
  expect: "C_tan = (1-DAMAGE_MAX)·C_e (el término consistente se corta)"
  tol_rel: 1.0e-12

- name: tangent_not_symmetric_in_loading

```

```

    setup: "carga activa con estado de deformación no degenerado (_xx _yy _zz,
        cortantes activos)"
    expect: "||C_tan - C_tan^||T_F / |||C_tan_F 0.01 (asimetría significativa)"

- name: tangent_finite_difference_consistency
  setup: "carga activa; comparar C_alg cerrada con diferencia finita centrada"
  expect: "||C_FD -|| C_alg / |||C_alg < 1e-5"
  tol_rel: 1.0e-5

- name: kappa_monotonic
  setup: "trayectoria multi-paso con carga, descarga, recarga"
  expect: " es monótono no decreciente en todos los pasos"

- name: descarga_recarga_irreversibilidad
  setup: "cargar a _load, descargar elásticamente, recargar"
  expect: "d permanece constante durante toda la descarga; al recargar, d sigue
        creciendo desde el valor de descarga"

- name: unit_invariance_MPa_vs_Pa
  setup: "mismo path adimensional con (MPa,mm,N) y (Pa,m,N)"
  expect: "d, y /E idénticos (_0 es una deformación, es adimensional →
        invariancia exacta)"
  tol_rel: 1.0e-12

- name: rechazo_inputs_invalidos
  setup: "construir con E0, _00, 0, (-1, 0.5), density<0"
  expect: "ValueError con mensaje claro en cada caso"

cross_consistency:
- name: equivalencia_plane_strain
  setup: "Damage3D con = (_xx, _yy, 0, _xy, 0, 0); Damage2D plane_strain con =
        (_xx, _yy, _xy);
        path multi-paso con carga, descarga, recarga"
  expect: "_xx, _yy, _xy, , d coinciden exactamente entre 3D y 2D PS;
        _zz en VM3D no nulo (consistente con C_e plane_strain interno);
        componentes 3D ausentes en 2D PS (_yz, _xz) permanecen nulas"
  tol_rel: 1.0e-12

integration:
- name: hex8_elastico
  setup: "Hex8 unitario con carga uniforme muy por debajo del umbral"
  expect: "d = 0 en todos los Gauss points; = C_e· exacto"
  tol_rel: 1.0e-8

- name: hex8_damage_newton_converges_in_few_iter
  setup: "Hex8 unitario, carga uniaxial hasta régimen plenamente dañado ( >> _0),
        8 pasos"
  expect: "Newton converge en 6 iteraciones por paso (evidencia de convergencia
        cuadrática con tangente consistente)"

- name: tet4_basic_pipeline
  setup: "Tet4 con confinamiento + tracción uniaxial, carga que activa daño"
  expect: "convergencia, d > 0 en el único Gauss point, _0"

references:
- "Lemaitre J., Chaboche J.-L. (1990). Mechanics of Solid Materials. Cambridge UP.
    Cap. 7 (continuum damage mechanics)."
```

```

- "Simó J.C., Ju J.W. (1987). Strain- and stress-based continuum damage models - I.
    Formulation. Int. J. Solids Struct. 23, 821-840 (tangente consistente para daño
```

```
isótropo)."
```

- "Oliver J. (1989). A consistent characteristic length for smeared cracking models. IJNME 28, 461-474."
- "de Souza Neto E.A., ĆPeri D., Owen D.R.J. (2008). Computational Methods for Plasticity. Wiley. §12 (damage models)."
- "ADR 0012 - Sólidos 3D: convención Voigt 6D del proyecto."
- "ADR 0003 - Despachador algebraico: tangente asimétrica fuerza LU global."
- "ADR 0006 - Tolerancias adimensionales: patrón atol + rtol·escala con escala física E·_0."

9.17 Implementación

- **Archivo:** solidum/materials/damage_3d.py.
- **Clase:** IsotropicDamage3D, registrada vía @MaterialRegistry.register. Declara IS_SYMMETRIC = False como ClassVar (tangente consistente asimétrica).
- **Composición:**
 - Elastic3D como base elástica (acceso a $\mathbf{C}_e$ 6×6 vía self.elastic_base.C).
 - evaluate_exponential_damage de solidum.materials._softening para la ley de softening (centralizada, compartida con IsotropicDamage1D e IsotropicDamage2D).
 - **Estado:** dict {'kappa': float, 'damage': float}. PRIMARY_STATE_VAR = 'damage'.
 - **Algoritmo compute_state:** idéntica estructura a IsotropicDamage2D extendida a Voigt 6D. Sin Numba (operaciones numpy puras; el coste dominante es el producto matriz-vector $\mathbf{C}_e \cdot \boldsymbol{\varepsilon}$, no se beneficia significativamente de JIT).
 - **admissibility_scale:** $E \cdot \kappa_0$ constante respecto al estado (idéntico al patrón 1D/2D — el criterio se mide contra κ_0 , no contra κ corriente).
- **Tests:**
 - tests/test_materials_unit.py · TestIsotropicDamage3D (12 tests unitarios) + TestIsotropicDamage3DvsPla (1 cruzado contra Damage2D plane_strain).
 - tests/test_solid_3d_plasticity.py · 3 tests de integración: TestHex8IsotropicDamage3DElastic (1 — sin daño), TestHex8IsotropicDamage3DActiveLoad (1 — daño activo con tangente asimétrica), TestTet4IsotropicDamage3DBasic (1 — smoke Tet4).
- **Notas de traducción:**
 - Matriz $\mathbf{M} = \text{diag}(1, 1, 1, 1/2, 1/2, 1/2)$ aplicada implícitamente en código (factor 1/2 sobre los slots cortantes de $\boldsymbol{\varepsilon}$) en lugar de alocar la 6×6, idéntico patrón al 2D.
 - Convención engineering input (γ_{ij}): el factor 1/2 en $\mathbf{M}$ y en $\partial \varepsilon_{eq} / \partial \boldsymbol{\varepsilon}$ corrige automáticamente para que la norma corresponda a la Frobenius del tensor ($\gamma_{ij}/2 = \varepsilon_{ij_tens}$).
 - Validación de parámetros en constructor: $E > 0$, $\kappa_0 > 0$, $\alpha > 0$, $\nu \in (-1, 0.5)$, $\text{density} \geq 0$ si se pasa. Mensajes en español.

9.18 Diálogo

- **2026-05-21** · Spec pre-redactada por la IA al cerrar el segundo material de A.bis (**DruckerPrager3D**). Base: **IsotropicDamage2D.md** (modelo físico, tangente algorítmica consistente, semánticas carga/descarga, cap **DAMAGE_MAX**) + **VonMises3D.md/DruckerPrager3D.md** (estructura Voigt 6D, patrón cross-consistency).
- **2026-05-21** · **Decisión de plumbing (decide la IA, Reglas §3)**: reuso de la función centralizada **evaluate_exponential_damage** de **solidum.materials._softening**. Esta función ya está compartida entre **IsotropicDamage1D** e **IsotropicDamage2D** (auditoría H-3.4 cerrada el 2026-05-19); añadir **IsotropicDamage3D** como tercer consumidor satisface la regla de centralización del proyecto. Sin duplicación de la ley de softening.
- **2026-05-21** · **Decisión de plumbing**: el material usa **Elastic3D** internamente como base elástica (paralelo a cómo **IsotropicDamage2D** usa **Elastic2D**). Acceso a la matriz constitutiva 6×6 vía **self.elastic_base.C** — idéntico patrón al 2D, sin duplicación de la construcción de $\mathbf{C}_e$.
- **2026-05-21** · **Decisión de plumbing**: la matriz **M** del cálculo de ε_{eq} y de la derivada $\partial\kappa/\partial\varepsilon$ se materializa implícitamente en código (multiplicaciones por 1/2 sobre los slots cortantes) en lugar de allocar una 6×6 — ahorro mínimo de memoria pero código más legible y consistente con el patrón seguido en **damage_2d.py**.
- **2026-05-21** · **Cross-consistency** contra **IsotropicDamage2D** **plane_strain**, no **plane_stress**. La equivalencia matemática es exacta bajo $\varepsilon_{zz} = \gamma_{yz} = \gamma_{xz} = 0$ impuesto (**plane strain**): la matriz $\mathbf{C}_e$ 3D restringida a $\varepsilon_{zz}=0$ input se reduce a la matriz $\mathbf{C}_e^{\text{plane strain } 2D}$, y ε_{eq}^{3D} se reduce a ε_{eq}^{2D} bajo la misma restricción. **Plane stress 2D** usa un $\mathbf{C}_e$ proyectado distinto y por tanto no es comparable directamente con 3D bajo restricción cinemática.
- **2026-05-21** · **Sin variantes de hipótesis** en 3D (igual que VM3D, DP3D, Elastic3D). Constructor más simple que el 2D.
- **2026-05-21** · **No se incluye consistent tangent para IsotropicDamage1D** (deuda gemela del 2D): fuera del alcance de esta entrega A.bis. La fórmula 1D es trivial reducción de la 3D si se prioriza después.
- **2026-05-21** · **Implementación completa, status validated**. Clase + 12 tests unitarios + 1 test cruzado + 3 tests de integración (Hex8 elástico, Hex8 con daño activo + tangente asimétrica, Tet4 smoke). **16 tests añadidos a la suite (848 → 866 verdes; suite global sin regresiones)**. Todos los tests pasaron a la primera, incluyendo:
 - Cruzado **Damage3D ↔ Damage2D plane_strain** a **14 decimales en d** y κ y 10 decimales en σ — la equivalencia bajo $\varepsilon_{zz} = \gamma_{yz} = \gamma_{xz} = 0$ es exacta a precisión máquina (misma ley centralizada, mismo $\mathbf{C}_e$ **plane_strain** bajo restricción cinemática).
 - Diferencia finita de la tangente algorítmica vs cerrada con error rel. $< 1e-5$ en las 36 componentes 6×6 en carga activa.
 - Asimetría significativa de la tangente en estados no degenerados (norma rel. ≥ 0.01).

- Invariancia bajo cambio de unidades (E en MPa vs Pa) — $\mathbf{d}$, κ idénticos a 12 decimales; σ escala linealmente con E.

Con esta entrega cierra la sub-etapa A.bis (materiales 3D no lineales): VonMises3D + DruckerPrager3D + IsotropicDamage3D. Validación contra benchmark publicado (e.g. ensayo tensión-softening con valor cuantitativo de carga máxima en barra cargada axialmente con softening exponencial calibrado por G_f efectivo) **diferida a la campaña 3D consolidada** que se hará al cierre de A.bis junto con Hill 3D para VM3D y esfera hueca para DP3D.

9.19 VonMises3D

*Orden de trabajo. El usuario revisa **especificación física, formulación numérica y contrato**; la IA propone, ejecuta y rellena **implementación + diálogo** durante el trabajo.*

*>Pre-redactada por la IA sobre la base de `VonMises2D.md` y `Elastic3D.md`. Apertura de la sub-etapa **A.bis** (materiales 3D no lineales) tras cierre de la Etapa 7 (sólidos 3D acotados con ADR 0012).*

9.20 Especificación física

9.20.1 0. Descripción general

Modelo elastoplástico independiente de la velocidad (rate-independent”) basado en el criterio **J2 / Von Mises** con regla de flujo **asociada** y **endurecimiento isótropo lineal**, formulado **íntegramente en 3D** sobre la convención Voigt 6D del proyecto (ADR 0012). Captura plasticidad de metales dúctiles en régimen de pequeñas deformaciones, sin efectos cinemáticos (Bauschinger), térmicos ni viscosos.

A diferencia de VonMises2D, **no tiene variantes de hipótesis cinemática**: en 3D todas las componentes de $\boldsymbol{\varepsilon}$ y $\boldsymbol{\sigma}$ son activas; no hay proyección a un subespacio. La consecuencia es algorítmica: el return mapping es **único y radial cerrado**, sin Newton local proyectado.

VonMises2D en hipótesis **plane strain** es matemáticamente la restricción de este modelo bajo $\varepsilon_{zz} = \gamma_{yz} = \gamma_{xz} = 0$ (deformación total impuesta); las componentes plásticas $\varepsilon_{zz}^p, \varepsilon_{yz}^p, \varepsilon_{xz}^p$ son libres y evolucionan por incompresibilidad/isotropía. Esta relación se verifica como test cruzado en §13.

9.20.2 1. Descomposición aditiva

En pequeñas deformaciones, el tensor de deformación se descompone aditivamente:

$$\boldsymbol{\varepsilon} = \boldsymbol{\varepsilon}^e + \boldsymbol{\varepsilon}^p$$

La parte elástica $\boldsymbol{\varepsilon}^e$ gobierna el esfuerzo vía la ley elástica isótropa; la parte plástica $\boldsymbol{\varepsilon}^p$ es **histórica** y evoluciona según la regla de flujo. La parte plástica es **incompresible**: $\text{tr}(\boldsymbol{\varepsilon}^p) = 0$.

9.20.3 2. Ley elástica

$$\boldsymbol{\sigma} = \mathbf{C}_e : \boldsymbol{\varepsilon}^e = \mathbf{C}_e : (\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}^p)$$

con $\mathbf{C}_e$ la matriz constitutiva isótropa 3D de Elastic3D (Voigt 6×6), parametrizada por (E, ν) o equivalentemente (K, G) :

$$K = \frac{E}{3(1-2\nu)}, \quad G = \frac{E}{2(1+\nu)}$$

Equivalente desviador + presión:

$$\boldsymbol{\sigma} = \mathbf{s} + p\mathbf{I}, \quad \mathbf{s} = 2G\mathbf{e}^e, \quad p = K \text{tr}(\boldsymbol{\varepsilon})$$

con $\mathbf{e}^e = \text{dev}(\boldsymbol{\varepsilon}^e)$ y, por incompresibilidad plástica, $\text{tr}(\boldsymbol{\varepsilon}^e) = \text{tr}(\boldsymbol{\varepsilon})$.

9.20.4 3. Superficie de fluencia (J2)

$$f(\boldsymbol{\sigma}, \alpha) = \|\mathbf{s}\| - \sqrt{\frac{2}{3}} (\sigma_y + H \alpha) \leq 0$$

donde $\mathbf{s} = \boldsymbol{\sigma} - \frac{1}{3} \text{tr}(\boldsymbol{\sigma}) \mathbf{I}$ es el tensor desviador, $\|\mathbf{s}\| = \sqrt{\mathbf{s} : \mathbf{s}}$ la norma de Frobenius tensorial, $\sigma_y > 0$ la fluencia inicial y $H \geq 0$ el módulo de endurecimiento isótropo lineal. El factor $\sqrt{2/3}$ alinea σ_y con la fluencia uniaxial estándar.

Cómputo de $\|\mathbf{s}\|$ en Voigt 6D del proyecto (componentes cortantes tensoriales sin factor 2):

$$\|\mathbf{s}\|^2 = s_{xx}^2 + s_{yy}^2 + s_{zz}^2 + 2(s_{xy}^2 + s_{yz}^2 + s_{xz}^2)$$

El factor 2 contabiliza las dos posiciones simétricas $s_{ij} = s_{ji}$ del tensor de 2º orden.

9.20.5 4. Regla de flujo (asociada)

$$\dot{\boldsymbol{\epsilon}}^p = \dot{\gamma} \frac{\partial f}{\partial \boldsymbol{\sigma}} = \dot{\gamma} \mathbf{N}, \quad \mathbf{N} = \frac{\mathbf{s}}{\|\mathbf{s}\|}$$

El flujo es puramente desviador ($\text{tr}(\mathbf{N}) = 0$) — **plasticidad incompresible**, propiedad central de J2. $\mathbf{N}$ es un tensor de 2º orden de norma unidad almacenado en Voigt 6D con cortantes tensoriales.

9.20.6 5. Endurecimiento isótropo lineal

$$\dot{\alpha} = \sqrt{\frac{2}{3}} \dot{\gamma}$$

α es la **deformación plástica acumulada equivalente** (escalar, monótona no decreciente). La superficie de fluencia se expande uniformemente con α manteniendo su centro en el origen del espacio de tensiones desviadoras. La fórmula es **idéntica a plane strain** porque la dimensión de $\mathbf{N}$ es la misma (norma unidad tensorial); contrasta con plane stress, donde $\Delta\gamma$ tiene unidades de [1/esfuerzo] por la proyección.

9.20.7 6. Condiciones de Kuhn-Tucker

$$\dot{\gamma} \geq 0, \quad f \leq 0, \quad \dot{\gamma} f = 0$$

Garantizan que $\dot{\gamma} > 0$ solo si el estado intenta salir de la superficie, y que el estado real siempre cumple $f \leq 0$.

9.20.8 7. Variables internas

Símbolo	Tipo	Significado
$\boldsymbol{\epsilon}^p$	tensor (6 componentes Voigt, ver §8)	deformación plástica
α	escalar	deformación plástica acumulada equivalente

`alpha` es la variable principal exportable (`PRIMARY_STATE_VAR = 'alpha'`); `eps_p` es interna al algoritmo de return mapping.

9.20.9 8. Notación Voigt y manejo de ε^p

Convención del proyecto para 3D (ADR 0012):

$$\varepsilon = [\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{zz}, \gamma_{xy}, \gamma_{yz}, \gamma_{xz}]^\top, \quad \gamma_{ij} = 2\varepsilon_{ij}$$

$$\sigma = [\sigma_{xx}, \sigma_{yy}, \sigma_{zz}, \sigma_{xy}, \sigma_{yz}, \sigma_{xz}]^\top$$

Almacenamiento interno de ε^p : 6 componentes con cortantes **tensoriales sin factor 2**:

$$\varepsilon^p = [\varepsilon_{xx}^p, \varepsilon_{yy}^p, \varepsilon_{zz}^p, \varepsilon_{xy}^p, \varepsilon_{yz}^p, \varepsilon_{xz}^p]$$

Decisión consistente con **VonMises2D** (que almacena ε_{xy}^p tensorial en su slot 4). Razón: el flujo plástico $\mathbf{N} = \mathbf{s}/\|\mathbf{s}\|$ es un tensor de 2º orden con cortantes tensoriales; almacenar ε^p en la misma convención evita conversiones repetidas en el return mapping y mantiene la fórmula $\|\varepsilon^p\|^2 = \sum_{\text{diag}} + 2\sum_{\text{off}}$ idéntica a la de $\mathbf{s}$.

Traducción al entrar al kernel: la deformación total llega con γ *engineering*; las componentes cortantes se dividen por 2 una sola vez al construir el tensor para deviator decomposition. Al salir, σ se devuelve en Voigt 6D *engineering-compatible* (cortantes tensoriales sin factor — esto es lo que el ensamblador espera).

9.20.10 9. Relación con VonMises2D plane strain

VonMises2D plane strain es la restricción de este modelo bajo:

$$\varepsilon_{zz} = \gamma_{yz} = \gamma_{xz} = 0 \quad (\text{deformación total impuesta por la hipótesis 2D})$$

con $\varepsilon_{zz}^p \neq 0$ libre (incompresibilidad plástica). El kernel 2D-plane-strain mantiene 4 componentes de ε^p porque $\varepsilon_{yz}^p = \varepsilon_{xz}^p = 0$ por isotropía + ausencia de cortantes activos en esos planos.

Implicación de testeo: si se ejecuta **VonMises3D** con $\varepsilon = [\varepsilon_{xx}, \varepsilon_{yy}, 0, \gamma_{xy}, 0, 0]$ (path plane strain), debe reproducir $\sigma_{xx}, \sigma_{yy}, \sigma_{xy}, \alpha$ de **VonMises2D plane strain** exactamente, además de devolver el σ_{zz} que **VonMises2D** calcula internamente pero no expone en el API público 2D. Ver §13.

9.21 Formulación numérica

9.21.1 10. Esquema temporal — Backward Euler implícito

Dado el estado convergido $\{\varepsilon_n^p, \alpha_n\}$ al paso n y la deformación total prescrita ε_{n+1} , resolver para $\{\sigma_{n+1}, \varepsilon_{n+1}^p, \alpha_{n+1}\}$ con discretización implícita:

$$\varepsilon_{n+1}^p = \varepsilon_n^p + \Delta\gamma \mathbf{N}_{n+1}, \quad \alpha_{n+1} = \alpha_n + \sqrt{\frac{2}{3}} \Delta\gamma$$

más $f(\sigma_{n+1}, \alpha_{n+1}) \leq 0$ con la condición de complementariedad.

9.21.2 11. Return mapping 3D — algoritmo radial cerrado

Descomposición volumétrica-desviadora del estado de prueba. Conversión de ε a tensorial dividiendo por 2 los cortantes engineering:

$$\varepsilon^{\text{tens}} = [\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{zz}, \frac{\gamma_{xy}}{2}, \frac{\gamma_{yz}}{2}, \frac{\gamma_{xz}}{2}]$$

(esta es la conversión que ya hace **Elastic3D** implícitamente vía la matriz $\mathbf{C}_e$ con sus factores $(1 - 2\nu)/2$ en los cortantes).

Deformación de prueba elástica y su descomposición:

$$\boldsymbol{\varepsilon}_{n+1}^{e,\text{trial}} = \boldsymbol{\varepsilon}_{n+1}^{\text{tens}} - \boldsymbol{\varepsilon}_n^p$$

$$p^{\text{trial}} = K \operatorname{tr}(\boldsymbol{\varepsilon}_{n+1}) = K (\varepsilon_{xx} + \varepsilon_{yy} + \varepsilon_{zz})$$

(la traza no depende de la sustracción de $\boldsymbol{\varepsilon}_n^p$ porque $\operatorname{tr}(\boldsymbol{\varepsilon}^p) = 0$.)

$$\mathbf{e}^{\text{trial}} = \operatorname{dev}(\boldsymbol{\varepsilon}_{n+1}^{e,\text{trial}}), \quad \mathbf{s}^{\text{trial}} = 2G \mathbf{e}^{\text{trial}}$$

con $\mathbf{e}^{\text{trial}}$ y $\mathbf{s}^{\text{trial}}$ ambos en Voigt 6D tensorial.

Función de fluencia trial:

$$f^{\text{trial}} = \|\mathbf{s}^{\text{trial}}\| - \sqrt{\frac{2}{3}} (\sigma_y + H \alpha_n)$$

con $\|\mathbf{s}^{\text{trial}}\|^2 = \sum_{\text{diag}} s_i^2 + 2 \sum_{\text{off}} s_{ij}^2$ (§3).

Si $f^{\text{trial}} \leq \mathbf{tol} \rightarrow$ paso elástico. $\boldsymbol{\sigma}_{n+1} = \mathbf{C}_e \boldsymbol{\varepsilon}_n^e + \mathbf{C}_e \Delta \boldsymbol{\varepsilon}$ equivalente, evaluado como $\mathbf{s}^{\text{trial}} + p^{\text{trial}} \mathbf{I}$. Estado interno intacto.

Si $f^{\text{trial}} > 0 \rightarrow$ corrector plástico radial. Por isotropía + asociado en J2, la dirección de flujo no cambia entre trial y final: $\mathbf{N}_{n+1} = \mathbf{N}^{\text{trial}} = \mathbf{s}^{\text{trial}} / \|\mathbf{s}^{\text{trial}}\|$. Forma cerrada:

$$\Delta\gamma = \frac{f^{\text{trial}}}{2G + \frac{2}{3}H}$$

Actualización (todo en Voigt 6D tensorial):

$$\mathbf{s}_{n+1} = \mathbf{s}^{\text{trial}} - 2G \Delta\gamma \mathbf{N}, \quad \boldsymbol{\sigma}_{n+1} = \mathbf{s}_{n+1} + p^{\text{trial}} \mathbf{I}$$

$$\boldsymbol{\varepsilon}_{n+1}^p = \boldsymbol{\varepsilon}_n^p + \Delta\gamma \mathbf{N}, \quad \alpha_{n+1} = \alpha_n + \sqrt{\frac{2}{3}} \Delta\gamma$$

Coste: $O(1)$ por punto de Gauss. Sin Newton local. **Algoritmo idéntico al de plane strain extendido a 6D** — solo cambian los índices de las componentes activas.

9.21.3 12. Tangente algorítmica consistente

Forma cerrada estándar (Simó-Hughes 1998, §3.3):

$$\mathbf{C}^{\text{alg}} = K \mathbf{1} \otimes \mathbf{1} + 2G(1 - \beta) \mathbf{I}_{\text{dev}} - 2G \bar{\gamma} \mathbf{N} \otimes \mathbf{N}$$

con

$$\beta = \frac{2G \Delta\gamma}{\|\mathbf{s}^{\text{trial}}\|}, \quad \bar{\gamma} = \frac{1}{1 + H/(3G)} - \beta$$

En Voigt 6D del proyecto:

- $\mathbf{1} \otimes \mathbf{1}$: matriz 6×6 con 1 en las entradas (i, j) con $i, j \in \{1, 2, 3\}$ y 0 resto.

- $\mathbf{I}_{\text{dev}}$: proyector desviador 6×6 . En la base tensorial, $\mathbf{I}_{\text{dev}} = \mathbf{I} - \frac{1}{3}\mathbf{1} \otimes \mathbf{1}$ en el bloque diagonal (3×3) y un bloque identidad escalado en los cortantes con el factor adecuado para que $\mathbf{I}_{\text{dev}} \boldsymbol{\varepsilon} = \text{dev}(\boldsymbol{\varepsilon})$ cuando $\boldsymbol{\varepsilon}$ está en convención engineering. En la convención del proyecto (entrada engineering, salida engineering), el bloque cortante de $\mathbf{I}_{\text{dev}}$ es la identidad 3×3 simple.
- $\mathbf{N} \otimes \mathbf{N}$: producto exterior 6×6 de $\mathbf{N}$ (Voigt 6D tensorial) consigo mismo. **Cuidado con la convención:** el producto $\mathbf{N} \otimes \mathbf{N}$ del kernel debe ser consistente con $\mathbf{C}^{\text{alg}} \boldsymbol{\varepsilon}$ engineering. La forma simple: si $\mathbf{N}$ está almacenado tensorial (sin factor 2 en cortantes), entonces $\mathbf{C}^{\text{alg}}$ en convención engineering tiene $\mathbf{N} \otimes \mathbf{N}$ con un factor de 2 en las posiciones cortantes — equivalente a multiplicar las componentes cortantes de $\mathbf{N}$ por $\sqrt{2}$ antes del producto exterior (Mandel-like rescaling local al kernel, idéntico al patrón de VM2D plane strain).

Simetría: $\mathbf{C}^{\text{alg}}$ es simétrica (J2 asociado), `IS_SYMMETRIC = True`.

Coste: precalcular los cuatro escalares $K, G, \beta, \bar{\gamma}$ y construir la 6×6 sin loops anidados — $O(1)$ constante.

9.21.4 13. Caveats numéricos

- **Locking volumétrico** en Hex8/Tet4 cuando la plasticidad domina: la incompresibilidad plástica $\text{tr}(\boldsymbol{\varepsilon}^p) = 0$ combinada con ν no muy lejano de 0.5 produce el mismo locking documentado para Elastic3D (limitación declarada, blindada por test en la Etapa 7). B-bar/F-bar diferidas a caso de uso real.
- $\nu \rightarrow 0,5$: $K \rightarrow \infty$ y el predictor elástico pierde precisión. Constructor rechaza $\nu \geq 0,5$. Recomendado $\nu \leq 0,49$ en metales.
- $H = 0$ (**plasticidad perfecta**): $\Delta\gamma = f^{\text{trial}}/(2G)$, sin singularidades. Carga aplicada que supere la capacidad plástica produce divergencia del Newton global del solver (`LoadExceedsCapacityError`, ADR 0011); responsabilidad del solver, no del material.
- **Tangente algorítmica vs continua:** con $\Delta\gamma > 0$, $\mathbf{C}^{\text{alg}} \neq \mathbf{C}_{\text{continuo}}^{\text{ep}}$ por el término β . Usar la algorítmica garantiza convergencia cuadrática del Newton global.
- **Tolerancia de fluencia** vía ADR 0006: `admissibility_scale =  $\sqrt{(2/3)(\sigma_y + H\alpha)}$` . Idéntica a plane strain — la escala física es la misma norma desviadora en la frontera de fluencia.

9.22 Contrato de implementación

```

name: VonMises3D
kind: material
status: validated           # draft → implemented → validated

interface:
  strain_dim: 6
  primary_state_var: alpha   # deformación plástica acumulada equivalente
  is_symmetric: true         # tangente algorítmica simétrica (J2 asociado)

parameters:
  - { name: E,               type: float, required: true, desc: "Módulo de Young (>0)" }
  - { name: nu,              type: float, required: true, desc: "Coeficiente de Poisson (-1, 0.5)" }
```

```

- { name: sigma_y, type: float, required: true, desc: "Esfuerzo de fluencia inicial (>0)" }
- { name: H, type: float, required: false, default: 0.0, desc: "Módulo de endurecimiento isótropo lineal (0). H=0 plasticidad perfecta" }
- { name: density, type: float, required: false, default: null, desc: "Densidad (kg/m³). Opcional al construir; obligatoria solo si se ensambla peso propio o masa (ADR 0008)" }

signature:
  compute_state: "(: ndarray(6,), state_vars=None) -> (: ndarray(6,), C_alg: ndarray(6,6), state_vars)"
  strain_kind: "3D Voigt - [_xx, _yy, _zz, _xy, _yz, _xz] con _ij = 2·_ij"

state_schema:
  eps_p: "ndarray(6,) - [p_xx, p_yy, p_zz, p_xy, p_yz, p_xz] (cortantes tensoriales, sin factor 2)"
  alpha: "float 0 - deformación plástica acumulada equivalente"

conventions:
  sign: "> 0 tracción (Reglas §5)"
  voigt: "[xx, yy, zz, xy, yz, xz] (ADR 0012); _ij = 2·_ij en deformación total engineering; p_ij almacenada en componente tensorial (sin factor 2); _ij tensorial sin factor"

validity:
- "E > 0, _y > 0, H 0"
- " (-1, 0.5) estricto; = 0.5 rechazado (incompresible)"
- "|| 1e-2 (régimen de pequeñas deformaciones)"

out_of_scope:
- "endurecimiento cinemático (back-stress); usar variante futura VonMises3DKinematic"
- "endurecimiento isótropo no lineal (Voce, exponencial, potencial)"
- "ablandamiento (H<0); requiere regularización por longitud característica o no-local"
- "regla de flujo no asociada"
- "acoplamiento con daño; usar modelo plástico-dañado dedicado"
- "viscoplasticidad y dependencia de la velocidad de deformación"
- "grandes deformaciones (descomposición multiplicativa F = Fe·Fp)"
- "anisotropía (Hill 1948, Barlat); el modelo es estrictamente J2 isótropo"
- "incompresibilidad estricta (=0.5)"

numerical_caveats:
- "Locking volumétrico en Hex8/Tet4 cuando la plasticidad domina (incompresibilidad plástica + moderado-alto). Mitigaciones B-bar/F-bar diferidas. Test específico documenta la limitación."
- " cercano a 0.5: K diverge; recomendado 0.49 (típico =0.3 en metales)."
- "H=0 (plasticidad perfecta): material correcto, divergencia bajo sobrecarga es responsabilidad del solver (ADR 0011)."
```

acceptance:

```

# Cubierto por tests/test_materials_unit.py · TestVonMises3D (a crear).
unit:
- name: paso_elastico_bajo_fluencia
  setup: " = (1e-5, 0, 0, 0, 0, 0): tracción uniaxial por debajo del yield"
  expect: " = C_e· exacto; = 0; eps_p = 0; tangente = C_e"
  tol_rel: 1.0e-12

```

```

- name: yield_uniaxial_at_sigma_y
  setup: "tracción uniaxial monotónica creciente con _yy = _zz = -·_xx, _xy =
        _yz = _xz = 0"
  expect: "primer yield exactamente en _xx = _y; eps_yield = _y/E"
  tol_rel: 1.0e-10

- name: traccion_biaxial_isotropia
  setup: " = (eps, eps, -2·eps/(1-), 0, 0, 0) - tracción biaxial isótropa en plano
        x-y con _zz libre por _zz=0"
  expect: "_xx = _yy convergen a yield uniforme; evoluciona; _zz 0"
  tol_rel: 1.0e-4

- name: cortante_puro_xy_yield
  setup: "_xy creciente con resto de componentes 0"
  expect: "primer yield en _xy = _y√/3 (criterio J2 en cortante puro)"
  tol_rel: 1.0e-10

- name: incompresibilidad_plastica
  setup: "cualquier paso plástico monotónico"
  expect: "tr(eps_p) = ^p_xx + ^p_yy + ^p_zz = 0 exacto"
  tol_abs: 1.0e-12

- name: alpha_monotonic
  setup: "carga monotónica creciente con incrementos plásticos no triviales"
  expect: " no decrece nunca; _final > 0 si hubo plasticidad"

- name: descarga_recupera_C_e
  setup: "tras estado plástico, paso con menor (descarga elástica)"
  expect: " devuelto coherente con eps_p_n acumulado; tangente devuelta = C_e;
        intacto"
  tol_rel: 1.0e-10

- name: tangente_simetrica
  setup: "evaluar C_alg en estado plástico arbitrario"
  expect: "máx |C_alg - C_alg.T| tol; IS_SYMMETRIC = True"
  tol_abs: 1.0e-8

- name: unit_invariance_MPa_vs_Pa
  setup: "mismo path adimensional con (E=210, _y=0.25) y (E=210e9, _y=0.25e9)"
  expect: ", eps_p (adimensionales) idénticos; /E idéntico"
  tol_rel: 1.0e-12

- name: degeneracion_a_elasticidad_sin_plasticidad
  setup: "trayectoria que nunca alcanza la frontera de fluencia"
  expect: "todos los pasos devuelven C_alg = C_e; eps_p y permanecen en cero"
  tol_rel: 1.0e-12

- name: rechazo_inputs_invalidos
  setup: "construir VonMises3D con E0, fuera de rango, _y0, H<0"
  expect: "ValueError con mensaje claro en cada caso"

# Test cruzado VM3D VM2D plane strain.
cross_consistency:
- name: equivalencia_plane_strain
  setup: "ejecutar VM3D con = (_xx, _yy, 0, _xy, 0, 0) y VM2D plane strain con
        = (_xx, _yy, _xy);
        mismo path multi-paso con plasticidad activa"
  expect: "_xx, _yy, _xy y coinciden entre 3D y 2D PS;
        VM3D además devuelve _zz coherente con la incompresibilidad plástica"

```

```

tol_rel: 1.0e-10

# Cubierto por tests/test_solid_3d_plasticity.py (a crear, fuera de esta spec -
# pertenece a la integración Hex8/Tet4 + VM3D).
integration:
- name: pipeline_hex8_traccion_uniaxial_elastoplastica
  setup: "Hex8 unitario, condiciones de contorno equivalentes a tracción uniaxial
        pura
        (4 caras laterales restringidas en su dirección normal, una cara tirada en
        x).
        Carga monotónica que cruza el yield."
  expect: "_xx en el centro coincide con material standalone bajo el path
        equivalente;
        coincide; convergencia Newton global en pocos pasos"
  tol_rel: 1.0e-6

- name: pipeline_hill_cylinder_3d
  setup: "cilindro hueco bajo presión interna creciente discretizado con Hex8 (
        octante con simetrías);
        cargar hasta plasticidad parcial"
  expect: "frontera elasto-plástica radial coincide con solución analítica cerrada
        de Hill (1950)
        al avanzar la presión"
  tol_rel: 1.0e-2

references:
- "Simó J.C., Hughes T.J.R. (1998). Computational Inelasticity. Springer.
  §3.3 (3D J2 return mapping radial cerrado, tangente algorítmica consistente)."
```

```

- "Crisfield M.A. (1991). Non-linear Finite Element Analysis of Solids and Structures
  , Vol. 1. Wiley.
  Cap. 6 (J2 plasticity)."
```

```

- "de Souza Neto E.A., ċPeri D., Owen D.R.J. (2008). Computational Methods for
  Plasticity. Wiley.
  §7 (3D J2 plasticity)."
```

```

- "Hill R. (1950). The Mathematical Theory of Plasticity. Oxford University Press.
  Cap. V (cilindro hueco bajo presión interna, solución analítica)."
```

```

- "Bathe K.J. (2014). Finite Element Procedures. Prentice Hall. §6.6, §6.8.4."
```

```

- "ADR 0012 - Sólidos 3D: convención Voigt 6D del proyecto."
```

```

- "ADR 0006 - Tolerancias adimensionales: patrón atol + rtol·escala con escala física
  ."
```

9.23 Implementación

- **Archivo:** solidum/materials/von_mises_3d.py.
- **Clase:** VonMises3D, registrada vía `@MaterialRegistry.register`. Sin despacho por hipótesis (no aplica en 3D).
- **Kernel Numba _compute_j2_3d:** descomposición volumétrica-desviadora en Voigt 6D tensorial, return mapping radial cerrado, tangente algorítmica consistente cerrada. Predictor elástico construido como $\mathbf{s}_{\text{trial}} + p\mathbf{I}$ con $\mathbf{s}_{\text{trial}} = 2G(\mathbf{e}_{\text{trial}}^{\text{dev}} - \epsilon_n^p)$ — respeta ϵ_n^p en descargas y reevaluaciones (aplicada desde el inicio la lección del bug detectado en VM2D plane strain el 2026-05-14).
- **State schema:** `eps_p ndarray(6,)` [xx, yy, zz, xy, yz, xz] con cortantes tensoriales, `alpha` escalar.

- **admissibility_scale**: $\sqrt{(2/3) \cdot (\sigma_y + H \cdot \alpha)}$ — idéntica a VM2D plane strain (misma escala física en la frontera J2).
- **Tests**:
- `tests/test_materials_unit.py · TestVonMises3D` (12 tests unitarios) + `TestVonMises3DvsPlaneStrain` (1 test cruzado VM3D $\leftrightarrow$ VM2D plane strain con path multi-paso).
- `tests/test_solid_3d_plasticity.py · 5 tests de integración`: `TestHex8VonMises3DUniaxialFree` (2 — cubo libre con Poisson, yield uniaxial), `TestHex8VonMises3DConfinedVsStandalone` (1 — cross-check contra material standalone), `TestHex8VonMises3DConvergence` (1 — Newton converge en pocas iteraciones), `TestTet4VonMises3DBasic` (1 — smoke test Tet4 + VM3D).
- **Notas de traducción**:
- Convención mixta engineering/tensorial idéntica a VM2D: entrada **strain** engineering ($\gamma_{ij} = 2 \cdot \varepsilon_{ij}$), salida **sigma** con cortantes tensoriales (lo que el ensamblador espera), estado **eps_p** con cortantes tensoriales. La conversión engineering $\rightarrow$ tensorial se hace una sola vez al construir el desviador (`strain[3..5]/2`).
- La tangente algorítmica $\mathbf{C}^{\text{alg}}$ usa el patrón $K \mathbf{1} \otimes \mathbf{1} + 2G(1 - \beta) \mathbf{I}_{\text{dev}} - 2G\bar{\gamma} \mathbf{N} \otimes \mathbf{N}$ con $\mathbf{I}_{\text{dev}}$ teniendo 1/2 en los cortantes (mapea engineering input a tensorial output) y $\mathbf{N} \otimes \mathbf{N}$ con $\mathbf{N}$ tensorial. Esta combinación produce la convención correcta engineering $\rightarrow$ tensorial en la salida sin factores adicionales — verificado por el test cruzado VM3D $\leftrightarrow$ VM2D plane strain (mismos $\sigma_{xx}, \sigma_{yy}, \sigma_{xy}, \alpha$ a 10 decimales) y por el test de integración Hex8 confinado vs material standalone.
- Validación de parámetros en constructor: $E > 0, \sigma_y > 0, H \geq 0, \nu \in (-1, 0.5), \text{density} \geq 0$ si se pasa. Mensajes en español.

9.24 Diálogo

- **2026-05-21** · Spec pre-redactada por la IA al abrir la sub-etapa **A.bis** (materiales 3D no lineales) tras cierre de la Etapa 7 (ADR 0012). Base: `VonMises2D.md` (modelo físico) + `Elastic3D.md` (estructura Voigt 6D + sin variantes de hipótesis).
- **2026-05-21** · **Decisión de plumbing (decide la IA, Reglas §3)**: kernels Numba separados `_compute_j2_3d` (este material) vs `_compute_j2_plane_strain` (existente en `VonMises2D`). **No** se extrae un `_J2Core` compartido en esta entrega. Razones:
 1. La diferencia operativa entre los dos kernels es el número de componentes activas (4 vs 6), que en Numba *nopython* requiere arrays de tamaño fijo — un kernel "genérico" obligaría a zero-padding sistemático con coste y oscurecería la lectura.
 2. Cada kernel es corto (~40 líneas): la duplicación cabe en el mismo PR sin coste de mantenimiento desproporcionado.
 3. La regla de centralización post-dos-casos del proyecto se satisface ya con la documentación cruzada (esta spec + `VonMises2D` referencian el mismo algoritmo de Simó-Hughes §3.3). Si aparece un tercer caso (e.g. `VonMises3DCorot` o variante con anisotropía leve) será el momento natural de abrir `_J2Core`.

4. Plane stress 2D queda explícitamente fuera de un eventual `_J2Core` por tener un algoritmo genuinamente distinto (Newton local proyectado).

Si al implementar la duplicación resulta más alta de lo previsto, se reconsidera y se eleva a zona gris.

- **2026-05-21 · Decisión de plumbing:** el predictor elástico debe construirse como $\sigma^{\text{trial}} = \mathbf{C}_e (\varepsilon - \varepsilon_n^p)$ (o equivalente desviador + presión), **no** como $\mathbf{C}_e \varepsilon$. Lección de VM2D plane strain (bug detectado el 2026-05-14): en descargas elásticas desde estado plástico o reevaluaciones vía `compute_gauss_state(U_final)`, ignorar ε_n^p devuelve σ inconsistente. Se aplica desde el inicio en VM3D y se cubre con `test_descarga_recupera_C_e` y con el test de integración `test_pipeline_hex8_traccion_uniaxial_elastoplastica`.
- **2026-05-21 · Tests cruzados VM3D ↔ VM2D plane strain (cross_consistency):** garantía adicional de que VM3D no introduce regresión silenciosa sobre el comportamiento ya validado de VM2D. Es la red de seguridad principal para confirmar que la formulación 6D no contiene errores de signo/factor en la convención Voigt — los tests unitarios analíticos por sí solos no cubren toda la combinatoria de componentes activas.
- **2026-05-21 · Validación contra Hill 3D:** el caso del cilindro hueco bajo presión interna con plasticidad perfecta tiene **solución analítica cerrada** (Hill 1950, §V) — la presión a la que la frontera elasto-plástica radial avanza a un radio dado es expresable como función explícita de σ_y, r_i, r_o . Es el benchmark canónico para validar J2 3D y ya está disponible en 2D plane strain (`tests/validation/test_hill_cylinder_j2.py`). El test 3D será su análogo discretizado por octante con simetrías. Se redactará y validará tras tener Hex8 + VonMises3D funcionando end-to-end.
- **2026-05-21 · Implementación completa, status validated.** Kernel + clase + 12 tests unitarios + 1 test cruzado + 5 tests de integración (Hex8 uniaxial libre con Poisson, Hex8 confinado vs material standalone, Hex8 convergencia, Tet4 smoke test). **18 tests añadidos a la suite (804 → 824 verdes; suite global sin regresiones).** Todos los tests pasaron a la primera, incluyendo el test cruzado VM3D ↔ VM2D plane strain a 10 decimales — señal fuerte de que la convención Voigt 6D en el kernel y la tangente algorítmica son correctas. Validación Hill 3D (cilindro hueco bajo presión interna con plasticidad perfecta) **diferida a entrega separada** porque requiere setup de malla por octante con simetrías y carga incremental hasta plasticidad parcial; será el primer test de `tests/validation/` que ejecute VM3D contra solución analítica cerrada.

Capítulo 10

Modelos Constitutivos — Cohesivos

10.1 CohesiveDamageIsotropic

Orden de trabajo. Pre-redactada por la IA como borrador para validación del usuario (autor de la formulación, Retama 2010). La física y la formulación numérica son traducción directa del Cap. 3 de la tesis; la mecánica de implementación (Voigt, registry, signatura, tests) sigue las convenciones del proyecto. El usuario revisa, corrige donde proceda, y aprueba antes de que la IA toque código.

10.2 Especificación física

10.2.1 0. Descripción general

Material cohesivo **traction-jump** para la superficie de discontinuidad Γ_d interior a un elemento finito sólido 2D. Modelo de **daño escalar isótropo** con activación tipo Rankine (Modo-I), ablandamiento gobernado por la energía de fractura G_F , y bulk vecino que descarga elásticamente sin disipación adicional.

Distinto en dominio de los materiales continuos `Elastic2D`, `VonMises2D`, `IsotropicDamage2D`:

- **No opera sobre puntos de Gauss del bulk.** Opera sobre el salto $[[u]] \in \mathbb{R}^2$ definido sobre Γ_d y devuelve tracciones $\mathbf{t} \in \mathbb{R}^2$ definidas sobre la misma superficie.
- **Unidades:** parámetros físicos son σ_{t0} (Pa, resistencia a tracción) y G_F (N/m, energía de fractura) — no E y ν .
- **Es la primera implementación** de la familia `CohesiveMaterial` introducida por ADR 0010.

Pensado para hormigón, roca, cerámicos en régimen cuasi-frágil bajo tracción dominante. Diseñado específicamente para alimentar el elemento `CST_Embedded2D` (fase 2 del ADR 0010), pero la abstracción es reutilizable por futuros elementos de interfaz cohesiva clásica (CZM) si entran al proyecto.

10.2.2 1. Descomposición del salto

Sin descomposición elástica-plástica (modelo de daño puro, no plasticidad cohesiva). El salto total $[[u]]$ se proyecta sobre el frame local $(\mathbf{n}, \mathbf{s})$ de Γ_d :

$$[\![\mathbf{u}]\!] = [\![u_n]\!] \mathbf{n} + [\![u_s]\!] \mathbf{s}$$

donde $\mathbf{n}$ es la normal a Γ_d (fijada al activarse el elemento, perpendicular a σ_I por criterio Rankine) y $\mathbf{s}$ es la tangente.

Modo-I (fase 1): sólo $\langle [\![u_n]\!] \rangle$ (parte positiva del salto normal) gobierna la disipación. La componente tangencial $[\![u_s]\!]$ desliza sin restricción y sin disipación. Modo mixto I-II queda diferido a la fase G del ADR 0010.

10.2.3 2. Ley elástica del salto (sin daño)

$$\mathbf{t} = (1 - \omega) \mathbf{T}^{el} [\![\mathbf{u}]\!]$$

con $\mathbf{T}^{el}$ la rigidez elástica del salto:

$$\mathbf{T}^{el} = K_e (\mathbf{n} \otimes \mathbf{n})$$

es decir: rigidez K_e en dirección normal, cero en dirección tangencial (consecuencia explícita de Modo-I).

K_e es una rigidez de penalty — debe escogerse suficientemente grande para que la fase elástica de la grieta sea geoméricamente despreciable ($\kappa_0 = \sigma_{t0}/K_e$ escala del problema), pero no tan grande que perjudique el condicionamiento del sistema. Guía típica: $K_e \approx E_{bulk} / _c$ con $_c$ una longitud característica del elemento. Ver §12.

10.2.4 3. Variable equivalente y función de fluencia

$$[\![u]\!]_{eq} = \langle [\![u_n]\!] \rangle$$

donde $\langle \cdot \rangle$ son los brackets de McAuley (parte positiva). La función de fluencia (Eq. 3.13 de Retama 2010):

$$f([\![\mathbf{u}]\!], \kappa) = [\![u]\!]_{eq} - \kappa \leq 0$$

donde κ es el historial del salto equivalente máximo.

Interpretación: la grieta sólo "siente" la tracción (y por tanto sólo daña) cuando se abre ($[\![u_n]\!] > 0$). En compresión ($[\![u_n]\!] < 0$) o deslizamiento puro, $\mathbf{f} = -\kappa < 0$, no hay evolución de daño.

10.2.5 4. Variable histórica y evolución (Kuhn-Tucker)

$$\kappa_{n+1} = \max(\kappa_n, \langle [\![u_n]\!]_{n+1} \rangle), \quad \kappa_0 = \frac{\sigma_{t0}}{K_e}$$

Codifica:

- **Carga activa** ($\langle [\![u_n]\!] \rangle > \kappa_n$): κ crece; el daño puede aumentar.
- **Descarga / recarga elástica** ($\langle [\![u_n]\!] \rangle \leq \kappa_n$): κ queda congelado; daño no cambia (secante respecto al origen con rigidez reducida $(1-\omega) \cdot K_e$).

- **Compresión** ($[[u_n]] < 0$): $\langle [[u_n]] \rangle = 0 \leq \kappa_n$, κ congelado. El daño no crece. *Caveat de cierre*: la rigidez efectiva en compresión sigue siendo $(1-\omega) \cdot K_e$, no se recupera el contacto rígido — limitación documentada en §12.
- **Irreversibilidad**: κ monótono no decreciente; $\omega(\kappa)$ monótono no decreciente.

Condiciones de Kuhn-Tucker estándar (Eq. 3.15):

$$\dot{\omega} \geq 0, \quad f \leq 0, \quad \dot{\omega} f = 0$$

10.2.6 5. Ley de evolución del daño — softening lineal y exponencial

El daño $\omega(\kappa)$ se define implícitamente para que la tracción en carga monótona siga una curva prescrita $T_{\text{soft}}(\kappa)$:

$$t_n(\kappa) = (1 - \omega(\kappa)) K_e \kappa \stackrel{!}{=} T_{\text{soft}}(\kappa) \Rightarrow \omega(\kappa) = 1 - \frac{T_{\text{soft}}(\kappa)}{K_e \kappa}$$

con $\kappa \geq \kappa_0$. Para $\kappa \leq \kappa_0$, $\omega = 0$ por definición.

Softening lineal:

$$T_{\text{soft}}(\kappa) = \sigma_{t0} \left(1 - \frac{\kappa - \kappa_0}{w_c - \kappa_0} \right) = \frac{\sigma_{t0} (w_c - \kappa)}{w_c - \kappa_0}, \quad w_c = \frac{2 G_F}{\sigma_{t0}}$$

válida en $\kappa \in [\kappa_0, w_c]$. Para $\kappa \geq w_c$: $T_{\text{soft}} = 0$, $\omega = 1$ (saturada a DAMAGE_MAX). Forma cerrada de $\omega(\kappa)$:

$$\omega(\kappa) = 1 - \frac{\sigma_{t0} (w_c - \kappa)}{K_e \kappa (w_c - \kappa_0)} \quad (\kappa \in [\kappa_0, w_c])$$

Verificación en los extremos: $\omega(\kappa_0) = 1 - \sigma_{t0}/(K_e \kappa_0) = 0$ (por $\kappa_0 = \sigma_{t0}/K_e$), y $\omega(w_c) = 1$.

Softening exponencial:

$$T_{\text{soft}}(\kappa) = \sigma_{t0} \exp \left(-\frac{\sigma_{t0} (\kappa - \kappa_0)}{H} \right), \quad H = G_F - \frac{\sigma_{t0} \kappa_0}{2}$$

asintótica a cero; $\omega(\kappa) \rightarrow 1$ cuando $\kappa \rightarrow \infty$, saturada a DAMAGE_MAX en la práctica. H se deriva imponiendo que la integral total de tracción a lo largo de la curva sea G_F (consistencia energética).

La validez de la rama exponencial exige $G_F > \sigma_{t0} \cdot \kappa_0 / 2$, es decir $K_e > \sigma_{t0}^2 / (2 \cdot G_F)$ — condición geométrica trivial cuando K_e se elige como penalty.

10.2.7 6. Variables internas

- κ : float — historial del salto equivalente máximo. **PRIMARY_STATE_VAR** alternativa (más fundamental, controla todo el resto).
- ω : float $\in [0, \text{DAMAGE_MAX}]$ — daño escalar. **PRIMARY_STATE_VAR** adoptada (más interpretable físicamente; se exporta al post-proceso).
- ω es función determinista de κ (ec. §5); se cachea en el estado para evitar recálculo y para diagnóstico.

10.2.8 7. Frame local y notación

Convención del proyecto:

- $\mathbf{n}$: normal a Γ_d , unitaria. Se fija al activar el elemento (perpendicular a σ_I).
- $\mathbf{s}$: tangente, ortogonal a $\mathbf{n}$, orientada por regla de la mano derecha con $\mathbf{z}$ saliendo del plano (consistente con Reglas.md §5).
- $[[\mathbf{u}]] = [[\mathbf{u}_n]] \cdot \mathbf{n} + [[\mathbf{u}_s]] \cdot \mathbf{s}$ con $[[\mathbf{u}_n]] = [[\mathbf{u}]] \cdot \mathbf{n}$, $[[\mathbf{u}_s]] = [[\mathbf{u}]] \cdot \mathbf{s}$.
- $\mathbf{t} = t_n \cdot \mathbf{n} + t_s \cdot \mathbf{s}$ con $t_n = K_e \cdot (1 - \omega) \cdot [[\mathbf{u}_n]]$ en Modo-I, $t_s = 0$.

El material recibe $[[\mathbf{u}]]$ y devuelve $\mathbf{t}$ en **ejes locales ($\mathbf{n}$, $\mathbf{s}$) de Γ_d** . La transformación a ejes globales es responsabilidad del elemento que lo consume, **no** del material. Mismo patrón que en VonMises2D (recibe ε en Voigt, no se ocupa de transformaciones de elemento).

10.3 Formulación numérica

10.3.1 8. Algoritmo en un paso (explícito, sin Newton local)

Dado κ_n y $[[\mathbf{u}]]_{n+1}$ (en ejes locales):

1. Calcular $[[\mathbf{u}]]_{eq} = \langle [[\mathbf{u}_n]] \rangle = \max(0, [[\mathbf{u}]] \cdot \mathbf{n}_{local})$. (En el frame local, $\mathbf{n}_{local} = (1, 0)$ y $[[\mathbf{u}_n]] = [[\mathbf{u}]] [0]$. Se mantiene la notación abstracta por claridad física.)
2. Si $[[\mathbf{u}]]_{eq} > \kappa_n$ y $[[\mathbf{u}]]_{eq} > \kappa_0$: **carga activa**, $\kappa_{n+1} = [[\mathbf{u}]]_{eq}$. Si no: **descarga / no daño**, $\kappa_{n+1} = \kappa_n$.
3. Aplicar $\omega(\kappa_{n+1})$ por la fórmula §5; saturar a DAMAGE_MAX.
4. Tracción: $t_n = (1 - \omega) \cdot K_e \cdot [[\mathbf{u}_n]]$, $t_s = 0$.
5. Tangente: ver §9.

El algoritmo es **explícito**: no requiere Newton local. La actualización de κ y ω es directa. Mismo patrón que IsotropicDamage2D.

10.3.2 9. Tangente algorítmica consistente

$$\mathbf{T}^{alg} = \frac{\partial \mathbf{t}_{n+1}}{\partial [[\mathbf{u}]]_{n+1}}$$

Derivando $\mathbf{t} = (1 - \omega) \cdot K_e \cdot [[\mathbf{u}_n]] \cdot \mathbf{n}$ en el frame local con $\mathbf{n} = (1, 0)$:

- **Sin daño activo** ($\kappa \leq \kappa_0$):

$$\mathbf{T}^{alg} = K_e (\mathbf{n} \otimes \mathbf{n}) = K_e \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$$

- **Descarga** ($[[\mathbf{u}]]_{eq} \leq \kappa_n$):

$$\mathbf{T}^{\text{alg}} = (1 - \omega) K_e (\mathbf{n} \otimes \mathbf{n})$$

(secante reducida; $\partial\kappa/\partial[[u]] = 0$ anula el segundo término).

- **Carga activa** ($[[u]]_{\text{eq}} > \kappa_n$, $\kappa > \kappa_0$, $\omega < \text{DAMAGE_MAX}$):

$$\begin{aligned} \mathbf{T}^{\text{alg}} &= (1 - \omega) K_e (\mathbf{n} \otimes \mathbf{n}) - \frac{d\omega}{d\kappa} K_e [[u_n]] (\mathbf{n} \otimes \mathbf{n}) \\ &= \left[(1 - \omega) - \frac{d\omega}{d\kappa} [[u_n]] \right] K_e (\mathbf{n} \otimes \mathbf{n}) \end{aligned}$$

- **Cap numérico de la tangente** ($\omega \geq \text{DAMAGE_MAX}$, o $\kappa \geq w_c$ en lineal):

$$\mathbf{T}^{\text{alg}} = (1 - \text{DAMAGE_MAX}) K_e (\mathbf{n} \otimes \mathbf{n})$$

El cap se aplica **sólo a la rigidez tangente** para evitar singularidad del Newton cuando $\omega \rightarrow 1$. La tracción se calcula con el ω físico (que llega a 1.0 exacto en lineal con $\kappa \geq w_c$, asintóticamente en exponencial) y por tanto $t_n = 0$ cuando la grieta está totalmente abierta, sin residuo numérico contaminando la energía disipada.

Las derivadas $d\omega/d\kappa$ cerradas:

- **Lineal**, en $\kappa \in [\kappa_0, w_c]$. Reescribiendo $\omega(\kappa) = 1 - C \cdot (w_c - \kappa) / \kappa = 1 - C \cdot w_c / \kappa + C$ con $C = \sigma_{t0} / [K_e \cdot (w_c - \kappa_0)]$ constante:

$$\frac{d\omega}{d\kappa} = \frac{C w_c}{\kappa^2} = \frac{\sigma_{t0} w_c}{K_e \kappa^2 (w_c - \kappa_0)}$$

Positiva y finita en todo el rango $[\kappa_0, w_c]$. Verificación en extremos: en $\kappa = \kappa_0$ (con $\kappa_0 = \sigma_{t0} / K_e \Rightarrow K_e \cdot \kappa_0^2 = \sigma_{t0} \cdot \kappa_0$) toma el valor $w_c / [\kappa_0 \cdot (w_c - \kappa_0)]$; en $\kappa = w_c$ toma $\sigma_{t0} / [K_e \cdot w_c \cdot (w_c - \kappa_0)]$. Dimensionalmente, $[d\omega/d\kappa] = m^{-1}$ (ω adimensional sobre κ con unidad de longitud). ✓

- **Exponencial**, en $\kappa > \kappa_0$:

$$\frac{d\omega}{d\kappa} = \frac{T_{\text{soft}}(\kappa)}{K_e \kappa} \left(\frac{1}{\kappa} + \frac{\sigma_{t0}}{H} \right) = (1 - \omega(\kappa)) \left(\frac{1}{\kappa} + \frac{\sigma_{t0}}{H} \right)$$

(Análoga a la forma de *IsotropicDamage2D* con $\alpha \leftrightarrow \sigma_{t0}/H$.)

10.3.3 10. Simetría de la tangente

A diferencia de *IsotropicDamage2D* (donde el producto externo $(C_e \cdot \varepsilon) \cdot (M \cdot \varepsilon)$ rompe la simetría), aquí ambos vectores que entran al rank-1 colapsan a $\mathbf{n}$:

$$\mathbf{T}^{\text{alg}} = \alpha(\kappa, \omega, [[u_n]]) K_e (\mathbf{n} \otimes \mathbf{n})$$

con α escalar. $\mathbf{n} \cdot \mathbf{n}$ es simétrica, por tanto $\mathbf{T}^{\text{alg}}$ es **simétrica** en Modo-I.

Consecuencia: `IS_SYMMETRIC = True` para este material. El despachador algebraico (ADR 0003) puede usar Cholesky/LDL^T si el resto del dominio también es simétrico. **Modo mixto (fase G) reintroducirá asimetría** — se evaluará entonces.

10.3.4 11. Decisión de régimen durante el Newton global

Dentro de una iteración del Newton global, el material recibe `[[u]]_trial` y decide carga/descarga comparando con κ_n (estado committed del paso anterior). Mismo patrón que `IsotropicDamage2D` §9. La transición loading↔unloading es discontinua en la tangente; en problemas patológicos cerca del umbral, considerar arc-length (`ArcLengthSolver` ya disponible).

10.3.5 12. Caveats numéricos

- **Elección de K_e (penalty).** Si K_e muy pequeño: $\kappa_0 = \sigma_{t0}/K_e$ no es despreciable y la respuesta elástica deforma significativamente la curva (la grieta "se abre" antes del pico). Si K_e muy grande: condicionamiento del sistema empeora ($K_e = E_{bulk} \cdot L_e$ introduce diferencia de magnitudes en $K_{\{u\}}$). Regla práctica: $K_e \approx 10 \cdot E_{bulk} / l_c$ con l_c la longitud característica del elemento (su lado más corto). El usuario lo declara explícitamente en YAML; no hay default automático.
- **Cierre en compresión.** Cuando `[[u_n]] < 0` el modelo devuelve $t_n = (1-\omega) \cdot K_e \cdot [[u_n]]$ (compresión reducida). Físicamente, la grieta cerrada debería transmitir compresión a rigidez completa K_e (contacto unilateral). En fase 1 esta limitación se acepta tal cual — los benchmarks de la tesis (Van Vliet, viga SEN) son traccionados puros. Un modelo con corrección de cierre se contempla como **deuda explícita** para casos con cambio de signo en la grieta.
- **Cap numérico de la rigidez tangente.** `DAMAGE_MAX < 1` se aplica **sólo** sobre `T_tan` para evitar singularidad del Newton (`stiffness = max(1- $\omega$ , 1-DAMAGE_MAX) ·  $K_e$`). El ω reportado en el estado y usado en $t_n = (1-\omega) \cdot K_e \cdot [[u_n]]$ es físico (puede valer exactamente 1.0 en lineal con $\kappa \geq w_c$). Diferencia respecto al patrón en `IsotropicDamage2D`: allí $K_e = E$ es moderado y truncar ω no introduce residuo significativo en σ ; aquí K_e es penalty (E) y u_n recorre rangos macroscópicos, así que truncar ω contaminaría la energía disipada en ~10–30 %.
- **Sensibilidad de malla** en régimen de softening. Sin regularización, el patrón de fallo localiza en función del tamaño y orientación de elementos. La integración con el elemento `CST_Embedded2D` (fase 2 del ADR 0010) y con el $l_d = (A/h) \cdot \cos(\theta-\alpha)$ de Cap. 6 da objetividad parcial respecto a la longitud de la grieta — pero la dirección de propagación sigue dependiendo de la dirección de tensiones principales en el elemento. Mitigación a nivel del *elemento*, no del material.
- **Validación energética.** Por construcción $\int_0^{w_c} t(w) \cdot dw = G_F$. Esto se verifica en los tests (§acceptance). Si la curva implementada no integra a G_F , hay error en w_c o en H .

10.4 Contrato de implementación

```
name: CohesiveDamageIsotropic
kind: cohesive_material          # nueva familia, ADR 0010
status: validated

interface:
  jump_dim: 2                   # [[u]]2 en 2D (componentes n, s en frame local)
  primary_state_var: damage     # , escalar exportable al post-proceso
  is_symmetric: true            # tangente simétrica en Modo-I (ver §10)

parameters:
- { name: sigma_t0,             type: float, required: true, desc: "Resistencia a
  tracción [Pa]; iniciación del daño" }
```

```

- { name: G_f,                type: float, required: true, desc: "Energía de
  fractura [N/m]; área bajo la curva -t[[u_n]]" }
- { name: K_e,                type: float, required: true, desc: "Rigidez elástica
  del salto en dirección normal [Pa/m] (penalty); ver §12" }
- { name: softening,          type: str,   required: true, desc: "Forma de la curva
  de softening: 'linear' | 'exponential'" }

signature:
  compute_traction: "([[u]]: ndarray(2,), state_vars=None) -> (t: ndarray(2,), T_tan:
    ndarray(2,2), state_vars)"
  jump_kind: "frame local - [[u]] = ([[u_n]], [[u_s]]) en ejes (n, s) de  $\Gamma_d$ "

state_schema:
  kappa: "float    _0 - historial del salto equivalente máximo"
  damage: "float    [0, DAMAGE_MAX] - variable de daño escalar"

conventions:
  sign: "[[u_n]] > 0  apertura (tracción); el modelo daña sólo en apertura (Modo-I)"
  frame: "ejes locales (n, s) de  $\Gamma_d$ ; transformación a globales por el elemento
    consumidor"
  damage_irreversibility: " monótono no decreciente vía max(_old, [[u_n]]); () monó
    tono creciente"
  units: "[_t0] = Pa, [G_F] = N/m, [K_e] = Pa/m (asumiendo SI; el sistema no convierte
    unidades)"

validity:
- "_t0 > 0, G_F > 0, K_e > 0"
- "K_e > _t02 / (2·G_F) - consistencia energética para softening exponencial (§5)"
- "softening {'linear', 'exponential'}"

out_of_scope:
- "Modo mixto I-II (cizalla acoplada al daño); diferido a fase G del ADR 0010"
- "Contacto / cierre unilateral en compresión; el modelo reduce rigidez al estar dañ
  ado, no recupera contacto rígido"
- "Anisotropía del daño (tensor de daño D vs escalar )"
- "Acoplamiento viscoso (rate-dependent); rate-independent puro en fase 1"
- "Fatiga / acumulación cíclica del daño"
- "Regularización para mesh-objectivity en propagación; se aborda a nivel de elemento
  (l_d correcto, Cap. 6) y no de material"

numerical_caveats:
- "Penalty K_e: balance entre _0 despreciable (K_e grande) y condicionamiento (K_e
  moderado). Guía: K_e 10·E_bulk/_c. Declarado por el usuario en YAML, sin
  default automático."
- "Cierre en compresión: rigidez reducida -(1)·K_e, no recuperación de contacto rí
  gido. Aceptable para benchmarks puramente traccionados; deuda explícita para
  casos con cambio de signo."
- "Cap por DAMAGE_MAX aplicado sólo a la rigidez tangente (stiffness = max-(1, -1
  DAMAGE_MAX)·K_e) para evitar singularidad del Newton. La tracción se calcula con
  físico (1.0 exacto en lineal con w_c) y por tanto t_n = 0 cuando la grieta está
  totalmente abierta, sin sesgo en la energía disipada."
- "Régimen elasticsoftening discontinuo en la tangente al cruzar _0; régimen
  loadingunloading discontinuo al cruzar _n committed. Mismo patrón que
  IsotropicDamage2D."
- "En la zona DAMAGE_MAX la tangente colapsa a la secante reducida -(1DAMAGE_MAX)·
  K_e, no a la consistente; el Newton degrada de convergencia cuadrática a lineal
  en esa rama (típicamente irrelevante porque la rama corresponde a un elemento prá
  cticamente roto)."

```

```

acceptance:
  verification:
    - name: respuesta_elastica_bajo_umbral
      setup: "carga monótona en [[u_n]] [0, _0]; [[u_s]] = 0"
      expect: " = 0; t_n = K_e · [[u_n]] exacto; t_s = 0; T_tan = K_e · (nn)"
      tol_rel: 1.0e-12

    - name: inicio_dano_en_umbral
      setup: "[[u_n]] cruza _0 = _t0/K_e por encima por primera vez"
      expect: " salta de 0 a un valor positivo continuo en _0 ((_0) = 0+); t_n(_0) =
        _t0 exacto"
      tol_rel: 1.0e-10

    - name: energia_fractura_softening_lineal
      setup: "carga monótona [[u_n]] de 0 a w_c = 2 · G_F / _t0 con softening='linear',
        K_e tal que _0 w_c"
      expect: " _0^{w_c} t_n([[u_n]]) · d[[u_n]] = G_F (cuadratura trapezoidal con 1000
        puntos)"
      tol_rel: 1.0e-3
      note: "La tolerancia 1e-3 absorbe el error de cuadratura; la consistencia analí
        tica es exacta por construcción de w_c."

    - name: energia_fractura_softening_exponencial
      setup: "carga monótona [[u_n]] de 0 a 20 · G_F / _t0 con softening='exponential',
        K_e tal que _0 G_F / _t0"
      expect: " t_n · d[[u_n]] G_F · (1 - _truncamiento), con _truncamiento < 1e-6 por
        la cola exponencial cortada"
      tol_rel: 1.0e-3

    - name: descarga_secante
      setup: "carga hasta (_0, w_c/2), después descarga [[u_n]] → 0"
      expect: "rama de descarga lineal con pendiente - (1()) · K_e; al volver a [[u_n]]
        ]]=0, t_n = 0 y no ha cambiado"
      tol_rel: 1.0e-10

    - name: recarga_hasta_mismo_kappa_sin_dano_extra
      setup: "carga → descarga → recarga hasta [[u_n]] = (mismo histórico)"
      expect: " no cambia (_{n+1} = max(_n, [[u_n]]) = _n)"
      tol_rel: 1.0e-12

    - name: recarga_mas_alla_continua_danando
      setup: "carga → descarga → recarga hasta [[u_n]] > histórico"
      expect: " evoluciona desde el valor previo hacia ([[u_n]]_nuevo) sin saltos"
      tol_rel: 1.0e-10

  specific:
    - name: tangente_simetrica_en_carga_y_descarga
      setup: "estados de carga activa, descarga, sin daño, saturado"
      expect: "T_tan = T_tan^T (simetría exacta en Modo-I); ||T_tan - T_tan^T||_F /
        ||T_tan||_F < 1e-14"
      tol_rel: 1.0e-14

    - name: tangente_diferencia_finita_consistente
      setup: "carga activa con [[u_n]] (_0, w_c/2); perturbar [[u]] con = 1e-7"
      expect: "||T_tan_FD - T_tan_analitica||_F / ||T_tan||_F < 1e-5"
      tol_rel: 1.0e-5

    - name: tangencial_no_dana_ni_transmite
      setup: "[[u_s]] arbitrario, [[u_n]] = 0"

```

```

expect: "t_s = 0; t_n = 0;  no cambia"
tol_rel: 1.0e-14

- name: compresion_no_dana
  setup: "[[u_n]] < 0 puro (penetración)"
  expect: "[[u_n]] = 0  f = -  0    no cambia. t_n = -(1_previo)·K_e·[[u_n]] (
    caveat de cierre, §12)"
  tol_rel: 1.0e-12

- name: saturacion_en_w_c_softening_lineal
  setup: "[[u_n]] = w_c·1.01 con softening='linear'"
  expect: " = 1.0 exacto, t_n = 0 exacto (grieta totalmente abierta). T_tan = -(1
    DAMAGE_MAX)·K_e (cap numérico sólo sobre la rigidez)."
  tol_rel: 1.0e-10

- name: degeneracion_a_elasticidad_intacta
  setup: "_t0 → (10~20 Pa) o cualquier [[u_n]] tan pequeño que jamás active"
  expect: " = 0, t_n = K_e·[[u_n]], T_tan = K_e·(nn) (material intacto puro)"
  tol_rel: 1.0e-12

arch:
- name: registry_kind_es_cohesive
  expect: "Registro vía CohesiveMaterialRegistry, no MaterialRegistry; kind='
    cohesive_material' en YAML"

- name: is_symmetric_attribute
  expect: "CohesiveDamageIsotropic.IS_SYMMETRIC == True"

references:
- "Retama Velasco, J. (2010). Formulation and Approximation to Problems in Solids by
  Embedded Discontinuity Models. Tesis Doctoral, UNAM. **Cap. 3 - Discrete damage
  models**. Fórmulas §3.1 (energía libre, ec. -3.13.4); §3.1.1 (tensor tangente, ec
  . -3.73.12); §3.1.2 (función de fluencia, ec. -3.133.14); §3.1.3 (Kuhn-Tucker, ec
  . -3.153.16)."
```

```

- "Hillerborg, A., Modéer, M., Petersson, P.-E. (1976). Analysis of crack formation
  and crack growth in concrete by means of fracture mechanics and finite elements.
  *Cement and Concrete Research* 6, 773-782. **Origen del modelo cohesivo con energ
  ía de fractura G_F**."
```

```

- "Barenblatt, G.I. (1962). The mathematical theory of equilibrium cracks in brittle
  fracture. *Advances in Applied Mechanics* 7, 55-129."
```

```

- "Simó, J.C., Ju, J.W. (1987). Strain- and stress-based continuum damage models - I.
  Formulation. *Int. J. Solids Struct.* 23, 821-840."
```

```

- "ADR 0010 - Discontinuidades interiores embebidas (familia CohesiveMaterial, hoja
  de ruta)."
```

```

- "Spec [IsotropicDamage2D](IsotropicDamage2D.md) - patrón análogo de daño isótopo
  escalar para el caso continuo."
```

10.5 Implementación

- Archivo: solidum/cohesive_materials/damage_isotropic.py
- Clase: CohesiveDamageIsotropic (registrada en CohesiveMaterialRegistry)
- Base abstracta: solidum/core/cohesive_material.py (CohesiveMaterial)
- Registry: solidum/registry.py (CohesiveMaterialRegistry, paralelo a MaterialRegistry)

- Autodiscover: `solidum.autodiscover.initialize()` descubre `solidum.cohesive_materials` automáticamente.
- Tests:
- `tests/test_cohesive_damage_isotropic.py` — todos los casos de acceptance (verification + specific + arch).
- Notas de traducción:
- El algoritmo §8 es explícito (sin Newton local); no se usa `is_admissible` en este material.
- `JUMP_DIM = 2, PRIMARY_STATE_VAR = 'damage', IS_SYMMETRIC = True`.
- Frame local interno: $\mathbf{n} = (1, 0)$, $\mathbf{s} = (0, 1)$; el material no transforma a globales (responsabilidad del elemento consumidor).
- `t_s` y `T_tan[1,1]` son 0 por construcción (Modo-I). El elemento que consuma el material debe estar preparado para rigidez tangencial nula.
- `K_e` obligatorio en el constructor (sin default). `kappa_0`, `w_c` (lineal) y `H` (exponencial) se derivan en `__init__`.

10.6 Diálogo

- **2026-05-13** · Spec pre-redactada por la IA como **orden de trabajo** sobre el Cap. 3 de Retama 2010 (tesis del usuario). La física es traducción directa de la tesis; la mecánica del proyecto (Voigt, registry, signatura, tests) sigue convenciones de `IsotropicDamage2D`. Pendiente de validación del usuario.
- **2026-05-18** · Usuario delega las tres decisiones a la IA. Cierre con justificación arquitectural:
- **PRIMARY_STATE_VAR = damage** (ω) — adoptado. Razones: (a) coherencia con `IsotropicDamage2D` (mismo criterio en toda la familia de daños), (b) $\omega \in [0, \text{DAMAGE_MAX}]$ es adimensional e interpretable físicamente y es lo que se colorea en el post-proceso VTK, (c) κ tiene unidades [m] y depende de `K_e` (no es portable entre materiales). κ se almacena como variable interna secundaria, controla todo determinísticamente vía $\omega(\kappa)$ cerrada.
- **$d\omega/d\kappa$ lineal cerrada en §9** — hecho. Forma simplificada: $d\omega/d\kappa = \sigma_{t0} \cdot w_c / [K_e \cdot \kappa^2 \cdot (w_c - \kappa_0)]$. Documentada en la spec para que el docstring del código pueda referenciarla en vez de derivarla.
- **`K_e` obligatorio en YAML, sin default automático** — adoptado. ADR 0010 §1 separa explícitamente `bulk_material` y `cohesive_material`; derivar `K_e` de `E_bulk`·`_c` los acoplaría a nivel de material y ensuciaría el contrato (el material recibiría `E_bulk` y `_c` como inputs ocultos). Si en una iteración futura el YAML pide derivado automático, la fábrica vivirá en el elemento (`CST_Embedded2D` construye el `CohesiveDamageIsotropic` con `K_e` derivado si se omite), no en el material.
- **2026-05-18** · Status $\rightarrow$ validated. La IA arranca implementación.

- **2026-05-18** · *Nota tras implementación:* durante los tests el corte por `DAMAGE_MAX` aplicado a ω introducía ~10% de exceso en $\int \mathbf{t} \cdot \mathbf{d}[[\mathbf{u}_n]]$ respecto a `G_F` (vs ~0.001 esperado). Razón física: en cohesivos `K_e` es penalty (`E_bulk`) y `u_n` recorre rango macroscópico, así que $(1 - \text{DAMAGE_MAX}) \cdot K_e \cdot u_n$ no es despreciable. Fix arquitectural: el cap por `DAMAGE_MAX` se aplica **sólo a la rigidez tangente** (para mantener el Newton no singular), no al ω reportado ni a `t_n`. Actualizadas §9, §12 y `acceptance.saturation_en_w_c.softening_lineal` consecuentemente. Distinto del patrón en `IsotropicDamage2D` por la diferencia en escalas de `K_e` y de la variable de entrada.

Capítulo 11

Esquemas de Solución — Estáticos

11.1 ArcLengthSolver

*Spec **retroactiva**: el solver existe desde la fase de daño/softening y está validado por tests de snap-through, snap-back y daño post-pico. Esta spec lo documenta sin cambiar comportamiento — H-5.3 de la auditoría 2026-05-18.*

11.2 Especificación física

11.2.1 0. Descripción general

Análisis estático **no lineal** que traza curvas de equilibrio $\mathbf{U}(\lambda)$ atravesando puntos límite (snap-through, snap-back) donde $d\lambda/dU$ cambia de signo. A diferencia de **NonlinearSolver** (control de carga, λ creciente impuesto), el arc-length **trata** λ **como incógnita adicional** y restringe el avance combinado en (U, λ) a un arco fijo dl en el espacio aumentado. Esto permite seguir la respuesta postcrítica de estructuras con softening, pandeo geométrico, y daño con rama descendente.

Algoritmo: **Crisfield cilíndrico** (1981) — restricción cuadrática $\|\Delta\mathbf{U}\|^2 = dl^2$, λ libre.

11.2.2 1. Ecuación de equilibrio resuelta

Forma fuerte:

$$\mathbf{F}_{\text{int}}(\mathbf{U}) = \lambda \mathbf{F}_{\text{ext}}^{\text{ref}}$$

con λ incógnita, sujeta a la restricción cilíndrica de Crisfield en cada paso:

$$\|\Delta\mathbf{U}^{\text{paso}}\|^2 = dl^2$$

(versión **cilíndrica**: no incluye $\Delta\lambda^2$, equivalente a tomar $\psi = 0$ en el parámetro de escala carga-desplazamiento; la versión esférica $\|\Delta\mathbf{U}\|^2 + \psi^2 \Delta\lambda^2 \|\mathbf{F}_{\text{ext}}^{\text{ref}}\|^2 = dl^2$ no está implementada).

11.2.3 2. Condiciones de contorno

Idénticas a **NonlinearSolver**: Dirichlet (homogéneo / no homogéneo) y MPC vía ADR 0004. Las condiciones Dirichlet no homogéneas se aplican proporcionalmente a λ .

11.2.4 3. Salidas físicas

- $\mathbf{U}$ final en $\lambda \rightarrow \lambda_{\text{máx}}$ (o cuando se alcance `max_steps`).
- Historia disponible vía `step_callback(step, U, lambda)`.
- Trazas $\lambda(U_i)$ para visualizar snap-through/snap-back.

11.3 Formulación numérica

11.3.1 4. Esquema operativo

```

= 0
dl = initial_dl
mientras < max_lambda y step < max_steps:
    # Predictor tangente
    K_t, F_int = assemble_non_linear_system(U_iter)
    u_t = K_t^{-1} · F_ext^ref          # desplazamiento tangente unitario
    sign = sign(U_prev · u_t)          # evitar revertir direcciónΔ
    _pred = sign · dl / ||u_tΔ
    U_pred = Δ_pred · u_t

    # Corrector iterativo
    para iter en range(max_iter):
        K_t, F_int = assemble_non_linear_system(U_iter)
        R = ( + Δ) · F_ext^ref - F_int
        u_R = K_t^{-1} · R              # corrección por residuo
        u_t = K_t^{-1} · F_ext^ref      # tangente (refresh)

        # Restricción cuadrática de Crisfield
        # ||ΔU_new + dd · ||u_t^2 = dl^2
        # resolver ecuación cuadrática en dd, elegir raíz por menor ángulo
        dd = raiz_de_menor_angulo(...)Δ
        U += u_R + dd · u_tΔ
        += dd
        U_iter = U_current + ΔU
        si converge (ADR 0007):
            commit_all_states()
            += Δ
            ajustar_dl_adaptativamente(iter)
            break
        si no converge:
            dl /= dl_shrink_factor

```

11.3.2 5. Predictor / corrector

Predictor tangente: $\delta \mathbf{u}_t = \mathbf{K}_t^{-1} \mathbf{F}_{\text{ext}}^{\text{ref}}$ (vector que mide la pendiente local del camino de equilibrio). $\Delta \lambda_{\text{pred}} = \pm dl / \|\delta \mathbf{u}_t\|$ con signo determinado por el producto escalar contra el incremento previo $\Delta \mathbf{U}_{\text{paso prev}}$ — así no se retrocede en el camino tras pasar un punto límite.

Corrector (Newton modificado para arc-length, Crisfield): cada iteración resuelve dos sistemas (residuo y tangente), combina con $dd\lambda$ que satisface la restricción cuadrática:

$$\|\Delta \mathbf{U} + dd\lambda \delta \mathbf{u}_t\|^2 = dl^2$$

Ecuación cuadrática en $dd\lambda$ con dos raíces. **Selección de raíz:** la que produce **menor ángulo** con el incremento acumulado $\Delta\mathbf{U}$ (evita pivotar el sentido del arco).

11.3.3 6. Criterio de convergencia (ADR 0007)

Idéntico a `NonlinearSolver` — patrón dual fuerza + desplazamiento con escala autoderivada. Compartido vía `ConvergenceCriterion`. La calibración del solver usa $\|\mathbf{F}_{\text{ext}}^{\text{ref}}\|$ como referencia de fuerza (no $\lambda \mathbf{F}_{\text{ext}}^{\text{ref}}$, porque λ varía).

11.3.4 7. Imposición de Dirichlet y MPC

Idéntico al `NonlinearSolver` — `Assembler.reduce(...)` con `U_current` y `load_factor=lambda_iter`.

11.3.5 8. Backend algebraico

Régimen **postcrítico**: $\mathbf{K}_t$ puede ser indefinida (autovalores negativos tras bifurcación). El solver fuerza `is_positive_definite=False` al construir el `linalg`, así el despachador ADR 0003 elige LU automáticamente. Si el usuario fuerza `linear_algebra=cholesky` desde YAML y $\mathbf{K}_t$ se vuelve indefinida en algún paso, hay fallback automático a LU con warning.

11.3.6 9. Adaptatividad — ajuste de dl

Política de auto-ajuste basada en iteraciones del paso:

- Converge en $< dl_grow_iter_threshold$ iteraciones $\Rightarrow dl \rightarrow dl \cdot dl_grow_factor$ (cota: $initial_dl \cdot dl_max_factor$).
- Converge en $> dl_shrink_iter_threshold$ iteraciones $\Rightarrow dl \rightarrow dl \cdot dl_shrink_factor$.
- No converge en `max_iter` $\Rightarrow dl / = 2$ y reintentar el paso.
- Cota inferior: $dl < ARCLength_min_dl_factor \cdot initial_dl \Rightarrow$ aborto.

11.3.7 10. Caveats numéricos

- **Cancelación o bifurcación verdadera:** en bifurcaciones simétricas múltiples, la selección de raíz por menor ángulo puede saltar a un camino paralelo. Para análisis de bifurcación auténtica usar perturbación + post-procesado modal.
- **Raíces imaginarias:** si el discriminante de la cuadrática es negativo (paso demasiado grande, mal condicionamiento severo), el solver aborta el paso y bisecta dl .
- **Último paso:** cuando $\lambda + \Delta\lambda_{\text{pred}} \geq \lambda_{\text{máx}}$, el solver fija $\Delta\lambda = \lambda_{\text{máx}} - \lambda$ y corrige solo en desplazamientos (Newton puro), sin restricción cuadrática. Permite cerrar exactamente en el target.
- **Softening con penalty cohesivo stiff** (`CST_Embedded2D`): no se atraviesa la transición elástico→softening por la quasi-singularidad de $\mathbf{K}_t$ cerca de κ_0 . Limitación específica al `embedded`.
- **Diagnóstico de bifurcación por inercia** (Sturm sequence): placeholder en `_negative_pivots()` — retorna `None` hasta que se implemente un backend $\text{LDL}^\top$ verdadero (Bunch-Kaufman) con conteo de pivots negativos.

11.4 Contrato de implementación

```

name: ArcLengthSolver
kind: solver
status: validated

interface:
  yaml_type: arclength
  output: SolveResult
  pipeline_kind: static

parameters:
  - { name: convergence,          type: ConvergenceCriterion, required: false,
      default: "ConvergenceCriterion()",
      desc: "Política dual fuerza+desplazamiento (ADR 0007), compartida con
            NonlinearSolver" }
  - { name: max_iter,            type: int,      required: false, default: 20,
      desc: "Iteraciones Newton (corrector) máximas por paso" }
  - { name: max_lambda,         type: float,    required: false, default: 1.0,
      desc: "Factor de carga objetivo" }
  - { name: initial_dl,         type: float,    required: false, default: 0.1,
      desc: "Longitud de arco inicial" }
  - { name: max_steps,          type: int,      required: false, default: 100,
      desc: "Cota dura sobre el número de pasos" }
  - { name: dl_grow_factor,      type: float,    required: false, default: 1.5 }
  - { name: dl_max_factor,      type: float,    required: false, default: 5.0,
      desc: "dl máximo = initial_dl · dl_max_factor" }
  - { name: dl_shrink_factor,    type: float,    required: false, default: 0.6 }
  - { name: dl_grow_iter_threshold, type: int,    required: false, default: 4 }
  - { name: dl_shrink_iter_threshold, type: int,    required: false, default: 8 }
  - { name: linear_algebra,      type: str,      required: false, default: "auto",
      desc: "'auto' (LU para postcrítico), 'cholesky', 'lu'. En arc-length el default
            fuerza LU por indefinición potencial" }

requirements:
  - "Modelo con potencial snap-through/snap-back (geometría inestable, softening del
    material)"
  - "Apoyos suficientes; sin modos rígidos"
  - "dl inicial razonable (orden de magnitud del desplazamiento característico del
    primer paso)"

conventions:
  units: "heredadas del modelo (ADR 0008)"
  stability: "incondicional respecto a puntos límite; bifurcaciones complejas pueden
    saltar de rama"

out_of_scope:
  - "Variante esférica del arc-length ( 0) - sólo cilíndrica implementada"
  - "Análisis dinámico usar NewmarkSolver y derivados"
  - "Continuación de bifurcaciones múltiples - requiere perturbación explícita por el
    usuario"
  - "Buckling lineal (eigenproblema) futuro BucklingSolver"
  - "Sign-of-pivot tracking (Sturm sequence) - placeholder hasta LDL verdadero"

acceptance:
  verification:
    - name: barra_traccion_elastica_reproduce_caso_lineal
      setup: "Truss2D elástica, carga axial; arc-length avanza hasta =1"

```

```

expect: "U(=1) = U_lineal exacto; equilibrio en cada paso"
tol_rel: 1.0e-10

- name: snap_through_lee_frame
  setup: "frame Lee con dos barras corotacionales en V, carga vertical en el vértice"
  expect: "se atraviesa el punto límite; respuesta U - coincide con Crisfield 1991 Vol.1 Fig. 9.5"
  tol_rel: 1.0e-3

- name: snap_back_truss_inestable
  setup: "armadura con elemento corotacional cuya rama descendente requiere  $d/dU < 0$ "
  expect: "se atraviesa el snap-back; signo de  $\Delta$  cambia coherentemente"
  tol_rel: 1.0e-3

- name: damage_softening_post_pico
  setup: "Quad4 con IsotropicDamage2D, tracción monotónica que entra a rama descendente"
  expect: "rama post-pico trazada; convergencia paso a paso"
  tol_rel: 1.0e-3

- name: ajuste_dl_adaptativo
  setup: "trayectoria con tramos elásticos rápidos y zona post-pico lenta"
  expect: "dl crece en tramos fáciles, se reduce cerca del pico; converge globalmente"

- name: ultimo_paso_cierra_en_target
  setup: "max_lambda = 0.7 sobre estructura elástica"
  expect: "_final = 0.7 exacto (no por overshoot)"
  tol_abs: 1.0e-12

specific:
- name: seleccion_raiz_no_revierte
  setup: "paso tras un snap-back con dos raíces de signos opuestos"
  expect: "se elige la raíz que continúa el camino (menor ángulo con incremento previo)"

- name: fallback_cholesky_lu_indefinitud
  setup: "linear_algebra='cholesky' forzado, modelo con punto límite"
  expect: "warning + LU al detectar no-PD; análisis continúa"

references:
- "Crisfield M.A. (1981). A fast incremental/iterative solution procedure that handles snap-through. Computers and Structures 13, 55-62."
- "Crisfield M.A. (1991). Non-linear Finite Element Analysis of Solids and Structures, Vol. 1. Wiley. §9 (arc-length cilíndrico)."
- "Riks E. (1979). An incremental approach to the solution of snapping and buckling problems. Int. J. Solids and Structures 15, 529-551."
- "de Borst R., Sluys L.J. (1999). Computational Methods in Non-linear Solid Mechanics. TU Delft. §3.5."
- "ADR 0003, ADR 0007."

```

11.5 Implementación

- Archivo: solidum/math/solvers/arclength.py.

- **Clase:** `ArcLengthSolver`, registrada vía `@SolverRegistry.register` con `PIPELINE_KIND = "static"`.
- **Restricción cuadrática y selección de raíz:** implementada en `solve`. Comparación de ángulos $\theta_1 = \Delta U_{\text{iter}} \cdot (\Delta U_{\text{new}} + \text{ddl1} \cdot \delta u_t)$, idem θ_2 , seleccionar el mayor $\rightarrow$ menor ángulo con la dirección previa.
- **`_make_linalg`:** fuerza `is_positive_definite=False` independientemente de la simetría del dominio (régimen postcrítico).
- **Entrypoint público:** `solidum.run_static(model, solver='arclength', ...)`.
- **Tests:**
 - `tests/test_arclength.py` · snap-through Lee frame, snap-back truss, recuperación lineal.
 - `tests/test_solver_robustness.py` · `test_arc_length_traverses_damage_softening`.

11.6 Diálogo

- **2026-05-19** · Spec creada retroactivamente para cerrar el hueco H-5.3. Solver anterior a la convención de specs. La spec recoge la implementación tal como está al cierre de la sesión de saneamiento post-auditoría.

11.7 DissipationArcLengthSolver

*Spec corta tipo extensión (Reglas.md §4). Variante de ArcLengthSolver que solo cambia la restricción del paso: pasa de cilíndrica ($\|\Delta\mathbf{U}\|^2 = dl^2$) a basada en **disipación de energía** ($g(\Delta\mathbf{U}, \Delta\lambda) = \tau$). Reusa todo el plumbing del padre (predictor tangente, corrector Newton, adaptatividad de paso, Dirichlet/MPC, backend algebraico). Sin ADR nuevo: reusa el contrato arquitectural ya aceptado del arc-length.*

*>Motivación: el ArcLengthSolver cilíndrico no atraviesa la transición elástico→softening con penalty stiff (K_e del embedded discontinuity, ADR 0010 fase 4). La restricción cuadrática queda casi degenerada cerca del pico — el discriminante de la cuadrática tiende a cero, el solver bisecta indefinidamente y aborta. La restricción de disipación es **lineal en** $(\Delta\mathbf{U}, \Delta\lambda)$ y se activa **proporcionalmente a la energía consumida** por la grieta cohesiva, navegando la rama post-pico de forma estable. Esta variante desbloquea el benchmark de fractura faithful Van Vliet (*docs/specs/CST_Embedded2D.md* §.^{out of scope}), el balance energético end-to-end de G_F (hueco abierto en `[[project_validacion_fase_b]]`), y validación cuantitativa de cohesivos en pipeline real.*

11.8 Especificación física

11.8.1 0. Descripción general

Solver estático no lineal con **traza por disipación**: la magnitud que se mantiene constante de paso a paso ya no es la longitud euclídea $\|\Delta\mathbf{U}\|$ sino la **disipación incremental de energía** ΔE_d asociada al paso. Adecuado para:

- Fractura cohesiva (Mode-I y, en el futuro, mixto) con penalty K_e stiff.
- Daño con softening severo donde la rama post-pico tiene pendiente casi vertical en el plano (U, λ) .
- Localización de deformación plástica con bandas estrechas.

Régimen elástico (sin disipación): la restricción degenera a $g = 0$. El solver detecta este caso y **conmuta automáticamente** a arc-length cilíndrico durante el régimen elástico, retornando a control por disipación cuando se detecta inicio de disipación ($\Delta\omega > 0$ en daño, $\Delta\alpha > 0$ en plasticidad, $\Delta\kappa > \kappa_0$ en cohesivo).

11.8.2 1. Ecuación de equilibrio resuelta

Idéntica al padre:

$$\mathbf{F}_{\text{int}}(\mathbf{U}) = \lambda \mathbf{F}_{\text{ext}}^{\text{ref}}$$

con λ incógnita. La **restricción del paso** cambia respecto al padre.

Restricción de disipación (Gutiérrez 2004, Verhoosel et al. 2009)

La energía disipada por paso, en problemas conservativos en el componente elástico, es:

$$\Delta E_d = \frac{1}{2} \left(\lambda_n \mathbf{F}_{\text{ext}}^{\text{ref}T} \Delta\mathbf{U} - \Delta\lambda \mathbf{F}_{\text{ext}}^{\text{ref}T} \mathbf{U}_n \right)$$

donde $(\mathbf{U}_n, \lambda_n)$ son los valores committed al inicio del paso y $(\Delta\mathbf{U}, \Delta\lambda)$ los incrementos del paso. La restricción es:

$$g(\Delta\mathbf{U}, \Delta\lambda) \equiv \frac{1}{2} \left(\lambda_n \mathbf{F}_{\text{ext}}^{\text{ref}T} \Delta\mathbf{U} - \mathbf{U}_n^T \mathbf{F}_{\text{ext}}^{\text{ref}} \Delta\lambda \right) = \tau$$

con τ el **incremento de disipación objetivo** (parámetro del solver, papel análogo al `initial_dl` del padre).

Linealidad: g es lineal en $(\Delta\mathbf{U}, \Delta\lambda)$. Por eso el corrector tiene **una sola raíz**, no dos como en la cuadrática cilíndrica — eliminando la selección por menor ángulo y la patología de raíces imaginarias en pasos grandes.

11.8.3 2. Condiciones de contorno

Idénticas al padre (`ArcLengthSolver` §2). Dirichlet homogéneo/no homogéneo y MPC vía ADR 0004.

11.8.4 3. Salidas físicas

Idénticas al padre — `SolveResult` con $\mathbf{U}$ final y `step_callback(step, U, λ)` para historia. Adicionalmente, se expone vía el callback el valor de disipación acumulada $\sum_n \Delta E_d^{(n)}$ — útil para validar G_F **end-to-end** integrando $\int_{\Gamma_d} t \cdot d[u] \approx \sum_n \Delta E_d^{(n)}$.

11.9 Formulación numérica — solo lo que cambia respecto al padre

11.9.1 4. Esquema operativo

Estructura idéntica al padre con tres diferencias:

```
= 0
= initial_tau          # análogo a initial_dl, pero en unidades de energía
modo = "cylindrical"   # arranque en cilíndrico mientras g > 0
mientras < max_lambda y step < max_steps:
    # Predictor tangente (idéntico al padre)
    u_t = K_t^{-1} · F_ext^ref
    sign = sign(U_prev · u_t)
    si modo == "cylindrical": Δ
        _pred = sign · dl / ||u_t
    sino:
        # modo == "dissipation"
        # Predictor por disipación: Δ_pred derivado de Δ
        _pred = sign · / (½ · |_n · (F_ext^ref · u_t) - F_ext^ref · U_n|) Δ
    U_pred = Δ_pred · u_t

    # Corrector iterativo
    para iter en range(max_iter):
        K_t, F_int = assemble(...)
        R = (+ Δ) · F_ext^ref - F_int
        u_R = K_t^{-1} · R
        u_t = K_t^{-1} · F_ext^ref

        si modo == "cylindrical":
            # Restricción cuadrática, idéntica al padre (ArcLengthSolver §5)
            dd = raiz_de_menor_angulo(...)
        sino:
            # modo == "dissipation"
```

```

    # Restricción lineal de Gutiérrez:  $g\Delta(U_{\text{new}}, \Delta_{\text{new}}) =$ 
    #  $\Delta U_{\text{new}} = \Delta U + u_R + dd \cdot u_t$ 
    #  $g\Delta(U_{\text{new}}, \Delta + dd) =$  dd explícito:
    num = - ½ · ( $u_n \cdot F_{\text{ext}}^{\text{ref}} \cdot \Delta(U + u_R) - \Delta(U) \cdot F_{\text{ext}}^{\text{ref}} \cdot u_n$ )
    den = ½ · ( $u_n \cdot F_{\text{ext}}^{\text{ref}} \cdot u_t - F_{\text{ext}}^{\text{ref}} \cdot u_n$ )
    dd = num / den
    U += u_R + dd · u_t
    += dd
    U_iter = U_current + ΔU
    si converge: break

si converge:
    commit_all_states()
    += Δ
    actualizar_modo(...) # ← switching cilíndrico  disipación
    ajustar_tau_adaptativamente(iter)
sino:
    /= shrink_factor

```

11.9.2 5. Predictor / corrector — punto clave del cambio

Switching automático cilíndrico ↔ disipación

El solver mantiene un atributo `modo` con dos estados:

- `cylindrical`: arc-length cilíndrico estándar; restricción $\|\Delta U\|^2 = dl^2$. **Régimen elástico**, sin disipación activa.
- `"dissipation"`: arc-length por energía; restricción $g = \tau$. **Régimen disipativo**.

Criterio de transición (Verhoosel et al. 2009 §3.2):

- Inicio de paso en `cylindrical`: tras commit, calcular ΔE_d del paso recién cerrado. Si $\Delta E_d > \varepsilon_{\text{diss}}$ (umbral pequeño, p. ej. $10^{-12} \cdot \|\mathbf{F}_{\text{ext}}^{\text{ref}}\| \cdot \|u_n\|$) $\Rightarrow$ siguiente paso entra en `"dissipation"`. Inicializa $\tau \leftarrow \Delta E_d$ del último paso para continuidad.
- Inicio de paso en `"dissipation"`: si el paso anterior tuvo iteraciones $\leq \text{tau_relax_iter_threshold}$ y el "predictor de disipación" arrojaría $\Delta\lambda$ desproporcionadamente grande (señal de retorno a elástico tras la rama post-pico), revertir a `cylindrical`. Patológico pero posible si hay descarga global tras grieta saturada.

Sign-of-pivot tracking (mejora sobre el padre)

`ArcLengthSolver._negative_pivots()` es placeholder hoy. Esta variante **lo implementa** mediante factorización LDL^T (Bunch-Kaufman) del backend algebraico, exponiendo el conteo de pivots negativos como diagnóstico de bifurcación. Uso interno: detectar paso del punto límite (cambio en el signo del primer pivot indefinido) y orientar la dirección del predictor con respecto al signo previo, no sólo al producto escalar con ΔU_{prev} .

Esto resuelve el caso patológico de bifurcación simétrica donde el "menor ángulo" elige una rama paralela en lugar de continuar el camino físico — caveat documentado en `ArcLengthSolver` §10.

11.9.3 6. Criterio de convergencia (ADR 0007)

Idéntico al padre — patrón dual fuerza + desplazamiento, escala autoderivada.

11.9.4 7. Imposición de Dirichlet y MPC

Idéntico al padre. La restricción de disipación opera sobre DOFs libres después de la reducción Dirichlet/MPC.

11.9.5 8. Backend algebraico

Idéntico al padre (`is_positive_definite=False`, despachador LU). **Adición:** para sign-of-pivot tracking auténtico se requiere LDL^T ; la implementación inicial puede usar la firma de la factorización LU (signo del determinante) como aproximación, con upgrade a LDL^T Bunch-Kaufman cuando se priorice.

11.9.6 9. Adaptatividad — ajuste de τ

Análoga a la adaptatividad de dl del padre, con τ en lugar de dl :

- Converge en $< \text{tau_grow_iter_threshold}$ iteraciones $\Rightarrow \tau \rightarrow \tau \cdot \text{tau_grow_factor}$ (cota: $\text{initial_tau} \cdot \text{tau_max_factor}$).
- Converge en $> \text{tau_shrink_iter_threshold}$ iteraciones $\Rightarrow \tau \rightarrow \tau \cdot \text{tau_shrink_factor}$.
- No converge en $\text{max_iter} \Rightarrow \tau / 2$ y reintentar.
- Cota inferior: $\tau < \text{DISS_MIN_TAU_FACTOR} \cdot \text{initial_tau} \Rightarrow$ aborto.

En modo `cylindrical` durante régimen elástico inicial, la adaptatividad opera sobre dl (idéntica al padre).

11.9.7 10. Caveats numéricos

Heredados del padre (`ArcLengthSolver` §10) más los específicos:

- **Régimen disipativo no estrictamente monótono:** si el material vuelve transitoriamente a régimen elástico (descarga local), ΔE_d puede ser negativo. El solver tolera este caso pero conmuta a `cylindrical` si la disipación neta del paso cae por debajo del umbral.
- **Inicialización de τ :** debe ser del orden de la disipación esperada por paso (típicamente fracción pequeña de $G_F \cdot l_d$ por elemento agrietado). Demasiado pequeño $\Rightarrow$ pasos numerosos; demasiado grande $\Rightarrow$ saltos sobre la rama post-pico.
- **Disipación de origen no físico:** si el modelo numérico introduce disipación espuria (algorithmic damping en transitorio, regularización no consistente), ΔE_d contiene esa componente. **Este solver es estático:** si el usuario invoca `DissipationArcLengthSolver` sobre un modelo con material no disipativo (elástico puro), el modo permanece en `cylindrical` indefinidamente — comportamiento equivalente al padre. No es un caveat operacional, es la definición del switching.
- **MPC con offset g no homogéneo:** la restricción de disipación se evalúa sobre DOFs libres reducidos; si las MPC introducen offset, g se evalúa sobre el sistema reducido y la disipación medida es consistente con la energía aportada por las MPC (tratamiento idéntico al residuo en el padre).

11.10 Contrato de implementación

```

name: DissipationArcLengthSolver
kind: solver
status: implemented          # draft → implemented → validated. Validación parcial - ver §"
                             Implementación".

parent_spec: ArcLengthSolver

interface:
  yaml_type: dissipation_arclength
  output: SolveResult
  pipeline_kind: static

parameters:
  # Heredados del padre (todos con misma semántica):
  - { name: convergence,          type: ConvergenceCriterion, required: false,
      default: "ConvergenceCriterion()" }
  - { name: max_iter,             type: int,      required: false, default: 20 }
  - { name: max_lambda,          type: float,    required: false, default: 1.0 }
  - { name: max_steps,           type: int,      required: false, default: 100 }
  - { name: initial_dl,          type: float,    required: false, default: 0.1,
      desc: "Longitud de arco inicial - usado en modo cilíndrico (régimen elástico)" }
  - { name: dl_grow_factor,       type: float,    required: false, default: 1.5 }
  - { name: dl_max_factor,       type: float,    required: false, default: 5.0 }
  - { name: dl_shrink_factor,     type: float,    required: false, default: 0.6 }
  - { name: dl_grow_iter_threshold, type: int,    required: false, default: 4 }
  - { name: dl_shrink_iter_threshold, type: int,    required: false, default: 8 }
  - { name: linear_algebra,       type: str,      required: false, default: "auto" }

  # Específicos de esta variante:
  - { name: initial_tau,          type: float,    required: true,
      desc: "Incremento de disipación objetivo (unidades de energía: F·U). Calibrar al
            orden de G·F·l_d/n_pasos_post_pico" }
  - { name: tau_grow_factor,       type: float,    required: false, default: 1.5 }
  - { name: tau_max_factor,       type: float,    required: false, default: 5.0 }
  - { name: tau_shrink_factor,     type: float,    required: false, default: 0.6 }
  - { name: tau_grow_iter_threshold, type: int,    required: false, default: 4 }
  - { name: tau_shrink_iter_threshold, type: int,    required: false, default: 8 }
  - { name: dissipation_threshold, type: float,    required: false, default: 1.0e-12,
      desc: "Umbral relativo bajo el cual ΔE_d se considera 0 (mantiene modo cilíndrico)" }

requirements:
  - "Modelo con material disipativo (plasticidad, daño, cohesivo) - en modelo puramente
    elástico el solver opera equivalente a ArcLengthSolver cilíndrico"
  - "F_ext^ref no nulo (la restricción de disipación se evalúa sobre F_ext^ref·U)"
  - "Heredados del padre: apoyos suficientes; sin modos rígidos; dl_inicial razonable"

conventions:
  units: "heredadas del modelo (ADR 0008); en unidades de energía consistentes"
  stability: "incondicional respecto a puntos límite y rama post-pico de softening
    severo. La estabilidad ante bifurcaciones múltiples mejora vs el padre por el
    sign-of-pivot tracking"

out_of_scope:
  - "Bifurcaciones múltiples simétricas - la detección por inercia desambigua
    bifurcaciones simples; las múltiples siguen requiriendo perturbación explícita"

```

```

- "Análisis dinámico - la formulación de disipación es válida solo en régimen cuasi-
  estático conservativo"
- "Modo II y mixto I-II en cohesivo - heredado del scope ADR 0010 fases F+"
- "3D - coherente con el scope global del proyecto en 2026-05"

acceptance:
  verification:
    - name: recuperacion_del_arclength_cilindrico_en_elastico
      setup: "Modelo elástico puro (sin disipación); ejecutar
        DissipationArcLengthSolver y ArcLengthSolver con los mismos parámetros y
        misma malla"
      expect: "El modo permanece 'cylindrical'; trayectoria U() coincide a paridad de
        bits con el padre"
      tol_rel: 1.0e-12

    - name: snap_through_lee_frame
      setup: "Mismo benchmark que ArcLengthSolver $acceptance - frame de Lee, dos
        barras corrotacionales"
      expect: "Atraviesa el punto límite; U- coincide con Crisfield 1991 Fig. 9.5"
      tol_rel: 1.0e-3

    - name: damage_softening_post_pico
      setup: "Quad4 con IsotropicDamage2D, tracción que entra en rama descendente"
      expect: "Modo conmuta a 'dissipation' al detectar  $\Delta > 0$ ; rama post-pico trazada;
        coincide con ArcLengthSolver del padre dentro de tolerancia (ambos resuelven
        el mismo problema)"
      tol_rel: 1.0e-3

    - name: cohesive_embedded_softening_postpeak          # benchmark CRÍTICO - desbloquea
      fase 4 ADR 0010
      setup: "Probeta rectangular con CST_Embedded2D + CohesiveDamageIsotropic; tracció
        n uniaxial monotónica que activa Rankine y entra en rama post-pico con
        penalty K_e stiff (1e13)"
      expect: "El solver atraviesa la transición elástico-softening (régimen donde el
        ArcLengthSolver cilíndrico aborta); cierra el ciclo de softening; daño
        cohesivo committed alcanza saturación"
      tol_rel: 1.0e-3

    - name: balance_energetico_GF
      setup: "Mismo modelo cohesivo+embedded del test anterior; integrar  $\Sigma \Delta E_d$  sobre
        toda la historia hasta saturación  $\rightarrow (1)$  y comparar con  $G_F \cdot$ 
        longitud_grieta_abierta del material"
      expect: " $\Sigma \Delta E_d \cdot G_F \cdot l_d$  a tolerancia de discretización temporal (1% típico)"
      tol_rel: 1.0e-2

    - name: convergencia_temporal_tau
      setup: "Reducir initial_tau en factor 2 sucesivamente sobre el benchmark cohesivo
        , observar U(_final)"
      expect: "Convergencia primer orden en (esquema lineal en restricción): error  $\rightarrow 0$ 
        como 0()"
      tol_rel: 0.2

  specific:
    - name: switching_cilindrico_a_disipacion_en_pico
      setup: "Modelo con daño que cruza  $t_0$  en el paso N"
      expect: "Modos: 'cylindrical' en pasos 1..N-1; 'dissipation' desde N+1 en
        adelante; sin oscilación de modo en pasos contiguos"

    - name: sign_of_pivot_tracking_bifurcacion_simple

```

```

setup: "Modelo con un punto límite simple (snap-back); registrar el número de
      pivots negativos antes y después del paso del límite"
expect: "Conteo de pivots negativos cambia por 1 al atravesar el límite;
      predictor mantiene la dirección física"

- name: fallback_a_cilindrico_en_descarga_global
  setup: "Modelo con grieta saturada (1) y descarga: decrece, F_ext invertida"
  expect: "Modo conmuta de vuelta a 'cylindrical' al detectar  $\Delta E_d < \text{umbral}$ "

references:
- "Gutiérrez M.A. (2004). Energy release control for numerical simulations of failure
  in quasi-brittle solids. *Communications in Numerical Methods in Engineering*
  20(1), 19-29. (Restricción lineal de disipación; fórmula central de esta spec.)"
- "Verhoosel C.V., Remmers J.J.C., Gutiérrez M.A. (2009). A dissipation-based arc-
  length method for robust simulation of brittle and ductile failure. *
  International Journal for Numerical Methods in Engineering* 77(9), 1290-1321. (
  Versión robusta con switching y aplicación a cohesivos.)"
- "Crisfield M.A. (1991). *Non-linear Finite Element Analysis of Solids and
  Structures*, Vol. 1, Wiley. §9.4 (alternative constraints to cylindrical/
  spherical)."
```

```

- "de Borst R., Sluys L.J. (1999). *Computational Methods in Non-linear Solid
  Mechanics*, TU Delft. §3.5 (controlled descent)."
```

```

- "ADR 0010 $Hoja de ruta (fase 4 diferida por solver)."
```

```

- "Spec padre: ArcLengthSolver.md."
```

11.11 Implementación

- **Archivo:** `solidum/math/solvers/dissipation_arclength.py`.
- **Clase:** `DissipationArcLengthSolver(ArcLengthSolver)` — subclase. Reusa predictor, corrector, ajuste de paso, manejo de Dirichlet/MPC y backend del padre. Override completo de `solve()` con bifurcación por `self._mode`.
- **Atributo** `self._mode: cylindrical` | `"dissipation"`. Transición tras commit según umbral `'dissipation_threshold·||F_ref||·||U||'`.
- **Sign-of-pivot tracking:** `_negative_pivots()` implementado vía signo del slogdet de `K_t`. Aproximación válida para distinguir indefinitud simple (0 vs número impar de pivots negativos) — base para detección de paso por punto límite simple. **LDL^T Bunch-Kaufman queda como deuda técnica** para tracking exacto del conteo.
- ****Salvaguarda contra 'final_step prematuro**:** si `|dλ_pred|` excede $3 \times (\max_lambda - \lambda_curr)$, se bisecta `dl` o `τ` antes de aceptar el paso. Esencial en pasos iniciales sin calibrar y tras switch cilíndrico→disipación donde α puede ser pequeño.
- **Detección de $\alpha \approx 0$ vía threshold relativo:** `'|α| < 1e-6·½·|λ_n·F·du_t|'` reverte a cilíndrico — para problemas lineales monotónicos $\alpha = 0$ exactamente, no es un error sino la matemática del régimen sin disipación física.
- **Entrypoint público:** registrado vía `@SolverRegistry.register`, expuesto como `solidum.math.solvers.Di`

11.11.1 Tests (suite `tests/test_dissipation_arclength.py`)

7/7 verde. Cubren:

- Recuperación cilíndrica en régimen elástico puro (regresión a paridad de bits con `ArcLengthSolver`).
- Daño 1D con softening pronunciado (rama post-pico atravesada).
- Switching automático cilíndrico↔disipación observable.
- Daño 2D continuo (Quad4 + IsotropicDamage2D plane strain).
- Balance energético cualitativo ($\Sigma \Delta E_d \geq 0$ con daño activo).
- Sign-of-pivot tracking via signo del determinante (0 para PD, 1 para indefinida).

11.11.2 Validación parcial — qué falta para `status validated`

El acceptance ‘`cohesive_embedded_softening_postpeak`’ **no está cubierto** en esta primera implementación. Diagnóstico:

La activación discreta del cohesivo en `prepare_step` (Rankine onset al cruzar σ_{t0}) introduce un salto en `F_int` y `K_t` entre el paso pre-activación y el paso post-activación. El predictor por `sign = sign(dot( $\delta U_{prev}$ ,  $du_t$ ))` puede heredar una orientación inconsistente cuando el Newton del paso de activación reorienta `dU_iter` para acomodar el nuevo `K_t` con el cohesivo activo. El paso siguiente entonces predice hacia atrás (descarga) en lugar de continuar la rama post-pico.

Resolverlo limpiamente requiere **una de las dos vías siguientes**, ambas más allá del scope incremental de esta spec:

1. **Sign-of-pivot tracking real con LDL^T Bunch-Kaufman**, no aproximación por signo del determinante. Permitiría detectar la inercia exacta de `K_t` y orientar el predictor por cambio de signo de pivot en lugar del producto escalar $\delta U \cdot du_t$. Esto desacopla la orientación del predictor de la cinemática perturbada por la activación. Requiere extender el backend algebraico (ADR 0003 fase 2) con un solver LDL^T verdadero.
2. **Control por CMOD/CTOD del salto** mediante MPC sobre el DOF interno del cohesivo, en lugar de la restricción de disipación global. Cambia la naturaleza del solver — sería una spec separada (`CMODControlSolver`) en lugar de variante de arc-length.

Recomendación: dejar esta validación como deuda técnica priorizada en STATUS.md §”Deuda técnica priorizada” hasta que el LDL^T Bunch-Kaufman entre en otro contexto (lo amerita por más razones, no solo esta).

11.12 Diálogo

- **2026-05-19 (tarde)** · Spec promovida de `draft` a `implemented`. 7/7 tests verde sobre los casos de daño continuo (1D y 2D); el caso crítico cohesivo+embedded reveló una limitación más profunda del `sign` por producto escalar tras activación discreta del Rankine, que requiere LDL^T Bunch-Kaufman real o control CMOD/CTOD — ambos fuera del scope incremental de esta variante. Documentado en §Implementación como deuda técnica. La spec mantiene su valor: provee la arquitectura del switching cilíndrico↔disipación y la fórmula lineal de Gutiérrez funcional para todos los problemas con softening continuo (daño bulk 1D y 2D), que es donde más casos prácticos de Solidum se concentran.

- **2026-05-19 (mañana)** · Spec creada como variante de `ArcLengthSolver` para desbloquear ADR 0010 fase 4 (embedded discontinuity con softening severo) y validación cuantitativa de G_F en pipeline real. Motivada por el cierre de la campaña de validación externa Tandas 12-14 ([`project_validacion_externa_tandas_12_14`]) que identificó esta pieza como **el primer componente nuevo priorizado** tras la campaña. Sin ADR nuevo: reusa decisiones arquitecturales del padre (estructura del bucle predictor/corrector, manejo de Dirichlet/MPC, ADR 0003 algebraico, ADR 0007 convergencia, ADR 0008 unidades). El cambio se circunscribe a la **restricción del paso** (cuadrática $\rightarrow$ lineal) más mejoras secundarias (switching automático cilíndrico $\leftrightarrow$ disipación, sign-of-pivot tracking).
- **Puntos abiertos para el usuario** (físicos/de alcance, no de plumbing — ver [`feedback_filtrar_puntos_a`])
 1. ¿La restricción de disipación de Gutiérrez 2004 es la fórmula que quieres, o prefieres una alternativa (p. ej. constraint sobre K_t -norma de Crisfield 1991 §9.4)? La de Gutiérrez es la más extendida en literatura cohesiva.
 2. ¿El switching automático cilíndrico $\leftrightarrow$ disipación es deseable, o prefieres dos solvers separados (`CylindricalArcLengthSolver` y `DissipationArcLengthSolver` puros) que el usuario elige según el caso? El switching es estándar en Verhoosel et al. 2009; dos solvers separados son más predecibles pero exigen que el usuario diagnostique cuándo cambiar.
 3. ¿Sign-of-pivot tracking se aborda en esta spec o se difiere a una spec independiente (`PivotTracker`)? Está acoplado al backend algebraico (ADR 0003) — si el LDL^T Bunch-Kaufman se introduce como helper compartido, otros solvers se beneficiarían. Mi propuesta es **abordarlo aquí en versión "signo del determinante LU"** (aproximación válida para indefinitud simple) y dejar el LDL^T verdadero como deuda técnica del proyecto.

11.13 LinearSolver

*Spec **retroactiva**: el solver existe desde la primera fase del proyecto y está validado por la inmensa mayoría de tests del pipeline estático. Esta spec lo documenta sin cambiar comportamiento — H-5.3 de la auditoría 2026-05-18.*

11.14 Especificación física

11.14.1 0. Descripción general

Análisis estático **lineal** de una estructura discretizada por FEM: resolución directa del sistema algebraico $\mathbf{K} \cdot \mathbf{U} = \mathbf{F}$. Hipótesis: linealidad geométrica y material (tangente constante = rigidez secante), apoyos Dirichlet (homogéneos o no homogéneos), restricciones lineales multipunto opcionales (ADR 0004). Sin iteración. Sin historia de carga.

11.14.2 1. Ecuación de equilibrio resuelta

Forma fuerte semidiscreta:

$$\mathbf{K} \mathbf{U} = \mathbf{F}_{\text{ext}}$$

donde $\mathbf{K}$ es la matriz de rigidez global ensamblada, $\mathbf{U}$ los desplazamientos nodales (incluidos los Dirichlet) y $\mathbf{F}_{\text{ext}}$ las cargas externas (incluidas peso propio si está activado).

11.14.3 2. Condiciones de contorno

Apoyos Dirichlet (homogéneos $u_s = 0$ o no homogéneos $u_s = g$) eliminados directamente por ADR 0004 fase 1. Restricciones lineales multipunto admitidas vía transformación $\mathbf{T}$ + offset $\mathbf{g}$ (ADR 0004 fase 2). El sistema reducido se resuelve en DOFs libres:

$$\mathbf{K}_{\text{red}} = \mathbf{T}^\top \mathbf{K} \mathbf{T}, \quad \mathbf{F}_{\text{red}} = \mathbf{T}^\top (\mathbf{F} - \mathbf{K} \mathbf{g})$$

11.14.4 3. Salidas físicas

- Vector de desplazamientos global $\mathbf{U} \in \mathbb{R}^{n_{\text{dof}}}$ (libres + prescritos consistentes).
- Reacciones se recuperan a posteriori vía `domain.reactions(U)`.
- Esfuerzos en puntos de Gauss vía `domain.recompute_gauss_state(U)`.
- El resultado se envuelve en `SolveResult` por la capa pública (`solidum.run_static`).

11.15 Formulación numérica

11.15.1 4. Esquema operativo

Un solo paso:

1. Ensamblar $\mathbf{K}$ (sparse COO $\rightarrow$ CSR) con caché topológica del Assembler.
2. Aplicar la reducción ADR 0004: $\mathbf{K}_{\text{red}}$, $\mathbf{F}_{\text{red}}$, operadores $\mathbf{T}$ y $\mathbf{g}$.

3. Calcular $\mathbf{F}_{\text{dir}} = \mathbf{T}^\top (\mathbf{K} \mathbf{g})$ (contribución de Dirichlet no homogéneo).
4. Factorizar $\mathbf{K}_{\text{red}}$ con el backend algebraico seleccionado.
5. Resolver $\mathbf{U}_{\text{red}} = \mathbf{K}_{\text{red}}^{-1} (\mathbf{F}_{\text{red}} - \mathbf{F}_{\text{dir}})$.
6. Expandir $\mathbf{U} = \mathbf{T} \mathbf{U}_{\text{red}} + \mathbf{g}$.

11.15.2 5. Predictor / corrector

No aplica — solver no iterativo.

11.15.3 6. Criterio de convergencia

No aplica — la solución directa es exacta (a precisión de máquina y condicionamiento de $\mathbf{K}$).

11.15.4 7. Imposición de Dirichlet y MPC

ADR 0004 fase 1 (eliminación directa de apoyos) y fase 2 (transformación $\mathbf{T}$ + offset $\mathbf{g}$ para MPC). Manejados uniformemente por `Assembler.reduce(...)`.

11.15.5 8. Backend algebraico

Despacho automático ADR 0003:

- Si el dominio es **simétrico** (todos los materiales con tangente simétrica) $\Rightarrow$ se asume PD y se factoriza con **Cholesky** (`scipy.sparse.linalg.splu` con `options={'SymmetricMode': True}` o equivalente Cholesky sparse).
- Si Cholesky reporta no-positividad (puede ocurrir con casi-singular o mal condicionado) $\Rightarrow$ **fallback automático** a LU.
- Si el dominio declara material **no simétrico** $\Rightarrow$ LU directo.

Override manual: parámetro `linear_algebra` $\in \{.auto, cholesky, lu\}$.

Cache de factorización (ADR 0003 fase 2): la primera llamada a `solve` ensambla $\mathbf{K}$, reduce y factoriza; la factorización se guarda en el solver. Llamadas posteriores con un $\mathbf{F}$ distinto reutilizan el factor sin reensamblaje — el coste se reduce a una resolución triangular barata. Si el usuario modifica el modelo entre llamadas (nuevos elementos, BCs, materiales) debe invocar `invalidate_cache()` explícitamente.

11.15.6 9. Adaptatividad / control de paso

No aplica.

11.15.7 10. Caveats numéricos

- **Suposición de PD por simetría:** el despachador asume que un dominio con materiales simétricos tiene $\mathbf{K}$ positiva definida. Es cierto para elasticidad estándar con suficientes apoyos; falso para casos degenerados (apoyos insuficientes, modos rígidos no restringidos, casi-singularidad geométrica). El fallback Cholesky $\rightarrow$ LU cubre estos casos.

- **Cache desactualizado:** si el modelo se modifica entre `solve` calls sin llamar a `invalidate_cache`, los resultados serán inconsistentes. La política deliberada es **cache silenciosa** (no se detectan modificaciones del modelo automáticamente) por velocidad — la responsabilidad de invalidar es del usuario.
- **No es no lineal:** si el problema es no lineal (material plástico, geometría corotacional con cargas significativas) usar `NonlinearSolver`. Este solver evaluará la tangente en estado virgen ($U = 0$) y producirá una respuesta lineal incorrecta sin avisar.

11.16 Contrato de implementación

```

name: LinearSolver
kind: solver
status: validated

interface:
  yaml_type: linear
  output: SolveResult
  pipeline_kind: static

parameters:
  - { name: linear_algebra, type: str, required: false, default: "auto",
      desc: "Selección del backend algebraico: 'auto' (→CholeskyLU según simetría), 'cholesky', 'lu' } }

requirements:
  - "Modelo lineal: materiales con tangente constante, geometría no corotacional o cargas suficientemente pequeñas"
  - "Apoyos suficientes para descartar modos rígidos (estructura cinemáticamente estable)"
  - "Compatibilidad ensamblaje-elemento-material verificada por Domain"

conventions:
  units: "heredadas del modelo (ADR 0008)"
  stability: "trivial - un solo paso"

out_of_scope:
  - "Análisis no lineal    usar NonlinearSolver"
  - "Snap-through / snap-back    usar ArcLengthSolver"
  - "Análisis dinámico    usar NewmarkSolver, HHTSolver, etc."
  - "Detección automática de modos rígidos (apoyos insuficientes)"

acceptance:
  verification:
    - name: viga_en_voladizo_carga_puntual
      setup: "viga de Frame2DEuler en voladizo con carga puntual P en el extremo"
      expect: " =  $P \cdot L^3 / (3 \cdot E \cdot I) \pm 0(\text{elementos})$ "
      tol_rel: 1.0e-8
    - name: barra_traccion_uniaxial
      setup: "Truss2D, longitud L, área A, carga axial F"
      expect: " =  $F \cdot L / (E \cdot A)$  exacto (sin discretización)"
      tol_rel: 1.0e-12
    - name: convergencia_h
      setup: "viga discretizada con N elementos, N {2, 4, 8, 16}"
      expect: "error decrece con orden teórico del elemento (4 para Hermite cúbicos)"
      tol_rel: 0.1

```

```

- name: reactions_equilibrio
  setup: "estructura cargada en una dirección"
  expect: " $\Sigma(\text{reacciones}) + \Sigma(\text{cargas externas}) = 0$  (Newton 3ª ley)"
  tol_abs: 1.0e-9
- name: dirichlet_no_homogeneo
  setup: "barra con apoyo prescrito  $u_s = g = 0$  en un extremo"
  expect: "solución exacta esperada (translación rígida + alargamiento si carga)"
  tol_rel: 1.0e-12
- name: cache_factorizacion
  setup: "dos llamadas consecutivas a solve con F distintos sin invalidar cache"
  expect: "segunda llamada reutiliza el factor (mensaje de log diferenciado);
           resultado igual al cómputo desde cero"
  tol_rel: 1.0e-12
- name: fallback_cholesky_lu
  setup: "matriz simétrica pero casi-singular o no PD efectiva (mecanismo cinemá
           tico añadido y luego soltado)"
  expect: "Cholesky aborta, LU resuelve sin fallar"
- name: mpc_simple
  setup: "Quad4 con MPC  $u_2 = u_1$  (master/slave)"
  expect: " $u_2 = u_1$  en la solución, equilibrio respetado"
  tol_abs: 1.0e-12

references:
- "Bathe K.J. (2014). Finite Element Procedures. Prentice Hall. §2."
- "Cook R.D., Malkus D.S., Plesha M.E., Witt R.J. (2002). Concepts and Applications
  of FEA. Wiley. §2.4 (eliminación de DOFs Dirichlet)."
- "ADR 0003 - selección automática de backend algebraico."
- "ADR 0004 - manejo de Dirichlet y constraints lineales."

```

11.17 Implementación

- **Archivo:** solidum/math/solvers/linear.py.
- **Clase:** LinearSolver, registrada vía @SolverRegistry.register con PIPELINE_KIND = "static".
- **Cache:** atributos _factor, _T, _g_full, _F_dir, _n_free se rellenan lazy en la primera llamada a solve. Método público invalidate_cache() los pone a None para forzar reensamblaje.
- **Fallback Cholesky→LU:** capturado en _build_cache vía CholeskyNotPositiveDefiniteError (ADR 0003 §5); registra un warning y reintenta con LU.
- **Entrypoint público:** solidum.run_static(model, solver="linear", ...) (despacho declarativo por PIPELINE_KIND, regla C de la auditoría).
- **Tests:** cobertura masiva implícita — prácticamente todos los tests del pipeline estático lineal lo usan. Tests específicos del cache: tests/test_solver_robustness.py::test_linear_solver_cache_reuse.

11.18 Diálogo

- **2026-05-19** · Spec creada retroactivamente para cerrar el hueco H-5.3. Solver anterior a la convención de specs validadas. La cache de factorización se añadió en la sesión de saneamiento post-auditoría (commit b517718, H-4.2); esta spec recoge ese comportamiento como parte del contrato actual.

11.19 NonlinearSolver

*Spec **retroactiva**: el solver existe desde la primera fase del proyecto y está validado por tests de plasticidad, daño, corotacional, embedded. Esta spec lo documenta sin cambiar comportamiento — H-5.3 de la auditoría 2026-05-18.*

11.20 Especificación física

11.20.1 0. Descripción general

Análisis estático **no lineal** con **control de carga** (load-controlled) en una estructura discretizada por FEM. La no-linealidad puede provenir de la geometría (formulación corotacional, grandes desplazamientos pequeñas deformaciones) o del material (plasticidad, daño, embedded discontinuity). Esquema **incremental-iterativo**: la carga objetivo se aplica por incrementos $\Delta\lambda$, y en cada incremento se resuelve el equilibrio por **Newton-Raphson** con tangente reensamblada y paso bisechado al fallar.

11.20.2 1. Ecuación de equilibrio resuelta

Forma fuerte semidiscreta, residuo:

$$\mathbf{R}(\mathbf{U}, \lambda) = \lambda \mathbf{F}_{\text{ext}}^{\text{ref}} - \mathbf{F}_{\text{int}}(\mathbf{U}) = \mathbf{0}$$

con $\lambda \in [0, 1]$ factor de carga aplicado a la carga de referencia $\mathbf{F}_{\text{ext}}^{\text{ref}}$, y $\mathbf{F}_{\text{int}}(\mathbf{U})$ fuerzas internas no lineales (dependientes del estado actual de los puntos de Gauss).

11.20.3 2. Condiciones de contorno

Idénticas a **LinearSolver**: Dirichlet (homogéneo / no homogéneo) y MPC vía ADR 0004. Las cargas Dirichlet no homogéneas se aplican proporcionalmente al factor de carga (incremento $\Delta\mathbf{g} = \mathbf{g} \cdot \Delta\lambda$).

11.20.4 3. Salidas físicas

- **U** final converged en $\lambda = 1$.
- Historia disponible vía `step_callback(step, U, load_factor)` que el usuario puede pasar al solver.
- Diagnóstico de divergencia con excepciones tipadas (ADR 0011): `OscillatingNewtonError`, `SingularTangentError`, `LoadExceedsCapacityError`, `UnknownDivergenceError`, todas subclases de `SolverDivergedError`.

11.21 Formulación numérica

11.21.1 4. Esquema operativo

```

= 0Δ
= 1 / num_steps
mientras < 1:
    intentar avanzar → + Δ:
        U_iter = U_current
        prepare_all_steps(U_current) # hook ADR 0010 (embedded)
        para iter en range(max_iter):
            K_t, F_int = assemble_non_linear_system(U_iter)
            R = Δ(+)·F_ext - F_int
            U = K_t^{-1} · R # reducido ADR 0004
            U_iter, |||R, F_int = line_search(...) # ADR 0011, opt-in
            si converge (ADR 0007 dual):
                commit_all_states()
                += Δ
                si iter < threshold y Δ < base:Δ←
                    minΔ( · growth, base) # adaptativo: acelerar
                break
            si no converge:Δ
                /= 2 # bisección
                si Δ < min_delta_lambda:
                    clasificar divergencia y raise # ADR 0011

```

11.21.2 5. Predictor / corrector

Predictor: $\Delta \mathbf{U}^{(0)} = \mathbf{0}$ — el incremento empieza desde el estado convergido del paso anterior.

Corrector (Newton iter k): $\mathbf{K}_t^{(k)} \delta \mathbf{U}^{(k)} = \mathbf{R}^{(k)}$, $\mathbf{U}^{(k+1)} = \mathbf{U}^{(k)} + \alpha \delta \mathbf{U}^{(k)}$, con α del line search (default $\alpha = 1$).

11.21.3 6. Criterio de convergencia (ADR 0007)

Patrón dual fuerza + desplazamiento, con escalas autoderivadas:

$$\|\mathbf{R}[\text{libres}]\| \leq \text{atol}_F + \text{rtol}_F \cdot \max(\|\mathbf{F}_{\text{ext}}\|, \|\mathbf{F}_{\text{int}}\|)$$

$$\|\Delta \mathbf{U}\| \leq \text{atol}_d + \text{rtol}_d \cdot \|\mathbf{U}\|$$

Semántica **AND** (ambos simultáneamente). Calibración: en el primer ensamblaje del solve, $\text{force_scale} = \max(\|\mathbf{F}_{\text{ext}}\|, \|\mathbf{F}_{\text{int}}\|, 1)$ y $\text{disp_scale} = \text{force_scale} / \max(|\text{diag}(\mathbf{K})|)$.

11.21.4 7. Imposición de Dirichlet y MPC

Idéntica a `LinearSolver` — ADR 0004. La carga Dirichlet no homogénea $\mathbf{g}$ se aplica proporcionalmente al factor de carga λ a través del operador del Assembler (`U_current=U_iter`, `load_factor=next_load_factor` en la llamada a `reduce`).

11.21.5 8. Backend algebraico

Despacho automático ADR 0003 idéntico a `LinearSolver`. Fallback Cholesky→LU activado.

Newton modificado (ADR 0003 fase 2): parámetro opcional `freeze_tangent_after_iter`. Si es int N , el solver factoriza fresca durante las primeras N iteraciones del paso y reutiliza la

factorización en las siguientes iteraciones del mismo paso (no entre pasos — la tangente cambia con $\mathbf{U}$). Default `None` = Newton estándar (factoriza cada iter).

11.21.6 9. Adaptatividad / control de paso

Acelerado: si un paso converge en $< \text{NEWTON_ADAPTIVE_GROWTH_ITER_THRESHOLD}$ iteraciones y $\Delta\lambda < 1/\text{num_steps}$, $\Delta\lambda$ se multiplica por `NEWTON\_ADAPTIVE\_GROWTH\_FACTOR` (cota: $1/\text{num_steps}$).

Bisección: si un paso no converge en `max_iter`, $\Delta\lambda \rightarrow \Delta\lambda/2$ y se reintenta. Aborto si $\Delta\lambda < \text{min_delta_lambda}$.

Line search ADR 0011 (opt-in, default `False`): variante GLL (Grippe-Lampariello-Lucidi) — backtracking simple por descenso no monótono. Justificación: line search Wolfe puro rechaza pasos Newton correctos en regímenes plásticos/dañados con residuo transitorio creciente. Documentado en ADR 0011.

11.21.7 10. Caveats numéricos

- **Snap-through / snap-back:** NO soportado — `NonlinearSolver` con control de carga falla en puntos límite con $dU/d\lambda \rightarrow \infty$ o $d\lambda/dU < 0$. Usar `ArcLengthSolver`. Excepción: el solver atraviesa puntos límite *suaves* (sin cambio de signo en $d\lambda$) con bisección + cinemática corotacional, hallazgo empírico de la auditoría 2026-05-18.
- **Softening severo con penalty cohesivo stiff** (embedded): el solver no atraviesa la transición elástico→softening con K_t casi singular en κ_0 . Limitación específica al embedded; el daño bulk continuo (1D/2D) sí se atraviesa.
- **Plasticidad perfecta con line search off:** puede oscilar (residuo periódico, no diverge ni converge). Mitigación: activar `line_search=True`.
- **Tangente singular:** lanza `SingularTangentError` con el último estado.
- **Newton modificado:** con `freeze_tangent_after_iter` no nulo, la convergencia cuadrática se degrada a lineal en las iteraciones congeladas. Útil cuando el coste de factorizar domina (K_t grande, materiales con tangente cara de evaluar).

11.22 Contrato de implementación

```
name: NonlinearSolver
kind: solver
status: validated

interface:
  yaml_type: nonlinear
  output: SolveResult
  pipeline_kind: static

parameters:
  - { name: convergence,                type: ConvergenceCriterion, required: false,
      default: "ConvergenceCriterion()",
      desc: "Política dual fuerza+desplazamiento (ADR 0007)" }
  - { name: max_iter,                   type: int,      required: false, default: 20,
      desc: "Iteraciones Newton máximas por paso antes de biseccionar (keyword-only)" }
  - { name: num_steps,                  type: int,      required: false, default: 10,
```



```

    desc: "Número base de incrementos para alcanzar =1 (keyword-only)" }
- { name: adaptive,                      type: bool,   required: false, default: true,
    desc: "Habilita acelerado + bisección de  $\Delta$  (keyword-only)" }
- { name: min_delta_lambda,              type: float, required: false, default:
    NEWTON_DEFAULT_MIN_DELTA_LAMBDA,
    desc: "Cota inferior para  $\Delta$ ; por debajo se aborta con SolverDivergedError (
        keyword-only)" }
- { name: linear_algebra,                 type: str,   required: false, default: "auto",
    desc: "'auto' ( $\rightarrow$ CholeskyLU según simetría), 'cholesky', 'lu' (keyword-only)" }
- { name: freeze_tangent_after_iter,      type: int|None, required: false, default: null
    ,
    desc: "Newton modificado: factoriza fresca primeras N iter, congela después (
        keyword-only)" }
- { name: line_search,                   type: bool,   required: false, default: false,
    desc: "Line search GLL ADR 0011 (keyword-only)" }

requirements:
- "Modelo con no-linealidad bien definida (material con tangente válida en todo el
    rango de carga)"
- "Cargas Dirichlet no homogéneas constantes en magnitud (se aplican
    proporcionalmente a )"
- "Apoyos suficientes para descartar modos rígidos"

conventions:
    units: "heredadas del modelo (ADR 0008)"
    stability: "no garantizada - los puntos límite con d /dU<0 abortan vía bisección"

out_of_scope:
- "Snap-through / snap-back con d /dU<0  usar ArcLengthSolver"
- "Análisis dinámico  usar NewtonNewmarkSolver, NewtonHHTSolver, etc."
- "Buckling lineal (eigenproblema generalizado)  futuro BucklingSolver"

acceptance:
    verification:
        - name: recuperacion_caso_lineal
          setup: "modelo enteramente lineal (Elastic2D + Quad4 + sin corotacional)"
          expect: "resultado idéntico a LinearSolver; Newton converge en 1 iteración por
              paso"
          tol_rel: 1.0e-12

        - name: plasticidad_uniaxial_truss
          setup: "Truss2D con Elastoplastic1D, carga axial creciente hasta plástico"
          expect: "respuesta bilineal exacta (E hasta yield, E_t = E·H/(E+H) después)"
          tol_rel: 1.0e-6

        - name: corotacional_voladizo_grande
          setup: "viga Frame2DEulerCorot, carga lateral grande (rotación >30°)"
          expect: " obtenido coincide con benchmark Bathe-Bolourchi 1979"
          tol_rel: 1.0e-3

        - name: damage_bulk_softening
          setup: "Quad4 con IsotropicDamage2D, tracción monotónica hasta post-pico"
          expect: "rama softening trazada; convergencia paso a paso con bisección"
          tol_rel: 1.0e-3

        - name: divergencia_clasificada
          setup: "carga que excede la capacidad última (plasticidad perfecta con mecanismo)
              "

```

```

expect: "raise OscillatingNewtonError o LoadExceedsCapacityError con métricas
estructuradas (ADR 0011)"

- name: paso_adaptativo_acelera_y_bisecta
  setup: "trayectoria con tramos elásticos rápidos y zona plástica lenta"
  expect: " $\Delta$  crece en tramos fáciles y bisecta en zonas duras; alcanza =1"

specific:
- name: newton_modificado_factoriza_menos
  setup: "freeze_tangent_after_iter=2, 5 iter típicas por paso"
  expect: "factorizaciones por paso 3 en lugar de 5; resultado dentro de
tolerancia"
  tol_rel: 1.0e-8

- name: line_search_corrige_oscilacion_DP
  setup: "Drucker-Prager 2D perfectamente plástico con carga que provoca oscilación
"
  expect: "line_search=True converge; line_search=False oscila o requiere bisección
extra"

references:
- "Crisfield M.A. (1991). Non-linear Finite Element Analysis of Solids and Structures
, Vol. 1. Wiley. $1-$9."
- "Bonet J., Wood R.D. (2008). Nonlinear Continuum Mechanics for Finite Element
Analysis. Cambridge. $8-$9."
- "de Borst R., Sluys L.J. (1999). Computational Methods in Non-linear Solid
Mechanics. TU Delft. $3 (Newton-Raphson y variantes)."
- "ADR 0003, ADR 0004, ADR 0006, ADR 0007, ADR 0011."

```

11.23 Implementación

- **Archivo:** `solidum/math/solvers/nonlinear.py`.
- **Clase:** `NonlinearSolver`, registrada vía `@SolverRegistry.register` con `PIPELINE_KIND = "static"`.
- **Argumentos keyword-only:** todos menos `assembler` y `convergence` son keyword-only (auditoría H-1.5, sesión 2026-05-19, commit 4e4ed54).
- **Bisección y clasificación:** en `solve`, al fallar $\Delta\lambda < \text{min_delta_lambda}$ se llama a `classify_divergence(res, delta_history, singular_tangent_detected)` que devuelve la subclase apropiada de `SolverDivergedError` con métricas tipadas.
- **Entrypoint público:** `solidum.run_static(model, solver="nonlinear", ...)`.
- **Tests:** cobertura amplísima — todos los tests de plasticidad/daño/corotacional/embedded del pipeline estático lo ejercitan. Tests específicos del comportamiento adaptativo y de divergencia tipada en `tests/test_solver_robustness.py` y `tests/test_solver_diagnostics.py`.

11.24 Diálogo

- **2026-05-19** · Spec creada retroactivamente para cerrar el hueco H-5.3. Solver anterior a la convención de specs. Comportamiento documenta el estado tras la sesión de saneamiento post-auditoría (commits 4e4ed54 para keyword-only, ADR 0011 ya integrado).

Capítulo 12

Esquemas de Solución — Modal y dinámicos

12.1 CentralDifferenceSolver

*Spec de **solver nuevo** (no variante).* Cubre el integrador explícito de segundo orden para análisis dinámico transitorio lineal y no lineal. Aprovecha la masa lumped diagonal habilitada por la fase 2 del ADR 0009 para evitar resolver sistema lineal en cada paso.

12.2 Especificación física

12.2.1 0. Descripción general

Análisis dinámico transitorio **explícito** de segundo orden. Resuelve $\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \mathbf{F}(\mathbf{t})$ (lineal) o $\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{F}_{\text{int}}(\mathbf{u}) = \mathbf{F}(\mathbf{t})$ (no lineal) avanzando la aceleración en cada paso por inversión trivial de la masa diagonal lumped — sin sistema lineal, sin Newton interno. Apropiado para wave propagation, impacto, dinámica de respuesta rápida.

Hipótesis cinemáticas globales: las mismas que admite el bloque de elementos del modelo (corrotacional, pequeñas deformaciones, etc.). Material lineal o no lineal con historia — en modo `nonlinear=True` el solver llama `Assembler.assemble_non_linear_system(u_n)` cada paso para reevaluar $\mathbf{F}_{\text{int}}$.

12.2.2 1. Ecuación de equilibrio resuelta

$$\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{F}_{\text{int}}(\mathbf{u}) = \mathbf{F}_{\text{ext}}(t)$$

con $\mathbf{F}_{\text{int}}(\mathbf{u}) = \mathbf{K}\mathbf{u}$ en el caso lineal.

12.2.3 2. Condiciones iniciales / de contorno

- Apoyos Dirichlet constantes en el tiempo (eliminación directa, ADR 0004).
- $\mathbf{u}(0) = \mathbf{u}_0$, $\dot{\mathbf{u}}(0) = \dot{\mathbf{u}}_0$ (cero por defecto).
- Cargas externas $\mathbf{F}_{\text{ext}}(t)$ vía `F_func(t)`.

- Amortiguamiento Rayleigh $\mathbf{C} = \alpha \mathbf{M} + \beta \mathbf{K}$ (mismo contrato que `NewmarkSolver`).

MPC no soportado en esta fase — no aparece en uso real con dinámica explícita.

12.2.4 3. Salidas físicas

`TransientResult` con historiales:

- `u_history[ndof, n_steps+1]` desplazamientos.
- `udot_history[ndof, n_steps+1]` velocidades.
- `uddot_history[ndof, n_steps+1]` aceleraciones.
- `t_history[n_steps+1]` instantes de tiempo.

12.3 Formulación numérica

12.3.1 4. Esquema operativo — leapfrog Belytschko-Liu-Moran

Inicialización:

$$\mathbf{a}_0 = \mathbf{M}^{-1}(\mathbf{F}(0) - \mathbf{F}_{\text{int}}(\mathbf{u}_0) - \mathbf{C} \mathbf{v}_0 - \mathbf{F}_{\text{dir}})$$

$$\mathbf{v}_{1/2} = \mathbf{v}_0 + \frac{\Delta t}{2} \mathbf{a}_0$$

Paso $n \rightarrow n + 1$:

$$\mathbf{u}_{n+1} = \mathbf{u}_n + \Delta t \mathbf{v}_{n+1/2}$$

$$\mathbf{F}_{\text{int}}^{n+1} = \mathbf{K} \mathbf{u}_{n+1} \quad (\text{lineal}) \quad \text{o} \quad \text{ensamble}(\mathbf{u}_{n+1}) \quad (\text{no lineal})$$

$$\mathbf{a}_{n+1} = \mathbf{M}^{-1}(\mathbf{F}(t_{n+1}) - \mathbf{F}_{\text{int}}^{n+1} - \mathbf{C} \mathbf{v}_{n+1/2} - \mathbf{F}_{\text{dir}})$$

$$\mathbf{v}_{n+1} = \mathbf{v}_{n+1/2} + \frac{\Delta t}{2} \mathbf{a}_{n+1} \quad (\text{salida de paso})$$

$$\mathbf{v}_{n+3/2} = \mathbf{v}_{n+1} + \frac{\Delta t}{2} \mathbf{a}_{n+1} \quad (\text{para el paso siguiente})$$

Sin Newton interno — la ecuación de aceleración es explícita en $\mathbf{F}_{\text{int}}^{n+1}$ que ya se evaluó en el estado actual.

12.3.2 5. Predictor / corrector

El esquema leapfrog **no tiene predictor/corrector** en el sentido implícito. La actualización $\mathbf{u}_{n+1}$ se calcula sólo con datos en n y $n + 1/2$; la velocidad $\mathbf{v}_{n+1}$ se obtiene del semipaso.

12.3.3 6. Criterio de convergencia

No aplica — esquema no iterativo. La convergencia "del problema" se materializa en la condición CFL y en la detección de divergencia.

12.3.4 7. Imposición de Dirichlet y MPC

Eliminación directa idéntica a `NewmarkSolver` (operador $\mathbf{T}$ + offset $\mathbf{g}$). MPC fuera de alcance en esta fase.

12.3.5 8. Backend algebraico

Ninguno — la única operación lineal es $\mathbf{M}^{-1} \cdot \mathbf{r}$ con $\mathbf{M}$ diagonal, implementada como $1/\text{diag}(\mathbf{M})$ y multiplicación elemento a elemento. El solver verifica al construir $\mathbf{M}_{\text{red}}$ que la diagonal es estrictamente positiva y que los off-diagonals son numéricamente nulos (tolerancia $10^{-12} \cdot \max|\text{diag}|$); si no, lanza `ValueError`.

12.3.6 9. Adaptatividad / control de paso

No adaptativo en esta fase — paso temporal constante Δt . La adaptatividad explícita (e.g. estimación de $\omega_{\text{máx}}$ por power iteration con actualización por paso) queda diferida hasta que aparezca un caso de uso con $\omega_{\text{máx}}$ variable significativamente en el tiempo.

12.3.7 10. Caveats numéricos

- **Estabilidad condicional.** La condición CFL exige $\Delta t < 2/\omega_{\text{máx}}$ donde $\omega_{\text{máx}}$ es la frecuencia natural máxima del sistema discreto. Para una barra axial elástica con elemento de longitud L_e y velocidad de propagación $c_p = \sqrt{E/\rho}$: $\Delta t_{\text{crit}} \approx L_e/c_p$. Para sólidos 2D la fórmula análoga es $\Delta t_{\text{crit}} \approx h_e/c_d$ donde c_d es la velocidad dilatacional. Si se excede, la solución diverge exponencialmente — el solver lo detecta a posteriori (umbral configurable `divergence_threshold`, default $10^8 \cdot |\mathbf{u}_0|$) y aborta con `RuntimeError` informativo.
- **Frame3D oblicuo.** La fase 2 del ADR 0009 documenta que $\mathbf{M}_{\text{lumped}}$ es bloque-diagonal (no estrictamente diagonal) en Frame3D cuando el eje del elemento no coincide con un eje global. Para central differences esto significaría invertir bloques 6×6 por nodo cada paso, **anulando** la ventaja del esquema explícito. El solver rechaza ese caso con `ValueError`.
- **Lumping obligatorio.** `lumping=consistent` se rechaza al construir el solver — invertir una $\mathbf{M}$ consistente cada paso es caro y elimina la razón de ser del explícito. El usuario debe pasar `lumping="lumped"` (default).
- **Modos espurios de altas frecuencias.** El esquema no introduce amortiguamiento numérico (a diferencia de HHT- α); las altas frecuencias de la malla pueden contaminar la respuesta de bajas frecuencias en problemas de wave propagation. Mitigaciones: refinamiento de malla, lumping puro (mejor filtrado que consistent), o cambio a HHT- α implícito.

12.4 Contrato de implementación

```
name: CentralDifferenceSolver
kind: solver
status: validated

interface:
  yaml_type: CentralDifferenceSolver
  output: TransientResult
```

```

parameters:
- { name: t_end, type: float, required: true, desc: "tiempo final (s)" }
- { name: dt, type: float, required: true, desc: "paso temporal constante (s); debe
  satisfacer  $\Delta t < \Delta t_{crit}$ " }
- { name: nonlinear, type: bool, required: false, desc: "default False; True
  recalcula  $F_{int}$  cada paso vía ensamble" }
- { name: rayleigh, type: dict | null, required: false, desc: " $C = \cdot M + \cdot K$ ; mismo
  contrato que NewmarkSolver" }
- { name: u0, type: ndarray | null, required: false, desc: "desplazamiento inicial
  global" }
- { name: u0_dot, type: ndarray | null, required: false, desc: "velocidad inicial
  global" }
- { name: F_func, type: callable | null, required: false, desc: " $F(t) \rightarrow$  ndarray
  global" }
- { name: lumping, type: str, required: false, desc: "default 'lumped'; 'consistent'
  rechazado" }
- { name: divergence_threshold, type: float, required: false, desc: "default 1e8;
  multiplicador sobre escala inicial para abortar" }

requirements:
- "Materiales con `density` declarada (ADR 0008)."
```

- "Todos los elementos del modelo deben soportar `compute_mass_matrix(lumping='lumped')` (ADR 0009 fase 2)."

- "Frame3D con eje oblicuo a globales NO soportado (M_{red} no es diagonal pura)."

```

conventions:
units:      "heredadas del modelo (ADR 0008)."
```

stability: "Condicional $-\Delta t < 2/\omega_{max}$ (CFL). Para barras: $\Delta t < L_{el}/c_p$. Sin power iteration en esta fase - responsabilidad del usuario calcular o estimar Δt_{crit} ."

```

out_of_scope:
- "Estimación automática de  $\Delta t_{crit}$  (power iteration sobre  $M \cdot K$ ) - diferida."
- "Paso adaptativo  $\Delta(t)$  variable - diferido."
- "MPC (restricciones lineales multipunto) - diferido."
- "Frame3D con eje oblicuo (rechazado por arquitectura del lumping)."
```

```

acceptance:
verification:
- name: free_vibration_1dof_undamped
  setup: "Truss2D 1 elemento con  $E=25$ ,  $\rho=2$ ,  $A=1$ ,  $L=1$  (lumped).  $u=1$ ,  $v=0$ .  $\Delta t = T/200$ ."
  expect: " $u(t) = \cos(t)$  con  $\omega=5$  rad/s, error < 2%"
  tol_rel: 0.02
- name: step_response_1dof
  setup: "Mismo modelo con  $F=50 \cdot H(t)$ "
  expect: " $u(t) = (F/K)(1 - \cos(t))$ , error < 2% del régimen permanente"
  tol_rel: 0.02
- name: matches_newmark_at_small_dt
  setup: "Mismo oscilador, Newmark  $\gamma=1/4$   $\beta=1/2$  con masa lumped vs CentralDiff con  $\Delta t = T/500$ "
  expect: " $u_{Newmark} - u_{CD}$  máximo < 1%"
  tol_rel: 0.01
specific:
- name: cfl_violation_diverges
  setup: " $\Delta t = 3 \cdot \Delta t_{crit}$ "
  expect: "RuntimeError con mensaje CFL informativo"
  tol_rel: 0
- name: cfl_threshold_matches_2_over_omega_max_1dof
```

```

setup: "Oscilador 1-GDL (=5, Δt_crit=2/√0.4): integrar con Δt = 0.95·Δt_crit vs
      1.05·Δt_crit"
expect: "0.95·Δt_crit produce trayectoria acotada (30 ciclos); 1.05·Δt_crit lanza
      RuntimeError CFL"
tol_rel: 0
- name: cfl_threshold_matches_2_over_omega_max_2dof
  setup: |
    Cadena 2-DOF (3 trusses E=A=L=1, m=1, lumped) con K=[[2,-1],[-1,2]], M=I →
    autovalores cerrados  $\omega^2$  {1, 3},  $\omega_{max}=\sqrt{3}$ ,  $\Delta t_{crit}=\sqrt{2/3}$ . Estado inicial
    excita modo antisimétrico (1, -1) - puro  $\omega_{max}$ .
  expect: |
    (1) Δt = 0.95·Δt_crit estable.
    (2) Δt = 1.05·Δt_crit lanza RuntimeError CFL.
    (3) Δt = Δt_crit/10 reproduce u(t) = (cos(√3·t), -cos(√3·t)) con error < 2%.
  tol_rel: 0.02
  ref: "tests/test_central_difference.py::TestCentralDifferenceCFLAnalytic"
- name: linear_mode_matches_nonlinear_mode_for_linear_material
  setup: "Mismo problema con material lineal puro, solver con nonlinear=False vs
      nonlinear=True"
  expect: "Coincidencia exacta a precisión de máquina"
  tol_rel: 1.0e-9
- name: consistent_lumping_rejected
  setup: "Construir solver con lumping='consistent'"
  expect: "ValueError"
  tol_rel: 0
- name: frame3d_oblique_rejected
  setup: "Frame3D con eje en dirección (1,1,1)/√3 e Iy Iz"
  expect: "ValueError 'no es estrictamente diagonal'"
  tol_rel: 0

references:
- "Belytschko, T., Liu, W. K., Moran, B. (2014). *Nonlinear Finite Elements for
  Continua and Structures*, §6.2."
- "Hughes, T. J. R. (2000). *The Finite Element Method*, §9.1."
- "Crisfield, M. A. (1997). *Non-Linear Finite Element Analysis of Solids and
  Structures*, vol. 2, §24.6."
- "Bathe, K.-J. (2014). *Finite Element Procedures*, §9.2 (estabilidad y CFL)."
```

12.5 Implementación

- Archivo: solidum/math/solvers/central_difference.py
- Clase: CentralDifferenceSolver (registrada en SolverRegistry)
- Atributo de despacho: PIPELINE_KIND = "transient" — run_yaml despacha a run_transient por el mecanismo declarativo de la regla C (aplicada 2026-05-18 al introducir este solver, primer caso fuera de la jerarquía de Newmark)
- Tests: tests/test_central_difference.py (10 tests verdes)

Notas de traducción: ninguna — la convención de signos del proyecto (Reglas §5) ya es la stress-resultant en el interior, sin reinterpretación específica de central differences.

12.6 Diálogo

- *2026-05-18*: regla C aplicada al introducir este solver. Antes de él, `run_yaml::isinstance(NewmarkSolver)` capturaba HHT- α por subclasificación; `CentralDifference` no podía heredar de `NewmarkSolver` (flujo radicalmente distinto, sin sistema lineal en cada paso, estabilidad condicional). En lugar de añadir una tercera rama `isinstance`, se refactorizó el dispatch a `PIPELINE_KIND` declarativo — futuros solvers no clásicos (`Harmonic`, `ResponseSpectrum`, ...) no requieren tocar `entry.py`.

12.7 HHTSolver

*Variante de NewmarkSolver según la regla de variantes (Reglas.md §4). Documenta **solo lo que cambia** respecto al padre. Implementa el integrador HHT- α (1977) para introducir ****amortiguamiento numérico controlado**** en altas frecuencias preservando segundo orden de precisión y estabilidad incondicional. Reusa todas las decisiones arquitecturales del ADR 0009 — sin ADR nuevo.*

12.8 Qué cambia respecto a NewmarkSolver

NewmarkSolver integra $\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \mathbf{F}(\mathbf{t})$ evaluando todas las fuerzas en el instante final $\mathbf{t}_{\{n+1\}}$ del paso. Con la elección $\beta=1/4$, $\gamma=1/2$ (regla trapezoidal, default) el integrador es **no disipativo**: los modos de alta frecuencia espurios introducidos por la discretización persisten indefinidamente en la respuesta y pueden enmascarar la señal de interés.

HHT- α modifica la ecuación de equilibrio temporal para evaluar las ****fuerzas viscosas, elásticas y externas**** en un instante intermedio $\mathbf{t}_{\{n+1-\alpha\}}$, mientras la **fuerza inercial** se mantiene en $\mathbf{t}_{\{n+1\}}$. Esto introduce disipación numérica controlada por un único parámetro escalar $\alpha \in [-1/3, 0]$, sin sacrificar la precisión de segundo orden global ni la estabilidad incondicional. Con $\alpha = 0$ se recupera Newmark trapezoidal.

12.8.1 Ecuación de movimiento HHT- α

$$\mathbf{M}\ddot{\mathbf{u}}_{n+1} + (1 + \alpha)\mathbf{C}\dot{\mathbf{u}}_{n+1} - \alpha\mathbf{C}\dot{\mathbf{u}}_n + (1 + \alpha)\mathbf{K}\mathbf{u}_{n+1} - \alpha\mathbf{K}\mathbf{u}_n = (1 + \alpha)\mathbf{F}(t_{n+1}) - \alpha\mathbf{F}(t_n)$$

Para preservar el orden 2 y la estabilidad incondicional, los parámetros Newmark se derivan automáticamente del α :

$$\beta = \frac{(1 - \alpha)^2}{4}, \quad \gamma = \frac{1 - 2\alpha}{2}$$

El usuario pasa solo **alpha**; β y γ se autoderivan salvo override explícito.

12.8.2 Sistema efectivo

Sustituyendo los predictores y correctores Newmark estándar (idénticos a la fase 3 con los β , γ autoderivados), el sistema efectivo a resolver en cada paso es:

$$\mathbf{A}_{\text{eff}}\ddot{\mathbf{u}}_{n+1} = \mathbf{b}$$

con

$$\mathbf{A}_{\text{eff}} = \mathbf{M} + (1 + \alpha)\gamma\Delta t\mathbf{C} + (1 + \alpha)\beta\Delta t^2\mathbf{K}$$

$$\mathbf{b} = (1 + \alpha)\mathbf{F}_{n+1} - \alpha\mathbf{F}_n - \mathbf{F}_{\text{dir}} - (1 + \alpha)\mathbf{C}\tilde{\dot{\mathbf{u}}} + \alpha\mathbf{C}\dot{\mathbf{u}}_n - (1 + \alpha)\mathbf{K}\tilde{\mathbf{u}} + \alpha\mathbf{K}\mathbf{u}_n$$

$\mathbf{A}_{\text{eff}}$ es constante (depende solo de $\mathbf{M}$, $\mathbf{C}$, $\mathbf{K}$ y de los coeficientes), por lo que se **factoriza una sola vez** al inicio del análisis (igual que Newmark). Cada paso temporal es una resolución triangular barata.

12.8.3 Disipación numérica controlada

El radio espectral del operador de amplificación en alta frecuencia ($\Delta t \rightarrow \infty$) es

$$\rho_{\infty} = \frac{1 + \alpha}{1 - \alpha}$$

Para $\alpha = 0$: $\rho_{\infty} = 1$ (sin disipación, persistencia indefinida del modo). Para $\alpha = -0.05$: $\rho_{\infty} \approx 0.905$ (disipación ligera, modo decae 9.5 % por periodo de alta frecuencia). Para $\alpha = -1/3$ (límite teórico): $\rho_{\infty} = 0.5$ (disipación máxima manteniendo orden 2).

La curva de disipación es **selectiva**: amortigua significativamente solo los modos con $\omega \Delta t > \pi$, dejando casi intactos los modos de interés con $\omega \Delta t \ll 1$. Es lo que distingue a HHT- α de simplemente subir el γ Newmark (que disipa en *todas* las frecuencias por igual y baja el orden a 1).

12.8.4 Recuperación del comportamiento Newmark

Con $\alpha = 0$, los coeficientes auto-derivados quedan $\beta = 1/4$, $\gamma = 1/2$ y la ecuación de movimiento HHT colapsa exactamente a la de Newmark trapezoidal. `HHTSolver(alpha=0.0)` reproduce **bit a bit** los resultados de `NewmarkSolver` con `beta=0.25`, `gamma=0.5`. Esto se valida en los acceptance.

12.8.5 Variante no lineal

`NewtonHHTSolver` (subclase de `NewtonNewmarkSolver`) aplica el mismo cambio de la ecuación de movimiento al residuo dinámico. El Newton interno y su jacobiano se ven afectados análogamente:

$$\mathbf{R}(\ddot{\mathbf{u}}_{n+1}) = (1 + \alpha) \mathbf{F}_{n+1} - \alpha \mathbf{F}_n - [(1 + \alpha) \mathbf{F}_{\text{int}}(\mathbf{u}_{n+1}) - \alpha \mathbf{F}_{\text{int}}(\mathbf{u}_n)] - [(1 + \alpha) \mathbf{C} \dot{\mathbf{u}}_{n+1} - \alpha \mathbf{C} \dot{\mathbf{u}}_n] - \mathbf{M} \ddot{\mathbf{u}}_{n+1}$$

$$\mathbf{J} = \mathbf{M} + (1 + \alpha) \gamma \Delta t \mathbf{C} + (1 + \alpha) \beta \Delta t^2 \mathbf{K}_t$$

Todo lo demás (criterio de convergencia, line search opt-in del ADR 0011, telemetría tipada de divergencia, commit/rollback de estado, amortiguamiento Rayleigh constante en el tiempo) se hereda de `NewtonNewmarkSolver` sin cambios.

12.9 Contrato de implementación

```
name: HHTSolver
kind: solver
status: validated
extends: NewmarkSolver          # variante según Reglas §4

interface:
  type_yaml: HHTSolver
  pipeline_kind: transient      # mismo que NewmarkSolver

parameters_extra_to_NewmarkSolver:
  - { name: alpha, type: float, required: false, default: -0.05,
      desc: "Parámetro HHT en [-1/3, 0]. =0 recupera Newmark trapezoidal.
            =-0.05 (default) es disipación ligera estándar; =-1/3 disipación máxima."
    }

parameters_overridden_with_autoderivation:
```

```

- { name: beta,    type: float, required: false, default: "(1-alpha)^2/4",
  desc: "Auto-derivado del alpha si no se pasa; override explícito posible
        (no recomendado salvo experimentación)."}
- { name: gamma,  type: float, required: false, default: "(1-2*alpha)/2",
  desc: "Auto-derivado del alpha si no se pasa." }

parameters_inherited:
# Idénticos a NewmarkSolver: t_end, dt, rayleigh, u0, u0_dot, F_func,
# linear_algebra, lumping. Ver docs/specs/NewmarkSolver.md.

acceptance:
  recovery:
    - name: alpha_zero_matches_NewmarkSolver
      setup: "Oscilador 1 GDL con HHTSolver(alpha=0.0) y NewmarkSolver(beta=0.25, gamma
        =0.5)"
      expect: "u_history coincide bit-a-bit (atol 1e-10)"

  damping:
    - name: high_frequency_dissipation
      setup: "Dos osciladores acoplados con  $\omega_1=1$  rad/s y  $\omega_2=100$  rad/s; condición inicial
        excita ambos modos"
      expect: "HHTSolver(alpha=-0.1) atenúa el modo alto en >50% tras 5 periodos del
        modo bajo,
        manteniendo amplitud del modo bajo dentro de 5% del valor inicial"

    - name: spectral_radius_at_high_frequency_limit
      setup: " $\Delta t$  grande (>10 periodos del modo); HHTSolver con varios  $\omega$ "
      expect: " $\omega_-$  observado =  $(1+)/(1-)$  dentro de tolerancia 5%"

    - name: amplification_matrix_spectral_radius_analytic
      setup: "Oscilador 1-GDL con HHTSolver ( $\{-0.05, -0.10, -1/3\}$ );  $\Delta t$  tal que  $\Omega = -\Delta t = 2$ "
      expect: |
        Razón de contracción por paso medida en la simulación
        coincide con  $\max |(A)|$ , donde A es la matriz de amplificación 3x3 HHT -
        (Hughes 1987 §9.3, Bathe 2014 §9.5.4):  $D = 1 + (1+)\Omega^2$ ,  $A \Omega[0,0] = (1+^2)/D$ , ...
        Tolerancia 1%.
      ref: "tests/test_hht.py::TestHHTNumericalDampingAnalytic"

  stability:
    - name: unconditional_stability_large_dt
      setup: "Sistema lineal con  $\Delta t = 100 \cdot T$  (T periodo natural), 10 pasos"
      expect: "u_history acotada (no diverge) para todo  $\omega \in [-1/3, 0]$ "

  nonlinear:
    - name: oscilador_plastico_con_disipacion_HHT
      setup: "Truss elastoplástico 1 GDL, vibración libre desde u_0 más allá del yield"
      expect: "NewtonHHTSolver(alpha=-0.1) reduce oscilación espuria post-yielding
        manteniendo decay físico de la disipación plástica"

references:
- "Hilber, H. M., Hughes, T. J. R. & Taylor, R. L. (1977). Improved numerical
  dissipation for time integration algorithms in structural dynamics.
  Earthquake Eng. Struct. Dyn. 5(3), 283-292."
- "Hughes, T.J.R. (2000). The Finite Element Method, §9.3 (HHT - y generalized -)."
- "Chopra, A. K. (2017). Dynamics of Structures, §15.4 (integradores con disipación
  numérica)."
- "ADR 0009 $Hoja de ruta - fila 3 (Newmark / HHT -)."
- "Spec padre: [docs/specs/NewmarkSolver.md](NewmarkSolver.md)."

```

```
- "Spec hermana: [docs/specs/NewtonNewmarkSolver.md](NewtonNewmarkSolver.md) - padre
  del NewtonHHTSolver."
```

12.10 Implementación

- **Archivo:** `solidum/math/solvers/newmark.py` — junto a `NewmarkSolver` y `NewtonNewmarkSolver` (lógicamente cercanos; comparten predictores Newmark, reducción Dirichlet y amortiguamiento Rayleigh).
- **Clases:**
- `HHTSolver(NewmarkSolver)` — variante lineal con disipación numérica.
- `NewtonHHTSolver(NewtonNewmarkSolver)` — variante no lineal con Newton interno + disipación numérica.

Ambas registradas con `@SolverRegistry.register`.

- **Auto-derivación de β , γ :** en `__init__`, si el usuario no pasa β o γ explícitos, se computan desde α antes de invocar `super().__init__`. Esto asegura que el padre vea la β , γ ya correctas y la mayor parte del flujo herede sin cambios.
- **Solve overrideado:** `solve()` se reescribe en ambas subclases para usar las ecuaciones HHT- α (ver §"Sistema efectivo" y §"Variante no lineal" arriba). El estado anterior `F_int_n`, `F_n`, `u_n` necesario para el término $-\alpha \cdot X_n$ se mantiene en variables locales del bucle.
- **Despacho YAML:** `solver: type: HHTSolver` (o `NewtonHHTSolver`). Como `HHTSolver` hereda de `NewmarkSolver` y `NewtonHHTSolver` hereda de `NewtonNewmarkSolver`, el `isinstance` en `entry.run_yaml` los detecta como `transient` automáticamente. **No dispara la regla C arquitectural** (que esperaría una rama no-clásica que NO sea subclase de las existentes).
- **Tests:** `tests/test_hht.py` nuevo con los acceptance arriba.

12.11 Diálogo

- **2026-05-18** · Spec creada como **variante** de `NewmarkSolver` (y `NewtonHHTSolver` como variante de `NewtonNewmarkSolver`) aplicando la regla §4. Sin ADR nuevo: reusa la decisión del ADR 0009 fila 3 ("Transitorio lineal Newmark / HHT- α "). Sin entrada propia en el manual estructural — visible al usuario solo a través del `type` YAML y el parámetro `alpha`.
- **2026-05-18** · Decisión sobre el default de `alpha`: **-0.05** (disipación ligera estándar) en lugar de 0 (sin disipación). Razones físicas: (i) si el usuario pide `HHTSolver`, está pidiendo disipación — default $\alpha=0$ lo convierte en un alias trivial de `NewmarkSolver`, ruido en el catálogo; (ii) -0.05 es el valor canónico de Hilber 1977 (artículo original) y de los códigos de referencia (Abaqus, OpenSees) por ser balance entre disipación útil (>9% por ciclo en altas frecuencias) y precisión preservada (<0.5% en modos de interés); (iii) no introduce regresiones — `HHTSolver` es un solver nuevo, nadie lo usa hoy. Diferente del default `line_search=False` del ADR 0011, que era *no* poder romper código existente; aquí no hay código existente que romper.

- **2026-05-18** · Decisión sobre auto-derivación de β, γ : por defecto sí, desde **alpha**. Override explícito posible pero no recomendado (combinaciones α, β, γ arbitrarias pueden perder estabilidad incondicional u orden 2). El parámetro de la API es **el** α , no la terna; β, γ son consecuencia matemática de α salvo experimentación expresa. Esto simplifica el uso típico y centraliza la responsabilidad: si el usuario solo conoce α , el solver hace el resto.

12.12 HarmonicSolver

*Spec de **solver nuevo**. Resuelve $(-\omega^2 \mathbf{M} + i\omega \mathbf{C} + \mathbf{K}) \cdot \hat{\mathbf{u}} = \mathbf{F}$ para un barrido de frecuencias y devuelve la amplitud compleja $\hat{\mathbf{u}}(\omega)$. Análisis lineal en frecuencia, sin transitorio.*

12.13 Especificación física

12.13.1 0. Descripción general

Análisis **lineal** de respuesta forzada armónica en estado estacionario. La excitación $\mathbf{F}(\mathbf{t}) = \text{Re}\{\mathbf{F} \cdot e^{i\omega \mathbf{t}}\}$ produce una respuesta $\mathbf{u}(\mathbf{t}) = \text{Re}\{\hat{\mathbf{u}}(\omega) \cdot e^{i\omega \mathbf{t}}\}$ también armónica con la misma frecuencia. El solver calcula $\hat{\mathbf{u}}(\omega)$ para un barrido de frecuencias. Útil para:

- Funciones de transferencia (FRFs) entrada→salida.
- Diagnóstico de resonancias previo a análisis transitorio caro.
- Respuesta forzada en estructuras con cargas periódicas (maquinaria rotante, sismo armónico, viento turbulento descompuesto en armónicos).

Hipótesis cinemáticas y material: lineales ($\mathbf{K}$ y $\mathbf{M}$ constantes). No se modelan grandes desplazamientos, plasticidad ni daño — quien quiera no linealidad debe usar **NewtonNewmarkSolver** con excitación armónica.

12.13.2 1. Ecuación de equilibrio resuelta

$$(-\omega^2 \mathbf{M} + i\omega \mathbf{C} + \mathbf{K}) \hat{\mathbf{u}}(\omega) = \hat{\mathbf{F}}$$

para cada ω del barrido. $\mathbf{C} = \alpha \cdot \mathbf{M} + \beta \cdot \mathbf{K}$ Rayleigh.

12.13.3 2. Condiciones iniciales / de contorno

- Apoyos Dirichlet constantes (eliminación directa, ADR 0004).
- $\mathbf{F} \in \hat{\text{ndof}}$. Real puro si la carga lleva fase cero; complejo si lleva fase.
- **No** hay condiciones iniciales — el estado estacionario es independiente de ellas (el transitorio se descarta por definición).
- MPC no soportado en esta fase.

12.13.4 3. Salidas físicas

HarmonicResult (inmutable):

- `omega[n_omega]` frecuencias evaluadas (rad/s).
- `frequencies_hz` propiedad derivada.
- `u_complex[ndof, n_omega]` amplitud compleja del desplazamiento.
- `F_complex[ndof]` amplitud compleja de la carga (informativa).
- Métodos auxiliares: `.amplitude()` → $|\hat{\mathbf{u}}|$, `.phase()` → $\arg(\hat{\mathbf{u}})$.

12.14 Formulación numérica

12.14.1 4. Esquema operativo

```

Ensamblar K, M.
C = .M + .K (Rayleigh).
Reducir por Dirichlet: K_red, M_red, C_red, F_red.
Para cada en el barrido:
    Z () = -2.M_red + i.C_red + K_red [matriz compleja]
    û_red () = Z ()-1 . F_red [spsolve complejo]
    û () = T . û_red () [expand a globales]

```

12.14.2 5. Predictor / corrector

No aplica — un solve directo por frecuencia.

12.14.3 6. Criterio de convergencia

No aplica — esquema no iterativo. La precisión depende del backend algebraico (`scipy.sparse.linalg.spsolve` LU complejo).

12.14.4 7. Imposición de Dirichlet y MPC

Eliminación directa idéntica a `NewmarkSolver`. En DOFs prescritos $\hat{\mathbf{u}} = 0$ (el apoyo no oscila). MPC fuera de alcance.

12.14.5 8. Backend algebraico

Factorización LU compleja por frecuencia vía `scipy.sparse.linalg.spsolve(Z.tocsc(), F_red)`. **No hay caché:** Z cambia con ω . Coste dominante del análisis es ‘ $O(n_{\text{omega}} \cdot \text{coste_factorizacion}(Z))$ ’.

Posible optimización futura (fuera de esta fase): superposición modal — ortogonalizar la respuesta sobre los primeros `n_modes` modos del problema generalizado, lo que reduce el barrido a `n_omega` operaciones escalares. Justifica el dispositivo cuando `n_modes` $\ll$ `n_dof` y el operador es simétrico.

12.14.6 9. Adaptatividad / control de paso

Espaciado del barrido controlado por el usuario:

- `scale="linear": np.linspace(omega_min, omega_max, n_omega).`
- `scale="log": np.geomspace(omega_min, omega_max, n_omega) —`

apropiado para FRFs sobre rangos de varias décadas.

- Vector explícito vía `omega=...` — usuarios con criterios específicos (densificar alrededor de resonancias conocidas, etc.).
- No hay refinamiento adaptativo de la malla de frecuencia.

12.14.7 10. Caveats numéricos

- **Resonancia exacta sin amortiguamiento:** $Z(\omega_n)$ es singular —

`spsolve` puede fallar o devolver valores enormes. Mitigación: evitar ω coincidente con un modo, o añadir amortiguamiento Rayleigh.

- **Mal condicionamiento cerca de resonancia:** la factorización LU

pierde precisión a medida que Z se aproxima a la singularidad. Es intrínseco al método directo; superposición modal es la alternativa estable.

- **Cargas con fase:** $\mathbf{F}$ compleja requiere construcción desde código

Python; YAML no soporta tipos complejos nativos. Si el usuario YAML necesita fase, debe construir el dominio programáticamente.

- **No analiza estabilidad** del estado estacionario (no aplica al

problema lineal — el estado estacionario siempre existe y es único para $\mathbf{C}$ definida positiva y $\mathbf{K}$ regular).

12.15 Contrato de implementación

```
name: HarmonicSolver
kind: solver
status: validated

interface:
  yaml_type: HarmonicSolver
  output: HarmonicResult

parameters:
  - { name: omega, type: array-like | null, required: false, desc: "vector explícito ( rad/s); excluye omega_min/omega_max/n_omega/scale" }
  - { name: omega_min, type: float, required: false, desc: "límite inferior del barrido (rad/s)" }
  - { name: omega_max, type: float, required: false, desc: "límite superior del barrido (rad/s)" }
  - { name: n_omega, type: int, required: false, desc: "número de puntos del barrido, default 100" }
  - { name: scale, type: str, required: false, desc: "'linear' (default) o 'log'" }
  - { name: F_amplitude, type: ndarray | null, required: false, desc: "amplitud compleja shape (ndof,); default ceros; YAML inyecta desde point_loads como real" }
  - { name: rayleigh, type: dict | null, required: false, desc: "C = .M + .K; sin amortiguamiento pico singular en resonancia" }
  - { name: lumping, type: str, required: false, desc: "'consistent' (default) o 'lumped'" }

requirements:
  - "Materiales con `density` declarada (ADR 0008)."
```

```
  - "Problema lineal - `K` y `M` constantes en el barrido."
```

```
conventions:
  units: "heredadas del modelo (ADR 0008). en rad/s - coherente con frecuencias de ModalSolver."
```



```

stability: "Lineal en frecuencia - estable para C definida positiva. Mal condicionado
           en → _n sin amortiguamiento."

out_of_scope:
- "Superposición modal - diferida; el barrido directo es más simple y suficiente para
  los tamaños actuales del catálogo."
- "MPC (restricciones lineales multipunto) - diferido."
- "No linealidad (geométrica o material) - usar NewtonNewmarkSolver con excitación
  armónica."
- "Cargas con fase declaradas desde YAML - usar construcción desde código."

acceptance:
  verification:
    - name: transfer_function_no_damping
      setup: "1 GDL con K=25, M=1; barrido en lejos de resonancia"
      expect: "û_real() = ^F/(K - ^2M); û_imag 0"
      tol_rel: 1.0e-10
    - name: transfer_function_rayleigh_damping
      setup: "1 GDL con -mass Rayleigh"
      expect: "û = ^F/(K - ^2M + ic) complejo"
      tol_rel: 1.0e-9
  specific:
    - name: resonance_peak_location
      setup: "1 GDL con =2% Rayleigh; barrido fino alrededor de _n"
      expect: "pico en = _n·√(1 - ^2)"
      tol_rel: 0.01
    - name: matches_newmark_steady_state
      setup: "Newmark con F(t)=Fcos(t), suficiente t_end para descartar transitorio
            (8)"
      expect: "amplitud Newmark estacionaria = |û_harmonic|"
      tol_rel: 0.02
    - name: log_sweep_geometric_spacing
      setup: "scale='log', omega_min=1, omega_max=1000, n_omega=4"
      expect: "omega = geomspace(1, 1000, 4) = [1, 10, 100, 1000]"
      tol_rel: 1.0e-12

references:
- "Bathe, K.-J. (2014). *Finite Element Procedures*, §9.5."
- "Clough, R. W., Penzien, J. (1993). *Dynamics of Structures*, §4.6."
- "Chopra, A. K. (2017). *Dynamics of Structures*, cap. 4."

```

12.16 Implementación

- Archivo: solidum/math/solvers/harmonic.py
- Clase: HarmonicSolver (registrada en SolverRegistry)
- Atributo de despacho: PIPELINE_KIND = "harmonic" → solidum.entry.run_harmonic → HarmonicResult.
- Tests: tests/test_harmonic.py (11 tests verdes).
- Resultado: HarmonicResult — dataclass inmutable con omega, u_complex, F_complex, propiedades frequencies_hz, n_omega, métodos amplitude() y phase().

12.17 Diálogo

- *2026-05-18*: nuevo `PIPELINE_KIND = "harmonic"` añadido al despacho declarativo de `run_yaml` (regla C, ya aplicada en D2). Tercer valor del literal junto a `"modal"`, `"transient"`, `"static"`. El parser YAML inyecta automáticamente las `point_loads` del bloque estándar como `F_amplitude` real cuando el usuario no la especifica explícitamente — convención coherente con el uso de `external_forces` en análisis estáticos.

12.18 ModalSolver

Orden de trabajo. La especificación física y la formulación numérica son patrimonio del usuario (revisa con detalle). La IA implementa y rellena las secciones marcadas.

12.19 Especificación física

12.19.1 0. Descripción general

Análisis modal lineal de una estructura discretizada por FEM: cálculo de frecuencias naturales y modos de vibración no amortiguados. Hipótesis: linealidad geométrica y material, amortiguamiento nulo ($\mathbf{C} = \mathbf{0}$), masa constante en el tiempo (lagrangeano total).

12.19.2 1. Problema físico

Vibración libre no amortiguada del sistema discreto:

$$\mathbf{M} \ddot{\mathbf{u}}(t) + \mathbf{K} \mathbf{u}(t) = \mathbf{0}.$$

Solución armónica $\mathbf{u}(t) = \boldsymbol{\phi} e^{i\omega t}$. Sustituyendo:

$$(\mathbf{K} - \omega^2 \mathbf{M}) \boldsymbol{\phi} = \mathbf{0}.$$

12.19.3 2. Problema de autovalor generalizado

Solución no trivial requiere $\det(\mathbf{K} - \omega^2 \mathbf{M}) = 0$. Equivalente:

$$\mathbf{K} \boldsymbol{\phi}_n = \omega_n^2 \mathbf{M} \boldsymbol{\phi}_n, \quad n = 1, \dots, N_{\text{dof}}.$$

Autovalores $\lambda_n = \omega_n^2 \geq 0$; autovectores $\boldsymbol{\phi}_n$ son los modos de vibración.

12.19.4 3. Propiedades del par $(\mathbf{K}, \mathbf{M})$

- $\mathbf{K}$ simétrica, positiva (semi)definida tras Dirichlet.
- $\mathbf{M}$ simétrica, **positiva definida** estrictamente para masa consistente (toda fila/columna tiene aporte de algún elemento con $\rho > 0$).
- Espectro real no negativo: $\omega_n \in \mathbb{R}_{\geq 0}$.

12.19.5 4. Salidas

- Frecuencias naturales ω_n (rad/s), $f_n = \omega_n/(2\pi)$ (Hz), períodos $T_n = 1/f_n$ (s).
- Modos $\boldsymbol{\phi}_n$ M-ortonormales: $\boldsymbol{\phi}_i^\top \mathbf{M} \boldsymbol{\phi}_j = \delta_{ij}$.
- Ordenación ascendente: $\omega_1 \leq \omega_2 \leq \dots \leq \omega_{n_{\text{modos}}}$.

12.20 Formulación numérica

12.20.1 5. Reducción por Dirichlet

Por eliminación directa (ADR 0004) con operador $\mathbf{T} \in \mathbb{R}^{N_{\text{dof}} \times N_{\text{libre}}}$ que selecciona DOFs libres:

$$\mathbf{K}_{\text{red}} = \mathbf{T}^\top \mathbf{K} \mathbf{T}, \quad \mathbf{M}_{\text{red}} = \mathbf{T}^\top \mathbf{M} \mathbf{T}.$$

El problema reducido $\mathbf{K}_{\text{red}} \phi_{\text{red}} = \omega^2 \mathbf{M}_{\text{red}} \phi_{\text{red}}$ se resuelve sobre los DOFs libres. El término $\mathbf{g}_{\text{indep}}$ de restricciones lineales no homogéneas no aplica: los modos son solución del problema homogéneo (los modos de la estructura con apoyos a desplazamiento prescrito no nulo coinciden con los de apoyos nulos; el campo prescrito solo entra en la respuesta estática). Tras resolver, expansión $\phi = \mathbf{T} \phi_{\text{red}}$; en DOFs prescritos $\phi_i = 0$.

12.20.2 6. Algoritmo numérico: Lanczos con shift-invert

ARPACK vía `scipy.sparse.linalg.eigsh`:

$$\text{eigsh}(\mathbf{K}_{\text{red}}, k = n_{\text{modos}}, \mathbf{M} = \mathbf{M}_{\text{red}}, \sigma, \text{which} = \text{'LM'}, \text{tol}).$$

- **Shift-invert** transforma el problema en $(\mathbf{K}_{\text{red}} - \sigma \mathbf{M}_{\text{red}})^{-1} \mathbf{M}_{\text{red}} \phi = \mu \phi$ con $\mu = 1/(\omega^2 - \sigma)$. Buscar `which="LM"` (mayor magnitud de μ) equivale a buscar autovalores cercanos al shift σ .
- $\sigma = 0$ por defecto: las frecuencias más bajas, que son las relevantes en ingeniería estructural.
- Internamente ARPACK factoriza $(\mathbf{K}_{\text{red}} - \sigma \mathbf{M}_{\text{red}})$ una sola vez y la reusa en cada iteración Lanczos.

12.20.3 7. Normalización

`eigsh` con $\mathbf{M}$ explícita devuelve autovectores M-ortonormales sin reescalado adicional. Signo arbitrario (autovector definido módulo signo); convención de signo se documenta solo si emerge como preferencia (no parte del contrato).

12.20.4 8. Post-procesado

- $\omega_n = \sqrt{\lambda_n}$ (clip a cero los autovalores ligeramente negativos por ruido numérico, $|\lambda| < 10^{-14} \lambda_{\text{max}}$).
- $f_n = \omega_n/(2\pi)$, $T_n = 1/f_n$ (con $T_n = \infty$ si $\omega_n = 0$, modo de cuerpo rígido).
- Reordenación ascendente por ω_n .

12.21 Contrato de implementación

```
name: ModalSolver
kind: solver
status: validated          # draft → implemented → validated

interface:
  yaml_type: modal         # solver: { type: modal }
```

```

output: ModalResult      # campos: frecuencias_hz, frecuencias_rad, periods, modes,
                          n_modes

parameters:
- { name: n_modes,      type: int,      required: true,  desc: "Número de modos a calcular"
  }
- { name: sigma,        type: float,    required: false, default: 0.0,      desc: "Shift (
rad2/s2) para shift-invert; 0 → frecuencias más bajas" }
- { name: which,         type: str,      required: false, default: "LM",      desc: "
Estrategia ARPACK: LM | SM | LA | SA | BE" }
- { name: tolerance,     type: float,    required: false, default: 1.0e-9,   desc: "
Tolerancia ARPACK sobre el residuo del autovalor" }
- { name: lumping,       type: str,      required: false, default: "consistent", desc: "
Discretización de masa; solo 'consistent' en fase 1" }
- { name: linear_algebra, type: str,     required: false, default: "auto",   desc: "Backend
para la factorización de (K - M) interna a shift-invert (ADR 0003)" }

requirements:
- "Todos los materiales del modelo declaran `density > 0` (ADR 0008). Densidad cero
  genera M_red singular → eigsh con sigma=0 falla; el solver propaga el error con
  mensaje físico."
- "Al menos un DOF libre tras Dirichlet."
- "n_modes < N_libre (ARPACK exige k < n)."
```

conventions:

```

units:      "heredadas del modelo (ADR 0008); en rad/s, f en Hz, T en s
            consistentes con kg/N/m o equivalentes."
frecuencias: "y f expuestos simultáneamente; el usuario consulta cualquiera."
modos:       "M-ortonormales  $\Phi$  (  $M \Phi = I$  ); signo arbitrario por modo."
ordering:    "ascendente en :      ...      _n_modes."
rigid_body:  "modos de cuerpo rígido aparecen con 0; Tm= se reporta sin error."
```

out_of_scope:

- "Modos complejos con amortiguamiento no proporcional."
- "Análisis modal sobre estado deformado (modos de pequeña amplitud alrededor de configuración prestressed) - diferido."
- "Masa lumped y factores de participación / masas efectivas - implementados (fase 2 y fase 7 del ADR 0009, cerradas 2026-05-18); accesibles vía `lumping='lumped'` y `ResponseSpectrumSolver` respectivamente."

acceptance:

```

verification:
- name: barra_axial_empotrada_libre
  setup: |
    Barra 1D elástica empotrada en x=0, libre en x=L. Discretización con
    N=20 elementos Truss2D alineados con el eje x. E=210e9 Pa, ρ=7850 kg/m3,
    A=1e-4 m2, L=1.0 m.
  expect: |
    Frecuencias propias analíticas (ondas axiales en barra empotrada-libre):
    _n = ((2n-1) / (2L)) · sqrt(E/ρ),    n = 1, 2, 3, ...
    Los primeros 3 modos calculados coinciden con la solución analítica.
  tol_rel: 0.01
- name: viga_bernoulli_simplemente_apoyada
  setup: |
    Viga Bernoulli-Euler simplemente apoyada en sus dos extremos.
    Discretización con N=20 elementos Frame2DEuler alineados con el eje x.
    E=210e9 Pa, ρ=7850 kg/m3, A=1e-4 m2, I=8.33e-10 m4, L=1.0 m.
  expect: |
    Frecuencias propias analíticas (flexión transversal):
```

```

    _n = (n/L)2 · sqrt(E·I / (·A)),    n = 1, 2, 3, ...
    Los primeros 3 modos coinciden con la solución analítica.
    tol_rel: 0.02
specific:
- name: ortonormalidad_de_masa
  setup: "Cualquier modelo bien formado tras resolver el problema modal."
  expect: "ΦT M Φ = I (n_modos × n_modos)."
  tol_abs: 1.0e-10
- name: ortogonalidad_de_rigidez
  setup: "Mismo modelo."
  expect: "ΦT K Φ = diag(2, ..., 2_n)."
  tol_rel: 1.0e-8
- name: dirichlet_anula_modos_en_prescritos
  setup: "Barra empotrada-libre; nodo x=0 con ux=0."
  expect: "Componente del modo en el DOF prescrito = 0 exacto."
  tol_abs: 1.0e-14
- name: density_faltante_lanza_value_error
  setup: "Material sin density declarada en el modelo."
  expect: "ModalSolver.solve() lanza ValueError listando los materiales afectados (
    mismo patrón que assemble_self_weight, ADR 0008)."

references:
- "Bathe K.-J., Finite Element Procedures (2014), cap. 9 (matriz de masa consistente)
  y cap. 11 (algoritmos de autovalor)."
- "Cook, Malkus & Plesha, Concepts and Applications of Finite Element Analysis, §
  -11.3§11.5."
- "Hughes T. J. R., The Finite Element Method, cap. 7."
- "Wilson E. L., Three-Dimensional Static and Dynamic Analysis of Structures, cap.
  10."
- "Lehoucq, Sorensen, Yang - ARPACK Users' Guide (Lanczos con shift-invert)."
- "ADR 0003 - Capa algebraica (factorización reutilizable de (K - M))."
- "ADR 0008 - Material.density."
- "ADR 0009 - Análisis modal y dinámico (este solver es la fase 1: el análisis modal
  estricto, problema generalizado K· = 2·M·; el análisis dinámico transitorio
  entra en fases 3-7)."
```

12.22 Implementación

- **Archivo:** solidum/math/solvers/modal.py (clase ModalSolver, registrada vía @SolverRegistry.register).
- **Backend algebraico:** solidum/math/linalg/eigen.py (clase EigenSolver envolviendo scipy.sparse.linalg).
- **Tipo de resultado:** ModalResult — dataclass frozen con frequencies_rad, frequencies_hz, periods, modes, n_modos, converged. Expone free_vibration(M, u0, u0_dot, t) para reconstruir analíticamente la respuesta temporal de vibración libre sin amortiguamiento por superposición modal — solución cerrada de $M\ddot{u} + K\cdot u = 0$ proyectada sobre los modos calculados (modos rígidos tratados aparte con evolución lineal $q_n(t) = a_n + b_n \cdot t$).
- **Entrypoint público:** solidum.run_modal para uso programático; solidum.run_yaml despacha automáticamente a run_modal cuando el YAML declara solver: type: ModalSolver.
- **Pipeline:** Assembler.assemble_system() → Assembler.assemble_mass_matrix() → Assembler.reduce(M) → EigenSolver.solve(K_red, M_red, n_modos) → expansión $\Phi = T \cdot \Phi_{\text{red}}$ → ModalResult.
- **Caché:** Assembler cachea M_global con el lumping usado; reusos posteriores del mismo análisis no recomputan.

- **Validación de density:** pre-chequeo agregado en `Assembler.assemble_mass_matrix` con el mismo patrón de ADR 0008 — `assemble_self_weight`.
- **Tests:**
- `tests/test_modal.py` — 13 tests cubriendo:
- **TestModalAxialBar:** barra empotrada-libre, 20 elementos `Truss2D`, frecuencias contra $\omega_n = ((2n-1)\pi/(2L)) \cdot \sqrt{E/\rho}$ (tol 1%).
- **TestModalSimplySupportedBeam:** viga biapoyada, 20 elementos `Frame2DEuler`, frecuencias contra $\omega_n = (n\pi/L)^2 \cdot \sqrt{EI/(\rho A)}$ (tol 2%).
- **TestModalAlgebraicProperties:** ortonormalidad $\Phi^T M \Phi = I$ (atol 1e-10) y ortogonalidad $\Phi^T K \Phi = \text{diag}(\omega^2)$ (rtol 1e-8) en ambos modelos.
- **TestModalSolverContract:** errores agregados (`density` faltante, `run_modal` sin solver ni `n_modes`) y verificación de que `lumping="lumped"` corre tras ADR 0009 fase 2 (cerrada 2026-05-18).
- **TestModalYamlPipeline:** pipeline end-to-end desde YAML.
- **Notas de traducción:**
- El parámetro `linear_algebra` está en el constructor por coherencia con los demás solvers, pero en fase 1 no se usa: ARPACK gestiona internamente la factorización LU de $(K - \sigma M)$ para shift-invert. Cuando se enchufe a la capa algebraica (ADR 0003), se podrá inyectar Cholesky o LDL^T .
- El bloque `convergence:` del YAML (ADR 0007) se omite silenciosamente para `ModalSolver` igual que para `LinearSolver`: no hay iteración Newton ni norma de residuo configurable; la tolerancia es la propia de ARPACK (`tolerance:`).
- Matriz de masa consistente "translacional invariante rotacional" para `Truss*` y `Cable*` ($M = \text{kron}([[2,1], [1,2]] \cdot \rho AL/6, I_{\text{dim}})$). Para frames se ensambla la masa local axial + Hermitiana cúbica y se rota con $T^T \cdot M_{\text{local}} \cdot T$. Para `Frame3D` se incluye masa torsional con momento polar geométrico $J_p = I_y + I_z$ (no la constante de Saint-Venant J , que rige la rigidez). Para sólidos 2D, integración por la cuadratura del propio elemento (`Tri3` usa fórmula analítica exacta porque su cuadratura 1-punto sería insuficiente).
- Inercia rotacional propia de sección ($\rho I \cdot L$) **incluida** en los DOFs rotacionales de `Frame2D` (Euler, Timoshenko, EulerCorot) y `Frame3D` — coherente con Timoshenko y con la rigidez `K_local` que tiene términos rotacionales puros (ADR 0009 §1).

12.23 Diálogo

- **2026-05-13** · Validado. Los 13 tests de `tests/test_modal.py` cubren los criterios de `acceptance` (verificación analítica + propiedades algebraicas + contrato de errores + pipeline YAML) y pasan con las tolerancias declaradas. Promovido `status: validated`.

12.24 NewmarkSolver

Orden de trabajo. La especificación física y la formulación numérica son patrimonio del usuario (revisa con detalle). La IA implementa y rellena las secciones marcadas.

12.25 Especificación física

12.25.1 0. Descripción general

Análisis dinámico transitorio **lineal** de una estructura discretizada por FEM: integración temporal directa de la ecuación semidiscreta de movimiento bajo cargas dependientes del tiempo. Hipótesis: linealidad geométrica y material ($\mathbf{K}$ constante), masa $\mathbf{M}$ constante (lagrangeano total), amortiguamiento Rayleigh proporcional $\mathbf{C} = \alpha \cdot \mathbf{M} + \beta \cdot \mathbf{K}$. Apoyos Dirichlet constantes en el tiempo. La no-linealidad (Newton dentro de cada paso) se difiere a la fase 4 del ADR 0009.

12.25.2 1. Ecuación de movimiento semidiscreta

Tras la discretización espacial por FEM, el problema continuo

$$\rho \ddot{\mathbf{u}} - \operatorname{div} \boldsymbol{\sigma} = \mathbf{b}(\mathbf{x}, t)$$

se reduce al sistema acoplado de ODEs:

$$\mathbf{M} \ddot{\mathbf{u}}(t) + \mathbf{C} \dot{\mathbf{u}}(t) + \mathbf{K} \mathbf{u}(t) = \mathbf{F}(t),$$

con condiciones iniciales $\mathbf{u}(0) = \mathbf{u}_0$, $\dot{\mathbf{u}}(0) = \dot{\mathbf{u}}_0$ y condiciones de contorno Dirichlet $\mathbf{u}_s(t) = \mathbf{g}$ constantes.

12.25.3 2. Amortiguamiento Rayleigh

$$\mathbf{C} = \alpha \mathbf{M} + \beta \mathbf{K},$$

con coeficientes calibrados a partir de dos pares modales (ξ_1, ω_1) y (ξ_2, ω_2) :

$$\alpha = \frac{2\omega_1\omega_2(\xi_1\omega_2 - \xi_2\omega_1)}{\omega_2^2 - \omega_1^2}, \quad \beta = \frac{2(\xi_2\omega_2 - \xi_1\omega_1)}{\omega_2^2 - \omega_1^2}.$$

La razón de amortiguamiento del modo n -ésimo resulta $\xi_n = \frac{1}{2}(\alpha/\omega_n + \beta\omega_n)$ — exacta en ω_1, ω_2 y suavemente sobre-amortiguada fuera del rango. El usuario también puede pasar (α, β) directos.

12.25.4 3. Salidas

- $\mathbf{u}(t_k)$, $\dot{\mathbf{u}}(t_k)$, $\ddot{\mathbf{u}}(t_k)$ para cada paso temporal $k = 0, 1, \dots, N_t$.
- Vector de instantes $t_k = k \Delta t$.
- Diagnósticos: α, β Rayleigh efectivos; número de pasos; tiempo de factorización vs tiempo de pasos.

12.26 Formulación numérica

12.26.1 4. Método de Newmark- β (a-form)

Familia paramétrica con (β, γ) . Predictores:

$$\begin{aligned}\tilde{\mathbf{u}}_{n+1} &= \mathbf{u}_n + \Delta t \dot{\mathbf{u}}_n + \frac{\Delta t^2}{2} (1 - 2\beta) \ddot{\mathbf{u}}_n, \\ \tilde{\dot{\mathbf{u}}}_{n+1} &= \dot{\mathbf{u}}_n + \Delta t (1 - \gamma) \ddot{\mathbf{u}}_n.\end{aligned}$$

Sistema efectivo en aceleraciones:

$$(\mathbf{M} + \gamma \Delta t \mathbf{C} + \beta \Delta t^2 \mathbf{K}) \ddot{\mathbf{u}}_{n+1} = \mathbf{F}_{n+1} - \mathbf{C} \tilde{\dot{\mathbf{u}}}_{n+1} - \mathbf{K} \tilde{\mathbf{u}}_{n+1}.$$

Correctores:

$$\mathbf{u}_{n+1} = \tilde{\mathbf{u}}_{n+1} + \beta \Delta t^2 \ddot{\mathbf{u}}_{n+1}, \quad \dot{\mathbf{u}}_{n+1} = \tilde{\dot{\mathbf{u}}}_{n+1} + \gamma \Delta t \ddot{\mathbf{u}}_{n+1}.$$

12.26.2 5. Aceleración inicial

$\ddot{\mathbf{u}}_0$ se calcula consistentemente con la ecuación de movimiento en $t = 0$:

$$\mathbf{M} \ddot{\mathbf{u}}_0 = \mathbf{F}(0) - \mathbf{C} \dot{\mathbf{u}}_0 - \mathbf{K} \mathbf{u}_0.$$

12.26.3 6. Parámetros canónicos

Esquema	β	γ	Propiedades
Average acceleration (default)	1/4	1/2	Incondicionalmente estable, sin amortiguamiento numérico, exactitud $O(\Delta t^2)$
Linear acceleration	1/6	1/2	Condicionally estable ($\Delta t \leq 2\sqrt{3}/\omega_{\max}$), $O(\Delta t^2)$
Fox-Goodwin	1/12	1/2	Condicionally estable, $O(\Delta t^4)$
Diferencias centradas	0	1/2	Explícito; condicionalmente estable ($\Delta t \leq 2/\omega_{\max}$), $O(\Delta t^2)$

12.26.4 7. Imposición de Dirichlet con apoyos constantes

Mismo mecanismo que estática (ADR 0004 fase 1) con operador $\mathbf{T}$ que selecciona DOFs libres y $\mathbf{g}$ que recoge los valores prescritos: $\mathbf{u} = \mathbf{T} \mathbf{u}_{\text{free}} + \mathbf{g}$. Como $\mathbf{g}$ es constante, $\dot{\mathbf{u}} = \mathbf{T} \dot{\mathbf{u}}_{\text{free}}$ y $\ddot{\mathbf{u}} = \mathbf{T} \ddot{\mathbf{u}}_{\text{free}}$. El sistema efectivo se proyecta a DOFs libres:

$$\mathbf{A}_{\text{eff,red}} \ddot{\mathbf{u}}_{\text{free},n+1} = \mathbf{T}^\top (\mathbf{F}_{n+1} - \mathbf{K} \mathbf{g}) - \mathbf{C}_{\text{red}} \tilde{\dot{\mathbf{u}}}_{\text{free}} - \mathbf{K}_{\text{red}} \tilde{\mathbf{u}}_{\text{free}},$$

donde $\mathbf{A}_{\text{eff,red}} = \mathbf{M}_{\text{red}} + \gamma \Delta t \mathbf{C}_{\text{red}} + \beta \Delta t^2 \mathbf{K}_{\text{red}}$. El término $\mathbf{T}^\top \mathbf{K} \mathbf{g}$ se precalcula una vez. La expansión final a globales es $\mathbf{u}_n = \mathbf{T} \mathbf{u}_{\text{free},n} + \mathbf{g}$.

12.26.5 8. Factorización reutilizable

Con Δt constante y problema lineal, $\mathbf{A}_{\text{eff,red}}$ es constante. Se factoriza una vez al inicio (ADR 0003 — **FactorizedSolver**) y cada paso es una resolución triangular barata. Para N_t pasos sobre N_{libre} DOFs: coste $\sim O(\text{factor})_{\text{una vez}} + N_t \cdot O(N_{\text{libre}})_{\text{por paso}}$.

12.27 Contrato de implementación

```

name: NewmarkSolver
kind: solver
status: validated      # draft → implemented → validated

interface:
  yaml_type: NewmarkSolver
  output: TransientResult  # campos: t_history, u_history, udot_history, uddot_history

parameters:
- { name: t_end,      type: float, required: true,      desc: "Tiempo final del análisis (s)" }
- { name: dt,         type: float, required: true,      desc: "Paso temporal constante (s)" }
- { name: beta,       type: float, required: false, default: 0.25, desc: "Parámetro de Newmark (default: average acceleration)" }
- { name: gamma,      type: float, required: false, default: 0.5,  desc: "Parámetro de Newmark" }
- { name: rayleigh,   type: dict,  required: false, default: null, desc: "Amortiguamiento Rayleigh: {alpha, beta} directos o {xi1, omega1, xi2, omega2} para calibración modal" }
- { name: u0,         type: ndarray, required: false, default: null, desc: "Desplazamiento inicial (ndof,); cero por defecto" }
- { name: u0_dot,     type: ndarray, required: false, default: null, desc: "Velocidad inicial (ndof,); cero por defecto" }
- { name: F_func,     type: callable, required: false, default: null, desc: "Callback Python F_func(t) -> ndarray (ndof,); cero por defecto" }
- { name: linear_algebra, type: str, required: false, default: "auto", desc: "Backend para factorizar A_eff (ADR 0003)" }
- { name: lumping,    type: str, required: false, default: "consistent", desc: "Discretización de masa; solo 'consistent' en esta fase" }

requirements:
- "Todos los materiales del modelo declaran `density > 0` (ADR 0008)."
- "K lineal y constante (hipótesis de pequeñas deformaciones, material elástico)."
- "Apoyos Dirichlet constantes en el tiempo."

conventions:
  units: "heredadas del modelo (ADR 0008); t en s, en rad/s, F en N consistente con kg/N/m."
  damping: "Rayleigh proporcional: C = ·M + ·K. Si se pasan (,) ,(,), (,) se derivan automáticamente."
  stability: |
    Default (=1/4, =1/2) es incondicionalmente estable. Para =0 (diferencias centradas) o =1/6 (lineal) el usuario es responsable de que Δt ≤ 2/_max; el solver no comprueba esta cota automáticamente.

out_of_scope:
- "Apoyos dependientes del tiempo (multi-support excitation sísmica) - diferido."
- "Δt adaptativo - diferido."
- "Generalized -, Bossak - - diferidos (variantes adicionales de la familia con amortiguamiento numérico)."
- "No-linealidad y disipación numérica controlada - implementados como variantes en clases hermanas (`NewtonNewmarkSolver`, `HHTSolver`, `NewtonHHTSolver`); fases 4 y 3-bis del ADR 0009, cerradas."

acceptance:

```

```

verification:
- name: oscilador_1gdl_no_amortiguado
  setup: |
    Sistema 1 GDL:  $M=1$ ,  $K=25$  ( $=5$  rad/s),  $C=0$ . Condiciones iniciales
     $u=1$ ,  $\dot{u}=0$ .  $F(t)=0$ .  $\Delta t=T/100$  (100 pasos por período). Integrar
    durante 4 períodos.
  expect: |
     $u(t) = \cos(\cdot t)$  exacto. Error máximo en  $u$  durante el intervalo
     $< 1\%$  ( $=1/4$  tiene error de período  $O(\Delta t^2)$  bajo control con 100
    pasos por período).
  tol_rel: 0.01
- name: oscilador_1gdl_amortiguado_rayleigh
  setup: |
    Sistema 1 GDL:  $M=1$ ,  $K=25$ ,  $C=2 \cdot \cdot \cdot M$  con  $=0.05$  (sub-amortiguado).
     $u=1$ ,  $\dot{u}=0$ ,  $F=0$ .  $\Delta t = T/100$ ,  $t_{\text{end}} = 4 \cdot T$ . Construir  $C$  vía
    Rayleigh con un único par ( $\cdot$ ,  $\cdot$ ).
  expect: |
     $u(t) = e^{-\gamma t} \cdot (\cos(\omega_d t) + (\gamma/\omega_d) \cdot \sin(\omega_d t))$  con
     $\gamma = \cdot \sqrt{1-\zeta^2}$ . Error máximo  $< 2\%$ .
  tol_rel: 0.02
- name: viga_voladizo_carga_subita_modal_vs_newmark
  setup: |
    Voladizo Frame2DEuler (20 elementos) con carga puntual aplicada en
    el extremo en  $t=0$  como step function.  $\Delta t = T/40$  ( $T$  período del
    primer modo),  $t_{\text{end}} = 2 \cdot T$ . Sin amortiguamiento.
  expect: |
    La reconstrucción modal de la respuesta (superposición de primeros
    5 modos forzados por  $F(t)=H(t) \cdot F$  con solución analítica conocida)
    coincide con Newmark a  $< 5\%$ .
  tol_rel: 0.05
- name: orden_de_convergencia
  setup: |
    Oscilador 1 GDL no amortiguado integrado con  $=1/4$ ,  $=1/2$  a  $\Delta$ 
     $t$ ,  $\Delta t/2$ ,  $\Delta t/4$ . Calcular el error máximo en  $u$  contra solución
    analítica.
  expect: |
    El cociente de errores  $e\Delta(t)/e\Delta(t/2)$  tiende a 4 (orden 2 exacto).
  tol_rel: 0.5 # tolerancia sobre el cociente:  $4 \pm 2$ 

specific:
- name: rayleigh_calibration
  setup: "Datos ( $=0.02$ ,  $=10$ ), ( $=0.05$ ,  $=100$ ), calibrar ( $\cdot$ ,  $\cdot$ )."= 2 \cdot \cdot \cdot (\cdot - \cdot) / (\cdot^2 - \cdot^2) \quad 0.303,
     $= 2 \cdot (\cdot - \cdot) / (\cdot^2 - \cdot^2) \quad 9.6e-4$ .
    Y verificar  $(\cdot_n) = (\cdot_n + \cdot_n)/2$  reproduce en  $\cdot$  y en  $\cdot$ .
  tol_rel: 1.0e-10

references:
- "Newmark N. M. (1959), A method of computation for structural dynamics, J. Eng.
  Mech. Div. ASCE."
- "Bathe K.-J., Finite Element Procedures (2014), cap. 9 (matriz de masa) y cap. 9.4
  (Newmark)."A_{\text{eff}})."

```

12.28 Implementación

- **Archivo:** `solidum/math/solvers/newmark.py` (clase `NewmarkSolver`, registrada vía `@SolverRegistry.register`).
- **Amortiguamiento:** `solidum/math/damping.py` (`rayleigh_from_modes`, `rayleigh_xi`).
- **Tipo de resultado:** `TransientResult` — dataclass frozen con `t_history`, `u_history`, `udot_history`, `uddot_history`, `n_steps`, `alpha_rayleigh`, `beta_rayleigh`, `converged`.
- **Entrypoint público:** `solidum.run_transient` para uso programático; `solidum.run_yaml` despacha automáticamente cuando el YAML declara `solver: type: NewmarkSolver`.
- **Pipeline:** `Assembler.assemble_system()` → `Assembler.assemble_mass_matrix()` → $C = \alpha \cdot M + \beta \cdot K$ → reducción simultánea por Dirichlet (operador T) → cálculo de $\ddot{u}_0$ → factorización única de $A_{\text{eff_red}} = M_{\text{red}} + \gamma \Delta t \cdot C_{\text{red}} + \beta \Delta t^2 \cdot K_{\text{red}}$ → bucle de pasos con resolución triangular reutilizada.
- **Tests:**
 - `tests/test_newmark.py` — 14 tests:
 - `TestRayleighCalibration`: (α, β) reproduce ξ objetivo en ω_1, ω_2 ; errores agregados (ω iguales, no positivos).
 - `TestNewmark1DofUndamped`: vibración libre $u(t) = \cos(\omega t)$ a 1 %; estado inicial preservado.
 - `TestNewmark1DofDamped`: respuesta sub-amortiguada $e^{(-\xi \omega t)} \cdot (\cos(\omega_d t) + \dots)$ a 2 %; coeficientes Rayleigh propagados al `TransientResult`.
 - `TestNewmark1DofStepResponse`: $u(t) = (F_0/K)(1 - \cos(\omega t))$ a 0.1 %.
 - `TestNewmarkConvergenceOrder`: cociente de errores al refinar $\Delta t \in T/40, T/80, T/160$ cae en $[3.5, 4.5]$ → orden 2 confirmado.
 - `TestNewmarkContract`: validaciones de constructor y `run_transient`.
 - `TestNewmarkYamlPipeline`: end-to-end desde YAML.
- **Notas de traducción:**
 - **Factorización única:** con Δt constante y problema lineal, $A_{\text{eff_red}}$ es invariante; se factoriza al inicio (`A_solver.factorize(A_eff)`) y cada paso es una sustitución triangular barata (ADR 0003 — `FactorizedSolver`).
 - **reduce_pair vs reduce triple:** en el problema modal `reduce_pair(K, M)` basta porque g no entra en autovalores. En Newmark se manejan tres matrices (K, M, C) y un término constante $F_{\text{dir}} = T^T K \cdot g$ para apoyos no nulos; se hace explícitamente en `NewmarkSolver.solve()` sin un helper unificado (no aporta valor centralizarlo con un solo consumidor).
 - **Aceleración inicial consistente:** $M \cdot \ddot{u}_0 = F(0) - C \cdot \dot{u}_0 - K \cdot u_0$ resuelve un sistema lineal aparte; se usa el dispatcher de la capa algebraica sobre M_{red} (siempre SPD si la masa es consistente y la densidad estrictamente positiva).

- **F_func opcional:** si es `None`, se trata como vector cero (vibración libre). El callback se invoca una vez por paso con `t_{n+1}`; el usuario es responsable de la coherencia temporal (no se reusan valores entre pasos).
- **El bloque `convergence` del `YAML`** (ADR 0007) se omite para `NewmarkSolver` igual que para `LinearSolver` y `ModalSolver`: la fase 3 es lineal y no hay iteración Newton interna; cuando entre la fase 4 (Newton dentro de cada paso), el bloque se reincorporará.

12.29 Diálogo

- **2026-05-13** · Validado. Los 14 tests de `tests/test_newmark.py` cubren `acceptance.verification` (osciladores 1 GDL no amortiguado / amortiguado / step / orden de convergencia) y `acceptance.specific` (calibración Rayleigh). Promovido `status: validated`.

12.30 NewtonHHTSolver

Variante de `NewtonNewmarkSolver` según la regla de variantes (Reglas.md §4). Aplica el esquema temporal HHT- α (Hilber-Hughes-Taylor 1977) al residuo dinámico no lineal: amortiguamiento numérico controlado en altas frecuencias dentro del bucle Newton de cada paso.
>Reusa todas las decisiones arquitecturales del ADR 0009 (esquema temporal, contrato `PIPELINE_KIND`, telemetría tipada, line search opt-in) y de `HHTSolver` (formulación temporal, autoderivación de β/γ).
Sin ADR nuevo. Documenta solo lo que cambia respecto a los dos padres.

12.31 Qué cambia respecto a `NewtonNewmarkSolver`

`NewtonNewmarkSolver` integra $\mathbf{M} \cdot \ddot{\mathbf{u}} + \mathbf{C} \cdot \dot{\mathbf{u}} + \mathbf{F}_{\text{int}}(\mathbf{u}) = \mathbf{F}(\mathbf{t})$ con Newmark- β clásico ($\beta=1/4$, $\gamma=1/2$ default, sin amortiguamiento numérico). Cada paso abre un bucle Newton porque $\mathbf{F}_{\text{int}}(\mathbf{u})$ es no lineal en $\mathbf{u}$ (plasticidad, daño, corotacional, embedded).

`NewtonHHTSolver` reemplaza el residuo Newmark por el residuo HHT- α : las **fuerzas internas, viscosas y externas** se evalúan en un instante intermedio $\mathbf{t}_{\{n+1-\alpha\}}$ mientras la **fuerza inercial** se mantiene en $\mathbf{t}_{\{n+1\}}$. El bucle Newton se ejecuta sobre este residuo modificado.

12.31.1 Residuo dinámico HHT- α no lineal

$$\mathbf{R}(\ddot{\mathbf{u}}_{n+1}) = (1+\alpha) \mathbf{F}_{n+1} - \alpha \mathbf{F}_n - [(1+\alpha) \mathbf{F}_{\text{int}}(\mathbf{u}_{n+1}) - \alpha \mathbf{F}_{\text{int}}(\mathbf{u}_n)] - [(1+\alpha) \mathbf{C} \dot{\mathbf{u}}_{n+1} - \alpha \mathbf{C} \dot{\mathbf{u}}_n] - \mathbf{M} \ddot{\mathbf{u}}_{n+1}$$

El término no lineal $\mathbf{F}_{\text{int}}(\mathbf{u}_{n+1})$ depende del estado actualizado por los correctores Newmark, así que el residuo es implícito en $\ddot{\mathbf{u}}_{n+1}$ y requiere Newton.

12.31.2 Jacobiano efectivo

$$\mathbf{J} = \mathbf{M} + (1+\alpha) \gamma \Delta t \mathbf{C} + (1+\alpha) \beta \Delta t^2 \mathbf{K}_t$$

donde $\mathbf{K}_t = \partial \mathbf{F}_{\text{int}} / \partial \mathbf{u}$ es la tangente material/geométrica reensamblada en cada iteración Newton.

12.31.3 Parámetros heredados de `HHTSolver`

- Rango admisible $\alpha \in [-1/3, 0]$. Valores fuera lanzan `ValueError`.
- β y γ auto-derivados desde α :

$$\beta = \frac{(1-\alpha)^2}{4}, \quad \gamma = \frac{1-2\alpha}{2}$$

- Override explícito de β/γ permitido pero genera `RuntimeWarning`

(combinaciones distintas pueden romper estabilidad incondicional u orden 2).

- Radio espectral en alta frecuencia: $\rho_\infty = (1+\alpha)/(1-\alpha)$.

Disipación selectiva en modos con $\omega \Delta t > \pi$.

12.31.4 Comportamiento que NO cambia

Todo lo heredado de `NewtonNewmarkSolver`:

- Convergencia dual fuerza + desplazamiento (ADR 0007) con autocalibración.
- Line search GLL opt-in (ADR 0011, default `False`).
- Diagnóstico tipado de divergencia (`SolverDivergedError` y subclases).
- Rayleigh con $\mathbf{K}_0$ constante (matriz de rigidez inicial).
- Commit/rollback del estado material por paso.
- Detección de oscilación por ventana móvil.
- Despacho algebraico ADR 0003 con fallback Cholesky→LU.
- Lumping de masa (consistent / lumped) ADR 0009 fase 2.

12.31.5 Recuperación de `NewtonNewmark` con $\alpha = 0$

Con $\alpha = 0$: $\beta = 1/4$, $\gamma = 1/2$, el residuo HHT colapsa al de Newmark estándar. Verificado en tests: `tests/test_dynamic_nonlinear_hht.py::test_alpha_zero_recovers_newton_newmark`.

12.31.6 Recuperación de `HHTSolver` con materiales lineales

Con materiales lineales ($\mathbf{F}_{\text{int}}(\mathbf{u}) = \mathbf{K}\mathbf{u}$), el residuo de Newton se anula en una iteración y el solver reproduce exactamente `HHTSolver`. Tangente constante = rigidez secante. Verificado: `tests/test_dynamic_nonlinear_hht.py::test_linear_material_matches_hht_solver`.

12.32 Contrato de implementación

```
name: NewtonHHTSolver
kind: solver
status: validated

extends: NewtonNewmarkSolver      # variante por Reglas §4
also_references: HHTSolver         # formulación temporal heredada

interface:
  yaml_type: newton_hht
  output: TransientResult
  pipeline_kind: transient

parameters:
  # Solo se listan los parámetros propios o con default distinto al padre.
  # Resto heredados de NewtonNewmarkSolver.
  - { name: alpha, type: float, required: false, default: -0.05,
      desc: "Parámetro HHT - [-1/3, 0]. =0 recupera NewtonNewmark; =-1/3 disipación m
          áxima preservando orden 2" }
  - { name: beta, type: float, required: false, default: null,
      desc: "Override del autoderivado (no recomendado - emite RuntimeWarning)" }
  - { name: gamma, type: float, required: false, default: null,
      desc: "Override del autoderivado (no recomendado - emite RuntimeWarning)" }
```

```

# heredados de NewtonNewmarkSolver:
#   t_end, dt, convergence, max_iter, freeze_tangent_after_iter,
#   line_search, rayleigh, u0, u0_dot, F_func, linear_algebra, lumping

requirements:
- "Modelo dinámico con no-linealidad de material o geometría"
- "M con density > 0 en todos los materiales activos (ADR 0008)"
- "Δt razonable (incondicionalmente estable para sistemas lineales, no garantizado para fuertes no-linealidades)"

conventions:
  units: "heredadas del modelo (ADR 0008)"
  stability: "incondicionalmente estable en problemas lineales para [-1/3, 0]; condicionalmente para no lineales severos"
  disipacion: "ω_ = (1+) / (1-); selectiva en alta frecuencia"

out_of_scope:
- "fuera de [-1/3, 0] (pierde estabilidad u orden 2)"
- "Generalized - / Bossak (variantes con disipación distinta) - diferidos"
- "Análisis estático usar NonlinearSolver"

acceptance:
  verification:
    - name: alpha_cero_recupera_newton_newmark
      setup: "mismo problema con NewtonNewmarkSolver y NewtonHHTSolver (=0)"
      expect: "respuesta idéntica paso a paso"
      tol_rel: 1.0e-12

    - name: material_lineal_recupera_hht
      setup: "mismo problema con HHTSolver y NewtonHHTSolver, ambos -=0.1"
      expect: "respuesta idéntica; Newton converge en 1 iteración por paso"
      tol_rel: 1.0e-12

    - name: disipacion_modo_alta_frecuencia
      setup: "respuesta libre con modos altos espurios excitados; -=0.1"
      expect: "amplitud de modos espurios decae ~ω_ por periodo"
      tol_rel: 0.05

    - name: respuesta_plastica_dinamica
      setup: "barra Truss2D con Elastoplastic1D, carga dinámica que cruza el yield"
      expect: "estado plástico converged paso a paso; histéresis dinámica coherente"
      tol_rel: 1.0e-3

    - name: rechazo_alpha_fuera_de_rango
      setup: "NewtonHHTSolver(alpha=-0.5) o (alpha=0.1)"
      expect: "ValueError con mensaje claro"

    - name: warning_override_beta_gamma
      setup: "NewtonHHTSolver(alpha=-0.1, beta=0.3)"
      expect: "RuntimeWarning emitido al construir"

  specific:
    - name: tangente_singular_dispara_excepcion_tipada
      setup: "modelo con material que produce K_t singular en algún paso"
      expect: "raise SingularTangentError con métricas estructuradas"

references:
- "Hilber H.M., Hughes T.J.R., Taylor R.L. (1977). Improved numerical dissipation for time integration algorithms in structural dynamics. Earthquake Eng. Struct. Dyn."

```



```

5, 283-292."
- "Hughes T.J.R. (2000). The Finite Element Method. Dover. §9.3."
- "Newmark N.M. (1959). A method of computation for structural dynamics. ASCE 85(EM3)
  , 67-94."
- "ADR 0009 (fase 4: HHT - - formulación temporal en residuo no lineal)."
```

12.33 Implementación

- **Archivo:** `solidum/math/solvers/newmark.py` (clase `NewtonHHTSolver`).
- **Clase:** hereda de `NewtonNewmarkSolver`; añade el atributo `self.alpha` y reemplaza el cómputo del residuo y del jacobiano en `solve()`.
- **Auto-derivación** de β y γ vía `_hht_autoderive_beta_gamma(alpha)` (función helper compartida con `HHTSolver`).
- **Validación temprana** en `__init__`: $\alpha \in [-1/3, 0]$, warning si override de β/γ .
- **Entrypoint público:** `solidum.run_transient(model, solver="newton_hht", ...)` (despacho por `PIPELINE_KIND = "transient"`).
- **Tests:**
- `tests/test_dynamic_nonlinear_hht.py` — incluye recuperación de `NewtonNewmark` con $\alpha=0$, recuperación de `HHTSolver` con materiales lineales, respuesta plástica dinámica.

12.34 Diálogo

- **2026-05-19** · Spec corta tipo extensión creada para cerrar el hueco H-5.3 (variante de `HHTSolver/NewtonNewmarkSolver` por Reglas §4). Sin ADR nuevo: reusa ADR 0009 (formulación temporal HHT) y ADR 0011 (Newton + diagnóstico). El comportamiento no se modifica.

12.35 NewtonNewmarkSolver

*Variante de NewmarkSolver según la regla de variantes (Reglas.md §4). Documenta **solo lo que cambia** respecto al padre. Implementa la **fase 4 del ADR 0009**; reusa todas las decisiones arquitecturales del ADR padre — sin ADR nuevo.*

12.36 Qué cambia respecto a NewmarkSolver

NewmarkSolver integra $\mathbf{M} \cdot \ddot{\mathbf{u}} + \mathbf{C} \cdot \dot{\mathbf{u}} + \mathbf{K} \cdot \mathbf{u} = \mathbf{F}(t)$ asumiendo **K constante**: factoriza $\mathbf{A}_{\text{eff}} = \mathbf{M} + \gamma \Delta t \cdot \mathbf{C} + \beta \Delta t^2 \cdot \mathbf{K}$ una vez y cada paso temporal es una resolución triangular barata.

NewtonNewmarkSolver resuelve el mismo problema cuando **la rigidez no es constante**: hay materiales con historia (plasticidad, daño) o no linealidad geométrica. En cada paso temporal se introduce un bucle de **Newton-Raphson** sobre el residuo dinámico, hasta cumplir el criterio de convergencia.

12.36.1 Residuo dinámico en cada paso

En el instante t_{n+1} las incógnitas son $(\mathbf{u}_{n+1}, \dot{\mathbf{u}}_{n+1}, \ddot{\mathbf{u}}_{n+1})$ ligadas por los correctores Newmark:

$$\mathbf{u}_{n+1} = \tilde{\mathbf{u}}_{n+1} + \beta \Delta t^2 \ddot{\mathbf{u}}_{n+1}, \quad \dot{\mathbf{u}}_{n+1} = \tilde{\dot{\mathbf{u}}}_{n+1} + \gamma \Delta t \ddot{\mathbf{u}}_{n+1}$$

(con predictores idénticos a la fase 3). El residuo dinámico es:

$$\mathbf{R}(\ddot{\mathbf{u}}_{n+1}) = \mathbf{F}_{\text{ext}}(t_{n+1}) - \mathbf{F}_{\text{int}}(\mathbf{u}_{n+1}) - \mathbf{C} \dot{\mathbf{u}}_{n+1} - \mathbf{M} \ddot{\mathbf{u}}_{n+1} - \mathbf{F}_{\text{dir}}$$

donde $\mathbf{F}_{\text{int}}(\mathbf{u})$ es el vector de fuerzas internas no lineales (plástico, dañado, etc.) que el `Assembler.assemble_non_` ya calcula para los solvers estáticos (ADR 0003 + ADR 0006 + ADR 0008).

12.36.2 Jacobiano dinámico tangente

Derivando $\mathbf{R}$ respecto a $\ddot{\mathbf{u}}_{n+1}$ usando los correctores:

$$\mathbf{J} = -\frac{\partial \mathbf{R}}{\partial \ddot{\mathbf{u}}_{n+1}} = \mathbf{M} + \gamma \Delta t \mathbf{C} + \beta \Delta t^2 \mathbf{K}_t$$

donde $\mathbf{K}_t = \partial \mathbf{F}_{\text{int}} / \partial \mathbf{u}$ es la rigidez tangente del estado corriente. La iteración es:

$$\mathbf{J} \delta \ddot{\mathbf{u}} = \mathbf{R}, \quad \ddot{\mathbf{u}}_{n+1}^{(k+1)} = \ddot{\mathbf{u}}_{n+1}^{(k)} + \delta \ddot{\mathbf{u}}$$

Los desplazamientos y velocidades se actualizan con los mismos correctores Newmark tras cada update de $\ddot{\mathbf{u}}_{n+1}$.

12.36.3 Amortiguamiento Rayleigh — heredado, constante en el tiempo

$\mathbf{C} = \alpha \mathbf{M} + \beta_R \mathbf{K}_0$ con $\mathbf{K}_0$ la rigidez **elástica de referencia** (la primera ensamblada en $t = 0$, $\mathbf{u} = 0$). No se actualiza con la plasticidad: el Rayleigh es modelo aproximado de amortiguamiento estructural, no consecuencia de la cinemática elastoplástica. Mantenerlo constante evita un acoplamiento ad-hoc que ningún código de referencia (Abaqus, ANSYS, OpenSees) realiza por defecto.

12.36.4 Reducción y commit de estado

- **Reducción por Dirichlet** heredada de la fase 3 (ADR 0004): mismo operador $\mathbf{T}$ y $\mathbf{g}$ constantes; el residuo y el jacobiano se proyectan a DOFs libres antes de cada solve.
- **Commit de variables internas**: al converger el paso, se llama `assembler.commit_all_states()` — los estados ($\epsilon^p, \alpha, \kappa, d, \dots$) pasan de *trial* a *committed*. Mismo patrón que `NonlinearSolver` y `ArcLengthSolver`. Si el paso no converge, los estados trial se descartan (no se llama commit) y se reporta error.

12.36.5 Convergencia (ADR 0007)

Criterio dual fuerza + desplazamiento, calibrado al primer paso temporal con la escala del ensamblaje inicial:

- $\|\mathbf{R}\|/(\text{atol} + \text{rtol} \|\mathbf{F}\|) \leq 1$
- $\|\delta\mathbf{u}\|/(\text{atol} + \text{rtol} \|\mathbf{u}\|) \leq 1$

Idéntico al `NonlinearSolver` estático. La calibración se hace una vez al inicio del análisis; las tolerancias son adimensionales (invariantes bajo cambio de unidades) gracias a la calibración con `force_scale/disp_scale`.

12.36.6 Factorización por iteración

A diferencia de `NewmarkSolver` (que factoriza $\mathbf{J}$ una sola vez al inicio), aquí $\mathbf{K}_t$ cambia con cada iteración, así que $\mathbf{J}$ debe re-factorizarse. Se mantiene el patrón de **Newton modificado opcional** (ADR 0003 fase 2): si `freeze_tangent_after_iter` es `int`, se factoriza fresco las primeras N iteraciones del paso y se reusa la factorización en las siguientes. `None` $\Rightarrow$ Newton estándar (re-factoriza cada iteración).

12.36.7 Recuperación del comportamiento lineal

Si todos los materiales del modelo son lineales, $\mathbf{K}_t \equiv \mathbf{K}_0$ constante y el residuo se anula en una iteración de Newton (Δx exacto en el primer solve). Por tanto **`**NewtonNewmarkSolver`** con materiales lineales reproduce exactamente `NewmarkSolver**` (sin discretización extra ni error adicional). Esto se valida en los acceptance.

12.37 Contrato de implementación

```
name: NewtonNewmarkSolver
kind: solver
status: validated
extends: NewmarkSolver          # variante según Reglas §4

interface:
  type_yaml: NewtonNewmarkSolver
  pipeline_kind: transient      # mismo que NewmarkSolver

parameters_extra_to_NewmarkSolver:
```

```

- { name: convergence,                type: ConvergenceCriterion, required: false,
  desc: "Política ADR 0007. Default: ConvergenceCriterion() con rtols de solidum.
  constants." }
- { name: max_iter,                    type: int,                      required: false,
  default: 20, desc: "Máximo iteraciones Newton por paso temporal." }
- { name: freeze_tangent_after_iter, type: int or None,                required: false,
  default: null, desc: "Si int, Newton modificado: factoriza fresco las primeras N
  iter y reusa después (ADR 0003 fase 2)." }

parameters_inherited:
  # Idénticos a NewmarkSolver: t_end, dt, beta, gamma, rayleigh, u0, u0_dot,
  # F_func, linear_algebra, lumping. Ver docs/specs/NewmarkSolver.md.

state_handling:
  commit: "assembler.commit_all_states() tras converger cada paso"
  rollback: "estado trial descartado si el paso no converge → RuntimeError"

acceptance:
  recovery_linear:
    - name: linear_material_matches_NewmarkSolver
      setup: "Mismo modelo y carga que un caso analítico de NewmarkSolver (oscilador 1
      GDL, viga), pero con NewtonNewmarkSolver"
      expect: "u_history idéntico (rtol 1e-10) al de NewmarkSolver - convergencia en
      1-2 iter por paso"

  nonlinear:
    - name: oscilador_1gdl_elastoplastico_libre
      setup: "1 nodo, Truss2D con ElastoplasticiD, masa concentrada, condición inicial
      u_0 más allá del yield, vibración libre (F=0)"
      expect: "u(t) refleja decaimiento por disipación plástica; (deformación plá
      stica acumulada) > 0 al final"

    - name: viga_voladizo_elastoplastica_pulso
      setup: "Frame2DEuler con varios elementos, masa distribuida, pulso de carga
      concentrada en el extremo libre"
      expect: "respuesta no lineal con redistribución plástica; converge en pocas iter
      por paso (max 6 típicamente)"

  convergence:
    - name: newton_converges_in_few_iter
      setup: "paso plástico activo en un benchmark"
      expect: "número de iteraciones Newton por paso max_iter; raramente excede 8 con
      tangente algorítmica consistente"

references:
  - "ADR 0009 $Fase 4 - Transitorio no lineal Newmark + Newton"
  - "Hughes T.J.R. (2000). The Finite Element Method, cap. 7-9 (Newmark, Newton dentro
    de paso temporal)"
  - "Crisfield M.A. (1991). Non-Linear Finite Element Analysis of Solids and Structures
    , vol. 2 cap. 24 (dinámica no lineal)"
  - "Spec padre: [docs/specs/NewmarkSolver.md](NewmarkSolver.md)"

```

12.38 Implementación

- Archivo: solidum/math/solvers/newmark.py — junto al padre.
- Clase: `NewtonNewmarkSolver(NewmarkSolver)`, registrada con `@SolverRegistry.register`.

Hereda `__init__` con kwargs adicionales `convergence`, `max_iter`, `freeze_tangent_after_iter`. Sobrescribe `solve()`.

■ **Flujo de `solve()`:**

1. Ensamblar $\mathbf{M}$, ensamblar sistema no lineal inicial para obtener $\mathbf{K}_0$ y $\mathbf{F}_{\text{int},0}$.
2. Calibrar amortiguamiento Rayleigh con $\mathbf{K}_0$ (constante en el tiempo).
3. Reducir por Dirichlet (mismo $\mathbf{T}$, $\mathbf{g}$, $\mathbf{F}_{\text{dir}}$ que la fase 3).
4. Aceleración inicial consistente: $\mathbf{M} \ddot{\mathbf{u}}_0 = \mathbf{F}_{\text{ext}}(0) - \mathbf{F}_{\text{int}}(\mathbf{u}_0) - \mathbf{C} \dot{\mathbf{u}}_0 - \mathbf{F}_{\text{dir}}$.
5. Bucle temporal con Newton interno: predictor $\rightarrow$ iteración ($\mathbf{R}, \mathbf{J} = \mathbf{M} + \gamma \Delta t \mathbf{C} + \beta \Delta t^2 \mathbf{K}_t$), factoriza/solve, actualiza correctores; converger; commit.
6. Volcar historial; reportar éxito.

■ **Refactor del padre:** la fase 3 (`NewmarkSolver`) sigue intacta. La subclase no extrae métodos del padre por ahora (el flujo no lineal es lo suficientemente distinto como para que la herencia sea estructural, no compartición de pasos internos). Cuando emergió la variante HHT- α (2026-05-18, `NewtonHHTSolver` como subclase de `NewtonNewmarkSolver`) la herencia estructural funcionó tal cual — los métodos `solve()` se sobrescriben íntegramente en cada subclase sin compartir pasos internos. Si en el futuro entran generalized- α u otras variantes y se observa que comparten *este* flujo no lineal específico, entonces se refactorizará al patrón con `_solve_step` virtual.

■ **Despacho YAML:** `solver: type: NewtonNewmarkSolver` activado vía `SolverRegistry` y `run_yaml`. Como en `NewmarkSolver`, `PIPELINE_KIND = "transient"` declarativo: si futura regla disparo C reemplaza el `isinstance` en `entry.run_yaml`, no se rompe.

■ **Tests:**

- `tests/test_newmark_nonlinear.py` nuevo: acceptance.
- Cobertura de regresión de los tests de `NewmarkSolver` permanece intacta (la subclase no toca el padre).

12.39 Diálogo

- **2026-05-14** · Spec creada como **variante** de `NewmarkSolver` aplicando la nueva regla §4 (registrada en el mismo commit que esta spec). Sin ADR nuevo: reusa las decisiones de ADR 0009 fase 4 (residuo, jacobiano, conmutar Newton dentro del paso). Sin entrada propia en el manual estructural — la subclase es invisible al usuario API (excepto por el `type` en YAML y los parámetros extra).
- **2026-05-14** · Decisión: **subclase** sobre `NewmarkSolver` en lugar de bandera `nonlinear=True` en la misma clase. Razones: (i) más explícito en el código y en el YAML (`type` distinto para análisis distinto); (ii) evita condicionales que ensucian el flujo lineal de la fase 3; (iii) deja sitio para variantes adicionales (HHT- α no lineal, generalized- α) por composición. El ADR 0009 fila "Fase 4" decía "`NewmarkSolver` (mismo, con `nonlinear=True`)– interpretación textualista vs. arquitectural: la regla §4 nueva legitima la elección de subclase, y el patrón conserva el espíritu (mismo análisis, mismas decisiones, mismo entripoint) sin la implementación literal.

- **2026-05-14** · Amortiguamiento Rayleigh queda fijado con $\mathbf{K}_0$ (rigidez elástica de referencia) y constante en el tiempo. Alternativa rechazada: Rayleigh con $\mathbf{K}_t$ corriente — no es la convención estándar y crea coupling artificial entre la disipación viscosa y la plástica. Documentado en sección .Amortiguamiento Rayleigh — heredado”.

12.40 ResponseSpectrumSolver

*Spec de **solver nuevo**. Combina las respuestas máximas modales bajo un espectro de respuesta dado (SRSS o CQC). Cierra el ADR 0009 completo y aplica la regla D de refactor arquitectural (free_vibration sale de `results.py` al nuevo `solidum/math/modal_response.py`).*

12.41 Especificación física

12.41.1 0. Descripción general

Análisis sísmico clásico: el suelo se caracteriza por un **espectro de respuesta** (en aceleración $S_a(T)$ o desplazamiento $S_d(T)$ vs período), no por un acelerograma temporal. Para cada modo del sistema se calcula la respuesta máxima individual $u_n^{\max} = \Gamma_n \cdot \varphi_n \cdot S_d(\omega_n)$ y se combinan las respuestas modales en una envolvente máxima. La fase temporal entre modos se pierde por construcción — sólo se conservan amplitudes envolventes.

Es **lineal** y **estadístico**: aproxima los máximos absolutos sin reconstruir la historia temporal. Útil para diseño normativo donde el espectro reglamentario es la entrada y los máximos modales son la salida solicitada (e.g., ASCE 7, Eurocódigo 8, NCh 433, NTC-DS México).

Hipótesis: lineales (K, M constantes). Para no-linealidad usar transitorio (Newton-Newmark con acelerograma).

12.41.2 1. Ecuación de equilibrio resuelta

Para cada modo n :

$$u_n^{\max} = \Gamma_n \cdot \phi_n \cdot S_d(\omega_n)$$

con $\Gamma_n = \phi_n^T M r$ el factor de participación modal en la dirección de excitación r .

Combinación SRSS (Square Root of Sum of Squares):

$$u_k^{\max} = \sqrt{\sum_n (u_{k,n}^{\max})^2}$$

Combinación CQC (Complete Quadratic Combination, Der Kiureghian 1980):

$$u_k^{\max} = \sqrt{\sum_i \sum_j \rho_{ij} \cdot u_{k,i}^{\max} \cdot u_{k,j}^{\max}}$$

$$\rho_{ij} = \frac{8\xi^2(1+r)r^{3/2}}{(1-r^2)^2 + 4\xi^2r(1+r)^2}, \quad r = \omega_i/\omega_j$$

Cuando $\xi \rightarrow 0$ o $r \rightarrow 0$: $\rho_{ij} \rightarrow \delta_{ij}$ y CQC degenera a SRSS.

12.41.3 2. Condiciones iniciales / de contorno

- Apoyos Dirichlet constantes (eliminación directa, ADR 0004).
- **Dirección de excitación rígida r** (vector unitario o por DOF name): vector r con valor 1 en los DOFs traslacionales en la dirección sísmica, 0 en el resto.
- **No** hay condiciones iniciales — la respuesta es envolvente máxima.

12.41.4 3. Salidas físicas

`ResponseSpectrumResult` (immutable):

- `u_combined[ndof]` respuesta máxima envolvente combinada (positiva).
- `u_per_mode[ndof, n_modes]` contribución modal individual con signo.
- `frequencies_rad[n_modes]` frecuencias naturales empleadas.
- `participation_factors[n_modes]` (Γ_n).
- `effective_masses[n_modes]` (Γ_n^2). Suma converge a la masa total proyectada en la dirección al incluir suficientes modos.
- `combination` ("SRSS" o CQC).
- `damping` (ξ usado para CQC; 0.0 si SRSS).
- `direction[ndof]` vector de excitación.
- Método `.cumulative_effective_mass_ratio()` para verificar el porcentaje de masa modal capturado.

12.42 Formulación numérica

12.42.1 4. Esquema operativo

```
1. Ensamble interno vía ModalSolver(n_modes, sigma, lumping):→
   modes, frequencies_rad
2. Resolver direction (np.ndarray o {"dof_name": "ux"}).
3. Resolver spectrum (callable o dict tabulado/constante).
4. Calcular  $\Gamma_n = \frac{1}{M} \mathbf{r}^T \text{participation\_factors}$ .
5. Calcular  $S_d(n)$  para cada modo.
6. u_per_mode[:, n] =  $\Gamma_n \cdot \mathbf{r} \cdot S_d(n)$ .
7. Combinar: SRSS (raíz cuadrada de suma de cuadrados elemento a elemento)
   o CQC (forma cuadrática con _ij(, )).
8. Devolver ResponseSpectrumResult.
```

12.42.2 5. Predictor / corrector

No aplica — un solve por análisis.

12.42.3 6. Criterio de convergencia

No aplica como Newton — pero la calidad del análisis depende del número de modos incluidos. Se verifica con `.cumulative_effective_mass_ratio() ≥ 0.9` (objetivo normativo típico).

12.42.4 7. Imposición de Dirichlet y MPC

Eliminación directa heredada del `ModalSolver` interno. MPC fuera de alcance.

12.42.5 8. Backend algebraico

Delega en `ModalSolver` para el cálculo de modos (`scipy.sparse.linalg.eigsh` con `shift-invert`) y luego operaciones de álgebra densa para la combinación modal (los modos son densos por columna).

12.42.6 9. Adaptatividad / control de paso

No aplica. El usuario controla `n_modes`. Recomendación: iterativamente subir `n_modes` hasta que la masa efectiva acumulada supere el 90 % en la dirección de excitación.

12.42.7 10. Caveats numéricos

- **Cuántos modos:** el método pierde precisión si los modos significativos en la dirección de excitación no están incluidos. Verifica con `cumulative_effective_mass_ratio`.
- **CQC vs SRSS:** usar CQC cuando los modos están cercanos en frecuencia (cociente < 1.3 según práctica habitual). Para modos bien separados, SRSS es la opción estándar y CQC se degenera a SRSS automáticamente.
- **Espectro fuera de rango:** `spectrum_tabulated` aplica clamp a los valores extremos (no extrapola). Si el barrido de frecuencias del modelo cae fuera del rango tabulado, el usuario debe extender la tabla.
- **Excitación multi-direccional:** este solver maneja una sola dirección por análisis. Para combinación direccional (CQC3, regla 100/30/30 de norma) se requieren dos o tres análisis independientes y una combinación adicional fuera del solver.
- **Cargas con fase:** no aplica al método espectral por construcción — la fase se pierde en el cómputo de máximos modales.

12.43 Contrato de implementación

```
name: ResponseSpectrumSolver
kind: solver
status: validated

interface:
  yaml_type: ResponseSpectrumSolver
  output: ResponseSpectrumResult

parameters:
  - { name: n_modes, type: int, required: true, desc: "Número de modos a calcular y combinar" }
  - { name: direction, type: "ndarray | dict", required: true, desc: "Vector r o {'dof_name': 'ux'}" }
  - { name: spectrum, type: "callable | dict", required: true, desc: "S_d() o {'type': 'tabulated'|'constant_acceleration', ...}" }
  - { name: combination, type: str, required: false, desc: "'SRSS' (default) o 'CQC'" }
  - { name: damping, type: float, required: false, desc: " modal (default 0.05); sólo usado por CQC" }
  - { name: sigma, type: float, required: false, desc: "Shift para ARPACK (default 0)" }
  - { name: lumping, type: str, required: false, desc: "'consistent' (default) o 'lumped'" }
```

```

requirements:
- "Materiales con `density` declarada (ADR 0008)."
```

```

- "Problema lineal - K y M constantes."
```

```

conventions:
  units: "heredadas del modelo (ADR 0008)."
```

```

  stability: "No iterativo; estabilidad numérica gobernada por ModalSolver interno."
```

```

out_of_scope:
- "Excitación multi-direccional simultánea (CQC3) - usuario combina afuera."
```

```

- "Espectros con dependencia explícita de por modo (_n distinto) - `damping` es
  global."
```

```

- "MPC y no linealidad."
```

```

acceptance:
  verification:
    - name: single_mode_recovers_gamma_phi_Sd
      setup: "Modelo 2-DOF, n_modes=1, S_d constante=1"
```

```

      expect: "u_combined = |u_per_mode[:, 0]| (SRSS trivial sobre un modo)"
```

```

      tol_rel: 1.0e-12
```

```

    - name: laplaciano_1d_3dof_analitico
      setup: |
        Cadena 4-truss n1-n2-n3-n4-n5 con n1, n5 fijos. E=A=L=1, =1, lumped →
        K Toeplitz tridiagonal [[2,-1,0],[-1,2,-1],[0,-1,2]], M=I. Autovalores
        cerrados del Laplaciano 1D discreto:  $\omega_n^2 = 2 - 2 \cdot \cos(n/4)$ ; modos
         $u_n[i] = \sqrt{2/4} \cdot \sin(n \cdot i \cdot /4)$ .
```

```

      expect: |
        (1)  $\omega_1 = \sqrt{2}$ ,  $\phi_1 = \sqrt{2}$  a precisión doble.
        (2) Con d = (1,  $\sqrt{1/2}$ , 0):  $\omega_d = 1$ ,  $\phi_d = \sqrt{1/2}$ ,  $\phi_d = 0$  (d = 0 por construcción).
        (3) Para Sd 1 y SRSS: u_combined =  $\sqrt{1/2, 1/2, 1/2}$  exacto.
        (4) Con d = e_{ux} (centro): sólo modo 1 contribuye, u_combined = (c $\sqrt{2}$ , c
          /2, c $\sqrt{2}$ ).
```

```

      tol_rel: 1.0e-10
```

```

      ref: "tests/test_response_spectrum.py::TestResponseSpectrumAnalytic2DOF"
```

```

    - name: srss_manual_combination
      setup: "Modelo 3-DOF, 2 modos, comparar contra raíz cuadrada de suma de cuadrados
        componente a componente"
```

```

      expect: "Coincidencia exacta a precisión de máquina"
```

```

      tol_rel: 1.0e-12
```

```

    - name: well_separated_modes_cqc_equals_srss
      setup: "Modelo 3-DOF con cociente / > 2"
```

```

      expect: "|u_CQC - u_SRSS| / |u_SRSS| < 1%"
```

```

      tol_rel: 0.01
```

```

  specific:
    - name: close_modes_cqc_differs_from_srss
      setup: "Sistema sintético con / = 1.05 y modos no-ortogonales-en-componentes"
```

```

      expect: "Diferencia relativa CQC vs SRSS > 5%"
```

```

      tol_rel: 0
```

```

    - name: dof_name_builds_unit_vector
      setup: "direction={'dof_name': 'ux'}"
```

```

      expect: "direction[node.dofs['ux']] = 1 en todos los nodos con ux"
```

```

      tol_rel: 0
```

```

    - name: cumulative_effective_mass_monotonic
      setup: "n_modes=2 sobre 3-DOF chain"
```

```

      expect: "cumulative crece monotonamente y termina en 1.0"
```

```

      tol_rel: 1.0e-10
```

```

    - name: free_vibration_wrapper_unchanged
      setup: "Regla D aplicada - ModalResult.free_vibration sigue funcionando"
```

```

expect: "Resultado idéntico a llamada directa a modal_response.free_vibration"
tol_rel: 1.0e-12

references:
- "Chopra, A. K. (2017). *Dynamics of Structures*, cap. 13."
- "Wilson, E. L. (2002). *Three-Dimensional Static and Dynamic Analysis of Structures
  *, cap. 15."
- "Der Kiureghian, A. (1980). 'Structural Response to Stationary Excitation'. *J. Eng
  . Mech. Div. ASCE*, 106(EM6), 1195-1213."

```

12.44 Implementación

- Archivo solver: solidum/math/solvers/response_spectrum.py
- Clase: ResponseSpectrumSolver (registrada en SolverRegistry)
- Atributo de despacho: PIPELINE_KIND = "spectrum" → solidum.entry.run_response_spectrum → ResponseSpectrumResult.
- Algoritmos centralizados: solidum/math/modal_response.py (response_spectrum_srss, response_spectrum_participation_factors, helpers spectrum_from_sa, spectrum_tabulated).
- Resultado: ResponseSpectrumResult — dataclass inmutable.
- Tests: tests/test_response_spectrum.py (19 tests verdes).

12.45 Diálogo

- *2026-05-18*: regla D aplicada al introducir este solver — segundo método de cómputo sobre ModalResult (el primero fue free_vibration). El algoritmo de free_vibration se movió a solidum/math/modal_response.py; ModalResult.free_vibration queda como wrapper delgado de 3 líneas, preservando la API histórica. Los nuevos response_spectrum_srss y response_spectrum_cqc viven en el mismo módulo. ResponseSpectrumSolver orquesta ModalSolver interno + delegación a modal_response.
- *2026-05-18*: el cuarto valor de PIPELINE_KIND ("spectrum") extiende el despacho declarativo de run_yaml introducido en D2. Cierra el ADR 0009 completo (fases 1-7 + variante HHT- α).

Capítulo 13

Elementos — catálogo

Referencia rápida de los elementos implementados. Una entrada por elemento. Para detalles físicos/numéricos → código fuente.

> **Convenciones:** *STRAIN_DIM* = dimensión Voigt esperada del material asociado (1 = axial escalar, 3 = 2D [ε_{xx} , ε_{yy} , γ_{xy}], 6 = 3D). DOFs por nodo = *DOF_NAMES*.

13.1 Truss2D — barra axial 2D

Elemento sólido 1D de primer orden, inmerso en el plano. Dos nodos; transmite exclusivamente fuerza axial.

13.1.1 Cinemática

- Interpolación lineal del desplazamiento entre los dos nodos.
- Deformación axial uniforme en el elemento.

13.1.2 Formulación

Cosenos directores del eje: $c = (x_2 - x_1)/L$, $s = (y_2 - y_1)/L$.

$$\begin{aligned} &= B \cdot u_e, & B &= (1/L) \cdot [c, -s, c, s] \\ K_e &= (E \cdot A / L) \cdot dd, & d &= -[c, -s, c, s] \\ F_{int} &= -A \cdot d \end{aligned}$$

13.1.3 Convenciones

- Convención de signos: $\sigma > 0 \Leftrightarrow \varepsilon > 0$ (elongación).
- *STRAIN_DIM* = 1: la deformación que recibe el material es un escalar (no Voigt).
- La orientación de los nodos no afecta el resultado (K_e y F_{int} son invariantes bajo su permutación).

13.1.4 Parámetros

- A — área de la sección transversal.

13.1.5 Cargas distribuidas

- `compute_body_load(b)` reparte $\mathbf{b} \cdot \mathbf{A} \cdot L_0 / 2$ por nodo en cada componente global (forma cerrada exacta para $\mathbf{b}$ uniforme). Útil para peso propio: $\mathbf{b} = (0, -\rho \cdot \mathbf{g})$.

13.1.6 Régimen de validez

- Pequeñas deformaciones ($|\varepsilon| \lesssim 10^{-2}$) y pequeños desplazamientos.
- Cargas exclusivamente axiales. Si la carga transversal sobre la barra no es despreciable $\rightarrow$ `Frame2DEuler` / `Frame2DTimoshenko`.

13.1.7 Validación

- Tests: `tests/test_truss.py` · `TestTruss2D`.
- Archivo fuente: `solidum/elements/truss.py` · clase `Truss2D`.
- Spec: `docs/specs/Truss2D.md`.
- Referencia: Bathe, *Finite Element Procedures*, §4.2.1.

13.2 Truss2DCorot — armadura 2D corotacional

Barra axial 2D en régimen de **grandes desplazamientos y rotaciones con pequeña deformación** (Updated Lagrangian). Hereda de `Truss2D`; comparte DOFs, parámetros y contrato con el material.

13.2.1 Cinemática

- Longitud y cosenos directores se recalculan en configuración **corriente** en cada evaluación.
- Deformación ingenieril corotacional: $\varepsilon = (l - L_0)/L_0$.

13.2.2 Formulación

```
d = -[ c_, -s_, c_, s_]      (dirección corriente del eje)
n = -[ s_,  c_, s_, -c_]    (perpendicular al eje)
K_M  = (E·A / L) · d·d
K_G  = (N / l) · n·n        (rigidez geométrica)
K_T  = K_M + K_G
F_int = N · d
```

13.2.3 Cargas distribuidas

- Heredado de `Truss2D`: `compute_body_load(b)` reparte $\mathbf{b} \cdot \mathbf{A} \cdot L_0 / 2$ por nodo, evaluado en geometría de referencia (aproximación estándar para cargas conservadoras en formulaciones corotacionales).

13.2.4 Régimen de validez

- $|\varepsilon| \lesssim 10^{-2}$ (pequeña deformación axial).
- Desplazamientos y rotaciones de cualquier magnitud.
- Cargas exclusivamente axiales.
- **Fuera de alcance:** pandeo por bifurcación (se capta ablandamiento precrítico pero no el punto crítico; para snap-through usar arc-length).

13.2.5 Validación

- Tests: tests/test_truss.py · TestTruss2DCorot (3 tests, uno por criterio de aceptación).
- Archivo fuente: solidum/elements/truss.py · clase Truss2DCorot.
- Spec: docs/specs/Truss2DCorot.md.
- Referencias: Crisfield §3.3, Belytschko §4.5.

13.3 Truss3D — barra axial 3D

Elemento sólido 1D de primer orden, inmerso en el espacio tridimensional. Dos nodos articulados; transmite exclusivamente fuerza axial. Régimen estrictamente de **linealidad geométrica**.

13.3.1 Cinemática

- Interpolación lineal del desplazamiento entre los dos nodos.
- Deformación axial uniforme en el elemento.
- Configuración inicial fija: L_0 , c , c_y , c_z no se actualizan con los desplazamientos.

13.3.2 Formulación

Cosenos directores del eje: $c = (x_2 - x_1)/L$, $c_y = (y_2 - y_1)/L$, $c_z = (z_2 - z_1)/L$.

$$\begin{aligned} &= B \cdot u_e, & B &= (1/L) \cdot [-c, -c_y, -c_z, c, c_y, c_z] \\ K_e &= (E \cdot A / L) \cdot d \cdot d, & d &= [-c, -c_y, -c_z, c, c_y, c_z] \quad (6 \times 6, \text{ rango } 1) \\ F_{int} &= A \cdot d \end{aligned}$$

13.3.3 Convenciones

- Convención de signos: $\sigma > 0 \Leftrightarrow \varepsilon > 0$ (elongación).
- STRAIN_DIM = 1: la deformación que recibe el material es un escalar (no Voigt).
- La orientación de los nodos no afecta el resultado (K_e y F_{int} son invariantes bajo su permutación).
- Acepta nodos con 2 ó 3 coordenadas (completa con $z = 0$ si la tercera falta).

13.3.4 Parámetros

- A — área de la sección transversal.

13.3.5 Cargas distribuidas

- `compute_body_load(b)` reparte $\mathbf{b} \cdot A \cdot L_0 / 2$ por nodo en cada componente global (forma cerrada exacta). Útil para peso propio: $\mathbf{b} = (0, 0, -\rho \cdot g)$.

13.3.6 Régimen de validez

- Pequeñas deformaciones ($|\varepsilon| \lesssim 10^{-2}$), pequeños desplazamientos y pequeñas rotaciones.
- Cargas exclusivamente axiales.
- Para grandes rotaciones en 3D $\rightarrow$ `Truss3DCorot`.

13.3.7 Validación

- Tests: `tests/test_truss.py · TestTruss3D`.
- Archivo fuente: `solidum/elements/truss.py · clase Truss3D`.
- Spec: `docs/specs/Truss3D.md`.
- Referencias: Bathe §4.2.1; Cook §2.3.

13.4 Truss3DCorot — barra axial 3D corotacional

Barra axial 3D en régimen de **grandes desplazamientos y rotaciones con pequeña deformación** (Updated Lagrangian). Hereda de `Truss3D`; comparte DOFs, parámetros y contrato con el material.

13.4.1 Cinemática

- Longitud y cosenos directores se recalculan en configuración **corriente** en cada evaluación.
- Deformación ingenieril corotacional: $\varepsilon = (1 - L_0)/L_0$.
- La dirección perpendicular al eje es un **plano** (dimensión 2), no un vector único.

13.4.2 Formulación

```

d = -[c, -c_y, -c_z, c, c_y, c_z]      (dirección corriente del eje)
ê = (c, c_y, c_z), P = I - ê · ê      (proyector 3×3 al plano perpendicular)

K_M  = (E · A / L) · d · d              (6×6, rango 1)
K_G  = (N / 1) · [[P, -P], [-P, P]]    (6×6, rango 2)
K_T  = K_M + K_G
F_int = N · d

```

13.4.3 Cargas distribuidas

- Heredado de `Truss3D`: `compute_body_load(b)` reparte $b \cdot A \cdot L_0 / 2$ por nodo, evaluado en geometría de referencia.

13.4.4 Régimen de validez

- $|\varepsilon| \lesssim 10^{-2}$ (pequeña deformación axial).
- Desplazamientos y rotaciones de cualquier magnitud en el espacio.
- Cargas exclusivamente axiales.

13.4.5 Validación

- Tests: `tests/test_truss.py` · `TestTruss3DCorot` (3 tests, uno por criterio de aceptación; el de rigidez geométrica verifica dos direcciones transversas independientes del plano perpendicular).
- Archivo fuente: `solidum/elements/truss.py` · clase `Truss3DCorot`.
- Spec: `docs/specs/Truss3DCorot.md`.
- Referencias: Crisfield §3.3; Belytschko §4.5.

13.5 Cable2DCorot — cable 2D corotacional

Elemento 1D inmerso en el plano que modela un cable: dos nodos, transmite solo tensión. Cinemática corotacional; la unilateralidad la aporta el material (típicamente `CableMaterial1D`).

13.5.1 Cinemática

- Idéntica a una barra corotacional: longitud y cosenos directores se recalculan en configuración corriente en cada evaluación.
- Deformación ingenieril corotacional: $\varepsilon = (l - L_0)/L_0$.
- La respuesta física de cable aparece **solo** si el material devuelve $\sigma = 0$ en compresión.

13.5.2 Formulación

```
d = -[ c_, -s_, c_, s_]
n = -[ s_, c_, s_, -c_]

K_M = (E_t · A / L) · d · d      (0 si material destensado)
K_G = (N / l) · n · n           (0 si N = 0)
K_T = K_M + K_G
F_int = N · d                   (0 si N = 0)
```

13.5.3 Cargas distribuidas

- `compute_body_load(b)` reparte $b \cdot A \cdot L_0 / 2$ por nodo en cada componente global, evaluado en geometría de referencia. Útil para peso propio.

13.5.4 Régimen de validez

- $|\varepsilon| \lesssim 10^{-2}$ en régimen tensado.
- Grandes desplazamientos y rotaciones en el plano.
- Cables completamente destensados aportan $K_T = 0$ al sistema global: otros elementos deben garantizar la estabilidad numérica.

13.5.5 Independencia del diseño

El elemento **no hereda** de `Truss2DCorot`. La cinemática corotacional se implementa dentro de la clase para que futuras modificaciones de las armaduras no afecten a los cables, y viceversa.

13.5.6 Validación

- Tests: `tests/test_cable_elements.py · TestCable2DCorot` (4 tests de aceptación: tensado, destensado, rotación rígida, cruce por cero).
- Archivo fuente: `solidum/elements/cable.py · clase Cable2DCorot`.
- Spec: `docs/specs/Cable2DCorot.md`.
- Referencias: Crisfield §3.3; Irvine §1.2.

13.6 Cable3DCorot — cable 3D corotacional

Elemento 1D inmerso en el espacio que modela un cable: dos nodos, transmite solo tensión, puede rotar libremente en el espacio. Cinemática corotacional 3D; unilateralidad aportada por el material (típicamente `CableMaterial1D`).

13.6.1 Cinemática

- Idéntica en filosofía a una barra corotacional 3D: longitud y cosenos directores se recalculan en configuración corriente.
- Deformación ingenieril corotacional: $\varepsilon = (l - L_0)/L_0$.
- La dirección perpendicular al eje es un **plano** (dimensión 2).

13.6.2 Formulación

```
d = -[ c, -c_y, -c_z,  c,  c_y,  c_z]
ê = ( c,  c_y,  c_z),  P = I - ê·ê      (proyector 3×3)

K_M   = (E_t · A / L) · d·d              (0 si material destensado)
K_G   = (N / l) · [[P, -P], -[P, P]]    (0 si N = 0)
K_T   = K_M + K_G
F_int = N · d                          (0 si N = 0)
```

13.6.3 Cargas distribuidas

- `compute_body_load(b)` reparte $\mathbf{b} \cdot \mathbf{A} \cdot L_0 / 2$ por nodo en cada componente global, evaluado en geometría de referencia.

13.6.4 Régimen de validez

- $|\varepsilon| \lesssim 10^{-2}$ en régimen tensado.
- Grandes desplazamientos y rotaciones en el espacio.
- Cables completamente destensados aportan $\mathbf{K}_T = 0$ al sistema global.

13.6.5 Independencia del diseño

No hereda de `Cable2DCorot` ni de `Truss3DCorot`. La maquinaria cinemática 3D se implementa íntegra dentro de la clase.

13.6.6 Validación

- Tests: `tests/test_cable_elements.py` · `TestCable3DCorot` (4 tests de aceptación).
- Archivo fuente: `solidum/elements/cable.py` · clase `Cable3DCorot`.
- Spec: `docs/specs/Cable3DCorot.md`.
- Referencias: Crisfield §3.3; Belytschko §4.5; Irvine §1.2.

13.7 Frame2DEuler — marco/viga 2D Euler-Bernoulli

Viga esbelta 2D con hipótesis de Euler-Bernoulli: secciones planas perpendiculares al eje neutro deformado. Dos nodos rígidamente conectados; transmite axial + cortante + momento flector.

13.7.1 Cinemática

- Interpolación lineal del desplazamiento axial, Hermite cúbica del desplazamiento transverso.
- Pequeños desplazamientos y rotaciones (régimen geoméricamente lineal).
- Configuración inicial fija: L_0 , $\mathbf{c}$, $\mathbf{s}$, $\mathbf{T}$ se calculan una vez y no se actualizan.

13.7.2 Formulación

Matriz de transformación global→local $\mathbf{T}$ (6×6) con cosenos directores del eje. Matriz local analítica:

```
K_local = [[ EA/L,      0,      0, -EA/L,      0,      0 ],
            [  0, 12EI/L3, 6EI/L2,  0, -12EI/L3, 6EI/L2 ],
            [  0,  6EI/L2,  4EI/L,   0, -6EI/L2,  2EI/L ],
            [-EA/L,      0,      0,  EA/L,      0,      0 ],
            [  0, -12EI/L3, -6EI/L2,  0, 12EI/L3, -6EI/L2 ],
            [  0,  6EI/L2,  2EI/L,   0, -6EI/L2,  4EI/L ] ]
K_global = T · K_local · T
F_int     = T · F_int_local      (componente axial corregida con  $\mathbf{c} \cdot \mathbf{A}$  del material)
```

13.7.3 Cargas distribuidas

- `compute_body_load(b)` distribuye $\mathbf{q} = \mathbf{A} \cdot \mathbf{b}$ (carga por unidad de longitud) con la fórmula consistente Hermite cúbica: axial mitad-mitad por nodo, transversal $\mathbf{q} \cdot L/2$ en fuerza y $\pm \mathbf{q} \cdot L^2/12$ en momento (signo positivo en nodo i , negativo en j). Transformación local $\leftrightarrow$ global vía la misma $\mathbf{T}$ del elemento. Útil para peso propio.

13.7.4 Régimen de validez

- Vigas esbeltas ($L/h \gtrsim 10$); peraltadas $\rightarrow$ `Frame2DTimoshenko`.
- Pequeños desplazamientos, pequeñas rotaciones.
- No captura **plasticidad por flexión** distribuida en la sección: `E_tangent` escala toda la matriz de rigidez por igual. La fluencia que ocurre primero en la fibra inferior bajo flexión no se modela hasta que entre `FiberSection` (deuda técnica documentada).

13.7.5 Independencia del diseño

Vive en el paquete `solidum/elements/frame/` junto a `Frame2DTimoshenko` y `Frame2DEulerCorot`, sin herencia entre ellas. La construcción de longitud, cosenos directores y matriz de transformación 6×6 se delega a `build_geometry_2d` en `solidum/elements/frame/_shared.py`, compartida con `Frame2DTimoshenko` (la versión corotacional reconstruye $\mathbf{T}$ desde `alpha0` por su lógica propia).

13.7.6 Validación

- Tests: `tests/test_frame.py` · `TestFrame2DEulerAcceptance` (3 tests de aceptación + registro, incluye flecha analítica del voladizo $PL^3/(3EI)$).
- Archivo fuente: `solidum/elements/frame/euler.py` · clase `Frame2DEuler`.
- Spec: `docs/specs/Frame2DEuler.md`.
- Referencias: Bathe cap. 5; Cook §2.7.

13.8 Frame2DTimoshenko — marco/viga 2D Timoshenko

Viga 2D con deformación por cortante transversal explícita. Dos nodos rígidamente conectados; transmite axial + cortante + momento flector. Apropiado para vigas peraltadas o cortas ($L/h \lesssim 10$).

13.8.1 Cinemática

- Secciones planas, **no** perpendiculares al eje neutro deformado (rotación independiente de la pendiente de la elástica).
- Distorsión angular $\gamma = dv/ds - \theta$ tratada como campo independiente.
- Configuración inicial fija (régimen geoméricamente lineal).

13.8.2 Formulación

Factor de corrección contra shear locking:

$$\Phi = 12 \cdot E \cdot I / (G \cdot A_s \cdot L^2), \quad G = E / [2(1 + \nu)]$$

Matriz local con coeficientes ajustados:

$$\begin{aligned} a &= 12 \cdot EI / [L^3 \Phi(1+)], & b &= 6 \cdot EI / [L^2 \Phi(1+)] \\ c &= \Phi(4+) \cdot EI / [L \Phi(1+)], & d &= \Phi(2-) \cdot EI / [L \Phi(1+)] \\ K_{\text{local}} &= \begin{bmatrix} EA/L, & 0, & 0, & -EA/L, & 0, & 0, \\ 0, & a, & b, & 0, & -a, & b, \\ 0, & b, & c, & 0, & -b, & d, \\ -EA/L, & 0, & 0, & EA/L, & 0, & 0, \\ 0, & -a, & -b, & 0, & a, & -b, \\ 0, & b, & d, & 0, & -b, & c \end{bmatrix} \end{aligned}$$

Para $\Phi \rightarrow 0$ (viga esbelta) se recupera la matriz Euler-Bernoulli.

13.8.3 Parámetros

- A, I, A_s (área efectiva de cortante).
- `nu` (opcional): se toma de `material.nu` si está disponible; en su defecto, del parámetro del elemento con default 0.3 y aviso por consola.

13.8.4 Cargas distribuidas

- `compute_body_load(b)` usa la misma fórmula consistente Hermite cúbica que `Frame2DEuler` (idéntica para carga uniforme — la diferencia entre formulaciones aparece solo con cargas no uniformes o concentradas).

13.8.5 Régimen de validez

- Vigas gruesas/peraltadas ($L/h \lesssim 10$); para esbeltas $\rightarrow$ `Frame2DEuler` (más simple y sin shear locking).
- Pequeños desplazamientos, pequeñas rotaciones.
- No captura plasticidad distribuida: `E_tangent` escala toda la matriz.

13.8.6 Independencia del diseño

Vive en el paquete `solidum/elements/frame/`, sin herencia con las otras vigas 2D. Comparte `build_geometry_2d` con `Frame2DEuler` en `solidum/elements/frame/_shared.py` — la construcción de la matriz de transformación 6×6 ya no se duplica.

13.8.7 Validación

- Tests: `tests/test_frame.py · TestFrame2DTimoshenkoAcceptance` (convergencia a Euler en viga esbelta, axial puro, simetría).
- Archivo fuente: `solidum/elements/frame/timoshenko.py · clase Frame2DTimoshenko`.

- Spec: docs/specs/Frame2DTimoshenko.md.
- Referencias: Reddy cap. 5; Bathe §5.4.

13.9 Frame2DEulerCorot — viga 2D Euler-Bernoulli corotacional

Viga 2D esbelta Euler-Bernoulli en formulación **corotacional** (Updated Lagrangian). Grandes desplazamientos y grandes rotaciones rígidas del elemento; deformaciones axiales pequeñas y rotaciones nodales deformacionales moderadas.

13.9.1 Cinemática corotacional

- Rotación rígida α_e del eje separada de las rotaciones deformacionales de las secciones $\bar{\theta}_1, \bar{\theta}_2$.
- DOFs deformacionales locales: $[u_l = l - L_0, \bar{\theta}_1, \bar{\theta}_2]$.
- Todo en configuración corriente: c, s, l, α se recalculan por evaluación.

13.9.2 Formulación

```
Rigidez local (3×3) en DOFs deformacionales:
K_ll = diag-block([EA/L], [[4EI/L, 2EI/L], [2EI/L, 4EI/L]])

Matriz B (3×6) en configuración corriente acopla DOFs locales y globales.

Rigidez tangente:
K_T = B · K_ll · B + K_
K_ = (N/l) · z · z - ((M+M)/l^2) · (r · z + z · r)

con r = -[c-, s, 0, c, s, 0], z = -[s, c, 0, s-, c, 0].
```

13.9.3 Cargas distribuidas

- `compute_body_load(b)` evalúa la integral consistente en la configuración de referencia (misma fórmula Hermite cúbica que **Frame2DEuler**). Para grandes rotaciones con cargas conservadoras (gravedad) la diferencia frente a la integración sobre la geometría corriente es de segundo orden y se asume aceptable.

13.9.4 Régimen de validez

- $|\varepsilon_{\text{axial}}| \lesssim 10^{-2}$.
- Rotaciones rígidas del elemento ilimitadas entre commits (hasta $|\alpha_e| < \pi$ por paso).
- Rotaciones deformacionales de las secciones moderadas ($|\theta| \lesssim 30^\circ$).
- Rotaciones continuas $> \pi$ requerirían tracking de vueltas (fuera de alcance).

13.9.5 Dominios que abre

Pandeo por flexión de columnas esbeltas, post-pandeo, snap-through de arcos, brazos flexibles, cables con rigidez a flexión. Problemas que la viga lineal **Frame2DEuler** no puede atacar.

13.9.6 Independencia del diseño

Vive en el paquete `solidum/elements/frame/` junto a las otras vigas 2D, **sin herencia**: la cinemática corotacional se implementa íntegra dentro de la clase (reconstruye T desde `alpha0` en lugar de usar `build_geometry_2d` compartido). Sí comparte con `Frame2DEuler` y `Frame2DTimoshenko` los helpers libres de `_shared.py` para carga de cuerpo (`_frame2d_consistent_body_load`), masa (`_frame2d_consistent_mass_local`) y traducción a `ElementForces` (`_frame2d_forces_from_local`).

13.9.7 Validación

- Tests: `tests/test_frame.py` · `TestFrame2DEulerCorotAcceptance` (4 criterios físicos + **chequeo por diferencias finitas de K_T contra F_{int}** + registro).
- Archivo fuente: `solidum/elements/frame/euler_corot.py` · clase `Frame2DEulerCorot`.
- Spec: `docs/specs/Frame2DEulerCorot.md`.
- Referencias: Crisfield §7.3; Belytschko §4.11; Wriggers cap. 4.

13.10 Frame3D — marco/viga 3D Euler-Bernoulli

Viga 3D esbelta con 6 DOFs por nodo (`ux`, `uy`, `uz`, `rx`, `ry`, `rz`). Transmite axial + cortante en dos planos + flexión en dos planos + torsión. Régimen geoméricamente lineal.

13.10.1 Cinemática

- Secciones planas perpendiculares al eje neutro deformado (Euler-Bernoulli).
- Torsión de Saint-Venant pura (sin alabeo).
- Flexiones en ejes principales desacopladas.
- Configuración inicial fija (no hay variante corotacional 3D por ahora).

13.10.2 Ejes locales y orientación de la sección

El eje $\hat{x}$ local va del nodo 1 al nodo 2. Los ejes $\hat{y}, \hat{z}$ de la sección se definen proyectando un **vector de referencia** (`ref_vector`) sobre el plano perpendicular al eje. Default: `[0, 0, 1]` (eje z global); fallback a `[1, 0, 0]` con advertencia si la barra es casi vertical.

13.10.3 Formulación

Matriz de transformación T 12×12 en bloques de λ (3×3). Matriz local 12×12 desacoplada:

```
Axial      (2×2) : EA/L
Torsión    (2×2) : GJ/L      con G = E / [2(1+)]
Flexión xy (4×4) : 12EIz/L3, 6EIz/L2, 4EIz/L, 2EIz/L
Flexión xz (4×4) : 12EIy/L3, 6EIy/L2, 4EIy/L, 2EIy/L (signos invertidos)

K_global = T K_local T
```

13.10.4 Parámetros

- A , I_y , I_z (área y momentos de inercia respecto a ejes locales y , z).
- J (constante torsional de Saint-Venant).
- ν (opcional, del material si lo expone, sino del elemento con default 0.3).
- `ref_vector` (opcional, default $[0, 0, 1]$).

13.10.5 Cargas distribuidas

- `compute_body_load(b)` proyecta $\mathbf{q} = \mathbf{A} \cdot \mathbf{b}$ sobre los ejes locales y reparte con la fórmula Hermite cúbica en ambos planos de flexión: $\mathbf{q} \cdot L / 2$ en fuerzas, $\pm \mathbf{q} \cdot L^2 / 12$ en momentos (con signos + en nodo i y - en nodo j para M_z , y signos opuestos para M_y por la regla de la mano derecha). No genera torsión (carga uniforme sin excentricidad respecto al centro de cortadura). Útil para peso propio: $\mathbf{b} = (0, 0, -\rho \cdot g)$.

13.10.6 Régimen de validez

- Vigas esbeltas ($L/h \gtrsim 10$).
- Pequeños desplazamientos, pequeñas rotaciones.
- Sin alabeo (secciones macizas o cerradas).
- Para grandes rotaciones en 3D: pendiente `Frame3DCorot`.

13.10.7 Masa lumped: limitación en orientación oblicua

La masa lumped es estrictamente diagonal en ejes locales ($\rho A L / 2$ traslacional, $\rho I \cdot L / 2$ rotacional con I específica por DOF). Tras la rotación $\mathbf{T}^T \mathbf{T} \cdot \mathbf{M} \cdot \mathbf{T}$ a globales, el bloque traslacional 3×3 por nodo permanece diagonal porque $\mathbf{m}_t \cdot \mathbf{I}_3$ es invariante bajo $\text{SO}(3)$, pero el bloque rotacional 3×3 queda **lleno** porque $\mathbf{m}_{rx} = \rho J_p \cdot L / 2$, $\mathbf{m}_{ry} = \rho I_y \cdot L / 2$, $\mathbf{m}_{rz} = \rho I_z \cdot L / 2$ son valores distintos para una sección real. `M_global` resulta entonces **bloque-diagonal por nodo** (no estrictamente diagonal). Es la limitación estándar del lumping en frames 3D (Cook-Malkus-Plesha §11.4); aceptable para Newmark/HHT pero `CentralDifferenceSolver` rechaza con `ValueError` y orienta al usuario hacia Newmark o hacia un eje del elemento alineado con un eje global.

13.10.8 Independencia del diseño

Archivo propio. No hereda ni comparte helpers con `Frame2DEuler`, `Frame2DTimoshenko`, ni con los trusses 3D.

13.10.9 Validación

- Tests: `tests/test_frame3d.py` · `TestFrame3DAcceptance` (5 criterios físicos + validación de `ref_vector` + registro).
- Archivo fuente: `solidum/elements/frame3d.py` · clase `Frame3D`.
- Spec: `docs/specs/Frame3D.md`.
- Referencias: Przemieniecki Tabla 11.3; Cook §2.8; Bathe cap. 5.

13.11 Quad4 — cuadrilátero bilineal 2D

- **Propósito:** continuo 2D isoparamétrico; problema mecánico plano.
- **DOFs por nodo:** ['ux', 'uy'] · 4 nodos (antihorario) · STRAIN_DIM = 3.
- **Cinemática:** matriz $B(\xi, \eta)$ calculada por mapeo isoparamétrico estándar; deformación Voigt $[\varepsilon_{xx}, \varepsilon_{yy}, \gamma_{xy}]$.
- **Integración:** por defecto Gauss 2×2 (4 puntos). Configurable vía `quadrature` desde `QuadratureRegistry`. **Aviso:** integración reducida 1×1 → riesgo de hourglassing.
- **Parámetros:** `thickness` (espesor para estado plano), `quadrature` (opcional).
- **Cargas distribuidas:** `compute_body_load(b)` integra $\int \mathbf{N}^T \mathbf{b} dV$ sobre el elemento; `compute_edge_traction( $\bar{\mathbf{t}}$ )` reparte una tracción uniforme sobre un borde (índices 0..3 según conectividad nodal). Tracción siempre en globales; sin soporte aún para tracción variable o presión normal.
- **Salida por Gauss:** `compute_gauss_state(U)` devuelve σ, ε y coordenadas (naturales y globales) en cada punto de Gauss. Habilita el suavizado nodal del exporter VTK (`Sigma*_nodal`).
- **Implementación:** kernels `_compute_kinematics` y `_compute_integrands` con `@njit` (Numba) — JIT en primera llamada. Viven en `solidum/elements/solid_2d/_shared.py`, compartidos con Tri3.
- **Limitaciones:** bloqueo volumétrico con materiales casi-incompresibles ($\nu \rightarrow 0.5$); en ese régimen usar formulación mixta (no implementada).
- **Archivo:** `solidum/elements/solid_2d/quad4.py`

13.12 Tri3 — triángulo lineal 2D (CST)

- **Propósito:** triángulo de deformación constante; útil para transiciones de malla.
- **DOFs por nodo:** ['ux', 'uy'] · 3 nodos · STRAIN_DIM = 3.
- **Cinemática:** matriz B constante (deformación uniforme dentro del elemento).
- **Integración:** 1 punto central, peso 0.5 (área triángulo en coordenadas naturales).
- **Parámetros:** `thickness`.
- **Cargas distribuidas:** `compute_body_load(b)` reparte 1/3 a cada nodo (exacto para b uniforme); `compute_edge_traction(edge,  $\bar{\mathbf{t}}$ )` reparte $L/2$ a cada uno de los 2 nodos del borde (índices 0..2). Tracción en globales.
- **Salida por Gauss:** `compute_gauss_state(U)` devuelve el único σ, ε y centroide del elemento (mismo formato que Quad4).
- **Limitaciones:** shear locking severo; convergencia lenta. **Preferir Quad4** salvo en transiciones donde Quad4 no encaja geométricamente.
- **Implementación:** kernel `_compute_kinematics_tri3` con `@njit` en `solidum/elements/solid_2d/_shared`
- **Archivo:** `solidum/elements/solid_2d/tri3.py`

13.13 Quad8 — cuadrilátero serendípito 2D de orden 2

- **Propósito:** continuo 2D cuadrático sin shear locking severo en flexión; reproduce campos cuadráticos exactamente.
- **DOFs por nodo:** ['ux', 'uy'] · 8 nodos (4 vértices antihorarios + 4 medios de borde) · STRAIN_DIM = 3.
- **Funciones de forma:** serendípitas (sin término $\xi^2\eta^2$).
- **Integración:** Gauss 3×3 (9 puntos) por defecto. Reducida 2×2 disponible vía `quadrature` (con riesgo de modos espurios).
- **Cargas:** body load por cuadratura; tracción de borde uniforme reparte 1/6, 4/6, 1/6 (vértice, medio, vértice).
- **Salida por Gauss:** `compute_gauss_state(U)` con 9 puntos por defecto.
- **Spec:** docs/specs/Quad8.md.
- **Archivo:** solidum/elements/solid_2d/quad8.py (subclase de la base interna `_HigherOrderSolid2D` definida en `_shared.py`, compartida con Quad9 y Tri6).

13.14 Quad9 — cuadrilátero Lagrangiano 2D de orden 2

- **Propósito:** continuo 2D cuadrático con espacio polinómico completo Q_2 ; añade un noveno nodo central interior al Quad8.
- **DOFs por nodo:** ['ux', 'uy'] · 9 nodos · STRAIN_DIM = 3.
- **Funciones de forma:** producto tensorial Lagrange 1D-1D.
- **Integración:** Gauss 3×3.
- **Cargas y salida por Gauss:** idénticas a Quad8 (el nodo 8 es interior y no participa en bordes).
- **Spec:** docs/specs/Quad9.md.
- **Archivo:** solidum/elements/solid_2d/quad9.py (subclase de la base interna `_HigherOrderSolid2D`, compartida con Quad8 y Tri6).

13.15 Tri6 — triángulo 2D cuadrático completo P_2

- **Propósito:** triángulo isoparamétrico cuadrático que cura el shear locking del Tri3; reproduce campos cuadráticos exactamente.
- **DOFs por nodo:** ['ux', 'uy'] · 6 nodos (3 vértices + 3 medios de borde) · STRAIN_DIM = 3.
- **Integración:** 3 puntos en los puntos medios (cuadratura `tri_3`).
- **Cargas:** body load por cuadratura; tracción de borde reparte 1/6, 4/6, 1/6.

- **Spec:** docs/specs/Tri6.md.
- **Archivo:** solidum/elements/solid_2d/tri6.py (subclase de la base interna `_HigherOrderSolid2D` en `_shared.py`, compartida con Quad8 y Quad9; declara `_MASS_QUADRATURE = "tri_6"` para integrar exactamente la masa consistente, que la cuadratura del elemento subintegraría).

Sólidos tridimensionales con convención Voigt 6D del proyecto (Reglas.md §5): $[\varepsilon_{xx}, \varepsilon_{yy}, \varepsilon_{zz}, \gamma_{xy}, \gamma_{yz}, \gamma_{xz}]$ con $\gamma_{ij} = 2 \cdot \varepsilon_{ij}$. Esta familia **no expone internal_forces** — la primitiva natural de salida en sólidos es `compute_gauss_state(U)`, que devuelve `{stress, strain, points_*}` por punto de integración (ADR 0012: cierre del contrato `internal_forces` por dominio explícito).

13.16 Hex8 — hexaedro trilineal 3D

- **Propósito:** sólido 3D isoparamétrico de primer orden, espejo natural de Quad4.
- **DOFs por nodo:** `['ux', 'uy', 'uz'] · 8 nodos (orden VTK_HEXAHEDRON) · STRAIN_DIM = 6 · N_INTEGRATION_POINTS = 8` (default Gauss $2 \times 2 \times 2$).
- **Cinemática:** matriz $B(\xi, \eta, \zeta)$ 6×24 por mapeo isoparamétrico estándar; deformación Voigt 3D.
- **Integración:** por defecto Gauss $2 \times 2 \times 2$ (8 puntos). Configurable vía `quadrature (hex_3x3x3` para no-lineales severos, `hex_1x1x1` reducido con riesgo de hourglass).
- **Caras (ADR 0012):** 6 caras numeradas con normal saliente — 0 ($-\zeta$), 1 ($+\zeta$), 2 ($-\eta$), 3 ($+\xi$), 4 ($+\eta$), 5 ($-\xi$). Ver `Hex8.FACE_NODES`.
- **Cargas distribuidas:** `compute_body_load(b)` integra $\int N^T b \, dV$; `compute_face_traction(face,  $\bar{t}$ )` reparte tracción uniforme sobre una cara con cuadratura 2D Gauss 2×2 (4 puntos por cara). $\bar{t}$ en globales.
- **Salida por Gauss:** `compute_gauss_state(U)` devuelve `{points_natural, points_global, strain (n_g, 6), stress (n_g, 6)}`. Sin `internal_forces` (ADR 0012).
- **Implementación:** `kernels_compute_kinematics_hex8, _det_jacobian_hex8, _shape_functions_hex8` con `@njit` en `solidum/elements/solid_3d/_shared.py`. Mass lumping vía `lump_hrz` con `n_translational_dirs=3` y `_expand_scalar_mass_3d`.
- **Limitaciones declaradas:**
- **Locking volumétrico** con $\nu \rightarrow 0.5$ — sin mitigación (política idéntica al Quad4 2D). Blindado por `tests/test_volumetric_locking_3d.py`.
- **Hourglass** con integración reducida `hex_1x1x1`: 12 modos espurios; sin estabilización Flanagan-Belytschko.
- **Shear locking** con malla coarse 1 capa en sección: documentado en `tests/validation/test_macneal_beam_3d.py`.
- **Spec:** docs/specs/Hex8.md.
- **Archivo:** solidum/elements/solid_3d/hex8.py.

13.17 Hex20 — hexaedro serendípito 3D de orden 2 (20 nodos)

- **Propósito:** sólido 3D isoparamétrico de orden cuadrático con interpolación serendípita; análogo 3D natural del Quad8. Reproduce campos cuadráticos completos en 3D y mitiga drásticamente el shear locking del Hex8 en problemas de flexión y geometrías curvas.
- **DOFs por nodo:** ['ux', 'uy', 'uz'] · 20 nodos (orden VTK_QUADRATIC_HEXAHEDRON: 8 vértices + 12 medios de arista, sin nodos de cara ni centroide) · STRAIN_DIM = 6 · N_INTEGRATION_POINTS = 27 (default Gauss 3×3×3).
- **Cinemática:** matriz $B(\xi, \eta, \zeta)$ 6×60 sobre Voigt 6D del proyecto (ADR 0012). Funciones de forma serendípitas: vértices con multiplicador $(\xi_i \xi + \eta_i \eta + \zeta_i \zeta - 2)$ que anula la función en todos los demás nodos; medios $(1 - \alpha^2)$ con α el eje del cual son punto medio.
- **Integración:** por defecto Gauss 3×3×3 (27 puntos, exacta para K con Jacobiano constante y para masa consistente cuadrática). Configurable vía `quadrature` (`hex_2x2x2` reducida con 6 modos de hourglass espurios por elemento aislado — sin estabilización implementada).
- **Caras (ADR 0012):** 6 caras numeradas con normal saliente, **8 nodos por cara** (4 vértices + 4 medios). Numeración paritaria con Hex8 en los vértices; cada cara añade los 4 medios en el mismo orden cíclico (medio entre vértice k y $k+1$). Ver `Hex20.FACE_NODES`.
- **Cargas distribuidas:** `compute_body_load(b)` integra $\int N^T b \, dV$ con la cuadratura del elemento (reparto **no uniforme** entre vértices y medios serendípticos, suma global $b \cdot V_e$ exacta). `compute_face_traction(face, t)` reparte la tracción uniforme sobre la cara serendípita Quad8 con Gauss 3×3 (exacto para tracción constante en cara plana): cada vértice recibe $-A \cdot t / 12$ y cada medio $4 \cdot A \cdot t / 12$; los signos negativos en vértices son el fenómeno serendípito conocido del Quad8 2D (Cook-Malkus-Plesha-Witt §6.5).
- **Salida por Gauss:** `compute_gauss_state(U)` devuelve {`points_natural`, `points_global`, `strain` (`n_g`, 6), `stress` (`n_g`, 6)}. Sin `internal_forces` (ADR 0012).
- **Masa:** consistente con cuadratura `hex_3x3x3 fija` (independiente de la cuadratura elegida para K, para que la masa siempre quede exacta). Lumped por HRZ canónico — para Hex20, row-sum produciría masas negativas en vértices serendípticos (Bathe FEP §9.2.4); HRZ es la única opción razonable.
- **Implementación:** funciones de forma y derivadas en `_N_hex20/_dN_hex20` (numpy puro, no @njit por la complejidad de la rama por tipo de nodo) en `solidum/elements/solid_3d/_shared.py`. Kinematics genérico de orden superior 3D en `_kinematics_higher_order_3d` (también `_shared.py`), reutilizable por Hex27 y Tet10 en sub-fases siguientes. Cara serendípita integrada con `_N_quad8/_dN_quad8` importados del paquete 2D.
- **Limitaciones declaradas:**
 - **Locking volumétrico** con $\nu \rightarrow 0.5$ **atenuado** respecto al Hex8 pero aún presente — sin mitigación implementada (política idéntica a Quad8/Hex8). Blindado por `tests/test_volumetric_locking_3d.py`: (ratio $u(0.4999)/u(0.3) \approx 0.80$ vs <0.6 en Hex8).
 - **Hourglass** con integración reducida `hex_2x2x2`: 6 modos espurios por elemento aislado; en mallas ensambladas con BC razonable los modos no propagan, pero un elemento aislado o capa pobre los mostraría. Sin estabilización.

- **Shear locking drásticamente mitigado** respecto al Hex8 en problemas de flexión: cuantificado en `tests/validation/test_macneal_beam_3d.py` (Hex20 $6\times 1\times 1$ alcanza 97 % de u_{EB} , vs <55 % de Hex8 $12\times 1\times 1$).
- **Validación externa:** NAFEMS LE10 thick plate pressure (`tests/validation/test_nafems_le10.py`, $6\times 6\times 2 \rightarrow \sigma_{yy}(D) \approx -5.38$ MPa canónico dentro de banda 10 %). **Lamé thick cylinder 3D** (`tests/validation/test_lame_cylinder_3d.py`, $4\times 4\times 1 \rightarrow$ error L^2 -rel <4 % en σ y <2 % en u_r contra solución cerrada de Timoshenko-Goodier §28; convergencia h monótona del error $L^2(\sigma_{\theta\theta})$ verificada). Es la primera demostración cuantitativa del proyecto de **capacidad isoparamétrica curva** — los mid-edges quedan automáticamente sobre la superficie cilíndrica por la no-linealidad cos/sin del mapeo angular.
- **Spec:** `docs/specs/Hex20.md`.
- **Archivo:** `solidum/elements/solid_3d/hex20.py`.

13.18 Hex27 — hexaedro Lagrangiano 3D triquadrático (27 nodos)

- **Propósito:** sólido 3D isoparamétrico Lagrangiano completo de orden cuadrático; análogo 3D del Quad9. Reproduce **todos** los polinomios triquadráticos ' $\xi^a \eta^b \zeta^c$ con $a, b, c \in \{0, 1, 2\}$ ', incluyendo los términos puramente triquadráticos ($\xi^2 \eta^2 \zeta^2$ y combinaciones) que faltan en el Hex20 **serendípito**. Se prefiere sobre Hex20 cuando la geometría tiene curvatura severa y se busca convergencia óptima en problemas donde el contenido espectral cubre el espacio completo; en flexión simple con geometría rectilínea Hex27 y Hex20⁺ dan resultados prácticamente idénticos a un coste mayor para el primero (Bathe FEP §5.3.2).
- **DOFs por nodo:** `['ux', 'uy', 'uz']` · 27 nodos (orden VTK_TRIQUADRATIC_HEXAHEDRON: 0-7 vértices, 8-19 medios de arista (paritarios con Hex20), 20-25 centros de cara en orden `'(-x, +x, -y, +y, -z, +z)`, 26 centro de cuerpo) · `STRAIN_DIM = 6` · `N_INTEGRATION_POINTS = 27` (default Gauss $3\times 3\times 3$).
- **Cinemática:** matriz $B(\xi, \eta, \zeta)$ 6×81 sobre Voigt 6D del proyecto (ADR 0012). Funciones de forma como producto tensorial de Lagrange cuadrático 1D: `'N_n = L_{i_n}(\xi) · L_{j_n}(\eta) · L_{k_n}(\zeta)` con $i_n, j_n, k_n \in \{-1, 0, +1\}$ '.
- **Integración:** por defecto Gauss $3\times 3\times 3$ (27 puntos, exacta para K y masa consistente sobre Jacobiano constante). Reducida `hex_2x2x2` disponible pero **introduce 27 modos de hour-glass espurios por elemento aislado** — la combinación más problemática del catálogo 3D. Recomendación operativa estricta: usar $3\times 3\times 3$.
- **Caras (ADR 0012):** 6 caras numeradas con normal saliente, **9 nodos por cara** (4 vértices + 4 medios + 1 centro de cara). Numeración paritaria con Hex8/Hex20 en los vértices y medios; el centro de cara correspondiente se añade al final (índice 20-25 según la cara). Las funciones de forma 2D de cara son las del Quad9 Lagrangiano (`_N_quad9/_dN_quad9` del paquete 2D).
- **Cargas distribuidas:** implementadas en la base `_HigherOrderSolid3D` (paritaria con `_HigherOrderSolid2D`). `compute_body_load(b)` integra $\int N^T b \, dV$; `compute_face_traction(face, t)` integra sobre la cara Quad9 con cuadratura 3×3 . Suma global `b · V_e` (body) y `A · t` (face) exactas en cara plana con campo uniforme.

- **Salida por Gauss:** `compute_gauss_state(U)` con 27 puntos. Sin `internal_forces` (ADR 0012).
- **Masa:** consistente con cuadratura `hex_3x3x3` fija (independiente de la cuadratura de `K`). Lumped HRZ canónico — row-sum produciría masas negativas en nodos interiores (Bathe FEP §9.2.4).
- **Implementación:** funciones de forma y derivadas en `_N_hex27/_dN_hex27` (numpy puro, vía productos tensoriales de los polinomios Lagrange 1D `_L_lagrange_quad/_dL_lagrange_quad`). Subclase de la base `_HigherOrderSolid3D` introducida con la entrada del `Hex27` (regla de los dos casos reales aplicada). El cuerpo de `hex27.py` queda como declaración de atributos.
- **Limitaciones declaradas:**
- **Coste computacional:** 81 DOFs/elemento vs 60 del `Hex20` (35 % más). Para flexión simple `Hex20` da resultados equivalentes a menor coste.
- **Locking volumétrico** con $\nu \rightarrow 0.5$ comparable al `Hex20` (ratio ≈ 0.80 vs <0.6 del `Hex8`). Sin mitigación implementada.
- **Hourglass** con integración reducida `hex_2x2x2`: 27 modos espurios por elemento aislado (peor del catálogo 3D); en mallas estructuradas la mayoría no propaga tras ensamblar.
- **Validación externa:** NAFEMS LE10 thick plate pressure (`tests/validation/test_nafems_le10.py`, $5 \times 5 \times 2 \rightarrow \sigma_{yy}(D) \approx -5.38$ MPa canónico dentro de banda 10 %). **Lamé thick cylinder 3D** (`tests/validation/test_lame_cylinder_3d.py`, $4 \times 4 \times 1 \rightarrow$ error L^2 -rel $<4\%$ en σ y $<2\%$ en u_r contra Timoshenko-Goodier §28; convergencia h monótona). Mid-edges automáticos sobre superficie cilíndrica por la no-linealidad del mapeo angular.
- **Spec:** `docs/specs/Hex27.md`.
- **Archivo:** `solidum/elements/solid_3d/hex27.py`.

13.19 Tet10 — tetraedro cuadrático 3D (10 nodos)

- **Propósito:** sólido 3D isoparamétrico cuadrático sobre simplex tetraédrico. Análogo 3D del `Tri6` 2D. Mitiga drásticamente el shear locking y el locking volumétrico del `Tet4` (CST 3D). Adecuado para mallas no estructuradas y geometrías complejas donde el `Hex20/Hex27` requeriría mapeo estructurado.
- **DOFs por nodo:** `['ux', 'uy', 'uz']` · 10 nodos (orden `VTK_QUADRATIC_TETRA`: 0-3 vértices idénticos al `Tet4`, 4-9 medios de arista en orden (0-1, 1-2, 2-0, 0-3, 1-3, 2-3)) · `STRAIN_DIM = 6` · `N_INTEGRATION_POINTS = 4` (default Stroud `tet_4`).
- **Cinemática:** matriz $B(\xi, \eta, \zeta)$ 6×30 sobre Voigt 6D del proyecto (ADR 0012). Funciones de forma en coordenadas baricéntricas L_i : vértices $N_i = L_i(2L_i - 1)$; medios $N_{ij} = 4 \cdot L_i \cdot L_j$. Reproduce todos los polinomios cuadráticos completos en (ξ, η, ζ) .
- **Integración:** por defecto Stroud `tet_4` (4 puntos, orden 2 — exacta para `K` con Jacobiano constante, pues B es lineal en barycentric). Alternativa `tet_15` (Keast 15 puntos, orden 5) para geometrías distorsionadas. **La masa consistente usa `tet_15` fijo** (independiente de la cuadratura de `K`) para integrar exactamente el producto cuadrático×cuadrático y garantizar correctitud en análisis modal/transitorio.

- **Caras (ADR 0012):** 4 caras triangulares con normal saliente, paritarias con **Tet4** en los vértices. Cada cara añade los 3 medios de arista correspondientes en orden **Tri6** (vértices antihorarios, medios entre 0-1, 1-2, 2-0). Cara *i* opuesta al vértice *i*. Las funciones de forma 2D de cara son las del **Tri6** (`_N_tri6/_dN_tri6` del paquete 2D).
- **Cargas distribuidas:** implementadas en la base `_HigherOrderSolid3D`. `compute_body_load(b)` integra $\int N^T b \, dV$ con la cuadratura del elemento; reparto no uniforme entre vértices y medios, suma global `b-V_e` exacta. `compute_face_traction(face, t)` integra sobre la cara triangular **Tri6** con cuadratura `tri_3` (exacta para tracción uniforme sobre cara plana).
- **Salida por Gauss:** `compute_gauss_state(U)` con 4 puntos por defecto.
- **Implementación:** funciones de forma y derivadas en `_N_tet10/_dN_tet10` (numpy puro, vía coordenadas baricéntricas) en `solidum/elements/solid_3d/_shared.py`. Subclase de la base `_HigherOrderSolid3D`. Las nuevas cuadraturas `tet_4` y `tet_15` viven en `solidum/math/integration.py`.
- **Limitaciones declaradas:**
- **Locking volumétrico** con $\nu \rightarrow 0.5$ atenuado respecto al **Tet4** pero presente; sin mitigación implementada.
- **Distorsión severa:** para **Tet10** con mid-edges curvos, el Jacobiano varía espacialmente; el usuario debería seleccionar `tet_15` vía el parámetro `quadrature`.
- **Superficie curva con descomposición ingenua hex→5tets:** no converge. La cara del tet que toca una superficie curva tiene 3 mid-edges; con descomposición genérica de cada hex en 5 tets, sólo 1 de los 3 mid-edges está sobre una arista del grid paramétrico (curva), los otros 2 corresponden a aristas diagonales del hex que se vuelven aristas del tet con mid-edges rectos. La representación isoparamétrica se pierde y el error σ_{rr} **crece con refinamiento h** (ver `tests/validation/test_lame_cylinder_3d.py` — tests **Tet10** con `@pytest.mark.skip` y motivo). Para validar **Tet10** sobre superficie curva se requiere mesher tetraédrico nativo (gmsh API) o descomposición específica del dominio en tets cuyas aristas estén todas sobre el grid paramétrico. Deuda técnica #8 en STATUS. La capacidad del **Tet10** sobre geometría plana está validada en `test_cube_lame_3d.py` (cubo Lamé exacto a precisión máquina), así que NO es fallo del elemento.
- **Validación externa:** cubo Lamé 3D tracción uniaxial sobre malla 5-Tet (exacto a precisión máquina, `test_cube_lame_3d.py`). Superficie curva diferida — ver limitación arriba.
- **Spec:** `docs/specs/Tet10.md`.
- **Archivo:** `solidum/elements/solid_3d/tet10.py`.

13.20 Tet4 — tetraedro lineal 3D (CST 3D)

- **Propósito:** sólido 3D isoparamétrico de primer orden, espejo natural de **Tri3**. Útil para mallas no estructuradas y transiciones.
- **DOFs por nodo:** `['ux', 'uy', 'uz'] · 4 nodos` (orden `VTX_TETRA`, volumen positivo) · `STRAIN_DIM = 6 · N_INTEGRATION_POINTS = 1`.

- **Cinemática:** B constante 6×12 (deformación uniforme dentro del elemento — CST 3D). Reproduce campos lineales exactamente.
- **Integración:** 1 punto baricéntrico con peso $1/6$ (exacto por linealidad de B).
- **Caras (ADR 0012):** 4 caras triangulares numeradas con normal saliente; cara i opuesta al nodo i . Ver `Tet4.FACE_NODES`.
- **Cargas distribuidas:** `compute_body_load(b)` reparte $\mathbf{b} \cdot \mathbf{V}_e / 4$ a cada nodo (exacto); `compute_face_traction(t)` reparte $\mathbf{A}_{cara} \cdot \mathbf{t} / 3$ a cada uno de los 3 nodos de la cara, cero al nodo opuesto.
- **Salida por Gauss:** `compute_gauss_state(U)` con 1 punto (centroide); σ y ε constantes sobre el elemento.
- **Implementación:** kernel `_compute_kinematics_tet4` con `@njit` en `_shared.py`. Masa consistente analítica $\mathbf{M}_{ij} = \rho \cdot \mathbf{V}_e \cdot (1 + \delta_{ij}) / 20$; lumped HRZ canónico (= row-sum por simetría).
- **Limitaciones declaradas:**
 - **Shear locking severo** en flexión — peor que Hex8 por la pobreza del espacio lineal. Recomendación: usar Hex8 en mallas hexaédricas; Tet4 para transiciones de malla no estructurada o cuando geometría compleja impide hexaedros.
 - **Locking volumétrico** con $\nu \rightarrow 0.5$ — aún peor que en Hex8.
 - **Spec:** docs/specs/Tet4.md.
 - **Archivo:** solidum/elements/solid_3d/tet4.py.

Subfamilia de elementos con **DOFs enriquecidos elementales** y **condensación estática local**: el ensamblador no ve los grados de libertad del salto. Cuando se cumple un criterio de activación (Rankine en fase 1), el elemento materializa una discontinuidad interna Γ_d y enriquece su cinemática con un salto $[[\mathbf{u}]]$ gobernado por un material cohesivo (`CohesiveMaterial`, ver el catálogo de materiales §"Materiales cohesivos"). La activación se evalúa en el hook `Element.prepare_step(U_committed)` que los solvers no lineales invocan **una vez por paso**, antes del Newton (anti-chattering, ADR 0010 §5).

13.21 CST_Embedded2D — triángulo CST con discontinuidad interior embebida (KOS)

- **Propósito:** introducir fractura computacional en aproximación discreta sobre el CST padre. Fiel a Retama (2010), Caps. 2, 5, 6 y 7: cinemática KOS, condensación estática local, longitud efectiva $l_d = (\mathbf{A}_e / h) \cdot \cos(\theta - \alpha)$.
- **DOFs por nodo:** $['ux', 'uy'] \cdot 3 \text{ nodos} \cdot \text{STRAIN_DIM} = 3 \cdot \text{N_INTEGRATION_POINTS} = 1$.
- **DOFs enriquecidos (elementales, no globales):** $[[\mathbf{u}]] \in \mathbb{R}^2$ en frame local $(\mathbf{n}, \mathbf{s})$ de Γ_d . Se condensan dentro de `compute_element_state` y nunca llegan al ensamblador.
- **Estado intacto:** bit-exact con Tri3 (mismo B, mismo material, misma cuadratura).

- **Estado agrietado:** Newton local sobre $[[u]]$ hasta $R^{\{[[u]]\}} = 0$; después la condensación $K_{\text{cond}} = K_{\text{dd}} - K_{\text{du}} \cdot K_{\{[[u]]\}}^{-1} \cdot K_{\text{du}}^T$ se devuelve al ensamblador. El bulk descarga elásticamente conforme $[[u]]$ crece (discrete approach); la disipación va al cohesivo en Γ_d .
- **Activación:** criterio Rankine ($\sigma_I > \sigma_{t0}$ del cohesivo) en el centroide del CST, con el estado convergido del paso anterior. **Irreversible:** una vez activada, la discontinuidad persiste aunque el siguiente paso descargue.
- **Materiales aceptados:** bulk solo **Elastic2D** en fase 1 (la discrete approach presupone bulk elástico, ADR 0010); cohesivo cualquier **CohesiveMaterial** con **JUMP_DIM** = 2.
- **Estado de la discontinuidad:** **DiscontinuityState** (en `solidum/core/discontinuity_state.py`) — paralela a **ElementState**, semántica `trial/commit`.
- **YAML:**

```
cohesive_materials:
  - {id: 1, type: CohesiveDamageIsotropic, sigma_t0: 2.5e6, G_f: 100.0,
      K_e: 1.0e13, softening: linear}
elements:
  - {id: 1, type: CST_Embedded2D, nodes: [1, 2, 3],
      material: 1, cohesive_material: 1}
```

- **Post-procesamiento:** `compute_gauss_state(U)` añade clave 'discontinuity' con {normal, tangent, centroid, solitary_node, l_d, jump, traction, damage} cuando el elemento está agrietado.
- **Limitaciones declaradas** (`out_of_scope` en la spec): modo mixto I-II (fase G del ADR 0010), reorientación de **n** tras activación (tracking no trivial, fase F), múltiples discontinuidades por elemento, contacto unilateral en compresión, bulks no elásticos, orden superior (**Tri6_Embedded**), 3D (**Tet4_Embedded**).
- **Spec:** `docs/specs/CST_Embedded2D.md`.
- **Archivo:** `solidum/elements/solid_2d/embedded_cst.py`.

13.22 Cómo añadir un elemento nuevo

1. **Spec primero** — el usuario crea `docs/specs/<Nombre>.md` a partir de `docs/specs/_template_element.md` (especificación física + formulación + contrato YAML). Sin spec, la IA no escribe código.
2. **Scaffolding** — `/solidum-new element <Nombre>` genera archivo en `solidum/elements/`, decorador `@ElementRegistry.register` y esqueleto de test.
3. **Implementación + validación** — la IA codifica contra la spec; los tests cubren los casos de `acceptance` declarados.
4. **Catálogo** — cuando la spec pasa a `status: validated`, se añade aquí una entrada breve siguiendo el formato de arriba (la spec sigue siendo la referencia detallada).

Capítulo 14

Modelos constitutivos — catálogo

Referencia rápida de los modelos constitutivos implementados. Una entrada por material. Para detalles del algoritmo → código fuente.

>**Convenciones:** `STRAIN_DIM` = dimensión Voigt de la deformación de entrada (1 = escalar, 3 = 2D [ε_{xx} , ε_{yy} , γ_{xy}], 6 = 3D). `PRIMARY_STATE_VAR` = variable interna que el `VtkExporter` lleva al output.

>**Densidad (ADR 0008):** todos los materiales aceptan un parámetro opcional *density* (kg/m³ en las unidades del problema). No requerido al construir — análisis estáticos sin peso propio funcionan sin declararla. Cuando un consumidor que requiere masa la pide (`Assembler.assemble_self_weight(g)` y futuras `assemble_mass_matrix`, etc.) y la encuentra como `None`, **falla con `ValueError`** listando los materiales sin densidad. Valor `0.0` declarado explícitamente cubre el caso legítimo de material sin masa física (penalty, restricción) y emite warning informativo.

14.1 Elastic1D — elástico lineal 1D

- **Ley:** $\sigma = E \cdot \varepsilon$ (Hooke 1D).
- **STRAIN_DIM:** 1.
- **Parámetros:** E (módulo de Young).
- **Variables internas:** ninguna (sin historia).
- **Tangente:** constante = E.
- **Compatible con:** Truss2D, Truss3D, Frame2DEuler, Frame2DTimoshenko.
- **Spec:** docs/specs/Elastic1D.md
- **Archivo:** solidum/materials/elastic.py

14.2 Elastic2D — elástico lineal 2D

- **Ley:** $\sigma = C \cdot \varepsilon$, con C el tensor constitutivo isótropo en notación Voigt [xx , yy , xy].
- **STRAIN_DIM:** 3.
- **Parámetros:** E, nu, hypothesis $\in \{\text{'plane_stress'}, \text{'plane_strain'}\}$.

- **Variables internas:** ninguna.
- **Tangente:** constante = $\mathbf{C}$.
- **Compatible con:** Quad4, Tri3, Tri6, Quad8, Quad9 (todos los sólidos 2D con STRAIN_DIM = 3).
- **Spec:** docs/specs/Elastic2D.md
- **Archivo:** solidum/materials/elastic_2d.py

14.3 Elastic3D — elástico lineal isótropo 3D

- **Ley:** $\sigma = \mathbf{C} \cdot \varepsilon$, con $\mathbf{C}$ el tensor constitutivo isótropo 6×6 en notación Voigt 3D del proyecto [xx, yy, zz, xy, yz, xz] (ADR 0012, Reglas.md §5).
- **STRAIN_DIM:** 6.
- **Parámetros:** E (>0), nu $\in (-1, 0.5)$, density (opcional, ADR 0008).
- **Variables internas:** ninguna.
- **Tangente:** constante = $\mathbf{C}$.
- **Hipótesis:** no aplican plane_stress/plane_strain en 3D — toda la deformación es activa.
- **Compatible con:** Hex8, Tet4 (todos los sólidos 3D con STRAIN_DIM = 6).
- **Caveat de compatibilidad:** ABAQUS usa orden Voigt [11, 22, 33, 12, 13, 23] (permutación yz $\leftrightarrow$ xz respecto al proyecto). Importar datos de ABAQUS requiere intercambiar los componentes 5 $\leftrightarrow$ 6 una vez en el preprocesador.
- **Spec:** docs/specs/Elastic3D.md
- **Archivo:** solidum/materials/elastic_3d.py

14.4 IsotropicDamage1D — daño isótropo 1D con softening exponencial

- **Modelo:** daño escalar $d \in [0, \text{DAMAGE_MAX}]$, esfuerzo $\sigma = (1 - d) \cdot E \cdot \varepsilon$. Versión 1D de IsotropicDamage2D.
- **Evolución del daño** (Kuhn-Tucker):
 - Deformación equivalente: $\varepsilon_{eq} = |\varepsilon|$.
 - Variable histórica: $\kappa = \max(\kappa_{old}, \varepsilon_{eq})$.
 - Si $\kappa \leq \kappa_0 \rightarrow d = 0$. Si no: $d = 1 - (\kappa_0/\kappa) \cdot \exp(-\alpha \cdot (\kappa - \kappa_0))$, saturado en DAMAGE_MAX.
- **Tangente:** **algorítmica consistente** en carga activa con daño no saturado; **secante** $(1-d) \cdot E$ en descarga, sin daño activo o al saturar.

- Fórmula consistente: $E_{\text{tan}} = -(1-d) \cdot E \cdot \alpha \cdot \kappa$ — **negativa** durante toda la fase de softening (refleja la pendiente descendente σ - ε post-pico).
- Derivación: de $(1-d) \cdot E - E \cdot \varepsilon \cdot (\partial d / \partial \kappa) \cdot \text{sign}(\varepsilon)$ con $\partial d / \partial \kappa = (1-d)(1/\kappa + \alpha)$ y $\text{sign}(\varepsilon) \cdot \varepsilon = |\varepsilon| = \kappa$ en carga.
- **STRAIN_DIM**: 1 · **PRIMARY_STATE_VAR**: 'damage' · **IS_SYMMETRIC**: True (la tangente escalar produce contribución elemental simétrica aunque pueda ser indefinida; el despachador algebraico ADR 0003 detecta no-PD y degrada Cholesky→LU automáticamente).
- **Parámetros**: E, kappa_0 (umbral elástico), alpha (velocidad de degradación), density (opcional, ADR 0008).
- **Variables internas**: kappa, damage.
- **Admisibilidad (ADR 0006)**: `admissibility_scale = E · kappa_0` — esfuerzo umbral inicial. Garantiza invariancia bajo cambio de unidades del check $f \leq \text{atol} + \text{rtol} \cdot \text{escala}$.
- **Limitaciones**: no distingue tracción/compresión en la activación del daño; control de carga global puede ser inestable tras el pico (requiere `ArcLengthSolver` para seguir la rama de softening).
- **Compatible con**: Truss2D, Truss3D.
- **Referencia**: ver docs/specs/IsotropicDamage1D.md y la hermana 2D para derivación detallada. Lemaitre & Chaboche, *Mechanics of Solid Materials*. Simó & Ju (1987, IJSS 23) marco de tangente consistente.
- **Archivo**: solidum/materials/damage_1d.py

14.5 IsotropicDamage3D — daño isótropo 3D con softening exponencial

- **Modelo**: extensión 3D de IsotropicDamage2D. Esfuerzo nominal $\sigma = (1 - d) \cdot C_e \cdot \varepsilon$ con C_e 6×6 isotrópica 3D; daño escalar $d \in [0, \text{DAMAGE_MAX}]$. **Sin variantes de hipótesis** — en 3D todas las componentes son activas.
- **Deformación equivalente** (Voigt 6D del proyecto): $\varepsilon_{\text{eq}} = \sqrt{(\varepsilon_{xx}^2 + \varepsilon_{yy}^2 + \varepsilon_{zz}^2 + \frac{1}{2}(\gamma_{xy}^2 + \gamma_{yz}^2 + \gamma_{xz}^2))}$ con $M = \text{diag}(1, 1, 1, 1/2, 1/2, 1/2)$. Equivale a la norma de Frobenius del tensor deformación 3D y es la extensión natural de la convención 2D ($M_{2D} = \text{diag}(1, 1, 1/2)$).
- **Evolución**: κ máxima histórica (Kuhn-Tucker $\kappa_{\{n+1\}} = \max(\kappa_n, \varepsilon_{\text{eq}})$); ley exponencial centralizada en `solidum.materials._softening.evaluate_exponential_damage` (comparada con IsotropicDamage1D e IsotropicDamage2D); saturación a DAMAGE_MAX.
- **Tangente**: **algorítmica consistente** en carga activa con daño no saturado; **secante** $(1-d) \cdot C_e$ en descarga, sin daño activo ($\kappa \leq \kappa_0$) o al saturar.
- Fórmula consistente: $C_{\text{alg}} = (1-d) \cdot C_e - [(1-d) \cdot (1/\kappa + \alpha) / \varepsilon_{\text{eq}}] \cdot (C_e \cdot \varepsilon) \cdot (M \cdot \varepsilon)$.
- Derivación: idéntica al 2D extendida a 6D. $\partial d / \partial \kappa = (1-d) \cdot (1/\kappa + \alpha)$; $\partial \kappa / \partial \varepsilon = M \cdot \varepsilon / \varepsilon_{\text{eq}}$ en carga activa, 0 en descarga.

- **No simétrica en general:** $(\mathbf{C}_e \cdot \boldsymbol{\varepsilon})$ y $(\mathbf{M} \cdot \boldsymbol{\varepsilon})$ no son proporcionales salvo en estados de deformación muy particulares (uniaxial puro alineado con ejes). Recupera convergencia cuadrática del Newton global frente a la secante (Simó-Ju 1987).
- **Relación con IsotropicDamage2D plane_strain:** bajo restricción $\varepsilon_{zz} = \gamma_{yz} = \gamma_{xz} = 0$, Damage3D se reduce **exactamente** a Damage2D plane_strain (misma $\mathbf{C}_e$, misma ε_{eq} , misma ley centralizada). Test cruzado TestIsotropicDamage3DvsPlaneStrain blindado a **14 decimales en d y κ** , 10 decimales en σ . Plane stress 2D usa un $\mathbf{C}_e$ proyectado distinto y no es comparable bajo restricción cinemática.
- **STRAIN_DIM: 6 · PRIMARY_STATE_VAR: 'damage' · IS_SYMMETRIC: False** (la tangente consistente no es simétrica; el despachador algebraico ADR 0003 elige LU para el sistema global).
- **Parámetros:** E (>0), $\nu \in (-1, 0.5)$, κ_0 (>0), α (>0), **density** (opcional, ADR 0008).
- **Variables internas:** κ , **damage**. Sin variables tensoriales (a diferencia de los modelos plásticos): el daño es escalar, el historial es escalar.
- **Admisibilidad (ADR 0006):** $\text{admissibility_scale} = E \cdot \kappa_0$ — idéntica al caso 1D/2D, constante respecto al estado.
- **Limitaciones** (idénticas al 2D): la ε_{eq} simétrica no distingue daño en tensión vs compresión; sin regularización por longitud característica $\rightarrow$ mesh-dependency en régimen de ablandamiento (localización en una banda de elementos). Para hormigón con resistencia diferente en tracción/compresión usar split tipo Mazars o de Vree (out-of-scope). Locking volumétrico en Hex8/Tet4 con ν cercano a 0.5 $\rightarrow$ mismo régimen ya documentado para los demás materiales 3D, blindado por test del elemento.
- **Compatible con:** Hex8, Tet4.
- **Referencia:** ver docs/specs/IsotropicDamage3D.md. Simó & Ju (1987, IJSS 23) tangente consistente para daño isótropo. Lemaitre & Chaboche (1990) marco de continuum damage mechanics. de Souza Neto, Perić & Owen (2008) §12.
- **Archivo:** solidum/materials/damage_3d.py

14.6 IsotropicDamage2D — daño isótropo 2D con softening exponencial

- **Modelo:** extensión 2D de IsotropicDamage1D. Esfuerzo nominal $\sigma = (1 - d) \cdot \mathbf{C}_e \cdot \boldsymbol{\varepsilon}$; daño escalar $d \in [0, \text{DAMAGE_MAX}]$.
- **Deformación equivalente:** $\varepsilon_{eq} = \sqrt{(\varepsilon_{xx}^2 + \varepsilon_{yy}^2 + \frac{1}{2} \gamma_{xy}^2)}$ (norma simple sin distinguir tensión/compresión).
- **Evolución:** κ máxima histórica (Kuhn-Tucker $\kappa_{n+1} = \max(\kappa_n, \varepsilon_{eq})$), ley exponencial $d(\kappa) = 1 - (\kappa_0/\kappa) \cdot \exp(-\alpha(\kappa - \kappa_0))$ para $\kappa > \kappa_0$, saturado a DAMAGE_MAX.
- **Tangente:** **algorítmica consistente** en carga activa con daño no saturado; **secante** $(1-d) \cdot \mathbf{C}_e$ en descarga, sin daño activo ($\kappa \leq \kappa_0$) o al saturar.

- Fórmula consistente: $\mathbf{C}_{\text{alg}} = (1-d) \cdot \mathbf{C}_e - [(1-d) \cdot (1/\kappa + \alpha) / \varepsilon_{\text{eq}}] \cdot (\mathbf{C}_e \cdot \varepsilon) \quad (\mathbf{M} \cdot \varepsilon)$ con $\mathbf{M} = \text{diag}(1, 1, 1/2)$.
- Derivación: $\partial d / \partial \kappa = (1-d) \cdot (1/\kappa + \alpha)$ por la ley exponencial; $\partial \kappa / \partial \varepsilon = \mathbf{M} \cdot \varepsilon / \varepsilon_{\text{eq}}$ en carga activa, 0 en descarga.
- **No simétrica en general** ($(\mathbf{C}_e \cdot \varepsilon)$ y $(\mathbf{M} \cdot \varepsilon)$ no son proporcionales). Recupera convergencia cuadrática del Newton global frente a la secante (que daba convergencia lineal).
- **STRAIN_DIM**: 3 · **PRIMARY_STATE_VAR**: 'damage' · **IS_SYMMETRIC**: False (la tangente consistente no es simétrica; el despachador algebraico ADR 0003 elige LU en lugar de Cholesky para el sistema global).
- **Parámetros**: E, nu, kappa_0, alpha, hypothesis $\in \{\text{'plane_stress' (default), 'plane_strain'}\}$, density (opcional, ADR 0008).
- **Variables internas**: kappa, damage.
- **Admisibilidad (ADR 0006)**: `admissibility_scale` = $E \cdot \kappa_0$ — idéntica al caso 1D, el criterio de daño tiene el mismo umbral característico independientemente de la dimensión.
- **Limitación**: la ε_{eq} simétrica no distingue daño en tensión vs compresión; para hormigón usar modelo de Mazars o split tensión/compresión (out-of-scope). Sin regularización por longitud característica $\rightarrow$ mesh-dependency en régimen de ablandamiento (banda localizada en una fila de elementos).
- **Compatible con**: Quad4, Tri3, Tri6, Quad8, Quad9.
- **Referencia**: ver docs/specs/IsotropicDamage2D.md. Simó & Ju (1987, IJSS 23) tangente consistente para daño isótropo. Lemaitre & Chaboche (1990) marco de continuum damage mechanics.
- **Archivo**: solidum/materials/damage_2d.py

14.7 Elastoplastic1D — plasticidad J2 1D con endurecimiento isótropo lineal

- **Modelo**: plasticidad asociativa con criterio $f = |\sigma_{\text{trial}}| - (\sigma_y + H \cdot \alpha) \leq 0$.
- **Algoritmo**: return mapping clásico 1D.
- Predictor elástico: $\sigma_{\text{trial}} = E \cdot (\varepsilon - \varepsilon_{\text{p_old}})$.
- Corrector: $\Delta \gamma = f / (E + H)$, $\sigma = \sigma_{\text{trial}} - \Delta \gamma \cdot E \cdot \text{sign}(\sigma_{\text{trial}})$.
- Tangente algorítmica consistente: $E_t = E \cdot H / (E + H)$.
- **STRAIN_DIM**: 1 · **PRIMARY_STATE_VAR**: 'alpha'.
- **Parámetros**: E, sigma_y (fluencia inicial), H (módulo de endurecimiento; 0 = perfectamente plástico).
- **Variables internas**: eps_p (deformación plástica), alpha (acumulada equivalente).

- **Admisibilidad (ADR 0006):** `admissibility_scale` = $\sigma_y + H \cdot \alpha$ — fluencia corriente (adaptativa al endurecimiento). El check $f \leq \text{atol} + \text{rtol} \cdot \text{escala}$ se hace contra la superficie de fluencia *en el estado de entrada del paso*.
- **Compatible con:** Truss2D, Truss3D, Frame2DEuler, Frame2DTimoshenko (en estos últimos solo se aplica al esfuerzo axial σ).
- **Referencia:** Simo & Hughes, *Computational Inelasticity*, cap. 1.
- **Spec:** docs/specs/Elastoplastic1D.md
- **Archivo:** solidum/materials/plastic_1d.py

14.8 CableMaterial1D — cable axial 1D con elasticidad unilateral

- **Ley:** $\sigma = E \cdot \varepsilon$ si $\varepsilon > 0$; $\sigma = 0$ si $\varepsilon \leq 0$ (sin rigidez en compresión).
- **STRAIN_DIM:** 1.
- **Parámetros:** E (módulo de Young en tensión, estrictamente positivo).
- **Variables internas:** ninguna (respuesta memoryless: la rigidez depende solo del signo instantáneo de ε).
- **Tangente:** $E_t = E$ en tensión, $E_t = 0$ en compresión o sin tensión.
- **Implicación numérica:** la rigidez tangente colapsa a cero cuando el cable se afloja, lo que requiere preconditionamiento o regularización en el solver para no degenerar la matriz global. El uso típico es a través del elemento Cable2DCorot/Cable3DCorot, que detecta la situación y la maneja correctamente.
- **Compatible con:** Cable2DCorot, Cable3DCorot.
- **Referencia:** ver docs/specs/CableMaterial1D.md.
- **Archivo:** solidum/materials/cable_1d.py

14.9 DruckerPrager3D — plasticidad friccional cohesivo-friccional 3D

- **Modelo:** cono circular suave de Mohr-Coulomb 3D con cohesión y fricción interna. Criterio $f = \sqrt{J_2} + \eta_f \cdot I_1 - k(\alpha) \leq 0$ con $k(\alpha) = k_0 + H \cdot \alpha$ (endurecimiento isótropo lineal en cohesión). **Sin variantes de hipótesis** — en 3D todas las componentes son activas.
- **Calibraciones disponibles (3D):**
- **outer_cone** (default): $\eta_f = 2 \cdot \sin(\varphi) / [\sqrt{3} \cdot (3 - \sin(\varphi))]$, $k_0 = 6 \cdot c_0 \cdot \cos(\varphi) / [\sqrt{3} \cdot (3 - \sin(\varphi))]$. Circumscribe Mohr-Coulomb en el meridiano de compresión triaxial.
- **inner_cone:** $\eta_f = 2 \cdot \sin(\varphi) / [\sqrt{3} \cdot (3 + \sin(\varphi))]$, $k_0 = 6 \cdot c_0 \cdot \cos(\varphi) / [\sqrt{3} \cdot (3 + \sin(\varphi))]$. Inscribe MC en el meridiano de extensión triaxial.

- **No incluye plane_strain_matched** (existente en DP2D): esa calibración es 2D-only por construcción — MC en 3D depende del ángulo de Lode y no admite coincidencia circular global. El constructor rechaza explícitamente esa variante con mensaje claro.
- **Plasticidad no asociada por defecto:** ángulo de dilatación ψ parámetro independiente, default $\psi = \varphi$ (asociada). Para $\psi \neq \varphi$ la tangente algorítmica es **asimétrica** → el despachador algebraico ADR 0003 elige LU.
- **Algoritmo — dos ramas cerradas con detección automática:**
 - **Return regular** (cone surface): $\Delta\gamma = \mathbf{f_trial} / (\mathbf{G} + 9\mathbf{K}\cdot\eta_f\cdot\eta_g + \mathbf{H_k})$, dirección desviadora invariante bajo carga radial. Tangente algorítmica $\mathbf{C_alg} = \mathbf{K}\cdot\mathbf{v}\mathbf{v} + 2\mathbf{G}\cdot(1-\beta)\cdot\mathbf{I_dev} + 4\mathbf{G}\cdot\beta\cdot\hat{\mathbf{n}}\hat{\mathbf{n}} - (1/A)\cdot\mathbf{b_g}\mathbf{b_f}$ con $\hat{\mathbf{n}}$ y $\mathbf{b_f}$, $\mathbf{b_g}$ en Voigt 6D tensorial. Idéntica estructura algebraica al kernel 2D, solo extendida a 6 componentes activas.
 - **Return al ápice** (vértice hidrostático): $\Delta\gamma_{\text{apex}} = (\mathbf{I_1_trial}\cdot\eta_f - k(\alpha))/ (9\mathbf{K}\cdot\eta_f\cdot\eta_g + \mathbf{H_k})$, $\sigma_{n+1} = (k/(3\eta_f))\cdot\mathbf{I}$ puramente hidrostático en las tres componentes diagonales. Tangente reducida $(\mathbf{K}\cdot\mathbf{H_k}/ (9\mathbf{K}\cdot\eta_f\cdot\eta_g + \mathbf{H_k}))\cdot\mathbf{v}\mathbf{v}$ (sin rigidez desviadora; salvaguarda numérica $\mathbf{K}\cdot\mathbf{1e-6}\cdot\mathbf{v}\mathbf{v}$ si la rigidez colapsa con $\mathbf{H_k}=0$ y $\eta_g\approx 0$).
- Detección regular $\leftrightarrow$ apex: si tras $\Delta\gamma_{\text{regular}}$ resulta $\sqrt{J_2_{\text{new}}} < 0 \rightarrow$ cae en apex; cambia a la fórmula apex.
- **Relación con DruckerPrager2D plane_strain:** DP2D plane_strain con variant='outer_cone' (o 'inner_cone') es la restricción de DP3D bajo $\varepsilon_{zz} = \gamma_{yz} = \gamma_{xz} = 0$. Test cruzado TestDruckerPrager3DvsPlaneStrain blindado a 10 decimales sobre σ_{xx} , σ_{yy} , σ_{xy} , α y componentes plásticas planas; las componentes 3D ausentes en plane strain (ε^p_{yz} , ε^p_{xz}) permanecen nulas en VM3D.
- **STRAIN_DIM:** 6 · **PRIMARY_STATE_VAR:** 'alpha' · **IS_SYMMETRIC:** False declarativo (override por instancia a True cuando $\psi = \varphi$, asociada).
- **Parámetros** (físicos):
 - E, nu: elásticos ($\nu \in (-1, 0.5)$ estricto).
 - cohesion: cohesión inicial c_0 (esfuerzo, >0).
 - phi_deg: ángulo de fricción interna (grados, $0 \leq \varphi < 90$).
 - psi_deg: ángulo de dilatación (grados, $0 \leq \psi \leq \varphi$; default $\varphi \Rightarrow$ asociada).
 - H (≥ 0 , default 0): endurecimiento isótropo lineal en cohesión.
 - variant: 'outer_cone' (default) o 'inner_cone'.
 - density (opcional, ADR 0008).
- **Variables internas:** eps_p (6 componentes Voigt 6D con cortantes tensoriales [xx, yy, zz, xy, yz, xz]), alpha (multiplicador plástico acumulado, adimensional). **Invariante cinemático:** $\text{tr}(\varepsilon^p) = 3\cdot\eta_g\cdot\alpha$ en cualquier estado plástico (distinto de J2 donde $\text{tr}(\varepsilon^p) = 0$).
- **Admisibilidad (ADR 0006):** $\text{admissibility_scale} = k(\alpha) = k_0 + H\cdot\alpha$.

- **Limitaciones declaradas** (`out_of_scope` en `spec`): MC con aristas (pirámide hexagonal — DP no captura efectos de Lode angle, diferencia hasta 15-20 % en frontera fuera de los meridianos puros), variantes con dependencia explícita del Lode angle (Matsuoka-Nakai, Lade-Duncan), endurecimiento por fricción/dilatancia, endurecimiento cinemático, cap-cone, ablandamiento ($H < 0$ requiere regularización), grandes deformaciones. Locking volumétrico en Hex8/Tet4 cuando $\psi > 0$ (flujo dilatante) + ν moderado-alto — mismo régimen ya documentado para Elastic3D/VonMises3D, sin mitigación implementada.
- **Compatible con:** Hex8, Tet4.
- **Referencia:** ver `docs/specs/DruckerPrager3D.md`. Drucker & Prager (1952). Simó & Hughes (1998) §6 (cone plasticity, apex return). de Souza Neto, Perić & Owen (2008) §8 (calibraciones outer/inner 3D, tangentes algorítmicas detalladas).
- **Archivo:** `solidum/materials/drucker_prager_3d.py`

14.10 DruckerPrager2D — plasticidad friccional cohesivo-friccional 2D plane strain

- **Modelo:** cono circular suave de Mohr-Coulomb con cohesión y fricción interna. Criterio $f = \sqrt{J_2} + \eta_f \cdot I_1 - k(\alpha) \leq 0$ con $k(\alpha) = k_0 + H \cdot \alpha$ (endurecimiento isótropo lineal en cohesión).
- **Hipótesis:** solo `plane_strain` en esta entrega. `plane_stress` declarado *out-of-scope* (proyección con $\sigma_{zz}=0$ acoplada a flujo dilatante es notoriamente delicada).
- **Plasticidad no asociada por defecto:** ángulo de dilatancia ψ parámetro independiente, default $\psi = \varphi$ (asociada). Para $\psi \neq \varphi$ la tangente algorítmica es **asimétrica** $\rightarrow$ `IS_SYMMETRIC = False`.
- **Calibración con Mohr-Coulomb** (variant):
- `plane_strain_matched` (default): $\eta_f = \tan(\varphi)/\sqrt{9 + 12\tan^2(\varphi)}$, $k_0 = 3c_0/\sqrt{9 + 12\tan^2(\varphi)}$. Coincide exactamente con MC en plane strain.
- `outer_cone`: $\eta_f = 2 \cdot \sin(\varphi)/[\sqrt{3} \cdot (3 - \sin(\varphi))]$. Circumscribe MC.
- `inner_cone`: $\eta_f = 2 \cdot \sin(\varphi)/[\sqrt{3} \cdot (3 + \sin(\varphi))]$. Inscribe MC.
- **Algoritmo — dos ramas cerradas con detección automática:**
- **Return regular** (cone surface): $\Delta\gamma = f_{\text{trial}} / (G + 9K \cdot \eta_f \cdot \eta_g + H)$, dirección desviadora preservada. Tangente algorítmica $C_{\text{alg}} = K \cdot v \cdot v + 2G \cdot (1-\beta) \cdot I_{\text{dev}} + 4G \cdot \beta \cdot \hat{n} \hat{n} - (1/A) \cdot b_g b_f$ con $\beta = G \cdot \Delta\gamma / \sqrt{J_2_{\text{trial}}}$, $b_g = 2G \cdot \hat{n} + 3K \cdot \eta_g \cdot v$, $b_f = 2G \cdot \hat{n} + 3K \cdot \eta_f \cdot v$.
- **Return al ápice** (vértice del cono, zona puramente hidrostática): $\Delta\gamma_{\text{apex}} = (I_1_{\text{trial}} \cdot \eta_f - k(\alpha)) / (9K \cdot \eta_f \cdot \eta_g + H)$, $\sigma_{n+1} = (k/3\eta_f) \cdot I$ puramente hidrostático. Tangente reducida a $K \cdot H / (9K \cdot \eta_f \cdot \eta_g + H) \cdot v \cdot v$ (simétrica, sin rigidez desviadora).
- Detección regular $\leftrightarrow$ apex: si tras $\Delta\gamma_{\text{regular}}$ resulta $\sqrt{J_2_{\text{new}}} < 0 \rightarrow$ cae en apex; cambia a la fórmula apex.

- **STRAIN_DIM:** 3 · **PRIMARY_STATE_VAR:** 'alpha' · **IS_SYMMETRIC:** False (declarativo, conservador; el despachador algebraico ADR 0003 elige LU).
- **Parámetros** (físicos):
 - E, nu: elásticos.
 - cohesion: cohesión inicial c_0 (esfuerzo).
 - phi_deg: ángulo de fricción interna (grados, $0 \leq \varphi < 90$).
 - psi_deg: ángulo de dilatación (grados, $0 \leq \psi \leq \varphi$; default $\varphi \Rightarrow$ asociada).
 - H (≥ 0 , default 0): endurecimiento isótropo lineal en cohesión.
 - variant: calibración con MC (default 'plane_strain_matched').
 - density (opcional, ADR 0008).
- **Variables internas:** eps_p (tensorial 4 componentes [xx, yy, zz, xy_tensorial]), alpha (multiplicador plástico acumulado, adimensional).
- **Admisibilidad (ADR 0006):** $\text{admissibility_scale} = k(\alpha) = k_0 + H \cdot \alpha$ (cohesión efectiva corriente, unidades de esfuerzo igual que f).
- **Limitaciones** (out-of-scope):
 - Sin plane_stress, endurecimiento por fricción/dilatación, endurecimiento cinemático, cap (modelo cap-cone), ablandamiento ($H < 0$ requiere regularización), grandes deformaciones.
 - Validación inicial cubierta por tests unitarios (incluyendo FD numérica de la tangente) y un benchmark de pipeline Quad4 (rama elástica + rama apex + caso asociado). El régimen rama regular pura.^{en} geometría confinada es difícil de calibrar sin caer en apex; se cubre por los tests unitarios de cortante puro.
- **Compatible con:** Quad4, Tri3, Tri6, Quad8, Quad9 (todos con material 2D plane strain).
- **Referencia:** ver docs/specs/DruckerPrager2D.md. Drucker & Prager (1952). de Souza Neto, Perić & Owen (2008) cap. 8 (Drucker-Prager, tangentes algorítmicas).
- **Archivo:** solidum/materials/drucker_prager_2d.py

14.11 VonMises3D — plasticidad J2 3D con endurecimiento isótropo lineal

- **Modelo:** J2 (Von Mises) con regla de flujo asociada y endurecimiento isótropo lineal, formulado íntegramente en Voigt 6D del proyecto (ADR 0012). **Sin variantes de hipótesis** — en 3D todas las componentes de ε y σ son activas.
- **Algoritmo** (Simó-Hughes §3.3): return mapping radial cerrado único sobre la parte desviadora 6D. Predictor elástico $\mathbf{s_trial} = 2G \cdot (\mathbf{e_dev} - \mathbf{e_p})$ con presión $p = K \cdot \text{tr}(\varepsilon)$; criterio $f = \|\mathbf{s}\| - \sqrt{(2/3) \cdot (\sigma_y + H \cdot \alpha)} \leq 0$ con norma Frobenius tensorial $\|\mathbf{s}\|^2 = \Sigma_{\text{diag}} \mathbf{s}_{ii}^2 + 2 \cdot \Sigma_{\text{off}} \mathbf{s}_{ij}^2$; corrector radial $\Delta\gamma = f_{\text{trial}} / (2G + \cdot H)$ con $\mathbf{N} = \mathbf{s_trial} / \|\mathbf{s_trial}\|$;

actualización $\alpha_{\text{new}} = \alpha + \sqrt{(2/3)} \cdot \Delta\gamma$. Tangente algorítmica consistente cerrada $\mathbf{C}_{\text{alg}} = \mathbf{K} \cdot \mathbf{v} \cdot \mathbf{v} + 2\mathbf{G}(1-\beta) \cdot \mathbf{I}_{\text{dev}} - 2\mathbf{G} \cdot \bar{\gamma} \cdot \mathbf{N} \cdot \mathbf{N}$. Idéntica estructura al kernel `plane_strain` de `VonMises2D` ampliada a 6 componentes activas; comparte el principio físico pero el código kernel se mantiene separado por simplicidad (zero-padding sistemático en Numba sería contraproducente).

- **Relación con VonMises2D plane strain:** VM2D plane strain es la restricción de este modelo bajo $\varepsilon_{zz} = \gamma_{yz} = \gamma_{xz} = 0$ (deformación total impuesta) con $\varepsilon^{\text{p}}_{zz}$, $\varepsilon^{\text{p}}_{yz}$, $\varepsilon^{\text{p}}_{xz}$ libres por incompresibilidad/isotropía. Test cruzado `TestVonMises3DvsPlaneStrain` blindado a 10 decimales sobre σ_{xx} , σ_{yy} , σ_{xy} , α y componentes plásticas planas.
- **STRAIN_DIM:** 6 · **PRIMARY_STATE_VAR:** 'alpha' · **IS_SYMMETRIC:** True (J2 asociado).
- **Parámetros:** E (>0), nu $\in (-1, 0.5)$, sigma_y (>0), H (≥ 0 , default 0), density (opcional, ADR 0008).
- **Variables internas:** eps_p (6 componentes Voigt 6D con cortantes tensoriales [xx, yy, zz, xy, yz, xz]), alpha (acumulada equivalente, adimensional). La parte plástica es **incompresible exacta:** $\text{tr}(\varepsilon^{\text{p}}) = 0$ por construcción del flujo desviador.
- **Admisibilidad (ADR 0006):** $\text{admissibility_scale} = \sqrt{(2/3)} \cdot (\sigma_y + H \cdot \alpha)$ — idéntica a plane strain (misma norma desviadora en la frontera J2).
- **Limitaciones declaradas** (out_of_scope en spec): no endurecimiento cinemático/Voce, no ablandamiento (H<0 requiere regularización), no regla de flujo no asociada, no daño acoplado, no viscoplasticidad, no grandes deformaciones, no anisotropía. Locking volumétrico en Hex8/Tet4 cuando la plasticidad domina + ν moderado-alto — mismo régimen ya documentado para Elastic3D, blindado por tests, sin mitigación implementada (B-bar/F-bar diferidas).
- **Implementación:** kernel `_compute_j2_3d` con @njit. Sin despacho por hipótesis (no aplica en 3D).
- **Compatible con:** Hex8, Tet4 (todos los sólidos 3D con STRAIN_DIM = 6).
- **Referencia:** ver docs/specs/VonMises3D.md. Simó & Hughes, *Computational Inelasticity* (1998), §3.3 (3D J2 return mapping radial cerrado). de Souza Neto, Perić & Owen, *Computational Methods for Plasticity* (2008), §7.
- **Archivo:** solidum/materials/von_mises_3d.py

14.12 VonMises2D — plasticidad J2 2D con endurecimiento isótropo lineal

- **Modelo:** J2 (Von Mises) con regla de flujo asociada y endurecimiento isótropo lineal. Soporta **dos hipótesis cinemáticas** mutuamente excluyentes (selección por kwarg `hypothesis` al construir): `plane_strain` ($\varepsilon_{zz} = 0$) y `plane_stress` ($\sigma_{zz} = 0$). Cada hipótesis usa un kernel Numba especializado.
- **Algoritmo plane strain** (Simó-Hughes §3.3): return mapping radial cerrado sobre la parte desviadora 3D extendida. Predictor elástico desviador $\mathbf{s}_{\text{trial}} = 2\mathbf{G} \cdot (\mathbf{e}_{\text{dev}} - \mathbf{e}_{\text{p}})$

con presión $p = K \cdot \text{tr}(\varepsilon)$; criterio $f = \|\mathbf{s}_{\text{trial}}\| - \sqrt{(2/3) \cdot (\sigma_y + H \cdot \alpha)} \leq 0$; corrector radial $\Delta\gamma = f_{\text{trial}} / (2G + H)$ con $N = \mathbf{s}_{\text{trial}} / \|\mathbf{s}_{\text{trial}}\|$; actualización $\alpha_{\text{new}} = \alpha + \sqrt{(2/3) \cdot \Delta\gamma}$. Tangente algorítmica consistente cerrada.

- **Algoritmo plane stress** (Simó-Hughes §3.4.1, "plane stress projected algorithm"): predictor $\sigma_{\text{trial}} = \mathbf{C}_{\text{e_ps}} \cdot (\varepsilon - \mathbf{e}_p)$ en subespacio plane stress; función de fluencia proyectada $\bar{\mathbf{f}} = \frac{1}{2} \cdot \sigma \cdot \mathbf{P} \cdot \sigma - R^2/3$ con operador $\mathbf{P} = (1/3) \cdot [[2, -1, 0], [-1, 2, 0], [0, 0, 6]]$. Newton local escalar sobre $\Delta\gamma$ en base de autovectores ortonormales $\mathbf{v}_1 = [1, 1, 0]/\sqrt{2}$, $\mathbf{v}_2 = [1, -1, 0]/\sqrt{2}$, $\mathbf{v}_3 = [0, 0, 1]$ de $\mathbf{C}_{\text{e_ps}} \cdot \mathbf{P}$ con autovalores $\mu_1 = E/(3(1-\nu))$, $\mu_2 = \mu_3 = 2G$. Converge en 3-6 iteraciones (tangente cerrada). Actualización de α con la regla físicamente correcta $\alpha_{\text{new}} = \alpha + \Delta\gamma \cdot \sqrt{(2\sigma\mathbf{P}\sigma/3)}$ (la heurística $\sqrt{(2/3) \cdot \Delta\gamma}$ válida en plane strain rompe la invariancia bajo cambio de unidades en plane stress, donde $\Delta\gamma$ tiene unidades $[1/\text{esfuerzo}]$). Incompresibilidad plástica cierra $\mathbf{e}_{p_zz} = -(\mathbf{e}_{p_xx} + \mathbf{e}_{p_yy})$. Tangente algorítmica consistente cerrada con corrección por $d\alpha/d\Delta\gamma \neq \sqrt{(2/3)}$.
- **STRAIN_DIM: 3 · PRIMARY_STATE_VAR: 'alpha'.**
- **Parámetros:** $E, \nu, \sigma_y, H (\geq 0, \text{default } 0), \text{hypothesis} \in \{\text{'plane_strain' (default), 'plane_stress'}\}, \text{density}$ (opcional, ADR 0008).
- **Variables internas:** eps_p (tensorial 4 componentes $[\text{xx}, \text{yy}, \text{zz}, \text{xy_tensorial}]$), α (acumulada equivalente, adimensional en ambas hipótesis).
- **Admisibilidad (ADR 0006):**
 - **plane_strain:** $\text{admissibility_scale} = \sqrt{(2/3) \cdot (\sigma_y + H \cdot \alpha)}$ — radio desviador de la superficie J2.
 - **plane_stress:** $\text{admissibility_scale} = (\sigma_y + H \cdot \alpha)^2/3$ — escala de $\bar{\mathbf{f}}$ proyectada.
 - En ambos, la tolerancia se precomputa fuera del kernel @njit y se pasa como float.
- **Limitaciones declaradas** (ver `out_of_scope` en spec): no endurecimiento cinemático/Voce, no ablandamiento ($H < 0$ requiere regularización), no regla de flujo no asociada, no daño acoplado, no viscoplasticidad, no grandes deformaciones, no anisotropía. Locking volumétrico en plane strain con elementos de bajo orden cuando $\nu \rightarrow 0.5$ o dominio plástico (mitigaciones B-bar/F-bar diferidas).
- **Implementación:** kernels `_compute_j2_plane_strain` y `_compute_j2_plane_stress` con @njit. Despacho por `hypothesis` en construcción (sin coste runtime).
- **Compatible con:** Quad4, Tri3, Tri6, Quad8, Quad9 (configurados en cualquiera de las dos hipótesis).
- **Referencia:** ver `docs/specs/VonMises2D.md`. Simó & Hughes, *Computational Inelasticity* (1998), §3.3 (plane strain), §3.4 (plane stress projected, Box 3.1). de Souza Neto, Perić & Owen, *Computational Methods for Plasticity* (2008), §9.4.
- **Archivo:** `solidum/materials/von_mises_2d.py`

Los materiales cohesivos son una **jerarquía paralela e independiente** de los materiales continuos: operan sobre el salto de desplazamientos $[[\mathbf{u}]]$ sobre una superficie de discontinuidad Γ_d y devuelven tracciones $\mathbf{t}$, no esfuerzos sobre ε . Viven en `solidum/cohesive_materials/`, heredan de

`solidum.core.cohesive_material.CohesiveMaterial`, se registran vía `CohesiveMaterialRegistry` y se declaran en YAML bajo la sección `cohesive_materials` (separada de `materials`). El bulk del elemento sigue siendo un `Material` continuo; el cohesivo entra al sistema sólo cuando el elemento activa una discontinuidad embebida (ver ADR 0010).

Convenciones de la familia: *JUMP_DIM* = dimensión del vector de salto (2 en 2D, 3 en 3D); *PRIMARY_STATE_VAR* = variable interna que se exporta al post-proceso; *IS_SYMMETRIC* = simetría de la contribución a la tangente del elemento. Parámetros físicos típicos: *sigma_t0* (Pa), *G_f* (N/m), *K_e* (Pa/m). No tienen densidad — la inercia es del bulk.

14.13 CohesiveDamageIsotropic — daño cohesivo isótropo Modo-I con softening lineal/exponencial

- **Modelo:** daño escalar $\omega \in [0, 1]$ sobre el salto. Tracción $\mathbf{t}_n = (1 - \omega) \cdot \mathbf{K}_e \cdot [[\mathbf{u}_n]]$ en dirección normal, $\mathbf{t}_s = 0$ en tangencial (Modo-I puro). Activación tipo Rankine ($f = \langle [[\mathbf{u}_n]] \rangle - \kappa$); historial monótono $\kappa_{n+1} = \max(\kappa_n, \langle [[\mathbf{u}_n]] \rangle)$ con $\kappa_0 = \sigma_{t0}/K_e$. Energía de fractura *G_f* cierra la curva analíticamente:
- **Lineal:** $T_{\text{soft}}(\kappa) = \sigma_{t0} \cdot (w_c - \kappa) / (w_c - \kappa_0)$ con apertura crítica $w_c = 2 \cdot G_f / \sigma_{t0}$. $\omega(\kappa) = 1 - \sigma_{t0} \cdot (w_c - \kappa) / [K_e \cdot (w_c - \kappa_0)]$. $\omega = 1$ en $\kappa \geq w_c$.
- **Exponencial:** $T_{\text{soft}}(\kappa) = \sigma_{t0} \cdot \exp[-\sigma_{t0} \cdot (\kappa - \kappa_0) / H]$ con $H = G_f - \sigma_{t0} \cdot \kappa_0 / 2$. $\omega(\kappa) = 1 - T_{\text{soft}}(\kappa) / (K_e \cdot \kappa)$. Asintótica a 1.
- **Tangente:** simétrica por construcción (rank-1 sobre $\mathbf{n} \mathbf{n}$). Algorítmica consistente en carga activa: $\mathbf{T}_{\text{tan}} = \mathbf{K}_e \cdot [(1-\omega) - [[\mathbf{u}_n]] \cdot d\omega/d\kappa] \cdot (\mathbf{n} \mathbf{n})$. Secante reducida $(1-\omega) \cdot \mathbf{K}_e \cdot (\mathbf{n} \mathbf{n})$ en descarga y sin daño. Cap por *DAMAGE_MAX* aplicado **sólo a la rigidez tangente** cuando $\omega \geq \text{DAMAGE_MAX}$; el valor de ω reportado y la tracción usan el valor físico (puede llegar a 1.0 exacto) — esto distingue al material de `IsotropicDamage2D`, donde *K_e* permite truncar ω sin sesgo.
- **JUMP_DIM:** 2 · **PRIMARY_STATE_VAR:** 'damage' · **IS_SYMMETRIC:** True.
- **Parámetros:** *sigma_t0* (resistencia a tracción, Pa), *G_f* (energía de fractura, N/m), *K_e* (rigidez del salto / penalty, Pa/m; sin default automático, ver §12 de la spec para guía $K_e \approx 10 \cdot E_{\text{bulk}} / w_c$), *softening* $\in \{\text{'linear'}, \text{'exponential'}\}$.
- **Variables internas:** *kappa* (historial del salto equivalente, [m]), *damage* (ω).
- **Validación energética:** por construcción $\int_0^{w_c} \mathbf{t} \cdot d[[\mathbf{u}_n]] = G_f$ (lineal) o $\int_0^\infty \approx G_f$ (exponencial); cubierto por tests con cuadratura trapezoidal.
- **Limitaciones declaradas** (*out_of_scope* en la spec): sólo Modo-I (mixto I-II diferido a fase G del ADR 0010), sin contacto unilateral en compresión (la grieta dañada transmite compresión con rigidez $(1-\omega) \cdot \mathbf{K}_e$, no rígida), sin anisotropía del daño, sin acoplamiento viscoso/cíclico, sin regularización para mesh-objectivity (se aborda a nivel del elemento `CST_Embedded2D`, no del material).
- **Compatible con:** pendiente de fase 2 del ADR 0010 (elemento `CST_Embedded2D`). En aislamiento, testeable con historial prescrito de $[[\mathbf{u}]]$.

- **Referencia:** ver docs/specs/CohesiveDamageIsotropic.md. Retama (2010) Cap. 3; Hillerborg, Modéer & Petersson (1976); Simó & Ju (1987).
- **Archivo:** solidum/cohesive_materials/damage_isotropic.py

14.14 Cómo añadir un material nuevo

`/solidum-new material <Name>` — genera archivo en `solidum/materials/`, decorador `@MaterialRegistry.register` esqueleto de test. Declarar `STRAIN_DIM` y, si tiene historia, `PRIMARY_STATE_VAR` (la variable que aparecerá en el VTK). Tras implementar el modelo constitutivo, **añadir una entrada a este catálogo** siguiendo el formato de arriba.

Para un material **cohesivo** nuevo: el patrón es análogo pero el archivo vive en `solidum/cohesive_materials/`, la clase hereda de `CohesiveMaterial` (no `Material`) y se registra con `@CohesiveMaterialRegistry.register`. Declarar `JUMP_DIM`, `PRIMARY_STATE_VAR` y `IS_SYMMETRIC`. Añadir la entrada en la sección "Materiales cohesivos" de este mismo catálogo.

Capítulo 15

Solvers — catálogo

Referencia rápida de los métodos de solución implementados. Una entrada por solver. Para detalles del algoritmo → código fuente.

15.1 LinearSolver — solución lineal directa en un paso

- **Propósito:** resolver $K \cdot U = F$ en un único `spsolve`.
- **Esquema:** ensamblaje único → eliminación directa de DOFs prescritos (ADR 0004) → `scipy.sparse.linalg.spsolve`.
- **Parámetros:** `linear_algebra` (default `auto`; ADR 0003 §4).
- **Cuándo usarlo:** problemas estrictamente lineales (todos los materiales con tangente constante, sin grandes desplazamientos, sin contacto).
- **No converge / no aplica:** cualquier no-linealidad material (daño, plasticidad) o geométrica (corotacional).
- **Spec:** `docs/specs/LinearSolver.md`
- **Archivo:** `solidum/math/solvers/linear.py`

15.2 NonlinearSolver — Newton-Raphson incremental con paso adaptativo

- **Propósito:** resolver problemas no lineales con control de carga, dividiendo la carga total en pasos y aplicando Newton-Raphson dentro de cada paso.
- **Esquema:**
 - Bucle externo: incrementar factor de carga λ desde 0 hasta 1 en `num_steps` pasos.
 - Bucle interno: Newton-Raphson con $K_t \cdot \Delta U = R = \lambda \cdot F_{\text{ext}} - F_{\text{int}}$.
- **Criterio de convergencia dual:** `max(err_disp, err_force) < tol`, con

`err_disp =  $\|\Delta U\| / (\|U\| + \varepsilon)$` y `err_force =  $\|R\| / \max(\|\lambda \cdot F_{\text{ext}}\|, \|F_{\text{int}}\|, \varepsilon)$` .

- **Adaptatividad** (`adaptive=True`): si converge en <5 iteraciones, agranda el siguiente paso ($\times 1.5$); si no converge, biseca ($\div 2$). Falla si $\Delta\lambda < \text{min_delta_lambda}$.
- Tras converger un paso $\rightarrow \text{assembler.commit_all_states}()$ (trial $\rightarrow$ committed en todos los elementos).
- **Parámetros**: `convergence`, `max_iter`, `num_steps`, `adaptive`, `min_delta_lambda`, `linear_algebra`, `freeze_tangent_after_iter`, `line_search` (default `False`, ADR 0011).
- **Cuándo usarlo**: la opción por defecto para no-linealidad material o geométrica suave (sin snap-back).
- **Cuándo activar `line_search=True`**: cuando se observe oscilación del residuo entre iteraciones (síntoma clásico: ratios alternados sin descenso monótono). Caso documentado: plasticidad perfecta con carga cerca/sobre la capacidad. **Default `False`** porque en problemas con tangente consistente cuasi-cuadrática (daño activo, plasticidad estándar) el line search rechaza pasos correctos del Newton donde el residuo sube transitoriamente — ver ADR 0011 §.Enmiendas”.
- **Diagnóstico al diverger**: lanza una subclase tipada de `RuntimeError` (`OscillatingNewtonError`, `SingularTangentError`, `LoadExceedsCapacityError`, `UnknownDivergenceError`) con métricas (último $\|R\|$, último $\|\delta U\|$, factor de carga, bisecciones consumidas) y `hint` textual. Ver `solidum/math/solvers/diagnostics.py`.
- **Limitación**: con bisección adaptativa y cinemática no lineal (corotacional) puede atravesar puntos límite suaves; no atraviesa snap-back con $du/d\lambda < 0$. Para eso $\rightarrow$ `ArcLengthSolver`. Hallazgo de la auditoría fase A: el solver es más capaz de lo asumido históricamente (ver `docs/auditorias/solvers_robustez_fase_A.md`).
- **Referencia**: Crisfield, *Non-linear Finite Element Analysis of Solids and Structures*, vol. 1, cap. 9. Line search: Grippo-Lampariello-Lucidi 1986 (variante de descenso no monótono).
- **Spec**: `docs/specs/NonlinearSolver.md`
- **Archivo**: `solidum/math/solvers/nonlinear.py`

15.3 ArcLengthSolver — método de longitud de arco cilíndrico (Crisfield)

- **Propósito**: trazar curvas de equilibrio con snap-through, snap-back o pérdida de unicidad de carga, controlando simultáneamente desplazamientos y factor de carga.
- **Esquema**:
- **Predictor tangente**: $du_t = K_t^{-1} \cdot F_{ext_ref}$; $d\lambda = \text{sign} \cdot dl / \|du_t\|$.

El `sign` se elige por proyección con el incremento del paso anterior (evitar regresar).

- **Corrector iterativo**: en cada iteración resuelve dos sistemas ($du_R = K^{-1} \cdot R$ y $du_t = K^{-1} \cdot F_{ext}$) y aplica la restricción cuadrática cilíndrica de Crisfield: $\|dU\|^2 = dl^2$.
- De las dos raíces se elige la que mantiene el ángulo positivo con el incremento previo.

- **Caso especial — final_step:** si el predictor sobrepasaría `max_lambda`, fija λ exactamente a `max_lambda` y resuelve solo desplazamientos (Newton-Raphson puro).
- **Auto-ajuste de dl:** <4 iter $\rightarrow$ ampliar ($\times 1.5$, tope `5·dl_initial`); >8 iter $\rightarrow$ reducir ($\times 0.6$); no converge $\rightarrow$ biseca ($\div 2$).
- **Convergencia:** mismo criterio dual que `NonlinearSolver`.
- **Parámetros:** `tol, max_iter, max_lambda, initial_dl, max_steps, dl_grow_factor, dl_max_factor, dl_shrink_factor, dl_grow_iter_threshold, dl_shrink_iter_threshold`.
- **Cuándo usarlo:** problemas con softening pronunciado (daño, post-pandeo, snap-through de cúpulas), o cuando `NonlinearSolver` diverge cerca de un punto límite.
- **Limitación:** más caro por paso (dos `spsolve` por iteración); requiere ajuste de `initial_dl` para problemas nuevos.
- **Referencia:** Crisfield, ^A fast incremental/iterative solution procedure that handles snap-through” (Computers & Structures, 1981); Crisfield vol. 1, cap. 9.
- **Spec:** docs/specs/ArcLengthSolver.md
- **Archivo:** solidum/math/solvers/arclength.py

15.4 DissipationArcLengthSolver — variante de arc-length por disipación (Gutiérrez 2004)

- **Propósito:** trazar la rama post-pico de problemas con **softening severo** controlando la disipación incremental de energía por paso en lugar de la longitud de arco euclídea. Más robusto que el cilíndrico cuando la curva U - λ tiene tramos casi-verticales en el plano (U, λ) .
- **Esquema:**
 - Subclase de `ArcLengthSolver`; reusa predictor tangente, corrector Newton, ajuste de paso, Dirichlet/MPC y backend algebraico.
- **Restricción de paso:** $g(\Delta U, \Delta \lambda) = \frac{1}{2} \cdot (\lambda_n \cdot F \cdot \Delta U - \Delta \lambda \cdot F \cdot U_n) = \tau$. Lineal en $(\Delta U, \Delta \lambda) \Rightarrow$ corrector con una sola raíz en $dd\lambda$, sin selección por menor ángulo ni patología de raíces imaginarias.
- **Switching automático cilíndrico $\leftrightarrow$ disipación** según se detecte disipación neta sobre el umbral relativo `dissipation_threshold·||F_ref||·||U||`. Arranque obligado en cilíndrico ($\alpha = \frac{1}{2} \cdot (\lambda_n \cdot F \cdot du_t - F \cdot U_n)$ es 0 cuando $\lambda_n = U_n = 0$).
- **Sign-of-pivot tracking aproximado** vía signo del `slogdet(K_t)`. Distingue 0 vs número impar de pivots negativos — diagnóstico de paso por punto límite simple. Tracking exacto requiere LDL^T Bunch-Kaufman (deuda técnica #7 STATUS.md).
- **Salvaguarda contra final_step prematuro:** si $|d\lambda_{pred}| > 3 \cdot (\max_lambda - \lambda_{curr})$, bisecta dl o τ antes de aceptar el paso.
- **Detección de $\alpha \approx 0$:** threshold relativo $|\alpha| < 1e-6 \cdot \text{escala}$; revierte temporalmente a cilíndrico (en problemas lineales monotónicos $\alpha = 0$ exactamente por construcción).

- **Parámetros heredados:** todos los de `ArcLengthSolver` (`max_iter`, `max_lambda`, `initial_dl`, `max_steps`, `factores_dl_*`, `linear_algebra`). **Nuevos:** `initial_tau` (obligatorio), `tau_grow_factor`, `tau_max_factor`, `tau_shrink_factor`, `tau_grow_iter_threshold`, `tau_shrink_iter_threshold`, `dissipation_threshold`.
- **Cuándo usarlo:** problemas con softening continuo donde `ArcLengthSolver` cilíndrico bisecta excesivamente cerca del pico — daño bulk 1D/2D, plasticidad con localización progresiva, post-pandeo con descarga elástica.
- **Cuándo NO usarlo (limitación documentada):** cohesivo+embedded discontinuity con `K_e` stiff ($\sim 1e13$). La activación discreta del Rankine introduce un salto en `F_int/K_t` que el `sign` por producto escalar no maneja graciosamente; el predictor del paso siguiente reorienta hacia descarga. Resolverlo requiere LDL^T Bunch-Kaufman real (item #7 deuda STATUS.md) o control por CMOD/CTOD del salto (spec separada). En problemas lineales puramente elásticos coincide con `ArcLengthSolver` cilíndrico a paridad de bits (regresión validada).
- **Referencias:** Gutiérrez (2004), *Communications in Numerical Methods in Engineering* 20, 19-29; Verhoosel, Remmers, Gutiérrez (2009), *IJNME* 77, 1290-1321; Crisfield (1991) vol. 1 §9.4.
- **Spec:** docs/specs/DissipationArcLengthSolver.md (status `implemented`, validación parcial).
- **Archivo:** solidum/math/solvers/dissipation_arclength.py

15.5 ModalSolver — análisis modal por autovalores generalizados (ADR 0009)

- **Propósito:** calcular frecuencias naturales ω_n y modos φ_n de vibración no amortiguados resolviendo $K \cdot \varphi = \omega^2 \cdot M \cdot \varphi$.
- **Esquema:**
- Ensamblaje de `K` (rigidez lineal en `u = 0`) y `M` (masa consistente, ADR 0009 §1) reusando la topología COO cacheada del `Assembler`.
- Reducción simultánea por `Dirichlet Assembler.reduce_pair(K, M) → K_red, M_red, T`.
- `EigenSolver` envuelve `scipy.sparse.linalg.eigsh` con `shift-invert` en `sigma` (`sigma=0` → frecuencias más bajas) y `which="LM"`. ARPACK factoriza $(K_{red} - \sigma \cdot M_{red})$ una sola vez y la reusa en todas las iteraciones Lanczos.
- Expansión de modos al espacio completo: $\varphi = T \cdot \varphi_{red}$. En DOFs prescritos por `Dirichlet` la componente es 0.
- Salida en `ModalResult`: `frequencies_rad`, `frequencies_hz`, `periods`, `modes` (M-ortonormales), `n_modes`, `converged`.
- **Parámetros:** `n_modes` (obligatorio), `sigma` (default 0.0), `which` (default "LM"), `tolerance` (default 1.0e-9), `lumping` (default `consistent`"; fase 1 solo admite ese valor), `linear_algebra` (reservado; ADR 0003).

- **Firma del solver:** `solve()` sin argumentos (no consume `F_applied`). El entrypoint público es `solidum.run_modal(domain, n_modes=N)` o YAML con `solver: type: ModalSolver`. `solidum.run_yaml` despacha automáticamente al detectar el tipo.
- **Cuándo usarlo:** caracterización dinámica de la estructura (frecuencias naturales, formas modales) antes de pasar a análisis transitorio o sísmico. Validación de la matriz de masa contra fórmulas analíticas (barra empotrada-libre, viga Bernoulli-Euler).
- **Limitaciones:** lineal (K evaluada en $\mathbf{u} = 0$); sin amortiguamiento; modos complejos no soportados. Cómputos derivados (factores de participación, masas efectivas, combinación espectral SRS-S/CQC) viven en `solidum/math/modal_response.py` y los consume `ResponseSpectrumSolver` (ADR 0009 fase 7 cerrada).
- **Referencia:** Bathe, *Finite Element Procedures*, cap. 9 (masa) y cap. 11 (algoritmos de autovalor); Hughes, *The Finite Element Method*, cap. 7; ARPACK Users' Guide (Lehoucq-Sorensen-Yang).
- **Spec:** `docs/specs/ModalSolver.md`
- **Archivos:** `solidum/math/solvers/modal.py` (clase `ModalSolver`), `solidum/math/linalg/eigen.py` (clase `EigenSolver`), `solidum/results.py` (`ModalResult`).

15.6 NewmarkSolver — análisis dinámico transitorio lineal (ADR 0009 fase 3)

- **Propósito:** integrar en el tiempo la ecuación de movimiento semidiscreta $\mathbf{M}\ddot{\mathbf{u}} + \mathbf{C}\dot{\mathbf{u}} + \mathbf{K}\mathbf{u} = \mathbf{F}(\mathbf{t})$ con amortiguamiento Rayleigh $\mathbf{C} = \alpha\mathbf{M} + \beta\mathbf{K}$ y apoyos Dirichlet constantes.
- **Esquema:**
- Familia Newmark- β con (β, γ) parametrizables; default $(1/4, 1/2)$ — average acceleration, incondicionalmente estable, sin amortiguamiento numérico, error $\mathcal{O}(\Delta t^2)$.
- **Predictores:** $\tilde{\mathbf{u}} = \mathbf{u}_n + \Delta t \cdot \dot{\mathbf{u}}_n + (\Delta t^2/2)(1-2\beta) \cdot \ddot{\mathbf{u}}_n$, $\tilde{\mathbf{u}} = \dot{\mathbf{u}}_n + \Delta t \cdot (1-\gamma) \cdot \ddot{\mathbf{u}}_n$.
- **Sistema efectivo:** $\mathbf{A}_{\text{eff}} \cdot \ddot{\mathbf{u}}_{\{n+1\}} = \mathbf{F}_{\{n+1\}} - \mathbf{C} \cdot \tilde{\mathbf{u}} - \mathbf{K} \cdot \tilde{\mathbf{u}}$ con $\mathbf{A}_{\text{eff}} = \mathbf{M} + \gamma \Delta t \cdot \mathbf{C} + \beta \Delta t^2 \cdot \mathbf{K}$.
- **Correctores:** $\mathbf{u}_{\{n+1\}} = \tilde{\mathbf{u}} + \beta \Delta t^2 \cdot \ddot{\mathbf{u}}_{\{n+1\}}$, $\dot{\mathbf{u}}_{\{n+1\}} = \tilde{\mathbf{u}} + \gamma \Delta t \cdot \ddot{\mathbf{u}}_{\{n+1\}}$.
- Reducción por Dirichlet (operador $\mathbf{T}$) aplicada a $(\mathbf{K}, \mathbf{M}, \mathbf{C})$ simultáneamente; factorización única de $\mathbf{A}_{\text{eff_red}}$ reusada en todos los pasos.
- Aceleración inicial consistente con la ecuación de movimiento en $\mathbf{t}=0$.
- **Parámetros:** `t_end`, `dt` (obligatorios), `beta` (default 0.25), `gamma` (default 0.5), `rayleigh` (`None` | `{alpha, beta}` directos | `{xi1, omega1, xi2, omega2}` para calibración modal), `u0`, `u0_dot` (default ceros), `F_func` (callback Python $\mathbf{t} \rightarrow \mathbf{F}(\mathbf{t})$; default vibración libre), `linear_algebra`, `lumping`.
- **Firma del solver:** `solve()` sin argumentos, retorna `TransientResult` con historiales `t_history`, `u_history`, `udot_history`, `uddot_history` (shape `(n_dof, n_steps + 1)`).

- **Cuándo usarlo:** respuesta a cargas dependientes del tiempo (sísmica time-history, impacto, vibración forzada), evolución transitoria desde condiciones iniciales arbitrarias, validación de respuesta libre/forzada de estructuras lineales.
- **Limitaciones:**
 - Solo lineal (K , M constantes). La extensión a no-linealidad (Newton dentro de cada paso) es fase 4 del ADR 0009.
 - Apoyos Dirichlet constantes en el tiempo. Multi-support excitation sísmica diferida.
 - Paso Δt fijo. Adaptativo diferido.
 - HHT- α y generalized- α (con amortiguamiento numérico controlado) no incluidos.
- **Referencia:** Newmark (1959), J. Eng. Mech. Div. ASCE; Bathe, *FEP* cap. 9.4; Hughes, *FEM* cap. 9; Chopra, *Dynamics of Structures* cap. 5 y 15.
- **Spec:** docs/specs/NewmarkSolver.md
- **Archivos:** solidum/math/solvers/newmark.py (clase `NewmarkSolver`), solidum/math/damping.py (Rayleigh), solidum/results.py (`TransientResult`).

15.7 NewtonNewmarkSolver — análisis dinámico transitorio no lineal (ADR 0009 fase 4)

- **Propósito:** integrar en el tiempo $M\ddot{u} + C\dot{u} + F_{\text{int}}(u) = F_{\text{ext}}(t)$ con materiales no lineales (plasticidad, daño) o no linealidad geométrica. **Variante** de `NewmarkSolver` (Reglas §4): hereda predictores/correctores y reducción de Dirichlet; añade Newton-Raphson dentro de cada paso temporal.
- **Esquema:**
 - Predictores Newmark idénticos a la fase 3.
 - **Bucle interno de Newton-Raphson** sobre el residuo dinámico

$R(\ddot{u}_{\{n+1\}}) = F_{\text{ext}}(t_{\{n+1\}}) - F_{\text{int}}(u_{\{n+1\}}) - C\dot{u}_{\{n+1\}} - M\ddot{u}_{\{n+1\}}$, con jacobiano $J = M + \gamma\Delta t \cdot C + \beta\Delta t^2 \cdot K_t$ y K_t la rigidez tangente corriente.

- **Convergencia dual** (ADR 0007): $R/(\text{atol} + \text{rtol} \cdot F_{\text{ref}}) \leq 1$ y $\delta u/(\text{atol} + \text{rtol} \cdot \|u\|) \leq 1$.
- **Commit de estado:** tras converger cada paso, `assembler.commit_all_states()` (trial $\rightarrow$ committed). Si no converge en `max_iter`, lanza `RuntimeError` y se descarta el state trial.
- **Amortiguamiento Rayleigh constante en el tiempo**, calibrado con la **rigidez elástica de referencia** K_0 (al inicio del análisis, $u = 0$). Convención estándar (Abaqus, ANSYS, OpenSees); evita acoplamiento ad-hoc entre disipación viscosa y plástica.
- **Newton modificado opcional** (`freeze_tangent_after_iter`, ADR 0003 fase 2): factoriza fresco las primeras N iter de cada paso y reusa la factorización en las siguientes.

- **Recuperación del caso lineal:** con materiales lineales ($K_t = K_0$), el residuo se anula en 1 iter y el resultado coincide exactamente con `NewmarkSolver`. Validado en tests.
- **Parámetros:** heredados de `NewmarkSolver` (`t_end`, `dt`, `beta`, `gamma`, `rayleigh`, `u0`, `u0_dot`, `F_func`, `linear_algebra`, `lumping`) más `convergence` (`ConvergenceCriterion`, ADR 0007), `max_iter` (default 20), `freeze_tangent_after_iter` (default None), `line_search` (default False, ADR 0011 — mismo criterio que `NonlinearSolver`).
- **Diagnóstico al diverger:** lanza una subclase tipada de `RuntimeError` con métricas y `hint` (mismo patrón que `NonlinearSolver`, ADR 0011).
- **Firma del solver:** `solve()` sin argumentos, retorna `TransientResult` (mismo formato que `NewmarkSolver`).
- **Cuándo usarlo:** respuesta dinámica de estructuras con plasticidad transitoria, fatiga de bajo ciclo, impacto en hormigón friccional, vibraciones de marcos con disipación plástica.
- **Limitaciones:** igual que `NewmarkSolver` en lo lineal (apoyos constantes, paso fijo). Además: no resuelve snap-back en problemas con softening severo — combinar con `ArcLengthSolver` si emerge.
- **Despacho YAML:** `solver.type: NewtonNewmarkSolver`. Hereda `PIPELINE_KIND = "transient"` de `NewmarkSolver`; `entry.run_yaml` despacha por ese atributo declarativo (regla C del ADR 0009, aplicada 2026-05-18) — sin `isinstance` ni cadenas hardcoded.
- **Referencia:** ADR 0009 §Fase 4; Hughes, *FEM* cap. 7-9 (Newton dentro de paso temporal); Crisfield vol. 2 cap. 24 (dinámica no lineal).
- **Spec:** docs/specs/NewtonNewmarkSolver.md
- **Archivo:** solidum/math/solvers/newmark.py (clase `NewtonNewmarkSolver`, junto al padre).

15.8 HHTSolver — análisis dinámico transitorio lineal con disipación numérica (Hilber-Hughes-Taylor α)

- **Propósito:** integrar $M\ddot{u} + C\dot{u} + Ku = F(t)$ con **amortiguamiento numérico controlado** que disipa selectivamente los modos de alta frecuencia espurios sin afectar los modos resueltos. **Variante** de `NewmarkSolver` (Reglas §4): hereda predictores/correctores, reducción Dirichlet y factorización única de A_{eff} .
- **Esquema:**
- Ecuación de equilibrio temporal: $M\ddot{u}_{n+1} + (1+\alpha)C\dot{u}_{n+1} - \alpha C\dot{u}_n + (1+\alpha)Ku_{n+1} - \alpha Ku_n = (1+\alpha)F_{n+1} - \alpha F_n$.
- **Auto-derivación de β, γ desde α** (Hilber 1977): $\beta = (1-\alpha)^2/4$, $\gamma = (1-2\alpha)/2$. Preserva orden 2 y estabilidad incondicional.
- **Sistema efectivo:** $A_{\text{eff}} = M + (1+\alpha)\gamma\Delta t C + (1+\alpha)\beta\Delta t^2 K$. Constante en el tiempo $\rightarrow$ factorización única reutilizable (como Newmark).

- **Radio espectral en alta frecuencia:** $\rho_{\infty} = (1+\alpha)/(1-\alpha)$. Con $\alpha=0 \rightarrow \rho_{\infty}=1$ (sin disipación, recupera Newmark trapezoidal). Con $\alpha=-0.05 \rightarrow \rho_{\infty}\approx 0.905$. Con $\alpha=-1/3 \rightarrow \rho_{\infty}=0.5$ (disipación máxima manteniendo orden 2).
- **Disipación selectiva:** modos con $\omega\Delta t > \pi$ se atenúan significativamente; modos con $\omega\Delta t \ll 1$ casi intactos. Es lo que distingue HHT- α de subir γ Newmark (que disipa en todas las frecuencias y baja a orden 1).
- **Parámetros:** `t_end`, `dt` obligatorios. `alpha` (default -0.05 , rango $[-1/3, 0]$). `beta`, `gamma` autoderivados desde `alpha` salvo override explícito (no recomendado fuera de experimentación). Resto heredado de `NewmarkSolver`: `rayleigh`, `u0`, `u0_dot`, `F_func`, `linear_algebra`, `lumping`.
- **Cuándo usarlo:** cuando la respuesta transitoria contenga modos de alta frecuencia espurios (mallas con modos altos no de interés, impacto, contacto, propagación con $\omega\Delta t$ cerca o por encima de π) que enmascaren la señal. Si quieres exactamente Newmark trapezoidal sin disipación, usa `NewmarkSolver`.
- **Default de alpha:** -0.05 es el valor canónico (Hilber 1977; Abaqus, OpenSees, ANSYS): disipación $>9\%$ por ciclo en altas frecuencias, $<0.5\%$ en modos de interés. Si pides `HHTSolver` con `alpha=0.0`, es funcionalmente idéntico a `NewmarkSolver(beta=0.25, gamma=0.5)`.
- **Despacho YAML:** `solver.type: HHTSolver`. Hereda `PIPELINE_KIND = "transient"` de `NewmarkSolver`; `entry.run_yaml` despacha por el atributo declarativo (regla C ADR 0009).
- **Referencia:** Hilber, Hughes & Taylor (1977), *Earthquake Eng. Struct. Dyn.* 5(3) 283-292; Hughes *FEM* §9.3; Chopra *Dynamics of Structures* §15.4.
- **Spec:** docs/specs/HHTSolver.md
- **Archivo:** solidum/math/solvers/newmark.py (clase `HHTSolver`).

15.9 NewtonHHTSolver — análisis dinámico transitorio no lineal HHT- α

- **Propósito:** combinar HHT- α (disipación numérica) con Newton-Raphson interno para problemas con materiales no lineales (plasticidad, daño) o no linealidad geométrica. **Variante** de `NewtonNewmarkSolver` (Reglas §4) que aplica el esquema temporal HHT- α al residuo dinámico no lineal.
- **Esquema:**
 - Residuo dinámico: $R(\ddot{u}_{\{n+1\}}) = (1+\alpha) \cdot F_{\{n+1\}} - \alpha \cdot F_n - [(1+\alpha) \cdot F_{\text{int}}(u_{\{n+1\}}) - \alpha \cdot F_{\text{int}}(u_n)] - [(1+\alpha) \cdot C \cdot \dot{u}_{\{n+1\}} - \alpha \cdot C \cdot \dot{u}_n] - M \cdot \ddot{u}_{\{n+1\}}$.
 - Jacobiano: $J = M + (1+\alpha) \cdot \gamma \Delta t \cdot C + (1+\alpha) \cdot \beta \Delta t^2 \cdot K_t$.
- Todo lo demás idéntico a `NewtonNewmarkSolver`: convergencia dual (ADR 0007), commit/-rollback, Rayleigh con `K_0`, Newton modificado opcional, `line_search` opt-in (ADR 0011), telemetría tipada al diverger.
- **Recuperación:** con materiales lineales, el residuo de Newton se anula en una iteración y el resultado coincide con `HHTSolver`. Validado en tests.

- **Parámetros:** `alpha` (default `-0.05`) + heredados de `NewtonNewmarkSolver` (`convergence`, `max_iter`, `freeze_tangent_after_iter`, `line_search`, ...).
- **Cuándo usarlo:** respuesta dinámica de estructuras con plasticidad transitoria + presencia de modos altos espurios que el Newmark trapezoidal no atenuaría.
- **Despacho YAML:** `solver.type: NewtonHHTSolver`. Vía atributo `PIPELINE_KIND="transient"` heredado de `NewmarkSolver` → `run_transient`.
- **Spec:** `docs/specs/NewtonHHTSolver.md` (variante corta sobre la spec padre `HHTSolver.md`).
- **Archivo:** `solidum/math/solvers/newmark.py` (clase `NewtonHHTSolver`).

15.10 CentralDifferenceSolver — integración explícita por diferencias centradas (ADR 0009 fase 5)

- **Propósito:** integrar $M\ddot{u} + C\dot{u} + F_{\text{int}}(u) = F(t)$ con esquema **explícito** de segundo orden — la aceleración se actualiza en cada paso por $M^{-1} \cdot \text{rhs}$ con M diagonal lumped, sin sistema lineal ni Newton interno. Cubre análisis lineal y no lineal con un solo solver (parámetro `nonlinear: bool`).
- **Esquema (leapfrog Belytschko-Liu-Moran):**
 - Inicialización: $a_0 = M^{-1} \cdot (F(0) - F_{\text{int}}(u_0) - C \cdot v_0 - F_{\text{dir}})$, $v_{\{1/2\}} = v_0 + (\Delta t/2) \cdot a_0$.
 - Paso $n \rightarrow n+1$: $u_{\{n+1\}} = u_n + \Delta t \cdot v_{\{n+1/2\}}$; reevaluar $F_{\text{int}}(u_{\{n+1\}})$; $a_{\{n+1\}} = M^{-1} \cdot (F_{\{n+1\}} - F_{\text{int}} - C \cdot v_{\{n+1/2\}} - F_{\text{dir}})$; $v_{\{n+1\}} = v_{\{n+1/2\}} + (\Delta t/2) \cdot a_{\{n+1\}}$.
 - Modo lineal: $F_{\text{int}} = K \cdot u$ (K constante, ensamblada una vez). Modo no lineal: `Assembler.assemble_non_line` cada paso (descarta la tangente).
- **Estabilidad condicional CFL:** $\Delta t < 2/\omega_{\text{max}}$. Para barras: $\Delta t < L_{\text{el}}/c_p = L_{\text{el}} \cdot \sqrt{(\rho/E)}$. Si se excede, el solver detecta la explosión exponencial a posteriori y aborta con `RuntimeError` informativo. **El solver no calcula Δt_{crit} automáticamente** — responsabilidad del usuario en esta versión inicial (power iteration sobre $M^{-1}K$ queda diferida).
- **Parámetros:** `t_end`, `dt`, `nonlinear` (default `False`), `rayleigh`, `u0`, `u0_dot`, `F_func`, `lumping` (default `"lumped"`; `"consistent"` rechazado), `divergence_threshold` (default `1e8`).
- **Cuándo usarlo:** wave propagation, impacto, dinámica de respuesta rápida — situaciones donde el Δt natural del problema cae por debajo del paso de estabilidad incondicional de Newmark. Para problemas con plasticidad transitoria de respuesta moderadamente rápida, sigue siendo competitivo frente a Newton-Newmark por evitar la factorización de la tangente cada iteración.
- **Rechazos defensivos:** `lumping=consistent` → `ValueError` (la inversión de M consistente cada paso anula la ventaja); `Frame3D` con eje oblicuo a globales → `ValueError` (M_{red} no es estrictamente diagonal; ADR 0009 fase 2 documenta la bloque-diagonalidad inherente).
- **Despacho YAML:** `solver.type: CentralDifferenceSolver`. Vía atributo `PIPELINE_KIND="transient"` → `run_transient`. **Primer solver fuera de la jerarquía de Newmark**, lo que disparó la regla C de refactor (atributo declarativo en lugar de cadena de `isinstance`).

- **Spec:** docs/specs/CentralDifferenceSolver.md.
- **Archivo:** solidum/math/solvers/central_difference.py.

15.11 HarmonicSolver — respuesta forzada armónica en el dominio de la frecuencia (ADR 0009 fase 6)

- **Propósito:** resolver $(-\omega^2\mathbf{M} + i\omega\mathbf{C} + \mathbf{K})\cdot\hat{\mathbf{u}} = \mathbf{F}$ para un barrido de frecuencias y devolver la amplitud compleja $\hat{\mathbf{u}}(\omega)$ en cada frecuencia. Lineal, en estado estacionario — sin transitorio. Apropiado para FRFs, diagnóstico de resonancias y respuesta forzada periódica.
- **Esquema:** ensamble único de $\mathbf{K}$, $\mathbf{M}$, $\mathbf{C} = \alpha\cdot\mathbf{M} + \beta\cdot\mathbf{K}$; reducción de Dirichlet; barrido `for  $\omega$  in omega: spsolve(Z( $\omega$ ), F_red)` con $\mathbf{Z}(\omega) = -\omega^2\mathbf{M} + i\omega\mathbf{C} + \mathbf{K}$ compleja. Factorización LU compleja por frecuencia (no se cachea — $\mathbf{Z}$ cambia con ω).
- **Barrido configurable:** lineal (`scale="linear"`), logarítmico (`scale="log"`), o vector explícito (`omega=array`). Default `n_omega=100`.
- **Parámetros:** `omega` o (`omega_min`, `omega_max`, `n_omega`, `scale`); `F_amplitude` (default ceros; YAML inyecta automáticamente las `point_loads`); `rayleigh`; `lumping`.
- **Cuándo usarlo:** diagnóstico de resonancias, FRFs, respuesta forzada armónica de máquinas rotantes, viento turbulento descompuesto en armónicos. Si el problema requiere no linealidad (geométrica o material), usar `NewtonNewmarkSolver` con excitación armónica.
- **Caveats:** resonancia exacta sin amortiguamiento $\Rightarrow \mathbf{Z}(\omega_n)$ singular (`spsolve` puede fallar). Mitigación: añadir Rayleigh. Cargas con fase compleja requieren construcción desde código — YAML no soporta tipos complejos nativos.
- **Despacho YAML:** `solver.type: HarmonicSolver`. Vía atributo `PIPELINE_KIND="harmonic"` $\rightarrow$ `run_harmonic`. Tercer valor del literal (después de "static", "modal", "transient").
- **Spec:** docs/specs/HarmonicSolver.md.
- **Archivo:** solidum/math/solvers/harmonic.py.

15.12 ResponseSpectrumSolver — análisis sísmico por combinación modal (ADR 0009 fase 7)

- **Propósito:** combinar las respuestas máximas de los primeros `n_modes` modos del sistema bajo un espectro de respuesta dado ($\mathbf{S}_d(\omega)$ o $\mathbf{S}_a(\omega)$) en una envolvente máxima. Análisis estadístico, no temporal — apropiado para diseño sísmico normativo (ASCE 7, Eurocódigo 8, NCh 433, NTC-DS).
- **Esquema:**
- Análisis modal interno vía `ModalSolver` para obtener (`modes`, `frequencies`).
- Factores de participación $\Gamma_n = \varphi_n^\top \cdot \mathbf{M} \cdot \mathbf{r}$ con $\mathbf{r}$ el vector de excitación rígida en la dirección sísmica.

- Respuesta máxima por modo: $u_n^{\text{max}} = \Gamma_n \cdot \varphi_n \cdot S_d(\omega_n)$.
- Combinación SRSS (default) o CQC con coeficientes de correlación de Der Kiureghian.
- **Parámetros:** `n_modes`, `direction` (ndarray o `{"dof_name": "ix"}`), `spectrum` (callable o dict tabulado/constante), `combination` ("SRSS"/"CQC"), `damping` (sólo CQC, default 0.05), `sigma`, `lumping`.
- **Cuándo usarlo:** diseño sísmico contra espectro normativo; diagnóstico de modos dominantes en una dirección de excitación. Para no-linealidad usar transitorio (Newton-Newmark con acelerograma).
- **Caveats:** precisión depende del número de modos incluidos — verifica `cumulative_effective_mass_ratio` ≥ 0.9 . CQC requerido con modos cercanos en frecuencia (cociente < 1.3). Excitación multi-direccional simultánea (CQC3, 100/30/30) requiere combinación adicional fuera del solver.
- **Despacho YAML:** `solver.type: ResponseSpectrumSolver`. Vía atributo `PIPELINE_KIND="spectrum"` $\rightarrow$ `run_response_spectrum`. Cuarto valor del literal (después de "static", "modal", "transient", "harmonic").
- **Algoritmos centralizados:** `solidum/math/modal_response.py` — `response_spectrum_srss`, `response_spectrum_cqc`, `participation_factors`, `spectrum_from_sa`, `spectrum_tabulated`. Compartido con `ModalResult.free_vibration` (wrapper delgado tras regla D 2026-05-18).
- **Spec:** `docs/specs/ResponseSpectrumSolver.md`.
- **Archivo:** `solidum/math/solvers/response_spectrum.py`.

15.13 Cómo añadir un solver nuevo

`/solidum-new solver <Name>` — genera archivo en `solidum/math/solvers/<snake>.py`, decorador `@SolverRegistry.register`, esqueleto de test.

Convenciones de interfaz:

- **Solvers estáticos** (lineales, no lineales, arc-length): `PIPELINE_KIND = "static"`. Constructor recibe `assembler` + parámetros; método `solve(F_ext_global, step_callback=None)` $\rightarrow$ `U_final`. Comprometen los estados internos vía `assembler.commit_all_states()` al converger cada paso. Retornan el campo de desplazamientos completo.
- **Solvers modales / autovalor** (modal — ADR 0009 — y futuros pandeo lineal): `PIPELINE_KIND = "modal"`. Constructor recibe `assembler` + parámetros; método `solve()` $\rightarrow$ `ModalResult` (u otro tipo específico). No consumen vector de cargas. `run_yaml` despacha a `run_modal`.
- **Solvers transitorios** (Newmark, HHT, central differences): `PIPELINE_KIND = "transient"`. Constructor recibe `assembler`, `t_end`, `dt` + parámetros; método `solve()` $\rightarrow$ `TransientResult`. `run_yaml` despacha a `run_transient`.
- **Solvers en frecuencia / espectrales** (harmonic con `PIPELINE_KIND = "harmonic"`, response spectrum con `"spectrum"`). Constructor recibe `assembler` + parámetros del análisis; método `solve()` devuelve el resultado específico (`HarmonicResult`, `ResponseSpectrumResult`). `run_yaml` despacha a `run_harmonic` o `run_response_spectrum` según el atributo.

El **dispatch en run_yaml** se hace por el atributo de clase PIPELINE_KIND (regla C, 2026-05-18). Solvers nuevos no clásicos (que no hereden de los existentes) sólo declaran su PIPELINE_KIND y quedan automáticamente cableados — no requieren tocar `entry.py`.

Tras implementar, **añadir una entrada a este catálogo** siguiendo el formato de arriba y promover la spec a `status: validated`.

Capítulo 16

Anexos técnicos

*Esta sección es referencia técnica del subsistema interno que resuelve el sistema lineal $K\mathbf{x} = \mathbf{b}$ que aparece en el corazón de cada solver no lineal. La elección del backend es **automática**; el usuario no decide. Esta sección documenta cómo se decide y cuáles son los puntos de override para diagnóstico.*

>Diseño completo: ver ADR 0003 en `docs/adr/0003-despachador-de-solver-algebraico.md`.

16.1 Dos capas que se llaman ambas «solver»

Conviene distinguirlas:

- **Solver no lineal** — `LinearSolver`, `NonlinearSolver`, `ArcLengthSolver`. Orquesta la estrategia de paso, las iteraciones de Newton, el control de longitud de arco, los criterios de convergencia. Es lo que el usuario elige en el YAML con `solver.type`.
- **Capa algebraica** — `solidum.math.linalg`. Resuelve el sistema lineal $K \cdot \delta U = \mathbf{R}$ que aparece dentro de cada iteración del solver no lineal. Tiene varios *backends* numéricos y un despachador interno que elige el adecuado según las propiedades de K . Es plumbing automático: el usuario no la ve salvo que pida diagnóstico.

16.2 Backends disponibles y criterio de selección

El despachador (`solidum.math.linalg.dispatcher.select_solver`) elige el backend en función de tres flags declarativos sobre K : simetría, positividad y talla.

Régimen de K	Backend elegido	Notas
Simétrica + positiva definida	Cholesky (CHOLMOD)	$\sim 2\times$ más rápido y $\sim 2\times$ menos memoria que LU. Requiere la dependencia opcional <code>scikit-sparse</code> . Si falta, degrada a LU con un <i>warning</i> una sola vez.
Simétrica indefinida	LU general (placeholder LDL^T)	Ideal para LDL^T con conteo de pivotes negativos (Sturm sequence) en pandeo y snap-through. La interfaz está reservada; mientras no haya backend nativo, se usa LU sin pérdida de corrección, solo sin diagnóstico de bifurcación.
No simétrica	LU general (SuperLU)	Backend universal. Cubre plasticidad no asociada, follower loads, contacto con fricción.

Origen de los flags, todos derivables del modelo:

- `is_symmetric`: AND lógico de `material.IS_SYMMETRIC` y `element.PRESERVES_SYMMETRY` sobre todos los componentes del dominio. Defaults True.
- `is_positive_definite`: arranca en True para `LinearSolver` y `NonlinearSolver`; en False para `ArcLengthSolver`. Se degrada automáticamente si Cholesky reporta no-positividad.
- `size`: número total de DOFs.

Ningún flag se pide al usuario.

EigenSolver (ADR 0009, problema generalizado). La capa algebraica incluye además `solidum.math.linalg.eigen.EigenSolver`, que resuelve el problema generalizado simétrico $K \cdot \varphi = \omega^2 \cdot M \cdot \varphi$ envolviendo `scipy.sparse.linalg.eigsh` (ARPACK Lanczos con shift-invert centrado en σ). No comparte el Protocol `LinearAlgebraSolver` de los backends de $K \cdot x = b$ porque su firma natural es `solve(K, M, n_modes) → (λ, φ)`. El `ModalSolver` lo invoca para el análisis modal; los cálculos internos de shift-invert generan una factorización de $(K - \sigma \cdot M)$ reutilizada en cada iteración Lanczos, así que la palanca de optimización es la misma — Cholesky enchufado en lugar de SuperLU bajaría $2\times$ el coste del modal en problemas SPD (pendiente, ver memoria de cierre).

16.3 Fallback automático SPD $\rightarrow$ LU

Si en algún paso K deja de ser positiva definida (paso a régimen postcrítico, daño con reblandecimiento), Cholesky lanza `CholeskyNotPositiveDefiniteError`. Los tres solvers no lineales lo capturan y reinstancian el backend con LU general para el resto del análisis. La transición es silenciosa — solo se imprime un mensaje informativo en stdout.

Esto significa que **forzar Cholesky desde YAML como diagnóstico nunca rompe el análisis**: si la matriz se vuelve incompatible, el sistema se autocorrigie.

16.4 Factorización reusable y Newton modificado

La interfaz expone `solver.factorize(K)` que retorna un objeto `FactorizedSolver` con `solve(b)` reutilizable. Habilita:

- **Newton modificado:** el `NonlinearSolver` admite el parámetro `freeze_tangent_after_iter: int` que congela la factorización tras N iteraciones del paso. Útil cuando la tangente cambia poco y el coste dominante es la factorización.
- **Dinámica implícita lineal** (ADR 0009 fase 3): el `NewmarkSolver` factoriza una sola vez la matriz efectiva $A_{\text{eff}} = M + \gamma \Delta t \cdot C + \beta \Delta t^2 \cdot K$ al inicio del análisis y reutiliza la factorización en cada paso temporal. Con Δt constante y problema lineal, los cientos o miles de pasos del análisis se reducen a sustituciones triangulares baratas. Es el mismo `FactorizedSolver` del despachador.
- **Análisis futuros** (pandeo lineal, dinámica implícita no lineal con factorización congelada entre pasos cuando la tangente cambia poco) reutilizan la misma interfaz sin extensiones adicionales.

Ejemplo YAML:

```
solver:
  type: NonlinearSolver
  num_steps: 10
  tol: 1.0e-8
  freeze_tangent_after_iter: 2
```

16.5 Override desde YAML — herramienta de diagnóstico

Solo en caso de necesidad de *diagnóstico* o *benchmarking*, el bloque `solver` admite el campo opcional `linear_algebra`:

```
solver:
  type: LinearSolver
  linear_algebra: auto           # default; el despachador decide
  # linear_algebra: cholesky    # forzar Cholesky (requiere scikit-sparse)
  # linear_algebra: lu          # forzar LU (siempre disponible)
  # linear_algebra: ldlt        # placeholder; degrada a LU con warning
```

Cuándo usar el override:

- Comparar tiempos entre Cholesky y LU en un mismo problema.
- Forzar LU si se sospecha un bug en la auto-detección de simetría tras añadir un material nuevo.
- Reproducir un caso de regresión bit-a-bit con un backend específico.

Cuándo NO usarlo: como parte del setup habitual de un caso de análisis. No es decisión de modelado; el default `auto` cubre todos los regímenes correctamente.

16.6 Activación de Cholesky (opcional)

```
conda install -c conda-forge scikit-sparse
```

Una vez instalado, los problemas estáticos lineales y los Newton estables empiezan a usar Cholesky automáticamente — sin tocar YAML, sin recompilar, sin reescribir specs. Si la dependencia no está disponible, Solidum funciona idéntico con LU.